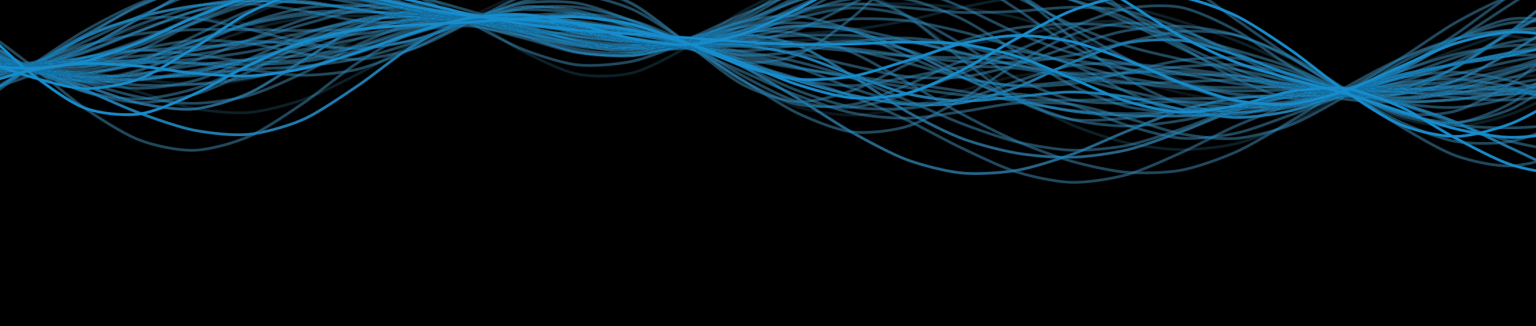
Algorithms for Optimization



Mykel J. Kochenderfer and Tim A. Wheeler

优化算法

优化算法

Mykel J. Kochenderfer
Tim A. Wheeler

The MIT Press
Cambridge, Massachusetts
London, England

© 2019 麻省理工学院

版权所有。免费授予使用与本书相关的代码片段的许可，条件是须注明代码来源。未经出版商书面许可，不得以任何电子或机械手段（包括影印、录制或信息存储与检索）以任何形式复制本书的其他部分。

本书由作者使用 LaTeX 中的 TEX Gyre Pagella 字体排版。在
美国印刷和装订。

美国国会图书馆在版编目数据

名称：Kochenderfer, Mykel J., 1980– 作者。| Wheeler, Tim A. (Tim Allan), 作者。标题：优化算法 / Mykel J. Kochenderfer 和 Tim A. Wheeler。描述：剑桥，马萨诸塞州 : 麻省理工学院出版社，[2019] | 包含参考文献和索引。标识符：LCCN 2018023091 | ISBN 9780262039420 (精装 : 酸性纸) 主题：LCSH：算法。| 算法—问题，练习等。分类：LCC QA9.58 .K65425 2019 | DDC 518/.1—dc23 LC记录可于 https://lccn.loc.gov/2018023091 获取

10 9 8 7 6 5 4 3 2 1

To our families.

Contents

Preface xvii

Acknowledgments xix

1 Introduction 1

1.1 A History 2

1.2 Optimization Process 4

1.3 Basic Optimization Problem 5

1.4 Constraints 6

1.5 Critical Points 7

1.6 Conditions for Local Minima 8

1.7 Contour Plots 11

1.8 Overview 11 *1.9 Summary* 17

1.10 Exercises 17

2 Derivatives and Gradients 19

2.1 Derivatives 19

2.2 *Derivatives in Multiple Dimensions* 21
2.3 *Numerical Differentiation* 23
2.4 *Automatic Differentiation* 27
2.5 *Summary* 32 2.6 *Exercises* 33

3 *Bracketing* 35
3.1 *Unimodality* 35
3.2 *Finding an Initial Bracket* 35
3.3 *Fibonacci Search* 37
3.4 *Golden Section Search* 39
3.5 *Quadratic Fit Search* 43
3.6 *Shubert-Piyavskii Method* 45
3.7 *Bisection Method* 49
3.8 *Summary* 51 3.9 *Exercises* 5
1

4 *Local Descent* 53
4.1 *Descent Direction Iteration* 53
4.2 *Line Search* 54
4.3 *Approximate Line Search* 55
4.4 *Trust Region Methods* 61
4.5 *Termination Conditions* 63
4.6 *Summary* 66 4.7 *Exercises* 66

5 *First-Order Methods* 69
5.1 *Gradient Descent* 69
5.2 *Conjugate Gradient* 70
5.3 *Momentum* 75
5.4 *Nesterov Momentum* 76
5.5 *Adagrad* 77 5.6 *RMSProp* 78
5.7 *Adadelta* 78 5.8 *Adam* 79
5.9 *Hypergradient Descent* 80
5.10 *Summary* 84 5.11 *Exercises* 84
6 *Second-Order Methods* 87
6.1 *Newton's Method* 87
6.2 *Secant Method* 91
6.3 *Quasi-Newton Methods* 91
6.4 *Summary* 95 6.5 *Exercises* 95
7 *Direct Methods* 99
7.1 *Cyclic Coordinate Search* 99
7.2 *Powell's Method* 100
7.3 *Hooke-Jeeves* 102
7.4 *Generalized Pattern Search* 103

7.5 Nelder-Mead Simplex Method 105
7.6 Divided Rectangles 108
7.7 Summary 120 *7.8 Exercises* 123
8 Stochastic Methods 125
8.1 Noisy Descent 125
8.2 Mesh Adaptive Direct Search 126
8.3 Simulated Annealing 128
8.4 Cross-Entropy Method 133
8.5 Natural Evolution Strategies 137
8.6 Covariance Matrix Adaptation 138
8.7 Summary 142 *8.8 Exercises* 142
9 Population Methods 147
9.1 Initialization 147
9.2 Genetic Algorithms 148
9.3 Differential Evolution 157
9.4 Particle Swarm Optimization 158
9.5 Firefly Algorithm 159
9.6 Cuckoo Search 161
9.7 Hybrid Methods 162
9.8 Summary 165 *9.9 Exercises* 165

10 *Constraints* 167
10.1 *Constrained Optimization* 167
10.2 *Constraint Types* 168
10.3 *Transformations to Remove Constraints* 169
10.4 *Lagrange Multipliers* 171
10.5 *Inequality Constraints* 174 10.6 *Duality* 177
10.7 *Penalty Methods* 178
10.8 *Augmented Lagrange Method* 183
10.9 *Interior Point Methods* 183 10.10 *Summary* 18
6 10.11 *Exercises* 186
11 *Linear Constrained Optimization* 189
11.1 *Problem Formulation* 189
11.2 *Simplex Algorithm* 195 11.3 *Dual Certificates*
206 11.4 *Summary* 210 11.5 *Exercises* 210

12 *Multiobjective Optimization* 211
12.1 *Pareto Optimality* 211
12.2 *Constraint Methods* 216
12.3 *Weight Methods* 218
12.4 *Multiobjective Population Methods* 221
12.5 *Preference Elicitation* 228 12.6 *Summary*
232 12.7 *Exercises* 232

13 Sampling Plans 235
13.1 Full Factorial 235
13.2 Random Sampling 236
13.3 Uniform Projection Plans 237
13.4 Stratified Sampling 238
13.5 Space-Filling Metrics 239
13.6 Space-Filling Subsets 244
13.7 Quasi-Random Sequences 245
13.8 Summary 251 *13.9 Exercises* 251
14 Surrogate Models 253
14.1 Fitting Surrogate Models 253
14.2 Linear Models 254
14.3 Basis Functions 255
14.4 Fitting Noisy Objective Functions 263
14.5 Model Selection 265 *14.6 Summary* 27
4 *14.7 Exercises* 274

15 Probabilistic Surrogate Models 275
15.1 Gaussian Distribution 275
15.2 Gaussian Processes 277 *15.3 Prediction* 28
0 *15.4 Gradient Measurements* 282

15.5 Noisy Measurements 285
15.6 Fitting Gaussian Processes 287
15.7 Summary 288 *15.8 Exercises* 288

16 Surrogate Optimization 291
16.1 Prediction-Based Exploration 291
16.2 Error-Based Exploration 292
16.3 Lower Confidence Bound Exploration 293
16.4 Probability of Improvement Exploration 293
16.5 Expected Improvement Exploration 294
16.6 Safe Optimization 296 *16.7 Summary* 305
16.8 Exercises 305

17 Optimization under Uncertainty 307
17.1 Uncertainty 307
17.2 Set-Based Uncertainty 309
17.3 Probabilistic Uncertainty 312
17.4 Summary 318 *17.5 Exercises* 3
18

18 Uncertainty Propagation 321
18.1 Sampling Methods 321
18.2 Taylor Approximation 322
18.3 Polynomial Chaos 323

18.4 Bayesian Monte Carlo 334
18.5 Summary 337
18.6 Exercises 337

19 Discrete Optimization 339
19.1 Integer Programs 340
19.2 Rounding 341
19.3 Cutting Planes 342
19.4 Branch and Bound 346
19.5 Dynamic Programming 351
19.6 Ant Colony Optimization 354
19.7 Summary 358 *19.8 Exercises* 35
8

20 Expression Optimization 361
20.1 Grammars 361
20.2 Genetic Programming 364
20.3 Grammatical Evolution 370
20.4 Probabilistic Grammars 375
20.5 Probabilistic Prototype Trees 377
20.6 Summary 382 *20.7 Exercises* 384

21 Multidisciplinary Optimization 387
21.1 Disciplinary Analyses 387
21.2 Interdisciplinary Compatibility 389

21.3 Architectures 393
21.4 Multidisciplinary Design Feasible 393
21.5 Sequential Optimization 396
21.6 Individual Discipline Feasible 398
21.7 Collaborative Optimization 403
21.8 Simultaneous Analysis and Design 406
21.9 Summary 407 *21.10 Exercises* 408

A Julia 411
A.1 Types 411
A.2 Functions 420
A.3 Control Flow 422
A.4 Packages 423

B Test Functions 425
B.1 Ackley's Function 425
B.2 Booth's Function 426
B.3 Branin Function 427
B.4 Flower Function 428
B.5 Michalewicz Function 429
B.6 Rosenbrock's Banana Function 430
B.7 Wheeler's Ridge 431
B.8 Circle Function 432

C Mathematical Concepts 433
C.1 Asymptotic Notation 433
C.2 Taylor Expansion 435
C.3 Convexity 436 *C.4 Norms* 439
C.5 Matrix Calculus 439
C.6 Positive Definiteness 442
C.7 Gaussian Distribution 442
C.8 Gaussian Quadrature 443

D Solutions 447
Bibliography 483 *Index*
495

Preface

本书广泛介绍了优化技术，重点聚焦于工程系统设计中的实用算法。我们涵盖了多种优化主题，介绍了 underlying 数学问题公式化及求解这些问题的算法。通过图表、示例与练习，旨在传达各种方法背后的直观理解。

本文面向高年级本科生、研究生及专业人士。本书要求读者具备一定的数学成熟度，并假定读者已接触过多变量微积分、线性代数和概率概念。附录中提供了一些复习材料。本书特别适用的学科包括数学、统计学、计算机科学、航空航天、电气工程和运筹学。

本教材的基础是算法，这些算法均以Julia编程语言实现。我们发现，该语言非常适合以人类可读的形式规定算法。本书附带的代码片段获准免费使用，条件是必须注明代码来源。我们预计，其他人可能希望贡献这些算法到其他编程语言的翻译。一旦有翻译版本可用，我们将从本书网页链接到它们。

Mykel J. Kochenderfer Tim
A. Wheeler 斯坦福，加利福
尼亚州 2018年10月30日

书中网页提供了辅助材料:
`http://mitpress.mit.edu/algorithms-for-optimization`

Acknowledgments

本教材源自斯坦福大学开设的工程设计优化课程。我们感谢在过去五年中帮助塑造该课程的学生和助教。我们也感谢曾基于Joaquim Martins、Juan Alonso、Ilan Kroo、Dev Rajnarayan和Jason Hicken的讲义，在我们系教授过该课程不同版本的教师们。本教材中讨论的许多主题都受到了他们讲义的启发。

作者希望感谢许多对本文手稿早期草稿提供了宝贵反馈的人，包括 Mohamed Abdelaty、Atish Agarwala、Piergiorgio Alotto、David Ata、Rishi Bedi、Felix Berkenkamp、Raunak Bhattacharyya、Hans Borchers、Maxime Bouton、Abhishek Cauligi、Mo Chen、Vince Chiu、Jonathan Cox、Katherine Driggs-Campbell、Thai Duong、Hamza El-Saawy、Sofiane Ennadir、Tamas Gal、Christopher Lazarus Garcia、Michael Gobble、Robert Goedman、Jayesh Gupta、Aaron Havens、Bogumi Kamiński、Walker Kehoe、Petr Krysl、Jessie Lauzon、Ruilin Li、Iblis Lin、Edward Londner、Charles Lu、Miles Lubin、Marcus Luebke、Jacqueline Machesky、Ashe Magalhaes、Zouhair Mahboubi、Jeremy Morton、Robert Moss、Santiago Padrón、Harsh Patel、Derek Phillips、Sidd Rao、Andreas Reschka、Alex Reynell、Per Rutquist、Orson Sandoval、Jeffrey Sarnoff、Sumeet Singh、Nathan Stacey、Alex Toews、Pamela Toman、Rachael Tompa、Zacharia Tuten、Yuri Vishnevsky、Julie Walker、Zijian Wang、Patrick Washington、Adam Wiktor、Robert Young 和 Andrea Zanette。此外，我们非常高兴能与麻省理工学院出版社的 Marie Lufkin Lee 和 Christine Bridget Savage 合作，为出版准备这份手稿。

本书的风格受到了爱德华·塔夫特的启发。在诸多风格元素中，我们采用了他宽页边距以及小多图的使用方式。事实上，本书的排版很大程度上基于凯文制作的Tufte-LaTeX包。

Godby、Bil Kleb 和 Bill Wood。我们还受到了 Donald Knuth 和 Stephen Boyd 教科书中清晰度的启发。

在过去几年中，我们受益于与Julia核心开发者的讨论，包括Jeff Bezanson、Stefan Karpinski和Viral Shah。我们还受益于本教科书所依赖的各种开源包（见附录A.4）。代码的排版是在Geoffrey Poore维护的pythontex帮助下完成的。绘图由Christian Feuersänger维护的pgfplots处理。本书的配色方案改编自Sublime Text的Jon Skinner设计的Monokai主题。对于图表，我们使用由Stéfan van der Walt和Nathaniel Smith定义的viridis色图。

1 Introduction

许多学科的核心都涉及优化。在物理学中，系统在物理定律的约束下趋向于最低能量状态。在商业领域，企业旨在最大化股东价值。在生物学中，适应环境更好的生物更有可能生存下来。本书将从工程的角度聚焦于优化，其目标是设计一个系统，在约束条件下优化一组指标。这个系统可以是一个复杂的物理系统，如飞机，也可以是一个简单的结构，如自行车车架。这个系统甚至可能不是物理的；例如，我们可能对设计一个自动驾驶汽车的控制系统或一个检测肿瘤活检图像是否为癌症的计算机视觉系统感兴趣。我们希望这些系统能够尽可能地表现出色。根据应用的不同，相关指标可能包括效率、安全性和准确性。设计上的约束可能包括成本、重量和结构可靠性。

本书讨论的是用于优化的*algorithms*，即计算过程。给定系统设计的某种表示，例如一组编码翼型几何形状的数字，这些算法将告诉我们如何在可能的设计空间中搜索，以找到最优设计。根据应用的不同，这种搜索可能涉及运行物理实验，例如风洞测试，也可能涉及评估解析表达式或运行计算机模拟。我们将讨论用于应对各种挑战的计算方法，例如如何搜索高维空间、处理存在多个相互竞争目标的问题，以及考虑度量中的不确定性。

1.1 A History

我们将从讨论优化算法历史[1]开始，首先介绍
古希腊哲学家。萨摩斯的毕达哥拉斯（公元前569-475年），勾股定理的发现者，声称“数学的原理是一切事物的原理”，[2]普及了数学可以模拟

世界。柏拉图和亚里士多德都运用推理来实现社会优化的目的。[3] 他们思考了人类的最佳风格
生命，这涉及个人生活方式和国家运作的优化。亚里士多德逻辑是一种早期的形式化过程——一种算法——通过它可以进行演绎。

数学抽象的优化也可以追溯到几千年前。亚历山大的欧几里得（公元前325–265年）解决了早期的几何优化问题，包括如何找到从一点到圆周的最短和最长线段。他还证明了在固定周长下，正方形是面积最大的矩形。[4] 希腊数学家芝诺多鲁斯（公元前200–140年）

研究了狄多问题，如图1.1所示。
其他人证明了自然界似乎会进行优化。亚历山大的海伦（公元10–75年）表明光在两点之间沿最短路径传播。亚历山大的帕普斯（公元290–350年）在其对优化的众多贡献中提出，蜂窝中重复的六边形是储存蜂蜜的最佳正多边形；其六边形结构使用最少的材料在平面上构建细胞网格。[5]

优化研究的核心在于使用*algebra*，即研究数学符号操作规则的学科。代数学归功于波斯数学家阿尔·花拉子米（公元790-850年），其著作《al-Kitāb al-jabr wal-muqābala》，或称《通过完成与平衡的计算简明书》。代数的优势在于使用印度-阿拉伯数字，包括在基数表示中使用零。*al'jabr*一词是波斯语中的“恢复”，是西方词汇*algebra*的来源。术语*algorithm*源自*algoritmi*，即阿尔·花拉子米名字的拉丁语翻译和发音。

优化问题通常被表述为在由一组坐标定义的空间中进行搜索。坐标的使用源自勒内·笛卡尔（1596–1650），他使用两个数字来描述二维平面上的一个点。他的洞见将代数及其解析方程与几何的描述性和可视化领域联系起来。[6] 他的工作还包括一种求任意曲线切线的方法

1 本讨论并非旨在全面详尽。更详细的历史由 X.-S. Yang 在 *Engineering Optimization*中的“优化简史”一文提供。Wiley，2010年，第1–13页。

2 亚里士多德，*Metaphysics*，W. D. Ross 译。公元前350年，第一卷，第5部分。

3 参见 S. Kiranyaz、T. Ince 和 M. Gabbouj 的讨论，*Multidimen-sional Particle Swarm Optimization for Machine Learning and Pattern Recognition*。Springer，2014年，第21–44页。

4 参见欧几里得的第三卷和第六卷，*The Elements*，由D. E. Joyce翻译。公元前300年。

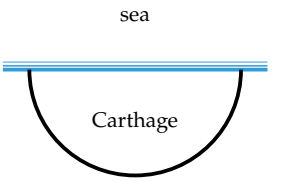


图1.1. 迦太基的创始人狄多女王被授予一块用地，其面积等于她能用一条牛皮绳围起来的土地。她把牛皮绳的两端沿海中海做成一个半圆形，从而围住了尽可能大的面积。这个问题在维吉尔的 *Aeneid* (19 BCE) 中被提及。

5 T. C. Hales，“蜂窝猜想”，*Discrete & Computational Geometry*，第25卷，第1–22页，2001年。

6 R. Descartes，“La Géométrie”，载于 *Discours de la Méthode*。1637年。

其方程是已知的。切线在确定函数的极小值和极大值方面很有用。皮埃尔·德·费马（1601–1665）开始求解导数为零的位置，以识别潜在的最优点。

*calculus*的概念，或连续变化的研究，在我们讨论优化时扮演着重要角色。现代微积分源于戈特弗里德·威廉·莱布尼茨（1646-1716）和艾萨克·牛顿爵士（1642-1727）的发展。微分和积分 calculus均利用了无限级数收敛到一个明确极限的概念。

20世纪中叶见证了电子计算机的兴起，激发了人们对优化数值算法的兴趣。计算的便捷性使得优化能够应用于各个领域中规模大得多的问题。其中一个重大突破是线性规划的引入，这是一种具有线性目标函数和线性约束的优化问题。列昂尼德·坎托罗维奇（1912–1986）提出了线性规划的一种表述及其求解算法。[7] 它被应用于最优资源

二战期间的分配问题。George Dantzig（1914–2005）开发了单纯形算法，这在高效求解线性规划方面代表了一个重大进步。[8] Richard Bellman（1920–1984）发展了动态

动态规划，这是一种常用的方法，通过将复杂问题分解为更简单的问题来最优地解决它们。[9] 动态规

规划已广泛用于最优控制。本教科书概述了为数字计算机开发的、已用于各种工程设计优化问题的许多关键算法。

数十年的大规模计算进步不仅催生了创新的物理工程设计，也促进了人工智能系统的设计。这些系统的智能已在国际象棋、《危险边缘！》和围棋等游戏中得到展现。1996年，IBM的深蓝通过评估数百万种棋局来优化走法，击败了世界国际象棋冠军加里·卡斯帕罗夫。2011年，IBM的沃森在《危险边缘！》节目中与前冠军布拉德·法特和肯·詹宁斯同台竞技。沃森通过对约2亿页结构化和非结构化数据的概率推断优化其回答，赢得了100万美元的一等奖。自那场比赛以来，该系统已发展至协助医疗决策和天气预报。2017年，谷歌的AlphaGo击败了当时世界排名第一的围棋选手柯洁。该系统使用了拥有数百万参数的神经网络，这些参数是通过自我对弈优化而来的，并且

7 L. V. Kantorovich，"求解某类极值问题的新方法"，载于 *Pro-*，第 2 卷，1940年。 *ceedings of the USSR Academy of Sciences*

8 单纯形算法将在第11章中介绍。

9 R. Bellman，“论动态规划理论”，*Proceedings*，第38卷，第8期，第716–719页，1952年。 *of the National Academy of Sciences of the United States of America*

来自人类游戏的数据。深度神经网络的优化正在推动人工智能领域的一场重大革命，这一趋势很可能会持续下去。[10]

10 I. Goodfellow, Y. Bengio, 和 A. Courville, *Deep Learning*. MIT Press, 2016.

1.2 *Optimization Process*

一个典型的工程设计优化过程如图1.2[11]所示。该 *designer* 的作用是提供一个详细说明要达到的参数、常量、目标和约束的问题 *specification*。设计者负责构建该问题并量化潜在设计的优点。设计者通常还向优化算法提供基线设计或初始设计点。

Design

工程设计中设计过程的进一步讨论见 J. Arora, *Introduction to Optimum*，第 4 版，Academic Press，2016 年。

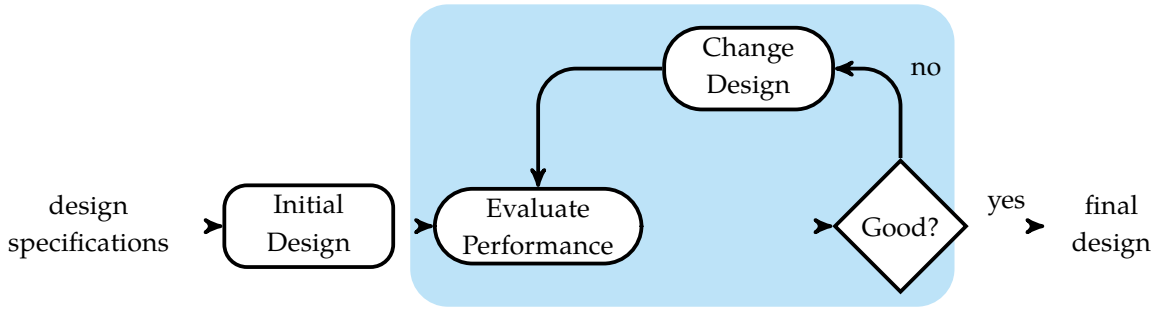


图1.2. 设计优化过程。我们力求自动化蓝色高亮显示的优化程序。

本书关注的是自动化地改进设计以提升性能的过程。优化算法被用来逐步改进设计，直到无法进一步改进，或者达到预算的时间或成本为止。设计人员负责分析优化过程的结果，以确保其适用于最终应用。问题描述中的错误、糟糕的基线设计，以及实现不当或不合适的优化算法，都可能导致次优甚至危险的设计。

将最优化方法应用于工程设计有几个优点。首先，最优化过程提供了一种系统、逻辑的设计流程。如果正确遵循，最优化算法有助于减少设计中人为错误的机会。有时工程设计中的直觉可能具有误导性；根据数据进行最优化可能要好得多。最优化可以加快设计过程，尤其是当某一流程可以编写一次并随后重新应用于其他问题时。传统的工程技术是

通常由人类在二维或三维中进行可视化和推理。然而，现代优化技术可以应用于具有数百万变量和约束的问题。

使用优化进行设计也存在一些挑战。我们通常在计算资源和时间上受到限制，因此我们的算法在选择探索设计空间的方式时必须有所取舍。从根本上说，优化算法受限于设计者指定问题的能力。在某些情况下，优化算法可能会利用建模错误，或者提供的解决方案不能充分解决预期的问题。当算法得出的看似最优设计却反直觉时，可能难以解释。另一个局限是，许多优化算法并不总能保证产生最优设计。

1.3 Basic Optimization Problem

基本优化问题是：

$$\begin{aligned} &\underset{\mathbf{x}}{\text{minimize}} && f(\mathbf{x}) \\ &\text{subject to} && \mathbf{x} \in \mathcal{X} \end{aligned} \tag{1.1}$$

这里，x 是一个 *design point*。一个设计点可以表示为对应于不同 *design variables* 的值向量。一个 n 维设计点表示为：[12]

$$[x_1, x_2, \cdots, x_n] \tag{1.2}$$

12 与Julia一样，用逗号分隔条目的方括号表示列向量。设计点是列向量。

其中第 i 个设计变量记为 x_i。可以调整该向量中的元素以最小化 *objective function* f。在 *feasible set* $\mathcal{X}$ 中的所有点里，任何使目标函数最小化的 x 值都称为 *solution* 或 *minimizer*。某个特定解写作 $\mathbf{x}^*$。图 1.3 展示了一个一维优化问题的示例。

这个表述具有普遍性，意味着任何优化问题都可以根据等式（1.1）重写。特别是，一个问题

$$\underset{\mathbf{x}}{\text{maximize}}\ f(\mathbf{x}) \text{ subject to } \mathbf{x} \in \mathcal{X} \tag{1.3}$$

可以被替换为

$$\underset{\mathbf{x}}{\text{minimize}}\ -f(\mathbf{x}) \text{ subject to } \mathbf{x} \in \mathcal{X} \tag{1.4}$$

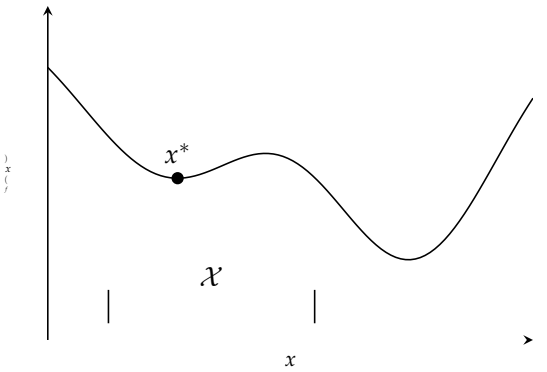


图1.3. 一维优化问题。请注意，最小值仅仅是可行集内的最优值——在可行域外可能存在更低的点。

新形式在具有相同解集这一点上是同一个问题。

在这个数学公式中建模工程问题可能具有挑战性。我们构建优化问题的方式可能会使求解过程变得容易或困难。[13] 我们将重点关注算法方面

tion

在问题被正确表述之后出现的优化。[14]

由于本书讨论了各种不同的优化算法，人们可能会好奇哪种算法最佳。正如Wolpert和Macready的*no free lunchtheorems*所阐述的那样，除非我们对可能目标函数空间上的概率分布做出假设，否则没有理由偏好一种算法胜过另一种。如果一种算法在一类问题上比另一种算法表现更好，那么它在另一类问题上就会表现得更差。[15]为了使许多优化算法有效工作，需要

mization: Algorithms and Applica-

tion

cepts and Applications in Engineer-

ing

Aerospace Design

ciples of Optimal Design

目标函数需要具备某种正则性，比如利普希茨连续性或凸性，这两个主题我们将在后续讨论。在探讨不同的算法时，我们将概述它们的假设条件、机制背后的动机以及各自的优缺点。

1.4 Constraints

许多问题都有约束条件。每个*constraint*限制了可能的解集，并且这些约束共同定义了可行集$\mathcal{X}$。可行设计点不违反任何约束。例如，考虑以下优化问题

Transactions on Evolutionary Com-

putation

13 参见 S. Boyd 和 L. Vandenberghe，*Convex Optimiza-* 中的讨论。剑桥大学出版社，2004年。

14 许多文献提供了如何将现实世界的优化问题转化为优化问题的示例。例如，参见以下内容：R. K. Arora, *Opti-*. Chapman and Hall/CRC, 2019. A. D. Belegundu and T. R. Chandrupatla, *Optimization Con-*, 2nd ed. Cambridge University Press, 2011. A. Keane and P. Nair, *Computational Approaches for*. Wiley, 2005. K. Y. Papalambros and D. J. Wilde, *Prin-*. Cambridge University Press, 2017.

15 无免费午餐定理的假设和结果由D. H. Wolpert和W. G. Macready提供，“优化的无免费午餐定理”，*IEEE*，第1卷，第1期，第67-82页，1997年。

问题：

$$\begin{aligned} \underset{x_1,x_2}{\text{minimize}} \quad & f(x_1, x_2) \\ \text{subject to} \quad & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_1 + x_2 \leq 1 \end{aligned} \tag{1.5}$$

可行集绘制在图1.4中。

约束通常使用 $\leq$、$\geq$ 或 $=$ 来表示。如果约束涉及 $<$ 或 $>$ (，即严格不等式) ，那么可行集不包含约束边界。不包含边界可能引发的一个问题，可通过以下示例说明：

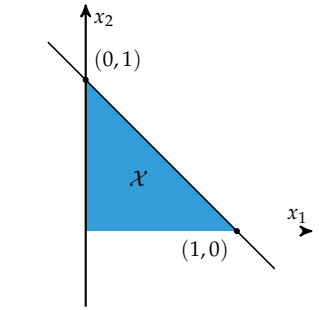


图1.4. 方程(1.5)所关联的可行集$\mathcal{X}$。

$$\begin{aligned} \underset{x}{\text{minimize}} \quad & x \\ \text{subject to} \quad & x > 1 \end{aligned} \tag{1.6}$$

可行集如图1.5所示。点 $x = 1$ 产生的值小于任何大于1的 x，但 $x = 1$ 是不可行的。我们可以选择任意接近但大于1的 x，但无论我们选择什么，总能找到无限多个更接近1的值。我们必须得出结论，该问题无解。为了避免此类问题，通常最好将约束边界包含在可行集中。

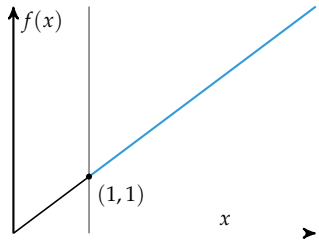


图1.5. 方程(1.6)中的问题无解，因为约束边界不可行。

1.5 Critical Points

图1.6展示了一个*univariate function*[16] $f(x)$，其中标有几个*critical points*，其中导数为零的点，在讨论优化问题时尤为关注。当最小化 f 时，我们希望找到一个 *global minimizer*，即 x 的一个值，使得 $f(x)$ 达到最小。一个函数最多可能有一个全局最小值，但可能有多个全局最小化点。

16 一元函数是单个标量的函数。术语 *uni-* 描述涉及一个变量的对象。*variate*

遗憾的是，通常很难证明一个给定的候选点处于全局最小值。我们能做到的往往是检查它是否处于 *local minimum*。如果存在一个 $\delta > 0$，使得对于所有满足 $|x - x^*| < \delta$ 的 x 都有 $f(x^*) \leq f(x)$，那么点 x^* 处于局部最小值（或是局部极小值点）。在多变量情形下，这一定义推广为存在一个 $\delta > 0$，使得只要 $\|\mathrm{x} - \mathrm{x}^*\| < \delta$，就有 $f(\mathrm{x}^*) \leq f(\mathrm{x})$。

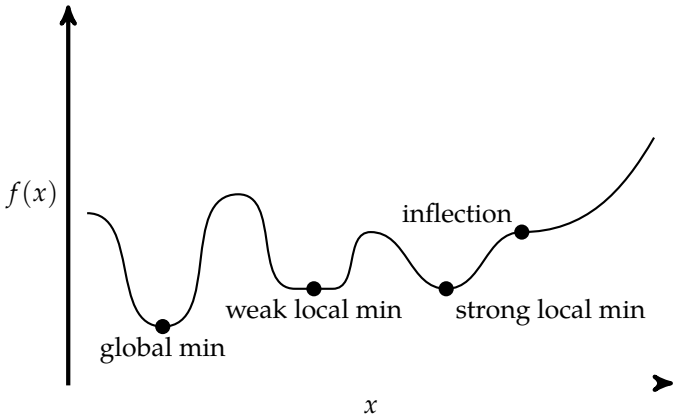


图1.6. 一元函数上优化算法关注的关键点（导数为零处）的示例。

图1.6展示了两种类型的局部极小值：*strong local minima*和*weak localminima*。*strong local minimizer*，也称为*strict local minimizer*，是在一个邻域内唯一最小化*f*的点。换言之，如果存在一个$\delta > > 0$，使得当$x^* \neq x$且$|x - x^*| < \delta$时，$f(x^*) < f(x)$成立，则x^*是一个严格局部极小值点。在多变量情形下，这推广为存在一个$\delta > > 0$，使得当$\mathrm{x}^* \neq \mathrm{x}$且$\|\mathrm{x} - \mathrm{x}^*\| < \delta$时，$f(\mathrm{x}^*) < f(\mathrm{x})$成立。弱局部极小值点是指不是强局部极小值点的局部极小值点。

对于连续无界目标函数，其所有局部和全局最小值的导数均为零。尽管导数为零是局部最小值的 *necessary condition*，[17] 但这并非 *sufficient condition*。

[17] Points with nonzero derivatives are never minima.

图1.6还展示了一个*inflection point*，其导数为零，但该点并未局部最小化*f*。拐点是*f*二阶导数符号改变的位置，对应于f'的局部最小值或最大值。拐点不一定具有零导数。

1.6 Conditions for Local Minima

许多数值优化方法寻求的是局部最小值。局部最小值是在局部范围内最优的，但我们通常无法确定一个局部最小值是否为全局最小值。本节讨论的条件假设目标函数是可微的。导数、梯度和黑塞矩阵在

下一章。在本节中，我们还假设问题是无约束的。关于约束问题的最优性条件将在第10章中介绍。

1.6.1 *Univariate*

如果局部导数为零且二阶导数为正，则保证设计点位于强局部极小值处：

1. $f'(x^*) = 0$
2. $f''(x^*) > 0$

零导数确保将点微小移动不会影响函数值。正二阶导数确保零一阶导数发生在*bowl*的底部。[18]

18 如果$f'(x) = 0$且$f''(x) < 0$，则x是局部最大值。

一个点也可能处于局部最小值，如果它的导数为零且二阶导数仅非负：

1. $f'(x^*) = 0$，即*first-order necessary condition* (FONC)[19]
2. $f''(x^*) \geq 0$，*second-order necessary condition* (SONC)

19 满足一阶必要条件的点有时被称为*stationary point*。

这些条件被称为 *necessary*，因为所有局部极小值都遵循这两条规则。不幸的是，并非所有一阶导数为零且二阶导数为零的点都是局部极小值，如图1.7所示。

可以使用泰勒展开[20]推导出第一个必要条件关于我们的候选点x^*：

20 泰勒展开在附录C中推导得出。

$$f(x^* + h) = f(x^*) + hf'(x^*) + O(h^2) \tag{1.7}$$
$$f(x^* - h) = f(x^*) - hf'(x^*) + O(h^2) \tag{1.8}$$
$$f(x^* + h) \geq f(x^*) \implies hf'(x^*) \geq 0 \tag{1.9}$$
$$f(x^* - h) \geq f(x^*) \implies hf'(x^*) \leq 0 \tag{1.10}$$
$$\implies f'(x^*) = 0 \tag{1.11}$$

其中渐近符号$O(h^2)$在附录C中有所涵盖。

二阶必要条件也可以从泰勒展开中得到：

$$f(x^* + h) = f(x^*) + \underbrace{hf'(x^*)}_{=0} + \frac{h^2}{2} f''(x^*) + O(h^3) \tag{1.12}$$

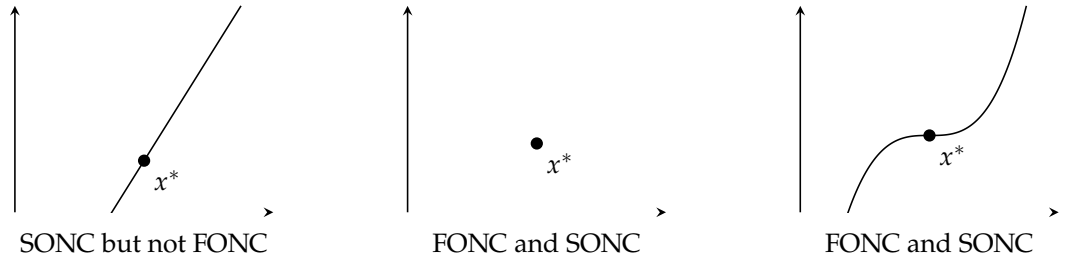


图1.7. 强局部极小值的必要但不充分条件的例子。

我们知道一阶必要条件必须适用：

$$f(x^* + h) \geq f(x^*) \implies \frac{h^2}{2} f''(x^*) \geq 0 \tag{1.13}$$

由于 $h > 0$。因此 $f''(x^*) \geq 0$ 必须成立，才能使 x^* 处于局部极小值　　妈妈。

1.6.2 *Multivariate*

以下是x处于f局部最小值的必要条件：

1. $\nabla f(\mathrm{x}) = 0$，一阶必要条件（FONC）

2. $\nabla^2 f(\mathrm{x})$ 是半正定的（关于此定义的回顾，见附录C.6），二阶必要条件（SONC）

FONC 和 SONC 是一元情况的推广。FONC 告诉我们函数在 x 处不变。图 1.8 展示了满足 FONC 的多元函数示例。SONC 告诉我们 x 位于一个碗状区域内。

FONC 和 SONC 可通过简单分析得到。为了使 x^* 处于局部最小值，它必须小于其周围的值：

$$f(\mathbf{x}^*) \leq f(\mathbf{x} + h\mathbf{y}) \quad \Leftrightarrow \quad f(\mathbf{x} + h\mathbf{y}) - f(\mathbf{x}^*) \geq 0 \tag{1.14}$$

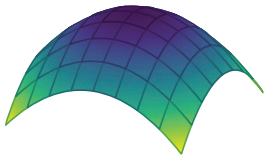
如果我们将$f(\mathrm{x}^*)$写成二阶近似，我们得到：

$$f(\mathbf{x}^* + h\mathbf{y}) = f(\mathbf{x}^*) + h\nabla f(\mathbf{x}^*)^\top \mathbf{y} + \frac{1}{2} h^2 \mathbf{y}^\top \nabla^2 f(\mathbf{x}^*)\mathbf{y} + O(h^3) \tag{1.15}$$

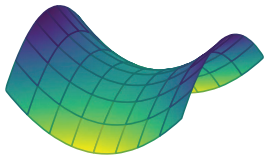
我们知道，至少一阶导数必须为零，并且我们忽略了高阶项。整理后，我们得到：

$$\frac{1}{2} h^2 \mathbf{y}^\top \nabla^2 f(\mathbf{x}^*)\mathbf{y} = f(\mathbf{x} + h\mathbf{y}) - f(\mathbf{x}^*) \geq 0 \tag{1.16}$$

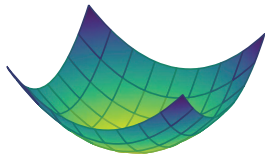
这是半正定矩阵的定义，并由此恢复出SONC。例1.1说明了如何将这些条件应用于Rosenbrock香蕉函数。



一个*local maximum*。中心的梯度为零，但海森矩阵是负定的。



一个 *saddle*。中心的梯度为零，但它不是局部最小值。



一个 *bowl*。中心的梯度为零，且海森矩阵正定。这是一个局部最小值。

图1.8. 梯度为零的三个局部区域。

虽然FONC和SONC对于最优性是必要的，但它们并不充分。对于二次可微函数的无约束优化问题，如果一个点满足FONC且$\nabla^2 f(\mathrm{x})$为正定矩阵，则该点必定处于强局部极小值。这些条件合称为*second-order sufficient condition* (SOSC)。

1.7 Contour Plots

本书将包含不同维数的问题，并需要在单一、两个或三个维度上展示信息。形如$f(x_1、x_2) = y$的函数可以在三维空间中渲染，但并非所有方向都能提供对定义域的完整视图。*contour plot*是通过在坐标轴分别以x_1和x_2为索引的二维图上绘制恒定y值（称为*contours*）的区域，从而获得的三维表面的可视化表示。示例1.2说明了如何解读等高线图。

1.8 Overview

本节简要概述本书各章内容。各章之间的概念依赖关系如图1.9所示。

考虑Rosenbrock香蕉函数，

$$f(\mathbf{x}) = (1 - x_1)^2 + 5(x_2 - x_1^2)^2$$

点 (1, 1) 是否满足 FONC 和 SONC?

梯度为：

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2\left(10x_1^3 - 10x_1x_2 + x_1 - 1\right) \\ 10(x_2 - x_1^2) \end{bmatrix}$$

而海森矩阵为：

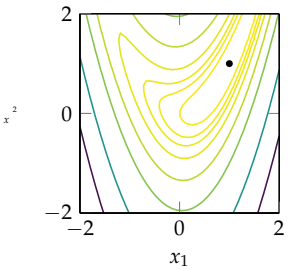
$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1 \partial x_1} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2 \partial x_2} \end{bmatrix} = \begin{bmatrix} -20(x_2 - x_1^2) + 40x_1^2 + 2 & -20x_1 \\ -20x_1 & 10 \end{bmatrix}$$

我们计算 $\nabla(f)([1, 1]) = 0$，因此一阶必要条件得到满足。在 [1, 1] 处的 Hessian 矩阵为：

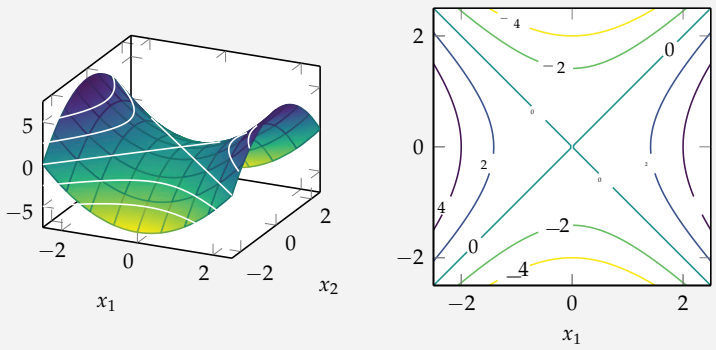
$$\begin{bmatrix} 42 & -20 \\ -20 & 10 \end{bmatrix}$$

它是正定的，因此满足SONC。

例1.1. 检查Rosenbrock函数上某点的一阶和二阶必要条件。极小值点在下图由点表示。



函数 $f(x_1、x_2) = x_1^2 - x_2^2$。该函数可以根据其两个输入和一个输出在三维空间中可视化。它也可以使用等高线图来可视化，该图显示恒定 y 值的线。三维可视化和等高线图如下所示。



示例 1.2. 一个三维可视化示例及相关的等高线图。

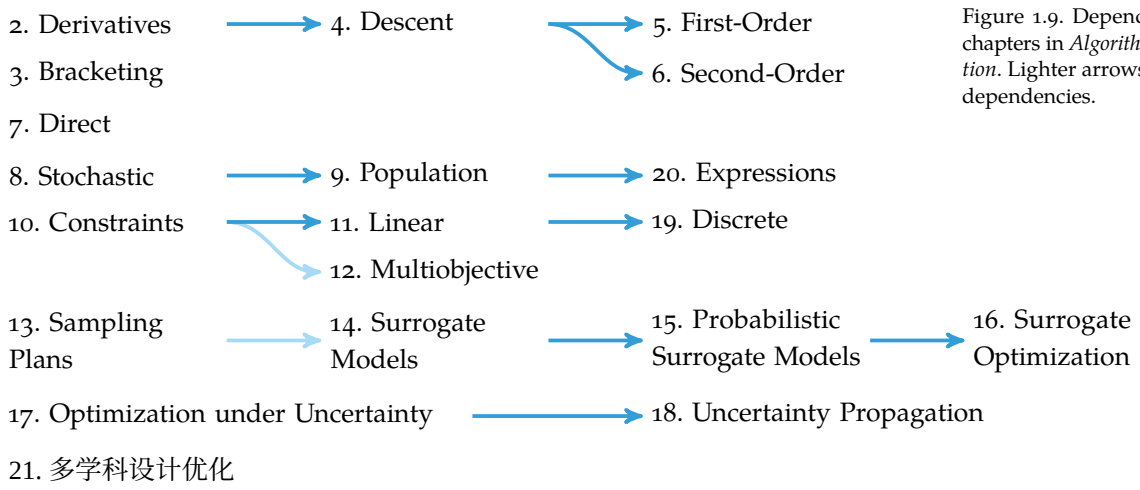


Figure 1.9. Dependencies for t他 chapters in *Algorithms for Optimization*. Lighter arrows indicate weak dependencies.

第2章首先讨论*derivatives*及其向多维的推广。导数在许多算法中被用于为最优值的搜索方向选择提供信息。通常导数无法解析地得知，因此我们讨论如何对其进行数值估计以及使用自动微分技术进行估计。

第3章讨论*bracketing*，它涉及确定一元函数局部最小值所在的区间。不同的区间收缩算法基于函数评估采用不同的方案来逐步缩小该区间。我们讨论的一种方法利用函数的Lipschitz常数知识来指导区间收缩过程。这些区间收缩算法通常作为子程序，用于文本后续讨论的优化算法中。

第4章介绍*local descent*作为优化多元函数的一般方法。局部下降涉及迭代选择下降方向，然后沿该方向迈出一步，并重复该过程，直至收敛或满足某个终止条件。选择步长有不同的方案。我们还将讨论自适应地将步长限制在局部模型可信区域内的方法。

第5章在上一章的基础上，阐释了如何利用通过梯度估计获得的*first-order*信息作为局部模型来指导下降方向。单纯沿着最陡下降方向迈步往往并非寻找最小值的最佳策略。本章讨论了多种不同方法，利用过去的梯度估计序列来更好地指导搜索。

第6章展示了如何使用基于*second-order*近似的局部模型来指导局部下降。这些模型基于对目标函数Hessian矩阵的估计。二阶近似的优势在于它能同时指导方向与步长。

第7章介绍了一组用于寻找最优解的*direct methods*，这些方法避免使用梯度信息来指导搜索方向。我们首先讨论沿一组方向迭代进行线搜索的方法。接着讨论模式搜索方法，这些方法不进行线搜索，而是沿一组方向在当前点之外一定步长处进行评估。随着搜索的进行，步长会逐步调整。另一种方法使用单纯形，在沿明显改进方向穿越设计空间时自适应地扩展和收缩。最后，我们讨论一种受Lipschitz连续性启发的方法，用于在可能包含全局最小值的区域提高分辨率。

第8章介绍了*stochastic*方法，其中随机性被纳入优化过程。我们展示了随机性如何改进前面章节讨论的一些算法，例如最速下降法和模式搜索。一些方法涉及增量遍历搜索空间，但其他方法涉及学习设计空间上的概率分布，对更可能包含最优解的区域赋予更大的权重。

第9章讨论了*population*方法，其中使用一组点来探索设计空间。在空间中分布大量点有助于降低陷入局部最小值的风险。群体方法通常依赖于随机性来促进群体中的多样性，并且它们可以与局部下降方法结合使用。

第10章介绍了优化问题中的*constraints*概念。我们首先讨论带约束最优性的数学条件。然后，我们介绍通过使用惩罚函数将约束融入到前面讨论的优化算法中的方法。我们还讨论了

确保如果我们从可行点出发，搜索将保持可行性的方法。

第11章假设目标函数和约束条件均为*linear*。尽管线性看似是一个较强的假设，但许多工程问题可以被构建为线性约束优化问题。针对利用这种线性结构，已有多种方法被开发出来。本章重点介绍单纯形算法，该算法保证能得到全局最小值。

第12章展示了如何解决*multiobjective*优化问题，即我们同时尝试优化多个目标的情况。工程领域经常涉及多个目标之间的权衡，而且往往不清楚如何优先考虑不同的目标。我们讨论了如何将多目标问题转化为标量值目标函数，以便能够使用前面章节中讨论的算法。我们还讨论了寻找代表目标之间最佳权衡的设计点集合的算法。

第13章讨论了如何创建由覆盖设计空间的点组成的*sampling plans*。对设计空间进行随机采样通常无法提供足够的覆盖。我们讨论了确保沿每个设计维度均匀覆盖的方法，以及衡量和优化空间覆盖的方法。此外，我们还讨论了可用于生成采样计划的准随机序列。

第14章解释了如何构建目标函数的*surrogate models*。代理模型通常用于评估目标函数非常昂贵的问题。优化算法可以使用代理模型的评估值来代替实际目标函数，以改进设计。这些评估值可以来自历史数据，或许是通过上一章介绍的采样计划获得的。我们讨论了不同类型的代理模型、如何将它们拟合到数据，以及如何识别合适的代理模型。

第15章介绍了*probabilistic surrogate models*，使我们能够量化对模型预测的信心。本章重点讨论一种特定类型的代理模型，称为高斯过程。我们展示了如何使用高斯过程进行预测，如何纳入梯度测量和噪声，以及如何从数据中估计控制高斯过程的一些参数。

第16章展示了如何使用前一章的概率模型来指导*surrogate optimization*。本章概述了几种选择下一个评估设计点的技术。我们还讨论了如何使用代理模型以安全的方式优化目标度量。

第17章解释了如何执行*optimization under uncertainty*，放宽了前几章中关于目标函数是设计变量的确定性函数的假设。我们讨论了表示不确定性的不同方法，包括基于集合和概率的方法，并解释了如何转换问题以提供对不确定性的鲁棒性。

第18章概述了*uncertainty propagation*的方法，其中已知的输入分布被用来估计与输出分布相关的统计量。理解目标函数的输出分布对于不确定性下的优化至关重要。我们讨论了多种方法，一些基于数学概念，如蒙特卡洛、泰勒级数近似、正交多项式和Gaussian过程。它们在所做的假设和估计的质量上有所不同。

第19章展示了如何解决设计变量被约束为*discrete*的问题。一种常见方法是放宽变量为离散的假设，但这可能导致不可行的设计。另一种方法涉及逐步添加线性约束，直到最优点为离散。我们还讨论了分支定界和动态规划方法，这两种方法都能保证最优性。本章还提到了一种基于种群的方法，该方法通常能扩展到大型设计空间，但不提供保证。

第20章讨论了如何搜索由文法定义的*expressions*所构成的设计空间。对于许多问题而言，变量数量是未知的，比如在图形结构或计算机程序的优化中。我们概述了几种算法，这些算法考虑了设计空间的语法结构以提高搜索效率。

第21章介绍了如何处理*multidisciplinary design optimization*。许多工程问题涉及多个学科之间复杂的相互作用，单独优化各个学科可能无法得到最优解。本章讨论了多种技术，利用多学科问题的结构来减少寻找良好设计所需的工作量。

附录包含补充材料。附录A从Julia编程语言的简短介绍开始，重点关注相关概念

用于指定本书中列出的算法。附录B详细说明了用于评估不同算法性能的多种测试函数。附录C涵盖了本文讨论的优化方法推导与分析中所使用的数学概念。

1.9 Summary

- 工程中的优化是在一组约束条件下寻找最佳系统设计的过程。

- 优化关注的是寻找函数的全局最小值。
- 最小值出现在梯度为零的地方，但零梯度并不意味着最优。

1.10 Exercises

练习 1.1。给出一个具有局部最小值但不是全局最小值的函数示例。

练习1.2。函数$f(x) = x^3 - x$的最小值是什么?

练习1.3。当x是约束问题的最优解时，一阶条件$f'(x) = 0$是否成立?

练习1.4. 在$x > y \geq 1$的条件下，$f(x、y) = x^2 + y$各有多少个极小值?

练习1.5。$x^3 - 10$有多少个拐点?

2 Derivatives and Gradients

优化关注的是寻找使目标函数最小化（或最大化）的设计点。了解函数值随输入变化而变化的方式是有用的，因为它告诉我们可以在哪个方向上移动以改进先前的点。函数值的变化在一维中由导数衡量，在多维中由梯度衡量。本章简要回顾了微积分中的一些基本要素。[1]

1 更全面的综述，参见 S. J. Colley, *Vector Calcu-*, 第4版. Pearson, 2011.

lus

2.1 Derivatives

函数 f（关于单变量 x）的 *derivative* $f'(x)$ 是 f 的值在 x 处的变化率。如图 2.1 所示，它通常通过函数图像在 x 处的切线来可视化。导数的值等于切线的斜率。

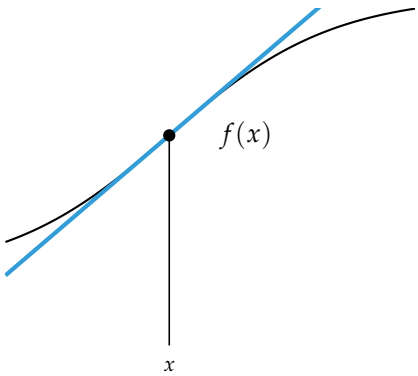


图2.1. 函数f以黑色绘制，而$f(x)$的切线以蓝色绘制。f在x处的导数是切线的斜率。

我们可以使用导数在 x 附近给出函数的线性近似：

$$f(x+\Delta x) \approx f(x) + f'(x)\Delta x \tag{2.1}$$

导数是点 x 处 f 的变化量与 x 的变化量之比：

$$f'(x) = \frac{\Delta f(x)}{\Delta x} \tag{2.2}$$

这是$f(x)$的变化量除以x的变化量，当步长变得无限小时，如图2.2所示。

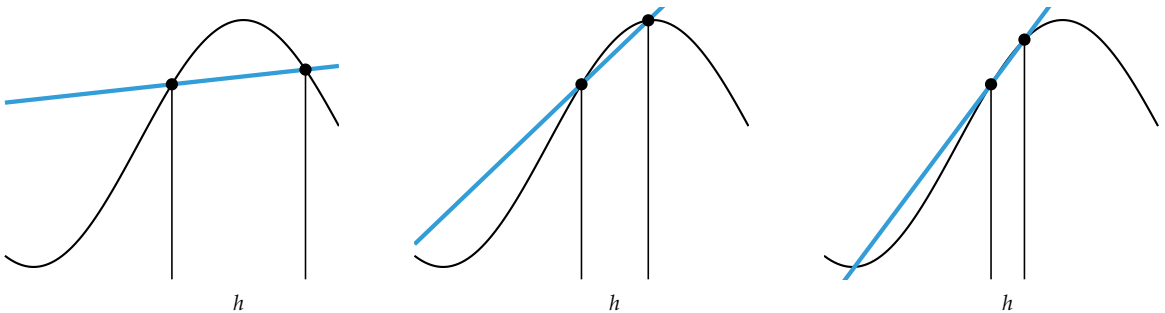


图2.2. 切线是通过连接步长差足够小的点得到的。

符号 $f'(x)$ 可归功于拉格朗日。我们还使用莱布尼茨创造的符号，

$$f'(x) \equiv \frac{df(x)}{dx} \tag{2.3}$$

这强调了这样一个事实：导数是点x处f的变化量与x的变化量之比。

定义导数的极限方程可以用三种不同的方式表示：*forward difference*、*central difference* 和 *backward difference*。每种方法都使用无限小的步长 h：

$$f'(x) \equiv \underbrace{\lim_{h\to 0} \frac{f(x+h)-f(x)}{h}}_{\text{forward difference}} = \underbrace{\lim_{h\to 0} \frac{f(x+h/2)-f(x-h/2)}{h}}_{\text{central difference}} = \underbrace{\lim_{h\to 0} \frac{f(x)-f(x-h)}{h}}_{\text{backward difference}} \tag{2.4}$$

如果 f 可以用符号表示，*symbolic differentiation* 通常能够通过应用微积分中的求导法则，为 f' 提供一个精确的解析表达式。然后，这个解析表达式可以在任何点 x 进行评估。这个过程在例2.1中进行了说明。

符号微分的实现细节超出了本文的范围。各种软件包，如 Julia 中的 `SymEngine.jl` 和 Python 中的 `SymPy`，提供了实现。这里我们使用 `SymEngine.jl` 来计算 $x^2 + x/2 - \sin(x)/x$ 的导数。

```
julia> using SymEngine
julia> @vars x; # define x as a symbolic variable
julia> f = x^2 + x/2 - sin(x)/x;
julia> diff(f, x)
1/2 + 2*x + sin(x)/x^2 - cos(x)/x
```

例2.1. 符号微分提供了分析导数。

2.2 *Derivatives in Multiple Dimensions*

gradient 是导数对多元函数的推广。它捕捉了函数的局部斜率，使我们能够预测从某一点沿任意方向迈出小步的效果。回忆一下，导数是切线的斜率。梯度指向切平面*hyperplane*的最陡上升方向，如图2.3所示。一个n维空间中的超平面是满足以下条件的点集：

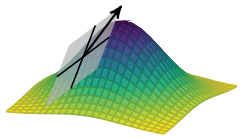


图2.3。梯度的每个分量定义了一条局部切线。这些切线定义了局部切超平面。梯度向量指向最大增加的方向。

$$w_1 x_1 + \cdots + w_n x_n = b \tag{2.5}$$

对于某个向量 w 和标量 b，一个超平面具有 $n - 1$ 维。

f 在 x 处的梯度记为 $\nabla f(\mathbf{x})$，是一个向量。该向量的每个分量是 f 关于该分量的 *partial derivative*[2]：

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f(\mathbf{x})}{\partial x_1}, \quad \frac{\partial f(\mathbf{x})}{\partial x_2}, \quad \ldots, \quad \frac{\partial f(\mathbf{x})}{\partial x_n}\right] \tag{2.6}$$

2 函数关于某个变量的偏导数是在假设所有其他输入变量保持不变时的导数。它记作 $\partial f/\partial x$。

我们采用的约定是，用逗号分隔书写的向量为列向量。例如，我们有 $[a, b, c] = [a\ b\ c]^\top$。例2.2展示了如何计算函数在某一点处的梯度。

多元函数的*Hessian*是一个包含所有关于输入的二次导数的矩阵。[3] 二次导数捕获信息

关于函数的局部曲率。

$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_n} \\ & \vdots & & \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_n \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_n \partial x_n} \end{bmatrix} \tag{2.7}$$

3 海森矩阵仅当f的二阶导数在求值点的一个邻域内都连续时才对称：

$$\frac{\partial^2 f}{\partial x_1 \partial x_2} = \frac{\partial^2 f}{\partial x_2 \partial x_1}$$

计算 $f(\mathrm{x}) = x_1 \sin(x_2) + 1$ 在 $\mathrm{c} = [2, 0]$ 处的梯度。

$$
\begin{aligned}
f(\mathbf{x}) &= x_1 \sin(x_2) + 1 \\
\nabla f(\mathbf{x}) &= \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right] = [\sin(x_2), x_1 \cos(x_2)] \\
\nabla f(\mathbf{c}) &= [0, 2]
\end{aligned}
$$

示例 2.2. 在特定点计算梯度。

多元函数f的*directional derivative* $\nabla_{\mathbf{s}} f(\mathrm{x})$是$f(\mathrm{x})$在x以速度s移动时的瞬时变化率。该定义与单变量函数的导数定义密切相关：[4]

ometry

4 有些文本要求s为单位向量。参见，例如，G. B. Thomas, *Calculus and Analytic Ge-*，第9版。Addison-Wesley，1968年。

$$
\nabla_{\mathbf{s}} f(\mathbf{x}) \equiv \underbrace{\lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{s}) - f(\mathbf{x})}{h}}_{\text{forward difference}} = \underbrace{\lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{s}/2) - f(\mathbf{x} - h\mathbf{s}/2)}{h}}_{\text{central difference}} = \underbrace{\lim_{h \to 0} \frac{f(\mathbf{x}) - f(\mathbf{x} - h\mathbf{s})}{h}}_{\text{backward difference}} \tag{2.8}
$$

可以使用函数的梯度来计算方向导数：

$$
\nabla_{\mathbf{s}} f(\mathbf{x}) = \nabla f(\mathbf{x})^{\top} \mathbf{s} \tag{2.9}
$$

另一种计算方向导数 $\nabla_{\mathbf{s}} f(\mathrm{x})$ 的方法是定义 $g(\alpha) \equiv f(\mathrm{x} + \alpha \mathrm{s})$，然后计算 $g'(0)$，如例2.3所示。

方向导数在梯度方向上最高，在梯度相反的方向上最低。这种方向依赖性源于方向导数定义中的点积，以及梯度是一个局部切超平面这一事实。

我们想要计算 $f(\mathrm{x}) = x_1 x_2$ 在点 $\mathrm{x} = [1, 0]$ 沿方向 $\mathrm{s} = [-1, -1]$ 的方向导数：

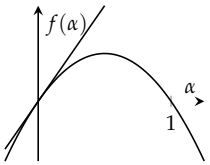
$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1}, & \frac{\partial f}{\partial x_2} \end{bmatrix} = [x_2, x_1]$$

$$\nabla_{\mathbf{s}} f(\mathbf{x}) = \nabla f(\mathbf{x})^\top \mathbf{s} = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix} = -1$$

我们也可以按如下方式计算方向导数：

$$g(\alpha) = f(\mathbf{x} + \alpha \mathbf{s}) = (1 - \alpha)(-\alpha) = \alpha^2 - \alpha$$
$$g'(\alpha) = 2\alpha - 1$$
$$g'(0) = -1$$

示例 2.3。计算方向导数。



2.3 Numerical Differentiation

对导数进行数值估计的过程被称为*numerical differentiation*。估计值可以通过不同的方式从函数求值中导出。本节讨论有限差分法和复数步长法。[5]

5 关于本章剩余部分所讨论主题的更全面处理，请参见A. Griewank和 *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*，第2版。SIAM，2008年。

2.3.1 Finite Difference Methods

顾名思义，*finite difference methods* 计算两个相差有限步长的值之间的差异。它们通过小差分近似公式(2.4)中的导数定义：

$$f'(x) \approx \underbrace{\frac{f(x+h) - f(x)}{h}}_{\text{forward difference}} \approx \underbrace{\frac{f(x+h/2) - f(x-h/2)}{h}}_{\text{central difference}} \approx \underbrace{\frac{f(x) - f(x-h)}{h}}_{\text{backward difference}} \tag{2.10}$$

数学上，步长h越小，导数的估计越好。
实际上，h的值太小可能会导致数值消去误差。这一效应将在图2.4中展示。算法2.1提供了这些方法的实现。

```
diff_forward(f, x; h=sqrt(eps(Float64))) = (f(x+h) - f(x))/h
diff_central(f, x; h=cbrt(eps(Float64))) = (f(x+h/2) - f(x-h/2))/h
diff_backward(f, x; h=sqrt(eps(Float64))) = (f(x) - f(x-h))/h
```

算法2.1。用于估计函数f在x处导数的有限差分方法，采用有限差分h。默认步长为浮点值机器精度的平方根或立方根。这些步长在机器舍入误差与步长误差之间取得平衡。

有限差分法可以通过泰勒展开推导得出。我们将从f在x处的泰勒展开开始，推导前向差分导数估计：

eps 函数提供 1.0 与下一个更大的可表示浮点值之间的步长。

$$f(x+h) = f(x) + \frac{f'(x)}{1!}h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \cdots \tag{2.11}$$

我们可以重新排列并解出第一导数：

$$f'(x)h = f(x+h) - f(x) - \frac{f''(x)}{2!}h^2 - \frac{f'''(x)}{3!}h^3 - \cdots \tag{2.12}$$

$$f'(x) = \frac{f(x+h) - f(x)}{h} - \frac{f''(x)}{2!}h - \frac{f'''(x)}{3!}h^2 - \cdots \tag{2.13}$$

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \tag{2.14}$$

前向差分对小h近似真实导数，误差取决于$\frac{f''(x)}{2!}h + \frac{f'''(x)}{3!}h^2 + \cdots$。该误差项为$O(h)$，这意味着当$h$趋近于零时，前向差分具有线性误差。[6]

6 渐近记号在附录C中讨论。

{translation}
使用泰勒展开表示该误差项。关于x的$f(x+h/2)$和$f(x-h/2)$的泰勒展开为：

Numerical Methods in MATLAB, 第4版, Pearson, 2004年。

$$f(x+h/2) = f(x) + f'(x)\frac{h}{2} + \frac{f''(x)}{2!}\left(\frac{h}{2}\right)^2 + \frac{f'''(x)}{3!}\left(\frac{h}{2}\right)^3 + \cdots \tag{2.15}$$

$$f(x-h/2) = f(x) - f'(x)\frac{h}{2} + \frac{f''(x)}{2!}\left(\frac{h}{2}\right)^2 - \frac{f'''(x)}{3!}\left(\frac{h}{2}\right)^3 + \cdots \tag{2.16}$$

减去这些展开式得到：

$$f(x+h/2) - f(x-h/2) \approx 2f'(x)\frac{h}{2} + \frac{2}{3!}f'''(x)\left(\frac{h}{2}\right)^3 \tag{2.17}$$

我们重新排列得到：

$$f'(x) \approx \frac{f(x+h/2) - f(x-h/2)}{h} - \frac{f'''(x)h^2}{24} \tag{2.18}$$

这表明该近似具有二次误差。

2.3.2 *Complex Step Method*

我们经常遇到需要选择足够小的步长h以提供良好近似，但又不能太小以免导致数值减法抵消问题的情况。*complex step method*通过使用单次函数评估来绕过减法抵消的影响。我们在沿虚数方向迈出一步后对函数进行一次评估。[8]

虚步长的泰勒展开式为：

$$f(x+ih) = f(x) + ihf'(x) - h^2\frac{f''(x)}{2!} - ih^3\frac{f'''(x)}{3!} + \cdots \tag{2.19}$$

仅取每边的虚部即可得到导数近似：

$$\operatorname{Im}(f(x+ih)) = hf'(x) - h^3\frac{f'''(x)}{3!} + \cdots \tag{2.20}$$

$$\Rightarrow f'(x) = \frac{\operatorname{Im}(f(x+ih))}{h} + h^2\frac{f'''(x)}{3!} - \cdots \tag{2.21}$$

$$= \frac{\operatorname{Im}(f(x+ih))}{h} + O(h^2) \text{ as } h \to 0 \tag{2.22}$$

实现由算法2.2提供。实部将$f(x)$近似到$O(h^2)$范围内，当$h \to$趋于0时：

$$\operatorname{Re}(f(x+ih)) = f(x) - h^2\frac{f''(x)}{2!} + \ldots \tag{2.23}$$

$$\Rightarrow f(x) = \operatorname{Re}(f(x+ih)) + h^2\frac{f''(x)}{2!} - \cdots \tag{2.24}$$

因此，我们可以通过单次评估带有复数参数的f来同时计算$f(x)$和$f'(x)$。示例 2.4 展示了在特定点估计函数导数所涉及的计算过程。算法 2.2 实现了复数步长方法。图 2.4 比较了随着步长变化，复数步长方法与前向差分法和中心差分法的数值误差。

```
diff_complex(f, x; h=1e-20) = imag(f(x + h*im)) / h
```

算法2.2。用于估计函数`f`在`x`处具有有限差分`h`的导数的复数步长方法。

8 J. R. R. A. Martins、P. Sturdza 和 J. J. Alonso，“复数步长导数近似”，*ACM*，第 29 卷，第 3 期，第 245–262 页，2003 年。必须特别注意确保f的实现正确支持复数作为输入。

Transactions on Mathematical Software

考虑$f(x) = \sin(x^2)$。在$x = \pi/2$处的函数值约为0.624266，导数为$\pi \cos(\pi^2/4) \approx -2.45425$。我们可以使用复数步长法得到这个结果：

```
julia> f = x -> sin(x^2);
julia> v = f(π/2 + 0.001im);
julia> real(v) # f(x)
0.6242698144866649
julia> imag(v)/0.001 # f'(x)
-2.4542516170381785
```

示例 2.4. 用于估计导数的复步方法。

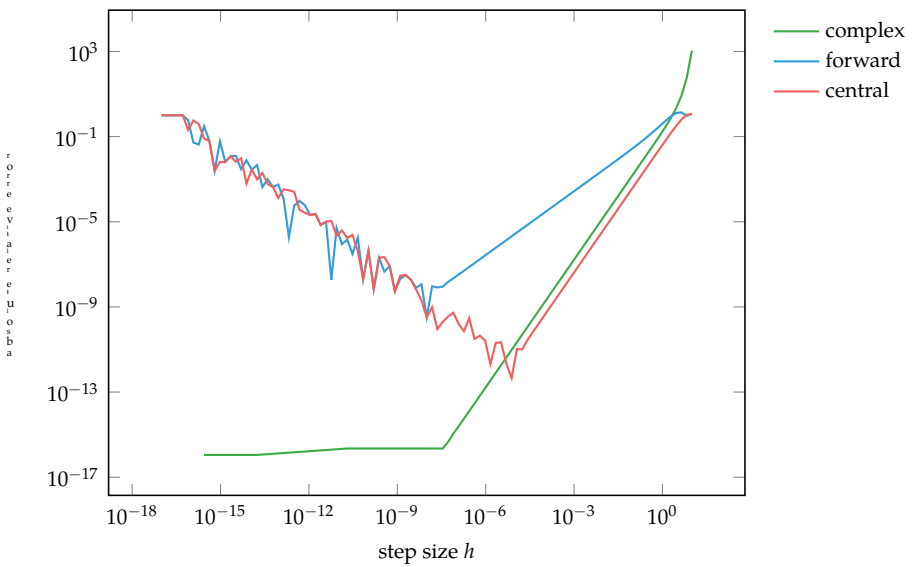


图2.4. 当步长变化时，函数$\sin(x)$在$x = 1/2$处的导数估计误差比较。右侧的恒定斜率显示了前向差分法的线性误差以及中心差分和复数方法的二次误差。复数步长方法避免了当两个相近的函数评估值相减时出现的减法消除误差。

2.4 Automatic Differentiation

本节介绍用于对由计算机程序指定的函数导数进行数值求值的算法。这些 *automatic differentiation* 技术的关键在于链式法则的应用：

$$\frac{d}{dx}f(g(x)) = \frac{d}{dx}(f \circ g)(x) = \frac{df}{dg}\frac{dg}{dx} \tag{2.25}$$

一个程序由加、减、乘、除等基本运算组成。

考虑函数 $f(a, b) = \ln(ab + \max(a, 2))$。如果我们要计算在某一点关于 a 的偏导数，我们需要多次应用链式法则：[9]

$$\begin{aligned}
\frac{\partial f}{\partial a} &= \frac{\partial}{\partial a}\ln(ab + \max(a, 2)) && (2.26)\\
&= \frac{1}{ab + \max(a, 2)}\frac{\partial}{\partial a}(ab + \max(a, 2)) && (2.27)\\
&= \frac{1}{ab + \max(a, 2)}\left[\frac{\partial(ab)}{\partial a} + \frac{\partial \max(a, 2)}{\partial a}\right] && (2.28)\\
&= \frac{1}{ab + \max(a, 2)}\left[\left(b\frac{\partial a}{\partial a} + a\frac{\partial b}{\partial a}\right) + \left((2 > a)\frac{\partial 2}{\partial a} + (2 < a)\frac{\partial a}{\partial a}\right)\right] && (2.29)\\
&= \frac{1}{ab + \max(a, 2)}[b + (2 < a)] && (2.30)
\end{aligned}$$

这个过程可以通过使用*computational graph*来自动化。计算图表示一个函数，其中节点是操作，边是输入-输出关系。计算图的叶节点是输入变量或常量，终端节点是函数输出的值。计算图如图2.5所示。

有两种方法可以利用 f 的计算图对其进行自动微分。*forward accumulation* 方法采用对偶数，从输入到输出遍历树，而 *reverse accumulation* 则需要通过图进行反向传递。

2.4.1 Forward Accumulation

Forward accumulation 将使用函数的计算图单次前向传递自动微分一个函数。该方法等价于

9 我们采用这样的约定：像$(2 < \alpha)$这样的布尔表达式，若为真则取值1，若为假则取值0。

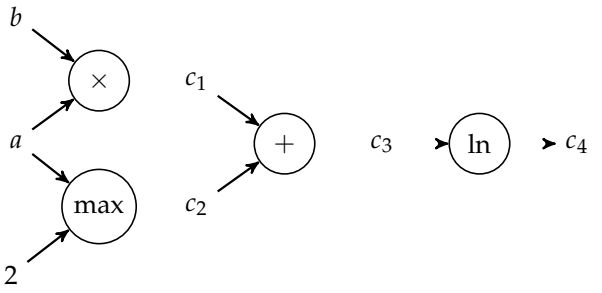


图2.5. 计算图 $\ln(ab + \max(a, 2))$。

迭代扩展内部操作的链式法则：

$$\frac{df}{dx} = \frac{df}{dc_4}\frac{dc_4}{dx} = \frac{df}{dc_4}\left(\frac{dc_4}{dc_3}\frac{dc_3}{dx}\right) = \frac{df}{dc_4}\left(\frac{dc_4}{dc_3}\left(\frac{dc_3}{dc_2}\frac{dc_2}{dx} + \frac{dc_3}{dc_1}\frac{dc_1}{dx}\right)\right) \qquad (2.31)$$

为了说明正向累积，我们将其应用于示例函数 $f(a、b) = \ln(ab + \max(a, 2))$，以计算在 $a = 3$、$b = 2$ 处关于 a 的偏导数。

1. 过程从图的源节点开始，这些节点包括函数输入和任何常数值。对于这些节点中的每一个，我们记录其值以及关于目标变量的偏导数，如图2.6.[10]所示。

10 为图简洁，我们使用*dot notation*或*Newton's no-*表示导数。例如，如果清楚我们是对a求导，就可以将$\partial b/\partial a$写作$\dot{b}$。

2. 接下来，我们沿着树向下进行，每次处理一个节点，选择下一个节点时，确保其输入已被计算。我们可以通过传递前几个节点的值来计算该值，并且利用前几个节点的值及其偏导数来计算关于a的局部偏导数。计算过程如图2.7所示。

我们最终得到正确结果，$f(3, 2) = \ln 9$ 和 $\partial f/\partial a = 1/3$。这是通过一次遍历计算图完成的。

这个过程可以通过计算机方便地自动化，使用一种编程语言，该语言已经重载了每个操作以同时产生其值及其导数。这样的对被称为*dual numbers*。

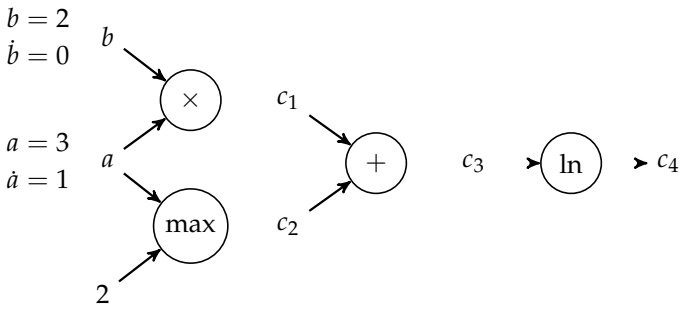


图2.6. 用于前向累积计算$\partial f/\partial a$的计算图，其中a =设为3，b =设为2，涉及$\ln(ab + \max(a, 2))$。

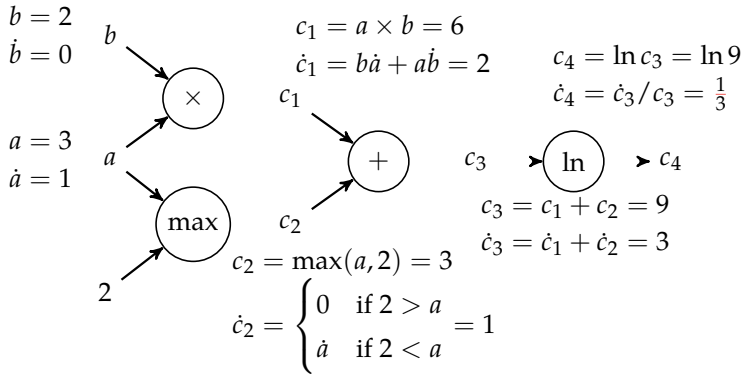


图 2.7. 应用前向累积计算 $\partial f / \partial a$ 时，$\ln(ab + \max(a, 2))$ 的计算图，其中 $a = 3$ 和 $b = 2$。

对偶数可以通过引入抽象量 ϵ 来数学地表示，其中 ϵ^2 被定义为 0。与复数类似，对偶数写作 $a + b\epsilon$，其中 a 和 b 均为实数值。我们有：

$$(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon \tag{2.32}$$

$$(a + b\epsilon) \times (c + d\epsilon) = (ac) + (ad + bc)\epsilon \tag{2.33}$$

事实上，通过将二元数传入任何光滑函数 f，我们可以得到求值及其导数。我们可以用泰勒级数来证明这一点：

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!}(x - a)^k \tag{2.34}$$

$$f(a + b\epsilon) = \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!}(a + b\epsilon - a)^k \tag{2.35}$$

$$= \sum_{k=0}^{\infty} \frac{f^{(k)}(a)b^k\epsilon^k}{k!} \tag{2.36}$$

$$= f(a) + bf'(a)\epsilon + \epsilon^2 \sum_{k=2}^{\infty} \frac{f^{(k)}(a)b^k}{k!}\epsilon^{(k-2)} \tag{2.37}$$

$$= f(a) + bf'(a)\epsilon \tag{2.38}$$

例2.5展示了一个实现。

2.4.2 *Reverse Accumulation*

前向累积需要 n 次传递才能计算 n 维梯度。*Reverse accumulation*[11] 只需运行一次即可计算

完整梯度，但需要两次遍历图：一次 *forward pass*，在此期间计算必要的中间值，以及一次 *backward pass*，其计算梯度。当需要梯度时，反向累积通常优于前向累积，但在计算图非常大且内存受限的系统中必须谨慎行事。[12]

11 S. Linnainmaa, “将算法的累积舍入误差表示为局部舍入误差的泰勒展开”，硕士论文，赫尔辛基大学，1970年。

12 反向累积是用于训练神经网络的误差反向传播算法的核心。D. E. Rumelhart, G. E. Hinton, and R. J. Williams, ‘‘Learning Representa- tions by Back-Propagating Errors,’’ *Nature*, vol. 323, pp. 533–536, 1986.

与正向累积类似，反向累积将计算关于所选目标变量的偏导数，但迭代地替换外部函数：

$$\frac{df}{dx} = \frac{df}{dc_4}\frac{dc_4}{dx} = \left(\frac{df}{dc_3}\frac{dc_3}{dc_4}\right)\frac{dc_4}{dx} = \left(\left(\frac{df}{dc_2}\frac{dc_2}{dc_3} + \frac{df}{dc_1}\frac{dc_1}{dc_3}\right)\frac{dc_3}{dc_4}\right)\frac{dc_4}{dx} \tag{2.39}$$

示例 2.5。对偶数的一种实现允许自动前向累积。

可以通过定义一个包含两个字段的结构体 `Dual` 来实现对偶数，这两个字段分别是值 `v` 和导数 `∂`。

```
struct Dual
    v
    ∂
end
```

然后，我们必须为所需的每个基础运算实现方法。这些方法接收对偶数，并根据该运算的链式法则逻辑生成新的对偶数。

```
Base.:+(a::Dual, b::Dual) = Dual(a.v + b.v, a.∂ + b.∂)
Base.:*(a::Dual, b::Dual) = Dual(a.v * b.v, a.v*b.∂ + b.v*a.∂)
Base.log(a::Dual) = Dual(log(a.v), a.∂/a.v)
function Base.max(a::Dual, b::Dual)
    v = max(a.v, b.v)
    ∂ = a.v > b.v ? a.∂ : a.v < b.v ? b.∂ : NaN
    return Dual(v, ∂)
end
function Base.max(a::Dual, b::Int)
    v = max(a.v, b)
    ∂ = a.v > b ? a.∂ : a.v < b ? 0 : NaN
    return Dual(v, ∂)
end
```

`ForwardDiff.jl` 包支持大量的数学运算，并且还提供梯度与海森矩阵。

```
julia> using ForwardDiff
julia> a = ForwardDiff.Dual(3,1);
julia> b = ForwardDiff.Dual(2,0);
julia> log(a*b + max(a,2))
Dual{Nothing}(2.1972245773362196,0.3333333333333333)
```

这个过程是反向传递，其评估需要在前向传递过程中获得的中间值。

反向累积可以通过*operation overloading*[13]在a中实现。
类似于使用对偶数实现前向累积的方式。每个基本操作必须实现两个函数：一个前向操作，在正向传递期间重载该操作以存储局部梯度信息；以及一个后向操作，利用该信息将梯度向后传播。像 Tensorflow[14] 或 `Zygote.jl` 这样的包可以

13 操作重载是指为自定义变量类型提供如 +、− 或 = 等常见操作的实现。重载在附录 A.2.5 中讨论。

14 Tensorflow 是一个开源软件库，用于使用数据流图进行数值计算，常用于深度学习应用。它可以从 `tensorflow.org` 获取。

自动构建计算图以及相关的前向和后向传递操作。示例2.6展示了如何使用 `Zygote.jl`。

`Zygote.jl` 包以反向累积的形式提供自动微分。这里使用 `gradient` 函数自动生成通过 `f` 源代码的反向传递，以获得梯度。

```
julia> import Zygote: gradient
julia> f(a, b) = log(a*b + max(a,2));
julia> gradient(f, 3.0, 2.0)
(0.3333333333333333, 0.3333333333333333)
```

示例 2.6. 使用 `Zygote.jl` 包进行自动微分。我们发现点 [3, 2] 处的梯度是 [1/3, 1/3]。

2.5 *Summary*

- 导数在优化中很有用，因为它们提供了关于如何改变给定点以改进目标函数的信息。
- 对于多元函数，多种基于导数的概念有助于指导寻找最优解，包括梯度、海森矩阵和方向导数。
- 数值微分的一种方法包括有限差分近似。
- 复数步长法可以在取小步长时消除减法抵消误差的影响，从而获得高质量的梯度估计。
- 解析微分方法包括计算图上的前向累积和反向累积。

2.6 Exercises

练习 2.1. 采用前向差分法，利用 $f(\mathrm{x})$ 的梯度 $\nabla f(\mathrm{x})$ 来近似其海森矩阵。

练习2.2。如果我们已经知道$f(\mathrm{x})$，那么中心差分法相较于其他有限差分法有什么缺点?

练习2.3。计算点x接近零时$f(x) = \ln x + e^x + \frac{1}{x}$的梯度。表达式中哪一项占主导?

练习2.4。假设$f(x)$是一个实值函数，同时也定义在复数输入上。如果$f(3 + ih) = 2 + 4ih$，那么$f'(3)$是多少?

练习2.5. 绘制$f(x、y) = \sin(x + y^2)$的计算图。利用该计算图通过前向累积计算在$(x, y) = (1, 1)$处的$\partial f/\partial y$。在通过图传播时，标记中间值和偏导数。

练习2.6。结合前向和后向差分方法，利用三个函数评估值，获得一个用于估计函数f在x处的二阶导数的差分方法。

3 *Bracketing*

本章介绍了针对单变量函数或涉及单个变量的函数的各种*bracketing*方法。区间定位是确定局部最小值所在区间并逐步縮小区间的过程。对于许多函数，导数信息有助于指导寻找最优解，但对于某些函数，这些信息可能不可用或不存在。本章概述了利用不同假设的多种方法。后续章节将在此基础上探讨多变量优化，并进一步拓展本章所介绍的概念。

3.1 *Unimodality*

本章介绍的若干算法都假设目标函数满足 *unimodality*。*unimodal function* f 是指存在一个 *unique* x^*，使得 f 在 $x \leq x^*$ 上单调递减，在 $x \geq x^*$ 上单调递增。由该定义可知，唯一的全局最小值位于 x^*，且不存在其他局部最小值。[1]

1 通常更常见的做法是反向定义单峰函数，即存在唯一的全局 *maximum* 而非最小值。然而，在本文中，我们致力于最小化函数，因此采用本段中的定义。

给定一个单峰函数，如果我们能找到三个点 $a < b < c$，使得 $f(a) > f(b) < f(c)$，我们就可以 *bracket* 一个包含全局最小值的区间 $[a, c]$。图3.1展示了一个例子。

3.2 *Finding an Initial Bracket*

在优化一个函数时，我们通常首先框定一个包含局部最小值的区间。然后，我们逐步缩小这个括号区间的大小，以收敛到局部最小值。一个简单的过程（算法3.1）可以用来找到初始括号。从一个给定的点开始，我们迈出一步。

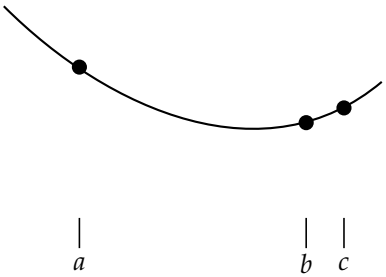


图3.1. 三个点显示包围一个最小值。

在正方向上。我们采用的步长是此算法的 *hyperparameter*，[2]，但提供的算法默认将其设为 1×10^{-2}。接着我们在

下坡方向寻找一个超越最低点的新点。每一步，我们通过某个因子扩大步长，这是该算法的另一个超参数，通常设置为2。图3.2展示了一个示例。没有局部最小值的函数，如exp(x)，无法被包围，将导致`bracket_minimum`失败。

2 超参数是控制算法功能的参数。它可以由专家设置，也可以通过优化算法进行调优。本文中的许多算法都有超参数。我们通常提供文献中建议的默认值。算法的成功可能对超参数的选择很敏感。

```
function bracket_minimum(f, x=0; s=1e-2, k=2.0)
    a, ya = x, f(x)
    b, yb = a + s, f(a + s)
    if yb > ya
        a, b = b, a
        ya, yb = yb, ya
        s = -s
    end
    while true
        c, yc = b + s, f(b + s)
        if yc > yb
            return a < c ? (a, c) : (c, a)
        end
        a, ya, b, yb = b, yb, c, yc
        s *= k
    end
end
```

算法3.1. 用于确定局部最小值必然存在的区间的算法。它接受一元函数`f`和起始位置`x`作为输入，其中起始位置默认为0。可以指定起始步长`s`和扩展因子`k`。它返回一个包含新区间$[a, b]$的元组。

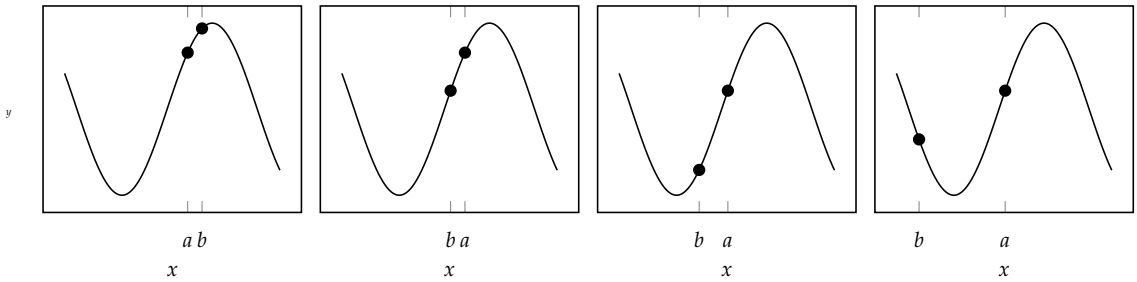


图3.2. 在一个函数上运行`bracket_minimum`的示例。该方法在第一次和第二次迭代之间改变方向，然后在第四次迭代时扩展直到bracketed 一个最小值。

3.3 *Fibonacci Search*

假设我们有一个由区间 $[a, b]$ 包围的单峰 f。给定对目标函数查询次数的限制，*Fibonacci search* (算法 3.2) 保证最大限度地缩小包围区间。

假设我们只能查询 f 两次。如果在区间的三分之一和三分之二处查询 f，那么无论 f 如何，我们都能确保去除区间的三分之一，如图3.3所示。

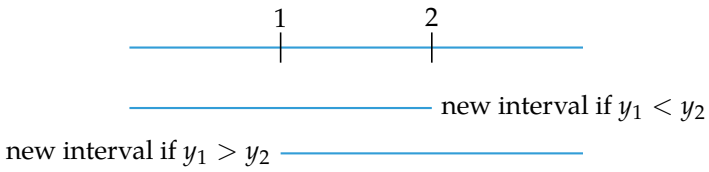


图3.3。我们对两个查询的初始猜测将移除初始区间的三分之一。

通过向中心移动我们的猜测，我们可以确保得到更紧凑的区间。当$\epsilon \to$趋近于0的极限情况下，如图3.4所示，我们保证我们的猜测会缩小两倍。

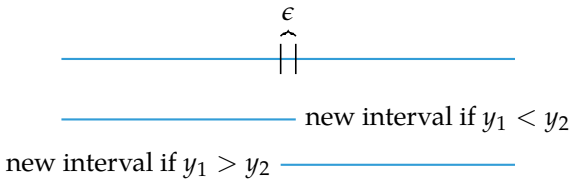


图3.4. 我们最多能保证将区间缩小到略少于一半。

通过三次查询，我们可以将区间缩小为原来的三分之一。首先，我们在区间的三分之一和三分之二处查询f，舍弃三分之一区间，然后在更优样本的紧邻位置采样，如图3.5所示。

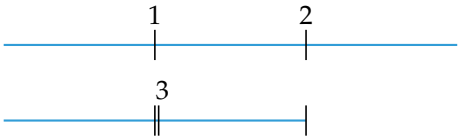


图3.5。通过三个查询，我们可以将域缩小为原来的三分之一。第三个查询是基于前两个查询的结果进行的。

对于 n 查询，区间的长度与斐波那契数列相关：1, 1, 2, 3, 5, 8，以此类推。前两项为1，后续每一项均为前两项之和：

$$F_n = \begin{cases} 1 & \text{if } n \leq 2 \\ F_{n-1} + F_{n-2} & \text{otherwise} \end{cases} \tag{3.1}$$

图3.6展示了区间之间的关系。例3.1演示了如何将其应用于单变量函数。

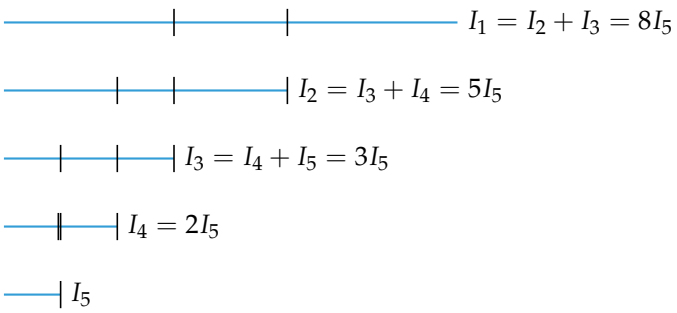


图3.6。对于n查询，我们保证将区间缩小F_{n+1}倍。斐波那契搜索过程中构造的每个区间的长度都可以表示为最终区间长度乘以一个斐波那契数。如果最终的最小区间长度为I_n，那么第二小的区间长度为$I_{n-1} = F_2 I_n$，第三小的区间长度为$I_{n-2} = F_3 I_n$，依此类推。

斐波那契数列可以通过*Binet's formula*解析确定：

$$F_n = \frac{\varphi^n - (1-\varphi)^n}{\sqrt{5}}, \tag{3.2}$$

其中 $\varphi = (1+\sqrt{5})/2 \approx 1.61803$ 是 *golden ratio*。

斐波那契数列中相邻两个值的比率是：

$$\frac{F_n}{F_{n-1}} = \varphi \frac{1 - s^{n+1}}{1 - s^n} \tag{3.3}$$

其中 $s = (1-\sqrt{5})/(1+\sqrt{5}) \approx -0.382$。

```
function fibonacci_search(f, a, b, n; ϵ=0.01)
    s = (1-√5)/(1+√5)
    ρ = 1 / (φ*(1-s^(n+1))/(1-s^n))
    d = ρ*b + (1-ρ)*a
    yd = f(d)
    for i in 1 : n-1
        if i == n-1
            c = ϵ*a + (1-ϵ)*d
        else
            c = ρ*a + (1-ρ)*b
        end
        yc = f(c)
        if yc < yd
            b, d, yd = d, c, yc
        else
            a, b = b, c
        end
        ρ = 1 / (φ*(1-s^(n-i+1))/(1-s^(n-i)))
    end
    return a < b ? (a, b) : (b, a)
end
```

算法3.2. 对单变量函数`f`执行斐波那契搜索，使用包围区间$[a, b]$，进行`n` >次函数评估。它返回新区间`(a, b)`。可选参数`ϵ`控制最低层区间。

3.4 Golden Section Search

如果我们取大n的极限，我们会发现斐波那契数列连续值的比例趋近于黄金比例：

$$\lim_{n\to\infty} \frac{F_n}{F_{n-1}} = \varphi. \tag{3.4}$$

Golden section search (算法3.3)使用黄金比例来近似斐波那契搜索。图3.7展示了区间之间的关系。图3.8和图3.9分别比较了在单峰和非单峰函数上的斐波那契搜索与黄金分割搜索。

示例 3.1。使用五次函数评估的斐波那契搜索来优化单变量函数。

考虑使用具有五次函数求值的斐波那契搜索，以在区间 $[a, b] = [-2, 6]$ 上最小化 $f(x) = \exp(x - 2) - x$。前两次函数求值在 $\frac{F_5}{F_6}$ 和 $1 - \frac{F_5}{F_6}$ 处进行，沿初始包围区间的长度：

$$\begin{aligned} f(x^{(1)}) &= f\left(a + (b-a)\left(1 - \frac{F_5}{F_6}\right)\right) && = f(1) = -0.632 \\ f(x^{(2)}) &= f\left(a + (b-a)\frac{F_5}{F_6}\right) && = f(3) = -0.282 \end{aligned}$$

在$x^{(1)}$处的评估值较低，得到新的区间$[a, b] = [-2, 3]$。下一次区间分割需要进行两次评估：

$$\begin{aligned} x_{\text{left}} &= a + (b-a)\left(1 - \frac{F_4}{F_5}\right) = 0 \\ x_{\text{right}} &= a + (b-a)\frac{F_4}{F_5} = 1 \end{aligned}$$

因此，在x_{left}处进行了第三次函数求值，因为x_{right}已经被求值：

$$f(x^{(3)}) = f(0) = 0.135$$

在$x^{(1)}$处的评估较低，从而得到新区间$[a, b] = [0, 3]$。下一次区间分割需要进行两次评估：

$$\begin{aligned} x_{\text{left}} &= a + (b-a)\left(1 - \frac{F_3}{F_4}\right) = 1 \\ x_{\text{right}} &= a + (b-a)\frac{F_3}{F_4} = 2 \end{aligned}$$

因此，在x_{right}处进行了第四次功能评估，因为x_{left}已经被评估过：

$$f(x^{(4)}) = f(2) = -1$$

新的区间是$[a, b] = [1, 3]$。在区间中心附近$2 + \epsilon$处进行最终评估，发现其值略高于$f(2)$。最终区间为$[1, 2 + \epsilon]$。

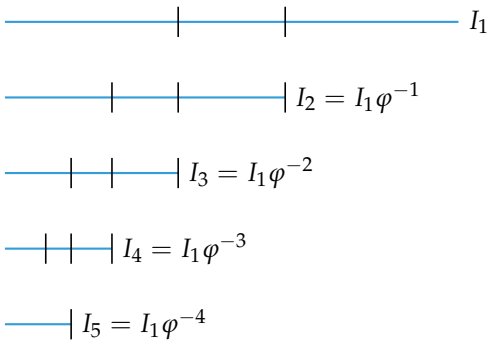


图3.7。对于单变量函数的n查询，我们保证将包围区间缩小φ^{n-1}倍。

```
function golden_section_search(f, a, b, n)
    ρ = φ-1
    d = ρ * b + (1 - ρ)*a
    yd = f(d)
    for i = 1 : n-1
        c = ρ*a + (1 - ρ)*b
        yc = f(c)
        if yc < yd
            b, d, yd = d, c, yc
        else
            a, b = b, c
        end
    end
    return a < b ? (a, b) : (b, a)
end
```

算法3.3. 对单变量函数`f`执行黄金分割搜索，给定区间$[a, b]$，进行`n` $>$次函数评估。返回新区间`(a, b)`。Julia中已定义了黄金比例`φ`。确保收敛至ϵ以内需要$n = (b-a)/(\epsilon \ln \varphi)$次迭代。

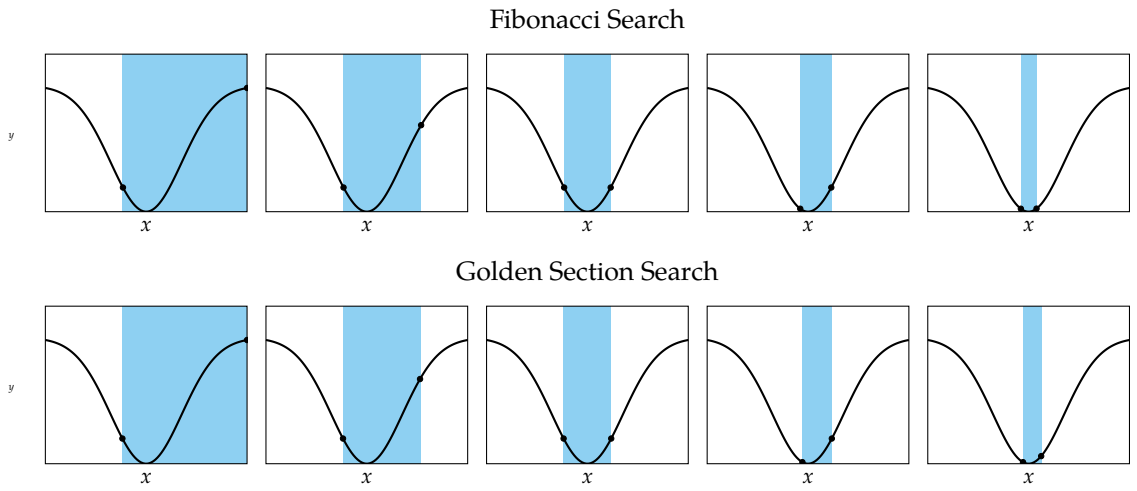


图3.8. 在单峰函数上的斐波那契与黄金分割搜索。

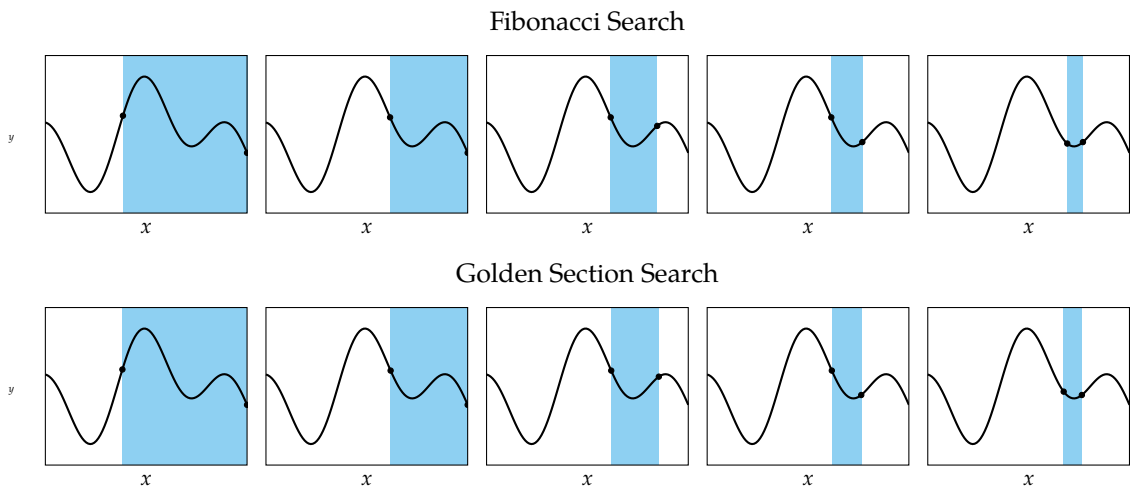


图3.9. 非单峰函数上的斐波那契与黄金分割搜索。

3.5 *Quadratic Fit Search*

Quadratic fit search 利用了我们对二次函数最小值进行解析求解的能力。许多局部最小值在放大到足够近时看起来都是二次的。二次拟合搜索迭代地将二次函数拟合到三个包围点，求解最小值，选择一组新的包围点，并重复，如图3.10所示。

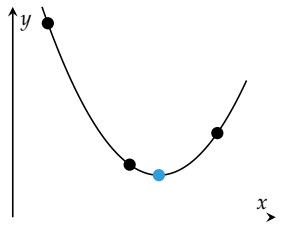


图3.10. 二次拟合搜索将二次函数拟合到三个包围点（黑点），并利用解析最小值（蓝点）来确定下一组包围点。

给定包围点 $a < b < c$，我们希望找到二次函数 q 的系数 p_1、p_2 和 p_3，该函数经过 (a, y_a)、(b, y_b) 和 (c, y_c)：

$$q(x) = p_1 + p_2 x + p_3 x^2 \tag{3.5}$$
$$y_a = p_1 + p_2 a + p_3 a^2 \tag{3.6}$$
$$y_b = p_1 + p_2 b + p_3 b^2 \tag{3.7}$$
$$y_c = p_1 + p_2 c + p_3 c^2 \tag{3.8}$$

以矩阵形式表示为

$$\begin{bmatrix} y_a \\ y_b \\ y_c \end{bmatrix} = \begin{bmatrix} 1 & a & a^2 \\ 1 & b & b^2 \\ 1 & c & c^2 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} \tag{3.9}$$

我们可以通过矩阵求逆来求解系数 n:

$$\begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} = \begin{bmatrix} 1 & a & a^2 \\ 1 & b & b^2 \\ 1 & c & c^2 \end{bmatrix}^{-1} \begin{bmatrix} y_a \\ y_b \\ y_c \end{bmatrix} \tag{3.10}$$

我们的二次函数则为

$$q(x) = y_a \frac{(x-b)(x-c)}{(a-b)(a-c)} + y_b \frac{(x-a)(x-c)}{(b-a)(b-c)} + y_c \frac{(x-a)(x-b)}{(c-a)(c-b)} \tag{3.11}$$

我们可以通过找到导数为零的位置来求解唯一的最小值：

$$x^* = \frac{1}{2} \frac{y_a(b^2 - c^2) + y_b(c^2 - a^2) + y_c(a^2 - b^2)}{y_a(b - c) + y_b(c - a) + y_c(a - b)} \tag{3.12}$$

二次拟合搜索通常比黄金分割搜索更快。对于下一个点非常接近其他点的情况，它可能需要保护措施。算法3.4提供了一个基本实现。图3.11展示了该算法的几次迭代。

```
function quadratic_fit_search(f, a, b, c, n)
    ya, yb, yc = f(a), f(b), f(c)
    for i in 1:n-3
        x = 0.5*(ya*(b^2-c^2)+yb*(c^2-a^2)+yc*(a^2-b^2)) /
                (ya*(b-c)    +yb*(c-a)    +yc*(a-b))
        yx = f(x)
        if x > b
            if yx > yb
                c, yc = x, yx
            else
                a, ya, b, yb = b, yb, x, yx
            end
        elseif x < b
            if yx > yb
                a, ya = x, yx
            else
                c, yc, b, yb = b, yb, x, yx
            end
        end
    end
    return (a, b, c)
end
```

算法3.4. 对单变量函数`f`进行二次拟合搜索，区间$[a, c]$，具有$a < b < c$。该方法将运行`n`次函数评估。它返回新的区间值作为元组，`(a, b, c)`。

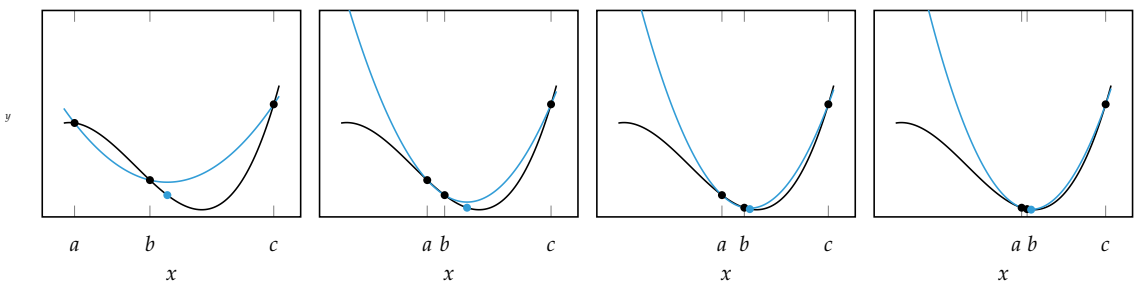


图3.11. 二次拟合方法的四次迭代。

3.6 Shubert-Piyavskii Method

与本 chapter 中的先前方法相比，*Shubert-Piyavskii method*[3]是一个在域 $[a, b]$ 上的 *global optimization method*，这意味着它保证收敛到函数的全局最小值，不受任何局部最小值或函数是否单峰的影响。算法3.5提供了一个基本实现。

3 S. Piyavskii，“寻找函数绝对极值的算法”，*USSR Computational Mathematics and Mathematical Physics*，第12卷，第4期，第57-67页，1972年。B. O. Shubert，“寻求函数全局最大值的序贯方法”，*SIAM Journal on Numerical Analysis*，第9卷，第3期，第379-388页，1972年。

Shubert-Piyavskii 方法要求函数是 *Lipschitz continuous*，即它是连续的，并且其导数的幅度存在上界。函数 f 在 $[a, b]$ 上是利普希茨连续的，如果存在一个 $\ell > 0$ 使得：[4]

$$|f(x) - f(y)| \leq \ell |x - y| \text{ for all } x, y \in [a, b] \tag{3.13}$$

4 我们可以将 Lipschitz 连续性的定义推广到多元函数，其中 x 和 y 是向量，绝对值由任意向量范数代替。

直观地说，ℓ与函数在$[a, b]$上的最大无符号瞬时变化率一样大。给定一个点$(x_0, f(x_0))$，我们知道$x > x_0$的$f(x_0) - \ell(x - x_0)$线和$x < x_0$的$f(x_0) + \ell(x - x_0)$线构成了f的下界。

舒伯特-皮亚夫斯基方法迭代地构建越来越紧的函数下界。给定一个有效的Lipschitz常数ℓ，算法从采样中点$x^{(1)} = (a + b)/2$开始。从这个点出发，使用斜率为$\pm\ell$的直线构建锯齿形下界。如果ℓ是有效的Lipschitz常数，如图3.12所示，这些直线将始终位于f之下。

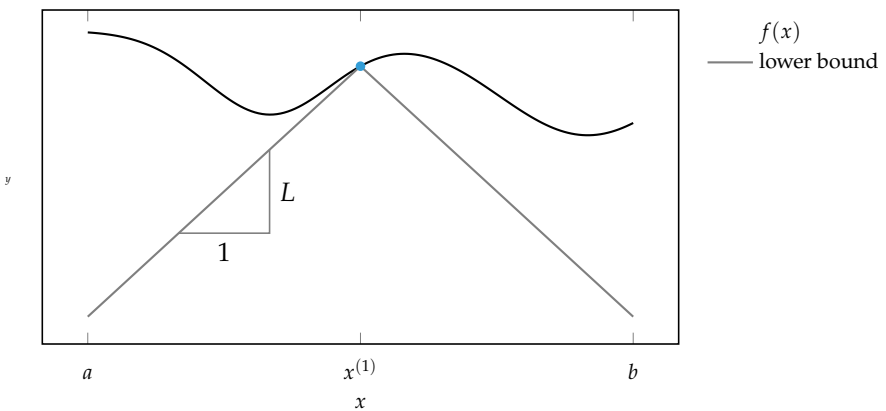


图3.12. Shubert-Piyavskii方法的第一轮迭代。

锯齿中的上顶点对应于采样点。下顶点对应于源自每个点的利普希茨线之间的交点

采样点。进一步的迭代在锯齿波中找到最小点，评估该x值处的函数，然后使用结果更新锯齿波。图3.13说明了这一过程。

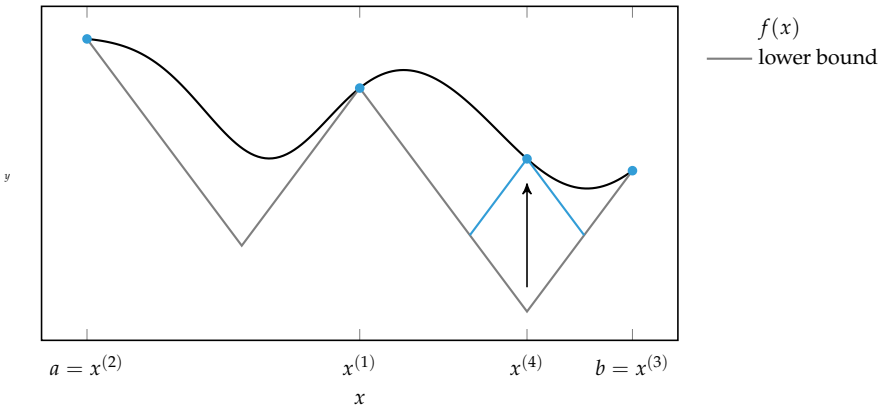


图3.13。更新下界涉及采样一个新点，并将新线与现有的锯齿形相交。

当最小锯齿值与函数在该点的评估值之间的高度差小于给定容差 ϵ 时，算法通常会停止。对于最小峰值（$x^{(n)}$，$y^{(n)}$）和函数评估 $f(x^{(n)})$，如果 $y^{(n)} - f(x^{(n)}) < \epsilon$，我们便终止。

可以利用这个更新信息计算最小值可能存在的区域。对于每个峰值，可以根据以下公式计算不确定区域：

$$\left[x^{(i)} - \frac{1}{\ell}(f(x_{\text{min}}) - y^{(i)}), x^{(i)} + \frac{1}{\ell}(y^{(i)} - y_{\text{min}})\right] \tag{3.14}$$

对于每个锯齿下顶点（$x^{(i)}$、$y^{(i)}$）和最小锯齿上顶点（x_{min}、y_{min}）。只有当 $y^{(i)} < y_{\text{min}}$ 时，一个点才会贡献一个不确定区域。最小值位于这些峰值不确定区域之一。

Shubert-Piyavskii 方法的主要缺点是需要知道一个有效的 Lipschitz 常数。较大的 Lipschitz 常数会导致下界较差。图 3.14 展示了 Shubert-Piyavskii 方法的几次迭代。

```
struct Pt
    x
    y
end
function _get_sp_intersection(A, B, l)
    t = ((A.y - B.y) - l*(A.x - B.x)) / 2l
    return Pt(A.x + t, A.y - t*l)
end
function shubert_piyavskii(f, a, b, l, ϵ, δ=0.01)
    m = (a+b)/2
    A, M, B = Pt(a, f(a)), Pt(m, f(m)), Pt(b, f(b))
    pts = [A, _get_sp_intersection(A, M, l),
           M, _get_sp_intersection(M, B, l), B]
    Δ = Inf
    while Δ > ϵ
        i = argmin([P.y for P in pts])
        P = Pt(pts[i].x, f(pts[i].x))
        Δ = P.y - pts[i].y

        P_prev = _get_sp_intersection(pts[i-1], P, l)
        P_next = _get_sp_intersection(P, pts[i+1], l)

        deleteat!(pts, i)
        insert!(pts, i, P_next)
        insert!(pts, i, P)
        insert!(pts, i, P_prev)
    end

    intervals = []
    i = 2*(argmin([P.y for P in pts[1:2:end]])) - 1
    for j in 2:2:length(pts)
        if pts[j].y < pts[i].y
            dy = pts[i].y - pts[j].y
            x_lo = max(a, pts[j].x - dy/l)
            x_hi = min(b, pts[j].x + dy/l)
            if !isempty(intervals) && intervals[end][2] + δ ≥ x_lo
                intervals[end] = (intervals[end][1], x_hi)
            else
                push!(intervals, (x_lo, x_hi))
            end
        end
    end
    return (pts[i], intervals)
end
```

算法3.5。对单变量函数f执行Shubert-Piyavskii方法，包含包围区间 a < b和利普希茨常数l。算法运行直至更新量小于容差ϵ。返回最佳点和不确定区间集。不确定区间以(a,b)元组数组形式返回。参数δ是用于合并不确定区间的容差。

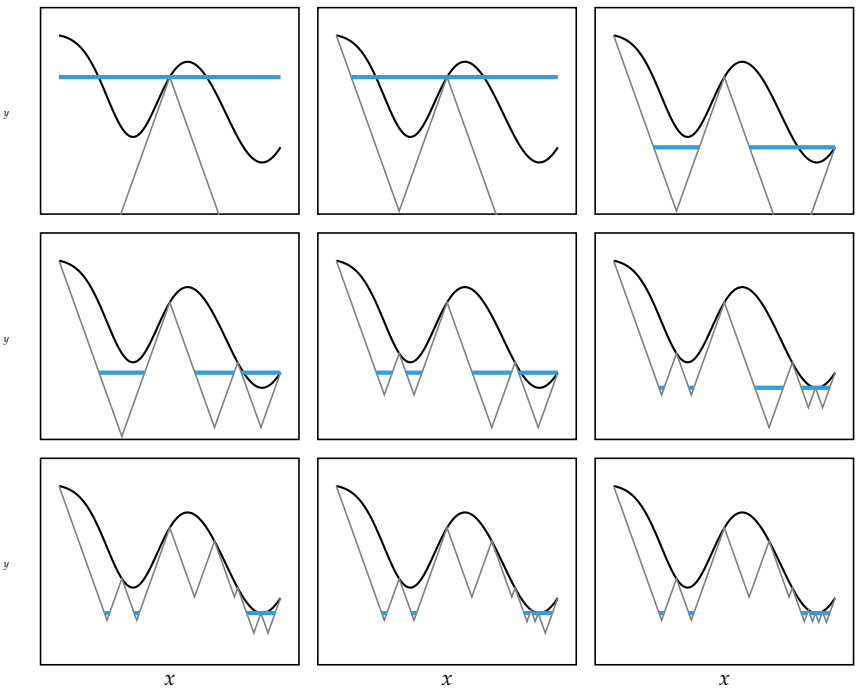


图3.14. Shubert-Piyavskii方法的九次迭代，按从左到右、从上到下的顺序进行。蓝线表示可能包含全局最小值的不确定区域。

3.7 Bisection Method

bisection method (算法3.6)可用于寻找函数的*roots*，或函数为零的点。这些*root-finding methods*可以通过应用于目标函数的导数来用于优化，定位$f'(x) =$为0的位置。一般来说，我们必须确保得到的点确实是局部最小值。

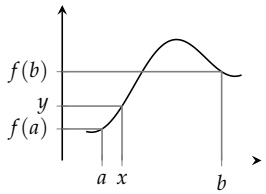


图3.15。从任意$y \in [f(a), f(b)]$绘制的水平线必须至少与图形相交一次。

二分法维持一个区间 $[a, b]$，其中已知至少存在一个根。如果 f 在 $[a, b]$ 上连续，并且存在某个 $y \in [f(a), f(b)]$，那么 *intermediate value theorem* 规定至少存在一个 $x \in [a, b]$，使得 $f(x) = y$ 如图3.15所示。由此可知，如果 $f(a)$ 和 $f(b)$ 的符号相反，则区间 $[a, b]$ 必定包含一个零点。

二分法在每次迭代时将包围区间减半。计算中点 $(a + b)/2$，并根据中点以及继续包围零点的那一侧形成新的区间。如果中点求值结果为零，我们可以立即终止。否则，我们可以在固定次数的迭代后终止。图3.16展示了二分法的四次迭代。该方法保证在$\lg\left(\frac{|b-a|}{\epsilon}\right)$次迭代内收敛到$x^*$的$\epsilon$范围内，其中lg表示以2为底的对数。

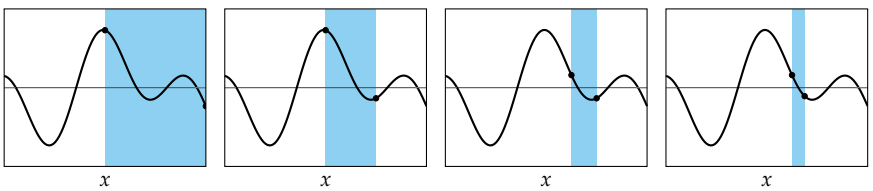


图3.16。二分法的四次迭代。水平线对应于{v*} 0。注意，初始区间内存在多个根。

像二分法这样的求根算法需要从零点的两侧开始区间 $[a, b]$。也就是说，sign$(f'(a)) \neq$ sign$(f'(b))$，或者等价地，$f'(a)f'(b) \leq 0$。算法3.7提供了一种自动确定此类区间的方法。它从一个猜测区间 $[a, b]$ 开始。只要区间无效，其宽度就按常数因子增加。将区间大小加倍是一种常见选择。如图3.17所示，这种方法并不总会成功。具有两个相近根的函数可能会被漏掉，导致区间无限增大而不终止。

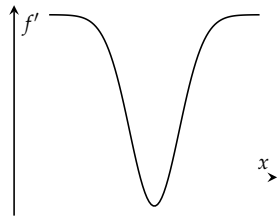


图 3.17. 若括住方法的初始区间设置得如同本图所示跨越两个根，则该区间将永远扩展，永远不会发现符号变化。此外，如果初始区间位于两个根之间，则区间加倍可能导致区间的两端同时越过这两个根。

Brent-Dekker 方法是二分法的扩展。它是一种求根算法，结合了割线法（第6.2节）和

```
function bisection(f′, a, b, ϵ)
    if a > b; a,b = b,a; end # ensure a < b

    ya, yb = f′(a), f′(b)
    if ya == 0; b = a; end
    if yb == 0; a = b; end

    while b - a > ϵ
        x = (a+b)/2
        y = f′(x)
        if y == 0
            a, b = x, x
        elseif sign(y) == sign(ya)
            a = x
        else
            b = x
        end
    end

    return (a,b)
end
```

算法3.6。二分法，其中`f′`是我们试图优化的单变量函数的导数。我们有`a < b`，它们包围了`f′`的一个零点。区间宽度容差是`ϵ`。调用`bisection`会返回新的包围区间$[a, b]$作为一个元组。

撇号字符 ′ 不是单引号。因此，`f′` 是变量名而非转置向量 `f`。该符号可通过输入 `\prime` 并按 tab 键来创建。

```
function bracket_sign_change(f′, a, b; k=2)
    if a > b; a,b = b,a; end # ensure a < b

    center, half_width = (b+a)/2, (b-a)/2
    while f′(a)*f′(b) > 0
        half_width *= k
        a = center - half_width
        b = center + half_width
    end

    return (a,b)
end
```

算法 3.7. 一种寻找符号变化区间的算法。输入是在实数上定义的实值函数 `f′`，以及起始区间 $[a, b]$。它通过扩展区间宽度，直到在区间边界上计算的函数值之间出现符号变化，返回新区间作为一个元组。扩展因子 `k` 默认为 2。

逆二次插值。它具有可靠且快速的收敛特性，是许多流行的数值优化软件包中首选的单变量优化算法。[5]

[5] 该算法的细节可在R. P. Brent, *Algorithms for Minimization Without Derivatives*中找到。Prentice Hall, 1973年。该算法是T. J. Dekker工作的扩展，‘‘通过逐次线性插值寻找零点’’，见 *Constructive Aspects of the Fundamental Theorem of Algebra*，B. Dejon和P Henrici，Interscience，1969年。

3.8 Summary

- 许多优化方法会缩小包围区间，包括斐波那契搜索、黄金分割搜索和二次拟合搜索。
- Shubert-Piyavskii方法在给定Lipschitz常数的情况下，输出一组包含全局最小值的区间包围。
- 像二分法这样的求根方法可以用来求函数的导数为零的位置。

3.9 Exercises

练习 3.1。给出一个例子，说明在什么情况下斐波那契搜索优于二分法。

练习3.2. Shubert-Piyavskii方法的一个缺点是什么?

练习3.3。给出一个非平凡函数的例子，使得一旦三个不同点上的函数值已知，二次拟合搜索就能正确找到最小值。

练习3.4。假设我们有$f(x) = x^2/2 - x$。应用二分法，从区间[0, 1000]开始，找到一个包含f极小值点的区间。执行算法的三个步骤。

练习 3.5。假设我们在区间 [0, 1] 上有一个函数$f(x) = (x + 2)^2$。2 是该区间上 f 的有效 Lipschitz 常数吗?

练习 3.6. 假设我们有一个定义在区间 [1, 32] 上的单峰函数。经过三次我们选择的功能评估后，我们是否能够将最优解缩小到一个长度最多为 10 的区间? 为什么能或为什么不能?

4 Local Descent

到目前为止，我们聚焦于涉及单个设计变量的优化问题。本章将介绍一种涉及*multivariate*函数，即包含多个变量的函数的通用优化方法。本章重点在于如何利用*local models*逐步改进设计点，直至满足某种收敛准则。我们首先讨论在每次迭代中基于局部模型选择下降方向，然后确定步长的方法。接着，我们将探讨限制步长处于认为局部模型有效的区域内的方法。本章最后将讨论收敛条件。接下来两章将讨论如何利用基于梯度或海森矩阵信息构建的一阶和二阶模型进行优化。

4.1 Descent Direction Iteration

一种常见的优化方法是逐步改进设计点 $\mathbf{x}$，通过采取基于局部模型最小化目标值的步骤。局部模型可以例如从一阶或二阶泰勒近似获得。遵循这一通用方法的优化算法被称为*descent direction methods*。它们从一个设计点 $\mathbf{x}^{(1)}$ 开始，然后生成一系列点，有时称为*iterates*，以收敛到局部最小值。[1]

迭代下降方向过程包括以下步骤：

1. 检查$\mathbf{x}^{(k)}$是否满足终止条件。若满足，则终止；否则继续执行下一步。

2. 使用梯度或海森矩阵等局部信息确定 *descent direction* $\mathbf{d}^{(k)}$。一些算法假设 $\|\mathbf{d}^{(k)}\| = 1$，但其他算法则不然。

1 选择 $\mathbf{x}^{(1)}$ 的值可能会影响算法寻找最小值的成功与否。领域知识常用于选择一个合理的值。当缺乏此类知识时，我们可以利用第13章中将要介绍的技术在设计空间中进行搜索。

3. 确定步长或学习率$\alpha^{(k)}$。一些算法尝试优化步长，以使步长最大程度地减少f。[2]

4. 根据以下内容计算下一个设计点：

$$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \underbrace{\alpha^{(k)}}_{\textit{rate}}\mathbf{d}^{(k)} \tag{4.1}$$

有许多不同的优化方法，每种方法都有其确定{v*}和d的独特方式。

2 我们使用*step size*来表示总体步长的幅度。使用公式（4.1）以步长$\alpha^{(k)}$获得新的迭代点意味着下降方向$\mathbf{d}^{(k)}$具有单位长度。我们使用*learning*来表示作用于下降方向向量上的标量倍数，该向量不一定具有单位长度。

4.2 Line Search

目前，假设我们已经选择了一个下降方向d，或许采用了后续章节中讨论的某种方法。我们需要选择步长因子α以获得下一个设计点。一种方法是使用*line search*，它选取使一维函数最小化的步长因子：

$$\underset{\alpha}{\text{minimize}}\, f(\mathbf{x} + \alpha\mathbf{d}) \tag{4.2}$$

线搜索是一个单变量优化问题，已在第3章中讨论过。我们可以应用所选的单变量优化方法。[3] 为了告知

搜索时，我们可以利用线搜索目标的导数，即沿方向d在点$\mathbf{x} + \alpha\mathbf{d}$ 处的方向导数。线搜索在例4.1中进行了演示，并在算法4.1中实现。

3 上一章提到的 Brent-Dekker 方法是一种常用的单变量优化方法。它结合了二分法的稳健性和割线法的速度。

```
function line_search(f, x, d)
    objective = α -> f(x + α*d)
    a, b = bracket_minimum(objective)
    α = minimize(objective, a, b)
    return x + α*d
end
```

算法4.1。一种进行线搜索的方法，该方法沿下降方向d从设计点x寻找最优步长因子，以最小化函数f。minimize函数可以使用单变量优化算法如Brent-Dekker方法来实现。

在每一步进行线搜索的一个缺点是，将α优化到高精度所需的计算成本。相反，通常的做法是快速找到一个合理的值，然后继续进行，选择$\mathbf{x}^{(k+1)}$，然后选取新方向$\mathbf{d}^{(k+1)}$。

一些算法使用固定的步长因子。大步长往往会导致更快的收敛，但存在越过最小值的风险。较小的步长往往更稳定，但可能导致收敛变慢。固定的步长因子 α 有时被称为 *learning rate*。

另一种方法是使用*decaying step factor*：

$$\alpha^{(k)} = \alpha^{(1)}\gamma^{k-1} \quad \text{for } \gamma \in (0, 1] \tag{4.3}$$

衰减步长因子在最小化噪声目标函数时尤其流行，[4]并且在机器学习应用中常用。

我们将在第17章讨论在噪声和其他形式的不确定性存在下的优化问题。

考虑对 $f(x_1、x_2、x_3) = \sin(x_1x_2) + \exp(x_2 + x_3) - x_3$ 从 $\mathrm{x} = [1, 2, 3]$ 沿方向 $\mathrm{d} = [0, -1, -1]$ 进行线搜索。相应的优化问题为：

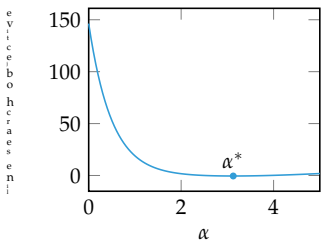
$$\underset{\alpha}{\text{minimize}}\ \sin((1 + 0\alpha)(2 - \alpha)) + \exp((2 - \alpha) + (3 - \alpha)) - (3 - \alpha)$$

简化为：

$$\underset{\alpha}{\text{minimize}}\ \sin(2 - \alpha) + \exp(5 - 2\alpha) + \alpha - 3$$

最小值位于 $\alpha \approx 3.127$，其中 $\mathrm{x} \approx [1, -1.126, -0.126]$。

例4.1. 沿下降方向使用线搜索以最小化函数。



4.3 Approximate Line Search

通常来说，进行更多次的下降法迭代往往比在每次迭代中执行精确线搜索在计算上更为高效，特别是当函数及其导数计算成本较高时。迄今讨论的许多方法都能受益于使用*approximate line search*来寻找一个只需少量评估的合适步长。由于下降法必须确保下降，只要步长α能导致目标函数值减小，它可能就是合适的。然而，为了促进更快的收敛，还可以施加多种其他条件。

sufficient decrease[5] 的条件要求步长导致一个足够的目标函数值的减少：

$$f(\mathbf{x}^{(k+1)}) \leq f(\mathbf{x}^{(k)}) + \beta\alpha\nabla_{\mathbf{d}^{(k)}} f(\mathbf{x}^{(k)}) \tag{4.4}$$

5 这个条件有时被称为 *Armijo condition*。

其中 $\beta \in [0, 1]$ 通常设置为 $\beta = 1 \times 10^{-4}$。图4.1说明了这一条件。如果 $\beta = 0$，那么任何减少都是可接受的。如果 $\beta = 1$，那么减少量至少要与一阶近似所预测的一样多。

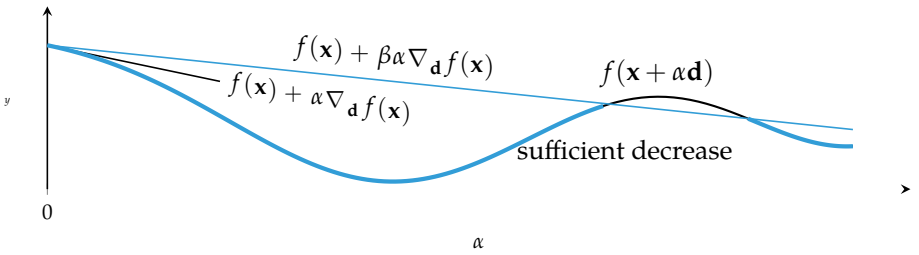


图4.1。充分下降条件，即第一个Wolfe条件，总能通过沿下降方向的足够小步长得到满足。

如果 d 是一个有效的下降方向，那么必定存在一个足够小的步长，使得充分下降条件得以满足。因此，我们可以从一个较大的步长开始，并以恒定的缩减因子逐步减小步长，直至满足充分下降条件。这个算法被称为*backtracking line search*[6]，因为关于它如何沿着下降方向回溯。回溯线搜索如图4.2所示，并在算法4.2中实现。我们通过例4.2来逐步说明这一过程。

6 也称为*Armijo line search*，L. Armijo，“具有Lipschitz连续一阶偏导数的函数的最小化”，*Pacific Journal of Mathematics*，第16卷，第1期，第1–3页，1966年。

```
function backtracking_line_search(f, ∇f, x, d, α; p=0.5, β=1e-4)
    y, g = f(x), ∇f(x)
    while f(x + α*d) > y + β*α*(g⋅d)
        α *= p
    end
    α
end
```

算法4.2。回溯线搜索算法，其目标函数为`f`，梯度为`∇f`，当前设计点为`x`，下降方向为`d`，最大步长为`α`。我们可以可选地指定缩减因子`p`和第一个Wolfe条件参数`β`。

第一个条件不足以保证收敛到局部最小值。非常小的步长能满足第一个条件，但可能会导致过早收敛。回溯线搜索通过接受经过连续缩放获得的最大满意步长，避免了过早收敛，并能保证收敛到局部最小值。

请注意，cdot字符`⋅`是`dot`函数的别名，因此`a⋅b`等同于`dot(a,b)`。该符号可以通过输入`\cdot`并按下制表键来创建。

另一个条件，称为*curvature condition*，要求下一次迭代的方向导数更平缓：

$$\nabla_{\mathbf{d}^{(k)}} f(\mathbf{x}^{(k+1)}) \geq \sigma \nabla_{\mathbf{d}^{(k)}} f(\mathbf{x}^{(k)}) \tag{4.5}$$

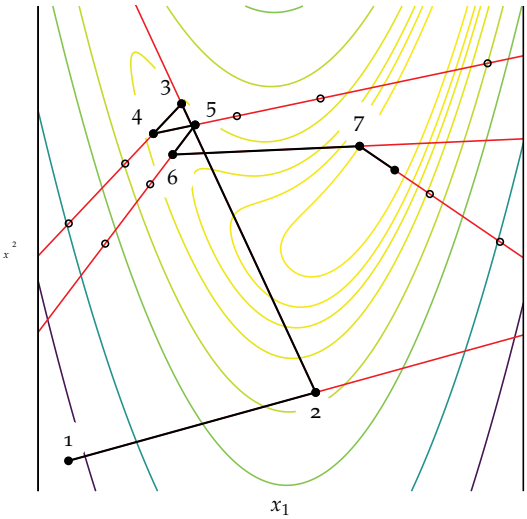


图4.2. 在Rosenbrock函数（附录B.6）上使用的回溯线搜索。黑线表示下降法所进行的七次迭代，红线表示每次线搜索过程中考虑的点。

其中 σ 控制下一个方向导数必须有多浅。图4.3和图4.4说明了这一条件。当近似线性搜索与共轭梯度法一起使用时，通常设置 $\beta < \sigma < 1$，$\sigma = 0.1$；当与牛顿法一起使用时，则设置为0.9。[7]

曲率条件的一个替代方案是*strong Wolfe condition*，这是一个更为严格的标准，因为它还要求斜率不能过于正：

$$|\nabla_{\mathbf{d}^{(k)}} f(\mathbf{x}^{(k+1)})| \leq -\sigma \nabla_{\mathbf{d}^{(k)}} f(\mathbf{x}^{(k)}) \tag{4.6}$$

图4.5说明了这一条件。

充分下降条件和第一曲率条件一起构成了 *Wolfe conditions*。充分下降条件通常被称为 *first Wolfe condition*，曲率条件被称为 *second Wolfe condition*。充分下降条件与第二曲率条件一起构成了 *strong Wolfe conditions*。

7 共轭梯度法在第5.2节中介绍，牛顿法在第6.1节中介绍。

满足强 Wolfe 条件需要更复杂的算法，*strong backtracking line search* (算法 4.3)。[8] 该方法分两个阶段进行。

第一阶段，*bracketing phase*，连续测试更大的步长，以括住一个保证含有满足Wolfe条件的步长的区间[$\alpha^{(k-1)}$, $\alpha^{(k)}$]。

8 J. Nocedal 和 S. J. Wright, *Numer-*, 第2版，Springer，2006年。

例4.2. 回溯线搜索的一个示例，这是一种近似线搜索方法。

考虑在方向 d = [−1, −1] 上，从 x = [1, 2] 中的 $f(x_1$、$x_2) = x_1^2 + x_1x_2 + x_2^2$ 进行近似线搜索，使用最大步长为 10、缩减因子为 0.5、第一 Wolfe 条件参数 $\beta = 1 \times 10^{-4}$ 和第二 Wolfe 条件参数 $\sigma = 0.9$。

我们检查最大步长是否满足第一个沃尔夫条件，其中 x 处的梯度为 g = [4, 5]：

$$f(\mathbf{x} + \mathbf{d}) \leq f(\mathbf{x}) + (\mathbf{g}^\top \mathbf{d})$$
$$f([1,2] + 10 \cdot [-1,-1]) \leq 7 + 1 \times 10^{-4} \cdot 10 \cdot [4,5]^\top [-1,-1]$$
$$217 \leq 6.991$$

它并不满足。

步长乘以0.5得到5，并再次检查第一个Wolfe条件：

$$f([1,2] + 5 \cdot [-1,-1]) \leq 7 + 1 \times 10^{-4} \cdot 5 \cdot [4,5]^\top [-1,-1]$$
$$37 \leq 6.996$$

它并不满足。

步长乘以0.5得到2.5，并再次检查第一个Wolfe条件：

$$f([1,2] + 2.5 \cdot [-1,-1]) \leq 7 + 1 \times 10^{-4} \cdot 2.5 \cdot [4,5]^\top [-1,-1]$$
$$3.25 \leq 6.998$$

第一个 Wolfe 条件得到满足。

候选设计点 $\mathbf{x}' = \mathbf{x} + \alpha\mathbf{d} = [-1.5, -0.5]$ 针对第二个 Wolfe 条件进行检验：

$$\nabla_{\mathbf{d}} f(\mathbf{x}') \geq \sigma \nabla_{\mathbf{d}} f(\mathbf{x})$$
$$[-3.5, -2.5]^\top [-1,-1] \geq \sigma\, [4,5]^\top [-1,-1]$$
$$6 \geq -8.1$$

第二个沃尔夫条件得到满足。

近似线搜索终止于 x = [−1.5, −0。 5]。

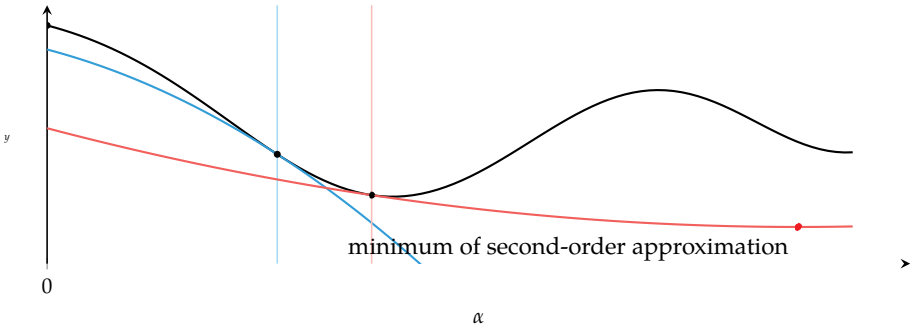


图4.3。曲率条件，即第二个Wolfe条件，是确保二阶函数近似具有正曲率，从而具有唯一全局最小值的必要条件。

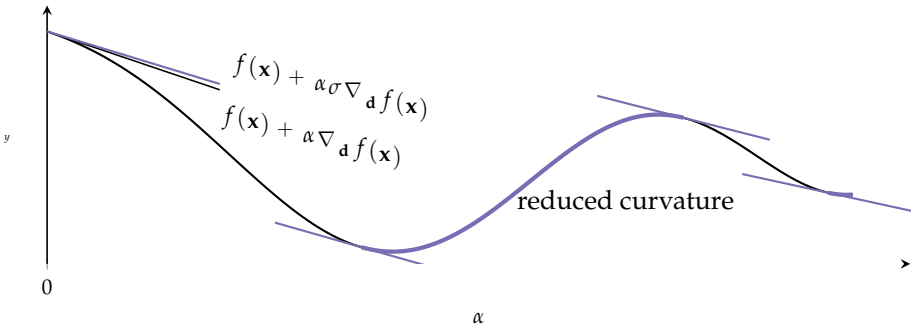


图4.4. 满足曲率条件的区域。

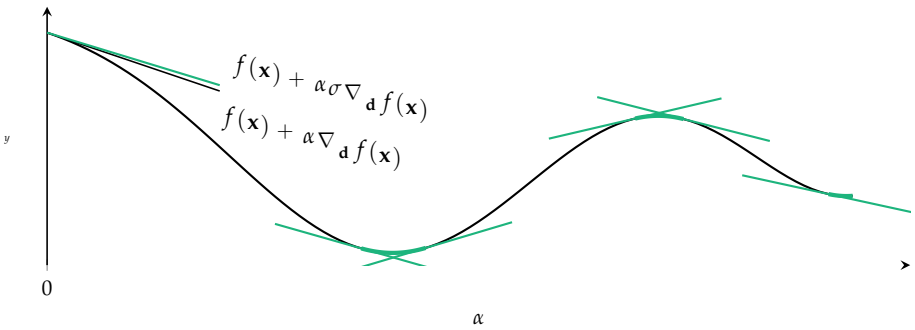


图4.5. 满足强曲率条件的区域。

当以下条件之一成立时，就找到了一个区间，该区间保证包含满足Wolfe条件的步长：

$$f(\mathbf{x} + \alpha^{(k)}\mathbf{d}) \geq f(\mathbf{x}) \tag{4.7}$$

$$f(\mathbf{x}^{(k)} + \alpha^{(k)}\mathbf{d}^{(k)}) > f(\mathbf{x}^{(k)}) + \beta\alpha^{(k)}\nabla_{\mathbf{d}^{(k)}} f(\mathbf{x}^{(k)}) \tag{4.8}$$

$$\nabla f(\mathbf{x} + \alpha^{(k)}\mathbf{d}) \geq \mathbf{0} \tag{4.9}$$

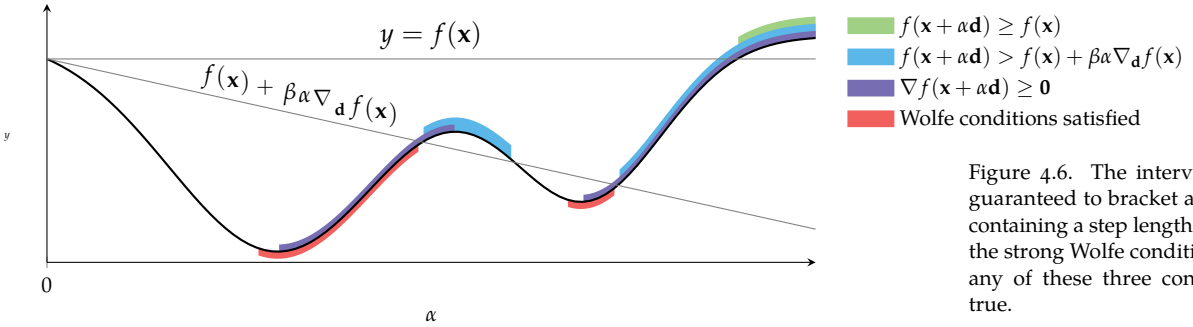


Figure 4.6. The interval $[0, \alpha]$ 是guaranteed to bracket an i间隔containing a step length sa满足the strong Wolfe conditions 当any of these three condi操作是true.

满足方程(4.8)等价于违反第一个Wolfe条件，从而确保缩短步长将保证得到一个满意的步长。类似地，方程(4.7)和方程(4.9)保证了下降步长已越过局部最小值，因此两者之间的区域必定包含一个满意的步长。

图4.6展示了在一个示例线搜索中，每个包围条件成立的位置。图中显示了包围区间$[0, \alpha]$，而高级回溯线搜索则逐步增加步长，以获得包围区间$[\alpha^{(k-1)}, \alpha^{(k)}]$。

在*zoom phase*中，我们缩小区间以寻找满足强沃尔夫条件的步长。缩小区间可采用二分法（第3.7节），根据相同的区间条件更新区间边界。这一过程如图4.7所示。

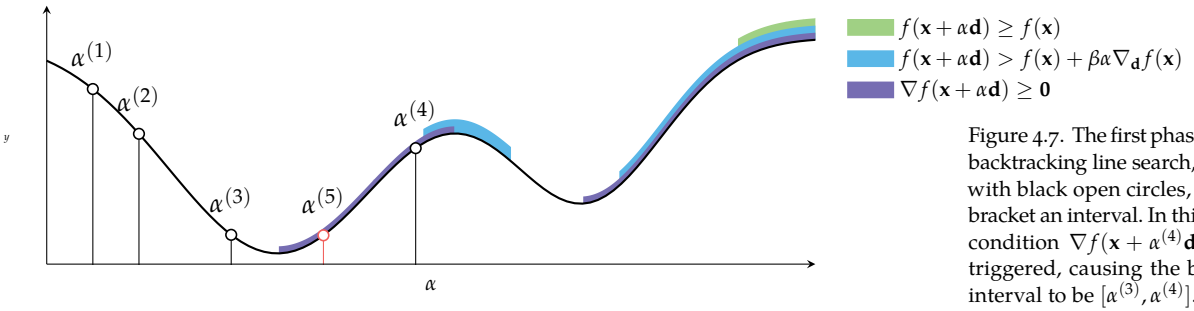


Figure 4.7. The first phase of 强 backtracking line search, ind指示 with black open circles, is used 到 bracket an interval. In this ca是, condition $\nabla f(\mathbf{x} + \alpha^{(4)}\mathbf{d}) \geq 0$ 是 triggered, causing the brac刻亭 interval to be $[\alpha^{(3)}, \alpha^{(4)}]$. Thzh 的 zoom phase, indicated by t红色 的空心圆会缩小括号区域，直到找 到合适的步长。

4.4 Trust Region Methods

下降方法可能过于信任其一阶或二阶信息，这可能导致步长过大或过早收敛。*trust region*[9]是设计空间中局部模型被认为可信的局部区域。

要可靠。信赖域方法，或称*restricted step method*，维护信赖域的局部模型，该模型既限制传统线搜索所采取的步长，又预测采取该步长所带来的改进。如果改进与预测值紧密匹配，则扩大信赖域。如果改进偏离预测值，则缩小信赖域。[10]

9 K. Levenberg，“一种求解特定非线性最小二乘问题的方法，” *Quar-*，第2卷，第2期，第164-168页，1944年。

terly of Applied Mathematics

10 关于信赖域方法的近期综述由Y. X. Yuan提供，“信赖域算法的 recent advances”，*Mathemati-*，第151卷，第1期，第249-281页，2015年。

cal Programming

图4.8展示了一个位于圆形信赖域中心的设计点。

信赖域方法首先选择最大步长，然后选择步方向，这与先选择步方向再优化步长的线搜索方法形成对比。信赖域方法通过在当前设计点$\mathbf{x}$为中心的信任域上最小化目标函数f的模型来寻找下一步。$\hat{f}$的一个例子是二阶泰勒近似（见附录C.2）。信任域的半径δ根据模型预测函数评估的准确程度进行扩展和收缩。下一个设计点$\mathbf{x}'$通过求解以下问题获得：

$$\begin{aligned} \underset{\mathbf{x}'}{\text{minimize}} \quad & \hat{f}(\mathbf{x}') \\ \text{subject to} \quad & \left\|\mathbf{x} - \mathbf{x}'\right\| \leq \delta \end{aligned} \tag{4.10}$$

其中信任区域由正半径δ和向量范数[11]定义。

上面的方程是一个约束优化问题，这在第10章中有所涉及。

11 有多种高效方法可以高效求解方程 (4.10)。关于应用于二次模型的信赖域方法的概述，参见 D. C. Sorensen，“Newton's Method with a Model Trust Region Modification，” *SIAM Journal on Numerical Analysis*, vol. 19, no. 2, pp. 409–426, 1982。

```
function strong_backtracking(f, ∇, x, d; α=1, β=1e-4, σ=0.1)
    y0, g0, y_prev, α_prev = f(x), ∇(x)⋅d, NaN, 0
    αlo, αhi = NaN, NaN

    # bracket phase
    while true
        y = f(x + α*d)
        if y > y0 + β*α*g0 || (!isnan(y_prev) && y ≥ y_prev)
            αlo, αhi = α_prev, α
            break
        end
        g = ∇(x + α*d)⋅d
        if abs(g) ≤ -σ*g0
            return α
        elseif g ≥ 0
            αlo, αhi = α, α_prev
            break
        end
        y_prev, α_prev, α = y, α, 2α
    end

    # zoom phase
    ylo = f(x + αlo*d)
    while true
        α = (αlo + αhi)/2
        y = f(x + α*d)
        if y > y0 + β*α*g0 || y ≥ ylo
            αhi = α
        else
            g = ∇(x + α*d)⋅d
            if abs(g) ≤ -σ*g0
                return α
            elseif g*(αhi - αlo) ≥ 0
                αhi = αlo
            end
            αlo = α
        end
    end
end
```

算法 4.3. 满足强 Wolfe 条件的强回溯近似线搜索。该算法以目标函数 `f`、梯度函数 `∇`、进行线搜索的设计点 `x` 和方向 `d`、初始步长 `α` 以及 Wolfe 条件参数 `β` 和 `σ` 作为输入。算法的括界阶段首先括住一个包含满足强 Wolfe 条件的步长的区间。然后，它在缩放阶段缩小该括住的区间，直到找到合适的步长。我们使用二分法进行插值，但也可以使用其他方案。

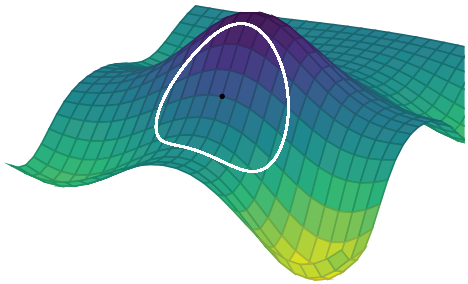


图4.8. 信赖域方法将下一步限制在局部区域内。信赖域的大小根据目标函数模型的预测性能进行扩展和收缩。

信赖域半径 δ 根据局部模型的预测性能进行扩大或缩小。信赖域方法将预测改进量 $\Delta y_{\text{pred}} = f(\mathbf{x}) - \hat{f}(\mathbf{x}')$ 与实际改进量 $\Delta y_{\text{act}} = f(\mathbf{x}) - f(\mathbf{x}')$ 进行比较：

$$\eta = \frac{\text{actual improvement}}{\text{predicted improvement}} = \frac{f(\mathbf{x}) - f(\mathbf{x}')}{f(\mathbf{x}) - \hat{f}(\mathbf{x}')} \tag{4.11}$$

当预测步长与实际步长匹配时，比率η接近于1。如果比率过小，例如低于阈值η_1，则认为改进程度远低于预期，信赖域半径按因子γ_1 <缩小1。如果比率足够大，例如高于阈值η_2，则认为我们的预测是准确的，信赖域半径按因子γ_2 >放大1。算法4.4提供了实现，图4.9可视化了优化过程。例4.3展示了如何构造非圆形信赖域。

4.5 Termination Conditions

下降方向法有四种常见的终止条件：

- *Maximum iterations.* 我们可能希望在迭代次数k超过某个阈值$k_{\max}$时终止。或者，我们可能希望在超过最大经过时间后终止。

$$k > k_{\max} \tag{4.12}$$

```
function trust_region_descent(f, ∇f, H, x, k_max;
    η1=0.25, η2=0.5, γ1=0.5, γ2=2.0, δ=1.0)
    y = f(x)
    for k in 1 : k_max
        x′, y′ = solve_trust_region_subproblem(∇f, H, x, δ)
        r = (y - f(x′)) / (y - y′)
        if r < η1
            δ *= γ1
        else
            x, y = x′, y′
            if r > η2
                δ *= γ2
            end
        end
    end
    return x
end

using Convex
function solve_trust_region_subproblem(∇f, H, x0, δ)
    x = Variable(length(x0))
    p = minimize(∇f(x0)⋅(x-x0) + quadform(x-x0, H(x0))/2)
    p.constraints += norm(x-x0) <= δ
    solve!(p)
    return (x.value, p.optval)
end
```

算法4.4. 信赖域下降法，其中`f`是目标函数，`∇f`生成导数，`H`生成海森矩阵，`x`是初始设计点，`k_max`是迭代次数。可选参数`η1`和`η2`决定何时增大或减小信赖域半径`δ`，而`γ1`和`γ2`控制变化幅度。必须提供一个求解方程(4.10)的`solve_trust_region_subproblem`实现。我们提供了一个示例实现，该实现在`x0`附近使用二阶泰勒近似，并采用圆形信赖域。

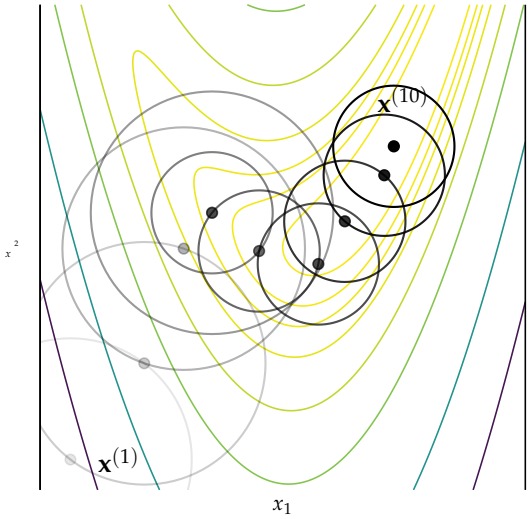


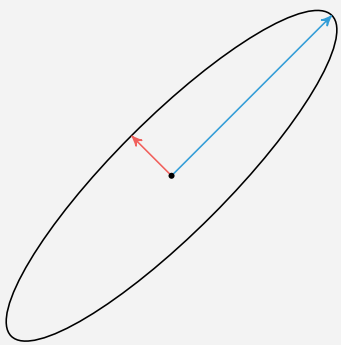
图4.9. 在罗森布鲁克函数上使用的信赖域优化（附录B.6）。

信赖域不必是圆形的。在某些情况下，某些方向可能比其他方向拥有更高的信任度。

可以构造一种范数以生成椭圆区域：

$$\|\mathbf{x} - \mathbf{x}_0\|_{\mathbf{E}} = (\mathbf{x} - \mathbf{x}_0)^\top \mathbf{E} (\mathbf{x} - \mathbf{x}_0)$$

使用 $\|\mathbf{x} - \mathbf{x}_0\|_{\mathbf{E}} \leq 1$，其中E是定义椭圆的对称矩阵。



椭圆矩阵 E 可以在每次下降迭代中更新，这涉及比缩放信任域更复杂的调整。

例4.3. 信赖域优化不必使用圆形信赖域。更多细节参见J. Nocedal和S. J. Wright的“信赖域方法”，收录于*Numerical Optimization*。Springer，2006年，第66-100页。

- *Absolute improvement.* 此终止条件考察后续步骤中函数值的变化。如果变化小于给定阈值，它就会终止：

$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^{(k+1)}) < \epsilon_a \tag{4.13}$$

- *Relative improvement.* 此终止条件同样关注函数值的变化，但使用了相对于当前函数值的步长因子：

$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^{(k+1)}) < \epsilon_r |f(\mathbf{x}^{(k)})| \tag{4.14}$$

- *Gradient magnitude.* 我们也可以基于梯度的大小来终止：

$$\|\nabla f(\mathbf{x}^{(k+1)})\| < \epsilon_g \tag{4.15}$$

在可能存在多个局部最小值的情况下，当满足我们的终止条件后，加入 *random restarts* 可能是有益的，此时我们从随机选择的初始点重新开始局部下降方法。

4.6 Summary

- 下降方向方法逐步下降以逼近局部最优解。
- 单变量优化可以在直线搜索期间应用 rch.
- 近似线搜索可用于确定合适的下降步长。
- 信赖域方法将步长限制在一个局部区域内，该区域根据预测准确性进行扩展或收缩。
- 下降方法的终止条件可以基于目标函数值的变化或梯度的模等准则。

4.7 Exercises

练习 4.1. 为什么拥有多个终止条件很重要?

练习 4.2. 第一个 Wolfe 条件要求

$$f(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)}) \leq f(\mathbf{x}^{(k)}) + \beta \alpha \nabla_{\mathbf{d}^{(k)}} f(\mathbf{x}^{(k)}) \tag{4.16}$$

在给定 $f(\mathrm{x}) = 5 + x_1^2 + x_2^2$、$\mathrm{x}(k) = [-1, -1]$、$\mathrm{d} = [1, 0]$ 和 $\beta = 10^{-4}$ 的情况下，满足该条件的最大步长 α 是多少?

5 First-Order Methods

上一章介绍了下降方向方法的一般概念。本章讨论使用*first-order*方法选择合适下降方向的各种算法。一阶方法依赖于梯度信息来帮助指导寻找最小值，这可以使用第2章中概述的方法获得。

5.1 Gradient Descent

下降方向 d 的一个直观选择是最速下降方向。只要目标函数光滑、步长足够小，并且我们尚未处于梯度为零的点，沿最速下降方向前进就一定能带来改进。[1] 该方向为

[1] A point where the gradient is zero is called a *stationary point*.

最速下降方向是梯度的相反方向 ∇f，因此得名 *gradient descent*。为方便本章讨论，我们定义

$$\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)}) \tag{5.1}$$

其中 x(k) 是我们在下降迭代 k 处的设计点。

在梯度下降中，我们通常对最陡下降方向进行归一化（见例5.1）：

$$\mathbf{d}^{(k)} = -\frac{\mathbf{g}^{(k)}}{\|\mathbf{g}^{(k)}\|} \tag{5.2}$$

如果选择的步长导致f最大程度减小，就会出现锯齿状的搜索路径。实际上，下一个方向总是与当前方向正交。我们可以这样证明：

例5.1. 计算梯度下降方向。

假设我们有 $f(\mathrm{x}) = x_1 x_2^2$。梯度是 $\nabla f = [x_2^2, 2x_1x_2]$。对于 $\mathrm{x}^{(k)} = [1, 2]$，我们得到未归一化的最陡下降方向 $\mathrm{d} = [-4, -4]$，归一化为 $\mathrm{d} = [-\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}]$。

如果我们在每一步都优化步长，我们有

$$\alpha^{(k)} = \arg\min_{\alpha} f(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)}) \tag{5.3}$$

上述优化意味着方向导数等于零。使用公式(2.9)，我们有

$$\nabla f(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)})^\top \mathbf{d}^{(k)} = 0 \tag{5.4}$$

我们知道

$$\mathbf{d}^{(k+1)} = -\frac{\nabla f(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)})}{\|\nabla f(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)})\|} \tag{5.5}$$

因此，

$$\mathbf{d}^{(k+1)\top} \mathbf{d}^{(k)} = 0 \tag{5.6}$$

这意味着$\mathrm{d}^{(k+1)}$和$\mathrm{d}^{(k)}$是正交的。

与下降方向一致的狭窄山谷不是问题。当下降方向横穿山谷时，为了沿着谷底前进，必须采取许多步骤，如图5.1所示。梯度下降的实现由算法5.1提供。

5.2 *Conjugate Gradient*

梯度下降在狭窄的山谷中可能表现不佳。*conjugate gradient*方法通过借鉴优化二次函数的方法来解决这个问题：

$$\operatorname*{minimize}_{\mathbf{x}} f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} + \mathbf{b}^\top \mathbf{x} + c \tag{5.7}$$

其中 A 是对称正定的，因此 f 具有唯一的局部极小值（第1.6.2节）。

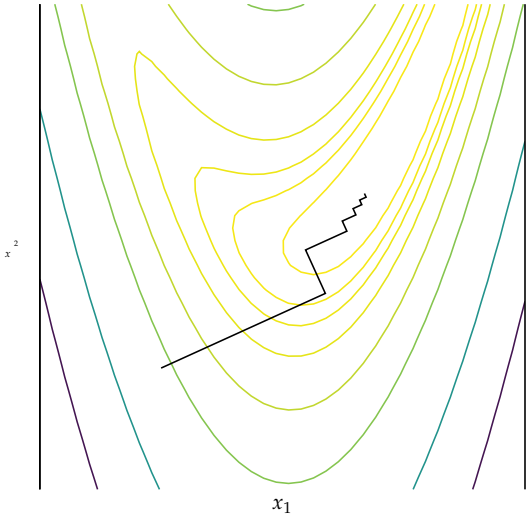


图5.1. 梯度下降在狭窄的峡谷中可能导致之字形移动。此处我们看到对Rosenbrock函数的影响（附录B.6）。

```
abstract type DescentMethod end
struct GradientDescent <: DescentMethod
    α
end
init!(M::GradientDescent, f, ∇f, x) = M
function step!(M::GradientDescent, f, ∇f, x)
    α, g = M.α, ∇f(x)
    return x - α*g
end
```

算法5.1. 梯度下降法，它以固定的学习率沿梯度下降方向进行。`step!` 函数产生下一次迭代，而 `init` 函数则不做任何事。

共轭梯度法可以在 n 步内优化 n 维二次函数，如图 5.2 所示。其方向关于 A 是 *mutually conjugate*：

$$\mathbf{d}^{(i)\top}\mathbf{A}\,\mathbf{d}^{(j)} = 0 \;\text{ for all }\; i \neq j \tag{5.8}$$

共轭向量是A的基向量。它们通常彼此不正交。

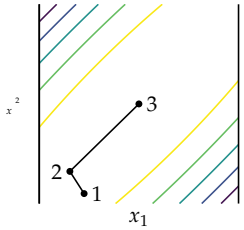


图 5.2. 当应用于一个 n 维二次函数时，共轭梯度下降在 n 步内收敛。

利用梯度信息和前一次下降方向来计算连续的共轭方向。该算法从最速下降方向开始：

$$\mathbf{d}^{(1)} = -\mathbf{g}^{(1)} \tag{5.9}$$

然后我们使用线搜索来寻找下一个设计点。对于二次函数，步长因子α可以精确计算（例5.2）。更新则为：

$$\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + \alpha^{(1)}\mathbf{d}^{(1)} \tag{5.10}$$

假设我们想要推导二次函数上线性搜索的最优步长因子：

$$\underset{\alpha}{\text{minimize}}\; f(\mathbf{x} + \alpha\mathbf{d})$$

我们可以计算关于α的导数：

$$\begin{aligned}\frac{\partial f(\mathbf{x}+\alpha\mathbf{d})}{\partial\alpha} &= \frac{\partial}{\partial\alpha}\left[\frac{1}{2}(\mathbf{x}+\alpha\mathbf{d})^\top\mathbf{A}(\mathbf{x}+\alpha\mathbf{d}) + \mathbf{b}^\top(\mathbf{x}+\alpha\mathbf{d}) + c\right] \\ &= \mathbf{d}^\top\mathbf{A}(\mathbf{x}+\alpha\mathbf{d}) + \mathbf{d}^\top\mathbf{b} \\ &= \mathbf{d}^\top(\mathbf{A}\mathbf{x}+\mathbf{b}) + \alpha\mathbf{d}^\top\mathbf{A}\mathbf{d}\end{aligned}$$

将 $\frac{\partial f(\mathbf{x}+\alpha\mathbf{d})}{\partial\alpha} =$ 设置为 0 会导致：

$$\alpha = -\frac{\mathbf{d}^\top(\mathbf{A}\mathbf{x}+\mathbf{b})}{\mathbf{d}^\top\mathbf{A}\mathbf{d}}$$

例5.2. 二次函数上进行线搜索的最优步长因子。

后续迭代根据下一个梯度和当前下降方向的贡献来选择d(k+1)：

$$\mathbf{d}^{(k+1)} = -\mathbf{g}^{(k+1)} + \beta^{(k)}\mathbf{d}^{(k)} \tag{5.11}$$

对于标量参数 β。β 的值越大，表明先前的下降方向贡献越强。

对于已知的 A，我们可以利用 d(k+1) 与 d(k) 共轭这一事实，推导出 β 的最佳值：

$$\mathbf{d}^{(k+1)\top}\mathbf{A}\mathbf{d}^{(k)} = 0 \tag{5.12}$$

$$\Rightarrow (-\mathbf{g}^{(k+1)} + \beta^{(k)}\mathbf{d}^{(k)})^\top \mathbf{A}\mathbf{d}^{(k)} = 0 \tag{5.13}$$

$$\Rightarrow -\mathbf{g}^{(k+1)^\top}\mathbf{A}\mathbf{d}^{(k)} + \beta^{(k)}\mathbf{d}^{(k)\top}\mathbf{A}\mathbf{d}^{(k)} = 0 \tag{5.14}$$

$$\Rightarrow \beta^{(k)} = \frac{\mathbf{g}^{(k+1)^\top}\mathbf{A}\mathbf{d}^{(k)}}{\mathbf{d}^{(k)\top}\mathbf{A}\mathbf{d}^{(k)}} \tag{5.15}$$

共轭梯度法也可以应用于非二次函数。光滑连续函数在接近局部最小值时表现得像二次函数，共轭梯度法在这些区域会很快收敛。

遗憾的是，我们不知道在 x(k) 附近最佳近似 f 的 A 值。相反，几种 $\beta^{(k)}$ 的选择往往效果不错：

Fletcher-Reeves：[2]

$$\beta^{(k)} = \frac{\mathbf{g}^{(k)\top}\mathbf{g}^{(k)}}{\mathbf{g}^{(k-1)\top}\mathbf{g}^{(k-1)}} \tag{5.16}$$

Polak-Ribière：[3]

$$\beta^{(k)} = \frac{\mathbf{g}^{(k)\top}\left(\mathbf{g}^{(k)} - \mathbf{g}^{(k-1)}\right)}{\mathbf{g}^{(k-1)\top}\mathbf{g}^{(k-1)}} \tag{5.17}$$

如果我们将Polak-Ribière方法（算法5.2）修改为允许自动重置，则可保证其收敛性：

$$\beta \leftarrow \max(\beta, 0) \tag{5.18}$$

图5.3展示了使用该方法的一个搜索示例。

2 R. Fletcher 和 C. M. Reeves，“共轭梯度函数最小化”，*The Computer Jour-nal*，第7卷，第2期，第149-154页，1964年。

3 E. Polak 和 G. Ribière，“关于共轭方向方法收敛性的注记”，*Revue Française d'informatique et de recherche opérationnelle, Série Rouge*，第3卷，第1期，第35-43页，1969年

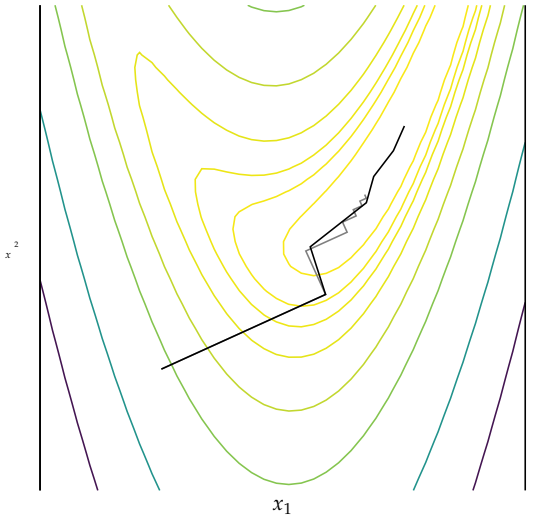


图5.3. 采用Polak-Ribière更新的共轭梯度法。梯度下降以灰色显示。

```
mutable struct ConjugateGradientDescent <: DescentMethod
    d
    g
end
function init!(M::ConjugateGradientDescent, f, ∇f, x)
    M.g = ∇f(x)
    M.d = -M.g
    return M
end
function step!(M::ConjugateGradientDescent, f, ∇f, x)
    d, g = M.d, M.g
    g′ = ∇f(x)
    β = max(0, dot(g′, g′-g)/(g⋅g))
    d′ = -g′ + β*d
    x′ = line_search(f, x, d′)
    M.d, M.g = d′, g′
    return x′
end
```

算法5.2. 采用Polak-Ribière更新的共轭梯度法，其中`d`是先前的搜索方向，`g`是先前的梯度。

5.3 *Momentum*

梯度下降在穿过几乎平坦的表面时会花费很长时间，如图5.4所示。允许动量累积是加快进度的一种方法。我们可以修改梯度下降以纳入动量。

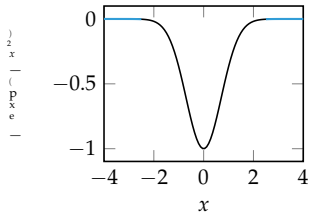


图5.4。几乎平坦的区域其梯度幅度较小，因此可能需要多次梯度下降迭代才能穿越。

momentum 的更新方程为：

$$\mathbf{v}^{(k+1)} = \beta \mathbf{v}^{(k)} - \alpha \mathbf{g}^{(k)} \tag{5.19}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)} \tag{5.20}$$

对于 $\beta = 0$，我们恢复为梯度下降。动量可以解释为一个球沿着几乎水平的斜坡滚下。就像重力使球加速，球自然地积累动量，同样地，梯度使动量在这种下降方法中累积。算法5.3提供了实现。图5.5中对比了动量下降与梯度下降。

```
mutable struct Momentum <: DescentMethod
    α # learning rate
    β # momentum decay
    v # momentum
end
function init!(M::Momentum, f, ∇f, x)
    M.v = zeros(length(x))
    return M
end
function step!(M::Momentum, f, ∇f, x)
    α, β, v, g = M.α, M.β, M.v, ∇f(x)
    v[:] = β*v - α*g
    return x + v
end
```

算法5.3. 加速下降的动量法。

`step!` 中的第一行复制标量 `α` 和 `β`，但创建对向量 `v` 的引用。因此，接下来的行 `v[:] = β*v - α*g` 修改结构体 `M` 中的原始动量向量。

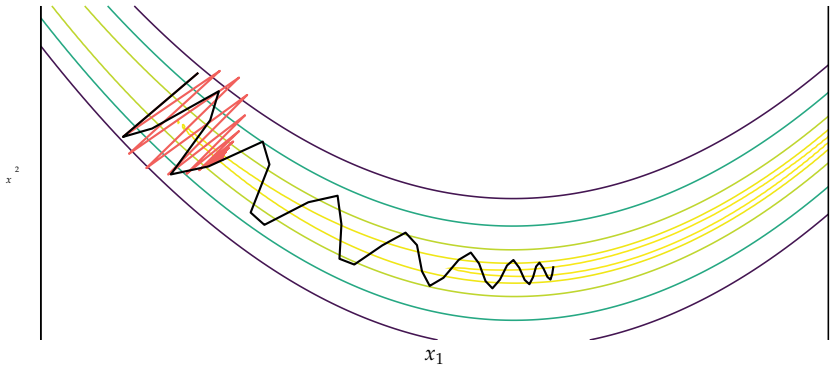


图5.5. 在Rosenbrock函数上比较梯度下降与动量法，取$b = 100$；参见附录B.6。

5.4 *Nesterov Momentum*

动量的一个问题是，在谷底时步长没有足够减慢，容易越过谷底。*Nesterov momentum*[4] 修改了要使用投影未来位置处梯度的动量算法：

$$\mathbf{v}^{(k+1)} = \beta\mathbf{v}^{(k)} - \alpha\nabla f(\mathbf{x}^{(k)} + \beta\mathbf{v}^{(k)}) \tag{5.21}$$

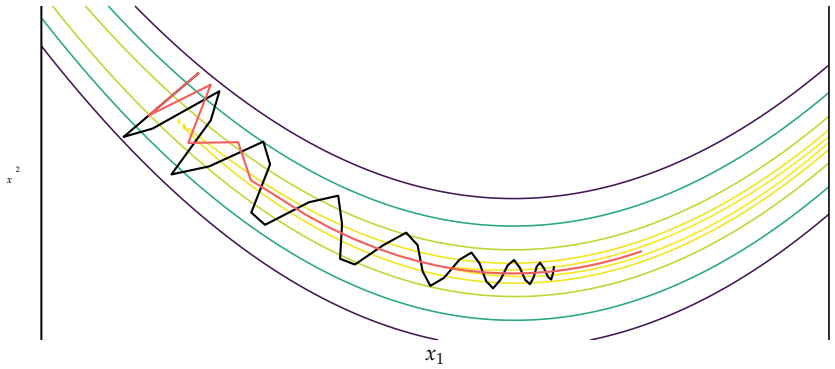
$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)} \tag{5.22}$$

4 Y. Nesterov，“一种求解凸规划问题且收敛率为 $O(1/k_2)$ 的方法”，*Soviet Mathematics Doklady*，第27卷，第2期，第543–547页，1983年。

算法5.4提供了一种实现。Nesterov动量法和动量下降法在图5.6中进行了比较。

```
mutable struct NesterovMomentum <: DescentMethod
    α # learning rate
    β # momentum decay
    v # momentum
end
function init!(M::NesterovMomentum, f, ∇f, x)
    M.v = zeros(length(x))
    return M
end
function step!(M::NesterovMomentum, f, ∇f, x)
    α, β, v = M.α, M.β, M.v
    v[:] = β*v - α*∇f(x + β*v)
    return x + v
end
```

算法5.4. Nesterov加速下降的动量法。



动量 Nesterov 动量

图5.6。动量法与Nesterov动量方法在b =为100时的Rosenbrock函数上的比较；参见附录B.6。

5.5 *Adagrad*

动量和Nesterov动量以相同的学习率更新x的所有分量。*adaptive subgradient*方法，或*Adagrad*、[5]自适应学习
每个 x 组件的速率。Adagrad 减弱了梯度持续较高的参数的影响，从而增加了更新不频繁的参数的影响。[6]

5 J. Duchi, E. Hazan, and Y. Singer, ‘‘自适应次梯度方法在在线学习和随机优化中的应用,’’ *Journal of Machine Learning Research*, vol. 12, pp. 2121–2159, 2011.

6 Adagrad在梯度稀疏时表现出色。原始论文的动机来源于随机梯度下降，它在每次迭代中随机选择一批训练数据来计算梯度。许多现实世界问题的深度学习数据集会产生稀疏梯度，其中某些特征出现的频率远低于其他特征。

Adagrad 更新步骤为：

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \sqrt{s_i^{(k)}}} g_i^{(k)} \tag{5.23}$$

其中 s(*k*) 是一个向量，其第 *i* 个元素是对 x_i 的偏导数的平方和，直到时间步 *k*，

$$s_i^{(k)} = \sum_{j=1}^{k} \left(g_i^{(j)}\right)^2 \tag{5.24}$$

而 ϵ 是一个小值，约为1×10^{-8}，以防止除法 n 乘零。

Adagrad 对 learning rate 参数 α 远不敏感。learning rate 参数通常默认设为 0.01。Adagrad 的主要弱点是 s 的各个分量都是严格非递减的。累积和导致有效 learning rate 在训练过程中减小，往往在收敛之前就变得极小。算法 5.5 提供了一个实现。

```
mutable struct Adagrad <: DescentMethod
    α # learning rate
    ϵ # small value
    s # sum of squared gradient
end
function init!(M::Adagrad, f, ∇f, x)
    M.s = zeros(length(x))
    return M
end
function step!(M::Adagrad, f, ∇f, x)
    α, ϵ, s, g = M.α, M.ϵ, M.s, ∇f(x)
    s[:] += g.*g
    return x - α*g ./ (sqrt.(s) .+ ϵ)
end
```

算法5.5. Adagrad加速下降法。

5.6 RMSProp

RMSProp[7] 扩展了 Adagrad 以避免单调递减 {v*} 的影响
学习率。RMSProp维护一个平方梯度的衰减平均值。这个平均值根据以下公式更新:[8]

$$\hat{\mathbf{s}}^{(k+1)} = \gamma\hat{\mathbf{s}}^{(k)} + (1-\gamma)\left(\mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}\right) \tag{5.25}$$

其中衰减因子 $\gamma \in [0, 1]$ 通常接近0.9。

过去平方梯度的衰减平均值可以代入RMSProp的更新方程中：[9]

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \sqrt{\hat{s}_i^{(k)}}} g_i^{(k)} \tag{5.26}$$

$$= x_i^{(k)} - \frac{\alpha}{\epsilon + \text{RMS}(g_i)} g_i^{(k)} \tag{5.27}$$

算法5.6提供了一种实现。

5.7 Adadelta

Adadelta[10] 是另一种克服 Adagrad 单调递减问题的方法。
学习率。在独立推导出RMSProp更新之后，作者们注意到在梯度下降、动量、新方程中的单位,

7 RMSProp 未发表，源自 Geoff Hinton 的 Coursera 课程第 6e 讲。

8 运算 a ⊙ b 是向量 a 和 b 之间的逐元素乘积。

9 分母类似于梯度分量的均方根（RMS）。在本章中，我们用RMS(x)来表示时间序列x的衰减均方根。

10 M. D. Zeiler, “ADADELTA：更种自适应学习率方法”，*ArXiv*，第 1212.5701 号，2012 年。

```
mutable struct RMSProp <: DescentMethod
    α # learning rate
    γ # decay
    ϵ # small value
    s # sum of squared gradient
end
function init!(M::RMSProp, f, ∇f, x)
    M.s = zeros(length(x))
    return M
end
function step!(M::RMSProp, f, ∇f, x)
    α, γ, ϵ, s, g = M.α, M.γ, M.ϵ, M.s, ∇f(x)
    s[:] = γ*s + (1-γ)*(g.*g)
    return x - α*g ./ (sqrt.(s) .+ ϵ)
end
```

算法5.6. RMSProp加速下降法。

而 Adagrad 并不匹配。为了解决这个问题，他们使用了平方更新的指数衰减平均值：

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\text{RMS}(\Delta x_i)}{\epsilon + \text{RMS}(g_i)} g_i^{(k)} \tag{5.28}$$

这完全消除了学习率参数。算法5.7提供了实现。

5.8 Adam

adaptive moment estimation 方法或 *Adam*、[11] 也适应学习率到每个参数（算法5.8）。它既像RMSProp和Adadelta那样存储指数衰减的平方梯度，也像动量那样存储指数衰减的梯度。

11 D. Kingma 和 J. Ba，“Adam：一种随机优化方法”，载于 *International Conference on Learning Representations (ICLR)*，2015年。

将梯度和平方梯度初始化为零会引入偏差。偏差校正步骤有助于缓解该问题。[12] 在每个

根据原始论文，良好的默认设置为 $\alpha = 0.001$、$\gamma_v = 0.9$、$\gamma_s = 0.999$，以及 $\epsilon = 1 \times 10^{-8}$。

```
mutable struct Adadelta <: DescentMethod
    γs # gradient decay
    γx # update decay
    ϵ # small value
    s # sum of squared gradients
    u # sum of squared updates
end
function init!(M::Adadelta, f, ∇f, x)
    M.s = zeros(length(x))
    M.u = zeros(length(x))
    return M
end
function step!(M::Adadelta, f, ∇f, x)
    γs, γx, ϵ, s, u, g = M.γs, M.γx, M.ϵ, M.s, M.u, ∇f(x)
    s[:] = γs*s + (1-γs)*g.*g
    Δx = - (sqrt.(u) .+ ϵ) ./ (sqrt.(s) .+ ϵ) .* g
    u[:] = γx*u + (1-γx)*Δx.*Δx
    return x + Δx
end
```

算法5.7. Adadelta加速下降法。小常数ϵ也被加到分子中，以防止进展完全衰减至零，并且开始于第一次迭代时Δx ==0的情况。

Adam的迭代如下：

$$\text{biased decaying momentum: } \mathbf{v}^{(k+1)} = \gamma_v \mathbf{v}^{(k)} + (1-\gamma_v)\mathbf{g}^{(k)} \tag{5.29}$$

$$\text{biased decaying sq. gradient: } \mathbf{s}^{(k+1)} = \gamma_s \mathbf{s}^{(k)} + (1-\gamma_s)\left(\mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}\right) \tag{5.30}$$

$$\text{corrected decaying momentum: } \hat{\mathbf{v}}^{(k+1)} = \mathbf{v}^{(k+1)} / (1-\gamma_v^k) \tag{5.31}$$

$$\text{corrected decaying sq. gradient: } \hat{\mathbf{s}}^{(k+1)} = \mathbf{s}^{(k+1)} / (1-\gamma_s^k) \tag{5.32}$$

$$\text{下次迭代：} \ \mathrm{x}^{(k+1)} = \mathrm{x}^{(k)} - \alpha \mathbf{v}^{(k+1)} / \left(\epsilon + \sqrt{\hat{\mathrm{s}}^{(k+1)}}\right) \tag{5.33}$$

5.9 *Hypergradient Descent*

加速下降方法要么对学习率极其敏感，要么在执行过程中竭力调整学习率。学习率决定了方法对梯度信号的敏感程度。学习率过高或过低往往都会极大地影响性能。

```
mutable struct Adam <: DescentMethod
    α # learning rate
    γv # decay
    γs # decay
    ϵ # small value
    k # step counter
    v # 1st moment estimate
    s # 2nd moment estimate
end
function init!(M::Adam, f, ∇f, x)
    M.k = 0
    M.v = zeros(length(x))
    M.s = zeros(length(x))
    return M
end
function step!(M::Adam, f, ∇f, x)
    α, γv, γs, ϵ, k = M.α, M.γv, M.γs, M.ϵ, M.k
    s, v, g = M.s, M.v, ∇f(x)
    v[:] = γv*v + (1-γv)*g
    s[:] = γs*s + (1-γs)*g.*g
    M.k = k += 1
    v_hat = v ./ (1 - γv^k)
    s_hat = s ./ (1 - γs^k)
    return x - α*v_hat ./ (sqrt.(s_hat) .+ ϵ)
end
```

算法5.8. Adam加速下降法。

Hypergradient descent[13] 的开发基于这样的理解：衍生学习率导数对于提升优化器性能应该是有用的。*hypergradient*是关于超参数的导数。超梯度算法降低了对于超参数的敏感性，使其能够更快地适应。

[13] A. G. Baydin, R. Cornish, D. M. Rubio, M. Schmidt, and F. Wood, "使用超梯度下降的在线学习率自适应"，载于 *International Conference on Learning Representations (ICLR)*, 2018年.

超梯度下降将梯度下降应用于基础下降方法的学习率。该方法需要目标函数关于学习率的偏导数。对于梯度下降，该偏导数为：

$$\frac{\partial f(\mathbf{x}^{(k)})}{\partial \alpha} = (\mathbf{g}^{(k)})^\top \frac{\partial}{\partial \alpha}\left(\mathbf{x}^{(k-1)} - \alpha \mathbf{g}^{(k-1)}\right) \tag{5.34}$$

$$= (\mathbf{g}^{(k)})^\top \left(-\mathbf{g}^{(k-1)}\right) \tag{5.35}$$

因此，计算超梯度需要跟踪最近的梯度。得到的更新规则是：

$$\alpha^{(k+1)} = \alpha^{(k)} - \mu \frac{\partial f(\mathbf{x}^{(k)})}{\partial \alpha} \tag{5.36}$$

$$= \alpha^{(k)} + \mu (\mathbf{g}^{(k)})^\top \mathbf{g}^{(k-1)} \tag{5.37}$$

其中 μ 是超梯度学习率。

此推导可应用于任何遵循方程(4.1)的基于梯度的下降方法。我们提供了梯度下降（算法5.9）和Nesterov动量（算法5.10）的超梯度版本的实现。这些方法在图5.7中可视化。

```
mutable struct HyperGradientDescent <: DescentMethod
    α0 # initial learning rate
    μ # learning rate of the learning rate
    α # current learning rate
    g_prev # previous gradient
end
function init!(M::HyperGradientDescent, f, ∇f, x)
    M.α = M.α0
    M.g_prev = zeros(length(x))
    return M
end
function step!(M::HyperGradientDescent, f, ∇f, x)
    α, μ, g, g_prev = M.α, M.μ, ∇f(x), M.g_prev
    α = α + μ*(g⋅g_prev)
    M.g_prev, M.α = g, α
    return x - α*g
end
```

算法5.9。梯度下降的超梯度形式。

```
mutable struct HyperNesterovMomentum <: DescentMethod
    α0 # initial learning rate
    μ # learning rate of the learning rate
    β # momentum decay
    v # momentum
    α # current learning rate
    g_prev # previous gradient
end
function init!(M::HyperNesterovMomentum, f, ∇f, x)
    M.α = M.α0
    M.v = zeros(length(x))
    M.g_prev = zeros(length(x))
    return M
end
function step!(M::HyperNesterovMomentum, f, ∇f, x)
    α, β, μ = M.α, M.β, M.μ
    v, g, g_prev = M.v, ∇f(x), M.g_prev
    α = α - μ*(g⋅(-g_prev - β*v))
    v[:] = β*v + g
    M.g_prev, M.α = g, α
    return x - α*(g + β*v)
end
```

算法5.10. Nesterov动量下降方法的超梯度形式。

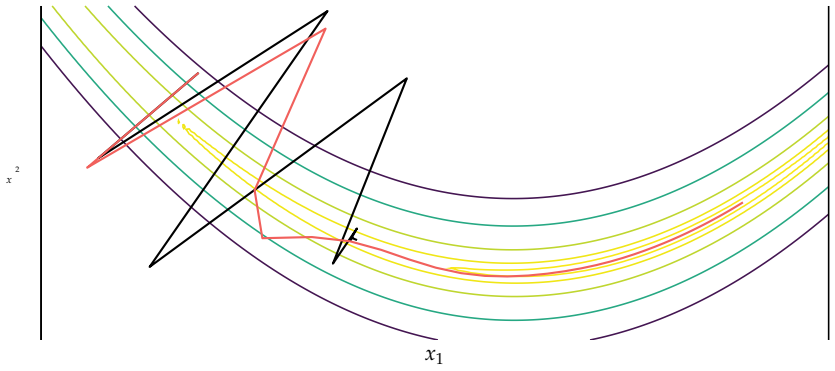


图 5.7。在 b = 100 的 Rosenbroc k 函数上，动量法和 Nesterov 动量法的对比；见附录 B.6。

5.10 Summary

- 梯度下降沿着最陡下降的方向进行。
- 共轭梯度法能够自动适应局部谷地。
- 具有动量的下降方法在有利的方向上积累进展。
- 多种加速下降方法使用特殊技术来加速下降。

- 超梯度下降将梯度下降应用于底层下降方法的学习率。

5.11 Exercises

练习 5.1. 当 A 对称时，计算 $x^\top Ax + b^\top x$ 的梯度。

练习5.2。从你选择的起始点出发，对$f(x) = x^4$应用单位步长的梯度下降法。计算两次迭代。

练习5.3。从$x^{(1)} = 10$出发，对$f(x) = e^x + e^{-x}$应用一步梯度下降，分别采用单位步长和精确线搜索。

练习 5.4。当函数在当前点有局部二次模型时，共轭梯度法也可用于寻找搜索方向d。以d作为搜索方向，设模型为

$$q(\mathbf{d}) = \mathbf{d}^\top \mathbf{H}\mathbf{d} + \mathbf{b}^\top \mathbf{d} + \mathbf{c}$$

对于对称矩阵 H。在这种情况下，海森矩阵是什么？当 d = 0 时，q 的梯度是什么？如果将共轭梯度法应用于二次模型以获取搜索方向 d，可能会出现什么问题？

练习5.5。Nesterov动量相较于动量有何改进?

练习 5.6. 共轭梯度法在哪方面比最速下降法有所改进?

练习 5.7. 在共轭梯度下降中，对于函数 $f(x, \ y) = x^2 + xy + y^2 + 5$ 在初始化为 $(x, y) = (1, 1)$ 时，第一次迭代的归一化下降方向是什么？共轭梯度法经过两步后得到的点是什么?

练习 5.8。我们有一个多项式函数 f，使得对于三维欧几里得空间中的所有 x，都有 $f(\mathrm{x}) > 2$。假设我们使用最速下降法，并且在每一步都优化步长，我们想要找到 f 的一个局部最小值。如果在第 k 步时，我们的未归一化下降方向是 [1, 2, 3]，那么在第 $k + 1$ 步时，我们的未归一化下降方向是否可能是 $[0, 0, -3]$? 为什么可能或为什么不可能?

6 Second-Order Methods

上一章重点讨论了利用目标函数梯度进行一阶近似的优化方法。本章则侧重于利用*second-order*近似来指导搜索，这些近似在一元优化中使用二阶导数，在多元优化中使用海森矩阵。这些额外的信息有助于改进用于指导下降算法中方向和步长选择的局部模型。

6.1 Newton's Method

知道设计点的函数值和梯度有助于确定行进方向，但这种一阶信息并不能直接帮助确定达到局部最小值所需的步长。另一方面，二阶信息使我们能够对目标函数进行二次近似，并估算出达到局部最小值的正确步长，如图6.1所示。正如我们在第3章中通过二次拟合搜索所看到的，我们可以解析地获得二次近似梯度为零的位置。然后，我们可以将该位置用作下一次迭代，以逼近局部最小值。

在单变量优化中，关于点 $x^{(k)}$ 的二次近似来源于二阶泰勒展开：

$$q(x) = f(x^{(k)}) + (x - x^{(k)})f'(x^{(k)}) + \frac{(x - x^{(k)})^2}{2} f''(x^{(k)}) \tag{6.1}$$

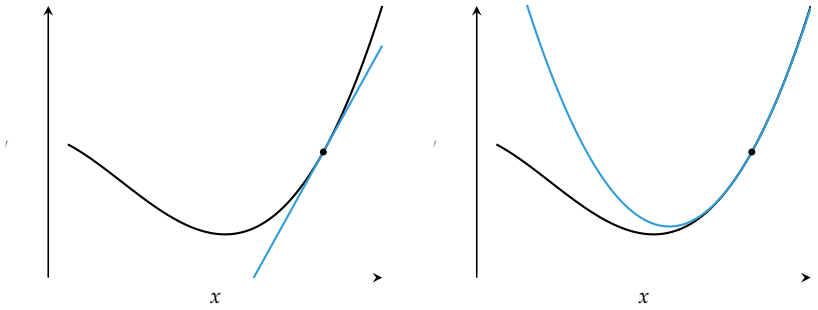


图6.1. 一阶与二阶近似的比较。碗状二次近似具有导数等于零的唯一位置。

将导数设为零并求解根，得到 *Newton's method* 的更新方程：

$$\frac{\partial}{\partial x} q(x) = f'(x^{(k)}) + (x - x^{(k)}) f''(x^{(k)}) = 0 \tag{6.2}$$

$$x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})} \tag{6.3}$$

此更新如图 6.2 所示。

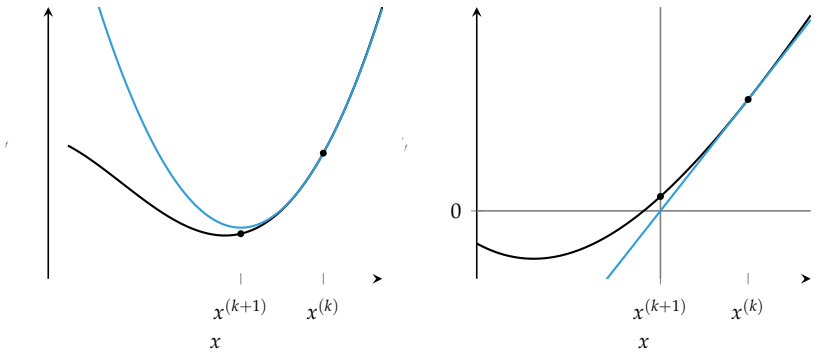


图6.2。牛顿法可以解释为应用于f'的求根方法，它通过在$(x, f'(x))$处取切线，找到与x轴的交点，并使用该x值作为下一个设计点，迭代地改进单变量设计点。

牛顿法的更新规则涉及除以二阶导数。当二阶导数为零时，更新未定义，这种情况发生在二次近似为水平线时。当二阶导数非常接近于零时，也会出现不稳定性，此时下一次迭代将远离当前设计点，远离局部二次近似有效的区域。局部近似不佳可能导致牛顿法性能低下。图6.3展示了三种失败情形。

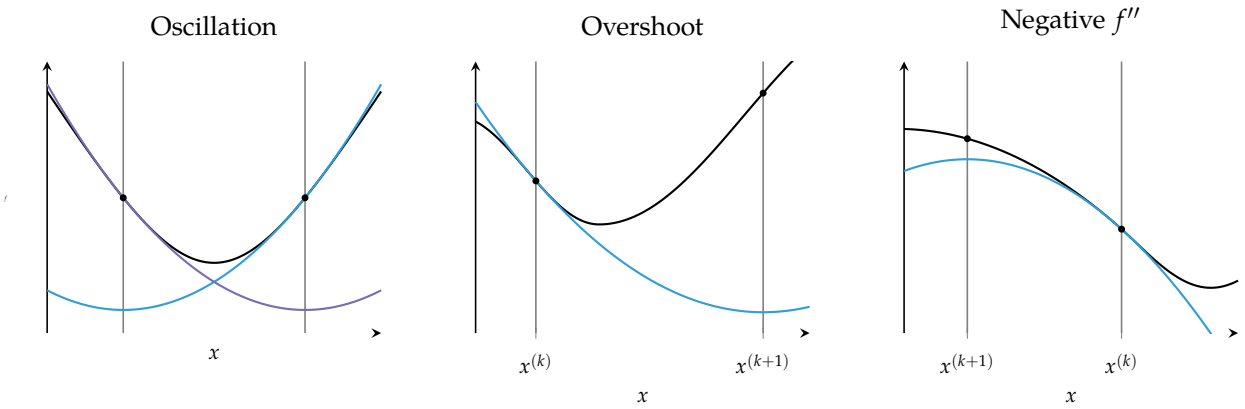


图6.3. 牛顿法失效案例示例。

牛顿法在碗状区域且充分接近局部最小值时，确实倾向于快速收敛。它具有*quadratic convergence*，意味着每次迭代时，最小值与迭代点之间的差值大约呈平方关系。如果[1]，则牛顿法从距离根x^*为δ的$x^{(1)}$开始，这种收敛速率成立。

1 最终条件强制 *suf-*，确保函数由*ficient closeness*泰勒展开充分近似。J. Stoer 和 R. Bulirsch，*Introduc-*，第3版。Springer，2002年。*tion to Numerical Analysis*

- $f''(x) \neq I$ 中所有点的值为 0，
- $f'''(x)$ 在I上连续，并且
- $12\,|\frac{f'''(x^{(1)})}{f''(x^{(1)})}| < c|\frac{f'''(x^*)}{f''(x^*)}|$ 对于某些 $c < \infty$

对于区间 $I = [x^* - \delta, x^* + \delta]$。最终条件用于防止超调。

牛顿法可以扩展到多变量优化（算法6.1）。

在x(k)处的多元二阶泰勒展开　　是：

$$f(\mathbf{x}) \approx q(\mathbf{x}) = f(\mathbf{x}^{(k)}) + (\mathbf{g}^{(k)})^\top(\mathbf{x} - \mathbf{x}^{(k)}) + \frac{1}{2}(\mathbf{x} - \mathbf{x}^{(k)})^\top\mathbf{H}^{(k)}(\mathbf{x} - \mathbf{x}^{(k)}) \tag{6.4}$$

其中　g(k) 和 H(k) 分别是 x(k) 处的梯度和海森矩阵，　　积极地。

我们计算梯度并将其设为零：

$$\nabla q(\mathbf{x}^{(k)}) = \mathbf{g}^{(k)} + \mathbf{H}^{(k)}(\mathbf{x} - \mathbf{x}^{(k)}) = \mathbf{0} \tag{6.5}$$

然后求解下一次迭代，从而得到多元形式的牛顿法：

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - (\mathbf{H}^{(k)})^{-1}\mathbf{g}^{(k)} \tag{6.6}$$

如果 f 是二次函数且其 Hessian 矩阵正定，则更新在一步内收敛到全局最小值。对于一般函数，牛顿法通常在 x 的变化不超过给定容差时终止。[2]

第5章给出了下降法的终止条件。

例6.1展示了如何使用牛顿法来最小化一个函数。

当 $\mathrm{x}^{(1)} = [9, 8]$ 时，我们将使用牛顿法来最小化 Booth 函数：

$$f(\mathbf{x}) = (x_1 + 2x_2 - 7)^2 + (2x_1 + x_2 - 5)^2$$

Booth 函数的梯度是：

$$\nabla f(\mathbf{x}) = [10x_1 + 8x_2 - 34, 8x_1 + 10x_2 - 38]$$

Booth函数的Hessian矩阵是：

$$\mathbf{H}(\mathbf{x}) = \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}$$

牛顿法的第一次迭代得到：

$$\begin{aligned} \mathbf{x}^{(2)} &= \mathbf{x}^{(1)} - \left(\mathbf{H}^{(1)}\right)^{-1} \mathbf{g}^{(1)} = \begin{bmatrix} 9 \\ 8 \end{bmatrix} - \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}^{-1} \begin{bmatrix} 10 \cdot 9 + 8 \cdot 8 - 34 \\ 8 \cdot 9 + 10 \cdot 8 - 38 \end{bmatrix} \\ &= \begin{bmatrix} 9 \\ 8 \end{bmatrix} - \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}^{-1} \begin{bmatrix} 120 \\ 114 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \end{aligned}$$

在$\mathrm{x}^{(2)}$处的梯度为零，因此经过一次迭代后我们就收敛了。海森矩阵处处正定，所以$\mathrm{x}^{(2)}$是全局最小值。

例6.1. 使用牛顿法最小化Booth函数；参见附录B.2。

牛顿法也可以用于为线搜索提供下降方向，或者可以修改为使用步长因子。[3] 向最小值迈出更小的步伐，或者
沿下降方向进行线搜索可以提高该方法的鲁棒性。下降方向为：[4]

$$\mathbf{d}^{(k)} = -(\mathbf{H}^{(k)})^{-1} \mathbf{g}^{(k)} \tag{6.7}$$

3 见第5章。

4 牛顿法给出的下降方向类似于*natural gradient*或*covariant gradient*。S. Amari，“自然梯度在学习中高效工作”，*Neural Computation*，第10卷，第2期，第251–276页，1998年。

```
function newtons_method(∇f, H, x, ϵ, k_max)
    k, Δ = 1, fill(Inf, length(x))
    while norm(Δ) > ϵ && k ≤ k_max
        Δ = H(x) \ ∇f(x)
        x -= Δ
        k += 1
    end
    return x
end
```

算法6.1. 牛顿法，该算法需要函数`∇f`的梯度，目标函数`H`的海森矩阵，初始点`x`，步长容差`ϵ`，以及最大迭代次数`k_max`。

6.2 Secant Method

牛顿法用于单变量函数最小化需要一阶和二阶导数 f' 和 f''。在许多情况下，已知 f'，但不知道二阶导数。*secant method* (算法 6.2) 使用二阶导数的估计值应用牛顿法，因此只需要 f'。这一特性使得割线法在实际使用中更为方便。

割线法使用最后两次迭代值来近似二阶导数：

$$f''(x^{(k)}) \approx \frac{f'(x^{(k)}) - f'(x^{(k-1)})}{x^{(k)} - x^{(k-1)}} \tag{6.8}$$

将此估计值代入牛顿法：

$$x^{(k+1)} \leftarrow x^{(k)} - \frac{x^{(k)} - x^{(k-1)}}{f'(x^{(k)}) - f'(x^{(k-1)})} f'(x^{(k)}) \tag{6.9}$$

割线法需要一个额外的初始设计点。它与牛顿法存在相同的问题，并且由于近似二阶导数，可能需要更多次迭代才能收敛。

6.3 Quasi-Newton Methods

正如割线法在单变量情况下近似 f''，*quasi-Newton* 方法近似逆 Hessian 矩阵。拟牛顿法更新具有以下形式：

$$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \alpha^{(k)} \mathbf{Q}^{(k)} \mathbf{g}^{(k)} \tag{6.10}$$

其中 $\alpha^{(k)}$ 是标量步长因子，而 Q(k) 近似表示 x(k) 处的 Hessian 矩阵的逆。

```
function secant_method(f′, x0, x1, ϵ)
    g0 = f′(x0)
    Δ = Inf
    while abs(Δ) > ϵ
        g1 = f′(x1)
        Δ = (x1 - x0)/(g1 - g0)*g1
        x0, x1, g0 = x1, x1 - Δ, g1
    end
    return x1
end
```

算法 6.2. 单变量函数最小化的割线法。输入为目标函数的一阶导数 `f′`，两个初始点 `x0` 和 `x1`，以及期望的容差 `ϵ`。返回最终的 x 坐标。

这些方法通常将 Q(1) 设为单位矩阵，然后应用更新来反映每次迭代所学习到的信息。为了简化各种拟牛顿方法的方程，我们定义如下：

$$\gamma^{(k+1)} \equiv \mathbf{g}^{(k+1)} - \mathbf{g}^{(k)} \tag{6.11}$$

$$\delta^{(k+1)} \equiv \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)} \tag{6.12}$$

The *Davidon-Fletcher-Powell* (DFP) 方法（算法 6.3）　　使用：[5]

$$\mathbf{Q} \leftarrow \mathbf{Q} - \frac{\mathbf{Q}\gamma\gamma^\top\mathbf{Q}}{\gamma^\top\mathbf{Q}\gamma} + \frac{\delta\delta^\top}{\delta^\top\gamma} \tag{6.13}$$

其中右侧的所有项均在迭代 k 处计算。

DFP方法中Q的更新具有三个性质：

1. Q 保持对称且正定。

2. 如果$f(\mathrm{x}) = \frac{1}{2}\mathrm{x}^\top\mathrm{A}\mathrm{x} + \mathrm{b}^\top\mathrm{x} + c$，那么$\mathrm{Q} = \mathrm{A}^{-1}$。因此，DFP算法具有与共轭梯度法相同的收敛性质。

3. 对于高维问题，与其他方法（如共轭梯度法）相比，存储和更新Q可能非常显著。

DFP的替代方法是*Broyden-Fletcher-Goldfarb-Shanno* (BFGS)方法（算法6.4），它使用：[6]

$$\mathbf{Q} \leftarrow \mathbf{Q} - \left(\frac{\delta\gamma^\top\mathbf{Q} + \mathbf{Q}\gamma\delta^\top}{\delta^\top\gamma}\right) + \left(1 + \frac{\gamma^\top\mathbf{Q}\gamma}{\delta^\top\gamma}\right)\frac{\delta\delta^\top}{\delta^\top\gamma} \tag{6.14}$$

5 最初的概念在技术报告中提出，W. C. Davidon，“用于最小化的变尺度法”，阿贡国家实验室，技术报告 ANL-5990，1959年。后来发表为：W. C. Davidon，“用于最小化的变尺度法”，*SIAM Journal on Optimization*，第1卷，第1期，第1-17页，1991年。该方法由 R. Fletcher 和 M. J. D. Powell 修改，“一种快速收敛的最小化下降法”，*The Computer Journal*，第6卷，第2期，第163-168页，1963年。

6 R. Fletcher，*Practical Methods of Optimization*，第2版，Wiley

```
mutable struct DFP <: DescentMethod
    Q
end
function init!(M::DFP, f, ∇f, x)
    m = length(x)
    M.Q = Matrix(1.0I, m, m)
    return M
end
function step!(M::DFP, f, ∇f, x)
    Q, g = M.Q, ∇f(x)
    x′ = line_search(f, x, -Q*g)
    g′ = ∇f(x′)
    δ = x′ - x
    γ = g′ - g
    Q[:] = Q - Q*γ*γ'*Q/(γ'*Q*γ) + δ*δ'/(δ'*γ)
    return x′
end
```

算法6.3. Davidon-Fletcher-Powell下降法。

```
mutable struct BFGS <: DescentMethod
    Q
end
function init!(M::BFGS, f, ∇f, x)
    m = length(x)
    M.Q = Matrix(1.0I, m, m)
    return M
end
function step!(M::BFGS, f, ∇f, x)
    Q, g = M.Q, ∇f(x)
    x′ = line_search(f, x, -Q*g)
    g′ = ∇f(x′)
    δ = x′ - x
    γ = g′ - g
    Q[:] = Q - (δ*γ'*Q + Q*γ*δ')/(δ'*γ) +
               (1 + (γ'*Q*γ)/(δ'*γ))[1]*(δ*δ')/(δ'*γ)
    return x′
end
```

算法6.4. Broyden-Fletcher-Goldfarb-Shanno下降法。

BFGS 在近似线搜索方面优于 DFP，但仍使用 $n \times n$ 稠密矩阵。对于空间成为问题的超大规模问题，可使用 *Limited- memory BFGS* 方法（算法 6.5）或 *L-BFGS* 来近似 BFGS。[7] L-BFGS 存储 $\boldsymbol{\delta}$ 和 $\boldsymbol{\gamma}$ 的最后 m 个值，而不是完整的逆矩阵。

7 J. Nocedal，“有限存储的拟牛顿矩阵更新”，*Mathematics of Computa-*，第35卷，第151期，第773–782页，1980年。

海森矩阵，其中 $i = 1$ 索引最旧的值，$i = m$ 索引最新的值。

在x处计算下降方向d的过程始于计算$q(m) = \nabla f(x)$。剩余的向量$q(i)$，对于从 $m - 1$递减到1的i，是通过以下方式计算的：

$$\mathbf{q}^{(i)} = \mathbf{q}^{(i+1)} - \frac{\left(\boldsymbol{\delta}^{(i+1)}\right)^\top \mathbf{q}^{(i+1)}}{\left(\boldsymbol{\gamma}^{(i+1)}\right)^\top \boldsymbol{\delta}^{(i+1)}} \boldsymbol{\gamma}^{(i+1)} \tag{6.15}$$

这些向量用于计算另一个 $m + 1$ 向量，从

$$\mathbf{z}^{(0)} = \frac{\boldsymbol{\gamma}^{(m)} \odot \boldsymbol{\delta}^{(m)} \odot \mathbf{q}^{(m)}}{\left(\boldsymbol{\gamma}^{(m)}\right)^\top \boldsymbol{\gamma}^{(m)}} \tag{6.16}$$

并对 i 从 1 到 m 按如下方式继续处理 $z(i)$：

$$\mathbf{z}^{(i)} = \mathbf{z}^{(i-1)} + \boldsymbol{\delta}^{(i-1)} \left(\frac{\left(\boldsymbol{\delta}^{(i-1)}\right)^\top \mathbf{q}^{(i-1)}}{\left(\boldsymbol{\gamma}^{(i-1)}\right)^\top \boldsymbol{\delta}^{(i-1)}} - \frac{\left(\gamma^{(i-1)}\right)^\top \mathbf{z}^{(i-1)}}{\left(\boldsymbol{\gamma}^{(i-1)}\right)^\top \boldsymbol{\delta}^{(i-1)}} \right) \tag{6.17}$$

下降方向是 $d = -z(m)$。

对于最小化问题，逆Hessian矩阵Q必须保持正定。初始Hessian矩阵通常设置为

$$\mathbf{Q}^{(1)} = \frac{\boldsymbol{\gamma}^{(1)} \left(\boldsymbol{\delta}^{(1)}\right)^\top}{\left(\boldsymbol{\gamma}^{(1)}\right)^\top \boldsymbol{\gamma}^{(1)}} \tag{6.18}$$

计算上述表达式的对角线并将结果代入$z(1) = Q(1)q(1)$，得到关于$z(1)$的方程。

本节讨论的拟牛顿法在图6.4中进行了比较。它们的表现往往相当相似。

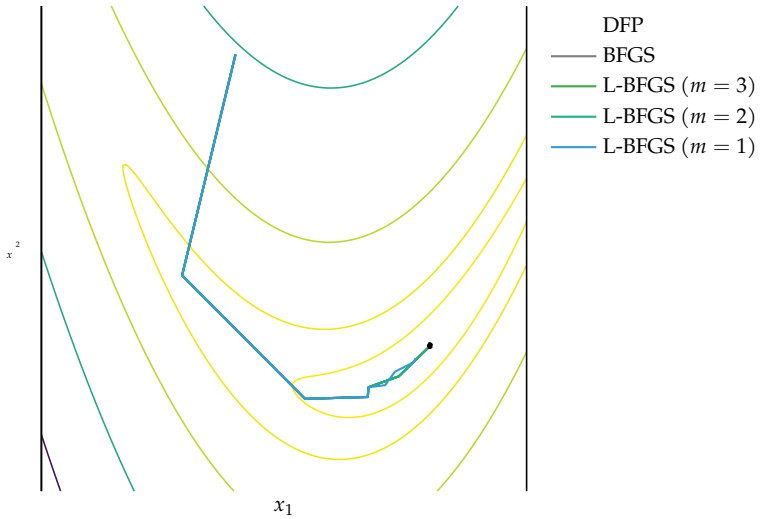


图6.4. 在Rosenbrock函数上比较了几种拟牛顿法；参见附录B.6。所有方法的更新几乎相同，仅当L-BFGS的历史记录m为1时，其表现明显偏离。

6.4 Summary

- 在下将方法中引入二阶信息通常会加速收敛。
- 牛顿法是一种利用二阶信息快速下降到局部最小值的求根方法。
- 割线法和拟牛顿法在二阶信息不可直接获得时近似牛顿法。

6.5 Exercises

练习 6.1。二阶信息在收敛性方面提供了哪些一阶信息所缺乏的优势?

练习 6.2. 在一维求根时，我们何时会选择使用牛顿法而非二分法?

练习6.3。对$f(x) = x^2$应用牛顿法，从你选择的起始点开始。我们需要多少步才能收敛?

```
mutable struct LimitedMemoryBFGS <: DescentMethod
    m
    δs
    γs
    qs
end
function init!(M::LimitedMemoryBFGS, f, ∇f, x)
    M.δs = []
    M.γs = []
    M.qs = []
    return M
end
function step!(M::LimitedMemoryBFGS, f, ∇f, x)
    δs, γs, qs, g = M.δs, M.γs, M.qs, ∇f(x)
    m = length(δs)
    if m > 0
        q = g
        for i in m : -1 : 1
            qs[i] = copy(q)
            q -= (δs[i]⋅q)/(γs[i]⋅δs[i])*γs[i]
        end
        z = (γs[m] .* δs[m] .* q) / (γs[m]⋅γs[m])
        for i in 1 : m
            z += δs[i]*(δs[i]⋅qs[i] - γs[i]⋅z)/(γs[i]⋅δs[i])
        end
        x′ = line_search(f, x, -z)
    else
        x′ = line_search(f, x, -g)
    end
    g′ = ∇f(x′)
    push!(δs, x′ - x); push!(γs, g′ - g)
    push!(qs, zeros(length(x)))
    while length(δs) > M.m
        popfirst!(δs); popfirst!(γs); popfirst!(qs)
    end
    return x′
end
```

算法6.5. 有限内存BFGS下降法，该方法避免了存储近似逆黑塞矩阵。参数`m`决定了历史大小。`LimitedMemoryBFGS`类型还存储步长差`δs`、梯度变化`γs`以及存储向量`qs`。

练习 6.4. 将牛顿法应用于从 $\mathrm{x}^{(1)} = [1, 1]$ 开始的 $f(x) = \frac{1}{2}\mathrm{x}^\top \mathrm{H}\mathrm{x}$。你观察到了什么？使用如下的 H：

$$\mathbf{H} = \begin{bmatrix} 1 & 0 \\ 0 & 1000 \end{bmatrix} \tag{6.19}$$

接下来，对相同的优化问题应用梯度下降法，采用未归一化的梯度进行步进。执行算法两步。你观察到了什么？最后，应用共轭梯度法。需要多少步才能收敛？

练习 6.5. 比较牛顿法与割线法在 $f(x) = x^2 + x^4$ 上的应用，其中 $x^{(1)} = -3$ 和 $x^{(0)} = -4$。对每种方法执行 10 次迭代。绘制两个图表：

1. 对每种方法，绘制 f 随迭代次数的变化图。

2. 绘制f'与x的关系图。对每种方法的进程进行叠加，为每次转变画出从$(x^{(i)}, f'(x^{(i)}))$到$(x^{(i+1)}, 0)$再到$(x^{(i+1)}, f'(x^{(i+1)}))$的连线。

关于这个比较，我们可以得出什么结论？

练习 6.6. 给出一个点序列 $x^{(1)}, x^{(2)}, \ldots$ 和一个函数 f 的例子，使得 $f(x^{(1)}) > f(x^{(2)}) > \cdots$ 成立，但该序列并不收敛到一个局部最小值。假设 f 是从下方有界的。

练习 6.7. 拟牛顿法相较于牛顿法有何优势？

练习 6.8. 举例说明BFGS更新不存在的情形。在这种情况下，你会怎么处理？

练习 6.9。假设我们有一个函数 $f(\mathrm{x}) = (x_1 + 1)^2 + (x_2 + 3)^2 + 4$。如果从原点出发，经过一步牛顿法后得到的点是什么？

练习 6.10. 在这个问题中，我们将推导得出 Davidon-Fletcher-Powell 更新所依据的优化问题。从 $\mathrm{x}^{(k)}$ 处的二次近似开始：

$$f^{(k)}(\mathbf{x}) = y^{(k)} + \left(\mathbf{g}^{(k)}\right)^\top \left(\mathbf{x} - \mathbf{x}^{(k)}\right) + \frac{1}{2}\left(\mathbf{x} - \mathbf{x}^{(k)}\right)^\top \mathbf{H}^{(k)} \left(\mathbf{x} - \mathbf{x}^{(k)}\right)$$

其中$y^{(k)}$、g(k)和H(k)分别是在x(k)处的目标函数值、真实梯度和正定Hessian近似。

下一次迭代通过线搜索选定以获得：

$$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \alpha^{(k)} \left(\mathbf{H}^{(k)}\right)^{-1} \mathbf{g}^{(k)}$$

我们可以在x(k+1)处构造一个新的二次近似$f^{(k+1)}$。该近似应确保局部函数评估是正确的：

$$f^{(k+1)}(\mathbf{x}^{(k+1)}) = y^{(k+1)}$$

并且局部梯度是正确的：

$$\nabla f^{(k+1)}(\mathbf{x}^{(k+1)}) = \mathbf{g}^{(k+1)}$$

并且先前的梯度是正确的：

$$\nabla f^{(k+1)}(\mathbf{x}^{(k)}) = \mathbf{g}^{(k)}$$

证明更新 Hessian 近似以获得 H(k+1) 需要：[8]

$$\mathbf{H}^{(k+1)} \boldsymbol{\delta}^{(k+1)} = \boldsymbol{\gamma}^{(k+1)}$$

然后，证明为了使H(k+1)为正定，我们需要：[9]

$$\left(\boldsymbol{\delta}^{(k+1)}\right)^{\top} \boldsymbol{\gamma}^{(k+1)} > 0$$

最后，假设曲率条件被强制执行，解释为什么随后要求解以下优化问题以获得 H(k+1)：[10]

$$\begin{aligned} \underset{\mathbf{H}}{\text{minimize}} \quad & \left\| \mathbf{H} - \mathbf{H}^{(k)} \right\| \\ \text{subject to} \quad & \mathbf{H} = \mathbf{H}^{\top} \\ & \mathbf{H}\delta^{(k+1)} = \gamma^{(k+1)} \end{aligned}$$

其中 $\left\| \mathrm{H} - \mathrm{H}(k) \right\|$ 是一个 *matrix norm*，它定义了 H 与 H(k) 之间的距离。

8 这个条件被称为*secant*。向量$\boldsymbol{\delta}$和$\boldsymbol{\gamma}$在等式(6.11)中定义。

equation

9 这个条件被称为*curvature condition*。它可以在线搜索过程中通过Wolfe条件来强制满足。

10 通过求解此类优化问题以获得解析解，进而找到逆Hessian近似对应的更新方程，即得到Davidon-Fletcher-Powell更新。

7 Direct Methods

Direct methods 仅依赖目标函数 f。这些方法也称为 *zero-order*、*black box*、*pattern search* 或 *derivative-free* 方法。直接方法不依赖导数信息来引导其趋向局部极小值或判断何时达到局部极小值。它们使用其他准则来选择下一个搜索方向并判断何时收敛。

7.1 Cyclic Coordinate Search

Cyclic coordinate search，也称为*coordinate descent*或*taxicab search*，其线搜索简单地在坐标方向之间交替。搜索从初始x(1)开始，并优化第一个输入：

$$\mathbf{x}^{(2)} = \arg\min_{x_1} f(x_1, x_2^{(1)}, x_3^{(1)}, \ldots, x_n^{(1)}) \tag{7.1}$$

解决这个问题后，它优化下一个坐标：

$$\mathbf{x}^{(3)} = \arg\min_{x_2} f(x_1^{(2)}, x_2, x_3^{(2)}, \ldots, x_n^{(2)}) \tag{7.2}$$

这个过程等价于沿着n基向量集合进行一系列线搜索，其中第i个基向量除了第i个分量为1外，其余全为零（算法7.1）。例如，在四维空间中，第三个基函数，记为e(3)，是：

$$\mathbf{e}^{(3)} = [0, 0, 1, 0] \tag{7.3}$$

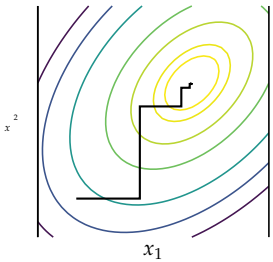


图7.1. 循环坐标下降法在坐标方向之间交替进行。

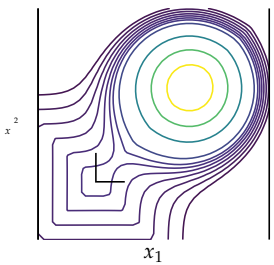


图7.2. 上面是一个示例，说明循环坐标搜索如何可能陷入困境。沿任一坐标方向移动只会导致f增加，但沿对角线移动，虽然在循环坐标搜索中不允许，却可能导致f降低。

```
basis(i, n) = [k == i ? 1.0 : 0.0 for k in 1 : n]
```

算法7.1。构造长度为n的第i个基向量的函数。

图7.1 sh ows 展示了一个在二维空间中进行搜索的示例 空间。

与最速下降法一样，循环坐标搜索保证每次迭代要么改进要么保持不变。若对所有坐标完成一个完整循环后没有显著改进，则表明该方法已收敛。算法 7.2 给出了实现。如图 7.2 所示，循环坐标搜索甚至可能无法找到局部最小值。

```
function cyclic_coordinate_descent(f, x, ϵ)
    Δ, n = Inf, length(x)
    while abs(Δ) > ϵ
        x′ = copy(x)
        for i in 1 : n
            d = basis(i, n)
            x = line_search(f, x, d)
        end
        Δ = norm(x - x′)
    end
    return x
end
```

算法7.2。循环坐标下降法以目标函数f和起始点x作为输入，并运行直到一个完整周期内的步长小于给定的容差ϵ。

该方法可以通过增加一个加速步骤来帮助穿越对角谷。对于每个完整循环，从沿e(1)优化x(1)开始，到沿e(n)优化后得到x(n+1)结束，还会沿方向x(n+1) − x(1)进行额外的线搜索。算法7.3中提供了实现，图7.3展示了一个搜索轨迹示例。

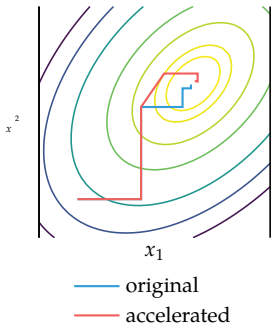


图7.3。将加速步骤添加到循环坐标下降有助于穿越山谷。原始版本和加速版本均展示了六个步骤。

7.2 *Powell's Method*

*Powell's method*1 可以在彼此不正交的方向上搜索。该
该方法可以自动适应长而窄的山谷，否则这些山谷可能需要循环坐标下降或其他沿轴对齐方向搜索的方法进行大量迭代。

1 Powell 方法最早由 M. J. D. Powel l 在“一种无需计算导数即可求多变量函数最小值的有效方法”中提出，*Computer Journal*，第 7 卷，第 2 期，第 155–162 页，1964 年。W. H. Press、S. A. Teukolsky、W. T. V etterling 和 B. P. Flannery 给出了概述，*Numerical*。剑桥大学出版社 *Recipes in C: The Art of Scientific Computing*，1982 年，第 2 卷。

```
function cyclic_coordinate_descent_with_acceleration_step(f, x, ϵ)
    Δ, n = Inf, length(x)
    while abs(Δ) > ϵ
        x′ = copy(x)
        for i in 1 : n
            d = basis(i, n)
            x = line_search(f, x, d)
        end
        x = line_search(f, x, x - x′) # acceleration step
        Δ = norm(x - x′)
    end
    return x
end
```

算法7.3。带有加速步骤的循环坐标下降方法以目标函数f和起始点x为输入，并运行直到一个完整周期内的步长小于给定容差ϵ。

该算法维护一个搜索方向列表 u(1)、...、u(n)，最初这些方向是坐标基向量，对所有 i，u(i) = e(i)。从 x(1) 开始，鲍威尔方法依次对每个搜索方向进行线搜索，每次更新设计点：

$$\mathbf{x}^{(i+1)} \leftarrow \texttt{line_search}(f, \mathbf{x}^{(i)}, \mathbf{u}^{(i)}) \text{ for all } i \text{ in } \{1, \ldots, n\} \tag{7.4}$$

接下来，所有搜索方向向下移动一个索引，丢弃最旧的搜索方向u(1)：

$$\mathbf{u}^{(i)} \leftarrow \mathbf{u}^{(i+1)} \text{ for all } i \text{ in } \{1, \ldots, n-1\} \tag{7.5}$$

最后一次搜索方向被替换为从 x(1) 到 x(n+1) 的方向，即上一个周期的总体进展方向：

$$\mathbf{u}^{(n)} \leftarrow \mathbf{x}^{(n+1)} - \mathbf{x}^{(1)} \tag{7.6}$$

然后沿着新方向进行另一次线搜索，以获得新的x(1)。重复此过程直到收敛。算法7.4给出了实现方式。图7.4展示了一个搜索轨迹示例。

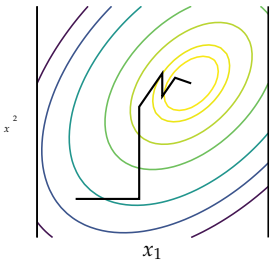


图7.4. 鲍威尔方法开始时与循环坐标下降相同，但通过迭代学习共轭方向。

Powell 表明，对于二次函数，在 k 次完整迭代之后，最后 k 个方向将互相共轭。回顾一下，沿互相共轭方向进行 n 次线搜索将优化一个二次函数。因此，Powell 方法的 n 次完整迭代，总计 $n(n+1)$ 次线搜索，将最小化一个二次函数。

```
function powell(f, x, ϵ)
    n = length(x)
    U = [basis(i,n) for i in 1 : n]
    Δ = Inf
    while Δ > ϵ
        x′ = x
        for i in 1 : n
            d = U[i]
            x′ = line_search(f, x′, d)
        end
        for i in 1 : n-1
            U[i] = U[i+1]
        end
        U[n] = d = x′ - x
        x′ = line_search(f, x, d)
        Δ = norm(x′ - x)
        x = x′
    end
    return x
end
```

算法7.4。鲍威尔方法，其采用目标函数f、起始点x以及容差ϵ。

放弃最旧的搜索方向而以整体进展方向取而代之的过程，可能导致搜索方向变为线性相关。若没有线性独立的搜索向量，搜索方向将无法覆盖整个设计空间，方法可能无法找到最小值。这一弱点可通过定期将搜索方向重置为基础向量来缓解。一项建议是每 n 或 $n+1$ 次迭代重置一次。

7.3 *Hooke-Jeeves*

Hooke-Jeeves method (算法 7.5) 根据每个坐标方向上的小步长评估来遍历搜索空间。[2] 在每次迭代中，Hooke-Jeeves方法在从*anchoring point* x开始的每个坐标方向上，针对给定步长α评估$f(x)$和$f(x \pm \alpha e^{(i)})$。它接受任何可能发现的改进。如果没有找到改进，它将减小步长。此过程重复进行，直到步长足够小。图7.5展示了该算法的几次迭代。

2 R. Hooke 和 T. A. Jeeves，“数值和统计问题的直接搜索解法”，*Journal of the ACM (JACM)*，第8卷，第2期，第212-229页，1961年。

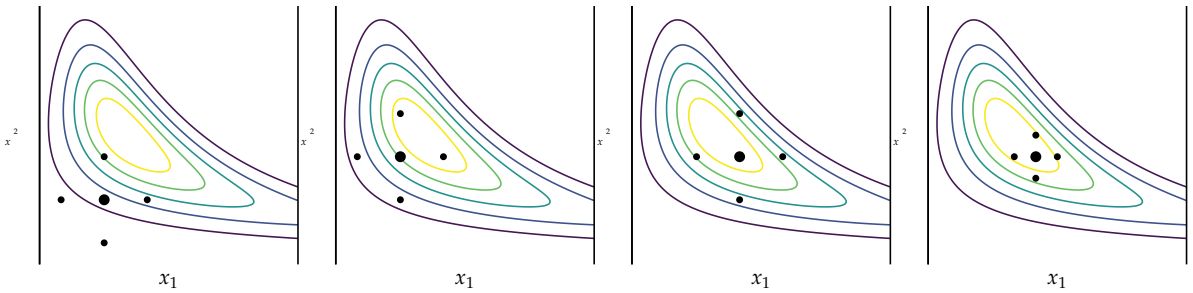


图7.5. Hooke-Jeeves方法，从左至右进行。它起初采用较大的步长，但一旦在任何坐标方向上迈出一步都无法改进时，便会减小步长。

Hooke-Jeeves方法的一步需要对n维问题进行$2n$次函数评估，对于维度较多的问题，这可能代价高昂。Hooke-Jeeves方法易受局部最小值的影响。该方法已被证明在某些类别的函数上收敛。[3]

3 E. D. Dolan, R. M. Lewis, 和 V. Torczon, ‘‘关于模式搜索的局部收敛性,’’ *SIAM*, 第14卷, 第2期, 第567–583页, 2003年.

7.4 Generalized Pattern Search

Journal on Optimization

与在坐标方向上进行搜索的Hooke-Jeeves方法不同，*generalized pattern search*能够在任意方向上进行搜索。根据以下方式，可以围绕锚点x，利用一组方向$\mathcal{D}$和步长α构建*pattern* $\mathcal{P}$:

$$\mathcal{P} = \{\mathbf{x} + \alpha\mathbf{d} \text{ for each } \mathbf{d} \text{ in } \mathcal{D}\} \tag{7.7}$$

Hooke-Jeeves 方法在 n 维问题中使用 $2n$ 个方向，但广义模式搜索可以使用少至 $n + 1$。

为使广义模式搜索收敛到局部最小值，必须满足某些条件。方向集必须是一个*positive spanning set*，这意味着我们可以使用$\mathcal{D}$中方向的非负线性组合来构造$\mathbb{R}^n$中的任何点。正生成集确保从具有非零梯度的位置出发，至少有一个方向是下降方向。[4]

4 广义模式搜索的收敛性保证要求所有采样点都落在一个缩放格点上。因此，每个方向必须是固定非奇异矩阵 G 与整数向量 z 的乘积 $d^{(j)} = Gz^{(j)}$。V. Torczon, “关于模式搜索算法的收敛性”，*SIAM Journal of Op-*，第 7 卷，第 1 期，第 1–25 页，1997 年。

我们可以判断给定的一组方向 $\mathcal{D} = \{d(1), d(2), \ldots, d(m)\}$ 在 $\mathbb{R}^n$ 中是否为正生成集。首先，我们构造矩阵 D，其列为 $\mathcal{D}$ (见图 7.6) 中的方向。方向集合 $\mathcal{D}$ 是正生成集，如果 D 具有满行秩，并且 $Dx = -D1$ 与 $x \geq 0$ 有解。[5] 这

timization

优化问题与线性规划的初始化阶段完全相同，这在第11章中有详细说明。

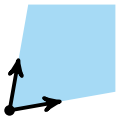
ing

5 R. G. Regis, “关于正生成集和正基的性质”，*Optimization and Engineer-*，第17卷，第1期，第229–262页，2016年。

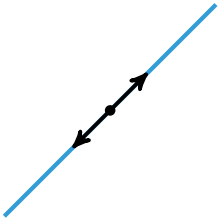
```
function hooke_jeeves(f, x, α, ϵ, γ=0.5)
    y, n = f(x), length(x)
    while α > ϵ
        improved = false
        x_best, y_best = x, y
        for i in 1 : n
            for sgn in (-1,1)
                x′ = x + sgn*α*basis(i, n)
                y′ = f(x′)
                if y′ < y_best
                    x_best, y_best, improved = x′, y′, true
                end
            end
        end
        x, y = x_best, y_best

        if !improved
            α *= γ
        end
    end
    return x
end
```

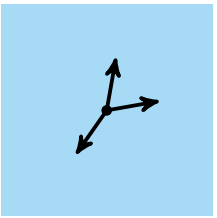
算法7.5. Hooke-Jeeves方法，其参数包括目标函数f、起始点x、初始步长α、容差ϵ以及步长衰减γ。该方法持续运行，直至步长小于ϵ且沿坐标方向采样的点不再提供改进。基于A. F. Kaupe Jr在“算法178：直接搜索”，*Communications of the ACM*，第6卷，第6期，第313-314页，1963年的实现。



仅正向张成锥体



仅正张成1维空间



正张成 $\mathbb{R}^2$

图7.6。广义模式搜索的有效模式需要一个正生成集。这些方向存储在集合{v*}中。

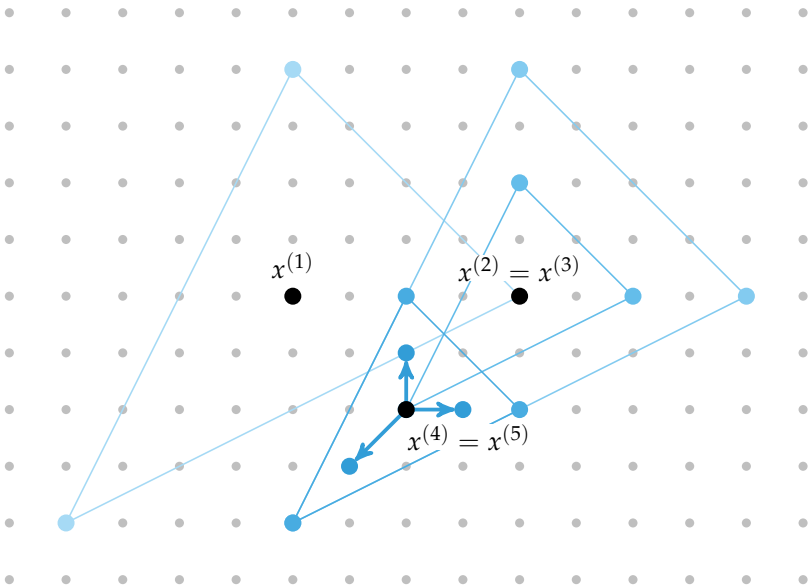


图 7.7. 广义模式搜索中的所有先前点都位于一个缩放格或网格上。该格并非显式构建，且无需轴对齐。

在算法7.6中，广义模式搜索的实现包含了对原始Hooke-Jeeves方法的额外增强。[6]首先，实现
实现方式是*opportunistic*——一旦某个评估改进了当前最佳设计，它就被接受为下一次迭代的锚定设计点。其次，该实现使用*dynamic ordering*来加速收敛——将导致改进的方向提升到方向列表的开头。图7.7展示了该算法的几次迭代。 *mization*

6 这些改进在C. Audet和J. E. Dennis Jr.的‘约束优化的网格自适应直接搜索算法’中提出，*SIAM Journal on Opti-*，第17卷，第1期，第188-217页，2006年。

7.5 *Nelder-Mead Simplex Method*

Nelder-Mead simplex method[7] 使用单纯形在搜索空间中遍历
的最小值。*simplex*是四面体到n维空间的推广。一维的单纯形是一条线，二维的单纯形是三角形（见图7.8）。单纯形得名于它是在任何给定空间中最简单的多胞形。 *Computer Journal*

tion

7 原始单纯形方法在 J. A. Nelder 和 R. Mead 的论文《A Simplex Method for Function Minimization》中有所介绍，*The*，第 7 卷，第 4 期，第 308–313 页，1965 年。我们采纳了 J. C. Lagarias、J. A. Reeds、M. H. Wright 和 P. E. Wright 在《Convergence Properties of the Nelder–Mead Simplex Method in Low Dimensions》中的改进，*SIAM Journal on Optimiza-*，第 9 卷，第 1 期，第 112–147 页，1998 年。

```
function generalized_pattern_search(f, x, α, D, ϵ, γ=0.5)
    y, n = f(x), length(x)
    while α > ϵ
        improved = false
        for (i,d) in enumerate(D)
            x′ = x + α*d
            y′ = f(x′)
            if y′ < y
                x, y, improved = x′, y′, true
                D = pushfirst!(deleteat!(D, i), d)
                break
            end
        end
        if !improved
            α *= γ
        end
    end
    return x
end
```

算法7.6. 广义模式搜索，该算法接受目标函数 `f`、起始点 `x`、初始步长 `α`、一组搜索方向 `D`、容差 `ϵ` 和步长衰减 `γ`。该方法运行直至步长小于 `ϵ` 且沿坐标方向采样的点不再提供改进。

Nelder-Mead方法使用一系列规则，这些规则根据目标函数在其顶点处的评估来决定单纯形如何更新。流程图7.9概述了该过程，算法7.7给出了实现。与Hooke-Jeeves方法类似，单纯形可以在大致保持其大小的同时移动，并且在接近最优解时可以缩小。

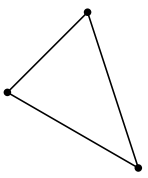


图7.8。二维空间中的单纯形是一个三角形。为了使单纯形有效，它必须具有非零面积。

单纯形由点 $\mathrm{x}_{(1)}$、⋯、$\mathrm{x}_{(n+1)}$ 组成。令 x_h 为函数值最高的顶点，令 x_s 为函数值第二高的顶点，并令 x_ℓ 为函数值最低的顶点。令 $\bar{\mathrm{x}}$ 为除最高点 x_h 外的所有顶点的均值。最后，对于任意设计点 x_θ，令 $y_\theta = f(\mathrm{x}_\theta)$ 为其目标函数值。随后，单次迭代会评估四种单纯形操作：

Reflection. $\mathrm{x}_r = \bar{\mathrm{x}} + \alpha(\bar{\mathrm{x}} - \mathrm{x}_h)$，反映了相对于质心的最高值点。这通常使单纯形从高区域向低区域移动。在这里，$\alpha > 0$，通常设为1。

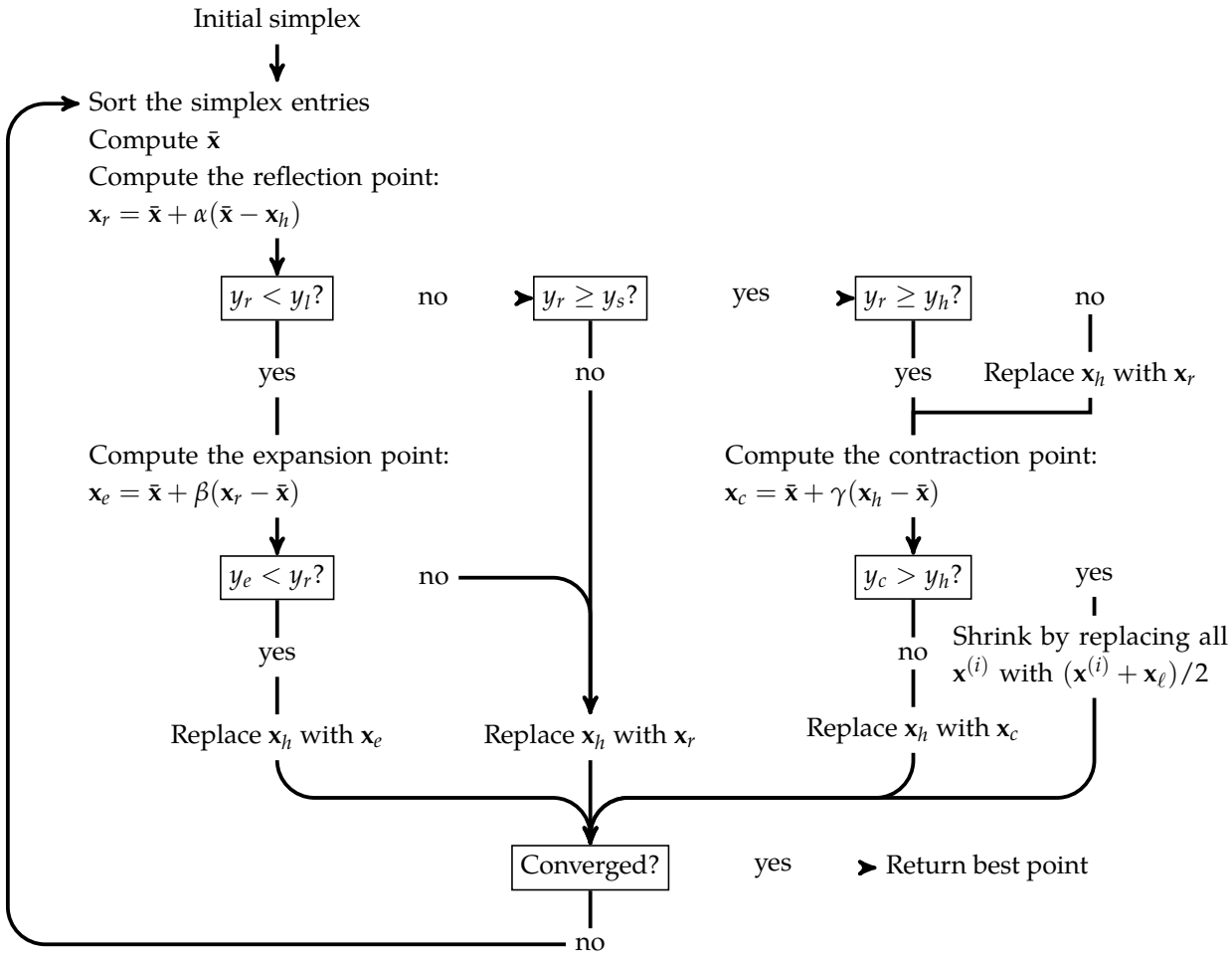


图7.9. Nelder-Mead算法流程图。

Expansion. $\mathrm{x}_e = \bar{\mathrm{x}} + \beta(\mathrm{x}_r - \bar{\mathrm{x}})$，类似于反射，但反射点被送得更远。这是在反射点的目标函数值小于单纯形中所有点时进行的。这里，$\beta > \max(1, \alpha)$，通常设定为 2。

Contraction. $\mathrm{x}_c = \bar{\mathrm{x}} + \gamma(\mathrm{x}_h - \bar{\mathrm{x}})$，单纯形通过远离最差点来收缩。它由 $\gamma \in (0, 1)$ 参数化，通常设为0.5。

Shrinkage. 所有点向最佳点移动，通常将分离距离减半。

图7.10展示了四种单纯形操作。图7.11展示了该算法的几次迭代。

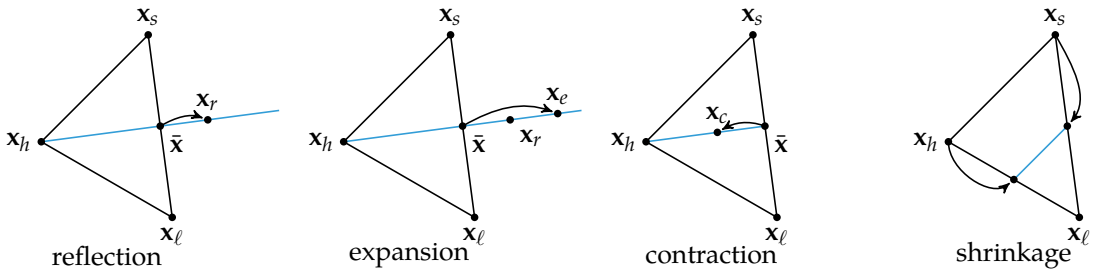


图7.10. 在二维空间中可视化的Nelder-Mead单纯形操作。

Nelder-Mead单纯形法的收敛准则与Powell方法不同，它考虑的是函数值的变化，而不是设计空间中点的变化。它将样本[8] $y^{(1)}, \ldots, y^{(n+1)}$的标准差与容差$\epsilon$进行比较。这个值对于

在高曲率区域上的单纯形，其值较大；而在平坦区域上的单纯形，其值较低。高曲率区域表明仍有进一步优化的可能。

8 *standard deviation of the sample* 也称为 *uncorrected sample*。在我们的情况下，它是 $\sqrt{\frac{1}{n+1}\sum_{i=1}^{n+1}\left(y^{(i)} - \bar{y}\right)^2}$，其中$\bar{y}$ 是 $y^{(1)}$，...，$y^{(n+1)}$ 的平均值。

standard deviation

7.6 Divided Rectangles

*divided rectangles*算法，或称*DIRECT*（用于分割矩形），是一种利普希茨优化方法，在某些方面与第3.6节中描述的Shubert-Piyavskii方法相似。[9] 然而，它消除了指定

Lipschitz常数，并且它可以更有效地扩展到多维。

9 D. R. Jones, C. D. Perttunen, and B. E. Stuckman, ‘‘Lipschitzian Op- timization Without the Lipschitz Constant,’’ *Journal of Optimization, Theory and Applications*, vol. 79, no. 1, pp. 157–181, 1993.

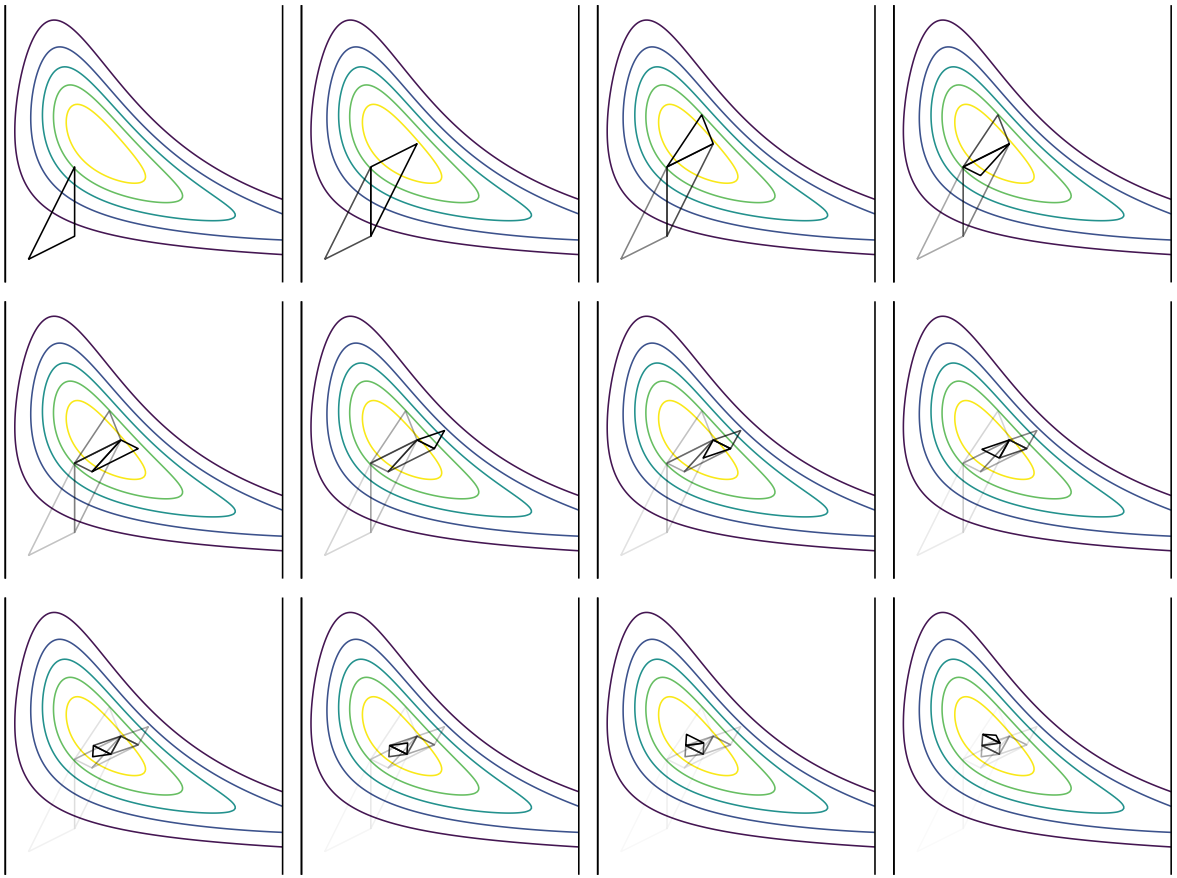


图7.11. Nelder-Mead方法，从左到右、从上到下进行。

```
function nelder_mead(f, S, ϵ; α=1.0, β=2.0, γ=0.5)
    Δ, y_arr = Inf, f.(S)
    while Δ > ϵ
        p = sortperm(y_arr) # sort lowest to highest
        S, y_arr = S[p], y_arr[p]
        xl, yl = S[1], y_arr[1] # lowest
        xh, yh = S[end], y_arr[end] # highest
        xs, ys = S[end-1], y_arr[end-1] # second-highest
        xm = mean(S[1:end-1]) # centroid
        xr = xm + α*(xm - xh) # reflection point
        yr = f(xr)

        if yr < yl
            xe = xm + β*(xr-xm) # expansion point
            ye = f(xe)
            S[end],y_arr[end] = ye < yr ? (xe, ye) : (xr, yr)
        elseif yr > ys
            if yr ≤ yh
                xh, yh, S[end], y_arr[end] = xr, yr, xr, yr
            end
            xc = xm + γ*(xh - xm) # contraction point
            yc = f(xc)
            if yc > yh
                for i in 2 : length(y_arr)
                    S[i] = (S[i] + xl)/2
                    y_arr[i] = f(S[i])
                end
            else
                S[end], y_arr[end] = xc, yc
            end
        else
            S[end], y_arr[end] = xr, yr
        end

        Δ = std(y_arr, corrected=false)
    end
    return S[argmin(y_arr)]
end
```

算法7.7. Nelder-Mead单纯形法，它采用目标函数f、由向量列表组成的初始单纯形S以及容差ϵ。Nelder-Mead参数也可以指定，并默认为推荐值。

Lipschitz连续性的概念可以推广到多维空间。如果f在定义域$\mathcal{X}$上Lipschitz连续，且Lipschitz常数为$\ell > 0$，则对于给定的设计$x^{(1)}$和$y = f(x^{(1)})$，圆锥

$$f(\mathbf{x}^{(1)}) - \ell\|\mathbf{x} - \mathbf{x}^{(1)}\|_2 \tag{7.8}$$

构成f的一个下界。给定m次函数评估，其设计点为$\{x^{(1)}, \ldots, x^{(m)}\}$，我们可以通过取这些下界的最大值来构造它们的叠加：

$$\underset{i}{\text{maximize}}\, f(\mathbf{x}^{(i)}) - \ell\|\mathbf{x} - \mathbf{x}^{(i)}\|_2 \tag{7.9}$$

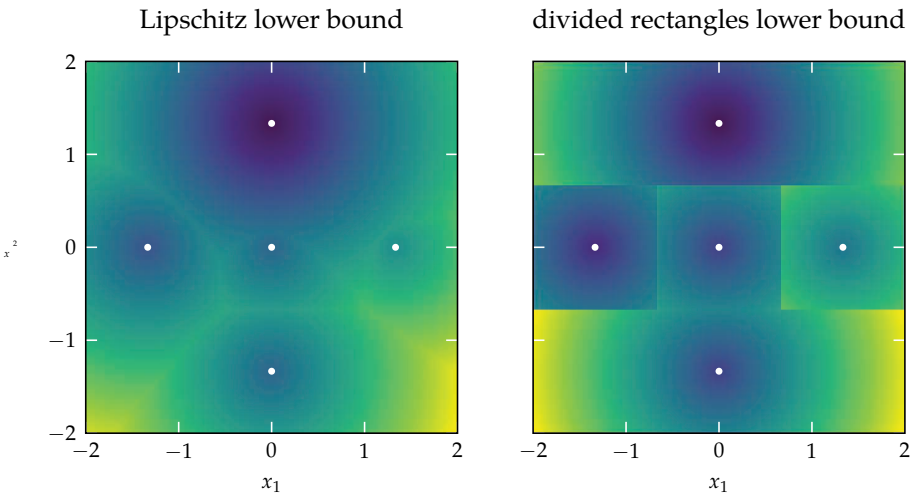


图7.12。Lipschitz下界是锥体的交集，这在高维空间中形成了复杂的表面。划分矩形下界将每个下界锥体隔离到其自身的超矩形区域，使得在给定Lipschitz常数的情况下，计算每个区域中的最小值变得简单。

Shubert-Piyavskii 方法在由已知 Lipschitz 常数导出的下界最低的点处进行采样。不幸的是，Lipschitz 下界具有复杂的几何形状，其复杂度随设计空间维数的增加而增加。图 7.12 左侧的等高线图展示了使用五次函数评估得到的这样一个下界。右侧的等高线图展示了 DIRECT 所做的近似，该方法将区域划分为超矩形——每个设计点周围各有一个。做出这一假设可以快速计算下界的最小值。

DIRECT方法不假设Lipschitz常数已知。图7.13和图7.14分别展示了利用Lipschitz连续性和DIRECT逼近构建的下界，针对几个不同的Lipschitz常数。注意到最小值的位置随着Lipschitz常数的变化而变化，较小的ℓ值导致设计接近最低函数评估值，而较大的ℓ值则导致设计远离先前函数评估值。

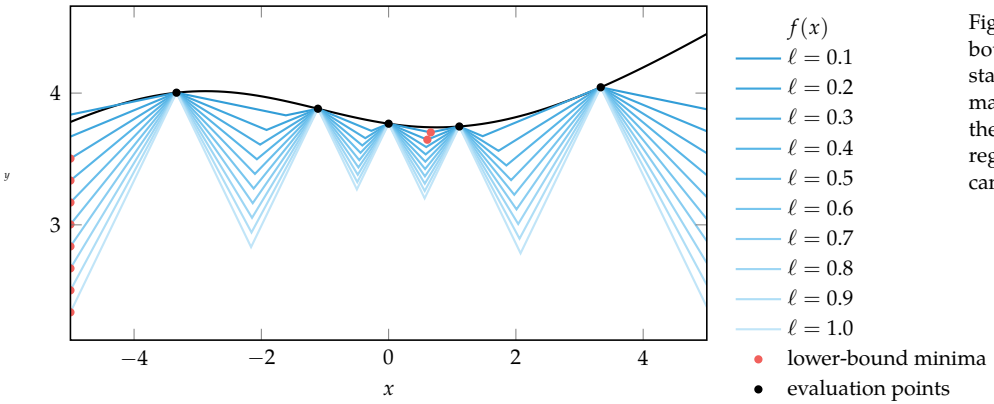


Figure 7.13. The Lipschitz 较低 bound for different Lipschitz constants ℓ. Not only does the estimated minimum change loc盟友作为 the Lipschitz constant is vari编辑了，该 region in which the minimum 位于 can vary as well.

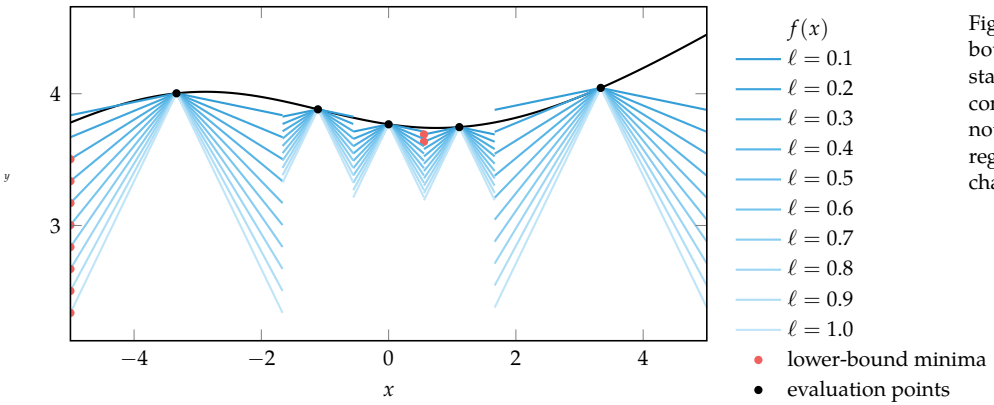


Figure 7.14. The DIRECT 较低 bound for different Lipschitz constants ℓ. The lower bound 不是 continuous. The minimum 做 not change locally but can c汉格 regionally as the Lipschitz constant changes.

7.6.1 *Univariate DIRECT*

在一维情况下，DIRECT算法递归地将区间三等分，然后在区间的中心处对目标函数进行采样，如图7.15所示。这种方法与Shubert-Piyavskii方法形成对比，后者的采样发生在由已知Lipschitz常数导出的下界最低的点上。

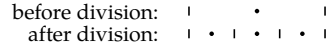


图7.15. 使用DIRECT方案的中心点采样将区间分为三等分。

对于中心为$c = (a + b)/2$的区间$[a, b]$，基于$f(c)$的下界是：

$$f(x) \geq f(c) - \ell|x - c| \tag{7.10}$$

其中 ℓ 是未知的利普希茨常数。该界在该区间上取得的最小值是 $f(c) - \ell(b - a)/2$，并且它在区间的边缘处取得。

即使我们不知道ℓ，我们也可以推断出某些区间的下界低于其他区间。例如，如果我们有两个长度相同的区间，且第一个区间在中点处的评估值低于第二个区间，那么第一个区间的下界就低于第二个区间的下界。尽管这并不意味着第一个区间包含最小值点，但它表明我们可能希望将搜索重点放在该区间内。

在我们的搜索过程中，我们会遇到许多宽度不同的区间$[a_1, b_1]$、...、$[a_n, b_n]$。我们可以根据区间的中心值和宽度来绘制这些区间，如图7.16所示。每个区间的下界是通过其中心点、斜率为ℓ的直线的垂直截距。当我们从下方以斜率ℓ向上移动直线时，首个相交的点将是具有最低下界的区间的中心。

DIRECT 分割所有存在 Lipschitz 常数的区间，使得它们具有最低的下界，如图 7.17 所示。我们将这些选中的区间称为潜在最优区间。尽管从技术上讲，任何区间都可能包含最优解，但根据启发式方法，选中的点最有可能包含最优解。

一维DIRECT算法的一次迭代包括识别可能最优区间集合，然后将每个区间分成三等份。例7.1演示了单变量DIRECT方法。

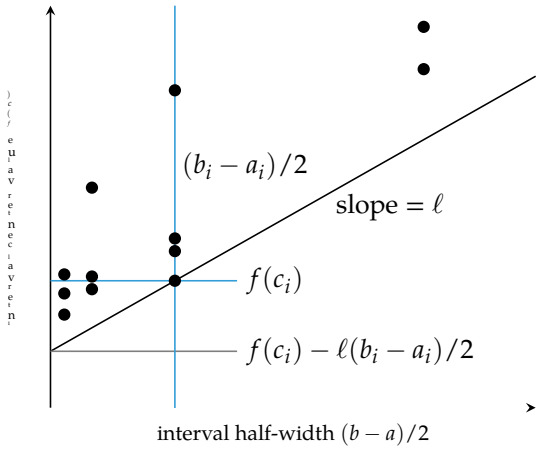


图7.16. 针对特定利普希茨常数 ℓ 的区间选择。黑点代表 DIRECT 区间及其中点对应的函数评估值。通过属于所选区间的点绘制一条斜率为 ℓ 的黑线。所有其他点必须位于此线上或其上方。

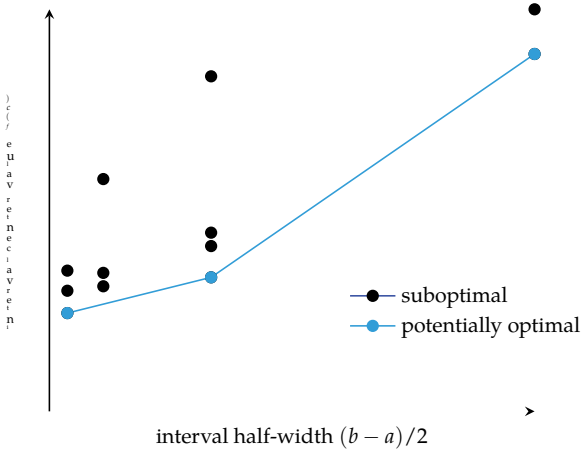
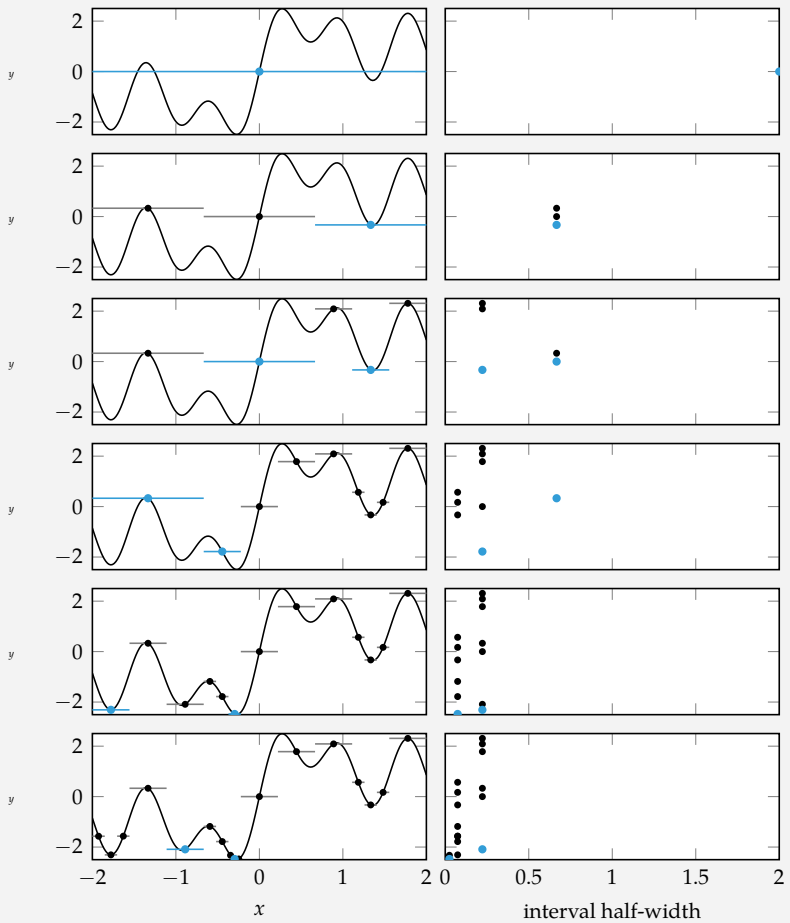


图7.17。DIRECT方法的潜在最优区间构成了一条分段边界，这条边界围住了右下方的所有区间。每个点对应一个区间。

考虑函数 $f(x) = \sin(x) + \sin(2x) + \sin(4x) + \sin(8x)$ 在区间 $[-2, 2]$ 上，全局最小值点约在 -0.272 附近。由于存在多个局部极小值点，优化较为困难。

下图展示了单变量 DIRECT 方法的进展，其中选择分割的区间以蓝色显示。左侧显示了叠加在目标函数上的区间。右侧显示了区间在半宽与中心值空间中的散点图。



示例 7.1. 将 DIRECT 方法应用于单变量函数。

7.6.2 *Multivariate DIRECT*

在多维情况下，我们划分的是矩形（或在超过两维时，超矩形）而非区间。与一维情况类似，我们沿轴线方向将矩形分成三等分。在开始划分矩形之前，DIRECT算法会将搜索空间标准化为单位超立方体。

如图7.18所示，分割单位超立方体时方向的选择顺序至关重要。DIRECT算法优先为函数评估值较低的点分配较大的矩形。较大的矩形会被优先考虑进行进一步分割。

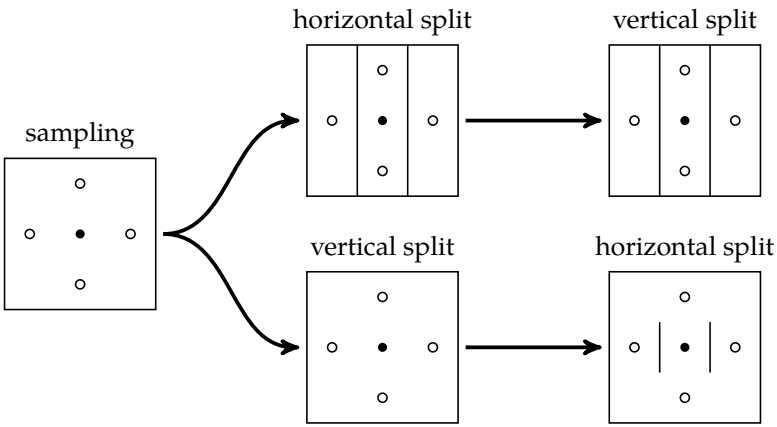


图7.18. 多维区间分割（使用DIRECT算法）需为分割维度选择排序顺序。

拆分边长不相等的区域时，仅拆分最长的维度（图7.19）。随后，按照与超立方体相同的方式继续在这些维度上进行拆分。

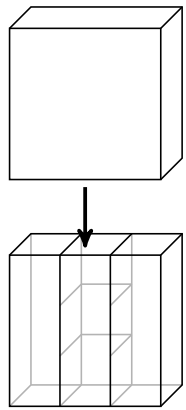


图7.19. DIRECT仅会划分超矩形中最长的维度。

潜在最优区间集如同在一维情况下一样获得。每个超矩形的下界可以根据最长边长和中心值来计算。我们可以构建一个类似于图7.16的图来识别潜在最优矩形。[10]

10 作为选择的额外要求，DIRECT还要求区间的下界能够显著改进当前最优值。

7.6.3 *Implementation*

将DIRECT（算法7.8）分解为子程序时，最能理解其原理。下面我们介绍这些子程序。

通过算法7.9完成到单位超立方体的归一化。

```
function direct(f, a, b, ϵ, k_max)
    g = reparameterize_to_unit_hypercube(f, a, b)
    intervals = Intervals()
    n = length(a)
    c = fill(0.5, n)
    interval = Interval(c, g(c), fill(0, n))
    add_interval!(intervals, interval)
    c_best, y_best = copy(interval.c), interval.y

    for k in 1 : k_max
        S = get_opt_intervals(intervals, ϵ, y_best)
        to_add = Interval[]
        for interval in S
            append!(to_add, divide(g, interval))
            dequeue!(intervals[min_depth(interval)])
        end
        for interval in to_add
            add_interval!(intervals, interval)
            if interval.y < y_best
                c_best, y_best = copy(interval.c), interval.y
            end
        end
    end

    return rev_unit_hypercube_parameterization(c_best, a, b)
end
```

算法7.8. DIRECT，该算法接受多维目标函数`f`、下界向量`a`、上界向量`b`、容差参数`ϵ`以及迭代次数`k_max`。它返回最佳坐标。

```
rev_unit_hypercube_parameterization(x, a, b) = x.*(b-a) + a
function reparameterize_to_unit_hypercube(f, a, b)
    Δ = b-a
    return x->f(x.*Δ + a)
end
```

算法7.9. 一个函数，它创建了一个定义在单位超立方体上的函数，该函数是定义在下界和上界为`a`和`b`的超立方体上的函数`f`的重参数化版本。

计算潜在最优矩形集合可以高效完成（算法7.10）。我们利用区间宽度只能取三分之一的幂这一事实，因此许多点将共享相同的x坐标。对于任何给定的x坐标，只有具有最低y值的点才可能是潜在最优区间。我们根据深度存储矩形区间，然后根据中心点值将其存储在优先队列中。

```
using DataStructures
struct Interval
    c
    y
    depths
end
min_depth(interval) = minimum(interval.depths)
const Intervals = Dict{Int,PriorityQueue{Interval, Float64}}
function add_interval!(intervals, interval)
    d = min_depth(interval)
    if !haskey(intervals, d)
        intervals[d] = PriorityQueue{Interval, Float64}()
    end
    return enqueue!(intervals[d], interval, interval.y)
end
```

算法7.10. DIRECT中使用的数据结构。在这里，`Interval`包含三个字段：区间中心`c`、中心点值`y = f(c)`以及每个维度上的分割数`depths`。函数`add_interval!`将一个新的`Interval`插入到数据结构中。

我们可以使用这个数据结构来获取所有潜在的最优区间(算法7.11)。该算法从最低的区间宽度进行到最高的区间宽度。对于每个点,我们首先判断它是在连接前两个点的直线的上方还是下方。如果它在下方,我们就跳过它。然后对下一个点进行同样的判断。

最后，我们需要一种划分区间的方法。这由`divide`方法实现（算法7.12）。

在二维空间中运行DIRECT得到的间隔在图7.20中展示。二维空间中DIRECT的两次迭代在例7.2中详细计算。

```
function get_opt_intervals(intervals, ϵ, y_best)
    max_depth = maximum(keys(intervals))
    stack = [DataStructures.peek(intervals[max_depth])[1]]
    d = max_depth-1
    while d ≥ 0
        if haskey(intervals, d) && !isempty(intervals[d])
            interval = DataStructures.peek(intervals[d])[1]
            x, y = 0.5*3.0^(-min_depth(interval)), interval.y

            while !isempty(stack)
                interval1 = stack[end]
                x1 = 0.5*3.0^(-min_depth(interval1))
                y1 = interval1.y
                l1 = (y - y1)/(x - x1)
                if y1 - l1*x1 > y_best - ϵ || y < y1
                    pop!(stack)
                elseif length(stack) > 1
                    interval2 = stack[end-1]
                    x2 = 0.5*3.0^(-min_depth(interval2))
                    y2 = interval2.y
                    l2 = (y1 - y2)/(x1 - x2)
                    if l2 > l1
                        pop!(stack)
                    else
                        break
                    end
                else
                    break
                end
            end

            push!(stack, interval) # add new point
        end
        d -= 1
    end
    return stack
end
```

算法7.11. 获取潜在最优区间的例程，其中`intervals`为`Intervals`类型，`ϵ`为容差参数，`y_best`为最佳函数评估值。

```
function divide(f, interval)
    c, d, n = interval.c, min_depth(interval), length(interval.c)
    dirs = findall(interval.depths .== d)
    cs = [(c + 3.0^(-d-1)*basis(i,n),
           c - 3.0^(-d-1)*basis(i,n)) for i in dirs]
    vs = [(f(C[1]), f(C[2])) for C in cs]
    minvals = [min(V[1], V[2]) for V in vs]

    intervals = Interval[]
    depths = copy(interval.depths)
    for j in sortperm(minvals)
        depths[dirs[j]] += 1
        C, V = cs[j], vs[j]
        push!(intervals, Interval(C[1], V[1], copy(depths)))
        push!(intervals, Interval(C[2], V[2], copy(depths)))
    end
    push!(intervals, Interval(c, interval.y, copy(depths)))
    return intervals
end
```

算法7.12。用于分割区间的`divide`例程，其中`f`是目标函数，`interval`是待分割的区间。它返回分割后得到的较小间隔的列表。

7.7 Summary

- 直接方法仅依赖于目标函数，不使用导数信息。
- 循环坐标搜索一次优化一个坐标方向。
- 鲍威尔方法根据进展方向调整搜索方向的集合。
- Hooke-Jeeves從當前點沿每個坐標方向搜索，使用隨時間調整的步長。
- 广义模式搜索与Hooke-Jeeves方法相似，但它使用的搜索方向较少，这些方向积极地覆盖了整个设计空间。
- Nelder-Mead单纯形法使用单纯形来搜索设计空间，并根据目标函数的评估结果自适应地扩大和缩小单纯形的大小。
- 分割矩形算法将Shubert-Piyavskii方法扩展到了多维空间，并且不需要指定有效的Lipschitz常数。

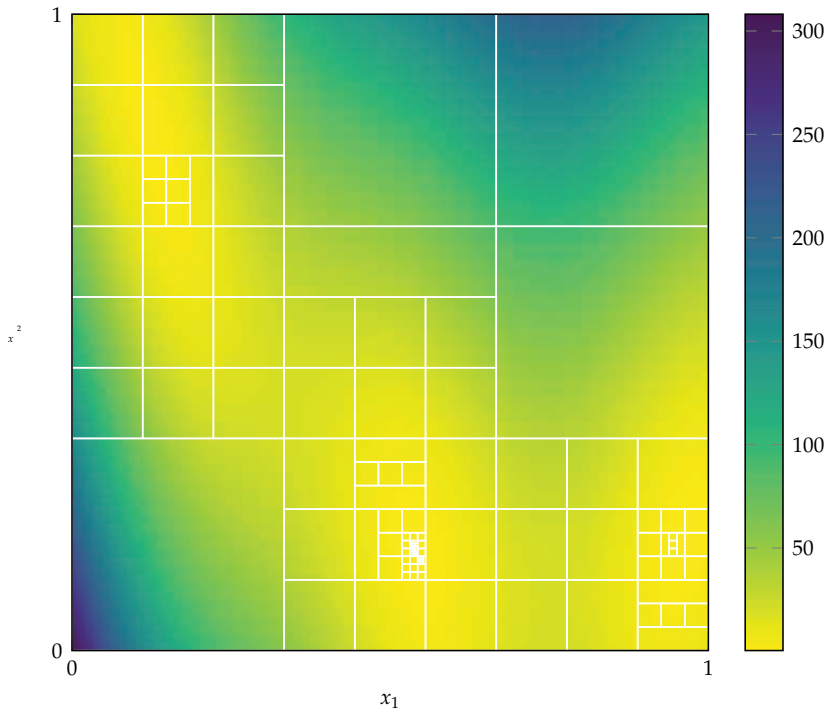


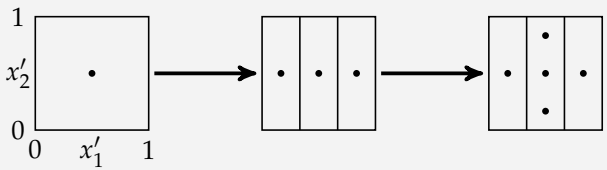
图 7.20. 对 Branin 函数应用 DIRECT 方法 16 次迭代后的结果，见附录 B.3。每个单元格由白线界定。由于 DIRECT 方法在这些区域逐步提高分辨率，单元格在 Branin 函数的极小值附近更加密集。

示例 7.2. 前两次 DIRECT 迭代的详细推导。

考虑使用DIRECT优化花朵函数（附录B.4），在$x_1 \in [-1, 3]$，$x_2 \in [-2, 1]$上。首先将函数归一化到单位超立方体，以便我们优化x'_1，$x'_2 \in [0, 1]$：

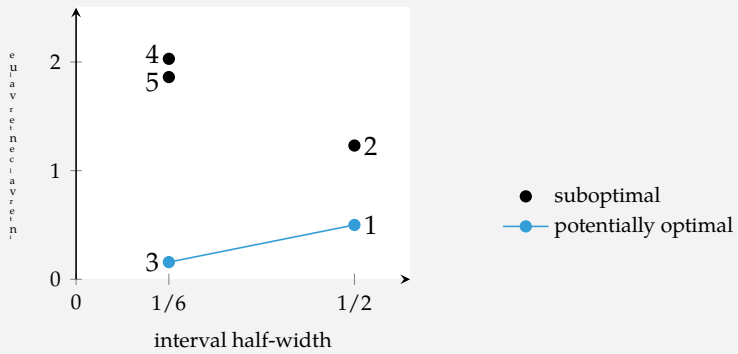
$$f(x'_1, x'_2) = \text{flower}(4x'_1 - 1, 3x'_2 - 2)$$

目标函数在 [0.5, 0.5] 处采样得到 0.158。我们有一个中心为 [0.5, 0.5]、边长为 [1, 1] 的单一区间。该区间被分割两次，首先在 x'_1 中三等分，然后在 x'_2 中将中心区间三等分。



我们现在有五个区间：

interval	center	side lengths	half width	center value
1	[0.25, 0.50]	[1/3, 1]	1/2	0.500
2	[0.75, 0.50]	[1/3, 1]	1/2	1.231
3	[0.50, 0.50]	[1/3, 1/3]	1/6	0.158
4	[0.50, 0.25]	[1/3, 1/3]	1/6	2.029
5	[0.50, 0.75]	[1/3, 1/3]	1/6	1.861



我们需要在中心为 [0.25, 0.5] 和 [0 的两个区间上分割 .5, 0.5]。

7.8 Exercises

练习 7.1。前几章介绍了利用导数下降至最小值的方法。直接方法只能使用零阶信息，即对 f 的评估。使用有限差分法近似一个 n 维目标函数的导数和海森矩阵需要多少次评估？你认为为什么拥有零阶方法是重要的？

练习 7.2. 设计一个目标函数和一个起始点{v*}，使得Hooke-Jeeves方法无法实现目标函数的减少。你需要选择{v*}，使其不是局部最小值点。

练习7.3。使用Hooke-Jeeves方法得到的设计点是否保证在局部最小值的ϵ范围内？

练习 7.4. 给出一个具体的工程问题示例，其中你可能无法计算解析导数。

练习7.5. 阐述一维中的分割矩形算法与Shubert-Piyavskii方法之间的一个差异。

练习 7.6。假设我们的搜索算法让我们从 $\mathrm{x}(k) = [1, 2, 3, 4]$ 转移到 $\mathrm{x}^{(k+1)} = [2, 2, 2, 4]$。我们的搜索算法可能是 (a) 循环坐标搜索，(b) 鲍威尔方法，(c) a 和 b 两者，还是 (d) 两者都不是？为什么？

8 Stochastic Methods

本章介绍了各种*stochastic methods*，它们策略性地利用随机化来帮助探索设计空间以寻找最优解。随机性有助于跳出局部最优，并增加找到全局最优的机会。随机方法通常使用*pseudo-random*数生成器来确保可重复性。[1]

1 尽管伪随机数生成器产生的数字看似随机，但实际上它们是确定性过程的结果。伪随机数可以通过调用`rand`函数来生成。可以通过`Random.jl`包中的`seed!`函数将过程重置为初始状态。

大量随机性通常是无效的，因为它阻碍了我们有效利用先前的评估点来帮助指导搜索。本章讨论了多种控制搜索中随机性程度的方法。

8.1 Noisy Descent

在大型非线性优化问题中，向梯度下降添加随机性可能是有益的。鞍点处梯度非常接近零，可能导致下降方法选择的步长过小，无法发挥作用。一种方法是在每个下降步骤中添加高斯噪声：[2]

$$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha \mathbf{g}^{(k)} + \boldsymbol{\epsilon}^{(k)} \tag{8.1}$$

其中 $\boldsymbol{\epsilon}^{(k)}$ 是均值为零、标准差为 σ 的高斯噪声。噪声量通常随时间减少。噪声的标准差通常是一个递减序列 $\sigma^{(k)}$，例如 $1/k$。[3] 算法 8.1 提供了

2 G. Hinton 和 S. Roweis，“随机邻居嵌入”，载于 *Advances in Neural Information Processing Systems*，2003年。

3 Hinton和Roweis的论文在前3,500次迭代中使用了固定的标准差，并在之后将标准差设为零。

该方法的实现。图8.1比较了在有噪声和无噪声情况下对鞍点函数的下降过程。

训练神经网络的一种常见方法是*stochastic gradient descent*，它使用了带噪声的梯度近似。除了有助于越过鞍点之外，使用随机选择的训练数据子集来评估带噪声的梯度[4]在计算成本上显著低于计算

4 这些子集称为 *batches*。

每次迭代的真实梯度。

```
mutable struct NoisyDescent <: DescentMethod
    submethod
    σ
    k
end                                                    submethod
function init!(M::NoisyDescent, f, ∇f, x)
    init!(M.submethod, f, ∇f, x)
    M.k = 1
    return M
end
function step!(M::NoisyDescent, f, ∇f, x)
    x = step!(M.submethod, f, ∇f, x)
    σ = M.σ(M.k)
    x += σ.*randn(length(x))
    M.k += 1
    return x
end
```

算法8.1。一种噪声下降方法，它通过添加高斯噪声来增强另一种下降方法。该方法接受另一个`DescentMethod`，一个噪声序列`σ`，并存储迭代次数`k`。

随机梯度下降的收敛性保证要求正步长应满足：

$$\sum_{k=1}^{\infty} \alpha^{(k)} = \infty \qquad \sum_{k=1}^{\infty} \left(\alpha^{(k)}\right)^2 < \infty \tag{8.2}$$

这些条件确保步长减小并使方法能够收敛，但不会过快以至于远离局部最小值而停滞。

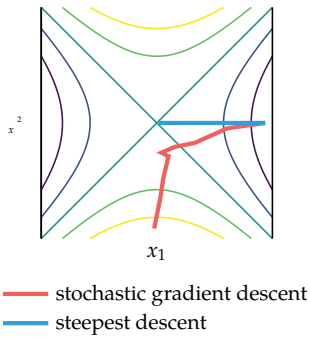


图8.1。在下降法中加入随机性有助于穿越鞍点，如图中所示的$f(x) = x_1^2 - x_2^2$。由于初始化的原因，最速下降法会收敛到梯度为零的鞍点。

8.2 *Mesh Adaptive Direct Search*

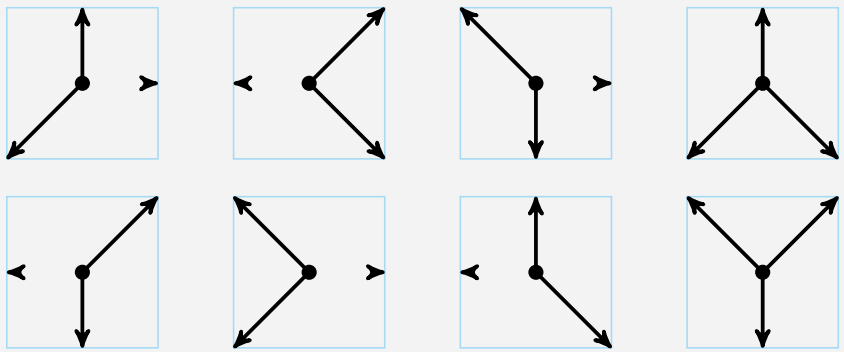
第7.4节中讨论的广义模式搜索方法将局部探索限制在一组固定的方向上。相比之下，*mesh adaptive direct search*使用了随机的正生成方向。[5]

用于采样正生成集的过程（见例8.1）首先构建一个形式为下三角矩阵L的初始线性生成集。L中的对角元素从$\pm 1/\sqrt{\alpha^{(k)}}$中采样，其中$\alpha^{(k)}$是迭代$k$时的步长。L的下部组件从

$$\left\{-1/\sqrt{\alpha^{(k)}} + 1, -1/\sqrt{\alpha^{(k)}} + 2, \ldots, 1/\sqrt{\alpha^{(k)}} - 1\right\} \tag{8.3}$$

5 本节遵循 C. Audet 和 J. E. Dennis Jr. 在“约束优化的网格自适应直接搜索算法”，*SIAM Journal on Optimization*第17卷，第1期，第188–217页，2006年中给出的下三角网格自适应直接搜索。

考虑由非零方向 d_1、$d_2 \in \{-1, 0, 1\}$ 构造的正生成集。由这些方向可以构造出8个包含3个元素的正生成集：



例8.1。针对 $\mathbb{R}^2$，其具有单位 ℓ_1 长度方向的正生成集。

然后随机置换L的行和列，得到一个矩阵D，其列对应于线性张成$\mathbb{R}^n$的n方向。这些方向中的最大幅度为$1/\sqrt{\alpha^{(k)}}$。

从线性张成集获得正张成集有两种常用方法：添加一个额外方向 $\mathrm{d}_{(n+1)} = -\sum_{i=1}^{n} \mathrm{d}_{(i)}$，以及对于 $\{$ 中的 j 添加 n 个额外方向 $\mathrm{d}_{(n+j)} = -\mathrm{d}_{(j)}$，其中 $n\} = 1, \ldots$。我们在算法8.2中使用第一种方法。

步长 α 从 1 开始，始终为 4 的幂，并且从不超过 1。使用 4 的幂会导致每次迭代中可能采取的最大步长按 2 的因子进行缩放，因为最大步长 $\alpha/\sqrt{\alpha}$ 的长度为 $4^m/\sqrt{4^m} = 2^m$，其中整数 $m < 1$。步长根据以下公式更新：

$$\alpha^{(k+1)} \leftarrow \begin{cases} \alpha^{(k)}/4 & \text{if no improvement was found in this iteration} \\ \min(1, 4\alpha^{(k)}) & \text{otherwise} \end{cases} \tag{8.4}$$

网格自适应直接搜索具有机会性，但无法支持动态排序[6]，因为在成功迭代后，步长会增加，并且另一

[6] See section 7.4.

在成功方向上会位于网格之外。算法沿接受的下行方向查询一个新的设计点。如果 $f(x^{(k)} = x^{(k-1)} + \alpha \mathrm{d}) < f(x^{(k-1)})$，那么被查询的点就是 $x^{(k-1)} + 4\alpha \mathrm{d}$ $= x^{(k)} + 3\alpha \mathrm{d}$。该过程是

```
function rand_positive_spanning_set(α, n)
    δ = round(Int, 1/sqrt(α))
    L = Matrix(Diagonal(δ*rand([1,-1], n)))
    for i in 1 : n-1
        for j in i+1:n
            L[i,j] = rand(-δ+1:δ-1)
        end
    end
    D = L[randperm(n),:]
    D = L[:,randperm(n)]
    D = hcat(D, -sum(D,dims=2))
    return [D[:,i] for i in 1 : n+1]
end
```

算法 8.2. 根据网格自适应直接搜索，以步长 α 和维度数 n 随机采样 $n+$ 个方向的正生成集。

如算法8.3所述。图8.2展示了该算法如何探索搜索空间。

8.3 Simulated Annealing

Simulated annealing[7] 从冶金学中汲取灵感。[8] *Temperature* 被使用用来控制随机搜索过程中的随机性程度。温度起初很高，使得过程能够自由地在搜索空间中移动，期望在此阶段能找到具有最佳局部最小值的好区域。随后温度慢慢降低，减少了随机性并迫使搜索收敛至一个最小值。模拟退火常因能逃脱局部最小值而被用于具有多个局部最小值的函数上。

7 S. Kirkpatrick、C. D. Gelatt Jr. 和 M. P. Vecchi，“通过模拟退火进行优化”，*Science*，第220卷，第4598期，第671-680页，1983年。

8 退火是一种将材料加热然后冷却的过程，使其更易加工。当材料加热时，其中的原子更自由地移动，并通过随机运动倾向于 settles 到更好的位置。缓慢冷却使材料达到有序的结晶状态。快速、突然的淬火会导致缺陷，因为材料被迫在其当前状态下 settle。

在每次迭代中，从 x 到 x′ 的一个候选转移是从转移分布 T 中采样的，并以如下概率被接受：

$$\begin{cases} 1 & \text{if } \Delta y \leq 0 \\ \min(e^{-\Delta y/t}, 1) & \text{if } \Delta y > 0 \end{cases} \tag{8.5}$$

其中 $\Delta y = f(\mathrm{x}') - f(\mathrm{x})$ 是目标值之差，t 是温度。正是这个接受概率，被称为 *Metropolis criterion*，使得算法在温度较高时能够逃离局部最小值。

```
function mesh_adaptive_direct_search(f, x, ϵ)
    α, y, n = 1, f(x), length(x)
    while α > ϵ
        improved = false
        for (i,d) in enumerate(rand_positive_spanning_set(α, n))
            x′ = x + α*d
            y′ = f(x′)
            if y′ < y
                x, y, improved = x′, y′, true
                x′ = x + 3α*d
                y′ = f(x′)
                if y′ < y
                    x, y = x′, y′
                end
                break
            end
        end
        α = improved ? min(4α, 1) : α/4
    end
    return x
end
```

算法8.3. 针对目标函数f、初始设计x和容差ϵ的网格自适应直接搜索。

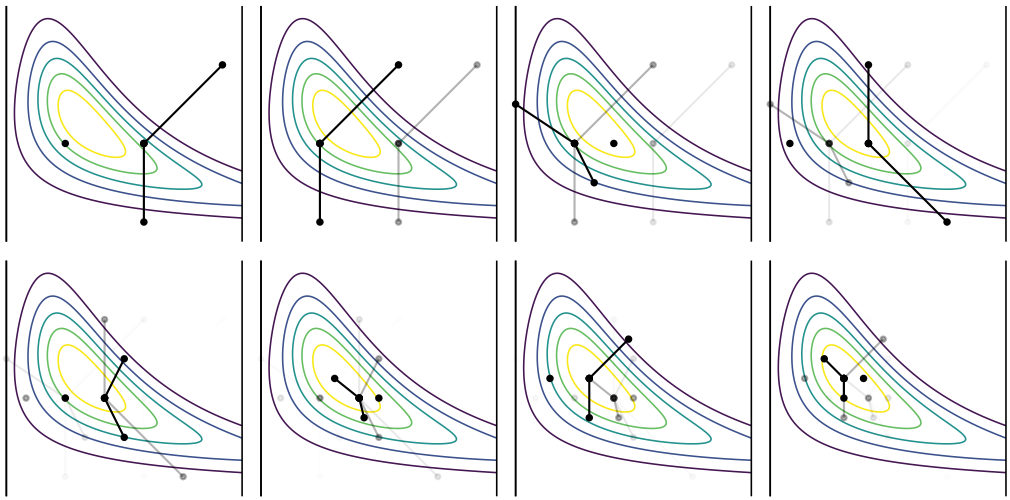


图8.2. 网格自适应直接搜索从左到右、从上到下进行。

温度参数t控制接受概率。随着算法的进行，采用退火计划逐渐降低温度，如图8.3所示。必须降低温度以确保收敛。如果温度下降过快，搜索方法可能无法覆盖包含全局最小值的搜索空间部分。

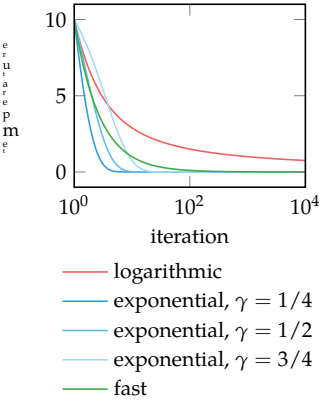


图8.3. 模拟退火中常用的几种退火计划。这些计划的初始温度为10。

可以证明，对于第k次迭代，$t^{(k)} = t^{(1)}$的*logarithmic annealing schedule*为$\ln(2)/\ln(k+1)$时，在一定条件下保证渐近达到全局最优，[9]但在实践中可能非常缓慢。*exponential annealing schedule*更常见的做法是使用简单的衰减因子：

$$t^{(k+1)} = \gamma t^{(k)} \tag{8.6}$$

对于某个$\gamma \in (0, 1)$。另一种常见的退火计划，*fast annealing*，[10] 使用了的温度

$$t^{(k)} = \frac{t^{(1)}}{k} \tag{8.7}$$

算法8.4提供了一个基本的模拟退火实现。例8.2展示了不同转移分布和退火计划对优化过程的影响。

9 B. Hajek，“最优退火中的冷却调度”，*Mathematics*，第13卷，第2期，第311–329页，1988年。*of Operations Research*,

```
function simulated_annealing(f, x, T, t, k_max)
    y = f(x)
    x_best, y_best = x, y
    for k in 1 : k_max
        x′ = x + rand(T)
        y′ = f(x′)
        Δy = y′ - y
        if Δy ≤ 0 || rand() < exp(-Δy/t(k))
            x, y = x′, y′
        end
        if y′ < y_best
            x_best, y_best = x′, y′
        end
    end
    return x_best
end
```

算法8.4。模拟退火算法，其输入为目标函数`f`、初始点`x`、转移分布`T`、退火计划`t`以及迭代次数`k_max`。

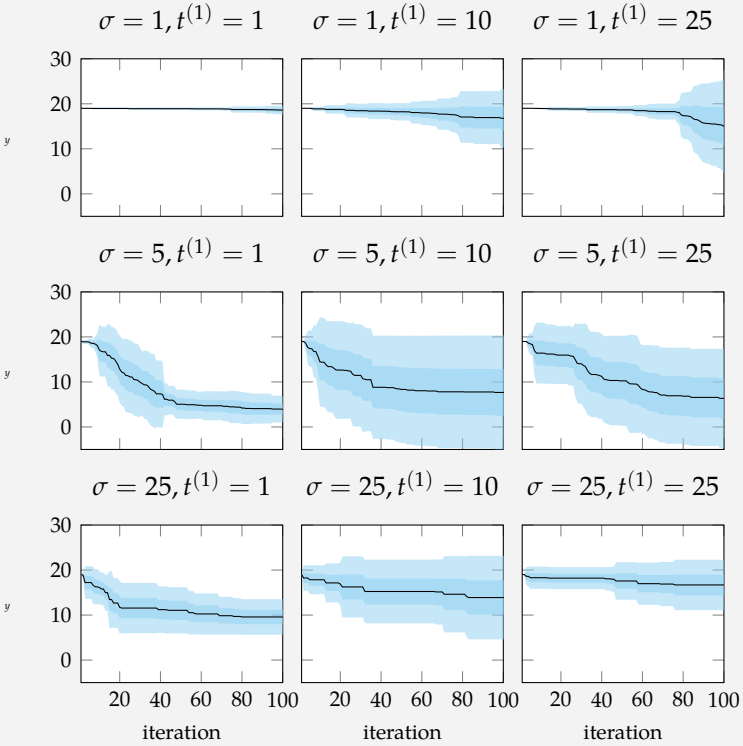
10 H. Szu和R. Hartley，“快速模拟退火”，*Physics Letters*，第122卷，第3-4期，第157-162页，1987年。

1987年，Corana等人提出了一种更为复杂的算法，允许在搜索过程中改变步长。[11]而不是使用固定的

11 A. Corana、M. Marchesi、C. Martini 和 S. Ridella，“使用‘模拟退火’算法最小化连续变量的多峰函数”，*ACM Transactions on Mathematical Software*，第 13 卷，第 3 期，第 262–280 页，1987 年。

我们可以使用模拟退火来优化Ackley函数，附录B.1。Ackley函数有许多局部最小值，使得基于梯度的方法容易陷入困境。

假设我们从 $\mathbf{x}^{(1)} = [15, 15]$ 出发，运行 100 次迭代。下面我们展示了使用三个零均值、对角协方差（$\sigma\mathbf{I}$）高斯转移分布的不同组合以及三种不同的温度调度 $t^{(k)} = t^{(1)}/k$ 进行多次运行时，迭代的分布情况。



在这种情况下，过渡分布的扩散对性能的影响最大。

示例 8.2. 探究分布方差与温度对模拟退火性能的影响。蓝色区域表示目标函数值的5%至95%及25%至75%经验高斯分位数。

过渡分布中，这种自适应模拟退火方法为每个坐标方向维护一个单独的步长v。对于给定点x，在每个坐标方向i上按照以下方式进行一轮随机移动循环：

$$\mathbf{x}' = \mathbf{x} + r v_i \mathbf{e}_i \tag{8.8}$$

其中 r 从 [1, −1] 中均匀随机抽取，而 v_i 是第 i 个坐标方向上的最大步长。每个新点根据 Metropolis 准则被接受。每个坐标方向上被接受的点的数量存储在一个向量 a 中。

在 n_s 个周期后，调整步长，目的是保持接受和拒绝的设计数量大致相等，平均接受率接近一半。拒绝过多的移动会浪费计算努力，而接受过多的移动则表明配置演化过慢，因为候选点与当前位置过于相似。Corana 等人使用的更新公式为：

$$v_i = \begin{cases} v_i\left(1 + c_i \frac{a_i/n_s - 0.6}{0.4}\right) & \text{if } a_i > 0.6 n_s \\ v_i\left(1 + c_i \frac{0.4 - a_i/n_s}{0.4}\right)^{-1} & \text{if } a_i < 0.4 n_s \\ v_i & \text{otherwise} \end{cases} \tag{8.9}$$

c_i 参数控制每个方向上的步长变化，通常设为2，如图8.4所示。更新的实现见算法8.5。

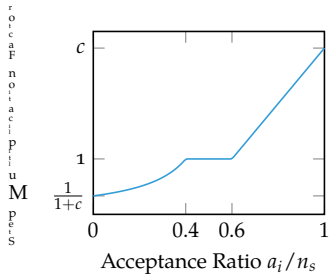


图8.4. $c = 2$的阶乘乘法因子与接受率的关系。

```
function corana_update!(v, a, c, ns)
    for i in 1 : length(v)
        ai, ci = a[i], c[i]
        if ai > 0.6ns
            v[i] *= (1 + ci*(ai/ns - 0.6)/0.4)
        elseif ai < 0.4ns
            v[i] /= (1 + ci*(0.4-ai/ns)/0.4)
        end
    end
    return v
end
```

算法8.5。Corana等人在自适应模拟退火中使用的更新公式，其中v是坐标步长向量，a是每个坐标方向上接受步数的向量，c是每个坐标方向的步长缩放因子向量，ns是运行步长调整前的周期数。

每进行 n_t 步调整，即每 $n_s \cdot n_t$ 个周期，执行温度降低。原始实现只是将温度乘以一个减少因子。

当温度降到足够低，以致无法期待进一步改进时，该过程终止。当最近一次的函数值与之前 n_ϵ 次迭代中的值以及执行过程中获得的最佳函数值相差不超过 ϵ 时，便发生终止。算法 8.6 提供了实现，该算法在图 8.5 中进行了可视化。

8.4 Cross-Entropy Method

cross-entropy method、[12]与我们已讨论的方法相比，在本章中，保持对设计空间的显式概率分布。[13] 这个概率分布，通常称为 *proposal distribution*，用于为下一次迭代提出新样本。在每次迭代中，我们从提议分布中抽样，然后更新提议分布以拟合一组最佳样本。收敛的目标是使提议分布聚焦于全局最优解。算法8.7提供了一种实现方式。

12 R. Y. Rubinstein 和 D. P. Kroese，*The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation, and Machine Learning*，Springer，2004。

13 这种方法的名称来源于拟合分布的过程涉及最小化 *cross-entropy*，这也被称为 *Kullback–Leibler divergence*。在某些条件下，最小化交叉熵对应于找到分布参数的最大似然估计。

交叉熵方法需要选择一族由θ参数化的分布。一个常见的选择是由均值向量和协方差矩阵参数化的多元正态分布族。该算法还要求我们指定在拟合下一次迭代的参数时要使用的*elite samples*、m_{elite}的数量。

根据分布族的选择，将分布拟合到精英样本的过程可以解析地完成。在多变量正态分布的情况下，参数依据最大似然估计进行更新：

$$\boldsymbol{\mu}^{(k+1)} = \frac{1}{m_{\text{elite}}} \sum_{i=1}^{m_{\text{elite}}} \mathbf{x}^{(i)} \tag{8.10}$$

$$\boldsymbol{\Sigma}^{(k+1)} = \frac{1}{m_{\text{elite}}} \sum_{i=1}^{m_{\text{elite}}} (\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(k+1)})(\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(k+1)})^\top \tag{8.11}$$

示例8.3将交叉熵方法应用于一个简单函数。图8.6展示了对一个更复杂函数的几次迭代。例8.4说明了使用多元正态分布拟合精英样本的潜在局限性。

```
function adaptive_simulated_annealing(f, x, v, t, ϵ;
    ns=20, nϵ=4, nt=max(100,5length(x)),
    γ=0.85, c=fill(2,length(x)) )

    y = f(x)
    x_best, y_best = x, y
    y_arr, n, U = [], length(x), Uniform(-1.0,1.0)
    a,counts_cycles,counts_resets = zeros(n), 0, 0

    while true
        for i in 1:n
            x′ = x + basis(i,n)*rand(U)*v[i]
            y′ = f(x′)
            Δy = y′ - y
            if Δy < 0 || rand() < exp(-Δy/t)
                x, y = x′, y′
                a[i] += 1
                if y′ < y_best; x_best, y_best = x′, y′; end
            end
        end

        counts_cycles += 1
        counts_cycles ≥ ns || continue

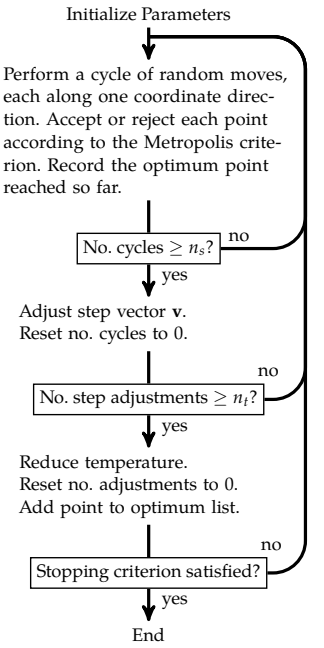
        counts_cycles = 0
        corana_update!(v, a, c, ns)
        fill!(a, 0)
        counts_resets += 1
        counts_resets ≥ nt || continue

        t *= γ
        counts_resets = 0
        push!(y_arr, y)

        if !(length(y_arr) > nϵ && y_arr[end] - y_best ≤ ϵ &&
             all(abs(y_arr[end]-y_arr[end-u]) ≤ ϵ for u in 1:nϵ))
            x, y = x_best, y_best
        else
            break
        end
    end
    return x_best
end
```

算法8.6. 自适应模拟退火算法，其中f是多变量目标函数，x是起始点，v是起始步长向量，t是起始温度，ϵ是终止准则参数。可选参数包括运行步长调整前的循环次数ns，降低温度前的循环次数nt，测试终止的连续降温次数nϵ，温度降低系数γ，以及方向变化准则c。

以下是原论文中提出的自适应模拟退火算法的流程图。



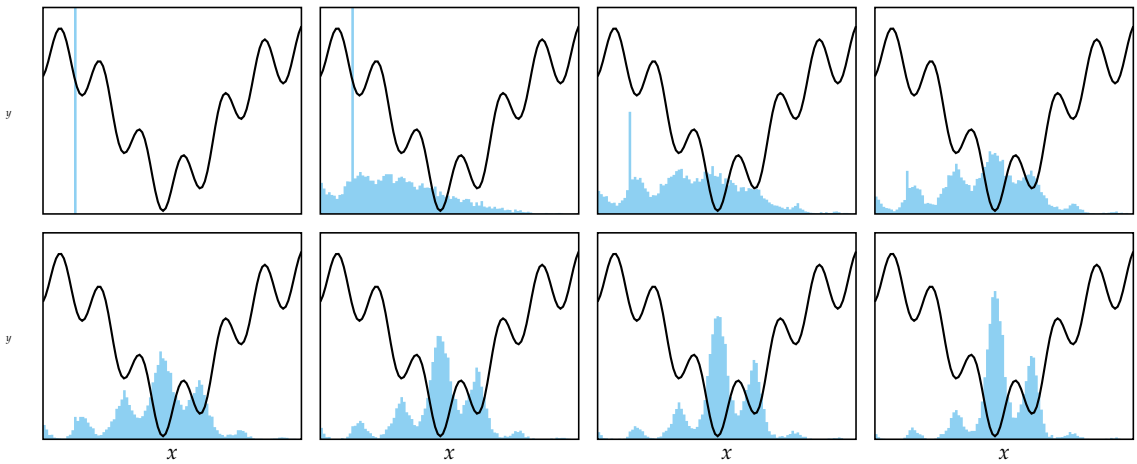


图8.5. 指数衰减温度下的模拟退火，其中直方图表示在该迭代中模拟退火处于特定位置的概率。

```
using Distributions
function cross_entropy_method(f, P, k_max, m=100, m_elite=10)
    for k in 1 : k_max
        samples = rand(P, m)
        order = sortperm([f(samples[:,i]) for i in 1:m])
        P = fit(typeof(P), samples[:,order[1:m_elite]])
    end
    return P
end
```

算法8.7. 交叉熵方法，其输入为一个待最小化的目标函数`f`，一个提议分布`P`，一个迭代次数`k_max`，一个样本大小`m`，以及重新拟合分布时要使用的样本数量`m_elite`。它返回更新后的分布，即全局最小值可能存在的分布。

我们可以使用`Distributions.jl`来表示、采样和拟合提议分布。参数向量θ被替换为分布`P`。调用`rand(P,m)`将生成一个$n \times m$矩阵，对应于来自`P`的m个n维样本，调用`fit`将拟合给定输入类型的新分布。

```
import Random: seed!
import LinearAlgebra: norm
seed!(0) # set random seed for reproducible results
f = x->norm(x)
μ = [0.5, 1.5]
Σ = [1.0 0.2; 0.2 2.0]
P = MvNormal(μ, Σ)
k_max = 10
P = cross_entropy_method(f, P, k_max)
@show P.μ

P.μ = [-6.13623e-7, -1.37216e-6]
```

示例 8.3. 使用交叉熵方法的例子。

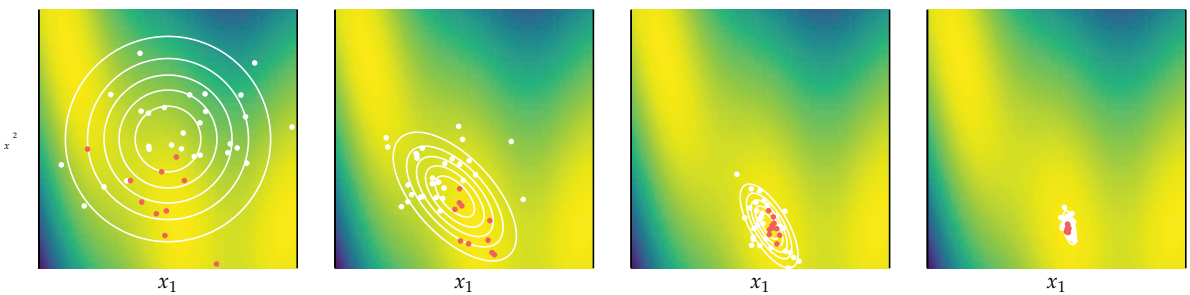
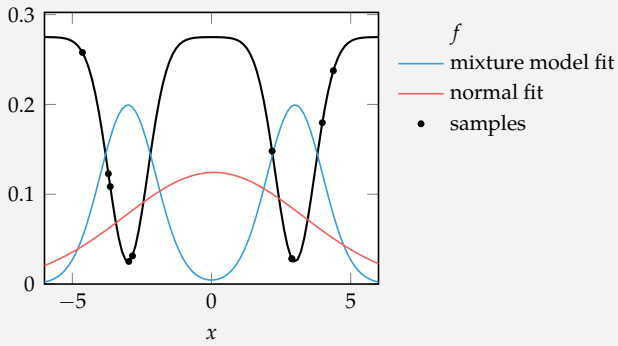


图 8.6. 应用带 $m = 40$ 的交叉熵方法于 Branin 函数（附录 B.3），使用多元高斯提议分布。每次迭代中的 10 个精英样本以红色表示。

分布族应足够灵活，以捕捉目标函数的相关特征。此处我们展示了在具有多模态的目标函数上使用正态分布的局限性，该分布会在两个最小值之间分配更高的密度。而混合模型则能够在每个最小值处自我定位。



例8.4. 正态分布无法捕捉多个局部极小值，这与能够维持多个局部极小值的混合模型形成对比。

8.5 Natural Evolution Strategies

与交叉熵方法类似，*natural evolution strategies*[14] 优化一个提议由 θ 参数化的分布。我们必须指定提议分布族和样本数量。目标是最小化 $\mathbb{E}_{\mathbf{x}\sim p(\cdot\mid\theta)}[f(\mathbf{x})]$。与拟合精英样本不同，进化策略应用梯度下降。梯度从样本中估计：[15]

$$\nabla_\theta \mathbb{E}_{\mathbf{x}\sim p(\cdot\mid\theta)}[f(\mathbf{x})] = \int \nabla_\theta p(\mathbf{x}\mid\theta) f(\mathbf{x})\, d\mathbf{x} \tag{8.12}$$

$$= \int \frac{p(\mathbf{x}\mid\theta)}{p(\mathbf{x}\mid\theta)} \nabla_\theta p(\mathbf{x}\mid\theta) f(\mathbf{x})\, d\mathbf{x} \tag{8.13}$$

$$= \int p(\mathbf{x}\mid\theta) \nabla_\theta \log p(\mathbf{x}\mid\theta) f(\mathbf{x})\, d\mathbf{x} \tag{8.14}$$

$$= \mathbb{E}_{\mathbf{x}\sim p(\cdot\mid\theta)}[f(\mathbf{x}) \nabla_\theta \log p(\mathbf{x}\mid\theta)] \tag{8.15}$$

$$\approx \frac{1}{m} \sum_{i=1}^{m} f(\mathbf{x}^{(i)}) \nabla_\theta \log p(\mathbf{x}^{(i)}\mid\theta) \tag{8.16}$$

虽然我们不需要目标函数的梯度，但确实需要对数似然的梯度，即 $\log p(\mathbf{x}\mid\theta)$。例8.5展示了如何计算

14 I. Rechenberg, *Evolutionsstrategie Optimierung technischer Systeme nach Prinzipien der biologischen Evolution*

15 这种梯度估计最近已成功应用于由深度神经网络表示的提议分布。T. Salimans、J. Ho、X. Chen 和 I. Sutskever，“进化策略作为强化学习的可扩展替代方案”，*ArXiv*，第 1703.03864 号，2017 年。

多元正态分布对数似然的梯度。估计的梯度可以与前面章节中讨论的任何下降方法一起使用，以改进 θ。算法8.8使用固定步长的梯度下降。图8.7展示了该算法的几次迭代。

```
using Distributions
function natural_evolution_strategies(f, θ, k_max; m=100, α=0.01)
    for k in 1 : k_max
        samples = [rand(θ) for i in 1 : m]
        θ -= α*sum(f(x)*∇logp(x, θ) for x in samples)/m
    end
    return θ
end
```

算法8.8。自然进化策略方法，其接收一个待最小化的目标函数 `f`、一个初始分布参数向量 `θ`、一个迭代次数 `k_max`、一个样本大小 `m` 以及一个步长因子 `α`。返回一个优化后的参数向量。方法 `rand(θ)` 应从由 `θ` 参数化的分布中采样，而 `∇logp(x, θ)` 应返回对数似然梯度。

8.6 Covariance Matrix Adaptation

另一种流行的方法是 *covariance matrix adaptation*、[16]，也被称为作为*CMA-ES*对于*covariance matrix adaptation evolutionary strategy*。它与第8.5节的自然进化策略有相似之处，但两者不应混淆。该方法维护一个协方差矩阵，具有鲁棒性和样本效率。与交叉熵方法和自然进化策略一样，分布会根据样本随时间改进。协方差矩阵自适应使用多元高斯分布。[17]

16 通常使用短语“进化策略”特指协方差矩阵自适应。

17 N. Hansen, “CMA进化策略：教程”，*ArXiv*，第1604.00772号，2016年。

协方差矩阵自适应维护一个均值向量 $\boldsymbol{\mu}$、一个协方差矩阵 $\boldsymbol{\Sigma}$ 以及一个额外的步长标量 σ。协方差矩阵在每次迭代中仅沿单一方向增大或减小，而步长标量则被调整以控制分布的整体扩散范围。在每次迭代中，从多元高斯分布中采样 m 个设计：[18]

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2\boldsymbol{\Sigma}) \tag{8.17}$$

18 对于 $\mathbb{R}^n$ 中的优化，建议每次迭代至少使用 $m = 4 + \lfloor 3 \ln n \rfloor$ 个样本，以及 $m_{\text{elite}} = \lfloor m/2 \rfloor$ 个精英样本。

然后根据目标函数值对设计进行排序，使得 $f(\mathbf{x}^{(1)}) \leq f(\mathbf{x}^{(2)}) \leq \cdots \leq f(\mathbf{x}^{(m)})$。使用采样设计的加权平均值形成新的均值向量 $\mu^{(k+1)}$：

$$\boldsymbol{\mu}^{(k+1)} \leftarrow \sum_{i=1}^{m} w_i \mathbf{x}^{(i)} \tag{8.18}$$

多元正态分布 $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$，其均值为 $\boldsymbol{\mu}$，协方差为 $\boldsymbol{\Sigma}$，因具有解析解而成为一类流行的分布族。在 d 维中的似然具有如下形式

$$p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = (2\pi)^{-\frac{d}{2}} |\boldsymbol{\Sigma}|^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

其中 $|\boldsymbol{\Sigma}|$ 是 $\boldsymbol{\Sigma}$ 的行列式。对数似然为

$$\log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\log|\boldsymbol{\Sigma}| - \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$$

可以使用对数似然梯度来更新参数：

$$\nabla_{(\boldsymbol{\mu})} \log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$$

$$\nabla_{(\boldsymbol{\Sigma})} \log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{2}\boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} - \frac{1}{2}\boldsymbol{\Sigma}^{-1}$$

术语$\nabla_{(\boldsymbol{\Sigma})}$包含$\boldsymbol{\Sigma}$的每个条目关于对数似然的偏导数。

直接更新 $\boldsymbol{\Sigma}$ 可能无法得到正定矩阵，而这是协方差矩阵所要求的。一种解决方案是将 $\boldsymbol{\Sigma}$ 表示为乘积 $\mathrm{A}^\top\mathrm{A}$，这保证了 $\boldsymbol{\Sigma}$ 保持半正定，然后改为更新 A。将 $\boldsymbol{\Sigma}$ 替换为 $\mathrm{A}^\top\mathrm{A}$ 并对 A 求梯度得到：

$$\nabla_{(\mathbf{A})} \log p(\mathbf{x} \mid \boldsymbol{\mu}, \mathbf{A}) = \mathbf{A}\left[\nabla_{(\boldsymbol{\Sigma})} \log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) + \nabla_{(\boldsymbol{\Sigma})} \log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})^\top\right]$$

例 8.5。多元高斯分布的对数似然梯度方程的推导。关于原始推导以及若干处理正定协方差矩阵的更精细解法，参见 D. Wierstra、T. Schaul、T. Glasmachers、Y. Sun 和 J. Schmidhuber，“Natural Evolution Strategies，” *ArXiv*, no. 1106.4487, 2011。

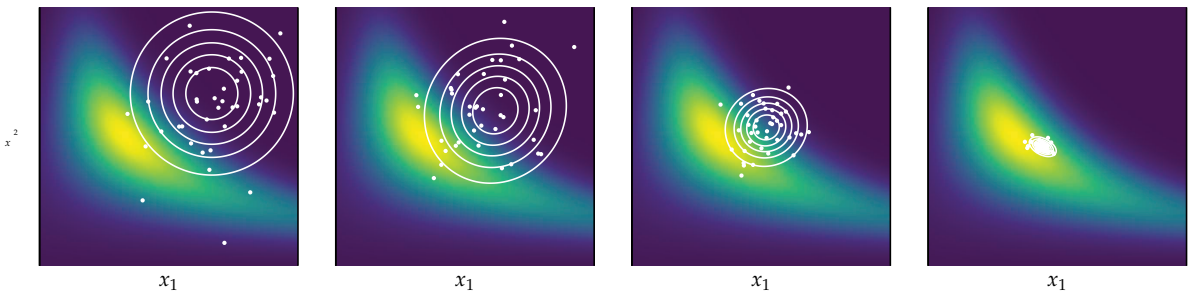


图8.7。应用于Wheeler山脊的多变量高斯分布自然进化策略，附录B.7。

其中权重之和为1，按从大到小排序，并且均为非负：[19]

$$\sum_{i=1}^{m} w_i = 1 \qquad w_1 \geq w_2 \geq \cdots \geq w_m \geq 0 \tag{8.19}$$

我们可以通过将前 m_{elite} 个权重设为 $1/m_{\text{elite}}$，并将其余权重设为零，来在交叉熵方法中恢复均值更新。协方差矩阵自适应也只为前 m_{elite} 个设计分配权重，但会不均匀地分配这些权重。推荐的权重通过归一化获得

$$w_i' = \ln \frac{m+1}{2} - \ln i \text{ for } i \in \{1, \ldots, m\} \tag{8.20}$$

以获得 $\mathbf{w} = \mathbf{w}'/\sum_i w_i'$。图 8.8 比较了协方差矩阵自适应和交叉熵方法的平均更新。

步长通过一个累积变量$\mathbf{p}_\sigma$进行更新，该变量随时间跟踪步数：

$$\begin{aligned} \mathbf{p}_\sigma^{(1)} &= \mathbf{0} \\ \mathbf{p}_\sigma^{(k+1)} &\leftarrow (1 - c_\sigma)\mathbf{p}_\sigma + \sqrt{c_\sigma(2 - c_\sigma)\mu_{\text{eff}}}(\boldsymbol{\Sigma}^{(k)})^{-1/2}\boldsymbol{\delta}_w \end{aligned} \tag{8.21}$$

其中 $c_\sigma < 1$ 控制衰减速率，右侧项根据观察到的样本相对于当前分布尺度决定步长应增加还是减少。方差有效选择质量 μ_{eff} 的形式为

$$\mu_{\text{eff}} = \frac{1}{\sum_i w_i^2} \tag{8.22}$$

并且$\boldsymbol{\delta}_w$是根据采样偏差计算得出的：

$$\boldsymbol{\delta}_w = \sum_{i=1}^{m_{\text{elite}}} w_i \boldsymbol{\delta}^{(i)} \text{ for } \delta^{(i)} = \frac{\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(k)}}{\sigma^{(k)}} \tag{8.23}$$

根据以下公式获得新的步长

$$\sigma^{(k+1)} \leftarrow \sigma^{(k)} \exp\left(\frac{c_\sigma}{d_\sigma}\left(\frac{\|\mathbf{p}_\sigma\|}{\mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|} - 1\right)\right) \tag{8.24}$$

其中

$$\mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\| = \sqrt{2}\frac{\Gamma\left(\frac{n+1}{2}\right)}{\Gamma\left(\frac{n}{2}\right)} \approx \sqrt{n}\left(1 - \frac{1}{4n} + \frac{1}{21n^2}\right) \tag{8.25}$$

19 原始论文（N. Hansen 和 A. Ostermeier，“在进化策略中自适应任意正态变异分布：协方差矩阵自适应”，载于 *IEEE International Conference on Evolutionary Computation*，1996）提供了无需遵守这些约束的泛化，但他们推荐的实现确实遵守了这些约束。算法8.9遵守了这些约束。

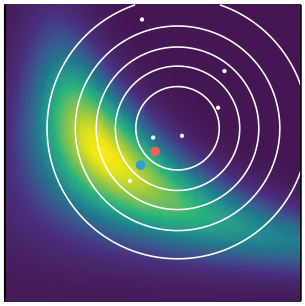


图8.8. 显示了初始提议分布（白色轮廓）、六个样本（白点），以及使用三个精英样本时协方差矩阵自适应（蓝点）和交叉熵方法（红点）各自更新后的均值。协方差矩阵自适应倾向于比交叉熵方法（红点）更积极地更新均值，因为它对更好的采样设计赋予更高的权重。

是从高斯分布中抽取的向量的期望长度。将p_σ的长度与其在随机选择下的期望长度进行比较，提供了σ增加或减少的机制。常数c_σ和d_σ推荐值如下：

$$\begin{aligned} c_\sigma &= (\mu_{\text{eff}} + 2)/(n + \mu_{\text{eff}} + 5) \\ d_\sigma &= 1 + 2\max\left(0, \sqrt{(\mu_{\text{eff}} - 1)/(n+1)} - 1\right) + c_\sigma \end{aligned} \tag{8.26}$$

协方差矩阵也使用累积向量进行更新：

$$\begin{aligned} \mathbf{p}_\Sigma^{(1)} &= \mathbf{0} \\ \mathbf{p}_\Sigma^{(k+1)} &\leftarrow (1 - c_\Sigma)\mathbf{p}_\Sigma^{(k)} + h_\sigma \sqrt{c_\Sigma(2 - c_\Sigma)\mu_{\text{eff}}}\boldsymbol{\delta}_w \end{aligned} \tag{8.27}$$

其中

$$h_\sigma = \begin{cases} 1 & \text{if } \frac{\|\mathbf{p}_\Sigma\|}{\sqrt{1-(1-c_\sigma)^{2(k+1)}}} < \left(1.4 + \frac{2}{n+1}\right) \mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\| \\ 0 & \text{otherwise} \end{cases} \tag{8.28}$$

当$\|\mathrm{p}_\Sigma\|$过大时，h_σ会阻止p_Σ的更新，从而在步长过小时防止$\boldsymbol{\Sigma}$的过度增加。

更新需要调整后的权重 w°：

$$w_i^\circ = \begin{cases} w_i & \text{if } w_i \geq 0 \\ \frac{n w_i}{\left\|\Sigma^{-1/2}\boldsymbol{\delta}^{(i)}\right\|^2} & \text{otherwise} \end{cases} \tag{8.29}$$

然后协方差更新为

$$\boldsymbol{\Sigma}^{(k+1)} \leftarrow \left(1 + \underbrace{c_1 c_c (1 - h_\sigma)(2 - c_c) - c_1 - c_\mu}_{\text{typically zero}}\right)\boldsymbol{\Sigma}^{(k)} + \underbrace{c_1 \mathbf{p}_\Sigma \mathbf{p}_\Sigma^\top}_{\text{rank-one update}} + \underbrace{c_\mu \sum_{i=1}^{\mu} w_i^\circ \boldsymbol{\delta}^{(i)} \left(\boldsymbol{\delta}^{(i)}\right)^\top}_{\text{rank-}\mu\text{ update}} \tag{8.30}$$

常数 c_Σ、c_1 和 c_μ 具有推荐值

$$\begin{aligned} c_\Sigma &= \frac{4 + \mu_{\text{eff}}/n}{n + 4 + 2\mu_{\text{eff}}/n} \\ c_1 &= \frac{2}{(n + 1.3)^2 + \mu_{\text{eff}}} \\ c_\mu &= \min\left(1 - c_1, 2\frac{\mu_{\text{eff}} - 2 + 1/\mu_{\text{eff}}}{(n+2)^2 + \mu_{\text{eff}}}\right) \end{aligned} \tag{8.31}$$

协方差更新由三个部分组成：先前的协方差矩阵 $\boldsymbol{\Sigma}^{(k)}$、一次秩一更新以及一次秩 μ 更新。秩一更新之所以得名，是因为 $\mathrm{p}_\Sigma \mathrm{p}_\Sigma^\top$ 的秩为一；它沿着 p_Σ 只有一个特征向量。使用累积向量的秩一更新能够利用连续步骤之间的相关性，从而使协方差矩阵能够沿有利轴更快地伸长。

秩-μ更新的名称来源于$\sum_{i=1}^{\mu} w_i^\circ \boldsymbol{\delta}^{(i)} \left(\boldsymbol{\delta}^{(i)}\right)^\top$的秩为$\min(\mu, n)$。交叉熵方法采用的经验协方差矩阵更新与秩-$\mu$更新之间的一个重要区别在于，前者估计的是围绕新均值$\boldsymbol{\mu}^{(k+1)}$的协方差，而后者估计的是围绕原始均值$\boldsymbol{\mu}^{(k)}$的协方差。因此，$\boldsymbol{\delta}^{(i)}$值有助于估计采样步长的方差，而非采样设计内的方差。

协方差矩阵自适应如图8.9所示。

8.7 Summary

- 随机方法在优化过程中使用随机数。
- 模拟退火使用温度来控制随机探索，温度随时间降低以收敛到局部最小值。
- 交叉熵方法和进化策略维护提议分布，从中采样以指导更新。
- 自然进化策略使用关于对数似然的梯度下降来更新其提议分布。
- 协方差矩阵自适应是一种稳健且样本高效的优化器，它维护具有完整协方差矩阵的多元高斯提议分布。

8.8 Exercises

练习 8.1。我们已经展示了混合提议分布能更好地捕捉多个最小值。为什么它们在交叉熵方法中的应用可能受到限制?

```
function covariance_matrix_adaptation(f, x, k_max;
    σ = 1.0,
    m = 4 + floor(Int, 3*log(length(x))),
    m_elite = div(m,2))

    μ, n = copy(x), length(x)
    ws = normalize!(vcat(log((m+1)/2) .- log.(1:m_elite),
                    zeros(m - m_elite)), 1)
    μ_eff = 1 / sum(ws.^2)
    cσ = (μ_eff + 2)/(n + μ_eff + 5)
    dσ = 1 + 2max(0, sqrt((μ_eff-1)/(n+1))-1) + cσ
    cΣ = (4 + μ_eff/n)/(n + 4 + 2μ_eff/n)
    c1 = 2/((n+1.3)^2 + μ_eff)
    cμ = min(1-c1, 2*(μ_eff-2+1/μ_eff)/((n+2)^2 + μ_eff))
    E = n^0.5*(1-1/(4n)+1/(21*n^2))
    pσ, pΣ, Σ = zeros(n), zeros(n), Matrix(1.0I, n, n)
    for k in 1 : k_max
        P = MvNormal(μ, σ^2*Σ)
        xs = [rand(P) for i in 1 : m]
        ys = [f(x) for x in xs]
        is = sortperm(ys) # best to worst

        # selection and mean update
        δs = [(x - μ)/σ for x in xs]
        δw = sum(ws[i]*δs[is[i]] for i in 1 : m_elite)
        μ += σ*δw

        # step-size control
        C = Σ^-0.5
        pσ = (1-cσ)*pσ + sqrt(cσ*(2-cσ)*μ_eff)*C*δw
        σ *= exp(cσ/dσ * (norm(pσ)/E - 1))

        # covariance adaptation
        hσ = Int(norm(pσ)/sqrt(1-(1-cσ)^(2k)) < (1.4+2/(n+1))*E)
        pΣ = (1-cΣ)*pΣ + hσ*sqrt(cΣ*(2-cΣ)*μ_eff)*δw
        w0 = [ws[i]≥0 ? ws[i] : n*ws[i]/norm(C*δs[is[i]])^2
              for i in 1:m]
        Σ = (1-c1-cμ) * Σ +
            c1*(pΣ*pΣ' + (1-hσ) * cΣ*(2-cΣ) * Σ) +
            cμ*sum(w0[i]*δs[is[i]]*δs[is[i]]' for i in 1 : m)
        Σ = triu(Σ)+triu(Σ,1)' # enforce symmetry
    end
    return μ
end
```

算法8.9. 协方差矩阵适应，它接受一个待最小化的目标函数`f`、一个初始设计点`x`和一个迭代次数`k_max`。可以选择性地指定步长标量`σ`、样本大小`m`和精英样本数量`m_elite`。

返回最佳候选设计点，即最终样本分布的均值。

协方差矩阵会经过额外操作以确保其保持对称；否则，微小的数值不一致可能导致矩阵不再为正定矩阵。

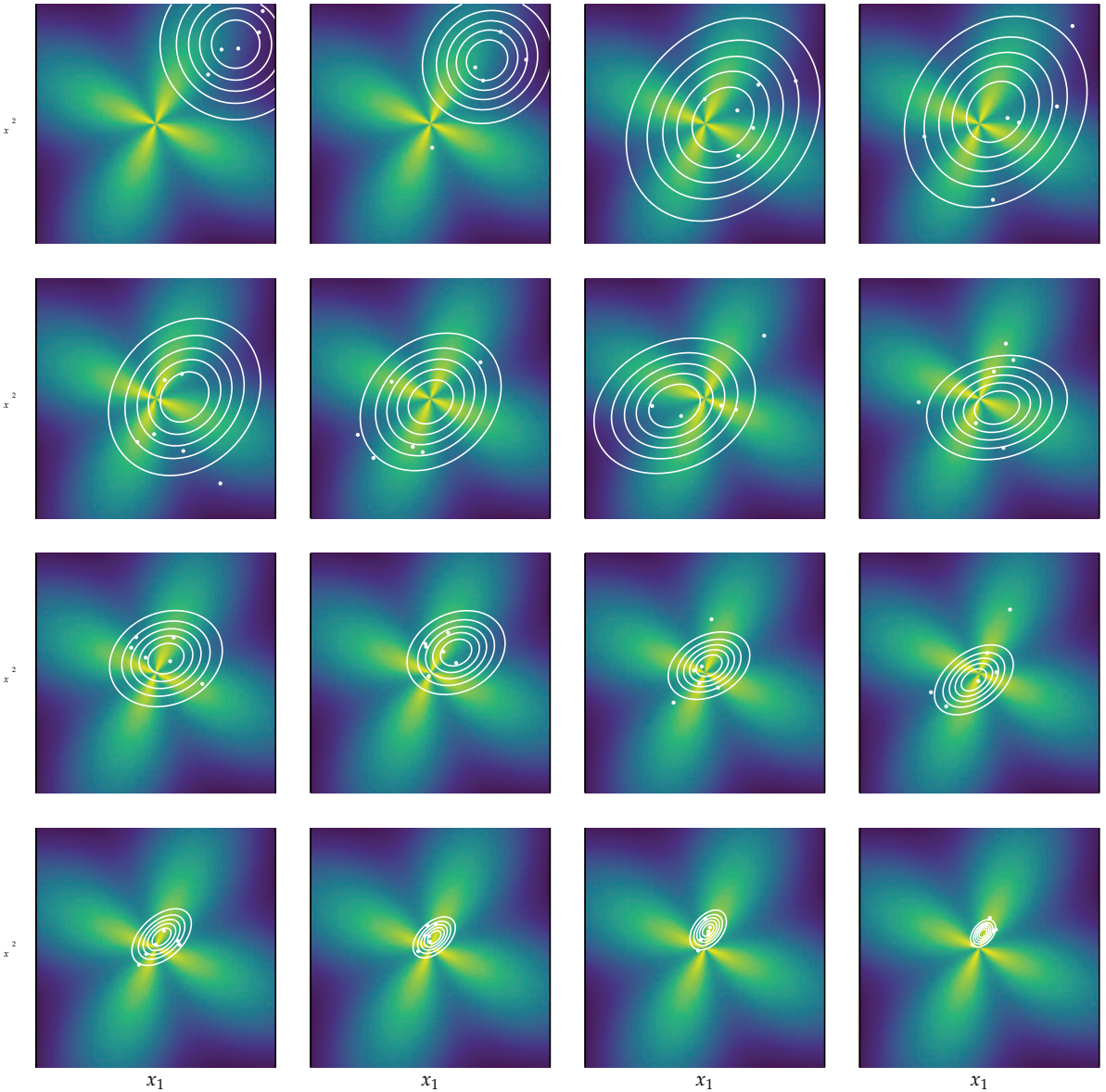


图8.9. 使用多元高斯分布对花函数进行协方差矩阵适应，附录B.4。

练习 8.2。在交叉熵方法中，使用接近总样本量的精英样本量可能产生什么潜在影响?

练习 8.3。从均值为 μ、方差为 ν 的高斯分布中采样得到的值的对数似然为:

$$\ell(x \mid \mu, \nu) = -\frac{1}{2}\ln 2\pi - \frac{1}{2}\ln \nu - \frac{(x-\mu)^2}{2\nu}$$

展示为什么使用高斯分布的进化策略在均值处于最优值 $\mu = x^*$ 时，对 $\cdot$ 方差 $\cdot$ 进行下降更新可能会遇到困难。

练习 8.4. 推导使用多元正态分布的交叉熵方法的最大似然估计:

$$\boldsymbol{\mu}^{(k+1)} = \frac{1}{m}\sum_{i=1}^{m}\mathbf{x}^{(i)}$$
$$\boldsymbol{\Sigma}^{(k+1)} = \frac{1}{m}\sum_{i=1}^{m}(\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(k+1)})(\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(k+1)})^\top$$

其中最大似然估计是使抽样个体 $\left\{\mathrm{x}(1)、\cdots、\mathrm{x}(m)\right\}$ 的似然最大化的参数值。

9 Population Methods

前几章重点讨论了将单一设计点逐步移向最小值的方法。本章介绍多种 *population methods*，这些方法利用一组设计点（称为*indi- viduals*）进行优化。在设计空间中分布大量个体有助于算法避免陷入局部最小值。个体之间可以共享设计空间中不同点的信息，以全局优化目标函数。大多数种群方法本质上是随机的，并且通常易于并行计算。

9.1 Initialization

群体方法从 *initial population* 开始，正如下降法需要初始设计点一样。初始群体应分布在整个设计空间中，以增加样本接近最佳区域的机会。本节介绍几种初始化方法，但更高级的采样方法将在第13章中详细讨论。

我们通常可以将设计变量约束到一个感兴趣的区域，该区域由下界和上界a和b定义的超矩形组成。初始种群可以从每个坐标的均匀分布中采样：[1]

$$\mathbf{x}_i^{(j)} \sim U(a_i, b_i) \tag{9.1}$$

其中 x(j) 是算法9.1中所见种群中的第 j 个个体。

另一种常见方法是在感兴趣区域上使用多元正态分布。协方差矩阵通常是对角矩阵，其对角元素经过缩放以充分覆盖搜索空间。算法9.2给出了实现。

1 一些种群方法需要与个体相关联的额外信息，例如粒子群优化中的速度，稍后讨论。速度通常根据均匀分布或正态分布初始化。

```
function rand_population_uniform(m, a, b)
    d = length(a)
    return [a+rand(d).*(b-a) for i in 1:m]
end
```

算法9.1. 一种在具有下界向量a和上界向量b的均匀超矩形上采样m个设计点的初始种群的方法。

```
using Distributions
function rand_population_normal(m, μ, Σ)
    D = MvNormal(μ,Σ)
    return [rand(D) for i in 1:m]
end
```

算法9.2. 一种使用均值为μ、协方差为Σ的多变量正态分布抽样m个设计点作为初始种群的方法。

均匀分布和正态分布将覆盖的设计空间限制在一个集中的区域。*Cauchy distribution* (图9.1)具有无界方差，可以覆盖更广阔的空间。算法9.3提供了一种实现。下一页的图9.2比较了使用不同方法生成的示例初始种群。

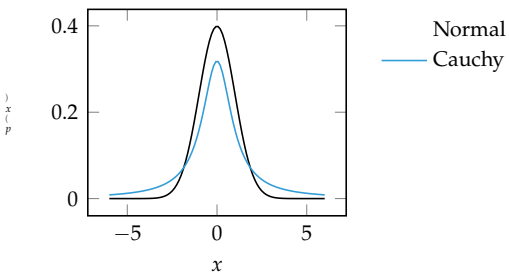


图9.1. 标准差为1的正态分布与尺度参数为1的柯西分布的比较。尽管有时用σ表示柯西分布的尺度参数，但这不应与标准差混淆，因为柯西分布的标准差是未定义的。柯西分布具有重尾特性，使其能够更广泛地覆盖设计空间。

9.2 *Genetic Algorithms*

Genetic algorithms (算法9.4)从生物进化中汲取灵感，适者更有可能将其基因传递给下一代。[2]个体繁殖的适应度与

该点处的目标函数。与个体相关的设计点表示为*chromosome*。在每一代中，较适应个体的染色体经过*crossover*和*mutation*的遗传操作后传递给下一代。

2 D. E. Goldberg, *Genetic Algorithms*. Addison-Wesley, *in Search, Optimization, and Machine Learning*

```
using Distributions
function rand_population_cauchy(m, μ, σ)
    n = length(μ)
    return [[rand(Cauchy(μ[j],σ[j])) for j in 1:n] for i in 1:m]
end
```

算法9.3. 一种使用柯西分布对初始种群进行抽样的方法，该分布对于每个维度具有位置参数 μ 和尺度参数 σ，用于生成 m 个设计点。位置参数和尺度参数类似于正态分布中使用的均值和标准差。

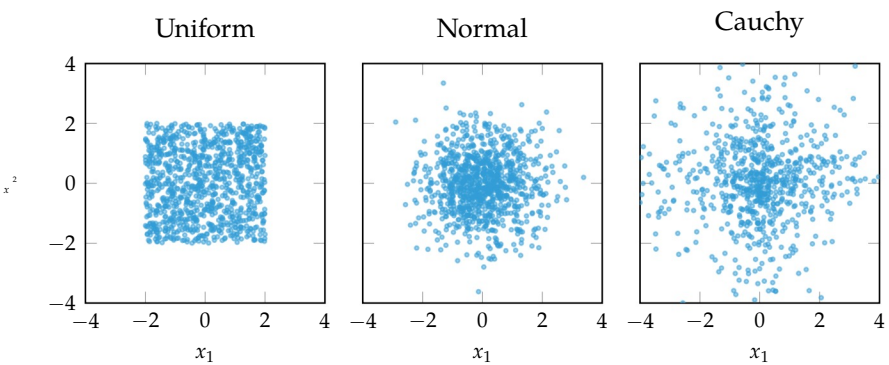


图9.2。使用均匀超矩形采样的初始种群大小为1,000，其中$= [-2, -2]$，$b = [2, 2]$，对角协方差为$\Sigma = I$的零均值正态分布，以及以原点为中心、尺度为$\sigma = 1$的柯西分布。

```
function genetic_algorithm(f, population, k_max, S, C, M)
    for k in 1 : k_max
        parents = select(S, f.(population))
        children = [crossover(C,population[p[1]],population[p[2]])
                    for p in parents]
        population .= mutate.(Ref(M), children)
    end
    population[argmin(f.(population))]
end
```

算法9.4。遗传算法，接受一个目标函数`f`、一个初始种群、迭代次数`k_max`、一个`SelectionMethod S`、一个`CrossoverMethod C`以及一个`MutationMethod M`。

9.2.1 *Chromosomes*

有几种表示染色体的方法。最简单的是*binarystring chromosome*，这种表示方式类似于DNA编码的方式。[3] A

可以使用`bitrand(d)`生成长度为d的随机二进制字符串。二进制字符串染色体如图9.3所示。

3 DNA并非二进制表示，而是包含四种核碱基：腺嘌呤、胸腺嘧啶、胞嘧啶和鸟嘌呤，通常缩写为A、T、C和G。



图9.3. 用二进制字符串表示的染色体。

二进制字符串常因便于表达交叉和变异而被使用。遗憾的是，解码二进制字符串并生成设计点的过程并不总是直截了当的。有时，二进制字符串可能并不代表设计空间中的有效点。通常，使用实数值列表来表示染色体更为自然。这样的*real-valued chromosomes*是$\mathbb{R}^d$中的向量，直接对应于设计空间中的点。

9.2.2 *Initialization*

遗传算法从一个随机的初始种群开始。二进制字符串染色体通常使用随机位字符串进行初始化，如算法9.5所示。实值染色体通常使用上一节中的方法进行初始化。

```
rand_population_binary(m, n) = [bitrand(n) for i in 1:m]
```

算法9.5. 一种采样长度为`n`的`m`位串染色体随机起始种群的方法。

9.2.3 *Selection*

选择是选择染色体作为下一代亲本的过程。对于具有 m 条染色体的种群，选择方法将为下一代的 m 个子代产生一个包含 m 个亲本对[4]的列表。该

选定的对可能包含重复项。

4 在某些情况下，如果希望结合两个以上的父代来生成一个子代，可以使用分组。

有几种方法可以使选择偏向最适应者（算法9.6）。在*truncation selection* (图9.4)中，我们从种群中最好的k条染色体中选择父母。在*tournament selection* (图9.5)中，每个父母是种群中随机选取的k条染色体中最适应的。在*roulette wheel selection* (图9.6)中，也称为*fitness proportionate selection*，每个父母的选择概率与其相对于种群的表现成正比。由于我们关注的是最小化目标函数f，第i个个体的适应度$x^{(i)}$与$y^{(i)} = f(x^{(i)})$成反比。有多种方法可以将集合$y^{(1)}$，...，$y^{(m)}$转换为适应度。一种简单的方法是按照$\max\{y^{(1)}, \ldots, y^{(m)}\} - y^{(i)}$来分配个体$i$的适应度。

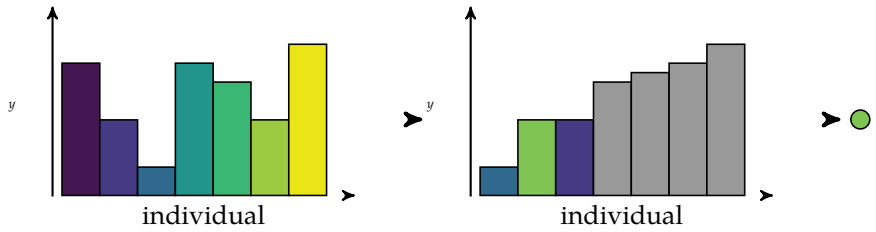


图9.4. 种群大小为$m = 7$、样本大小为$k = 3$的截断选择。条形的高度表示其目标函数值，而颜色表示它所对应的个体。

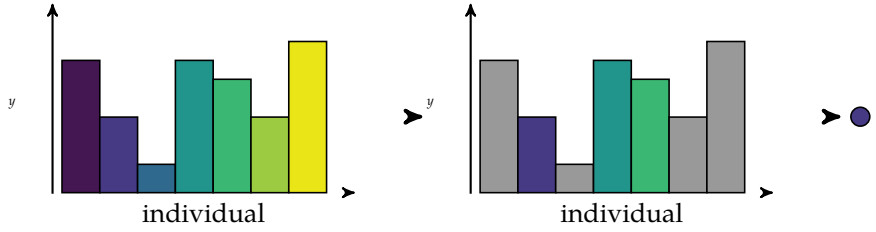


图 9.5. 锦标赛选择，种群大小 $m = 7$，样本大小 $k = 3$，分别为每个父代独立运行。条形的高度表示其目标函数值，而颜色表示它对应哪个个体。

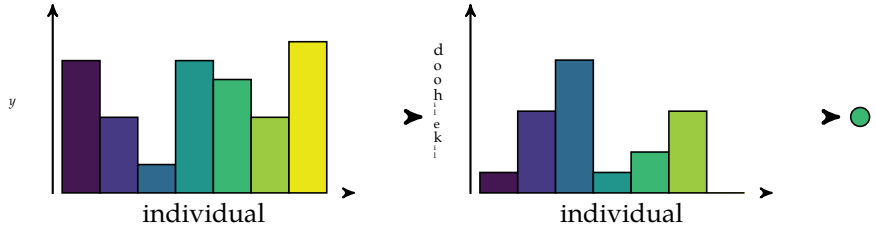


图 9.6. 种群规模为 $m = 7$ 的轮盘赌选择，对每个父代单独运行。所采用的方法导致目标函数值最差的个体被选中的可能性为零。条形的高度表示其目标函数值（左）或其可能性（右），而颜色表示其对应的个体。

```
abstract type SelectionMethod end

struct TruncationSelection <: SelectionMethod
    k # top k to keep
end
function select(t::TruncationSelection, y)
    p = sortperm(y)
    return [p[rand(1:t.k, 2)] for i in y]
end

struct TournamentSelection <: SelectionMethod
    k
end
function select(t::TournamentSelection, y)
    getparent() = begin
        p = randperm(length(y))
        p[argmin(y[p[1:t.k]])]
    end
    return [[getparent(), getparent()] for i in y]
end

struct RouletteWheelSelection <: SelectionMethod end
function select(::RouletteWheelSelection, y)
    y = maximum(y) .- y
    cat = Categorical(normalize(y, 1))
    return [rand(cat, 2) for i in y]
end
```

算法9.6. 遗传算法的几种选择方法。使用`SelectionMethod`和目标函数值列表`f`调用选择将生成一对父母列表。

9.2.4 *Crossover*

交叉操作将父母的染色体结合起来形成后代。与选择操作一样，存在多种交叉方案（算法9.7）。

- 在*single-point crossover* (图9.7)中，父本A染色体的一部分构成了子代染色体的前段，而父本B染色体的后一部分则构成了子代染色体的后段。发生转换的*crossoverpoint*是均匀随机决定的。

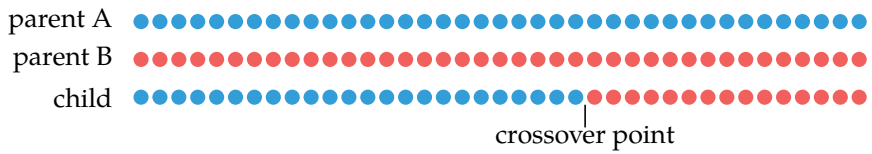


图9.7. 单点交叉。

- 在*two-point crossover* (图9.8)中，我们使用两个随机交叉点。

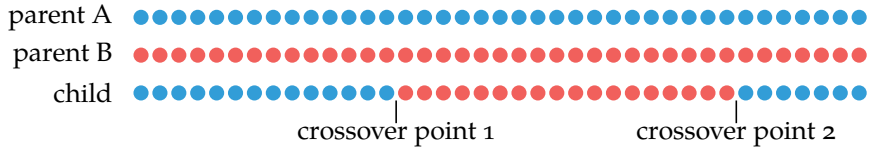


图9.8. 两点交叉。

- 在*uniform crossover* (图9.9)中，每个位有50%的概率来自两个亲本中的任意一个。这个方案等同于每个点有50%的概率成为交叉点。



图 9.9. 均匀交叉。

先前的交叉方法也适用于实值染色体。然而，我们可以额外定义一个在实数值之间进行插值的交叉例程（算法9.8）。在这里，实数值在父代值 x_a 和 x_b 之间进行线性插值：

$$\mathbf{x} \leftarrow (1-\lambda)\mathbf{x}_a + \lambda\mathbf{x}_b \tag{9.2}$$

其中λ是通常设置为二分之一的标量参数。

```
abstract type CrossoverMethod end
struct SinglePointCrossover <: CrossoverMethod end
function crossover(::SinglePointCrossover, a, b)
    i = rand(1:length(a))
    return vcat(a[1:i], b[i+1:end])
end

struct TwoPointCrossover <: CrossoverMethod end
function crossover(::TwoPointCrossover, a, b)
    n = length(a)
    i, j = rand(1:n, 2)
    if i > j
        (i,j) = (j,i)
    end
    return vcat(a[1:i], b[i+1:j], a[j+1:n])
end

struct UniformCrossover <: CrossoverMethod end
function crossover(::UniformCrossover, a, b)
    child = copy(a)
    for i in 1 : length(a)
        if rand() < 0.5
            child[i] = b[i]
        end
    end
    return child
end
```

算法9.7. 遗传算法的几种交叉方法。使用CrossoverMethod以及两个父代a和b调用交叉操作，将产生一个子代染色体，该染色体包含父代遗传代码的混合。这些方法适用于二进制字符串和实值染色体。

```
struct InterpolationCrossover <: CrossoverMethod
    λ
end
crossover(C::InterpolationCrossover, a, b) = (1-C.λ)*a + C.λ*b
```

算法9.8. 一种实值染色体的交叉方法，该方法在父代之间执行线性插值。

9.2.5 *Mutation*

如果新染色体仅通过交叉产生，那么初始随机种群中不存在的许多性状将永远无法出现，而最适应的基因可能会使种群饱和。突变允许新性状自发出现，使遗传算法能够探索更大的状态空间。子染色体在交叉后进行突变。

二进制值染色体中的每个位通常都有很小的翻转概率（图9.10）。对于具有m位的染色体，这个*mutation rate*通常设置为$1/m$，平均每个子染色体发生一次突变。实值染色体的突变可以通过按位翻转来实现，但更常见的是添加零均值高斯噪声。算法9.9提供了实现。



图9.10. 二进制串染色体的突变会给每个位赋予一个小的翻转概率。

```
abstract type MutationMethod end
struct BitwiseMutation <: MutationMethod
    λ
end
function mutate(M::BitwiseMutation, child)
    return [rand() < M.λ ? !v : v for v in child]
end

struct GaussianMutation <: MutationMethod
    σ
end
function mutate(M::GaussianMutation, child)
    return child + randn(length(child))*M.σ
end
```

算法9.9. 二进制串染色体的位变异方法及实值染色体的高斯变异方法。其中，λ 是变异率，而 σ 是标准差。

图9.11展示了遗传算法的几代演化过程。例9.1展示了如何结合本节讨论的选择、交叉和突变策略。

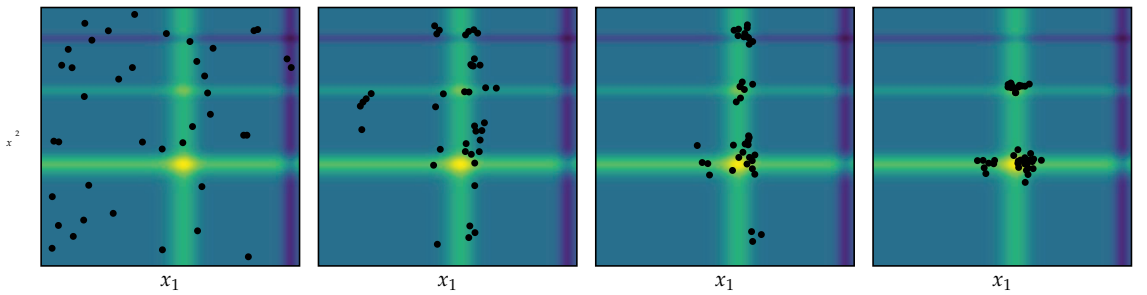


图9.11. 一种采用截断选择、单点交叉和高斯变异（其中 $\sigma = 0.1$）的遗传算法，应用于附录B.5中定义的Michalewicz函数。

我们将演示使用遗传算法来优化一个简单的函数。

```
import Random: seed!
import LinearAlgebra: norm
seed!(0) # set random seed for reproducible results
f = x->norm(x)
m = 100 # population size
k_max = 10 # number of iterations
population = rand_population_uniform(m, [-3, 3], [3,3])
S = TruncationSelection(10) # select top 10
C = SinglePointCrossover()
M = GaussianMutation(0.5) # small mutation rate
x = genetic_algorithm(f, population, k_max, S, C, M)
@show x
```

```
x = [0.0367471, -0.090237]
```

例 9.1. 展示使用遗传算法优化一个简单函数。

9.3 *Differential Evolution*

Differential evolution (算法 9.10) 试图通过根据一个简单公式重组种群中的其他个体来改进种群中的每个个体。[5] 它由一个交叉概率 p 和一个差分权重 w。通常，w 介于 0.4 和 1 之间。对于每个个体 x:

5 S. Das 和 P. N. Suganthan，“差分进化：最新技术综述”，*IEEE Transactions on Evolutionary Computation*，第15卷，第1期，第4-31页，2011年。

1. 选择三个随机不同的个体a、b和c。
2. 构建一个临时设计 $\mathrm{z} = \mathrm{a} + w \cdot (\mathrm{b} - \mathrm{c})$，如图9.12所示。
3. 在n维中，选择一个用于优化的随机维度$j \in [1, \ldots, n]$。
4. 使用二进制交叉构建候选个体 x′。

$$x_i' = \begin{cases} z_i & \text{if } i = j \text{ or with probability } p \\ x_i & \text{otherwise} \end{cases} \tag{9.3}$$

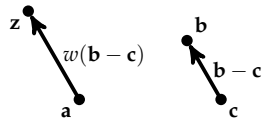


图9.12. 差分进化取三个个体a、b和c，并将它们组合成候选个体z。

5. 在下一次生成中，将更优设计插入到x和x′之间。

该算法在图9.13中演示。

```
using StatsBase
function differential_evolution(f, population, k_max; p=0.5, w=1)
    n, m = length(population[1]), length(population)
    for k in 1 : k_max
        for (k,x) in enumerate(population)
            a, b, c = sample(population,
                Weights([j!=k for j in 1:m]), 3, replace=false)
            z = a + w*(b-c)
            j = rand(1:n)
            x′ = [i == j || rand() < p ? z[i] : x[i] for i in 1:n]
            if f(x′) < f(x)
                x[:] = x′
            end
        end
    end
    return population[argmin(f.(population))]
end
```

算法9.10。差分进化，它接受一个目标函数`f`、一个种群`population`、迭代次数`k_max`、交叉概率`p`和差分权重`w`。返回最佳个体。

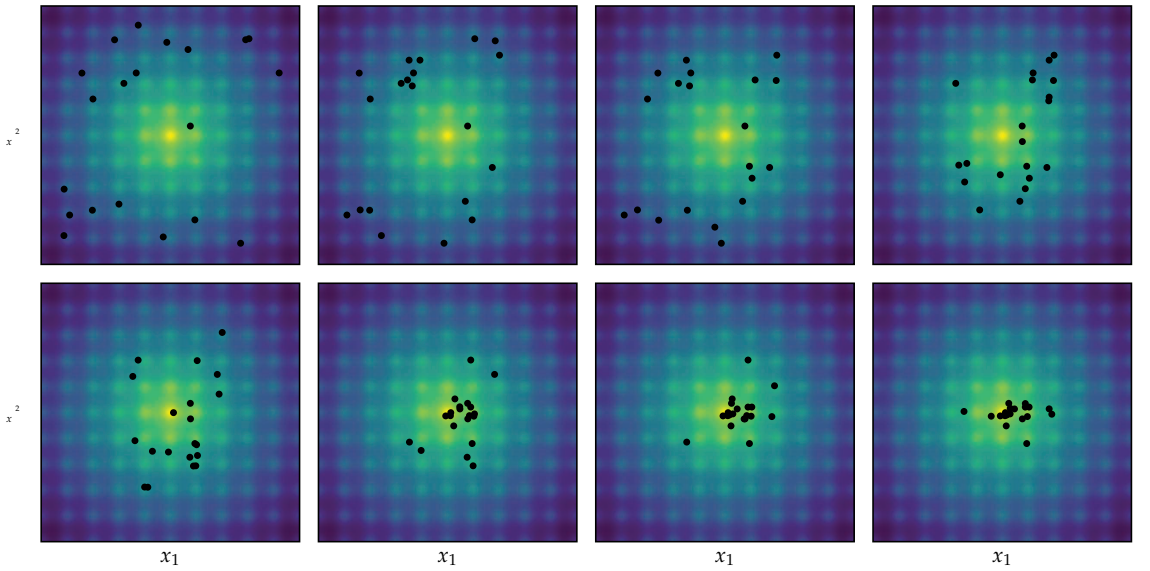


图9.13. 使用$p = 0.5$和$w = 0.2$的差分进化算法应用于附录B.1中定义的Ackley函数。

9.4 Particle Swarm Optimization

Particle swarm optimization 引入动量以加速向极小值收敛。[6] 种群中的每个个体，或 *particle*，会跟踪其当前位置、速度以及迄今为止它所见过的最佳位置 (算法 9.11)。动量允许个体在有利方向上积累速度，不受局部扰动的影响。

6 J. Kennedy, R. C. Eberhart, 和 Y. Shi, *Swarm Intelligence*. Morgan Kaufmann, 2001.

```
mutable struct Particle
    x
    v
    x_best
end
```

算法9.11。在粒子群优化中，每个粒子在设计空间中都有一个位置 `x` 和速度 `v`，并追踪迄今为止找到的最佳设计点 `x_best`。

在每次迭代中，每个个体都会向其曾经见过的最佳位置以及迄今为止任何个体找到的最佳位置加速。加速受到随机项的加权，其中会生成独立的随机数

对于每个加速度。更新方程为：

$$x^{(i)} \leftarrow x^{(i)} + v^{(i)} \tag{9.4}$$

$$v^{(i)} \leftarrow wv^{(i)} + c_1 r_1 \left(x_{\text{best}}^{(i)} - x^{(i)}\right) + c_2 r_2 \left(x_{\text{best}} - x^{(i)}\right) \tag{9.5}$$

其中 x_{best} 是迄今为止在所有粒子中找到的最佳位置；w、c_1 和 c_2 是参数；而 r_1 和 r_2 是从 $U(0, 1)$ 中抽取的随机数。[7] 算法 9.12 提供了一个实现。图9.14展示了该算法的几次迭代。

7 一种常见策略是让惯性 `w` 随时间衰减。

```
function particle_swarm_optimization(f, population, k_max;
    w=1, c1=1, c2=1)
    n = length(population[1].x)
    x_best, y_best = copy(population[1].x_best), Inf
    for P in population
        y = f(P.x)
        if y < y_best; x_best[:], y_best = P.x, y; end
    end
    for k in 1 : k_max
        for P in population
            r1, r2 = rand(n), rand(n)
            P.x += P.v
            P.v = w*P.v + c1*r1.*(P.x_best - P.x) +
                          c2*r2.*(x_best - P.x)
            y = f(P.x)
            if y < y_best; x_best[:], y_best = P.x, y; end
            if y < f(P.x_best); P.x_best[:] = P.x; end
        end
    end
    return population
end
```

算法9.12. 粒子群优化，该算法接受一个目标函数`f`，一个粒子列表`population`，迭代次数`k_max`，惯性权重`w`，以及动量系数`c1`和`c2`。默认值采用的是R. Eberhart和J. Kennedy在《使用粒子群理论的新优化器》一文中的设定，发表于*International Symposium on Micro Machine and Human Science*，1995年。

9.5 Firefly Algorithm

firefly algorithm (算法 9.13) 是受到萤火虫通过闪烁光亮来吸引配偶的方式启发而提出的。[8] 在萤火虫算法中，每个个体在

种群中的每只萤火虫都能闪烁以吸引其他萤火虫。在每次迭代中，所有萤火虫都会向所有更具吸引力的萤火虫移动。萤火虫 a 根据以下方式被移向具有更大吸引力的萤火虫 b

$$\mathbf{a} \leftarrow \mathbf{a} + \beta I(\|\mathbf{b} - \mathbf{a}\|)(\mathbf{b} - \mathbf{a}) + \alpha\boldsymbol{\epsilon} \tag{9.6}$$

8 X.-S. Yang, *Nature-Inspired Metaheuristic Algorithms*. Luniver Press, 2008. 有趣的是，雄性萤火虫闪烁发光以吸引异性，而雌性有时闪烁是为了吸引其他种类的雄性，随后将其吃掉。

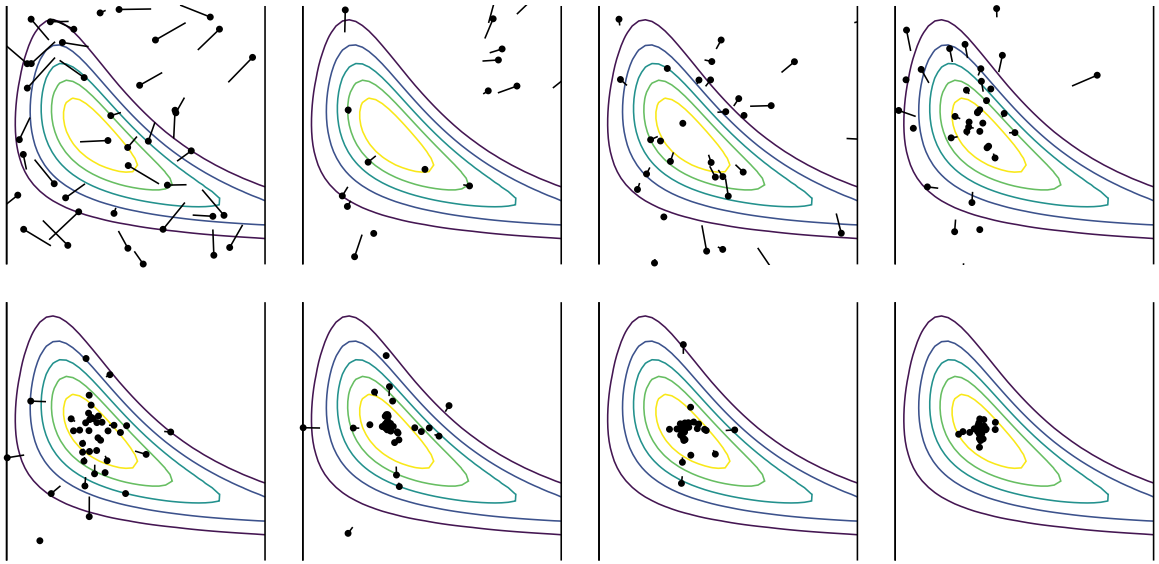


图9.14. 将$w = 0.1$、$c_1 = 0.25$和$c_2 = 2$的粒子群方法应用于附录B.7中的Wheeler's Ridge。

其中 I 是吸引力的强度，β 是光源强度。随机游走部分也包括在内，其中 $\boldsymbol{\epsilon}$ 是从零均值、单位协方差的多变量高斯分布中抽取的，而 α 则缩放步长。最终更新是朝向更亮萤火虫的偏置随机游走。[9]

9 Yang 推荐 $\beta = 1$ 和 $\alpha \in [0, 1]$。如果 $\beta = 0$，则该行为是随机游走。

强度 I 随着两只萤火虫之间的距离 r 增大而减弱，并且当 $r =$ 为 0 时定义为 1 。一种方法是将强度建模为一个向空间辐射的点源，在这种情况下，强度根据平方反比定律减弱

$$I(r) = \frac{1}{r^2} \tag{9.7}$$

或者，如果光源悬浮在吸收光线的介质中，强度将按照指数衰减的方式减少

$$I(r) = e^{-\gamma r} \tag{9.8}$$

其中 γ 是光吸收系数。[10]

10 当γ趋近于零时，萤火虫之间的距离不再重要。

在实际应用中，我们通常希望避免使用公式(9.8)，因为当 r = 趋于0时会出现奇点。逆平方定律与吸收的结合可以近似为高斯亮度衰减：

$$I(r) = e^{-\gamma r^2} \tag{9.9}$$

萤火虫的吸引力与其性能成正比。吸引力仅影响一只萤火虫是否被另一只吸引，而强度则影响吸引力较低的萤火虫移动的程度。图9.15展示了该算法的几次迭代。

```
using Distributions
function firefly(f, population, k_max;
    β=1, α=0.1, brightness=r->exp(-r^2))

    m = length(population[1])
    N = MvNormal(Matrix(1.0I, m, m))
    for k in 1 : k_max
        for a in population, b in population
            if f(b) < f(a)
                r = norm(b-a)
                a[:] += β*brightness(r)*(b-a) + α*rand(N)
            end
        end
    end
    return population[argmin([f(x) for x in population])]
end
```

算法9.13。萤火虫算法，该算法接受一个目标函数`f`、一个由设计点组成的种群`flies`、迭代次数`k_max`、光源强度`β`、随机游走步长`α`以及强度函数`I`。返回最佳设计点。

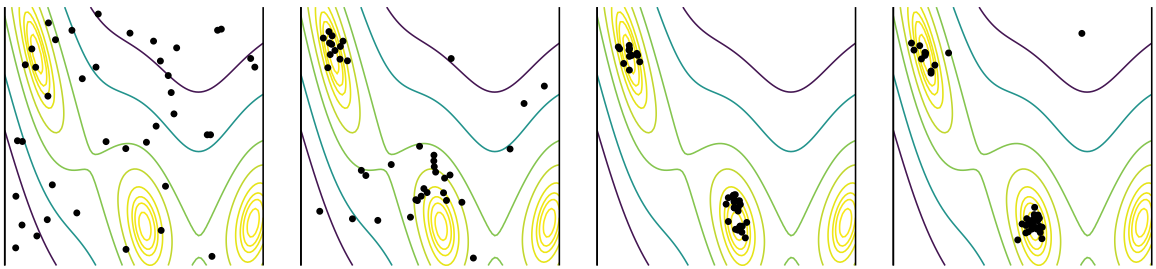


图9.15. 应用$\alpha = 0.5$、$\beta = 1$和$\gamma = 0.1$的萤火虫搜索于Branin函数（附录B.3）。

9.6 Cuckoo Search

Cuckoo search (算法9.14)是另一种以杜鹃鸟命名的自然启发式算法，它参与了一种巢寄生形式。[11]杜鹃会产下它们的

在其他鸟类的巢中产卵，通常是其他物种的鸟。当这种情况发生时，宿主鸟可能会发现侵入的蛋，然后将其破坏或建立一个新的

11 X.-S. Yang 和 S. Deb，“通过 Lévy 飞行进行布谷鸟搜索”，载于 *World Congress on Nature & Biologically Inspired Computing* (*NaBIC*)，2009 年。

在其他地方筑巢。然而，也有可能这颗蛋被宿主鸟接受并抚养。[12]

在布谷鸟搜索中，每个巢代表一个设计点。利用来自巢的*Lévy flights*可以生成新的设计点，这些点是从重尾分布中抽取步长的随机游走。如果新的设计点具有更好的目标函数值，它可以替换一个巢，这类似于布谷鸟蛋替换其他种类鸟的蛋。

核心规则是：

1. 杜鹃鸟会在随机选择的巢中产卵。
2. 拥有最佳蛋的最佳巢穴将存活到下一代。
3. 杜鹃蛋有可能被宿主鸟发现，一旦被发现，这些蛋就会被破坏。

布谷鸟搜索依赖随机飞行来建立新的巢穴位置。这些飞行从现有的巢穴出发，然后随机移动到新位置。虽然我们可能倾向于使用均匀分布或高斯分布来进行游走，但这些分布将搜索限制在相对集中的区域。相反，布谷鸟搜索使用柯西分布，其尾部更重。此外，柯西分布已被证明更能代表其他动物在野外的运动。[13] 图9.16展示了布谷鸟搜索的几次迭代。

其他受自然启发的算法包括人工蜂群、灰狼优化器、蝙蝠算法、萤火虫群优化、智能水滴和和声搜索。[14] 对于这种激增，已有一些批评

将方法与自然进行类比，而不从根本上贡献新方法及理解的种种做法。[15]

9.7 Hybrid Methods

许多群体方法在全局搜索中表现良好，能够避免局部极小值并找到设计空间中的最佳区域。遗憾的是，与下降方法相比，这些方法在局部搜索中的表现并不理想。为了扩展群体方法，已经开发了几种*hybrid methods*[16]，

基于下降的特征以改善其在局部搜索中的性能。将群体方法与局部搜索技术结合有两种通用方法：[17]

12 有趣的是，新孵化的杜鹃鸟有一种本能，就是将其他蛋或雏鸟（属于宿主的）推出巢外。

13 例如，某种果蝇利用柯西式步长探索周围环境，其间以90°转向分隔。A. M. Reynolds和M. A. Frye，“果蝇自由飞行气味追踪与最优间歇无标度搜索一致”，*PLoS ONE*，第2卷，第4期，e354，2007年。

14 参见，例如，D. Simon，*Evolutionary Optimization Algorithms*，Wiley，2013。

15 这一观点由K. Sörensen提出，“元启发式——隐喻的暴露”，*International Transactions in Operational Research*，第22卷，第1期，第3-18页，2015年。

16 在文献中，这类技术也被称为*memetic algorithms*或*genetic local segrch*

17 K. W. C. Ku 和 M.-W. Mak，“探索拉马克与鲍德温学习在演化循环神经网络中的效应，” 载于*IEEE Congress on Evolutionary Computation (CEC)*，1997年。

```
using Distributions
mutable struct Nest
    x # position
    y # value, f(x)
end
function cuckoo_search(f, population, k_max;
    p_a=0.1, C=Cauchy(0,1))
    m, n = length(population), length(population[1].x)
    a = round(Int, m*p_a)
    for k in 1 : k_max
        i, j = rand(1:m), rand(1:m)
        x = population[j].x + [rand(C) for k in 1 : n]
        y = f(x)
        if y < population[i].y
            population[i].x[:] = x
            population[i].y = y
        end

        p = sortperm(population, by=nest->nest.y, rev=true)
        for i in 1 : a
            j = rand(1:m-a)+a
            population[p[i]] = Nest(population[p[j]].x +
                                    [rand(C) for k in 1 : n],
                                    f(population[p[i]].x)
                                    )
        end
    end
    return population
end
```

算法9.14. 布谷鸟搜索，输入包括目标函数`f`、初始巢穴集`population`、迭代次数`k_max`、放弃巢穴的百分比`p_a`以及飞行分布`C`。飞行分布通常为中心柯西分布。

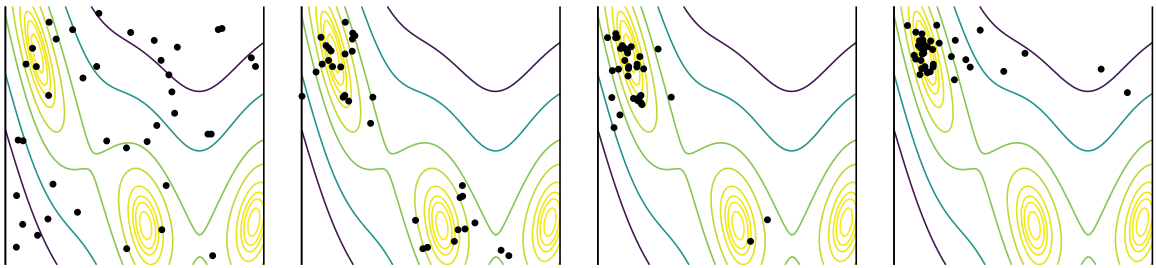


图9.16. 应用于Branin函数的布谷鸟搜索（附录B.3）。

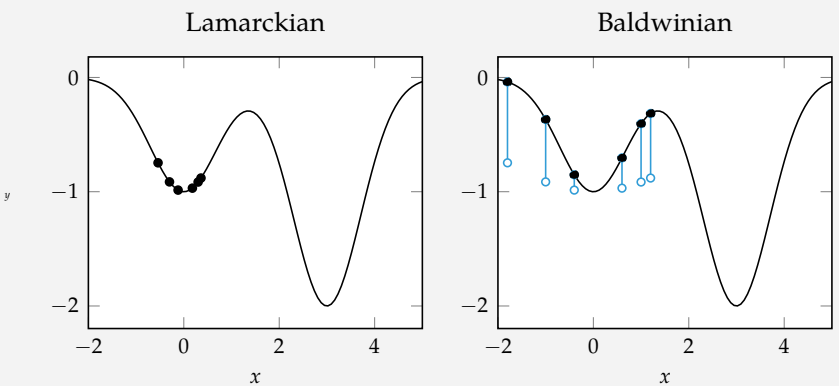
- 在*Lamarckian learning*中，通过局部搜索方法扩展了种群方法，该方法对每个个体进行局部改进。原始个体及其目标函数值被个体优化后的对应物及其目标函数值所取代。

- 在*Baldwinian learning*中，对每个个体应用相同的局部搜索方法，但结果仅用于更新个体的感知目标函数值。个体不会被替换，而只是与优化后的目标函数值关联，这些值与其实际目标函数值不同。鲍德温学习可以帮助防止过早收敛。

这些方法之间的差异在示例9.2中进行了说明。

例9.2。拉马克式和鲍德温式混合方法的比较。

考虑使用在$x = 0$附近初始化的个体种群来优化$f(x) = -e^{-x^2} - 2e^{-(x-3)^2}$。



将拉马克式局部搜索更新应用于该种群会使个体向局部最小值移动，从而降低未来个体逃离并在$x = 3$附近找到全局最优解的机会。鲍德温式方法将计算相同的更新，但保持原始设计不变。选择步骤将根据每个设计在局部搜索中的价值来评估其价值。

9.8 Summary

- 群体方法利用设计空间中的一组个体来引导向最优解的进展。
- 遗传算法利用选择、交叉和突变来产生更优的后代。
- 差分进化、粒子群优化、萤火虫算法和布谷鸟搜索都包含将设计点吸引到种群中最优个体的规则和机制，同时保持适当的状态空间探索。
- 可以使用局部搜索方法扩展种群方法，以改善收敛性。

9.9 Exercises

练习 9.1. 遗传算法中选择操作的动机是什么?

练习9.2。为什么变异在遗传算法中扮演如此基础的角色？如果我们怀疑存在更好的最优解，应该如何选择变异率?

练习 9.3. 如果我们观察到粒子群优化算法快速收敛到一个非全局最小值，我们应该如何调整算法的参数?

10 Constraints

前几章聚焦于无约束问题，其中每个设计变量的定义域均为实数空间。许多问题是有约束的，这迫使设计点必须满足特定条件。本章介绍多种方法，将带有约束的问题转化为无约束问题，从而允许使用我们已经讨论过的优化算法。同时也会探讨解析方法，包括对偶概念及约束优化下的最优性必要条件。

10.1 Constrained Optimization

回想核心优化问题方程（1.1）：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{x} \in \mathcal{X} \end{aligned} \tag{10.1}$$

在无约束问题中，可行集$\mathcal{X}$是$\mathbb{R}^n$。在约束问题中，可行集是它的某个子集。

一些约束仅仅是设计变量的上界或下界，正如我们在区间线搜索中所见，其中 x 必须位于 a 和 b 之间。一个区间约束 $x \in [a, b]$ 可以替换为两个不等式约束：$a \leq x$ 和 $x \leq b$，如图 10.1 所示。在多变量问题中，对输入变量进行区间约束会迫使它们位于一个超矩形内，如图 10.2 所示。

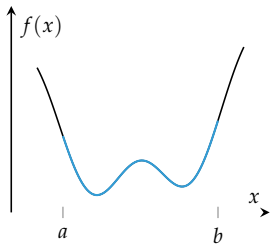


图10.1。一个受上下界约束的简单优化问题。

$$\min_x f(x), \text{ 约束条件为 } x \in [a, b]$$

在表述实际问题时，约束条件自然而然地出现。对冲基金经理卖出的股票不能超过其持有量，飞机的机翼不可能厚度为零，每周花在作业上的时间也不能超过168小时。在这类问题中加入约束条件，是为了防止优化算法给出不可行的解决方案。

对问题施加约束可能会影响解，但如图10.3所示，情况未必如此。

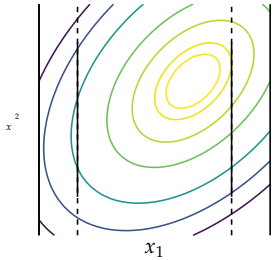


图10.2. 框式约束迫使解位于超矩形内。

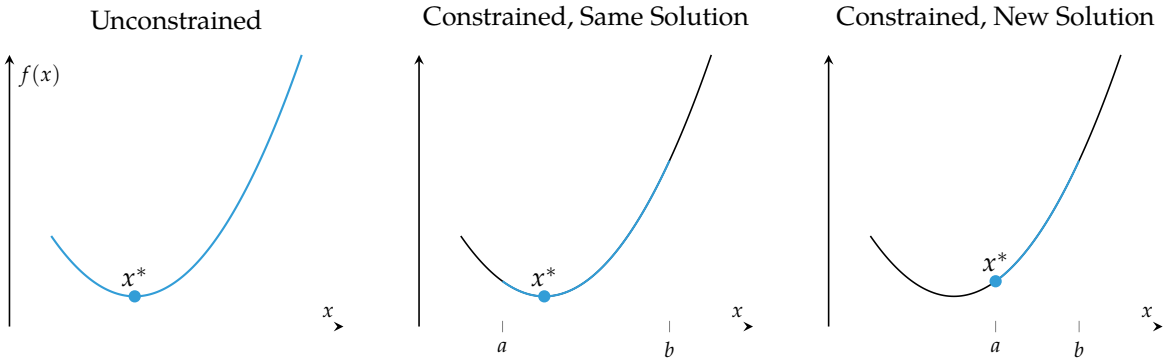


图10.3. 约束可以改变问题的解决方案，但并非必须如此。

10.2 *Constraint Types*

约束通常不直接通过已知的可行集$\mathcal{X}$来指定。相反，可行集通常由两类约束构成：[1]

1. 等式约束，$h(\mathrm{x}) = 0$
2. 不等式约束，$g(\mathrm{x}) \leq 0$

1 我们有g表示一个小于不等式约束。大于等于约束可以通过引入负号转换为小于不等式约束。

任何优化问题都可以使用这些约束条件重写：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & h_i(\mathbf{x}) = 0 \text{ for all } i \text{ in } \{1, \ldots, \ell\} \\ & g_j(\mathbf{x}) \leq 0 \text{ for all } j \text{ in } \{1, \ldots, m\} \end{aligned} \tag{10.2}$$

当然，约束可以从可行集$\mathcal{X}$构造得出：

$$h(\mathbf{x}) = (\mathbf{x} \notin \mathcal{X}) \tag{10.3}$$

其中布尔表达式的计算结果为 0 或 1。

我们经常使用等式和不等式函数（$h(\mathrm{x}) = 0$，$g(\mathrm{x}) \leq 0$）来定义约束，而不是集合隶属关系（$\mathrm{x} \in \mathcal{X}$），因为这些函数可以提供关于给定点离可行还有多远的信息。这些信息有助于推动求解方法趋向可行性。

等式约束有时会被分解为两个不等式约束：

$$h(\mathbf{x}) = 0 \quad \Longleftrightarrow \quad \begin{cases} h(\mathbf{x}) \leq 0 \\ h(\mathbf{x}) \geq 0 \end{cases} \tag{10.4}$$

然而，有时我们希望单独处理等式约束，正如我们将在本章后面讨论的那样。

10.3 *Transformations to Remove Constraints*

在某些情况下，可以通过变换问题来移除约束条件。例如，通过将 x 通过变换（图10.4），可以移除边界约束 $a \leq x \leq b$：

$$x = t_{a,b}(\hat{x}) = \frac{b+a}{2} + \frac{b-a}{2}\left(\frac{2\hat{x}}{1+\hat{x}^2}\right) \tag{10.5}$$

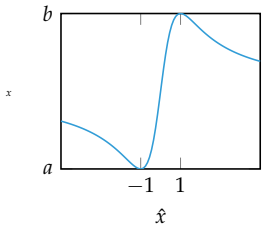


图10.4. 此变换确保x位于a和b之间。

示例 10.1 演示了这一过程。

一些等式约束可用于在给定 x_1、...、x_{n-1} 的情况下求解 x_n。换句话说，如果我们知道 x 的前 $n-$ 个 1 分量，就可以利用约束方程得到 x_n。在这种情况下，优化问题可以改为在 x_1、...、x_{n-1} 上重新表述，从而去掉该约束并减少一个设计变量。例 10.2 演示了这一过程。

示例 10.1. 通过对输入变量进行变换来移除边界约束。

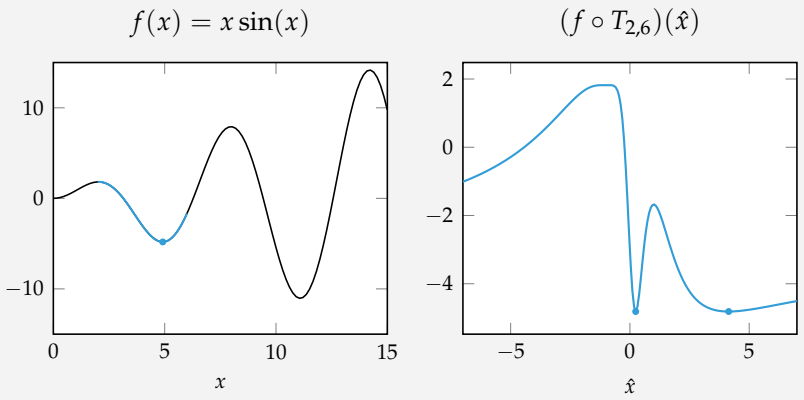
考虑优化问题

$$\begin{aligned} \underset{x}{\text{minimize}} \quad & x\sin(x) \\ \text{subject to} \quad & 2 \leq x \leq 6 \end{aligned}$$

我们可以将问题转化为消除约束的形式：

$$\begin{aligned} \underset{\hat{x}}{\text{minimize}} \quad & t_{2,6}(\hat{x})\sin(t_{2,6}(\hat{x})) \\ \underset{\hat{x}}{\text{minimize}} \quad & \left(4 + 2\left(\frac{2\hat{x}}{1+\hat{x}^2}\right)\right)\sin\left(4 + 2\left(\frac{2\hat{x}}{1+\hat{x}^2}\right)\right) \end{aligned}$$

我们可以使用我们选择的优化方法来解决这个无约束问题。在此过程中，我们找到了两个极小值：$\hat{x} \approx 0.242$ 和 $\hat{x} \approx 4.139$，这两个极小值的函数值大约为 -4.814。



原问题的解通过将 $\hat{x}$ 通过变换得到。两个 $\hat{x}$ 值都产生 $x = t_{2,6}(\hat{x}) \approx 4.914$。

示例 10.2. 使用约束方程消除设计变量。

考虑约束条件：

$$h(\mathbf{x}) = x_1^2 + x_2^2 + \cdots + x_n^2 - 1 = 0$$

我们可以使用前 $n-$ 个 1 变量来求解 x_n：

$$x_n = \pm\sqrt{1 - x_1^2 + x_2^2 + \cdots + x_{n-1}^2}$$

我们可以变换

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & h(\mathbf{x}) = 0 \end{aligned}$$

到

$$\underset{x_1,\ldots,x_{n-1}}{\text{minimize}} \quad f\left(\left[x_1, \ldots, x_{n-1}, \pm\sqrt{1 - x_1^2 + x_2^2 + \cdots + x_{n-1}^2}\right]\right)$$

10.4 Lagrange Multipliers

使用*Lagrange multipliers*的方法来优化受等式约束的函数。考虑具有单个等式约束的优化问题：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & h(\mathbf{x}) = 0 \end{aligned} \tag{10.6}$$

其中 f 和 h 具有连续偏导数。例10.3讨论了这样一个问题。

拉格朗日乘数法用于计算 f 的等高线与 $h(\mathrm{x}) = 0$ 的等高线对齐的位置。由于函数在某点的梯度垂直于通过该点的该函数的等高线，我们知道 h 的梯度将垂直于等高线 $h(\mathrm{x}) = 0$。因此，我们需要找到 f 的梯度和 h 的梯度对齐的位置。

我们寻找最佳的x，使得约束条件

$$h(\mathbf{x}) = 0 \tag{10.7}$$

考虑最小化问题：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & -\exp\left(-\left(x_1 x_2 - \frac{3}{2}\right)^2 - (x_2 - \frac{3}{2})^2\right) \\ \text{subject to} \quad & x_1 - x_2^2 = 0 \end{aligned}$$

我们将约束 $x_1 = x_2^2$ 代入目标函数，得到一个无约束的目标：

$$f_{\text{unc}} = -\exp\left(-\left(x_2^3 - \frac{3}{2}\right)^2 - \left(x_2 - \frac{3}{2}\right)^2\right)$$

其导数为：

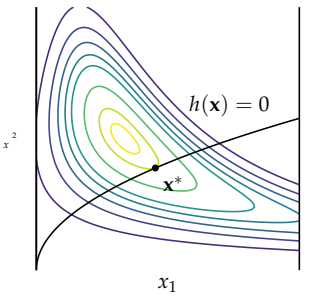
$$\frac{\partial}{\partial x_2} f_{\text{unc}} = 6\exp\left(-\left(x_2^3 - \frac{3}{2}\right)^2 - \left(x_2 - \frac{3}{2}\right)^2\right)\left(x_2^5 - \frac{3}{2}x_2^2 + \frac{1}{3}x_2 - \frac{1}{2}\right)$$

将导数设为零并求解 x_2 得到 $x_2 \approx 1.165$。因此，原始优化问题的解为 $\mathbf{x}^* \approx [1.358, 1.165]$。最优解位于 f 的等高线与 h 对齐之处。

如果点 x^* 沿着 h 优化 f，那么它在 x^* 处沿 h 的方向导数必须为零。也就是说，x^* 沿 h 的小幅移动不会带来改进。

f 的等值线是常数 f 的线。因此，如果 f 的等值线与 h 相切，那么在该点处，h 沿轮廓 $h(\mathbf{x}) = 0$ 方向的方向导数必定为零。

例10.3. 拉格朗日乘数法的一个启发性的例子。



满足且梯度对齐

$$\nabla f(\mathbf{x}) = \lambda \nabla h(\mathbf{x}) \tag{10.8}$$

对于某个 *Lagrange multiplier* λ。我们需要标量 λ，因为梯度的大小可能不相同。[2]

2 当 ∇f 为零时，拉格朗日乘子 λ 等于零，与 ∇h 无关。

我们可以将*Lagrangian*公式化，它是设计变量和乘数的函数。

$$\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda h(\mathbf{x}) \tag{10.9}$$

求解 $\nabla \mathcal{L}(\mathrm{x},\ \lambda) = 0$ 解方程（10.7）和（10.8）。具体来说，$\nabla_{\mathbf{x}} \mathcal{L} = 0$ 给出条件 $\nabla f = \lambda \nabla h$，而 $\nabla_\lambda \mathcal{L} = 0$ 给出 $h(\mathrm{x}) = 0$。任何解都被视为 *critical point*。临界点可以是局部极小值、全局极小值或鞍点。[3] 例 10.4 演示了这种方法。

3 拉格朗日乘子法为我们提供了检验最优性的一阶必要条件。我们将扩展此方法以包含不等式。

例10.4. 使用拉格朗日乘子法解决例10.3中的问题。

我们可以使用拉格朗日乘数法来解决例10.3中的问题。我们构造拉格朗日函数

$$\mathcal{L}(x_1, x_2, \lambda) = -\exp\left(-\left(x_1 x_2 - \frac{3}{2}\right)^2 - \left(x_2 - \frac{3}{2}\right)^2\right) - \lambda(x_1 - x_2^2)$$

并计算梯度

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial x_1} &= 2x_2 f(\mathbf{x})\left(\frac{3}{2} - x_1 x_2\right) - \lambda \\
\frac{\partial \mathcal{L}}{\partial x_2} &= 2\lambda x_2 + f(\mathbf{x})\left(-2x_1(x_1 x_2 - \frac{3}{2}) - 2(x_2 - \frac{3}{2})\right) \\
\frac{\partial \mathcal{L}}{\partial \lambda} &= x_2^2 - x_1
\end{aligned}$$

将这些导数设为零并求解，得到 $x_1 \approx 1.358$、$x_2 \approx 1.165$ 和 $\lambda \approx 0.170$。

拉格朗日乘数法可以扩展到多个等式约束。考虑一个包含两个等式约束的问题：

$$\begin{aligned}
\underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\
\text{subject to} \quad & h_1(\mathbf{x}) = 0 \\
& h_2(\mathbf{x}) = 0
\end{aligned} \tag{10.10}$$

我们可以将这些约束合并为一个约束。新约束满足的点与之前完全相同，因此解保持不变。

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & h_{\text{comb}}(\mathbf{x}) = h_1(\mathbf{x})^2 + h_2(\mathbf{x})^2 = 0 \end{aligned} \tag{10.11}$$

我们现在可以像之前一样应用拉格朗日乘数法。特别地，我们计算梯度条件

$$\nabla f - \lambda \nabla h_{\text{comb}} = \mathbf{0} \tag{10.12}$$

$$\nabla f - 2\lambda(h_1 \nabla h_1 + h_2 \nabla h_2) = \mathbf{0} \tag{10.13}$$

我们对h_{comb}的选择有些随意。我们本可以使用

$$h_{\text{comb}}(\mathbf{x}) = h_1(\mathbf{x})^2 + c \cdot h_2(\mathbf{x})^2 \tag{10.14}$$

对于某个常数 $c > 0$。

通过这种更通用的表述，我们得到

$$\mathbf{0} = \nabla f - \lambda \nabla h_{\text{comb}} \tag{10.15}$$

$$= \nabla f - 2\lambda h_1 \nabla h_1 + 2c\lambda h_2 \nabla h_2 \tag{10.16}$$

$$= \nabla f - \lambda_1 \nabla h_1 + \lambda_2 \nabla h_2 \tag{10.17}$$

因此，对于含有 ℓ 个等式约束的问题，我们可以用 ℓ 个拉格朗日乘子定义一个拉格朗日函数

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) - \sum_{i=1}^{\ell} \lambda_i h_i(\mathbf{x}) = f(\mathbf{x}) - \boldsymbol{\lambda}^\top \mathbf{h}(\mathbf{x}) \tag{10.18}$$

10.5 *Inequality Constraints*

考虑一个具有单个不等式约束的问题：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & g(\mathbf{x}) \leq 0 \end{aligned} \tag{10.19}$$

我们知道，如果解位于约束边界上，那么拉格朗日条件成立

$$\nabla f - \mu \nabla g = \mathbf{0} \tag{10.20}$$

对于某个常数μ。当这种情况发生时，约束被认为是*active*，目标函数的梯度受到的限制与等式约束时完全相同。图10.5展示了一个示例。

如果问题的解不在约束边界上，那么该约束被认为是*inactive*。f的解将简单地位于f梯度为零的地方，就像无约束优化一样。在这种情况下，通过将μ设置为零，方程(10.20)将成立。图10.6显示了一个例子。

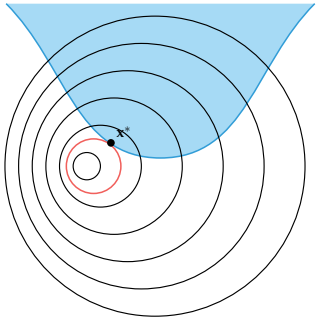


图10.5. 一个有效的不等式约束。相应的轮廓线以红色显示。

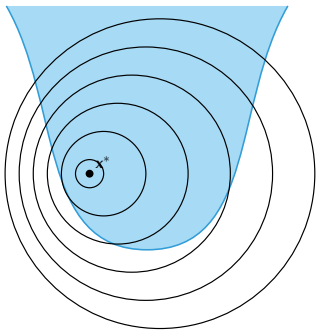


图10.6. 非活动不等式约束。

我们可以通过为不可行点引入无穷步长惩罚来优化具有不等式约束的问题：

$$f_{\infty\text{-step}}(\mathbf{x}) = \begin{cases} f(\mathbf{x}) & \text{if } g(\mathbf{x}) \leq 0 \\ \infty & \text{otherwise} \end{cases} \tag{10.21}$$

$$= f(\mathbf{x}) + \infty \cdot (g(\mathbf{x}) > 0) \tag{10.22}$$

不幸的是，$f_{\infty\text{-step}}$ 优化起来很不方便。[4] 它不连续且不可微。搜索程序无法获得方向信息来引导自己朝向可行性。

我们可以改用线性惩罚 $\mu g(\mathbf{x})$，它对 $\infty \cdot (g(\mathbf{x}) > 0)$ 形成一个下界，并且只要 $\mu > 0$，就惩罚目标趋向可行性。这个线性惩罚在图10.7中可视化。

我们可以使用这个线性惩罚来构造不等式约束的拉格朗日函数

$$\mathcal{L}(\mathbf{x}, \mu) = f(\mathbf{x}) + \mu g(\mathbf{x}) \tag{10.23}$$

我们可以通过相对于 μ 最大化来恢复 $f_{\infty\text{-step}}$

$$f_{\infty\text{-step}} = \underset{\mu \geq 0}{\text{maximize}}\ \mathcal{L}(\mathbf{x}, \mu) \tag{10.24}$$

对于任何不可行的x，我们得到无穷大；对于任何可行的x，我们得到$f(\mathbf{x})$。

新的优化问题因此为

$$\underset{\mathbf{x}}{\text{minimize}}\ \underset{\mu \geq 0}{\text{maximize}}\ \mathcal{L}(\mathbf{x}, \mu) \tag{10.25}$$

这种重新表述被称为*primal*问题。

4 这样的问题表述可以使用直接方法进行优化，例如网格自适应直接搜索（第8.2节）。

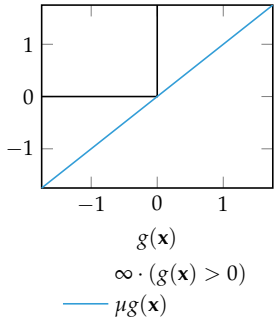


图 10.7. 线性函数 $\mu g(\mathbf{x})$ 是无限步惩罚的下界，只要 $\mu \geq 0$。

优化原始问题需要找到临界点 x^*，使得：

1. $g(x^*) \leq 0$ 该点是可行的。

2. $\mu \geq 0$
惩罚必须指向正确的方向。这个要求有时被称为{v*}。

3. $\mu g(x^*) = 0$
边界上的可行点将满足$g(x) = 0$，而满足$g(x) < 0$的可行点则需要$\mu = 0$，以便从拉格朗日函数中恢复$f(x^*)$。

4. $\nabla f(x^*) - \mu \nabla g(x^*) = 0$
当约束条件起作用时，我们要求f和g的等高线对齐，这等价于说它们的梯度对齐。当约束条件不起作用时，我们的最优解将具有$\nabla f(x^*) = 0$和$\mu = 0$。

这四个要求可以推广到具有任意数量的等式和不等式约束的优化问题：[5]

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{g}(\mathbf{x}) \leq \mathbf{0} \\ & \mathbf{h}(\mathbf{x}) = \mathbf{0} \end{aligned} \tag{10.26}$$

5 如果u和v是长度相同的向量，则当对于所有i都有$u_i \leq v_i$时，我们说u ≤ v。我们类似地为向量定义≥、<和>。

其中 g 的每个分量是不等式约束，h 的每个分量是等式约束。这四个条件称为 *KKT conditions*。[6]

6 以 Harold W. Kuhn 和 Albert W. Tucker 命名，他们在 1951 年发表了这些条件。后来发现 William Karush 在 1939 年的一篇未发表的硕士论文中研究了这些条件。T. H. Kjeldsen 提供了历史展望，“非线性规划中 Kuhn-Tucker 定理的语境化历史分析：第二次世界大战的影响”，*Historia Mathe-*，第 27 卷，第 4 期，第 331–361 页，2000 年。

1. 可行性：所有约束条件均得到满足。

$$\mathbf{g}(\mathbf{x}^*) \leq \mathbf{0} \tag{10.27}$$

$$\mathbf{h}(\mathbf{x}^*) = \mathbf{0} \tag{10.28}$$

2. 对偶可行性：惩罚趋向于可行性。

$$\boldsymbol{\mu} \geq 0 \tag{10.29}$$

matica

3. 互补松弛性：拉格朗日乘子填补松弛。要么μ_i为零，要么$g_i(x^*)$为零。[7]

$$\boldsymbol{\mu} \odot \mathbf{g} = \mathbf{0} \tag{10.30}$$

7 运算 a ⊙ b 表示向量 a 和 b 之间的逐元素乘积。

4. 平稳性：目标函数等高线与每个有效约束相切。

$$\nabla f(\mathbf{x}^*) - \sum_i \mu_i \nabla g_i(\mathbf{x}^*) - \sum_j \lambda_j \nabla h_j(\mathbf{x}^*) = \mathbf{0} \tag{10.31}$$

这四个条件是最优性的一阶必要条件，因此对于具有光滑约束的问题是FONCs。正如无约束优化的一阶必要条件一样，必须特别注意确保所识别的临界点实际上是局部最小值。

10.6 Duality

在推导约束优化的FONCs时，我们也发现了拉格朗日量的一种更一般的形式。这个 *generalized Lagrangian* 是[8]

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \sum_i \mu_i g_i(\mathbf{x}) + \sum_j \lambda_j h_j(\mathbf{x}) \tag{10.32}$$

8 由于λ的符号不受限制，我们可以反转拉格朗日乘子法中等式约束的符号。

优化问题的{v*}是使用广义拉格朗日公式表述的原始优化问题

$$\underset{\mathbf{x}}{\text{minimize}} \; \underset{\boldsymbol{\mu} \geq \mathbf{0}, \boldsymbol{\lambda}}{\text{maximize}} \; \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) \tag{10.33}$$

原始问题与原始问题完全相同，且同样难以优化。

优化问题的*dual*形式将方程（10.33）中的最小化和最大化顺序颠倒：

$$\underset{\boldsymbol{\mu} \geq \mathbf{0}, \boldsymbol{\lambda}}{\text{maximize}} \; \underset{\mathbf{x}}{\text{minimize}} \; \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) \tag{10.34}$$

max-min inequality 指出，对于任意函数 $f(\mathrm{a}, \mathrm{b})$：

$$\underset{\mathbf{a}}{\text{maximize}} \; \underset{\mathbf{b}}{\text{minimize}} \; f(\mathbf{a}, \mathbf{b}) \leq \underset{\mathbf{b}}{\text{minimize}} \; \underset{\mathbf{a}}{\text{maximize}} \; f(\mathbf{a}, \mathbf{b}) \tag{10.35}$$

对偶问题的解因此是原问题解的下界。也就是说，$d^* \leq p^*$，其中 d^* 是 *dual value*，而 p^* 是 *primal value*。

对偶问题中的内层最大化通常被并入一个*dual function*，

$$\mathcal{D}(\boldsymbol{\mu} \geq \mathbf{0}, \boldsymbol{\lambda}) = \underset{\mathbf{x}}{\text{minimize}} \; \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) \tag{10.36}$$

为方便表达。对偶函数是凹函数。[9] 在其上的梯度上升凹函数总是收敛到全局最大值。只要相对于 x 最小化拉格朗日函数容易，优化对偶问题就容易。

[9] 有关详细概述，请参见 S. Nash 和 A. Sofer，*Linear and Non-linear Programming*。McGraw-Hill，1996 年。

我们知道，最大化$_{\boldsymbol{\mu} \geq \mathbf{0}, \boldsymbol{\lambda}} \mathcal{D}(\boldsymbol{\mu}、\boldsymbol{\lambda}) \leq p^*$。由此可知，对偶函数总是原始问题的下界（参见例10.5）。对于任何$\boldsymbol{\mu} \geq 0$及任何$\boldsymbol{\lambda}$，我们有

$$\mathcal{D}(\boldsymbol{\mu} \geq \mathbf{0}, \boldsymbol{\lambda}) \leq p^* \tag{10.37}$$

对偶值与原始值之间的差 $p^* - d^*$ 被称为 *duality gap*。在某些情况下，对偶问题保证与原问题有相同的解——对偶间隙为零。[10] 在这种情况下，对偶性提供了一种优化问题的替代方法。例10.6展示了这种方法。

[10] 在S. Boyd和L. Vandenberghe的著作*Con-vex Optimization*中，讨论了保证零对偶间隙的10个条件。剑桥大学出版社，2004年。

10.7 Penalty Methods

我们可以使用 *penalty methods* 通过向目标函数添加惩罚项，将约束优化问题转化为无约束优化问题，从而可以使用前几章中开发的方法。

考虑一个一般优化问题：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{g}(\mathbf{x}) \leq \mathbf{0} \\ & \mathbf{h}(\mathbf{x}) = \mathbf{0} \end{aligned} \tag{10.38}$$

一种简单的惩罚方法计算被违反的约束方程的数量：

$$p_{\text{count}}(\mathbf{x}) = \sum_i (g_i(\mathbf{x}) > 0) + \sum_j (h_j(\mathbf{x}) \neq 0) \tag{10.39}$$

这导致了一个惩罚不可行性的无约束优化问题

$$\underset{\mathbf{x}}{\text{minimize}}\, f(\mathbf{x}) + \rho \cdot p_{\text{count}}(\mathbf{x}) \tag{10.40}$$

其中 $\rho > 0$ 调整惩罚的幅度。图10.8展示了一个示例。

例10.5。对偶函数是原始问题的下界。

考虑优化问题：

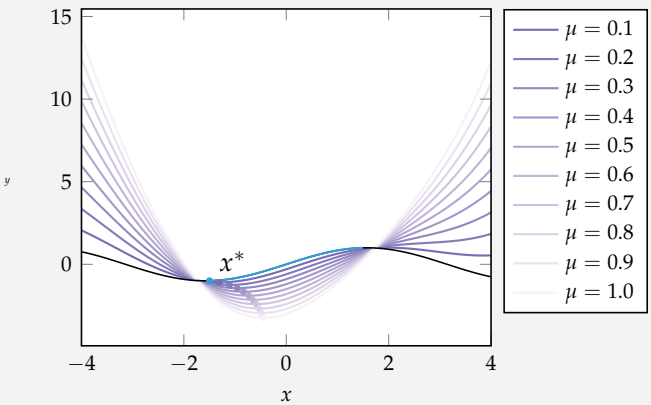
$$\begin{aligned} \underset{x}{\text{minimize}} \quad & \sin(x) \\ \text{subject to} \quad & x^2 \leq 3 \end{aligned}$$

广义拉格朗日量为$\mathcal{L}(x、\mu) = \sin(x) + \mu(x^2 - 3)$，由此得到原始问题：

$$\underset{x}{\text{minimize}} \, \underset{\mu \geq 0}{\text{maximize}} \, \sin(x) + \mu(x^2 - 3)$$

以及对偶问题：

$$\underset{\mu \geq 0}{\text{maximize}} \, \underset{x}{\text{minimize}} \, \sin(x) + \mu(x^2 - 3)$$



目标函数用黑色绘制，可行域用蓝色描绘。最小值位于 $x^* = -1.5$ 处，$p^* \approx -0.997$。紫色线是拉格朗日量 $\mathcal{L}(x、\mu)$，对应 $\mu = 0.1、0.2、...、1$，其中每条线的最小值都低于 p^*。

考虑这个问题：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & x_1 + x_2 + x_1 x_2 \\ \text{subject to} \quad & x_1^2 + x_2^2 = 1 \end{aligned}$$

拉格朗日量为 $\mathcal{L}(x_1、x_2、\lambda) = x_1 + x_2 + x_1 x_2 + \lambda(x_1^2 + x_2^2 - 1)$。
我们应用拉格朗日乘子法：

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial x_1} &= 1 + x_2 + 2\lambda x_1 = 0 \\ \frac{\partial \mathcal{L}}{\partial x_2} &= 1 + x_1 + 2\lambda x_2 = 0 \\ \frac{\partial \mathcal{L}}{\partial \lambda} &= x_1^2 + x_2^2 - 1 = 0 \end{aligned}$$

解得四个可能的解，因此有四个临界点：

x_1	x_2	λ
-1	0	$1/2$
0	-1	$1/2$
$\frac{\sqrt{2}+1}{\sqrt{2}+2}$	$\frac{\sqrt{2}+1}{\sqrt{2}+2}$	$\frac{1}{2}\left(-1-\sqrt{2}\right)$
$\frac{\sqrt{2}-1}{\sqrt{2}-2}$	$\frac{\sqrt{2}-1}{\sqrt{2}-2}$	$\frac{1}{2}\left(-1+\sqrt{2}\right)$

对偶函数的形式为

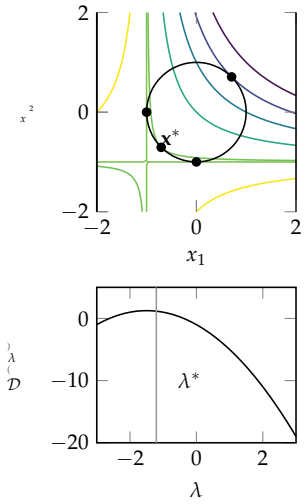
$$\mathcal{D}(\lambda) = \underset{x_1, x_2}{\text{minimize}}\, x_1 + x_2 + x_1 x_2 + \lambda(x_1^2 + x_2^2 - 1)$$

我们可以在 $x_1 = x_2 = x$ 中代入，并将关于 x 的导数设为零，得到 $x = -1 - \lambda$。进行代入后得到

$$\mathcal{D}(\lambda) = -1 - 3\lambda - \lambda^2$$

对偶问题最大化 $_\lambda\, \mathcal{D}(\lambda)$在$\lambda = \left(-1 - \sqrt{2}\right)/2$处取得最大值。

例10.6. 拉格朗日对偶应用于具有等式约束问题的一个示例。上图展示了目标函数的等高线及带有四个标记为散点的临界约束。下图显示了该对偶函数。



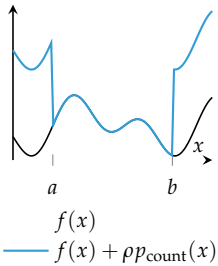


图10.8。在满足$x \in [a, b]$的条件下，使f最小化的原始目标函数及计数惩罚目标函数。

惩罚方法从初始点 x 和 ρ 的一个小值开始。然后求解无约束优化问题方程（10.40）。得到的设计点随后被用作增加惩罚后的另一次优化的起点。我们继续这一过程，直到所得点可行，或达到最大迭代次数。算法10.1提供了一个实现。

```
function penalty_method(f, p, x, k_max; ρ=1, γ=2)
    for k in 1 : k_max
        x = minimize(x -> f(x) + ρ*p(x), x)
        ρ *= γ
        if p(x) == 0
            return x
        end
    end
    return x
end
```

算法10.1. 针对目标函数`f`、惩罚函数`p`、初始点`x`、迭代次数`k_max`、初始惩罚项`ρ > 0`以及惩罚乘数`γ > 1`的惩罚方法。方法`minimize`应替换为合适的无约束最小化方法。

对于较大的 ρ 值，这种惩罚将保持问题解不变，但会引入一个尖锐的不连续性。不在可行集内的点缺乏梯度信息来指导搜索向可行性方向进行。

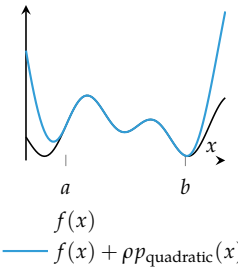


图10.9. 使用二次罚函数最小化f受限于$x \in [a, b]$。

我们可以使用*quadratic penalties*来生成一个平滑的目标函数（图10.9）：

$$p_{\text{quadratic}}(\mathbf{x}) = \sum_i \max(g_i(\mathbf{x}), 0)^2 + \sum_j h_j(\mathbf{x})^2 \tag{10.41}$$

接近约束边界的二次惩罚非常小，可能需要ρ趋于无穷大，才能使解不再违反约束。

也可以将计数和二次惩罚函数混合使用（图10.10）：

$$p_{\text{mixed}}(\mathbf{x}) = \rho_1 p_{\text{count}}(\mathbf{x}) + \rho_2 p_{\text{quadratic}}(\mathbf{x}) \tag{10.42}$$

这种惩罚混合方式在可行域与不可行域之间提供了清晰的边界，同时为求解器提供了梯度信息。

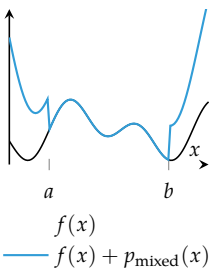


图10.10. 同时使用二次和离散惩罚函数，在满足a, b的条件下最小化f[$x \in$]。

图10.11显示了随着ρ的增加，惩罚函数的进展。如例10.7所述，二次惩罚函数无法确保可行性。

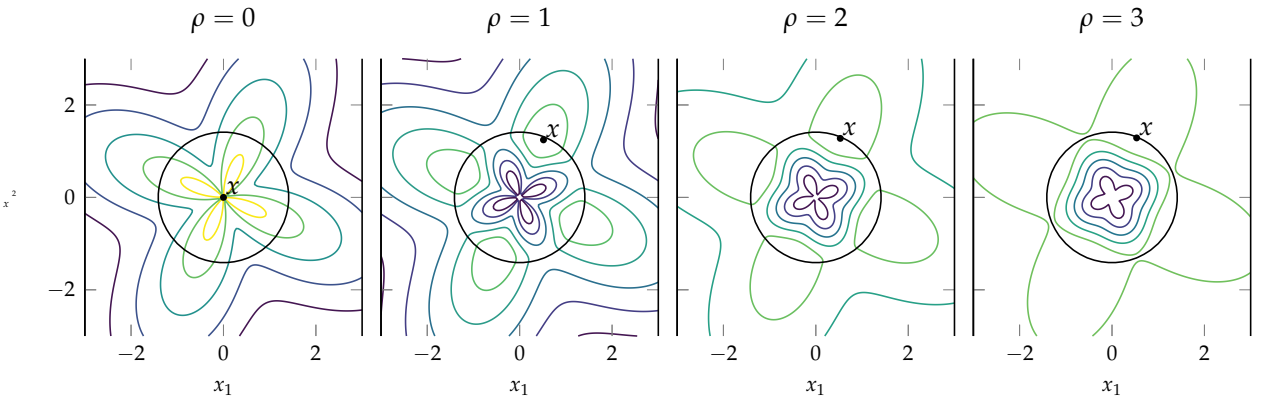


图10.11. 应用于花朵函数的惩罚方法，附录B.4，以及圆形约束

$$x_1^2 + x_2^2 \geq 2$$

考虑这个问题

$$\begin{aligned} &\text{最小化} && x \\ &x \\ &\text{得 } x \geq 5 && \text{使} \end{aligned}$$

使用二次罚函数。

无约束目标函数为

$$f(x) = x + \rho \max(5 - x, 0)^2$$

无约束目标函数的最小值为

$$x^* = 5 - \frac{1}{2\rho}$$

虽然约束优化问题的最小值显然是 $x = 5$，但惩罚优化问题的最小值仅趋近于 $x = 5$，需要无限大的惩罚才能达到可行性。

示例 10.7. 一个展示二次惩罚项无法确保可行性的例子。

10.8 Augmented Lagrange Method

augmented Lagrange method[11] 是惩罚方法规范的改编
特别适用于等式约束。与惩罚方法不同，后者有时必须先使ρ趋于无穷大才能找到可行解，增广拉格朗日方法则能在较小的ρ值下工作。它对每个约束同时使用二次和线性惩罚。

11 不要与拉格朗日乘数法混淆。

对于具有等式约束 \(h(x) \) = 0 的优化问题，罚函数为：

$$p_{\text{Lagrange}}(\mathbf{x}) = \frac{1}{2}\rho \sum_i (h_i(\mathbf{x}))^2 - \sum_i \lambda_i h_i(\mathbf{x}) \tag{10.43}$$

其中 {v*} 收敛于拉格朗日乘子。

除了在每次迭代中增加ρ之外，线性惩罚向量根据以下公式更新：

$$\boldsymbol{\lambda}^{(k+1)} = \boldsymbol{\lambda}^{(k)} - \rho \mathbf{h}(\mathbf{x}) \tag{10.44}$$

算法10.2提供了一个实现。

```
function augmented_lagrange_method(f, h, x, k_max; ρ=1, γ=2)
    λ = zeros(length(h(x)))
    for k in 1 : k_max
        p = x -> f(x) + ρ/2*sum(h(x).^2) - λ⋅h(x)
        x = minimize(x -> f(x) + p(x), x)
        ρ *= γ
        λ -= ρ*h(x)
    end
    return x
end
```

算法10.2. 针对目标函数 `f`、等式约束函数 `h`、初始点 `x`、迭代次数 `k_max`、初始惩罚标量 `ρ > 0` 以及惩罚乘子 `γ > 1` 的增广拉格朗日方法。函数 `minimize` 应替换为您选择的最小化方法。

10.9 Interior Point Methods

Interior point methods (算法10.3)，有时被称为*barrier methods*，是确保搜索点始终可行的优化方法。[12]

内点法使用一种障碍函数，当接近约束边界时，该函数趋于无穷大。此障碍函数$p_{\text{barrier}}(x)$必须满足以下几个性质：

12 早期停止的内点方法产生几乎最优但可行的设计点。方法可能因时间或处理限制而提前停止。

1. $p_{\text{barrier}}(x)$ 是连续的

2. $p_{\text{barrier}}(\mathrm{x})$ 非负（$p_{\text{barrier}}(\mathrm{x}) \geq 0$）

3. 当x接近任何约束边界时，$p_{\text{barrier}}(\mathrm{x})$趋于无穷大

以下是一些障碍函数的示例：
逆障碍：

$$p_{\text{barrier}}(\mathbf{x}) = -\sum_i \frac{1}{g_i(\mathbf{x})} \tag{10.45}$$

对数障碍：

$$p_{\text{barrier}}(\mathbf{x}) = -\sum_i \begin{cases} \log(-g_i(\mathbf{x})) & \text{if } g_i(\mathbf{x}) \geq -1 \\ 0 & \text{otherwise} \end{cases} \tag{10.46}$$

一个带有不等式约束的问题可以转化为无约束优化问题

$$\underset{\mathbf{x}}{\text{minimize}}\, f(\mathbf{x}) + \frac{1}{\rho} p_{\text{barrier}}(\mathbf{x}) \tag{10.47}$$

当ρ增加时，接近边界的惩罚会减少（图10.12）。

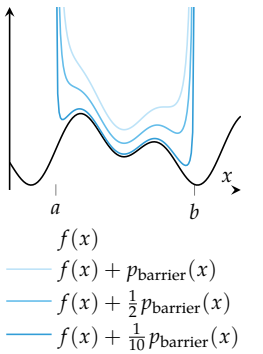


图10.12. 应用内点法及逆向障碍函数，以在约束条件 $x \in [a, b]$ 下最小化 f。

必须特别注意确保线搜索不离开可行域。线搜索 $f(\mathrm{x} + \alpha \mathrm{d})$ 被约束在区间 $\alpha = [0, \alpha_u]$ 内，其中 α_u 是到最近边界的最小步长。实际中，α_u 的选择使得 x $+ \alpha$d 恰好位于边界内部，以避免边界奇异性。

与惩罚方法类似，内点法从一个较小的ρ值开始，并逐渐增大直至收敛。当连续点之间的差异小于某个阈值时，内点法通常会被终止。图10.13展示了逐步增大ρ效果的示例。

内点法需要一个可行点作为搜索的起点。寻找可行点的一种简便方法是优化二次罚函数

$$\underset{\mathbf{x}}{\text{minimize}}\, p_{\text{quadratic}}(\mathbf{x}) \tag{10.48}$$

```
function interior_point_method(f, p, x; ρ=1, γ=2, ϵ=0.001)
    delta = Inf
    while delta > ϵ
        x′ = minimize(x -> f(x) + p(x)/ρ, x)
        delta = norm(x′ - x)
        x = x′
        ρ *= γ
    end
    return x
end
```

算法10.3. 目标函数`f`、障碍函数`p`、初始点`x`、初始惩罚项`ρ > 0`、惩罚乘数`γ > 1`及停止容差`ϵ > 0`的内点法。

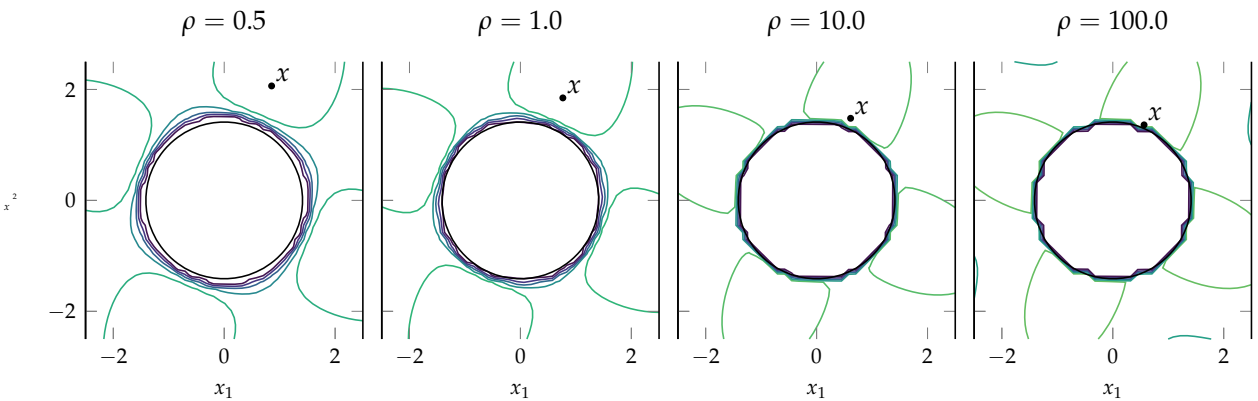


图10.13. 应用逆障碍函数的内点法于花朵函数，附录B.4，及约束条件

$$x_{21} + x_{22} \geq 2$$

10.10 Summary

- 约束是设计点必须满足的要求。
- 一些约束可以通过变换或代入问题中，从而转化为无约束优化问题。
- 使用拉格朗日乘子的分析方法得出广义拉格朗日量和约束下最优性的必要条件。
- 一个有约束的优化问题具有一个对偶问题形式，该形式更易于求解，且其解是原问题解的下界。
- 惩罚方法对不可行解进行惩罚，并通常向优化器提供梯度信息，以引导不可行点趋向可行。
- 内点法保持可行性，但使用障碍函数以避免离开可行集。

10.11 Exercises

练习 10.1. 求解

$$\begin{aligned} \underset{x}{\text{minimize}} \quad & x \\ \text{subject to} \quad & x \geq 0 \end{aligned} \tag{10.49}$$

使用二次惩罚方法，其中$\rho >$为0。以闭式形式求解该问题。

练习10.2。使用计数惩罚方法（其中$\rho >$为1）求解上述问题，并与二次惩罚方法进行比较。

练习 10.3. 假设你正在使用惩罚方法解决一个约束问题。你注意到迭代点始终不可行，于是决定停止算法。为了在下一次尝试中更成功，你可以做些什么?

练习 10.4。考虑一个简单的单变量最小化问题，其中你最小化一个函数$f(x)$，受约束$x \geq 0$。假设你知道约束是活跃的，即在最小化点x^*处$x^* = 0$，且根据最优性条件$f'(x^*) > 0$。证明使用惩罚方法求解同一问题

$$f(x) + (\min(x, 0))^2 \tag{10.50}$$

针对原问题产生了一个不可行解。

练习10.5。与二次罚函数方法相比，增广拉格朗日方法有何优势?

练习10.6。何时会使用障碍法代替惩罚法?

练习 10.7. 给出一个光滑优化问题的例子，使得对于任何惩罚参数 $\rho >> 0$，都存在一个起始点 $x^{(1)}$，使得最速下降法发散。

练习 10.8。假设你有一个优化问题 em

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{h}(\mathbf{x}) = \mathbf{0} \\ & \mathbf{g}(\mathbf{x}) \leq \mathbf{0} \end{aligned} \tag{10.51}$$

但没有初始可行设计。如果存在可行点，你将如何找到关于约束的可行点?

练习 10.9. 解决约束优化问题

$$\begin{aligned} \underset{x}{\text{minimize}} \quad & \sin\left(\frac{4}{x}\right) \\ \text{subject to} \quad & x \in [1, 10] \end{aligned} \tag{10.52}$$

同时使用变换 $x = t_{a,b}(\hat{x})$ 和用于约束边界的 S 形变换 $x \in [a, \quad b]$:

$$x = s(\hat{x}) = a + \frac{(b-a)}{1 + e^{-\hat{x}}} \tag{10.53}$$

为什么 t 变换比 sigmoid 变换更好?

练习 10.10。给出一个涉及两个设计变量的二次目标函数的例子，其中添加一个线性约束会导致不同的最优解。

练习 10.11。假设我们希望在约束条件 $x_1 + 2x_2 + 3x_3 = 6$ 下最小化 {v*}。我们如何将其转换为具有相同最小值的无约束问题?

练习 10.12. 假设我们要在约束条件 $ax_1 + x_2 \leq 5$ 和 $x_1, x_2 \geq 0$ 下最小化 $-x_1 - 2x_2$ 的目标函数。如果 a 是一个有界常数，那么 a 取哪些范围的值会导致无穷多个最优解?

练习 10.13. 考虑使用惩罚方法来优化

$$\begin{aligned} \underset{x}{\text{minimize}} \quad & 1 - x^2 \\ \text{subject to} \quad & |x| \leq 2 \end{aligned} \tag{10.54}$$

使用惩罚方法进行优化通常涉及以递增的惩罚权重运行多次优化。缺乏耐心的工程师可能希望仅使用非常大的惩罚权重优化一次。请解释计数惩罚方法和二次惩罚方法各自遇到的问题。

11 Linear Constrained Optimization

线性规划涉及求解具有线性目标函数和线性约束的优化问题。许多问题自然可以用线性规划来描述，包括来自交通、通信网络、制造业、经济学和运筹学等各个领域的问题。许多本质上非线性的问题通常可以通过线性规划来近似。已经开发了几种方法来利用线性结构。现代技术和硬件能够全局最小化具有数百万个变量和数百万个约束的问题。[1]

1 本章简要介绍线性规划以及用于求解线性规划的一种单纯形算法变体。有多本教科书完全致力于线性规划，包括R. J. Vanderbei, *Linear Programming: Foundations and Extensions* , 4th ed. Springer, 2014。有多种用于求解线性规划的软件包，例如 `Convex.jl` 和 `JuMP.jl` ，两者都包含与开源和商业求解器的接口。

11.1 Problem Formulation

一个称为*linear program*、[2]的线性规划问题可以表示为几种形式。每个线性规划由一个线性目标函数和一组线性约束组成：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{w}_{\text{LE}}^{(i)\top} \mathbf{x} \leq b_i \text{ for } i \in \{1, 2, \ldots\} \\ & \mathbf{w}_{\text{GE}}^{(j)\top} \mathbf{x} \geq b_j \text{ for } j \in \{1, 2, \ldots\} \\ & \mathbf{w}_{\text{EQ}}^{(k)\top} \mathbf{x} = b_k \text{ for } k \in \{1, 2, \ldots\} \end{aligned} \tag{11.1}$$

2 *quadratic program*是线性规划的一种推广，其中目标函数是二次的，约束是线性的。解决这类问题的常见方法包括前几章讨论的一些算法，包括内点法、增广拉格朗日法和共轭梯度法。本章介绍的单纯形法也已被改编用于优化二次规划。J. Nocedal和S. J. Wright, *Numerical Optimization*，第2版，Springer，2006年。

其中 i、j 和 k 在有限的约束集合上变化。此类优化问题在例 11.1 中给出。将实际问题转化为这种数学形式通常并非易事。本文侧重于求解的算法，但其他文献讨论了如何对实际问题进行建模。[3] 若干有趣的

转换见例 11.2。

3 参见，例如，H. P. Williams, *Model Building in Mathematical Programming*，第5版，Wiley，2013年。

示例 11.1. 一个线性规划示例。

以下问题的目标函数和约束条件都是线性的，因此是一个线性规划问题。

$$
\begin{aligned}
\underset{x_1,x_2,x_3}{\text{minimize}} \quad & 2x_1 - 3x_2 + 7x_3 \\
\text{subject to} \quad & 2x_1 + 3x_2 - 8x_3 \leq 5 \\
& 4x_1 + x_2 + 3x_3 \leq 9 \\
& x_1 - 5x_2 - 3x_3 \geq -4 \\
& x_1 + x_2 + 2x_3 = 1
\end{aligned}
$$

例11.2. 可以转换为线性规划的常见范数最小化问题。

许多问题都可以转化为具有相同解的线性规划。两个例子是L_1和L_∞最小化问题：

$$\text{minimize} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_1 \qquad \text{minimize} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_\infty$$

第一个问题等价于求解

$$
\begin{aligned}
\underset{\mathbf{x},\mathbf{s}}{\text{minimize}} \quad & \mathbf{1}^\top \mathbf{s} \\
\text{subject to} \quad & \mathbf{A}\mathbf{x} - \mathbf{b} \leq \mathbf{s} \\
& \mathbf{A}\mathbf{x} - \mathbf{b} \geq -\mathbf{s}
\end{aligned}
$$

带有额外变量s。

第二个问题等价于求解

$$
\begin{aligned}
\underset{\mathbf{x},t}{\text{minimize}} \quad & t \\
\text{subject to} \quad & \mathbf{A}\mathbf{x} - \mathbf{b} \leq t\mathbf{1} \\
& \mathbf{A}\mathbf{x} - \mathbf{b} \geq -t\mathbf{1}
\end{aligned}
$$

带有额外变量 t。

11.1.1 General Form

我们可以使用矩阵更紧凑地编写线性规划，并得到 *general form*:[4]

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}_{\text{LE}}\mathbf{x} \leq \mathbf{b}_{\text{LE}} \\ & \mathbf{A}_{\text{GE}}\mathbf{x} \geq \mathbf{b}_{\text{GE}} \\ & \mathbf{A}_{\text{EQ}}\mathbf{x} = \mathbf{b}_{\text{EQ}} \end{aligned} \tag{11.2}$$

[4] 这里，每个约束都是逐元素的。例如，在写作

$$\mathbf{a} \leq \mathbf{b},$$

我们指的是所有i的$a_i \leq b_i$。

11.1.2 Standard Form

(11.2) 式中给出的一般线性规划可以转换为 *standard form*，其中所有约束都是小于等于不等式，且设计变量非负。

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned} \tag{11.3}$$

大于不等式反转，等式约束拆分为两个

$$\begin{aligned} \mathbf{A}_{\text{GE}}\mathbf{x} \geq \mathbf{b}_{\text{GE}} \quad &\rightarrow \quad -\mathbf{A}_{\text{GE}}\mathbf{x} \leq -\mathbf{b}_{\text{GE}} \\ \mathbf{A}_{\text{EQ}}\mathbf{x} = \mathbf{b}_{\text{EQ}} \quad &\rightarrow \quad \begin{cases} \mathbf{A}_{\text{EQ}}\mathbf{x} \leq \mathbf{b}_{\text{EQ}} \\ -\mathbf{A}_{\text{EQ}}\mathbf{x} \leq -\mathbf{b}_{\text{EQ}} \end{cases} \end{aligned} \tag{11.4}$$

我们要确保所有 x 项也都是非负的。假设我们从一个线性规划开始，其中 x 不受非负约束：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} \leq \mathbf{b} \end{aligned} \tag{11.5}$$

我们将 x 替换为 $x^+ - x^-$ 并约束 $x^+ \geq 0$ 和 $x^- \geq 0$:

$$\begin{aligned} \underset{\mathbf{x}^+,\mathbf{x}^-}{\text{minimize}} \quad & \begin{bmatrix}\mathbf{c}^\top & -\mathbf{c}^\top\end{bmatrix}\begin{bmatrix}\mathbf{x}^+\\ \mathbf{x}^-\end{bmatrix} \\ \text{subject to} \quad & \begin{bmatrix}\mathbf{A} & -\mathbf{A}\end{bmatrix}\begin{bmatrix}\mathbf{x}^+\\ \mathbf{x}^-\end{bmatrix} \leq \mathbf{b} \\ & \begin{bmatrix}\mathbf{x}^+\\ \mathbf{x}^-\end{bmatrix} \geq \mathbf{0} \end{aligned} \tag{11.6}$$

线性目标函数$c^\top x$形成了一个平坦的斜坡。函数沿c的方向增加，因此，如图11.1所示，所有等高线都垂直于c且彼此平行。

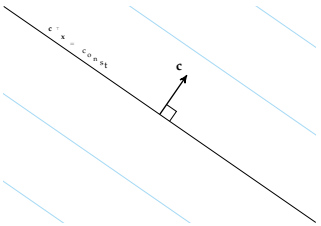


图11.1. 线性目标函数 $c^\top x$ 的等值线，沿 c 方向递增。

一个不等式约束 $w^\top x \leq b$ 构成一个 *half-space*，或超平面一侧的区域。该超平面垂直于 w，并由 $w^\top x = b$ 定义，如图 11.2 所示。区域 $w^\top x > b$ 位于超平面的 +w 侧，而 $w^\top x < b$ 位于超平面的 −w 侧。

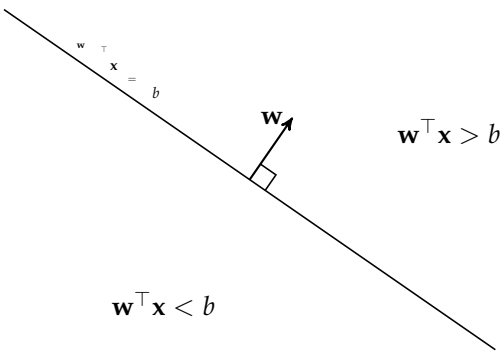


图11.2. 线性约束。

半空间是凸集（见附录C.3），而凸集的交集也是凸集，如图11.3所示。因此，线性规划的可行域总是构成一个凸集。可行域的凸性，加之目标函数的凸性，意味着一旦找到局部可行极小值，它同时也是全局可行极小值。

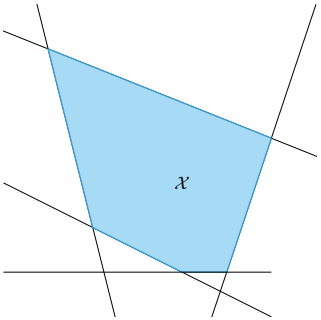


图11.3. 线性约束的交集是一个凸集。

可行集是由平坦面围成的凸区域。根据该区域的配置，解可能位于一个顶点、一条边上，或整个面上。如果问题没有适当的约束，解可能是无界的，

并且，如果系统为*over-constrained*，则不存在可行解。图11.4中展示了几个这样的情况。

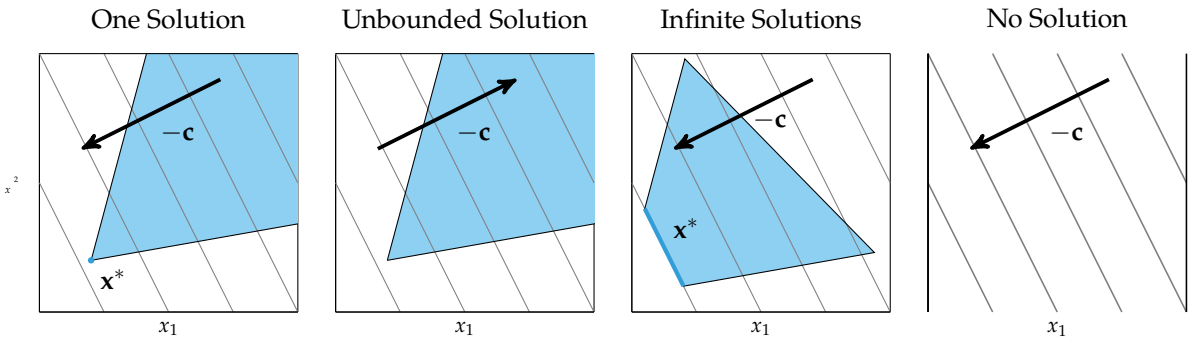


图11.4. 几种不同的线性问题形式及其不同的解。

11.1.3 *Equality Form*

线性规划通常以第三种形式求解，即*equality form*

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned} \tag{11.7}$$

其中 x 和 c 各有 n 个分量，A 是一个 $m \times n$ 矩阵，b 有 m 个分量。换句话说，我们有 n 个非负设计变量和一个由 m 个方程定义的等式约束系统。

等式形式有两部分约束。第一部分，Ax = b，强制解位于仿射子空间内。[5] 这样的约束很方便，因为
搜索技术可以将自身限制在约束仿射子空间内以保持可行性。约束的第二部分要求 x ≥ 0，这迫使解位于正象限内。因此，可行集是仿射子空间的非负部分。例11.3为一个简单的线性规划提供了可视化。

[5] 非正式地说，一个 *affine subspace* 是一个已被平移的向量空间，使得其在更高维空间中的原点不一定是0。

标准形式的任何线性规划都可以转换为等式形式。约束条件被转换为：

$$\mathbf{A}\mathbf{x} \leq \mathbf{b} \quad \rightarrow \quad \mathbf{A}\mathbf{x} + \mathbf{s} = \mathbf{b}, \quad \mathbf{s} \geq \mathbf{0} \tag{11.8}$$

考虑标准形式的线性规划：

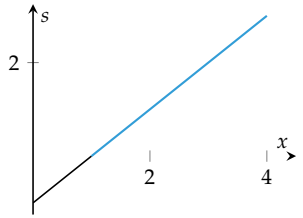
$$\begin{aligned} \underset{x}{\text{minimize}} \quad & x \\ \text{subject to} \quad & x \geq 1 \end{aligned}$$

当我们将其转换为等式形式时，我们得到

$$\begin{aligned} \underset{x,s}{\text{minimize}} \quad & x \\ \text{subject to} \quad & x - s = 1 \\ & x, s \geq 0 \end{aligned}$$

等式约束要求可行点落在直线 $x - s = 1$ 上。这条直线是二维欧几里得空间中的一维仿射子空间。

例11.3。等式形式的可行集是超平面。



通过引入*slack variables* s。这些变量占用额外的松弛量以强制实现等式。

从一个线性规划开始：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned} \tag{11.9}$$

我们引入松弛变量：

$$\begin{aligned} \underset{\mathbf{x},\mathbf{s}}{\text{minimize}} \quad & \begin{bmatrix} \mathbf{c}^\top & \mathbf{0}^\top \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{s} \end{bmatrix} \\ \text{subject to} \quad & \begin{bmatrix} \mathbf{A} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{s} \end{bmatrix} = \mathbf{b} \\ & \begin{bmatrix} \mathbf{x} \\ \mathbf{s} \end{bmatrix} \geq \mathbf{0} \end{aligned} \tag{11.10}$$

示例 11.4 演示了从标准形式转换为等式形式。

Consider the linear program

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & 5x_1 + 4x_2 \\ \text{subject to} \quad & 2x_1 + 3x_2 \leq 5 \\ & 4x_1 + x_2 \leq 11 \end{aligned}$$

To convert to equality form, we first introduce two slack variables:

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & 5x_1 + 4x_2 \\ \text{subject to} \quad & 2x_1 + 3x_2 + s_1 = 5 \\ & 4x_1 + x_2 + s_2 = 11 \\ & s_1, s_2 \geq 0 \end{aligned}$$

We then split $\mathbf{x}$:

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & 5(x_1^+ - x_1^-) + 4(x_2^+ - x_2^-) \\ \text{subject to} \quad & 2(x_1^+ - x_1^-) + 3(x_2^+ - x_2^-) + s_1 = 5 \\ & 4(x_1^+ - x_1^-) + (x_2^+ - x_2^-) + s_2 = 11 \\ & x_1^+, x_1^-, x_2^+, x_2^-, s_1, s_2 \geq 0 \end{aligned}$$

示例 11.4. 将线性规划转换为等式形式。

11.2 Simplex Algorithm

*simplex algorithm*通过从可行集的一个顶点移动到另一个顶点来求解线性规划问题。[6]该方法保证能找到最优解，因此

只要线性规划可行且有界。

单纯形算法作用于等式形式的线性规划（$\text{Ax} = \text{b}, \text{x} \geq 0$）。我们假设A的行是

Working Draft

线性无关的。[7] 我们还假设

该问题的等式约束数量不超过设计变量数量（$m \leq n$），这确保了问题不会过度约束。预处理阶段保证A满足这些条件。

6 单纯形算法最初由 George Dantzig 于 20 世纪 40 年代开发。有关其发展历史的资料可见：G. B. Dantzig，“单纯形法的起源”，载于 *A History of Sci-*，S. G. Nash 编，ACM，1990 年，第 141–151 页。

7 行线性无关的矩阵称为满行秩。线性无关是通过移除冗余的等式约束来实现的。

11.2.1 *Vertices*

等式形式的线性规划具有凸*polytopes*形式的可行集，这些是拥有平坦面的几何对象。这些多面体由等式约束与正象限的交集形成。与多面体相关的是*vertices*，即可行集中不位于任何其他可行集点之间的点。

可行集由几种不同类型的设计点组成。内部点永远不是最优的，因为可以通过沿 $-c$ 移动来改进。面上的点只有在面垂直于 c 时才可能最优。不垂直于 c 的面上的点可以通过沿 $-c$ 在面上的投影方向在面内滑动来改进。类似地，边上的点只有在边垂直于 c 时才可能最优，否则可以通过沿 $-c$ 在边上的投影滑动来改进。最后，顶点也可能最优。

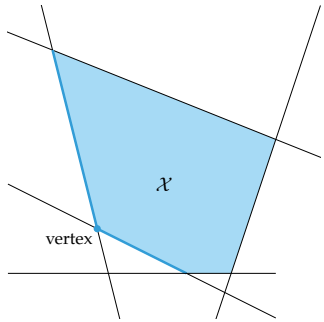


图11.5。在二维空间中，任何顶点至少会有两个活跃约束。

单纯形算法产生一个最优顶点。如果一个线性规划包含可行点，它也至少包含一个顶点。此外，如果一个线性规划有解，那么至少有一个解位于一个顶点上。在整个边或面都是最优的情况下，顶点解与其他任何解一样好。

等式形式的线性规划的每个顶点都可以由 x 中等于零的 $n - m$ 个分量唯一定义。这些分量受到 $x_i \geq 0$ 的积极约束。图 11.5 可视化了识别一个顶点所需的积极约束。

当A为方阵时，等式约束$\mathrm{A}\mathbf{x} = \mathrm{b}$具有唯一解。我们假定$m \leq n$，因此选择$m$个设计变量并将其余变量置零，实际上移除了A中的$n - m$列，从而得到一个$m \times m$约束矩阵（参见例11.5）。

任何顶点的分量 $\{1, \ldots, n\}$ 的索引可以划分为两个集合 $\mathcal{B}$ 和 $\mathcal{V}$，使得：

- $\mathcal{V}$ 中索引对应的设计值为零：

$$i \in \mathcal{V} \implies x_i = 0 \tag{11.11}$$

- 与$\mathcal{B}$中索引相关的设计值可能为零，也可能不为零：

$$i \in \mathcal{B} \implies x_i \geq 0 \tag{11.12}$$

- $\mathcal{B}$ 恰好有 m 个元素，且 $\mathcal{V}$ 恰好有 $n - m$ 个元素。

对于含有5个设计变量和3个约束的问题，将2个变量置零唯一确定了一个点。

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \\ a_{21} & a_{22} & a_{23} & a_{24} & a_{25} \\ a_{31} & a_{32} & a_{33} & a_{34} & a_{35} \end{bmatrix} \begin{bmatrix} x_1 \\ 0 \\ x_3 \\ x_4 \\ 0 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{13} & a_{14} \\ a_{21} & a_{23} & a_{24} \\ a_{31} & a_{33} & a_{34} \end{bmatrix} \begin{bmatrix} x_1 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

例11.5。将x的$n - m$分量置零可以唯一定义一个点。

我们用$\mathbf{x}_\mathcal{B}$表示由x中属于$\mathcal{B}$的分量组成的向量，用$\mathbf{x}_\mathcal{V}$表示由x中属于$\mathcal{V}$的分量组成的向量。注意$\mathbf{x}_\mathcal{V} = 0$。

与划分$(\mathcal{B}, \mathcal{V})$相关联的顶点可以通过使用由$\mathcal{B}$选出的A的$m$列形成的$m \times m$矩阵$\mathbf{A}_\mathcal{B}$获得：[8]

$$\mathbf{A}\mathbf{x} = \mathbf{A}_\mathcal{B}\mathbf{x}_\mathcal{B} = \mathbf{b} \quad \rightarrow \quad \mathbf{x}_\mathcal{B} = \mathbf{A}_\mathcal{B}^{-1}\mathbf{b} \tag{11.13}$$

已知 $\mathbf{x}_\mathcal{B}$ 就足以构造 x；其余设计变量为零。算法 11.1 实现了此过程，示例 11.6 演示了验证给定设计点是否为顶点。

8 如果$\mathcal{B}$和$\mathcal{V}$确定一个顶点，那么$\mathbf{A}_\mathcal{B}$的列必须线性无关，因为$\mathbf{A}\mathbf{x} = \mathbf{b}$必须恰有一个解。因此，$\mathbf{A}_\mathcal{B}\mathbf{x}_\mathcal{B} = \mathbf{b}$必须恰有一个解。这种线性无关性保证了$\mathbf{A}_\mathcal{B}$是可逆的。

```
mutable struct LinearProgram
    A
    b
    c
end
function get_vertex(B, LP)
    A, b, c = LP.A, LP.b, LP.c
    b_inds = sort!(collect(B))
    AB = A[:,b_inds]
    xB = AB\b
    x = zeros(length(c))
    x[b_inds] = xB
    return x
end
```

算法11.1. 一种提取与分区B和等式形式的线性规划LP相关的顶点的方法。我们为线性规划引入了特殊类型LinearProgram。

虽然每个顶点都有一个关联的划分（$\mathcal{B}$，$\mathcal{V}$），但并非每个划分都对应一个顶点。一个划分只有当$A_{\mathcal{B}}$是非奇异的，并且通过应用方程（11.13）获得的设计是可行的时，才对应一个顶点。[9] 识别配对

对应于顶点的划分是非平凡的，我们在11.2.4节中展示，寻找这样的划分涉及求解一个线性规划！单纯形算法分两个阶段运行——一个*initialization phase*用于识别顶点划分，一个*optimization phase*用于在顶点划分之间转换，朝向对应于最优顶点的划分。我们将在本节后面讨论这两个阶段。

9 例如，对于约束 $\mathcal{B} = \{1, 2\}$

$$\begin{bmatrix} 1 & 2 & 0 \\ 1 & 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

对应于

$$\begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

这不会产生可逆的$A_{\mathcal{B}}$，并且没有唯一解。

示例11.6。验证设计点是否等式形式约束的顶点。

考虑以下约束条件：

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & -1 & 2 & 3 \\ 2 & 1 & 2 & -1 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}, \quad \mathbf{x} \geq \mathbf{0}$$

考虑设计点 x = [1, 1, 0, 0]。我们可以验证 x 是可行的，并且其非零分量不超过三个。我们可以选择 $\mathcal{B} = \{1, 2, 3\}$ 或 $\mathcal{B} = \{1, 2, 4\}$。两者

$$\mathbf{A}_{\{1,2,3\}} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & -1 & 2 \\ 2 & 1 & 2 \end{bmatrix}$$

和

$$\mathbf{A}_{\{1,2,4\}} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & -1 & 3 \\ 2 & 1 & -1 \end{bmatrix}$$

–可逆。因此，x 是可行集多面体的一个顶点 pe.

11.2.2 *First-Order Necessary Conditions*

最优性的一阶必要条件（FONCs）用于确定一个顶点何时为最优，并指导如何过渡到更有利的顶点。我们为线性规划的等式形式构造拉格朗日函数：[10]

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = \mathbf{c}^\top \mathbf{x} - \boldsymbol{\mu}^\top \mathbf{x} - \boldsymbol{\lambda}^\top (\mathbf{A}\mathbf{x} - \mathbf{b}) \tag{11.14}$$

10 注意在 $\mathbf{x} \geq 0$ 中，必须通过两边乘以 -1 来反转不等式的极性，从而在 $\boldsymbol{\mu}$ 前面产生负号。拉格朗日量可以用正 $\boldsymbol{\lambda}$ 或负 $\boldsymbol{\lambda}$ 来定义。

具有以下 FONC：

1. 可行性：$Ax = b, x \geq 0$

2. 对偶可行性：$\mu \geq 0$

3. 互补松弛性：$\mu \odot x = 0$

4. 平稳性：$A^\top \lambda + \mu = c$

FONC 是线性规划最优性的充分条件。因此，如果对于给定顶点能够计算出 $\{v^*\}$ 并且四个 FONC 方程均得到满足，则该顶点是最优的。

我们可以将平稳性条件分解为 $\mathcal{B}$ 和 $\mathcal{V}$ 分量：

$$\mathbf{A}^\top \boldsymbol{\lambda} + \boldsymbol{\mu} = \mathbf{c} \quad \rightarrow \quad \begin{cases} \mathbf{A}_\mathcal{B}^\top \boldsymbol{\lambda} + \boldsymbol{\mu}_\mathcal{B} = \mathbf{c}_\mathcal{B} \\ \mathbf{A}_\mathcal{V}^\top \boldsymbol{\lambda} + \boldsymbol{\mu}_\mathcal{V} = \mathbf{c}_\mathcal{V} \end{cases} \tag{11.15}$$

我们可以选择 $\boldsymbol{\mu}_\mathcal{B} = 0$ 来满足互补松弛性。$\boldsymbol{\lambda}$ 的值可以从 $\mathcal{B}$:[11] 计算得出。

$$\begin{aligned} \mathbf{A}_\mathcal{B}^\top \boldsymbol{\lambda} + \underbrace{\boldsymbol{\mu}_\mathcal{B}}_{=\mathbf{0}} &= \mathbf{c}_\mathcal{B} \\ \boldsymbol{\lambda} &= \mathbf{A}_\mathcal{B}^{-\top} \mathbf{c}_\mathcal{B} \end{aligned} \tag{11.16}$$

我们可以使用这个来获得

$$\begin{aligned} \mathbf{A}_\mathcal{V}^\top \boldsymbol{\lambda} + \boldsymbol{\mu}_\mathcal{V} &= \mathbf{c}_\mathcal{V} \\ \boldsymbol{\mu}_\mathcal{V} &= \mathbf{c}_\mathcal{V} - \mathbf{A}_\mathcal{V}^\top \boldsymbol{\lambda} \\ \boldsymbol{\mu}_\mathcal{V} &= \mathbf{c}_\mathcal{V} - \left(\mathbf{A}_\mathcal{B}^{-1} \mathbf{A}_\mathcal{V}\right)^\top \mathbf{c}_\mathcal{B} \end{aligned} \tag{11.17}$$

知道 $\boldsymbol{\mu}_\mathcal{V}$ 让我们能够评估顶点的最优性。如果 $\boldsymbol{\mu}_\mathcal{V}$ 包含负分量，则对偶可行性不满足，该顶点是次优的。

11 我们用 $A^{-\top}$ 来表示 A 的逆的转置：

$$\mathbf{A}^{-\top} = \left(\mathbf{A}^{-1}\right)^\top = \left(\mathbf{A}^\top\right)^{-1}$$

11.2.3 *Optimization Phase*

单纯形算法维护一个划分 $(\mathcal{B}, \mathcal{V})$，它对应于可行集多面体的一个顶点。该划分可以通过在 $\mathcal{B}$ 和 $\mathcal{V}$ 之间交换索引来更新。这种交换等同于沿着可行集多面体的一条边从一个顶点移动到另一个顶点。如果初始划分对应于一个顶点且问题有界，则单纯形算法保证收敛到最优解。

顶点之间的转移 $\mathrm{x} \to \mathrm{x}'$ 必须满足 $\mathrm{Ax}' = \mathrm{b}$。从一个由 $\mathcal{B}$ 定义的分区开始，我们选择 *entering index* $q \in \mathcal{V}$ 进入 $\mathcal{B}$，使用本节末尾描述的启发式方法之一。新顶点 x' 必须满足：

$$\mathbf{A}\mathbf{x}' = \mathbf{A}_{\mathcal{B}}\mathbf{x}'_{\mathcal{B}} + \mathbf{A}_{\{q\}}x'_q = \mathbf{A}_{\mathcal{B}}\mathbf{x}_{\mathcal{B}} = \mathbf{A}\mathbf{x} = \mathbf{b} \tag{11.18}$$

在$x'_{\mathcal{B}}$中的一个*leaving index* $p \in \mathcal{V}$在过渡期间变为零，并被A中与索引q对应的列所替换。此操作称为*pivoting*。

我们可以求解新的设计点

$$\mathbf{x}'_{\mathcal{B}} = \mathbf{x}_{\mathcal{B}} - \mathbf{A}_{\mathcal{B}}^{-1}\mathbf{A}_{\{q\}}x'_q \tag{11.19}$$

一个特定的离开索引 $p \in \mathcal{B}$ 在以下情况下变得活跃：

$$\left(\mathbf{x}'_{\mathcal{B}}\right)_p = 0 = \left(\mathbf{x}_{\mathcal{B}}\right)_p - \left(\mathbf{A}_{\mathcal{B}}^{-1}\mathbf{A}_{\{q\}}\right)_p x'_q \tag{11.20}$$

并且因此通过将 $x_q = 0$ 增加到 x'_q 来获得，其中：

$$x'_q = \frac{\left(\mathbf{x}_{\mathcal{B}}\right)_p}{\left(\mathbf{A}_{\mathcal{B}}^{-1}\mathbf{A}_{\{q\}}\right)_p} \tag{11.21}$$

离开索引是通过*minimum ratio test*获得的，它针对每个可能的离开索引进行计算，并选择x'_q最小的一个。随后，我们在$\mathcal{B}$和$\mathcal{V}$之间交换p和q。边缘转换在算法11.2中实现。

```
function edge_transition(LP, B, q)
    A, b, c = LP.A, LP.b, LP.c
    n = size(A, 2)
    b_inds = sort(B)
    n_inds = sort!(setdiff(1:n, B))
    AB = A[:,b_inds]
    d, xB = AB\A[:,n_inds[q]], AB\b

    p, xq′ = 0, Inf
    for i in 1 : length(d)
        if d[i] > 0
            v = xB[i] / d[i]
            if v < xq′
                p, xq′ = i, v
            end
        end
    end
    return (p, xq′)
end
```

算法11.2。一种计算等式形式线性规划 `LP` 中由划分 `B` 定义的顶点的索引 p 及通过增加索引 `q` 得到的新坐标值 x'_q 的方法。

变换对目标函数的影响可以使用 x'_q 计算。新顶点处的目标函数值为[12]

$$\begin{aligned}
\mathbf{c}^\top \mathbf{x}' &= \mathbf{c}_\mathcal{B}^\top \mathbf{x}'_\mathcal{B} + c_q x'_q && (11.22)\\
&= \mathbf{c}_\mathcal{B}^\top \left(\mathbf{x}_\mathcal{B} - \mathbf{A}_\mathcal{B}^{-1}\mathbf{A}_{\{q\}} x'_q\right) + c_q x'_q && (11.23)\\
&= \mathbf{c}_\mathcal{B}^\top \mathbf{x}_\mathcal{B} - \mathbf{c}_\mathcal{B}^\top \mathbf{A}_\mathcal{B}^{-1}\mathbf{A}_{\{q\}} x'_q + c_q x'_q && (11.24)\\
&= \mathbf{c}_\mathcal{B}^\top \mathbf{x}_\mathcal{B} - \left(c_q - \mu_q\right) x'_q + c_q x'_q && (11.25)\\
&= \mathbf{c}^\top \mathbf{x} + \mu_q x'_q && (11.26)
\end{aligned}$$

12 这里，我们利用了 $\boldsymbol{\lambda} = \mathrm{A}_\mathcal{B}^{-\top} \mathrm{c}_\mathcal{B}$ 以及 $\mathrm{A}_{\{q\}}^\top \boldsymbol{\lambda} = c_q - \mu_q$ 这一事实。

选择进入索引q会使目标函数值减少

$$\mathbf{c}^\top \mathbf{x}' - \mathbf{c}^\top \mathbf{x} = \mu_q x'_q \tag{11.27}$$

目标函数只有在 μ_q 为负时才会减少。为了向最优解推进，我们必须在 $\mathcal{V}$ 中选择一个索引 q，使得 $\boldsymbol{\mu}_q$ 为负。如果 $\boldsymbol{\mu}_\mathcal{V}$ 的所有分量都为正，我们就找到了全局最优解。

由于 $\boldsymbol{\mu}_\mathcal{V}$ 中可能存在多个负项，可以使用不同的启发式方法来选择入基索引:[13]

13 现代实现使用更复杂的规则。例如，参见 J. J. Forrest 和 D. Goldfarb，“线性规划的最速边单纯形算法”，*Mathematical Programming*，第 57 卷，第 1 期，第 341–374 页，1992 年。

- *Greedy heuristic*，它选择一个 q 以最大程度地减少 $c^\top x$。
- *Dantzig's rule*它选择μ中负项最多的q。这个规则易于计算，但它不能保证$c^\top x$的最大减少量。它对约束的缩放也很敏感。[14]
- *Bland's rule*，它选择μ中第一个具有负值的q。单独使用时，Bland规则在实践中往往导致性能不佳。然而，这一规则可以帮助我们防止*cycles*的发生，即当我们回到之前访问过的顶点而没有减少目标函数时。此规则通常仅在采用另一种规则连续多次迭代未取得改进后使用，以打破循环并确保收敛。

14 对于约束 $A^\top x = b \to \alpha A^\top x = \alpha b, \alpha > 0$，我们不改变解，但拉格朗日乘子被缩放，$\lambda \to \alpha^{-1}\lambda$。

单纯形法优化阶段的一次迭代根据进入索引的启发式规则将一个顶点划分移动到相邻顶点。算法11.3使用贪婪启发式实现了这样的迭代。例11.7演示了从已知顶点划分出发使用单纯形算法求解线性规划。

11.2.4 Initialization Phase

单纯形算法的*optimization phase*在算法11.4中实现。遗憾的是，算法11.4需要一个对应于顶点的初始划分。如果没有这样的划分，我们必须解决一个*auxiliary linear program*，作为*initialization phase*的一部分来获得这个划分。

初始化阶段要求解的辅助线性规划包含额外变量 $z \in \mathbb{R}^m$，我们的目标是将其置零：[15]

$$\begin{aligned} \underset{x,z}{\text{minimize}} \quad & \begin{bmatrix} 0^\top & 1^\top \end{bmatrix} \begin{bmatrix} x \\ z \end{bmatrix} \\ \text{subject to} \quad & \begin{bmatrix} A & Z \end{bmatrix} \begin{bmatrix} x \\ z \end{bmatrix} = b \\ & \begin{bmatrix} x \\ z \end{bmatrix} \geq 0 \end{aligned} \tag{11.28}$$

15 z 的值表示 $Ax = b$ 被违反的程度。通过将 z 置零，我们找到一个可行点。如果在求解辅助问题时，我们没有找到一个 z 被置零的顶点，那么我们可以得出结论：该问题是不可行的。此外，并不总是需要添加所有 m 个额外变量，特别是当在标准形式和等式形式之间转换时包含松弛变量的情况下。

例11.7. 使用单纯形算法求解线性规划。

考虑具有以下形式的等式线性规划

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 & 0 \\ -4 & 2 & 0 & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 9 \\ 2 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 3 \\ -1 \\ 0 \\ 0 \end{bmatrix}$$

以及由$\mathcal{B} = \{3, 4\}$定义的初始顶点。在验证$\mathcal{B}$定义了一个可行顶点后，我们可以开始单纯形算法的一次迭代。

我们提取$\mathrm{x}_{\mathcal{B}}$:

$$\mathbf{x}_{\mathcal{B}} = \mathbf{A}_{\mathcal{B}}^{-1}\mathbf{b} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 9 \\ 2 \end{bmatrix} = \begin{bmatrix} 9 \\ 2 \end{bmatrix}$$

然后我们计算$\boldsymbol{\lambda}$:

$$\boldsymbol{\lambda} = \mathbf{A}_{\mathcal{B}}^{-\top}\mathbf{c}_{\mathcal{B}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}^{-\top} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \mathbf{0}$$

和$\boldsymbol{\mu}_{\mathcal{V}}$:

$$\boldsymbol{\mu}_{\mathcal{V}} = \mathbf{c}_{\mathcal{V}} - \left(\mathbf{A}_{\mathcal{B}}^{-1}\mathbf{A}_{\mathcal{V}}\right)^{\top}\mathbf{c}_{\mathcal{B}} = \begin{bmatrix} 3 \\ -1 \end{bmatrix} - \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 \\ -4 & 2 \end{bmatrix}\right)^{\top} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

我们发现 $\boldsymbol{\mu}_{\mathcal{V}}$ 包含负元素，因此当前的 $\mathcal{B}$ 不是最优的。我们将在唯一负元素的索引 $q = 2$ 上进行主元操作。从 $\mathrm{x}_{\mathcal{B}}$ 沿着方向 $-\mathrm{A}_{\mathcal{B}}^{-1}\,\mathrm{A}_{\{q\}} = [1, 2]$ 执行边转移。

使用方程（11.19），我们增加 x'_q，直到一个新的约束变为活跃。在这种情况下，$x'_q = 1$ 导致 x_3 变为零。我们将基本索引集更新为 $\mathcal{B} = \{2, 3\}$。

在第二次迭代中，我们发现:

$$\mathbf{x}_{\mathcal{B}} = \begin{bmatrix} 1 \\ 8 \end{bmatrix}, \quad \boldsymbol{\lambda} = \begin{bmatrix} 0 \\ -1/2 \end{bmatrix}, \quad \boldsymbol{\mu}_{\mathcal{V}} = \begin{bmatrix} 1 \\ 1/2 \end{bmatrix}.$$

顶点是最优的，因为$\boldsymbol{\mu}_{\mathcal{V}}$没有负元素。因此，我们的算法以$\mathcal{B} = \{2, 3\}$终止，此时设计点为$\mathrm{x}^* = [0, 1, 8, 0]$。

```
function step_lp!(B, LP)
    A, b, c = LP.A, LP.b, LP.c
    n = size(A, 2)
    b_inds = sort!(B)
    n_inds = sort!(setdiff(1:n, B))
    AB, AV = A[:,b_inds], A[:,n_inds]
    xB = AB\b
    cB = c[b_inds]
    λ = AB' \ cB
    cV = c[n_inds]
    μV = cV - AV'*λ

    q, p, xq′, Δ = 0, 0, Inf, Inf
    for i in 1 : length(μV)
        if μV[i] < 0
            pi, xi′ = edge_transition(LP, B, i)
            if μV[i]*xi′ < Δ
                q, p, xq′, Δ = i, pi, xi′, μV[i]*xi′
            end
        end
       end
       if q == 0
        return (B, true) # optimal point found
    end

    if isinf(xq′)
        error("unbounded")
    end

    j = findfirst(isequal(b_inds[p]), B)
    B[j] = n_inds[q] # swap indices
    return (B, false) # new vertex but not optimal
end
```

算法11.3。单纯形算法的一次迭代，在此迭代中，集合$\mathcal{B}$从一个顶点移动到相邻顶点，同时使目标函数最大程度地减少。在此，`step_lp!`采用由`B`和线性规划`LP`定义的划分。

```
function minimize_lp!(B, LP)
    done = false
    while !done
        B, done = step_lp!(B, LP)
    end
    return B
end
```

算法11.4. 给定由`B`定义的顶点划分和一个线性规划`LP`，最小化一个线性规划。

其中 Z 是一个对角矩阵，其对角线元素为

$$Z_{ii} = \begin{cases} +1 & \text{if } b_i \geq 0 \\ -1 & \text{otherwise.} \end{cases} \tag{11.29}$$

辅助线性规划通过由 $\mathcal{B}$ 定义的分区求解，该分区仅选择 z 值。相应的顶点具有 $\mathrm{x} = 0$，并且每个 z 元素是对应 b 值的绝对值：$z_j = |b_j|$。这个初始顶点可以很容易地被证明是可行的。

示例11.8演示了使用辅助线性规划来获得一个可行顶点。

示例 11.8. 使用辅助线性规划获得可行顶点。

考虑等式形式的线性规划：

$$\begin{aligned} \underset{x_1,x_2,x_3}{\text{minimize}} \quad & c_1x_1 + c_2x_2 + c_3x_3 \\ \text{subject to} \quad & 2x_1 - 1x_2 + 2x_3 = 1 \\ & 5x_1 + 1x_2 - 3x_3 = -2 \\ & x_1, x_2, x_3 \geq 0 \end{aligned}$$

我们可以通过求解以下问题来确定一个可行顶点：

$$\begin{aligned} \underset{x_1,x_2,x_3,z_1,z_2}{\text{minimize}} \quad & z_1 + z_2 \\ \text{subject to} \quad & 2x_1 - 1x_2 + 2x_3 + z_1 = 1 \\ & 5x_1 + 1x_2 - 3x_3 - z_2 = -2 \\ & x_1, x_2, x_3, z_1, z_2 \geq 0 \end{aligned}$$

以初始顶点由 $\mathcal{B} = \{4, 5\}$ 定义。

初始顶点具有：

$$\mathbf{x}_{\mathcal{B}}^{(1)} = \mathbf{A}_{\mathcal{B}}^{-1}\mathbf{b}_{\mathcal{B}} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}^{-1} \begin{bmatrix} 1 \\ -2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

因此$\mathrm{x}(1) = [0, 0, 0, 1, 2]$。求解辅助问题得到$\mathrm{x}^* \approx [0.045, 1.713, 1.312, 0, 0]$。因此，[0.045, 1.713, 1.312]是原问题中的一个可行顶点。

通过求解辅助线性规划得到的分割将产生一个可行设计点 $Ax = b$，因为 z 将被归零。如果 z 非零，则原始线性规划不可行。如果 z 为零，则得到的分割可用作单纯形算法优化阶段的初始分割。原始问题必须稍作修改以纳入新的 z 变量：

$$\begin{aligned} \underset{\mathbf{x},\mathbf{z}}{\text{minimize}} \quad & \begin{bmatrix} \mathbf{c}^\top & \mathbf{0}^\top \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{z} \end{bmatrix} \\ \text{subject to} \quad & \begin{bmatrix} \mathbf{A} & \mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{z} \end{bmatrix} = \begin{bmatrix} \mathbf{b} \\ \mathbf{0} \end{bmatrix} \\ & \begin{bmatrix} \mathbf{x} \\ \mathbf{z} \end{bmatrix} \geq \mathbf{0} \end{aligned} \tag{11.30}$$

必须包含 z 值。尽管它们的向量对应项为零，但初始分区 $\mathcal{B}$ 中可能包含 z 的分量中的某些索引。可以检查初始分区并仅包含所需的特定分量。

通过求解第二个线性规划得到的解 (x^*, z^*) 将具有 $z^* = 0$。因此，x^* 将成为原始线性问题的解。

算法11.5给出了完整单纯形算法的一个实现。

11.3 *Dual Certificates*

假设我们有一个候选解，并希望验证其最优性。使用*dual certificates* (算法11.6)来验证最优性在许多情况下都很有用，例如调试我们的线性规划代码。

我们从约束优化的FONCs得知，对偶问题的最优值d^*是原问题最优值p^*的下界。线性规划是线性和凸的，可以证明对偶问题的最优值也是原问题的最优值，$d^* = p^*$。

```
function minimize_lp(LP)
    A, b, c = LP.A, LP.b, LP.c
    m, n = size(A)
    z = ones(m)
    Z = Matrix(Diagonal([j ≥ 0 ? 1 : -1 for j in b]))

    A′ = hcat(A, Z)
    b′ = b
    c′ = vcat(zeros(n), z)
    LP_init = LinearProgram(A′, b′, c′)
    B = collect(1:m).+n
    minimize_lp!(B, LP_init)

    if any(i-> i > n, B)
        error("infeasible")
    end

    A′′ = [A          Matrix(1.0I, m, m);
           zeros(m,n) Matrix(1.0I, m, m)]
    b′′ = vcat(b, zeros(m))
    c′′ = c′
    LP_opt = LinearProgram(A′′, b′′, c′′)
    minimize_lp!(B, LP_opt)
    return get_vertex(B, LP_opt)[1:n]
end
```

算法11.5. 当初始划分未知时，求解等式形式线性规划的单纯形算法。

原始线性规划可以转换为如下对偶形式：[16]

Primal Form (equality)	Dual Form
$\underset{\mathbf{x}}{\text{minimize}} \quad \mathbf{c}^\top \mathbf{x}$	$\underset{\boldsymbol{\mu}}{\text{maximize}} \quad \mathbf{b}^\top \boldsymbol{\mu}$
$\text{subject to} \quad \mathbf{A}\mathbf{x} = \mathbf{b}$	$\text{subject to} \quad \mathbf{A}^\top \boldsymbol{\mu} \leq \mathbf{c}$
$\mathbf{x} \geq \mathbf{0}$	

16 单纯形算法的一种替代方法，*self-dual simplex algorithm*，在实践中往往更快。它不要求矩阵 $A_\mathcal{B}$ 满足 $x_\mathcal{B} = A_\mathcal{B}^{-1} b \geq 0$。自对偶单纯形算法是对标准形式线性规划问题对偶的单纯形算法的一种修改。

如果原始问题有 n 个变量和 m 个等式约束，那么对偶问题有 m 个变量和 n 个约束。此外，对偶问题的对偶是原始问题。

最优性可以通过验证三个性质来评估。如果有人声称（x^*，$\boldsymbol{\mu}^*$）是最优的，我们可以通过检查以下三个条件是否全部满足来快速验证该声明：

1. x^*在原问题中可行。
2. $\boldsymbol{\mu}^*$ 在对偶问题中是可行的。
3. $p^* = \mathbf{c}^\top \mathbf{x}^* = \mathbf{b}^\top \boldsymbol{\mu}^* = d^*$.

对偶证书在示例 11.9 中用于验证线性规划的解。

```
function dual_certificate(LP, x, μ, ϵ=1e-6)
    A, b, c = LP.A, LP.b, LP.c
    primal_feasible = all(x .≥ 0) && A*x ≈ b
    dual_feasible = all(A'*μ .≤ c)
    return primal_feasible && dual_feasible &&
           isapprox(c⋅x, b⋅μ, atol=ϵ)
end
```

算法11.6. 一种检查由设计点x和对偶点μ给出的线性规划问题LP的等式形式候选解是否最优的方法。参数ϵ控制等式约束的容差。

考虑标准形式的线性规划，其中

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & -1 \\ -1 & 2 & 0 \\ 1 & 2 & 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ -2 \\ 5 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$$

我们想要确定$x^* = [2, 0, 1]$和$\boldsymbol{\mu}^* = [1, 0, 0]$是否为一个最优解对。我们首先验证$x^*$是可行的：

$$\mathbf{A}\mathbf{x}^* = [1, -2, 5] = \mathbf{b}, \qquad \mathbf{x}^* \geq \mathbf{0}$$

然后我们验证$\boldsymbol{\mu}^*$是对偶可行的：

$$\mathbf{A}^\top \boldsymbol{\mu}^* \approx [1, 1, -1] \leq \mathbf{c}$$

最后，我们验证 p^* 和 d^* 是相同的：

$$p^* = \mathbf{c}^\top \mathbf{x}^* = 1 = \mathbf{b}^\top \boldsymbol{\mu}^* = d^*$$

我们得出结论，$(x^*, \boldsymbol{\mu}^*)$ 是最优的。

示例 11.9. 使用双重证书验证解。

11.4 Summary

- 线性规划是由线性目标函数和线性约束条件组成的问题。
- 单纯形算法能够以高效的方式全局优化线性规划问题。
- 对偶证书使我们能够验证候选的原始-对偶解对是否最优。

11.5 Exercises

练习 11.1. 假设你不知道任何求解线性规划的优化算法。你决定评估所有顶点，并通过检查确定哪一个使目标函数最小。给出你将检查的可能最小化点的数量的宽松上界。此外，这种方法是否妥善处理所有线性约束优化问题?

练习 11.2. 如果例11.1中的程序下方有界，论证单纯形法必然收敛。

练习 11.3. 假设我们想要最小化 $6x_1 + + 5x_2$，受约束条件 $3x_1 - + 2x_2 \geq = 5$。我们如何将这个问题转换为具有相同最小化点的等式形式的线性规划?

练习 11.4。假设你的优化算法找到了一个搜索方向 d，你想进行线搜索。然而，你已知存在一个线性约束 $w^\top x \geq 0$。你会如何修改线搜索以考虑这一约束？可以假设你当前的设计点是可行的。

练习11.5。重新表述线性规划

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} \geq \mathbf{0} \end{aligned} \tag{11.31}$$

转化为带有对数障碍罚项的无约束优化问题。

12 Multiobjective Optimization

前几章介绍了优化单目标函数的方法，但本章关注的是*multiobjective optimization*，即*vector op-timization*，其中我们必须同时针对多个目标进行优化。工程往往是在成本、性能和上市时间之间进行权衡，而且通常不清楚如何确定不同目标的优先级。我们将讨论将向量值目标函数转换为标量值目标函数的各种方法，以便我们可以使用前几章讨论的算法来达到最优解。此外，我们还将讨论用于识别代表目标之间最佳权衡的设计点集合的算法，而无需承诺对目标进行特定的优先级排序。然后，这些设计点可以呈现给专家，由他们识别出最理想的设计。[1]

12.1 Pareto Optimality

Pareto optimality 的概念在讨论存在多个目标的问题时很有用。如果一个设计在不使至少一个其他目标变差的情况下无法在某个目标上得到改进，那么它就是帕累托最优的。在多目标设计优化中，我们通常可以将精力集中在帕累托最优的设计上，而不必在目标之间承诺某种特定的权衡。本节介绍一些在讨论识别帕累托最优设计的方法时有帮助的定义和概念。

1 其他方法在 R. T. Marler 和 J. S. Arora 的‘工程多目标优化方法综述’中进行了调查，*Structural and Multidisciplinary Optimization*，第 26 卷，第 6 期，第 369-395 页，2004 年。关于完全致力于多目标优化的教科书，请参见 K. Miettinen，*Nonlinear*。Kluwer Academic Publishers，1999 年。

12.1.1 Dominance

在单目标优化中，两个设计点 x 和 x′ 可以根据它们的标量函数值进行客观排序。当 $f(\mathbf{x}')$ 小于 $f(\mathbf{x})$ 时，点 x′ 更优。

在多目标优化中，当在设计点 x 处评估时，我们的目标函数 f 返回一个 m 维的值向量 y。y 的不同维度对应于不同的目标，有时也称为指标或准则。只有当某个设计点在至少一个目标上更好且在其他任何目标上都不差时，我们才能客观地对两个设计点 x 和 x′ 进行排序。也就是说，x *dominates* x′ 当且仅当

$$\begin{aligned} f_i(\mathbf{x}) &\leq f_i(\mathbf{x}') \text{ for all } i \text{ in } \{1, \ldots, m\} \\ \text{and } f_i(\mathbf{x}) &< f_i(\mathbf{x}') \text{ for some } i \end{aligned} \tag{12.1}$$

如算法 12.1 中紧凑实现的那样。

```
dominates(y, y′) = all(y .≤ y′) && any(y .< y′)
```

算法12.1. 一种检查x是否支配x′的方法，其中y是f(x)的目标值向量，y′是f(x′)的目标值向量。

图12.1显示，在多维度中存在支配模糊性的区域。这种模糊性出现在当一个解在某些目标上更优而另一个解x′在其他目标上更优时。解决这些模糊性的方法有多种。

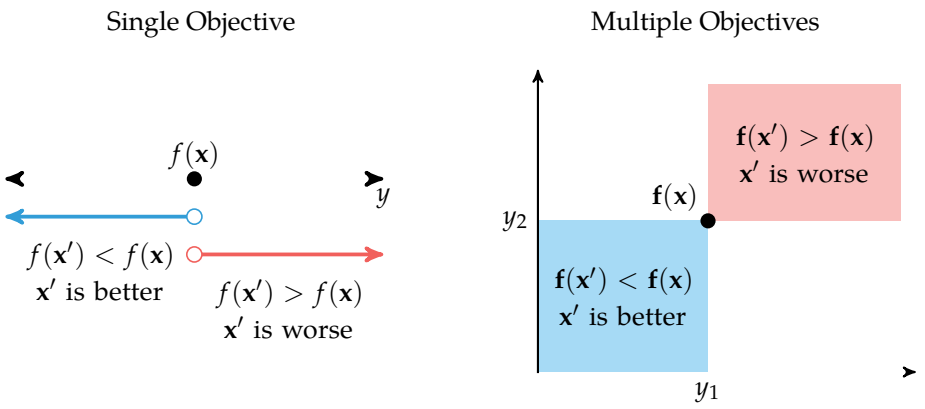


图12.1. 在单目标优化中，设计点可以客观排序，但在多目标优化中，只有在某些情况下设计点才能被客观排序。

12.1.2 *Pareto Frontier*

在数学中，输入集通过某个函数的*image*是该函数在输入集元素上求值时所有可能输出的集合。我们将$\mathcal{X}$通过f的像表示为$\mathcal{Y}$，并将$\mathcal{Y}$称为*criterion space*。图12.2展示了单目标和多目标问题的判据空间示例。如图所示，单目标优化中的判据空间是一维的。所有全局最优解共享单一的目标函数值y^*。在多目标优化中，判据空间是m维的，其中m是目标的数量。通常不存在全局最佳的目标函数值，因为当目标之间的权衡未明确指定时可能存在模糊性。

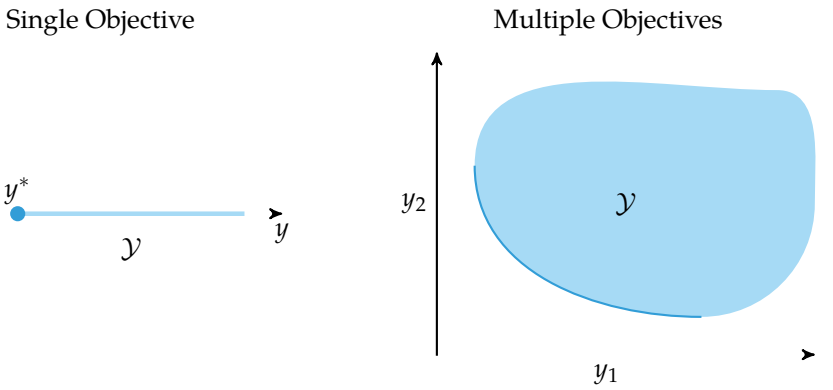


图12.2. 准则空间是由可行设计点获得的所有目标值集合。适定问题的准则空间从下方有界，但不必从上方有界。帕累托前沿以深蓝色高亮显示。

在多目标优化中，我们可以定义*Pareto optimality*的概念。当没有点支配设计点x时，x就是帕累托最优的。也就是说，如果不存在$\mathbf{x}' \in \mathcal{X}$使得$\mathbf{x}'$支配x，那么$\mathbf{x} \in \mathcal{X}$就是帕累托最优的。帕累托最优点构成的集合称为*Pareto frontier*。如例12.1所述，帕累托前沿对于帮助决策者做出设计权衡决策非常有价值。在二维情况下，帕累托前沿也称为*Pareto curve*。

所有帕累托最优点位于准则空间的边界上。一些多目标优化方法也能找到*weakly Pareto-optimal*点。而帕累托最优点是指那些没有任何其他点能在至少一个目标上改进的点。

jective，弱帕累托最优点是指不存在其他点能改进所有目标的点（图12.3）。也就是说，如果不存在一个$x' \in \mathcal{X}$使得$f(x') < f(x)$，那么$x \in \mathcal{X}$是弱帕累托最优的。帕累托最优点也是弱帕累托最优的。弱帕累托最优点不一定是帕累托最优的。

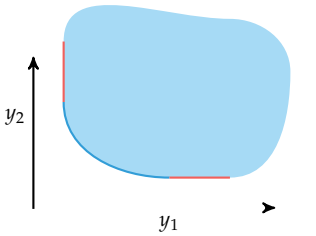


图12.3. 弱帕累托最优点，以红色显示，无法在所有目标上同时得到改进。

下面讨论的几种方法使用了另一个特殊点。我们将*utopia point*定义为准则空间中由各分量最优值组成的点：

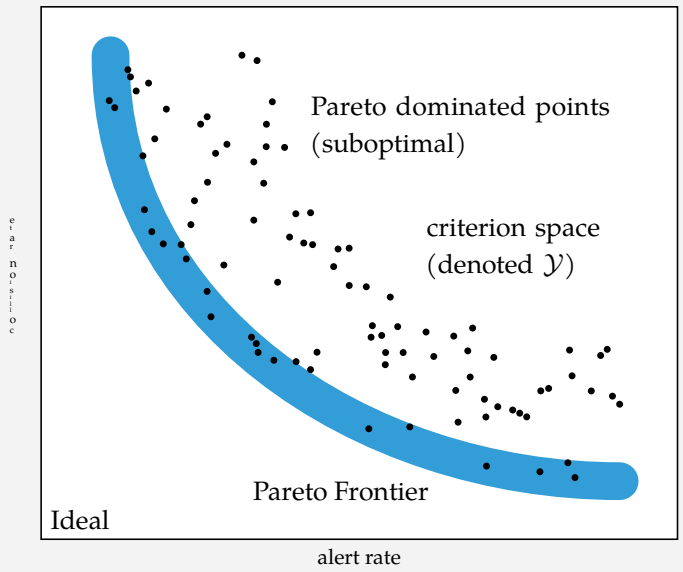
$$y_i^{\text{utopia}} = \underset{\mathbf{x} \in \mathcal{X}}{\text{minimize}}\, f_i(\mathbf{x}) \tag{12.2}$$

乌托邦点通常无法实现；优化一个组件通常需要在另一个组件中进行权衡。

12.1.3 *Pareto Frontier Generation*

有几种方法可以生成帕累托前沿。一种简单的方法是在整个设计空间中抽样设计点，然后识别非支配点（算法12.2）。这种方法通常效率低下，会导致许多被支配的设计点，如图12.4所示。此外，这种方法不能保证帕累托前沿的平滑或正确。本章剩余部分将讨论生成帕累托前沿的更有效方法。

在为飞机构建防撞系统时，必须同时最小化碰撞率和警报率。虽然如果遵循警报，更多警报可以防止更多碰撞，但过多的警报可能导致飞行员对系统失去信任，并降低对系统的遵从性。因此，系统设计者必须谨慎权衡警报和碰撞风险。



通过改变碰撞避免系统的设计参数，我们可以得到许多不同的避碰系统，但如图所示，其中一些会比其他的更好。可以提取帕累托前沿，以帮助领域专家和监管者理解目标权衡将对优化系统产生的影响。

例12.1. 通过评估飞机防撞系统中众多不同设计点得到的近似帕累托前沿。

```
function naive_pareto(xs, ys)
    pareto_xs, pareto_ys = similar(xs, 0), similar(ys, 0)
    for (x,y) in zip(xs,ys)
        if !any(dominates(y′,y) for y′ in ys)
            push!(pareto_xs, x)
            push!(pareto_ys, y)
        end
    end
    return (pareto_xs, pareto_ys)
end
```

算法12.2. 一种利用随机采样的设计点xs及其多目标值ys生成帕累托前沿的方法。返回帕累托最优设计点及其目标值。

12.2 Constraint Methods

约束可用于削减帕累托前沿的部分区域，并在准则空间中获取单一最优点。约束既可以由问题设计者提供，也可以根据目标排序自动获得。

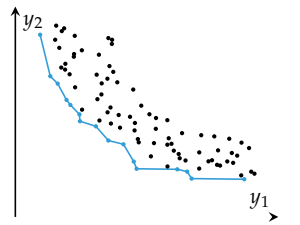


图12.4. 使用简单散布点生成帕累托前沿直截了当，但效率低且近似。

12.2.1 Constraint Method

constraint method 约束了除一个目标以外的所有目标。在这里，我们不失一般性地选择 f_1：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f_1(\mathbf{x}) \\ \text{subject to} \quad & f_2(\mathbf{x}) \leq c_2 \\ & f_3(\mathbf{x}) \leq c_3 \\ & \vdots \\ & f_m(\mathbf{x}) \leq c_m \\ & \mathbf{x} \in \mathcal{X} \end{aligned} \tag{12.3}$$

给定向量c，约束法在准则空间中产生一个唯一的最优解，前提是约束条件可行。如图12.5所示，通过改变c，约束法可用于生成帕累托前沿。

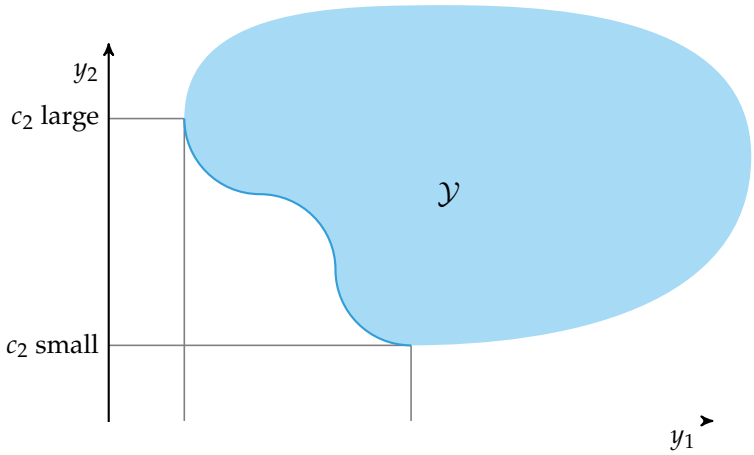


图 12.5. 生成帕累托前沿的约束方法。该方法能够识别帕累托前沿凹陷区域中的点。

12.2.2 *Lexicographic Method*

lexicographic method 按照重要性对目标进行排序。按照重要性顺序对目标执行一系列单目标优化。每个优化问题都包含约束，以保持对于先前优化目标的最优性，如图12.6所示。

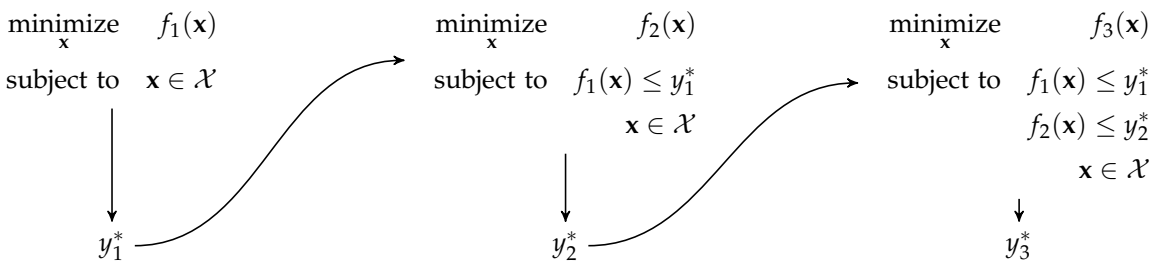


图12.6. 具有三个目标的优化问题的字典序方法。

迭代总是可行的，因为前一次优化的最小点始终是可行的。约束条件也可以替换为等式，但不等式通常更易于优化器执行。此外，如果所采用的优化方法并非最优，那么后续优化可能会遇到原本会被拒绝的更优解。字典序方法对目标函数的排序很敏感。

12.3 Weight Methods

设计师有时可以识别目标之间的偏好，并将这些偏好编码为权重向量。在权重选择不明显的情况下，我们可以通过遍历权重空间来生成帕累托前沿。本节还讨论了将多目标函数转换为单目标函数的各种替代方法。

12.3.1 Weighted Sum Method

weighted sum method (算法12.3)使用权重向量w将f转换为单个目标f: [2]

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{f}(\mathbf{x}) \tag{12.4}$$

其中权重非负且总和为1。这些权重可以被解释为与每个目标相关的成本。通过改变w并求解与方程(12.4)中目标相关的优化问题，可以提取帕累托前沿。在二维情况下，我们从0到1变化w_1，设定$w_2 = 1 - w_1$。这种方法在图12.7中进行了说明。

2 L. Zadeh, “最优性和非标量值性能标准”，*IEEE Transactions on Automatic Control*，第8卷，第1期，第59-60页，1963年。

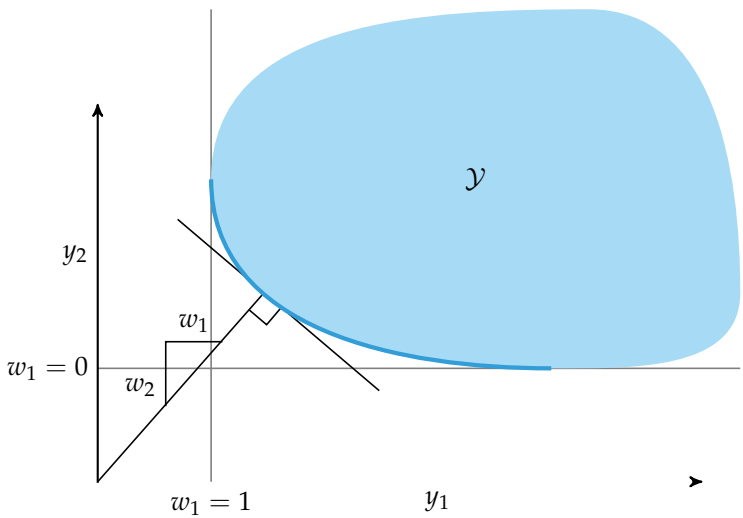


图12.7. 用于生成帕累托前沿的加权和法。通过变化权重，我们可以描绘出帕累托前沿。

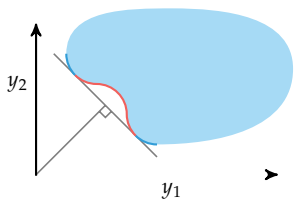


图12.8. 红点表示帕累托最优解，但无法通过加权和法获得。

与约束方法相比，加权和方法无法获得帕累托前沿非凸区域中的点，如图12.8所示。

给定的一组权重形成一个线性目标函数，其等值线平行且远离原点。如果可行集偏离原点弯曲，则在边界上会有其他帕累托最优点，这些点无法通过最小化方程(12.4)来恢复。

```
function weight_pareto(f1, f2, npts)
    return [
        optimize(x->w1*f1(x) + (1-w1)*f2(x))
        for w1 in range(0,stop=1,length=npts)
    ]
end
```

算法12.3. 生成帕累托前沿的加权和法，该方法采用目标函数`f1`和`f2`以及帕累托点数量`npts`。

12.3.2 *Goal Programming*

*Goal programming*3是一种将多目标函数转换为单
通过最小化 f(x) 与目标点之间的 L_p 范数来优化目标函数：

$$\underset{\mathbf{x}\in\mathcal{X}}{\text{minimize}} \left\| \mathbf{f}(\mathbf{x}) - \mathbf{y}^{\text{goal}} \right\|_p \tag{12.5}$$

其中目标点通常是乌托邦点。上述方程不涉及权重向量，但本章讨论的其他方法可以视为目标规划的推广。该方法如图12.9所示。

3 目标规划通常是指使用 $p = 1$. 概述见 D. Jones 和 M. Tamiz, *Practical Goal Programming*. Springer, 2010.

12.3.3 *Weighted Exponential Sum*

weighted exponential sum 结合了目标规划和加权求和法4

$$f(\mathbf{x}) = \sum_{i=1}^{m} w_i \left(f_i(\mathbf{x}) - y_i^{\text{goal}} \right)^p \tag{12.6}$$

其中w是一个正权重向量，其和为1，而$p \geq 1$ 是指数与L_p范数中使用的类似。如之前所述，零值权重可能导致弱帕累托最优点。

4 P. L. Yu, “锥凸性、锥极点和多目标决策问题中的非支配解”，*Journal of Optimization Theory and Applications*，第14卷，第3期，第319–377页，1974年。

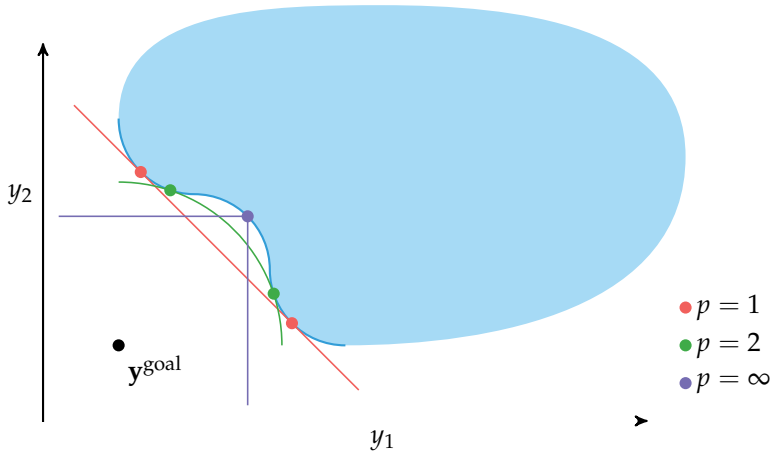


图12.9. 当p的值改变时，目标规划的解决方案。

加权指数和衡量解点与目标点在准则空间中距离的每个分量。增加 p 会增加 f(x) 与目标点之间最大坐标偏差的相对惩罚。虽然可以通过连续变化 p 来获得帕累托最优集的部分内容，但我们无法保证获得完整的帕累托前沿，通常更可取的是使用常数 p 来变化 w。

12.3.4 *Weighted Min-Max Method*

使用较高的p值与加权指数和目标函数倾向于产生更好的帕累托前沿覆盖，因为距离轮廓能够进入帕累托前沿的非凸区域。*weighted min-maxmethod*，也称为 *weighted Tchebycheff method*，是p趋于无穷大时的极限：[5]

$$f(\mathbf{x}) = \max_i \left[w_i \left(f_i(\mathbf{x}) - y_i^{\text{goal}} \right) \right] \tag{12.7}$$

加权最小-最大方法可以通过扫描权重来提供完整的帕累托最优集，但也会产生弱帕累托最优点。该方法可以扩展为仅产生帕累托前沿

$$f(\mathbf{x}) = \max_i \left[w_i \left(f_i(\mathbf{x}) - y_i^{\text{goal}} \right) \right] + \rho \, \mathbf{f}(\mathbf{x})^\top \mathbf{y}^{\text{goal}} \tag{12.8}$$

5 通过包含一个附加参数λ，可以去除最大化：

$$\begin{aligned} &\underset{\mathbf{x},\lambda}{\text{minimize}} && \lambda \\ &\text{subject to} && \mathbf{x} \in \mathcal{X} \\ &\mathbf{w} \odot \left(\mathbf{f}(\mathbf{x}) - \mathbf{y}^{\text{goal}} \right) - \lambda \mathbf{1} \leq \mathbf{0} \end{aligned}$$

其中 ρ 是一个小的正标量，其值通常在 0.0001 到 0.01 之间。添加的项要求 y_{goal} 中的所有项都为正，这可以通过平移目标函数来实现。根据定义，对于所有 x，都有 $f(x) \geq y_{goal}$。任何弱帕累托最优点都会使 $f(x)^\top y^{goal}$ 大于更接近 y_{goal} 的强帕累托最优点。

12.3.5 Exponential Weighted Criterion

exponential weighted criterion[6] 的动机是加权
加权和法无法获得帕累托前沿非凸部分上的点。它根据以下公式构建标量目标函数：

$$f(\mathbf{x}) = \sum_{i=1}^{m} (e^{pw_i} - 1) e^{pf_i(\mathbf{x})} \quad \textit{tion} \tag{12.9}$$

每个目标都分别进行变换和重新加权。p 的高值可能导致数值溢出。

6 T. W. Athan 和 P. Y. Papalambros，“关于多目标优化中妥协解的加权准则方法的注记”，*Engineering Optimiza-*，第27卷，第2期，第155-176页，1996年。

12.4 Multiobjective Population Methods

种群方法也已被应用于多目标优化。[7] 我们
可以调整标准算法，以鼓励种群在帕累托前沿上分布。

第9章涵盖了7种群方法。

12.4.1 Subpopulations

种群方法可以将其注意力分散到多个可能相互竞争的目标上。种群可以被划分为 *subpopulations*，其中每个子种群针对不同的目标进行优化。例如，传统的遗传算法可以被修改，使重组个体的选择偏向每个子种群内最适合的个体。被选中的个体可以与来自不同子种群的个体形成后代。

将种群方法应用于多目标优化的首批适应性算法之一是 *vector evaluated genetic algorithm*[8]（算法12.4）。图12.10展示了子种群如何在向量评估遗传算法中被用于在多个目标上保持多样性。向量评估遗传算法的进展如图12.11所示。

8 J. D. Schaffer，“使用向量评估遗传算法的多目标优化”，载于 {v*}，1985年。

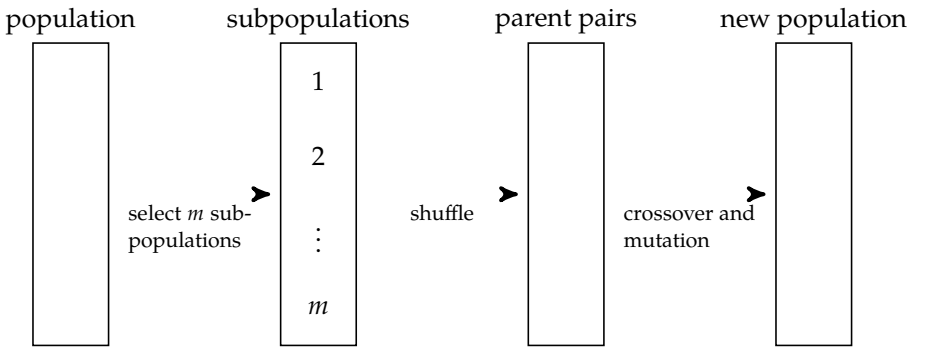


图12.10。在向量评估遗传算法中使用子种群。

```
function vector_evaluated_genetic_algorithm(f, population,
    k_max, S, C, M)
    m = length(f(population[1]))
    m_pop = length(population)
    m_subpop = m_pop ÷ m
    for k in 1 : k_max
        ys = f.(population)
        parents = select(S, [y[1] for y in ys])[1:m_subpop]
        for i in 2 : m
            subpop = select(S,[y[i] for y in ys])[1:m_subpop]
            append!(parents, subpop)
        end

        p = randperm(2m_pop)
        p_ind=i->parents[mod(p[i]-1,m_pop)+1][(p[i]-1)÷m_pop + 1]
        parents = [[p_ind(i), p_ind(i+1)] for i in 1 : 2 : 2m_pop]
        children = [crossover(C,population[p[1]],population[p[2]])
                    for p in parents]
        population = [mutate(M, c) for c in children]
    end
    return population
end
```

算法12.4。向量评价遗传算法，该算法接受一个向量值目标函数`f`、初始种群、迭代次数`k_max`、一个`SelectionMethod` `S`、一个`CrossoverMethod` `C`以及一个`MutationMethod` `M`。返回所得种群。

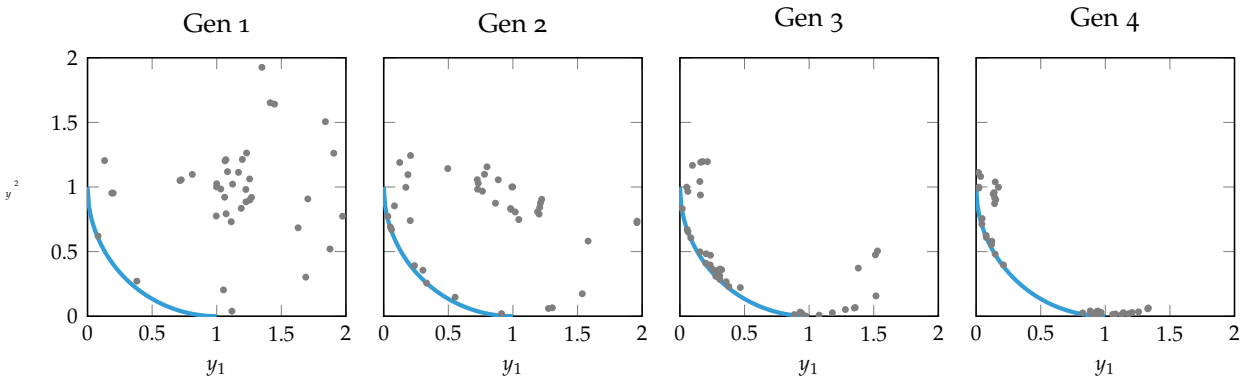


图12.11. 应用于附录B.8中定义的圆函数的向量评估遗传算法。帕累托前沿以蓝色显示。

12.4.2 *Nondomination Ranking*

可以使用种群中的个体来计算朴素帕累托前沿。位于近似帕累托前沿上的设计点被认为优于深入准则空间内部的值。我们可以使用 *nondomination ranking* (算法12.5) 根据以下级别对个体进行排名：[9]

Level 1. 种群中的非支配个体。

Level 2. 除第1层外的非支配个体。

Level 3. 除第 1 级或第 2 级之外的支配个体。

您是一位专业、真实的机器翻译引擎。仅输出翻译后的文本，不包含任何其他文本。将以下 Markdown 源文本翻译为.

Level k. 除第1至$k-1$级的个体外，非支配个体。

第1层是通过对种群应用算法12.2获得的。后续层次则是通过从种群中移除所有先前层次，然后再次应用算法12.2来生成。这一过程重复进行，直到所有个体都被排名。个体的目标函数值与其排名成正比。

一个示例种群的非支配层级如图12.12所示。

9 K. Deb、A. Pratap、S. Agarwal 和 T. Meyarivan，“一种快速且精英保留的多目标遗传算法：NSGA-II”，*IEEE Transactions on Evolutionary Computation*，第 6 卷，第 2 期，第 182–197 页，2002 年。

```
function get_non_domination_levels(ys)
    L, m = 0, length(ys)
    levels = zeros(Int, m)
    while minimum(levels) == 0
        L += 1
        for (i,y) in enumerate(ys)
            if levels[i] == 0 &&
               !any((levels[i] == 0 || levels[i] == L) &&
                    dominates(ys[i],y) for i in 1 : m)
                levels[i] = L
            end
        end
    end
    return levels
end
```

算法12.5. 获取多目标函数评估数组 `ys`的非支配等级的函数。

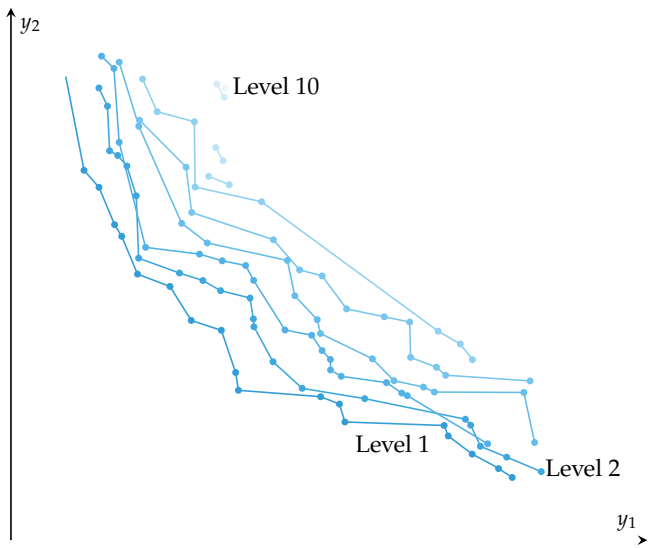


图12.12。种群的非支配层级。较暗的层级具有较低（较好）的排名。

12.4.3 *Pareto Filters*

种群方法可以用 *Pareto filter* 来增强，*Pareto filter* 是一个近似帕累托前沿的种群。[10] 过滤器通常随着每次生成（算法12.7）。将种群中未被过滤器内任何个体支配的个体加入。过滤器中被支配的点则被移除。帕累托过滤器中的个体可注入种群，从而降低帕累托前沿各部分在世代间丢失的可能性。

10 H. Ishibuchi 和 T. Murata，“一种多目标遗传局部搜索算法及其在流水车间调度中的应用”，*IEEE Transactions on Systems, Man, and Cybernetics*，第28卷，第3期，第392-403页，1998年。

过滤器通常具有最大容量。[11] 超过容量的过滤器可能通过在准则空间中找到最接近的设计点对，并从该对中移除一个个体来进行缩减。这种修剪方法在算法12.6中实现。使用遗传算法获得的帕累托过滤器如图12.13所示。

11 通常为种群规模。

```
function discard_closest_pair!(xs, ys)
    index, min_dist = 0, Inf
    for (i,y) in enumerate(ys)
        for (j, y′) in enumerate(ys[i+1:end])
            dist = norm(y - y′)
            if dist < min_dist
                index, min_dist = rand([i,j]), dist
            end
        end
    end
    deleteat!(xs, index)
    deleteat!(ys, index)
    return (xs, ys)
end
```

算法12.6。方法`discard_closest_pair!`用于从超出容量的过滤器中移除一个个体。该方法接收过滤器的设计点列表`xs`及相关的目标函数值`ys`。

```
function update_pareto_filter!(filter_xs, filter_ys, xs, ys;
    capacity=length(xs),
    )
    for (x,y) in zip(xs, ys)
        if !any(dominates(y′,y) for y′ in filter_ys)
            push!(filter_xs, x)
            push!(filter_ys, y)
        end
    end
    filter_xs, filter_ys = naive_pareto(filter_xs, filter_ys)
    while length(filter_xs) > capacity
        discard_closest_pair!(filter_xs, filter_ys)
    end
    return (filter_xs, filter_ys)
end
```

算法12.7. 一种更新帕累托过滤器的方法，该方法涉及设计点 `filter_xs`、相应的目标函数值 `filter_ys`、包含设计点 `xs` 和目标函数值 `ys` 的种群，以及默认为种群大小的过滤容量 `capacity`。

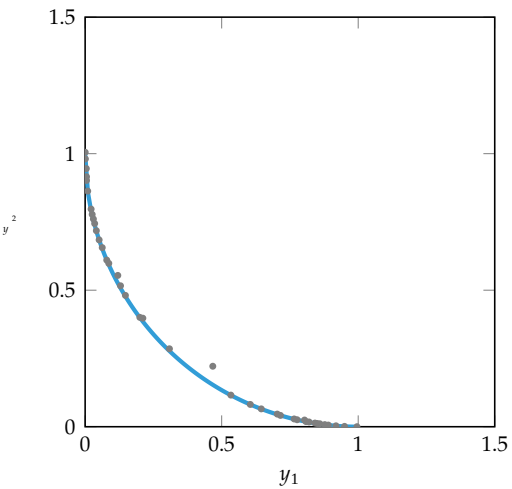


图12.13。用于图12.11中遗传算法的帕累托滤波器，以逼近帕累托前沿。

12.4.4 Niche Techniques

术语*niche*指的是一个集中的点簇，通常位于准则空间中，如图12.14所示。种群方法可能会收敛到少数几个小生境上，这限制了它们在帕累托前沿上的分布。*Niche techniques*有助于促进点的均匀分布。

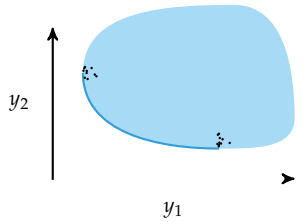


图12.14. 二维标准空间中种群的两个明显生态位。

在*fitness sharing*、[12]（如图12.15所示）中，个体的目标值为在准则空间中，被等于指定距离内其他点数量的因子所惩罚。[13] 这种方案导致局部区域内的所有点都与局部区域内其他点共享适应度。适应度共享可与非支配排序和子种群评估一起使用。

12 适应度与所最小化的目标呈反比关系。

Equivalence class sharing 可以应用于非支配排序。当比较两个个体时，首先基于非支配排序确定较优个体。如果它们相等，则在准则空间中指定距离内个体数量最少的个体为更优个体。

13 D. E. Goldberg 和 J. Richardson，“用于多模态函数优化的共享遗传算法”，载于 *International Conference on Genetic Algorithms*，1987年

另一种针对遗传算法的特定技术已被提出，其中为交叉选择的父母在准则空间中不能过于接近。仅选择非支配个体也是推荐做法。[14]

14 S. Narayanan 和 S. Azarm，“关于改进设计优化的多目标遗传算法”，*Structural Optimization*，第18卷，第2-3期，第146-155页，1999年。

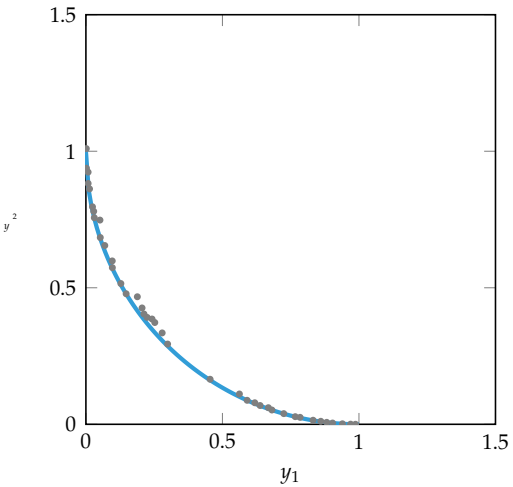


图12.15. 将适应度共享应用于图12.13中的帕累托过滤器后，其覆盖范围得到显著提升的结果。

12.5 Preference Elicitation

Preference elicitation 涉及从专家关于目标之间权衡的偏好中推断标量值目标函数。[15] 有许多有多种方式表示标量值目标函数，但本节将重点讨论加权求和模型，其中$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{f}(\mathbf{x})$。一旦确定了合适的$\mathbf{w}$，我们就能利用这一标量值目标函数来寻找最优设计。

15 本章概述了偏好获取的非贝叶斯方法。关于贝叶斯方法，参见：S. Guo 和 S. Sanner，“Real-Time Multiattribute Bayesian Preference Elicitation with Pairwise Comparison Queries,” in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010. J. R. Lepird, M. P. Owen, 和 M. J. Kochenderfer, “Bayesian Preference Elicitation for Multi-objective Engineering Design Optimization,” *Journal of Aerospace Information Systems*, vol. 12, no. 10, pp. 634–645, 2015.

12.5.1 Model Identification

在我们偏好模型中识别权重向量 w 的一种常用方法是让专家陈述他们在准则空间 $\mathcal{Y}$ (图12.16) 中两个点 a 和 b 之间的偏好。这些点中的每一个都是使用相关权重向量 $\mathbf{w_a}$ 和 $\mathbf{w_b}$ 针对帕累托前沿上的一个点进行优化得到的结果。专家的回应要么是对 a 的偏好，要么是对 b 的偏好。还有其他引出偏好信息的方案，例如对准则空间中的点进行排序，但已证明这种二元偏好询问对专家造成的认知负担最小。[16]

16 V. Conitzer, “使用比较查询引出单峰偏好”，*Journal of Artificial Intelligence Research*，第 35 卷，第 161–191 页，2009 年。

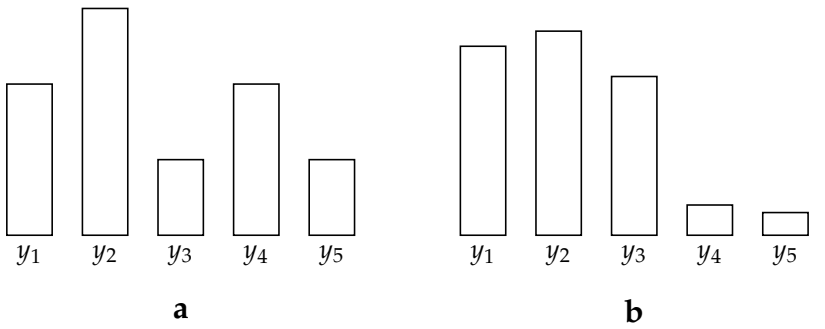


图12.16。偏好引出方案通常涉及询问专家在准则空间中两点之间的偏好。

假设专家查询的结果产生了一组准则对

$$\left\{(\mathbf{a}^{(1)}, \mathbf{b}^{(1)}), \ldots, (\mathbf{a}^{(n)}, \mathbf{b}^{(n)})\right\} \tag{12.10}$$

其中在每一对中 a(i) 优于 b(i)。对于这些偏好中的每一个，权重向量必须满足

$$\mathbf{w}^\top \mathbf{a}^{(i)} < \mathbf{w}^\top \mathbf{b}^{(i)} \implies (\mathbf{a}^{(i)} - \mathbf{b}^{(i)})^\top \mathbf{w} < 0 \tag{12.11}$$

为了与数据保持一致，权重向量必须满足

$$\begin{cases} (\mathbf{a}^{(i)} - \mathbf{b}^{(i)})^\top \mathbf{w} < 0 \text{ for all } i \text{ in } \{1, \ldots, n\} \\ \mathbf{1}^\top \mathbf{w} = 1 \\ \mathbf{w} \geq \mathbf{0} \end{cases} \tag{12.12}$$

许多不同的权重向量都可能满足上述方程。一种方法是选择一个能最好地将 $w^\top a^{(i)}$ 与 $w^\top b^{(i)}$ 分开的 w。

$$\begin{aligned} \underset{\mathbf{w}}{\text{minimize}} \quad & \sum_{i=1}^{n} (\mathbf{a}^{(i)} - \mathbf{b}^{(i)})^\top \mathbf{w} \\ \text{subject to} \quad & (\mathbf{a}^{(i)} - \mathbf{b}^{(i)})^\top \mathbf{w} < 0 \text{ for } i \in \{1, \ldots, n\} \\ & \mathbf{1}^\top \mathbf{w} = 1 \qquad \mathbf{w} \geq \mathbf{0} \end{aligned} \tag{12.13}$$

通常希望选择下一个权重向量，使其与上一个权重向量的距离最小化。我们可以将方程 (12.13) 中的目标函数替换为 $\|w - w^{(n)}\|_1$，从而确保我们的新权重向量 $w^{(n+1)}$ 尽可能接近当前权重向量。[17]

17 先前的权重向量可能与新增约束 $(a^{(n)} - b^{(n)})^\top w < 0$ 一致，也可能不一致。

12.5.2 *Paired Query Selection*

我们通常希望在准则空间中选择这两个点，以使查询的结果尽可能具有信息量。对于此类*paired query selection*，有许多不同的方法，但我们将重点放在那些试图减小与*expert responses*（由领域专家提供的偏好信息）一致的权重空间的方法上。

我们将与专家回答一致的权重集合表示为$\mathcal{W}$，它由方程(12.12)中的线性约束定义。由于权重被限制在0和1之间，可行集是一个封闭区域，形成一个具有有限体积的凸多面体。我们通常希望以尽可能少的查询来减小$\mathcal{W}$的体积。

Q-Eval[18]，如图12.17所示，是一种启发式的贪心诱导策略。s试图在每次迭代中尽快减小$\mathcal{W}$的体积。它选择的查询最接近于将$\mathcal{W}$等分为两部分。该方法在帕累托最优设计点的有限样本上运行。选择查询对的步骤如下：

18 V. S. Iyengar, J. Lee, and M. Campbell, ‘‘Q-EVAL: 使用查询评估多属性项目,’’ 载于 *ACM Conference on Electronic Commerce*, 2001.

1. 计算$\mathcal{W}$的*prime analytic center*c，即在$\mathcal{W}$中每个非冗余约束上，使自身与最近点之间距离的对数之和最大化的点：

$$\mathbf{c} = \arg\max_{\mathbf{w} \in \mathcal{W}} \sum_{i=1}^{n} \ln\left((\mathbf{b}^{(i)} - \mathbf{a}^{(i)})^\top \mathbf{w} \right) \tag{12.14}$$

2. 计算每对点之间的平分超平面到中心的垂直距离。

3. 按距离递增的顺序对设计点对进行排序。

4. 对于最接近c的k个超平面，计算沿超平面分割$\mathcal{W}$所形成的两个多面体的体积比。

5. 选择分流比最接近1的设计点对。

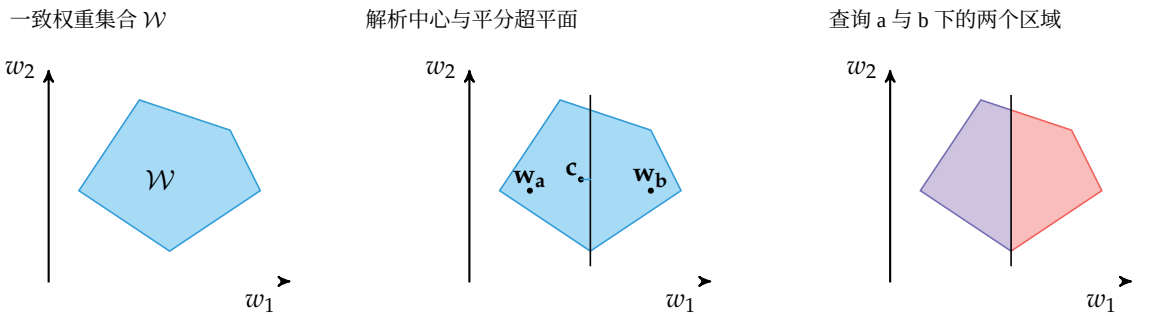


图12.17. Q-Eval贪心启发策略的可视化。图中的进展展示了与先前偏好一致的初始权重集$\mathcal{W}$、一对权重向量及其对应的平分超平面，以及沿平分超平面分割形成的两个多面体。该算法考虑了来自帕累托最优设计点有限采样的所有可能对，并选择最均匀地划分$\mathcal{W}$的查询。

polyhedral method[19] 通过用边界椭球近似 $\mathcal{W}$ 来工作以$\mathcal{W}$的解析中心为中心，如图12.18所示。查询被设计为将边界椭球体划分为近似相等的部分，并倾向于选择垂直于椭球体最长轴的切割，以减少不确定性并平衡每个维度上的广度。

12.5.3 *Design Selection*

上一节讨论了选择查询对以有效缩小搜索空间的查询方法。查询选择完成后，仍然必须选择最终设计。这一过程称为*design selection*。

19 D. Braziunas 和 C. Boutilier，“分解效用的诱导”，*AI Magazine*，第29卷，第4期，第79-92页，2009年。

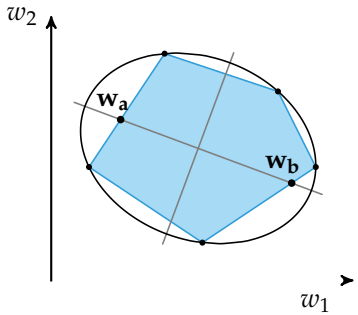


图12.18。多面体方法对$\mathcal{W}$使用包围椭球。

其中一种方法，*decision quality improvement*，[20]，基于这样的想法：如果我们必须致力于一个特定的权重，应当选择那个最坏情况目标值最低的权重：

20 D. Braziunas 和 C. Boutilier，“基于最小最大遗憾的广义加性效用获取”，载于 *Conference on Uncertainty in Artificial Intelligence (UAI)*，2007年。

$$\mathbf{x}^* = \arg\min_{\mathbf{x}\in\mathcal{X}} \max_{\mathbf{w}\in\mathcal{W}} \mathbf{w}^\top \mathbf{f}(\mathbf{x}) \tag{12.15}$$

这个 *minimax decision* 是稳健的，因为它为目标值提供了一个上界。

minimax regret[21] 反而最小化了最大遗憾量用户在选定特定设计时可以拥有：

21 C. Boutilier, R. Patrascu, P. Poupart, and D. Schuurmans，“基于约束的优化与使用极小极大决策准则的效用诱导，” *Artificial Intelligence*, vol. 170, no. 8-9, pp. 686–713, 2006.

$$\mathbf{x}^* = \arg\min_{\mathbf{x}\in\mathcal{X}} \underbrace{\max_{\mathbf{w}\in\mathcal{W}} \max_{\mathbf{x}'\in\mathcal{X}} \mathbf{w}^\top \mathbf{f}(\mathbf{x}) - \mathbf{w}^\top \mathbf{f}(\mathbf{x}')}_{\text{maximum regret}} \tag{12.16}$$

其中 $\mathbf{w}^\top \mathbf{f}(\mathbf{x}) - \mathbf{w}^\top \mathbf{f}(\mathbf{x}')$ 是与在偏好权重向量 $\mathbf{w}$ 下选择设计 $\mathbf{x}$ 而非设计 $\mathbf{x}'$ 相关的 *regret*。极小极大后悔可以视为考虑决策系统关于设计者真实效用函数的不确定性。

极小极大后悔值可用作偏好引出策略的停止准则。一旦极小极大后悔值降至某一阈值以下，我们即可终止偏好引出过程。

12.6 Summary

- 多目标设计问题往往需要在不同目标之间权衡性能。
- 帕累托前沿表示潜在最优解的集合。
- 向量值目标函数可以使用基于约束或基于权重的方法转换为标量值目标函数。
- 种群方法可以扩展以生成跨越帕累托前沿的个体。
- 了解专家在准则空间中成对点之间的偏好，有助于指导标量值目标函数的推断。

12.7 Exercises

练习 12.1. 加权和法是一种非常简单的策略，工程师在多目标优化中确实会使用它。当用于计算帕累托前沿时，该方法的一个缺点是什么?

练习12.2。为什么种群方法非常适合多目标优化?

练习 12.3。假设你在准则空间中有一些点 {[1, 2]、[2, 1]、[2, 2]、[1, 1]}，并希望近似一条帕累托前沿。相对于其余点，哪些点是帕累托最优的？是否存在弱帕累托最优点?

练习 12.4. 多目标优化不易通过二阶方法实现。原因何在?

练习12.5。考虑一个正方形准则空间 $\mathcal{Y}$，其中 $y_1 \in [0, 1]$ 且 $y_2 \in [0, 1]$。绘制该准则空间，标出帕累托最优点，并标出弱帕累托最优点。

练习 12.6。在加权求和法中，强制 $\mathbf{w} \geq 0$ 和 $\|\mathbf{w}\|_1 = 1$ 不足以保证帕累托最优性。给出一个例子，说明权重为零的坐标会找到弱帕累托最优点。

练习12.7。提供一个目标规划不产生帕累托最优点实例。

练习 12.8. 使用约束方法获得优化问题的帕累托曲线：

$$\underset{x}{\text{minimize}} \ [x^2, (x-2)^2] \tag{12.17}$$

练习12.9。假设我们有一个多目标优化问题，其中两个目标如下：

$$f_1(x) = -(x-2)\sin(x) \tag{12.18}$$
$$f_2(x) = -(x+3)^2\sin(x) \tag{12.19}$$

用$x \in \{-5、-3、-1、1、3、5\}$，在准则空间中绘制这些点。帕累托前沿上有多少个点?

13 Sampling Plans

对于许多优化问题，函数评估可能相当昂贵。例如，评估一个硬件设计可能需要冗长的制造过程，飞机设计可能需要风洞测试，而新的深度学习超参数可能需要一周的GPU训练。在评估设计点成本高昂的情况下，一种常见的优化方法是构建*surrogate model*，这是一种优化问题的模型，可以高效地进行优化，以替代真实的目标函数。对真实目标函数的进一步评估可用于改进模型。拟合此类模型需要一组初始点，理想情况下这些点是*space-filling*的；也就是说，这些点尽可能覆盖区域。本章涵盖了在资源有限时，用于覆盖搜索空间的不同*sampling plans*。[1]

1 还有其他参考文献更详细地讨论了本章的主题。例如，参见：G. E. P. Box、W. G. Hunter 和 J. S. Hunter，*Statistics for Experimenters:*，第2版。Wiley，2005年。A. Dean、D. Voss 和 D. Draguljić，*Design and Analysis*，第2版。Springer，2017年。D. C. Montgomery，*Design*。Wiley，2017年。

13.1 Full Factorial

*full factorial*采样计划（算法13.1）在搜索空间上放置了一个均匀分布的网格点。这种方法易于实现，不依赖于随机性，且能覆盖整个空间，但使用的点数较多。如图13.1所示，均匀分布的网格点遍布于搜索空间。在完全因子采样计划中对这些点进行优化被称为*gridsearch*。

采样网格由下界向量 a 和上界向量 b 定义，使得对于每个分量 i，$a_i \leq x_i \leq b_i$。对于第 i 维中有 m_i 个样本的网格，最近点之间的距离为 $(b_i - a_i)/(m_i - 1)$。

全因子方法所需的样本数量随维度呈指数增长。[2]对于n个维度，每个维度有m个样本，我们有m^n

2 全因子方法的名字并非来源于因子样本数（它是指数级的），而是来源于使用两个或多个 *discrete* 进行设计。这里的因子是与每个变量相关联的 m 离散化水平。

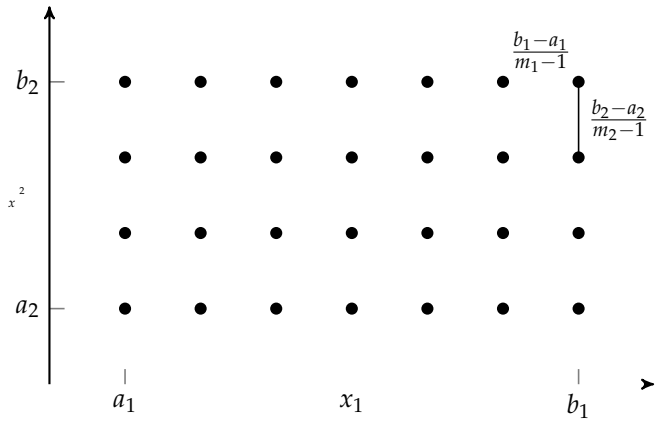


图13.1. 全因子搜索以网格点的形式覆盖搜索空间。

总样本数。当变量多于几个时，这种指数级增长远远过高，无法实际应用。即使能够使用全因子采样，网格点通常也被迫相当粗糙，因此很容易错过优化景观中的小型局部特征。

```
function samples_full_factorial(a, b, m)
    ranges = [range(a[i], stop=b[i], length=m[i])
              for i in 1 : length(a)]
    collect.(collect(product(ranges...)))
end
```

算法13.1. 一个用于获取全因子网格所有样本位置的函数。这里，a是变量下界的向量，b是变量上界的向量，m是每个维度样本计数的向量。

13.2 *Random Sampling*

全因子采样的一种直接替代方法是*random sampling*，它仅使用伪随机数生成器在设计空间上抽取m个随机样本。为了生成随机样本x，我们可以从分布中独立地对每个变量进行采样。如果变量有边界，例如$a_i \leq x_i \leq b_i$，常用方法是在$[a_i, b_i]$上使用均匀分布，尽管也可以使用其他分布。对于某些变量，使用对数均匀分布可能更合理。[3]设计点的样本是不相关的

彼此之间。希望这种随机性本身能够充分覆盖设计空间。

3 某些参数，如深度神经网络的学习率，最适合在对数空间中搜索。

13.3 *Uniform Projection Plans*

假设我们有一个二维优化问题，其按照全因子方法离散化为一个 $m \times m$ 采样网格，但不是取全部 m^2 个样本，我们只想采样 m 个位置。我们可以随机选择样本，但并非所有排列都同样有用。我们希望样本分布在整个空间中，并且希望样本分布在每个单独的分量上。

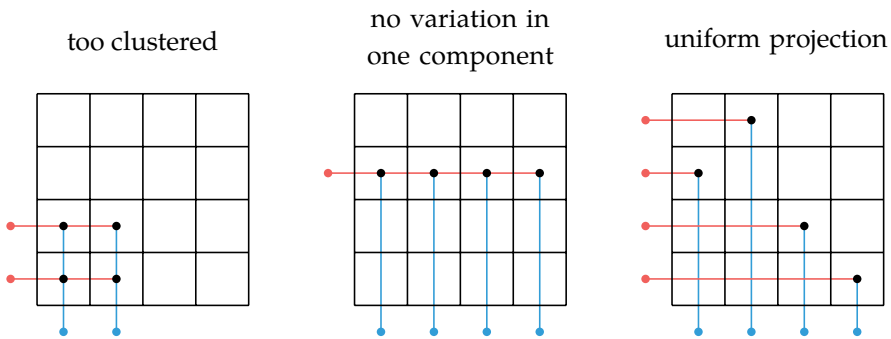


图13.2. 从二维网格中选择m样本的几种方法。我们通常更倾向于那些能够覆盖整个空间并在每个分量上都有所变化的抽样方案。

uniform projection plan 是在离散网格上的采样计划，其中每个维度上的分布都是均匀的。例如，在图13.2最右侧的采样计划中，每一行恰好有一个条目，每一列也恰好有一个条目。

一个在 $m \times m$ 网格上具有 m 个样本的均匀投影计划，可以使用如图 13.3 所示的 m 元素排列来构建。因此，存在 $m!$ 种可能的均匀投影计划。[4]

4 对于 $m = 5$，这已经是 $5! = 120$ 种可能的计划。对于 $m = 10$，有 3,628,800 种计划。

$$p = 4\,2\,1\,3\,5$$

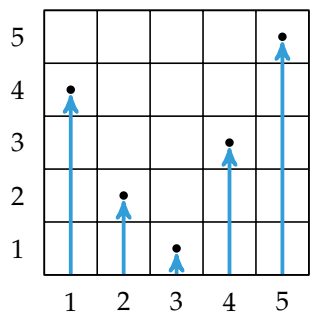


图13.3. 使用排列构建均匀投影计划。

使用均匀投影计划进行抽样有时被称为*Latin-hypercubesampling*，因为它与*Latin squares* (图13.4)有关。拉丁方是一个$m \times m$网格，其中每一行包含从1到m的每个整数，每一列包含从1到m的每个整数。拉丁超立方体是对任意维度的推广。

对于 n 维，可以使用每个维度上的一个排列（算法13.2）来构造均匀投影方案。

```
function uniform_projection_plan(m, n)
    perms = [randperm(m) for i in 1 : n]
    [[perms[i][j] for i in 1 : n] for j in 1 : m]
end
```

4	1	3	2
1	4	2	3
3	2	1	4
2	3	4	1

图13.4. 一个4 × 4拉丁方。通过选择一个值$i \in \{1、2、3、4\}$并对具有该值的所有单元格进行抽样，可以构建均匀投影计划。

算法13.2. 构建一个n维超立方体的均匀投影计划的函数，每维有m个样本。它返回一个索引向量组成的向量。

13.4 *Stratified Sampling*

许多采样计划，包括均匀投影和全因子计划，都是基于$m \times m$网格的。这种网格即使完全采样，也可能因系统规律性而错过重要信息，如图13.5所示。为了有机会命中每一个点，一种方法是使用*stratified sampling*。

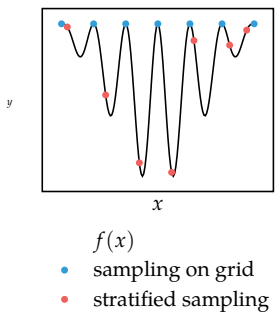


图13.5。在具有系统性规律性的函数上使用均匀间隔的网格可能会遗漏重要信息。

分层抽样会修改任何基于网格的抽样计划，包括全因子和均匀投影计划。单元格的抽样点是从单元格内均匀随机选择的点，而不是像图13.6所示那样从单元格中心选择。

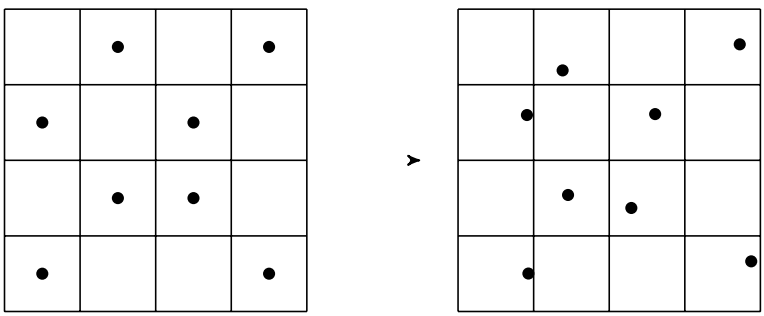


图13.6. 分层抽样应用于均匀投影计划。

13.5 Space-Filling Metrics

一个好的采样计划能够填充设计空间，因为代理模型从样本泛化的能力会随着与这些样本距离的增加而衰减。并非所有计划，即便是均匀投影计划，在覆盖搜索空间方面都同样出色。例如，网格对角线（图13.7）是一种均匀投影计划，但仅覆盖了一条狭窄的带状区域。本节讨论不同的*space-filling metrics*，用于衡量采样计划$X \subseteq \mathcal{X}$填充设计空间的程度。

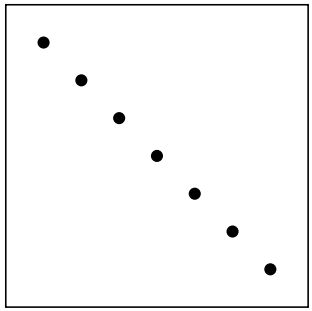


图13.7. 一个非空间填充的均匀投影方案。

13.5.1 Discrepancy

抽样计划填充超矩形设计空间的能力可以通过其*discrepancy*.[5]来衡量。如果X具有低差异，那么随机选择的

设计空间子集应包含与子集体积成比例的样本分数。[6] 与 X 相关的差异是最大差异

在超矩形子集$\mathcal{H}$中样本的比例与该子集的体积之间：

$$d(X) = \operatorname*{supremum}_{\mathcal{H}} \left| \frac{\#(X \bigcap \mathcal{H})}{\#X} - \lambda(\mathcal{H}) \right| \tag{13.1}$$

其中 $\#X$ 和 $\#(X \bigcap \mathcal{H})$ 分别是 X 中的点数以及位于 $\mathcal{H}$ 中的 X 的点数。值 $\lambda(\mathcal{H})$ 是 $\mathcal{H}$ 的 n 维体积，即 $\mathcal{H}$ 的边长之积。术语 *supremum* 与最大化非常相似，但对于 $\mathcal{H}$ 仅趋近于某个特定矩形子集的问题，它允许存在解，如例 13.1.7 所示。

计算单位超矩形上采样计划的差异通常很困难，而且对于非矩形可行集，我们如何计算差异也并不总是很清楚。

13.5.2 Pairwise Distances

确定两个m点抽样计划中哪一个更具空间填充性的另一种方法是比较每个抽样计划内所有点之间的*pairwise distances*。更分散的抽样计划往往具有更大的成对距离。

通常通过将每个集合的成对距离按升序排序来进行比较。第一个成对距离超过另一个的方案被认为更具空间填充性。

5 L. Kuipers 和 H. Niederreiter, *Uniform Distribution of Sequences*. Dover, 2012.

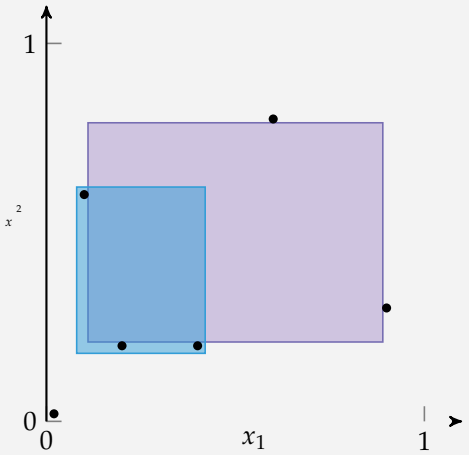
6 在任意维度中，我们可以使用 *Lebesgue measure*，它是体积概念在n维欧几里得空间任意子集上的推广。在一维空间中它是长度，在二维空间中它是面积，在三维空间中它是体积。

7 偏差的定义需要超矩形，并且通常假设 X 是单位超立方体的有限子集。偏差的概念可以扩展，允许 $\mathcal{H}$ 包含其他集合，例如凸多面体。

考虑集合：

$$X = \left\{ \left[\frac{1}{5}, \frac{1}{5}\right], \left[\frac{2}{5}, \frac{1}{5}\right], \left[\frac{1}{10}, \frac{3}{5}\right], \left[\frac{9}{10}, \frac{3}{10}\right], \left[\frac{1}{50}, \frac{1}{50}\right], \left[\frac{3}{5}, \frac{4}{5}\right] \right\}$$

X 相对于单位正方形的偏差由矩形子集 $\mathcal{H}$ 决定，该子集要么面积非常小但包含非常多的点，要么面积非常大但包含非常少的点。



例13.1. 计算单位正方形上抽样计划的差异。矩形的尺寸略微夸大，以清楚显示它们包含哪些点。

蓝色矩形 $x_1 \in \left[\frac{1}{10}, \frac{2}{5}\right]$、$x_2 \in \left[\frac{1}{5}, \frac{3}{5}\right]$ 的体积为 0.12，包含 3 个点。因此其对应的差异度量为 0.38。

紫色矩形，$x_1 \in \left[\frac{1}{10} + \epsilon, \frac{9}{10}\right]$，$x_2 \in \left[\frac{1}{5} + \epsilon, \frac{4}{5}\right]$，会产生更大的差异。当 ϵ 趋近于零时，体积和差异趋近于0.48，因为该矩形不包含任何点。请注意，这里需要求极限，这反映了在差异定义中使用上确界的必要性。

算法13.3计算抽样计划中各点之间的所有成对距离。算法13.4利用各自的成对距离比较两个抽样计划填充空间的效果。

```
import LinearAlgebra: norm
function pairwise_distances(X, p=2)
    m = length(X)
    [norm(X[i]-X[j], p) for i in 1:(m-1) for j in (i+1):m]
end
```

算法13.3. 一个函数，用于根据p指定的L_p范数，获取采样计划X中点之间的成对距离列表。

```
function compare_sampling_plans(A, B, p=2)
    pA = sort(pairwise_distances(A, p))
    pB = sort(pairwise_distances(B, p))
    for (dA, dB) in zip(pA, pB)
        if dA < dB
            return 1
        elseif dA > dB
            return -1
        end
    end
    return 0
end
```

算法13.4。一个函数，用于比较两个抽样计划A和B根据p指定的L_p范数的空间填充程度。如果A比B更填充空间，则函数返回-1。如果B比A更填充空间，则返回1。如果它们等价，则返回0。

生成空间填充均匀投影计划的一种方法是随机生成若干候选方案，然后选择其中空间填充性最好的一个。

我们可以通过反复变异均匀投影方案来寻找填满空间的均匀投影方案，同时保持均匀投影性质（算法13.5）。例如，可以利用模拟退火在空间中搜索具有良好覆盖率的抽样方案。

```
function mutate!(X)
    m, n = length(X), length(X[1])
    j = rand(1:n)
    i = randperm(m)[1:2]
    X[i[1]][j], X[i[2]][j] = X[i[2]][j], X[i[1]][j]
    return X
end
```

算法13.5. 一种用于变异均匀投影计划X的函数，同时保持其均匀投影性质。

13.5.3 Morris-Mitchell Criterion

13.5.2 节中的比较方案通常会导致一个具有许多局部极小值的具有挑战性的优化问题。另一种方法是针对算法 13.6*Morris-Mitchell criterion* (进行优化：)[8]

$$\Phi_q(X) = \left(\sum_i d_i^{-q}\right)^{1/q} \tag{13.2}$$

8 M. D. Morris 和 T. J. Mitchell，“计算实验的探索性设计”，*Journal of Statistical Planning and Inference*，第 43 卷，第 3 期，第 381–402 页，1995 年。

其中 d_i 是 X 和 $q > 0$ 中两点之间的第 i 个成对距离，是一个可调参数。[9] Morris 和 Mitchell 建议优化：

$$\underset{X}{\text{minimize}} \quad \underset{q \in \{1,2,3,10,20,50,100\}}{\text{maximize}} \quad \Phi_q(X) \tag{13.3}$$

9 q 值越大，对远距离的惩罚就越高。

```
function phiq(X, q=1, p=2)
    dists = pairwise_distances(X, p)
    return sum(dists.^(-q))^(1/q)
end
```

算法13.6。Morris-Mitchell准则的一种实现，该实现接收设计点列表`X`、准则参数`q > 0`以及范数参数`p ≥ 1`。

图13.8展示了针对若干随机生成的均匀投影方案评估的Morris-Mitchell准则。

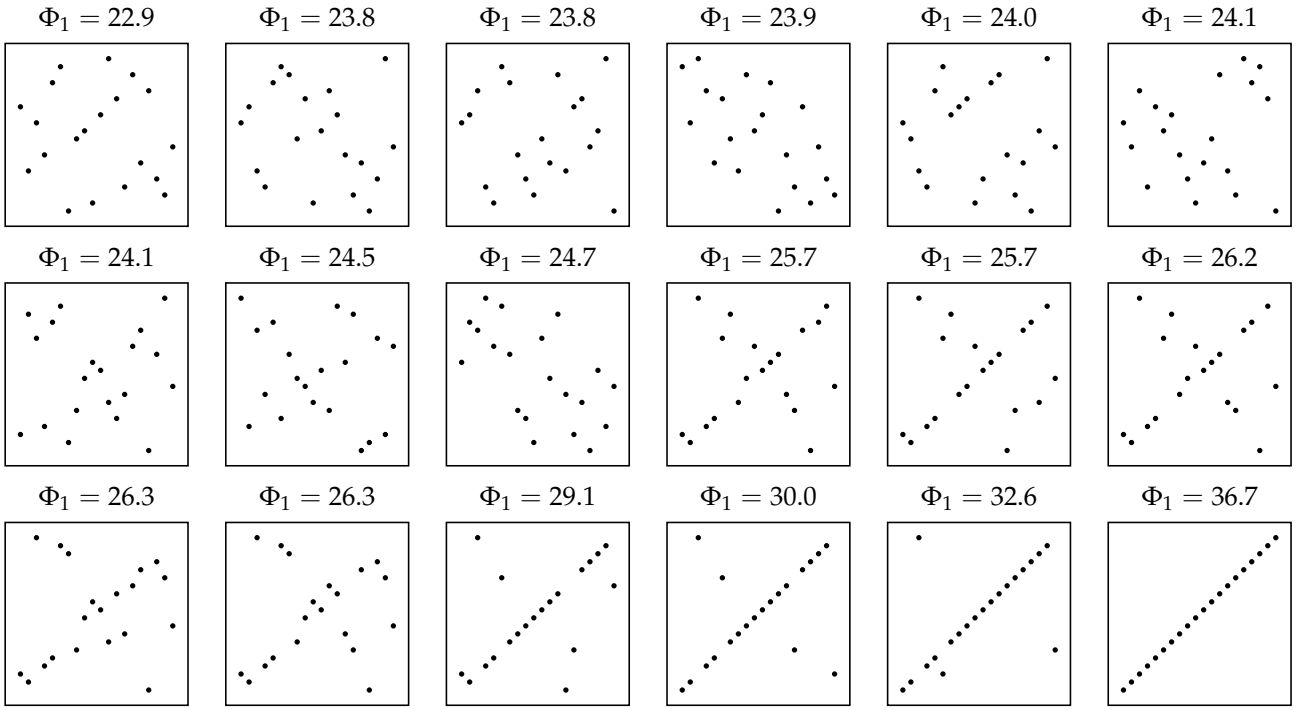


图13.8. 根据{v*}从最优到最差排序的均匀投影计划。

13.6 Space-Filling Subsets

在某些情况下，我们有一组点X，并希望找到一个子集的点$S \subset X$，这些点仍然最大限度地填充X。在多功能模型中，识别X的*space-filling subsets*的需求出现。[10] 例如，假设我们使用了一个采样计划X用于确定多种飞机机翼设计，以便在仿真中使用计算流体动力学模型进行评估。我们只能选择这些设计点的一个子集S来构建并在风洞中进行测试。我们仍然希望S是空间填充的。

10 A. I. J. Forrester, A. Sóbester, and A. J. Keane, ‘‘Multi-Fidelity Op- timization via Surrogate Mod- elling,’’ *Proceedings of the Royal Society of London A: Mathematical, Physical and Engineering Sciences*, vol. 463, no. 2088, pp. 3251–3269, 2007

S 填充设计空间的程度可以通过 X 中的一点与 S 中最近点之间的最大距离来量化。这个度量可以推广到任意两个有限集合 A 和 B (算法 13.7)。我们可以使用任何 L_p 范数，但通常使用 L_2，即欧几里得距离：

$$d_{\max}(X, S) = \underset{\mathbf{x} \in X}{\text{maximize}} \, \underset{\mathbf{s} \in S}{\text{minimize}} \|\mathbf{s} - \mathbf{x}\|_p \tag{13.4}$$

```
min_dist(a, B, p) = minimum(norm(a-b, p) for b in B)
d_max(A, B, p=2) = maximum(min_dist(a, B, p) for a in A)
```

算法13.7。两个离散集合之间的距离度量集合L_p，其中A和B是设计点列表，p是L_p范数参数。

一种空间填充抽样方案就是使该指标最小化的方案。[11] 寻找一个具有m个元素的填空间抽样计划是一个优化问题

$$\begin{aligned} \underset{S}{\text{minimize}} \quad & d_{\max}(X, S) \\ \text{subject to} \quad & S \subseteq X \\ & \#S = m \end{aligned} \tag{13.5}$$

11 我们也可以针对 S 最小化莫里斯-米切尔准则。

优化方程(13.5)通常是计算上不可行的。一种暴力方法会尝试所有$d!/m!(d-m)!$个大小为m的子集，对于包含d个设计点的数据集而言。*greedy local search* (算法13.8)和*exchange algorithm* (算法13.9)都是克服这一难题的启发式策略。它们通常能找到可接受的X空间填充子集。

贪婪局部搜索从 X 中随机选择的一个点开始，然后逐步添加下一个能最小化距离度量的最佳点。持续添加点，直到达到所需的点数。由于点的是随机初始化的，通过多次运行贪婪局部搜索并保留最佳采样方案（算法13.10），可以获得最佳结果。

```
function greedy_local_search(X, m, d=d_max)
    S = [X[rand(1:m)]]
    for i in 2 : m
        j = argmin([x ∈ S ? Inf : d(X, push!(copy(S), x))
                    for x in X])
        push!(S, X[j])
    end
    return S
end
```

算法13.8. 贪婪局部搜索，用于为离散集X寻找最小化距离度量d的m元素抽样计划。

交换算法将S初始化为X的随机子集，并反复将S中的点替换为X中尚未在S中的不同点，以改进距离度量。交换算法通常也会运行多次。

图13.9比较了使用贪婪局部搜索和交换算法得到的空间填充子集。

13.7 Quasi-Random Sequences

Quasi-random sequences，12 也称为 *low-discrepancy sequences*，常用于尝试近似多维空间上的积分时：

$$\int_{\mathcal{X}} f(\mathbf{x})\, d\mathbf{x} \approx \frac{v}{m} \sum_{i=1}^{m} f(\mathbf{x}^{(i)}) \tag{13.6}$$

其中每个 $\mathbf{x}^{(i)}$ 是在定义域 $\mathcal{X}$ 上均匀随机采样的，且 v 是 $\mathcal{X}$ 的体积。这种近似被称为 *Monte Carlo integration*。

与其依赖随机数或伪随机数来生成积分点，准随机序列是确定性序列，以系统化的方式填充空间，从而使积分随点数的增加尽可能快地收敛m。[13] 这些 *quasi-Monte Carlo methods*具有误差收敛

$O(1/m)$ 的收敛性，与典型蒙特卡洛积分中的 $O(1/\sqrt{m})$ 相反，如图 13.10 所示。

12 C. Lemieux, *Monte Carlo and Quasi-Monte Carlo Sampling*. Springer, 2009.

13 伪随机数序列，例如通过一系列对rand的调用产生的序列，在给定特定种子的情况下是确定性的，但它们看起来是随机的。准随机数也是确定性的，但看起来并不随机。

```
function exchange_algorithm(X, m, d=d_max)
    S = X[randperm(m)]
    δ, done = d(X, S), false
    while !done
        best_pair = (0,0)
        for i in 1 : m
            s = S[i]
            for (j,x) in enumerate(X)
                if !in(x, S)
                    S[i] = x
                    δ′ = d(X, S)
                    if δ′ < δ
                        δ = δ′
                        best_pair = (i,j)
                    end
                end
            end
            S[i] = s
        end
        done = best_pair == (0,0)
        if !done
            i,j = best_pair
            S[i] = X[j]
        end
    end
    return S
end
```

算法13.9。用于寻找离散集合X上最小化距离度量d的m元素抽样计划的交换算法。

```
function multistart_local_search(X, m, alg, k_max, d=d_max)
    sets = [alg(X, m, d) for i in 1 : k_max]
    return sets[argmin([d(X, S) for S in sets])]
end
```

算法 13.10. 多起点局部搜索多次运行特定搜索算法并返回最佳结果。这里，`X` 是点的列表，`m` 是所需采样计划的大小，`alg` 是 `exchange_algorithm` 或 `greedy_local_search`，`k_max` 是要运行的迭代次数，`d` 是距离度量。

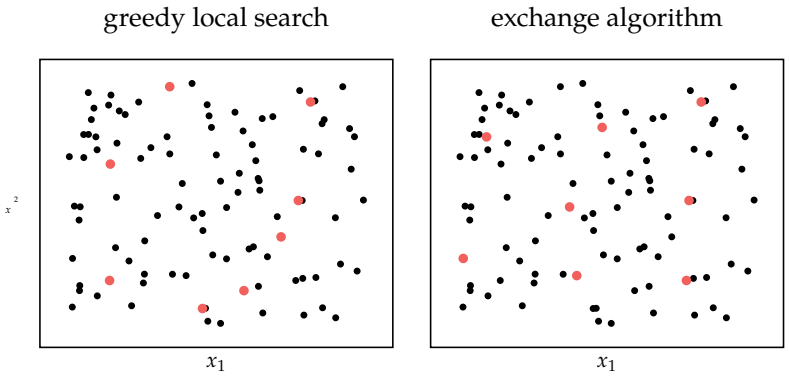


图13.9. 使用贪心局部搜索和交换算法得到的空间填充子集。

拟随机序列通常是针对单位n维超立方体$[0, 1]^n$构建的。任何对每个变量都有边界约束的多维函数都可以变换到这样的超立方体上。这一变换在算法7.9中实现。

存在多种生成准随机序列的方法。图13.12中将几种此类方法与随机采样进行了比较。

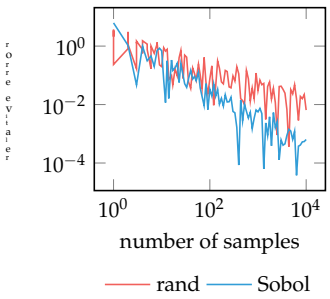


图13.10. 使用来自{v7}0, 1{v8}的随机数和Sobol序列进行蒙特卡洛积分估计{v4} sin(10{v5}) {v6}时的误差。第13.7.3节中介绍的Sobol序列收敛更快。

13.7.1 *Additive Recurrence*

形式为以下内容的简单递推关系：

$$x^{(k+1)} = x^{(k)} + c \pmod 1 \tag{13.7}$$

生成空间填充集，前提是 c 是无理数。导致最小差异的 c 值是

$$c = 1 - \varphi = \frac{\sqrt{5} - 1}{2} \approx 0.618034 \tag{13.8}$$

其中 φ 是黄金比例。[14]

我们可以使用每个坐标的*additive recur-rence*序列，在n维上构造一个空间填充集，每个序列都有自己的c值。已知素数的平方根是无理数，因此可以用来为每个坐标获得不同的序列：

$$c_1 = \sqrt{2}, \quad c_2 = \sqrt{3}, \quad c_3 = \sqrt{5}, \quad c_4 = \sqrt{7}, \quad c_5 = \sqrt{11}, \quad \ldots \tag{13.9}$$

14 C. Schretter、L. Kobbelt 和 P.-O. Dehaye，“低差异采样的黄金比例序列”，*Journal of Graphics Tools*，第16卷，第2期，第95-104页，2016年。

加法递推的方法在算法13.11中实现。

```
using Primes
function get_filling_set_additive_recurrence(m; c=φ-1)
    X = [rand()]
    for i in 2 : m
        push!(X, mod(X[end] + c, 1))
    end
    return X
end
function get_filling_set_additive_recurrence(m, n)
    ps = primes(max(ceil(Int, n*(log(n) + log(log(n)))), 6))
    seqs = [get_filling_set_additive_recurrence(m, c=sqrt(p))
            for p in ps[1:n]]
    return [collect(x) for x in zip(seqs...)]
end
```

算法13.11. 用于构造n维单位超立方体上m元填充序列的加法递推。使用Primes包生成前n个素数，其中第k个素数受限于

$$k(\text{日志}\, k + \text{日志}\, \text{日志}\, k)$$

对于 $k > 6$。

13.7.2 *Halton Sequence*

*Halton sequence*是一个多维准随机空间填充集。[15]
单维版本，称为*van der Corput sequences*，生成序列，其中单位区间被分为基数b的幂。例如，$b = 2$产生：

15 J. H. Halton, “算法247：根式逆序准随机点序列”，*Communications of the ACM*，第7卷，第12期，第701-702页，1964年。

$$X = \left\{ \frac{1}{2}, \frac{1}{4}, \frac{3}{4}, \frac{1}{8}, \frac{5}{8}, \frac{3}{8}, \frac{7}{8}, \frac{1}{16}, \ldots \right\} \tag{13.10}$$

而 $b = 5$ 产生：

$$X = \left\{ \frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}, \frac{1}{25}, \frac{6}{25}, \frac{11}{25}, \ldots \right\} \tag{13.11}$$

多维空间填充序列为每个维度使用一个van der Corput序列，每个序列都有自己的基b。然而，这些基必须是*coprime*[16]。
为了不相关。构造 Halton 序列的方法在算法13.12中实现。

16 如果两个整数的唯一正整数公约数是1，则这两个整数互质。

```
using Primes
function halton(i, b)
    result, f = 0.0, 1.0
    while i > 0
        f = f / b;
        result = result + f * mod(i, b)
        i = floor(Int, i / b)
    end
    return result
end
get_filling_set_halton(m; b=2) = [halton(i,b) for i in 1: m]
function get_filling_set_halton(m, n)
    bs = primes(max(ceil(Int, n*(log(n) + log(log(n)))), 6))
    seqs = [get_filling_set_halton(m, b=b) for b in bs[1:n]]
    return [collect(x) for x in zip(seqs...)]
end
```

算法13.12. 基于b进制的Halton准随机m元素填充序列，覆盖n维单位超立方体，其中基础数bs必须互质。

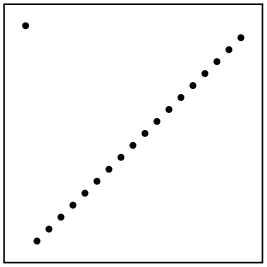


图13.11. 具有b = [19, 23] 的Halton序列，其前18个样本完全线性相关。

对于大素数，我们可以在前几个数中得到相关性。这种相关性如图13.11所示。可以通过*leaped Halton method*、[17]来避免相关性，它取每第p个点，其中p是一个与所有

坐标基。

17 L. Kocis 和 W. J. Whiten，“低差异序列的计算研究”，*ACM Transactions on Mathematical Software*，第23卷，第2期，第266-294页，1997年。

13.7.3 Sobol Sequences

Sobol sequences 是用于 n 维超立方体的准随机空间填充序列。[18]它们通过将前一个 Sobol 数与一个集合进行异或生成

方向编号：[19]

$$X_j^{(i)} = X_j^{(i-1)} \veebar v_j^{(k)} \tag{13.12}$$

其中 $v_j^{(k)}$ 是第 k 个方向数的第 j 位。多位作者已提供了优质方向数表。[20]

图13.12展示了这些方法与先前方法的比较。对于高值，若干方法展现出明显的底层结构。

18 I. M. Sobol，“关于立方体中点的分布及积分的近似求值”，*USSR Computational Mathematics and Mathematical Physics*，第7卷，第4期，第86-112页，1967年。

19 符号$\veebar$表示异或操作，当且仅当两个输入不同时返回真。

20 `Sobol.jl` 包提供了最高支持 1,111 维度的实现。

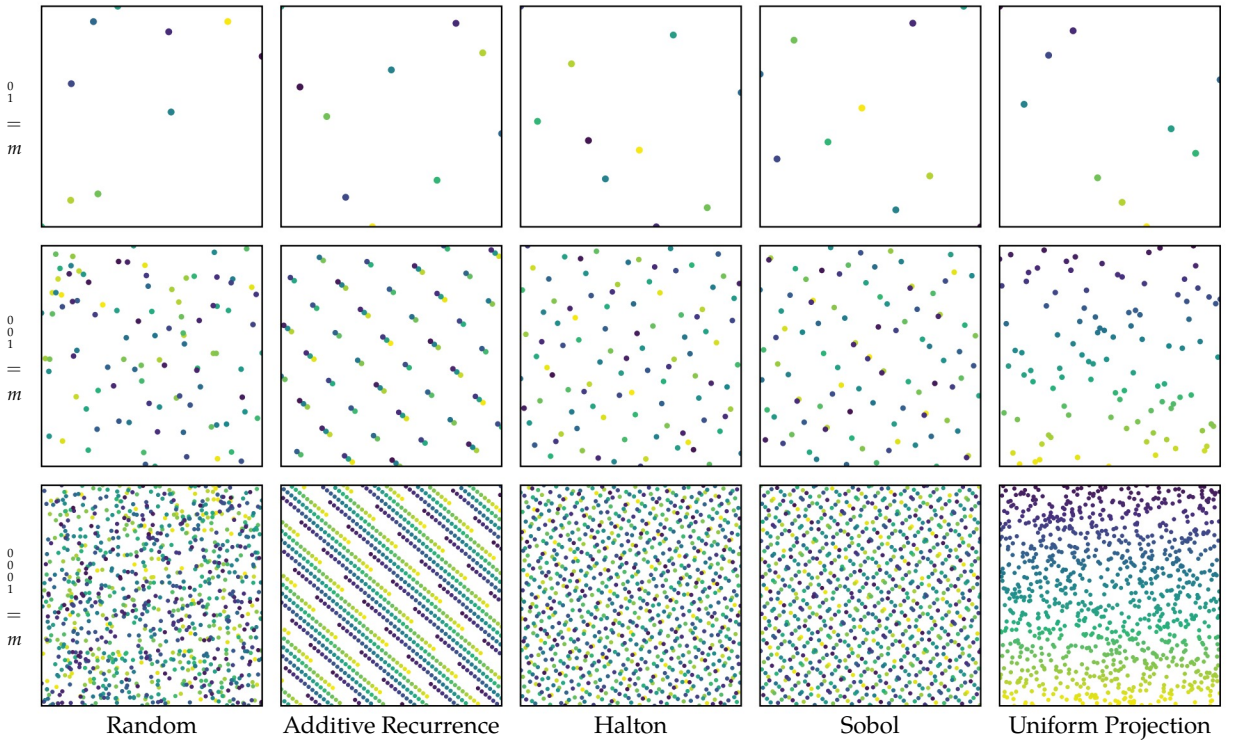


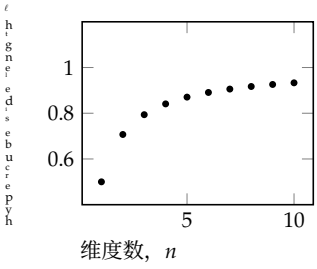
图13.12. 二维空间填充抽样计划的比较。样本根据抽样顺序着色。均匀投影计划是随机生成的，未经过优化。

13.8 Summary

- 抽样计划用于以有限数量的点覆盖搜索空间。
- 全因子采样，即在均匀离散化网格的顶点处采样，需要随维度数量指数增长的点数。
- 均匀投影方案在每个维度上均匀投影，能够高效生成并可优化以实现空间填充。
- 贪心局部搜索和交换算法可用于找到能最大程度填充空间的点子集。
- 准随机序列是确定性过程，通过它可以生成空间填充采样计划。

13.9 Exercises

练习 13.1。填充多维空间所需的点随维度增加呈指数级增长。为了帮助建立这种直觉，确定一个 n 维超立方体的边长，使其体积为 n 维单位超立方体体积的一半。



练习13.2。假设你在一个n维的单位球体内随机抽样。计算随机抽取的点距离球面ϵ以内的概率，当$n \to \infty$时？提示：球体的体积是$C(n)r^n$，其中r是半径，$C(n)$仅是维度n的函数。

练习 13.3。假设我们有一个抽样计划 $X = \{\mathrm{x}(1), \ldots, \mathrm{x}(10)\}$，其中

$$\mathbf{x}^{(i)} = [\cos(2\pi i/10), \sin(2\pi i/10)] \tag{13.13}$$

使用 L_2 范数计算 X 的 Morris-Mitchell 准则，当参数 q 设为 2 时。换句话说，评估 $\Phi_2(X)$。如果我们将 [2, 3] 加到每个 $\mathrm{x}^{(i)}$ 上，$\Phi_2(X)$ 会改变吗？为什么？

练习 13.4. 加法递推要求方程(13.7)中的乘数因子 c 是无理数。为什么 c 不能是有理数?

14 Surrogate Models

上一章讨论了生成采样计划的方法。本章将展示如何使用这些样本来构建目标函数的模型，以替代真实的目标函数。此类*surrogate models*旨在平滑且评估成本低廉，以便能够高效地进行优化。随后，代理模型可用于辅助指导寻找真实目标函数最优解的搜索过程。

14.1 Fitting Surrogate Models

一个以θ为参数的代理模型$\hat{f}$被设计用来模拟真实目标函数f。这些参数θ可以根据从f收集的样本进行调整以拟合模型。一个示例代理模型如图14.1所示。

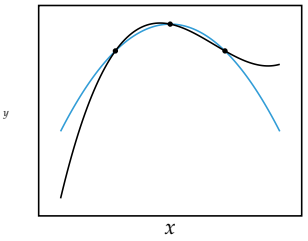


图14.1. 代理模型近似真实目标函数。该模型拟合于已评估的设计点，但偏离这些点越远。

假设我们有m个设计点

$$X = \left\{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots, \mathbf{x}^{(m)}\right\} \tag{14.1}$$

以及相关的功能评估

$$\mathbf{y} = \left\{y^{(1)}, y^{(2)}, \ldots, y^{(m)}\right\} \tag{14.2}$$

对于一组特定的参数，模型将预测

$$\hat{\mathbf{y}} = \left\{\hat{f}_\theta(\mathbf{x}^{(1)}), \hat{f}_\theta(\mathbf{x}^{(2)}), \ldots, \hat{f}_\theta(\mathbf{x}^{(m)})\right\} \tag{14.3}$$

将模型拟合到一组点需要调整参数，以最小化真实评估值与模型预测值之间的差异，通常依据 L_p 范数：[1]

$$\underset{\theta}{\text{minimize}} \quad \|\mathbf{y} - \hat{\mathbf{y}}\|_p \tag{14.4}$$

1 通常使用 L_2 范数。使用 L_2 范数最小化此方程等价于最小化这些数据点处的均方误差。

方程（14.4）仅在数据点处惩罚模型的偏差。无法保证模型在观测数据之外仍能良好拟合，而且随着离采样点越来越远，模型的准确性通常会降低。

这种模型拟合形式称为*regression*。有大量工作致力于解决回归问题，并且在机器学习中得到了广泛研究。[2]

本章其余部分介绍了几种常用的代理模型及拟合数据以构建代理模型的算法，最后总结了在不同类型模型间进行选择的方法。

2 K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.

14.2 Linear Models

一个简单的代理模型是*linear model*，其形式为[3]

$$\hat{f} = w_0 + \mathbf{w}^\top \mathbf{x} \qquad \theta = \{w_0, \mathbf{w}\} \tag{14.5}$$

对于一个 n 维的设计空间，线性模型有 $n+1$ 个参数，因此至少需要 $n+1$ 个样本才能无歧义地拟合。

不是将 w 和 w_0 都作为参数，通常构建一个参数向量 $\theta = [w_0, \mathrm{w}]$，并在向量 x 前面加上1，得到

$$\hat{f} = \theta^\top \mathbf{x} \tag{14.6}$$

找到最优的θ需要解决一个*linear regression*问题：

$$\underset{\theta}{\text{minimize}} \quad \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2 \tag{14.7}$$

这等价于求解

$$\underset{\theta}{\text{minimize}} \quad \|\mathbf{y} - \mathbf{X}\theta\|_2^2 \tag{14.8}$$

其中 X 是由 m 个数据点形成的 *design matrix*

$$\mathbf{X} = \begin{bmatrix} (\mathbf{x}^{(1)})^\top \\ (\mathbf{x}^{(2)})^\top \\ \vdots \\ (\mathbf{x}^{(m)})^\top \end{bmatrix} \tag{14.9}$$

3 这个方程可能看起来很熟悉。它是超平面的方程。

```
function design_matrix(X)
    n, m = length(X[1]), length(X)
    return [j==0 ? 1.0 : X[i][j] for i in 1:m, j in 0:n]
end
function linear_regression(X, y)
    θ = pinv(design_matrix(X))*y
    return x -> θ⋅[1; x]
end
```

算法14.1。一种从设计点列表X构建设计矩阵的方法，以及一种使用线性回归对设计点列表X和目标函数值向量y进行拟合代理模型的方法。

算法14.1实现了计算设计矩阵和解决线性回归问题的方法。线性回归的几种情况展示在图14.2中。

线性回归有解析解

$$\theta = \mathbf{X}^{+}\mathbf{y} \tag{14.10}$$

其中 $\mathbf{X}^{+}$ 是 $\mathbf{X}$ 的 Moore-Penrose *pseudoinverse*。

如果 $\mathbf{X}^{\top}\mathbf{X}$ 可逆，则伪逆可以计算为

$$\mathbf{X}^{+} = \left(\mathbf{X}^{\top}\mathbf{X}\right)^{-1}\mathbf{X}^{\top} \tag{14.11}$$

如果$\mathbf{X}\mathbf{X}^{\top}$可逆，则伪逆可计算为

$$\mathbf{X}^{+} = \mathbf{X}^{\top}\left(\mathbf{X}\mathbf{X}^{\top}\right)^{-1} \tag{14.12}$$

函数`pinv`计算给定矩阵的伪逆。[4]

4 函数 `pinv` 使用奇异值分解 $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{*}$ 来计算伪逆：

$$\mathbf{X}^{+} = \mathbf{V}\mathbf{\Sigma}^{+}\mathbf{U}^{*}$$

其中对角矩阵$\mathbf{\Sigma}$的伪逆是通过取对角线上每个非零元素的倒数，然后转置结果得到的。

14.3 Basis Functions

线性模型是x各分量的线性组合：

$$\hat{f}(\mathbf{x}) = \theta_1 x_1 + \cdots + \theta_n x_n = \sum_{i=1}^{n} \theta_i x_i = \theta^{\top}\mathbf{x} \tag{14.13}$$

这是*basis functions*更一般的线性组合的一个具体例子

$$\hat{f}(\mathbf{x}) = \theta_1 b_1(\mathbf{x}) + \cdots + \theta_q b_q(\mathbf{x}) = \sum_{i=1}^{q} \theta_i b_i(\mathbf{x}) = \theta^{\top}\mathbf{b}(\mathbf{x}) \tag{14.14}$$

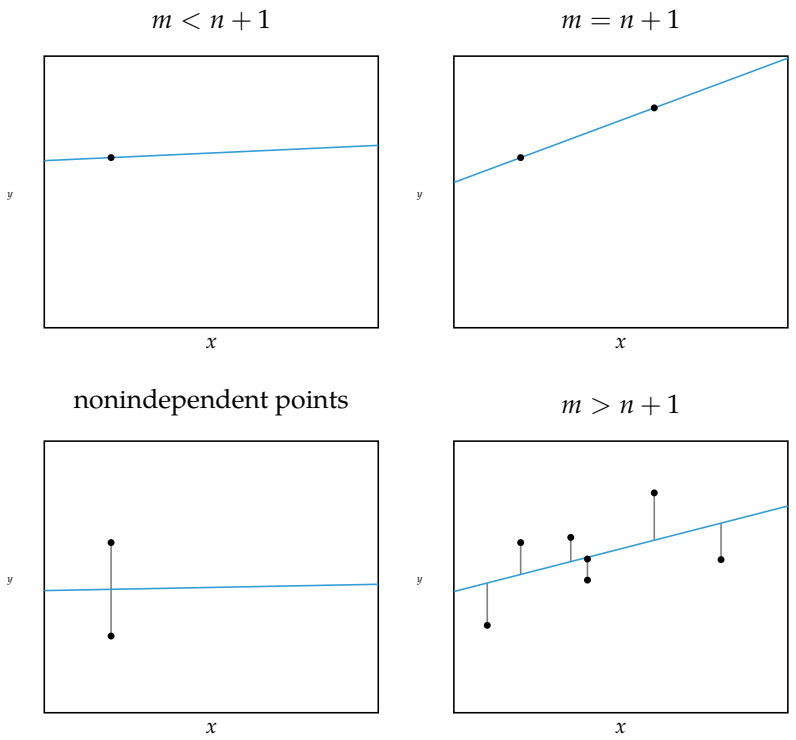


图14.2. 线性回归产生的模型，该模型最小化每个点到模型的垂直距离的平方。伪逆对于任何非空点配置产生唯一解。

左下方子图显示了两个重复点得到的模型，即$m = n + 1$。由于这两个条目重复，矩阵X是非奇异的。尽管在这种情况下X没有逆矩阵，但伪逆产生了一个穿过这两个点的唯一解。

在线性回归的情况下，基函数仅仅提取每个分量，$b_i(\mathbf{x}) = x_i$。

任何表示为基函数线性组合的代理模型都可以使用回归进行拟合：

$$\underset{\boldsymbol{\theta}}{\text{minimize}} \quad \|\mathbf{y} - \mathbf{B}\boldsymbol{\theta}\|_2^2 \tag{14.15}$$

其中 B 是由 m 数据点构成的基础矩阵：

$$\mathbf{B} = \begin{bmatrix} \mathbf{b}(\mathbf{x}^{(1)})^\top \\ \mathbf{b}(\mathbf{x}^{(2)})^\top \\ \vdots \\ \mathbf{b}(\mathbf{x}^{(m)})^\top \end{bmatrix} \tag{14.16}$$

加权参数可以使用伪逆得到

$$\boldsymbol{\theta} = \mathbf{B}^+\mathbf{y} \tag{14.17}$$

算法14.2实现了这个更通用的回归过程。

```
using LinearAlgebra
function regression(X, y, bases)
    B = [b(x) for x in X, b in bases]
    θ = pinv(B)*y
    return x -> sum(θ[i] * bases[i](x) for i in 1 : length(θ))
end
```

算法14.2. 一种使用包含在`bases`数组中的基函数进行回归，以将代理模型拟合到设计点列表`X`及其对应的目标函数值`y`的方法。

线性模型无法捕捉非线性关系。还有多种其他基函数族可以表示更具表现力的代理模型。本节剩余部分将讨论几种常见的族。

14.3.1 Polynomial Basis Functions

Polynomial basis functions 由设计向量分量的乘积组成，每个分量都乘以幂。线性基函数是多项式基函数的特例。

根据泰勒级数展开式[5]，我们知道任何无限可微的函数可以通过足够次数的多项式很好地近似。我们可以使用算法14.3来构造这些基。

5 在附录 C.2 中涵盖。

在一维情况下，次数为k的多项式模型形式为

$$\hat{f}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \cdots + \theta_k x^k = \sum_{i=0}^{k} \theta_i x^i \tag{14.18}$$

因此，我们有一组基函数$b_i(x) = x^i$，用于i，范围从0到k。

在二维情况下，一个k次多项式模型具有形式为

$$b_{ij}(\mathbf{x}) = x_1^i x_2^j \text{ for } i, j \in \{0, \ldots, k\}, \; i + j \leq k \tag{14.19}$$

拟合多项式代理模型是一个回归问题，因此多项式模型在高维空间中是线性的（图14.3）。任何基函数的线性组合都可以视为高维空间中的线性回归。

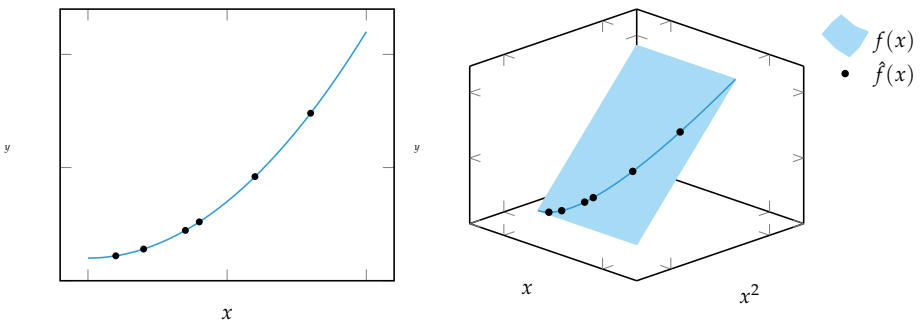


图14.3. 高维空间中的多项式模型呈线性。该函数存在于由其基底构成的平面中，但由于各项并非独立，它并未占据整个平面。

14.3.2 *Sinusoidal Basis Functions*

任何有限域上的连续函数都可以用无限集合 *sinusoidal basis functions*.[6] 来表示。对于任何

区间$[a, b]$上的可积一元函数f

$$f(x) = \frac{\theta_0}{2} + \sum_{i=1}^{\infty} \theta_i^{(\sin)} \sin\left(\frac{2\pi i x}{b-a}\right) + \theta_i^{(\cos)} \cos\left(\frac{2\pi i x}{b-a}\right) \tag{14.20}$$

6 如果函数是周期的，傅里叶级数对于定义在整个实数线上的函数也是精确的。

```
polynomial_bases_1d(i, k) = [x->x[i]^p for p in 0:k]
function polynomial_bases(n, k)
    bases = [polynomial_bases_1d(i, k) for i in 1 : n]
    terms = Function[]
    for ks in product([0:k for i in 1:n]...)
        if sum(ks) ≤ k
            push!(terms,
                x->prod(b[j+1](x) for (j,b) in zip(ks,bases)))
        end
    end
    return terms
end
```

算法14.3. 一种为设计点的第i个分量构造直至k次的多项式基函数数组的方法，以及一种为直至k次的项构造n维多项式基列表的方法。

其中

$$\theta_0 = \frac{2}{b-a}\int_a^b f(x)\,dx \tag{14.21}$$

$$\theta_i^{(\sin)} = \frac{2}{b-a}\int_a^b f(x)\sin\left(\frac{2\pi i x}{b-a}\right)dx \tag{14.22}$$

$$\theta_i^{(\cos)} = \frac{2}{b-a}\int_a^b f(x)\cos\left(\frac{2\pi i x}{b-a}\right)dx \tag{14.23}$$

正如泰勒级数的前几项用于多项式模型一样，傅里叶级数的前几项也用于正弦模型。在域$x \in [a, b]$上，单个分量的基为：

$$\begin{cases} b_0(x) & = 1/2 \\ b_i^{(\sin)}(x) & = \sin\left(\frac{2\pi i x}{b-a}\right) \\ b_i^{(\cos)}(x) & = \cos\left(\frac{2\pi i x}{b-a}\right) \end{cases} \tag{14.24}$$

我们可以像在多项式模型中组合项一样，为多维正弦模型组合项。算法14.4可用于构建正弦基函数。图14.4展示了正弦回归的几种情况。

14.3.3 Radial Basis Functions

一个*radial function* ψ是指仅依赖于点到某个中心点c的距离的，可以写成$\psi(\mathrm{x}, \mathrm{c}) = \psi(\|\mathrm{x} - \mathrm{c}\|) = \psi(r)$。

```
function sinusoidal_bases_1d(j, k, a, b)
    T = b[j] - a[j]
    bases = Function[x->1/2]
    for i in 1 : k
        push!(bases, x->sin(2π*i*x[j]/T))
        push!(bases, x->cos(2π*i*x[j]/T))
    end
    return bases
end
function sinusoidal_bases(k, a, b)
    n = length(a)
    bases = [sinusoidal_bases_1d(i, k, a, b) for i in 1 : n]
    terms = Function[]
    for ks in product([0:2k for i in 1:n]...)
        powers = [div(k+1,2) for k in ks]
        if sum(powers) ≤ k
            push!(terms,
                x->prod(b[j+1](x) for (j,b) in zip(ks,bases)))
        end
    end
    return terms
end
```

算法14.4。方法`sinusoidal_bases_1d`根据下界`a`和上界`b`，为设计向量的第`i`个分量生成最高次数为`k`的基函数列表。方法`sinusoidal_bases`根据下界向量`a`和上界向量`b`，生成最高次数为`k`的所有基函数组合。

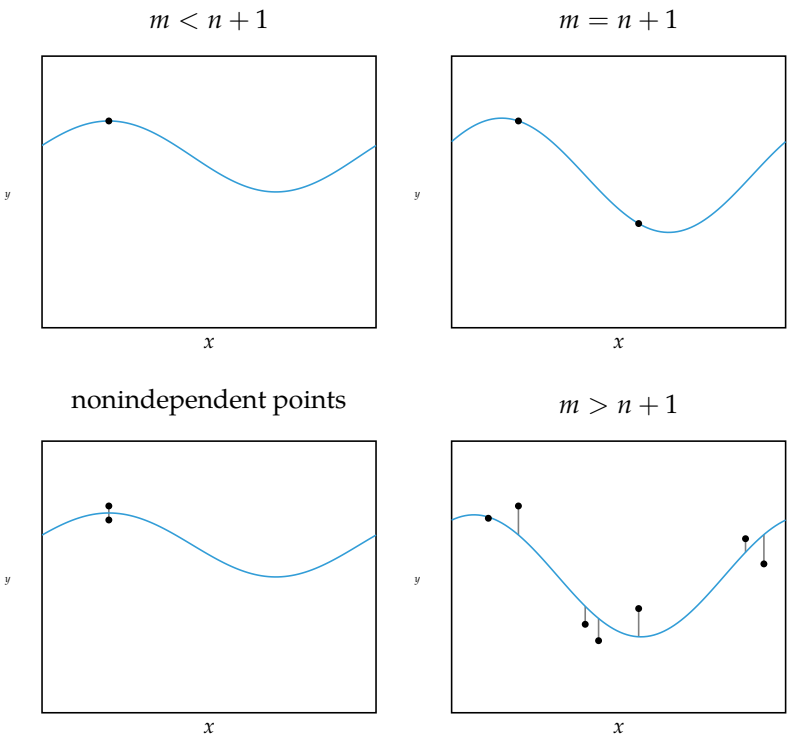


图14.4. 对含噪点拟合正弦模型。

径向函数是方便的基函数，因为在函数景观中放置一个径向函数会形成山峰或山谷。一些常见的径向基函数如图14.5所示。

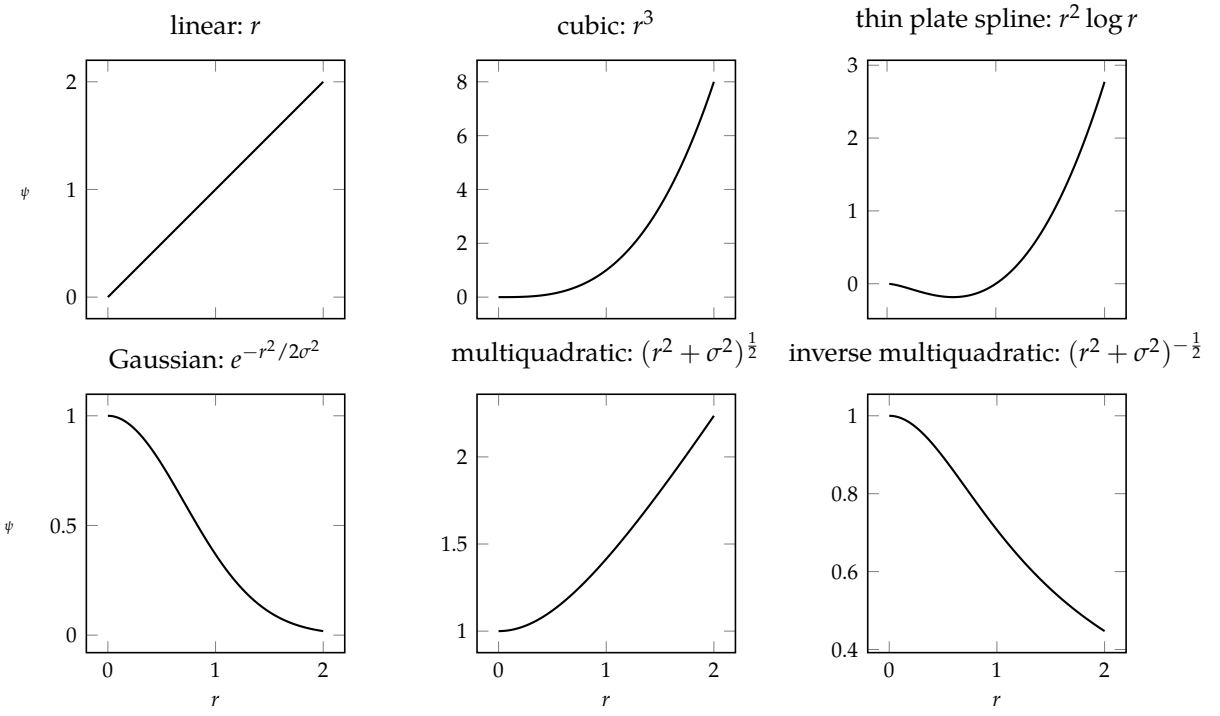


图14.5. 几种径向基函数。

径向基函数需要指定中心点。在将径向基函数拟合到一组数据点时，一种方法是使用数据点作为中心。对于一组 m 个点，因此构造 m 个径向基函数

$$b_i(\mathbf{x}) = \psi(\|\mathbf{x} - \mathbf{x}^{(i)}\|) \quad \text{for } i \in \{1, \ldots, m\} \tag{14.25}$$

相应的 $m \times m$ 基矩阵总是半正定的。算法 14.5 可用于构建具有已知中心点的径向基函数。使用不同径向基函数的代理模型如图 14.6 所示。

```
radial_bases(ψ, C, p=2) = [x->ψ(norm(x - c, p)) for c in C]
```

算法14.5. 一种获取基函数列表的方法，给定径向基函数 `ψ`、中心列表 `C` 以及 L_p 范数参数 `p`。

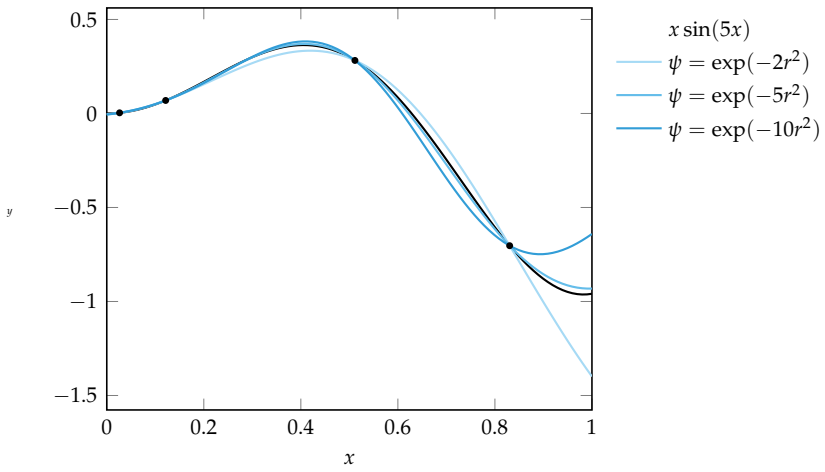


图14.6. 用于根据4个无噪声样本拟合x sin(5x)的几种不同高斯径向基函数。

14.4 *Fitting Noisy Objective Functions*

使用回归拟合的模型将尽可能接近每个设计点。当目标函数评估存在噪声时，复杂模型可能会过度扭曲自身以穿过每一个点。然而，更平滑的拟合通常能更好地预测真实的底层目标函数。

公式（14.15）中规定的基础回归问题可以通过增强来偏好更平滑的解。为了偏好具有较低权重的解，除了预测误差之外，还添加了一个*regularization term*。带有L_2 *regularization*[7]的最终基础回归问题为：

$$\underset{\theta}{\text{minimize}} \quad \|\mathbf{y} - \mathbf{B}\theta\|_2^2 + \lambda\|\theta\|_2^2 \tag{14.26}$$

其中 $\lambda \geq 0$ 是平滑参数，$\lambda = 0$ 表示不进行平滑处理。最优参数向量由下式给出：[8]

$$\theta = \left(\mathbf{B}^\top\mathbf{B} + \lambda\mathbf{I}\right)^{-1}\mathbf{B}^\top\mathbf{y} \tag{14.27}$$

其中 I 是单位矩阵。

算法14.6实现了带有L_2正则化的回归。图14.7展示了使用不同径向基函数的替代模型对含噪样本的拟合情况。

7 其他 L_p 范数，见附录 C.4，同样可以使用。使用 L_1 范数将鼓励稀疏解，即影响力较小的分量权重被设为零，这有助于识别重要的基函数。

8 矩阵 $(\mathbf{B}^\top\mathbf{B} + \lambda\mathbf{I})$ 并不总是可逆的。我们总可以生成一个 λ 足够大的可逆矩阵。

```
function regression(X, y, bases, λ)
    B = [b(x) for x in X, b in bases]
    θ = (B'B + λ*I)\B'y
    return x -> sum(θ[i] * bases[i](x) for i in 1 : length(θ))
end
```

算法14.6. 一种在存在噪声情况下进行回归的方法，其中λ是平滑项。它通过使用基函数bases的回归，返回一个拟合到设计点列表X及相应目标函数值y的代理模型。

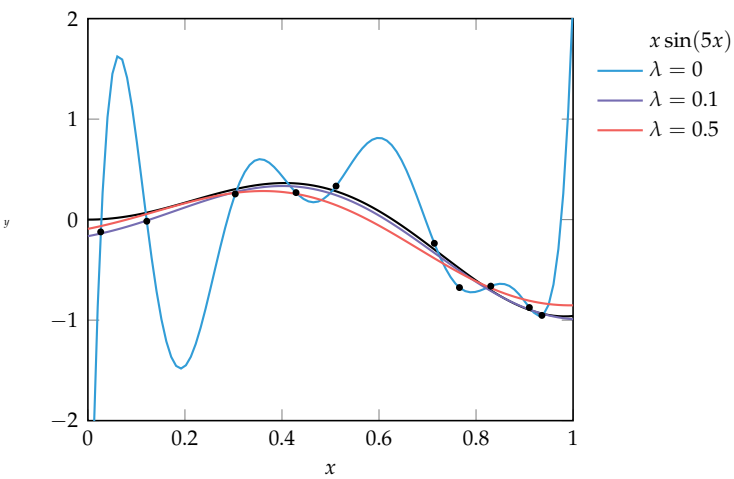


图14.7. 用于拟合$x\sin(5x)$的几种不同高斯径向基函数，基于10个噪声样本，均值为零，标准差为0.1误差，以及径向基函数$\psi = \exp(-5r^2)$。

14.5 Model Selection

到目前为止，我们已经讨论了如何将特定模型拟合到数据。本节解释如何选择使用哪个模型。我们通常希望最小化 *gen- eralization error*，这是模型在整个设计空间上的误差度量，包括可能不包含在用于训练模型的数据中的点。衡量泛化误差的一种方法是使用其预测的期望平方误差：

$$\epsilon_{\text{gen}} = \mathbb{E}_{\mathbf{x}\sim\mathcal{X}}\left[\left(f(\mathbf{x}) - \hat{f}(\mathbf{x})\right)^2\right] \tag{14.28}$$

当然，我们无法精确计算这个泛化误差，因为这需要知道我们试图逼近的函数。人们可能倾向于从*training error*来估计模型的泛化误差。衡量训练误差的一种方法是使用在用于训练的m样本上评估的模型的*mean squared error* (MSE)：

$$\epsilon_{\text{train}} = \frac{1}{m}\sum_{i=1}^{m}\left(f(\mathbf{x}^{(i)}) - \hat{f}(\mathbf{x}^{(i)})\right)^2 \tag{14.29}$$

然而，在训练数据上表现良好并不一定对应于低的泛化误差。复杂模型可能会减少训练集上的误差，但它们可能无法在设计空间中的其他点上提供良好的预测，如例14.1所示。[9]

本节讨论几种估计泛化误差的方法。这些方法在数据子集上进行训练和测试。我们引入`TrainTest`类型（算法14.7），它包含一个训练索引列表和一个测试索引列表。方法`fit`接收一个训练集并生成一个模型。方法`metric`接收一个模型和一个测试集并生成一个度量，如均方误差。方法`train_and_validate` (算法14.7)是一个用于训练然后评估模型的实用函数。尽管在估计泛化误差时我们在数据子集上进行训练，但一旦我们决定了要使用哪个模型，我们就可以在完整数据集上进行训练。

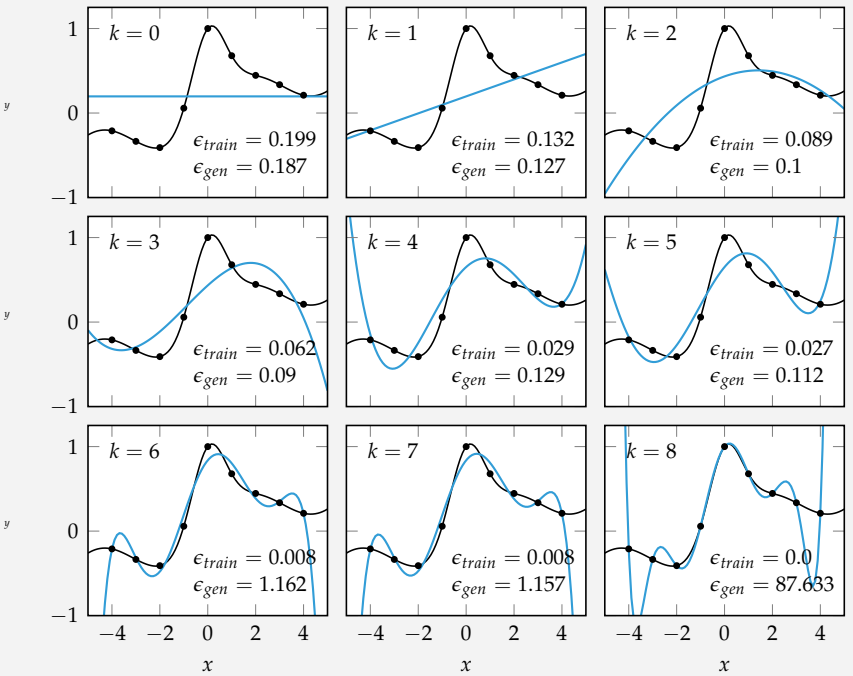
9 机器学习中的一个主要主题是平衡模型复杂度，以避免对训练数据*overfitting*。K. P. Murphy, *Machine Learn-*。麻省理工学院出版社，2012年。

例14.1. 多项式代理模型度数变化时训练误差与泛化误差的比较。

考虑将不同次数的多项式拟合到目标函数的评估值上

$$f(x) = x/10 + \sin(x)/4 + \exp\left(-x^2\right)$$

下面我们绘制了不同次数的多项式代理模型，这些模型使用了在$[-4, 4]$区间上均匀分布的同样九个评估点。训练误差和泛化误差也一并展示，其中泛化误差是在$[-5, 5]$区间上计算的。



该图显示，对于k的极低值和高值，泛化误差均较高，并且随着多项式次数的增加，训练误差减小。高次多项式对于$[-4, 4]$以外的设计尤其不擅长预测。

```
struct TrainTest
    train
    test
end
function train_and_validate(X, y, tt, fit, metric)
    model = fit(X[tt.train], y[tt.train])
    return metric(model, X[tt.test], y[tt.test])
end
```

算法14.7. 一种用于训练模型并根据指标进行验证的实用类型和方法。此处，`train`和`test`是训练数据中索引的列表，`X`是设计点列表，`y`是对应函数评估的向量，`tt`是训练-测试划分，`fit`是模型拟合函数，而`metric`在测试集上评估模型以估计泛化误差。

14.5.1 *Holdout*



图14.8。留出法（左）将数据分为训练集和测试集。

$$\text{train}(\bullet) \rightarrow \text{test}(\hat{f}, \bullet) \rightarrow \text{generalization error estimate}$$

一种估计泛化误差的简单方法是*holdout method*，它将可用数据划分为包含h个样本的*test set* $\mathcal{D}_h$和由所有剩余$m - h$个样本组成的*trainingset* $\mathcal{D}_t$，如图14.8所示。训练集用于拟合模型参数。留出的测试集在模型拟合期间不使用，因此可用于估计泛化误差。根据数据集的大小和性质，使用不同的划分比例，通常从50%训练、50%测试到90%训练、10%测试不等。用于训练的样本太少可能导致拟合不佳（图14.9），而使用太多则会导致泛化估计不佳。

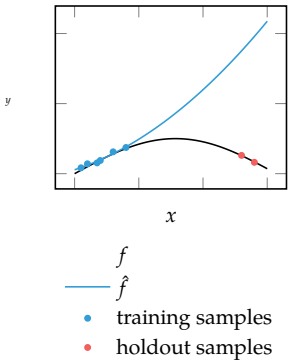


图14.9. 不良的训练-测试集划分可能导致模型性能不佳。

模型$\hat{f}$ 拟合到训练集后的留出误差为

$$\epsilon_{\text{holdout}} = \frac{1}{h} \sum_{(\mathbf{x},y)\in\mathcal{D}_h} \left(y - \hat{f}(\mathbf{x})\right)^2 \tag{14.30}$$

即使分区比例固定，留出误差也会取决于所选的特定训练-测试分区。随机选择分区（算法14.8）仅会给出点估计。在*random subsampling* (算法14.9)中，我们多次应用留出法，并随机选择训练-测试分区。估计的泛化误差是所有运行结果的平均值。[10]因为验证

由于验证集是随机选择的，这种方法不能保证我们在所有数据点上进行验证。

10 所有运行的标准差可用于估计估计泛化误差的标准差。

```
function holdout_partition(m, h=div(m,2))
    p = randperm(m)
    train = p[(h+1):m]
    holdout = p[1:h]
    return TrainTest(train, holdout)
end
```

算法14.8。一种将m个数据样本随机划分为训练集和留出集的方法，其中h个样本被分配到留出集。

```
function random_subsampling(X, y, fit, metric;
    h=div(length(X),2), k_max=10)
    m = length(X)
    mean(train_and_validate(X, y, holdout_partition(m, h),
          fit, metric) for k in 1 : k_max)
end
```

算法14.9。使用保留法的k_max次运行来获得模型泛化误差的均值和标准差估计的随机子采样方法。

14.5.2 *Cross Validation*

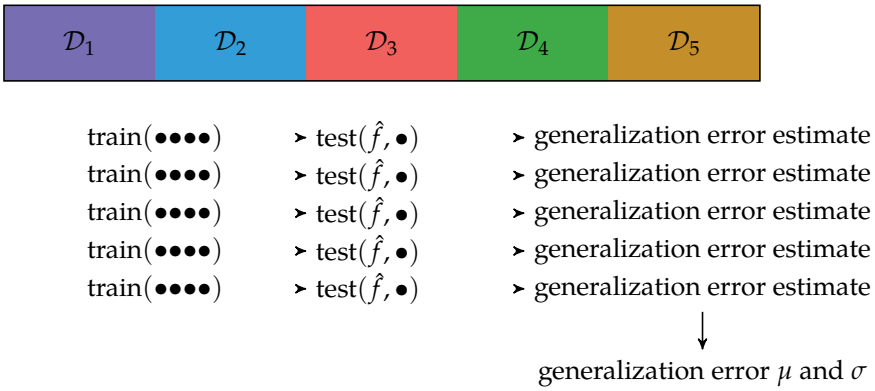


图14.10. 交叉验证将数据划分为大小相等的集合。每个集合轮流作为保留集一次。这里我们展示了5折交叉验证。

使用训练-测试划分可能造成浪费，因为我们的模型调优只能利用数据的一部分。通过使用*k-fold cross validation*往往能获得更好的结果。[11] 这里，原始数据集$\mathcal{D}$被随机划分为k

将大小相等或近似相等的集合 $\mathcal{D}_1$、$\mathcal{D}_2$、……、$\mathcal{D}_k$ 划分，如图 14.10 所示，并在算法 14.10 中实现。然后我们训练 k 个模型，每个模型基于 $k-1$ 个集合中的一个子集，并使用保留集来估计泛化误差。泛化误差的交叉验证估计是所有折上泛化误差的均值：[12]

11 也称为 *rotation estimation*。

12 与随机子采样一样，可以通过各折的标准差来获得方差的估计。

$$\begin{aligned}\epsilon_{\text{cross-validation}} &= \frac{1}{k}\sum_{i=1}^{k}\epsilon^{(i)}_{\text{cross-validation}} \\ \epsilon^{(i)}_{\text{cross-validation}} &= \frac{1}{|\mathcal{D}^{(i)}_{\text{test}}|}\sum_{(\mathbf{x},y)\in\mathcal{D}^{(i)}_{\text{test}}}\left(y-\hat{f}^{(i)}(\mathbf{x})\right)^2\end{aligned} \tag{14.31}$$

其中 $\epsilon^{(i)}_{\text{cross-validation}}$ 和 $\mathcal{D}^{(i)}_{\text{test}}$ 分别是第 i 折的交叉验证估计和留出测试集。

```
function k_fold_cross_validation_sets(m, k)
    perm = randperm(m)
    sets = TrainTest[]
    for i = 1:k
        validate = perm[i:k:m];
        train = perm[setdiff(1:m, i:k:m)]
        push!(sets, TrainTest(train, validate))
    end
    return sets
end
function cross_validation_estimate(X, y, sets, fit, metric)
    mean(train_and_validate(X, y, tt, fit, metric)
         for tt in sets)
end
```

算法14.10。方法 `k_fold_cross_validation_sets` 为`m`个样本构建`k`折交叉验证所需的集合，其中`k ≤ m`。方法 `cross_validation_estimate`通过在`sets`中包含的训练-验证集列表上进行训练和验证，计算泛化误差估计的均值。其他变量包括设计点列表`X`、相应的目标函数值`y`、训练代理模型的函数`fit`以及在数据集上评估模型的函数`metric`。

交叉验证还依赖于特定的数据划分。一个例外是带有 $k = m$ 的 *leave-one-out cross-validation*，它有一个确定性的划分。它尽可能在最多的数据上进行训练，但需要训练 m 个模型。[13] 平均-对所有的 $\binom{m}{m/k}$ 可能划分进行平均，称为*complete cross-validation*，往往代价过高。虽然可以对多次交叉验证取平均，但更常见的做法是对单次交叉验证划分得到的模型取平均。

交叉验证在第14.2节中演示。

13 M. Stone, “交叉验证选择与统计预测评估”, *Journal of the Royal Statistical Society*, 第36卷, 第2期, 第111–147页, 1974年。

14.5.3 The Bootstrap

bootstrap method[14] 使用多个 *bootstrap samples*，这些 *bootstrap samples* 由 m 索引组成到一个大小为m的数据集中，这些数据点是独立且均匀随机选取的。由于索引是有放回地选取的，因此某些索引可能被多次选中，而有些索引则可能完全未被选中，如图14.11所示。自举样本用于拟合一个模型，随后该模型在原始训练集上进行评估。获取自举样本的方法见算法14.11。

[14] B. Efron, “自助法：刀切法的另一种视角”，*The Annals of Statistics*，第 7 卷，第 1–26 页，1979 年。

如果制作了b个自助样本，那么泛化误差的自助估计就是相应的泛化误差估计$\epsilon_{\text{test}}^{(1)}, \ldots, \epsilon_{\text{test}}^{(b)}$的均值：

$$\epsilon_{\text{boot}} = \frac{1}{b}\sum_{i=1}^{b} \epsilon_{\text{test}}^{(i)} \tag{14.32}$$

$$= \frac{1}{m}\sum_{j=1}^{m} \frac{1}{b}\sum_{i=1}^{b} \left(y^{(j)} - \hat{f}^{(i)}(\mathbf{x}^{(j)})\right)^2 \tag{14.33}$$

其中$\hat{f}^{(i)}$ 是拟合到第 i 个自助样本的模型。自助法在算法 14.12 中实现。

方程（14.32）中的自助法误差在模型拟合所依据的数据点上对模型进行测试。*leave-one-out bootstrap estimate*通过仅对留出数据评估拟合模型来消除这一偏差来源：

$$\epsilon_{\text{leave-one-out-boot}} = \frac{1}{m}\sum_{j=1}^{m} \frac{1}{c_{-j}} \sum_{i=1}^{b} \begin{cases} \left(y^{(j)} - \hat{f}^{(i)}(\mathbf{x}^{(j)})\right)^2 & \text{if } j\text{th index was not in the } i\text{th bootstrap sample} \\ 0 & \text{otherwise} \end{cases} \tag{14.34}$$

其中 c_{-j} 是不包含索引 j 的 bootstrap 样本数量。留一 bootstrap 方法在算法 14.13 中实现。

概率　　特定索引不在自助样本中的概率　　ple是：

$$\left(1 - \frac{1}{m}\right)^m \approx e^{-1} \approx 0.368 \tag{14.35}$$

因此，一个自助样本平均预期包含来自原始数据集的0.632m个不同索引。

假设我们想要使用带有噪声超参数 λ（第14.4节）的径向基函数来拟合一个有噪声的目标函数。我们可以使用交叉验证来确定 λ。我们被给定从我们的噪声目标函数中抽取的十个样本。在实践中，目标函数将是未知的，但这个示例使用的是

$$f(x) = \sin(2x)\cos(10x) + \epsilon/10$$

其中 $x \in \in [0, 1]$，且 ϵ 是均值为零、方差为单位方差的随机噪声，$\epsilon \sim \mathcal{N}(0, 1)$。

```
Random.seed!(0)
f = x->sin(2x)*cos(10x)
X = rand(10)
y = f.(X) + randn(length(X))/10
```

我们将使用随机分配的三个折：

```
sets = k_fold_cross_validation_sets(length(X), 3)
```

接下来，我们实现我们的度量标准。我们使用均方误差：

```
metric = (f, X, y)->begin
    m = length(X)
    return sum((f(X[i]) - y[i])^2 for i in m)/m
end
```

我们现在遍历不同的λ值并拟合不同的径向基函数。我们将使用高斯径向基。使用交叉验证来获得每个值的MSE：

```
λs = 10 .^ range(-4, stop=2, length=101)
es = []
basis = r->exp(-5r^2)
for λ in λs
    fit = (X, y)->regression(X, y, radial_bases(basis, X), λ)
    push!(es,
          cross_validation_estimate(X, y, sets, fit, metric)[1])
end
```

得到的曲线在 $\lambda \approx 0.2$ 处有一个最小值。

示例 14.2. 用于拟合超参数的交叉验证。

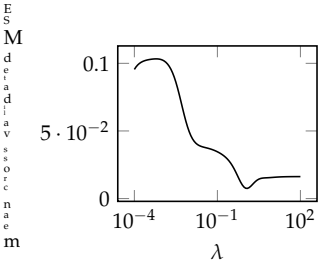




图14.11. 一个单一的bootstrap样本由m个从数据集中通过有放回抽样得到的索引组成。该bootstrap样本用于训练模型，并在整个数据集上进行评估，以估计泛化误差。

```
bootstrap_sets(m, b) = [TrainTest(rand(1:m, m), 1:m) for i in 1:b]
```

算法14.11. 一种获取b个自助样本的方法，每个样本针对大小为m的数据集。

算法14.12。一种通过在sets中包含的训练-验证集列表上进行训练和验证来计算自助法泛化误差估计的方法。其他变量包括设计点列表X、相应的目标函数值y、一个训练代理模型的函数fit，以及一个在数据集上评估模型的函数metric。

```
function bootstrap_estimate(X, y, sets, fit, metric)
    mean(train_and_validate(X, y, tt, fit, metric) for tt in sets)
end
```

```
function leave_one_out_bootstrap_estimate(X, y, sets, fit, metric)
    m, b = length(X), length(sets)
    ε = 0.0
    models = [fit(X[tt.train], y[tt.train]) for tt in sets]
    for j in 1 : m
        c = 0
        δ = 0.0
        for i in 1 : b
            if j ∉ sets[i].train
                c += 1
                δ += metric(models[i], [X[j]], [y[j]])
            end
        end
        ε += δ/c
    end
    return ε/m
end
```

算法14.13. 一种计算留一法自助泛化误差估计的方法，使用训练-验证集sets。其他变量包括设计点列表X、相应的目标函数值y、一个用于训练代理模型的函数fit，以及一个在数据集上评估模型的函数metric。

很遗憾，留一法自助估计由于测试集大小变化而引入了新的偏差。*0.632 bootstrap estimate*[15] (算法14.14)缓解这种偏差：

$$\epsilon_{\text{0.632-boot}} = 0.632\epsilon_{\text{leave-one-out-boot}} + 0.368\epsilon_{\text{boot}} \tag{14.36}$$

15 0.632 自助法估计由 B. Efron 在《预测规则错误率的估计：交叉验证的改进》中引入，*Journal of the American Statistical Association*，第78卷，第382期，第316–331页，1983年。其变体0.632+自助法估计由 B. Efron 和 R. Tibshirani 在《交叉验证的改进：.632+自助法》中引入，*Journal of the American Statistical Association*，第92卷，第438期，第548–560页，1997年。

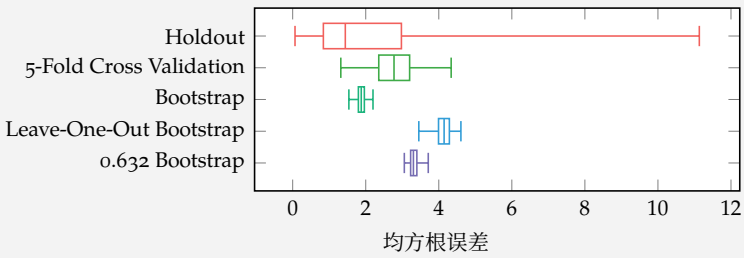
```
function bootstrap_632_estimate(X, y, sets, fit, metric)
    models = [fit(X[tt.train], y[tt.train]) for tt in sets]
    ϵ_loob = leave_one_out_bootstrap_estimate(X,y,sets,fit,metric)
    ϵ_boot = bootstrap_estimate(X,y,sets,fit,metric)
    return 0.632ϵ_loob + 0.368ϵ_boot
end
```

算法14.14. 一种获取数据点 `X` 的0.632自助法估计的方法，目标函数值 `y`，自助样本数量 `b`，拟合函数 `fit`，以及度量函数 `metric`。

示例 14.3 中比较了几种泛化估计方法。

考虑 $f(x) = x^2 + \epsilon/2$ 在 $x \in [-3, 3]$ 上的十个均匀分布样本，其中 ϵ 是零均值、单位方差的高斯噪声。我们希望测试几种不同的泛化误差估计方法，用于将线性模型拟合到这些数据。我们的指标是 *root mean squared error*，即均方误差的平方根。

所采用的方法包括：使用八个训练样本的留出法、五折交叉验证，以及每种各有十个自助样本的自助法。每种方法均拟合了100次，所得统计量如下所示。



示例 14.3. 泛化误差估计方法的比较。*box and whisker* 图中的竖线表示在50次试验中每种泛化误差估计方法的最小值、最大值、第一四分位数、第三四分位数以及中位数。

14.6 Summary

- 代理模型是一种函数近似，可以优化这些模型，而不用优化真正可能成本高昂的目标函数。
- 许多代理模型可以用基函数的线性组合来表示。
- 模型选择涉及低复杂度模型与高复杂度模型之间的偏差-方差权衡，前者无法捕捉重要趋势，后者则会对噪声过拟合。
- 泛化误差可以通过留出法、k折交叉验证和自助法等技术进行估计。

14.7 Exercises

练习 14.1. 通过将梯度设为零，推导回归问题方程（14.8）最优解所满足的表达式。不要对任何矩阵求逆。得到的关系称为*normal equation*。

练习14.2. 何时我们会采用更具描述性的模型，比如包含多项式特征的模型，而非如线性回归这样的简单模型?

练习 14.3. 形如方程 (14.8) 的线性回归问题并不总是通过解析方法求解，而是采用优化技术。这是为什么?

练习14.4。假设我们在四个点上评估目标函数：1、2、3和4，得到的结果分别是0、5、4和6。我们想要拟合一个多项式模型$f(x) = \sum_{i=0}^{k} \theta_i x^i$。计算当 k 在0到4之间变化时，均方误差的留一交叉验证估计。根据这个度量，k 的最佳值是什么，以及 θ 的元素的最佳值是什么?

15 Probabilistic Surrogate Models

上一章讨论了如何根据已评估的设计点构建代理模型。当使用代理模型进行优化时，量化我们对其预测的信心通常很有用。量化信心的一种方法是对代理建模采取概率方法。一种常见的概率代理模型是 *Gaussian process*，它表示函数上的概率分布。本章将解释如何使用高斯过程，在给定先前已评估设计点的值的情况下，推断不同设计点取值上的分布。我们将讨论如何纳入梯度信息以及目标函数的噪声测量。由于高斯过程做出的预测由一组参数控制，我们将讨论如何直接从数据中推断这些参数。

15.1 Gaussian Distribution

在介绍高斯过程之前，我们将首先回顾多元*Gaussian distribution*的一些相关性质，这些性质通常也被称为多元*normal distribution*。[1]一个n维高斯分布是由参数

通过其均值 $\boldsymbol{\mu}$ 和协方差矩阵 $\boldsymbol{\Sigma}$ 来描述。在 x 处的概率密度为

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = (2\pi)^{-n/2}|\boldsymbol{\Sigma}|^{-1/2} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right) \tag{15.1}$$

图15.1展示了不同协方差矩阵下密度函数的等高线图。协方差矩阵总是半正定的。

从高斯分布中采样的值写作

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \tag{15.2}$$

[1] 单变量高斯分布在附录 C.7 中讨论。

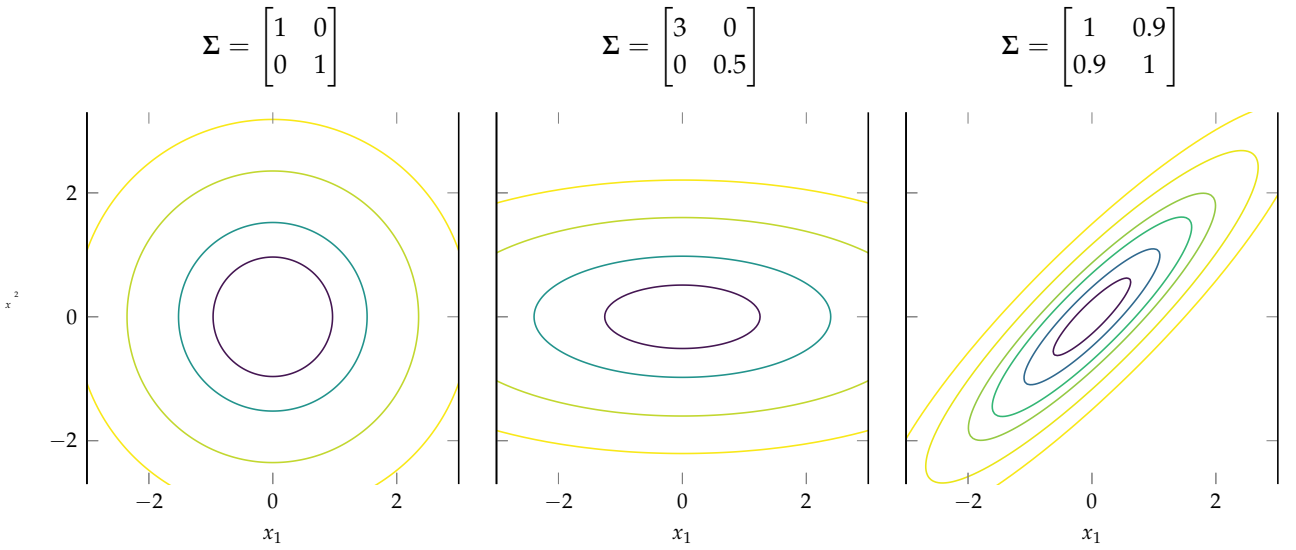


图15.1. 具有不同协方差矩阵的多变量高斯分布。

两个联合高斯随机变量a和b可以写成

$$\begin{bmatrix} \mathbf{a} \\ \mathbf{b} \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \boldsymbol{\mu}_\mathbf{a} \\ \boldsymbol{\mu}_\mathbf{b} \end{bmatrix}, \begin{bmatrix} \mathbf{A} & \mathbf{C} \\ \mathbf{C}^\top & \mathbf{B} \end{bmatrix} \right) \tag{15.3}$$

随机变量向量的 *marginal distribution*[2] 由其相关相应的均值和协方差

$$\mathbf{a} \sim \mathcal{N}(\boldsymbol{\mu}_\mathbf{a}, \mathbf{A}) \qquad \mathbf{b} \sim \mathcal{N}(\boldsymbol{\mu}_\mathbf{b}, \mathbf{B}) \tag{15.4}$$

多元高斯分布的 *conditional distribution* 也有一个便捷的闭式解：

$$\mathbf{a} \mid \mathbf{b} \sim \mathcal{N}\left(\boldsymbol{\mu}_{\mathbf{a}|\mathbf{b}}, \boldsymbol{\Sigma}_{\mathbf{a}|\mathbf{b}}\right) \tag{15.5}$$

$$\boldsymbol{\mu}_{\mathbf{a}|\mathbf{b}} = \boldsymbol{\mu}_\mathbf{a} + \mathbf{C}\mathbf{B}^{-1}(\mathbf{b} - \boldsymbol{\mu}_\mathbf{b}) \tag{15.6}$$

$$\boldsymbol{\Sigma}_{\mathbf{a}|\mathbf{b}} = \mathbf{A} - \mathbf{C}\mathbf{B}^{-1}\mathbf{C}^\top \tag{15.7}$$

示例15.1展示了如何从多元高斯分布中提取边缘分布和条件分布。

2 边缘分布是当其余变量被积分掉或边缘化时，变量子集的分布。对于两个变量a和b上的分布，关于a的边缘分布是：

$$p(a) = \int p(a, b)\, db$$

例15.1。多元高斯分布的边缘分布和条件分布。

例如，考虑

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix} \right)$$

x_1 的边缘分布为 $\mathcal{N}(0, 3)$，而 x_2 的边缘分布为 $\mathcal{N}(1, 2)$。

条件分布 x_1 在给定 $x_2 = 2$ 的情况下是

$$\begin{aligned} \boldsymbol{\mu}_{x_1|x_2=2} &= 0 + 1 \cdot 2^{-1} \cdot (2 - 1) = 0.5 \\ \boldsymbol{\Sigma}_{x_1|x_2=2} &= 3 - 1 \cdot 1^{-1} \cdot 1 = 2.5 \\ x_1 \mid (x_2 = 2) &\sim \mathcal{N}(0.5, 2.5) \end{aligned}$$

15.2 Gaussian Processes

在上一章中，我们使用拟合到先前评估的设计点的代理模型函数$\hat{f}$来近似目标函数f。一种特殊类型的代理模型，称为*Gaussian process*，不仅允许我们预测f，还允许我们使用概率分布[3]来量化预测中的不确定性。

3 C. E. Rasmussen 和 C. K. I. Williams 提供了更全面的高斯过程介绍，*Gaussian Processes for Machine Learning*。MIT Press，2006年。

高斯过程是函数的分布。对于任意有限点集 $\{\mathrm{x}(1), \ldots, \mathrm{x}(m)\}$，相关联的函数值 $\{y_1, \ldots, y_m\}$ 的分布依据：

$$\begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} m(\mathbf{x}^{(1)}) \\ \vdots \\ m(\mathbf{x}^{(m)}) \end{bmatrix}, \begin{bmatrix} k(\mathbf{x}^{(1)}, \mathbf{x}^{(1)}) & \cdots & k(\mathbf{x}^{(1)}, \mathbf{x}^{(m)}) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}^{(m)}, \mathbf{x}^{(1)}) & \cdots & k(\mathbf{x}^{(m)}, \mathbf{x}^{(m)}) \end{bmatrix} \right) \tag{15.8}$$

其中 $m(\mathrm{x})$ 是一个 *mean function*，且 $k(\mathrm{x}、\mathrm{x}')$ 是 *covariance function*，或者 *kernel*。均值函数可以表示关于函数的先验知识。核函数控制函数的平滑度。使用均值函数和协方差函数构建均值向量和协方差矩阵的方法在算法15.1中给出。

4 均值函数产生期望：

$$m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$$

并且协方差函数产生协方差：

$$k(\mathbf{x}, \mathbf{x}') = \mathbb{E}\left[(f(\mathbf{x}) - m(\mathbf{x}))(f(\mathbf{x}') - m(\mathbf{x}'))\right]$$

```
μ(X, m) = [m(x) for x in X]
Σ(X, k) = [k(x,x′) for x in X, x′ in X]
K(X, X′, k) = [k(x,x′) for x in X, x′ in X′]
```

算法15.1。函数μ用于根据设计点列表和均值函数m构建均值向量，函数Σ用于根据一个或两个设计点列表和协方差函数k构建协方差矩阵。

一个常见的核函数是*squared exponential kernel*，其中

$$k(x, x') = \exp\left(-\frac{(x - x')^2}{2\ell^2}\right) \tag{15.9}$$

参数 ℓ 对应于所谓的 *characteristic length-scale*，可以将其视为在设计空间中必须行进的距离，直到目标函数值发生显著变化。[5] 因此，ℓ 的较大值导致在更平滑的函数中。图15.2展示了从零均值函数和具有不同特征长度的平方指数核的高斯过程中采样的函数。

5 特征长度尺度的数学定义由 C. E. Rasmussen 和 C. K. I. Williams 提供，*Gaussian Processes for Machine Learning*，MIT Press，2006.

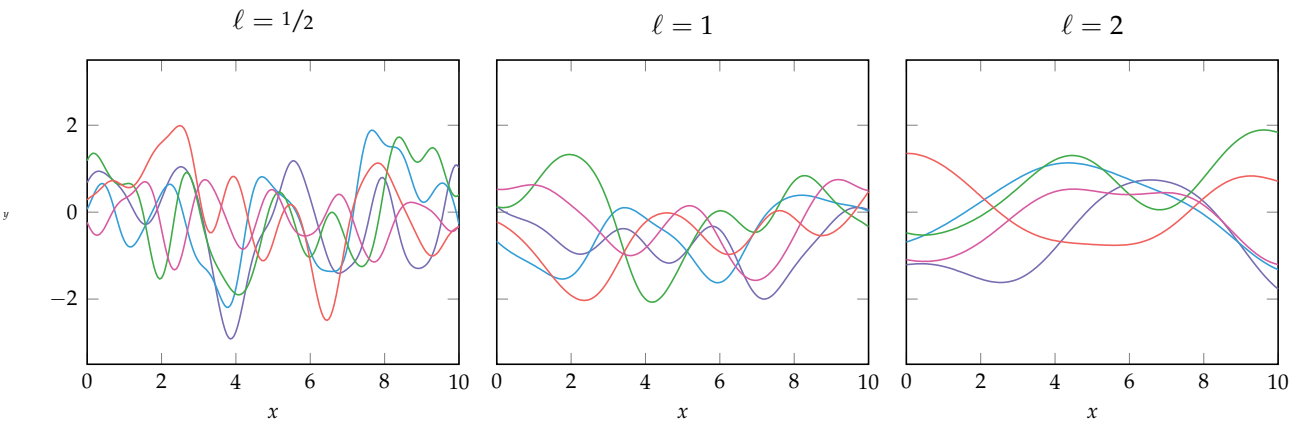


图15.2 从具有平方指数核的高斯过程中采样的函数。

除了平方指数核之外，还有许多其他核函数。图15.3展示了其中几种。许多核函数使用r，即x与x′之间的距离。通常使用欧几里得距离。*Matérn kernel* 使用*gamma function* Γ，由`SpecialFunctions.jl`包中的`gamma`实现，而$K_\nu(x)$是*modified Bessel function of the second kind*，由`besselk(ν,x)`实现。*neural network kernel*为每个设计向量添加一个1以便于表示：$\bar{\mathbf{x}} = [1, x_1, x_2, \ldots]$和$\bar{\mathbf{x}}' = [1, x'_1, x'_2, \ldots]$。

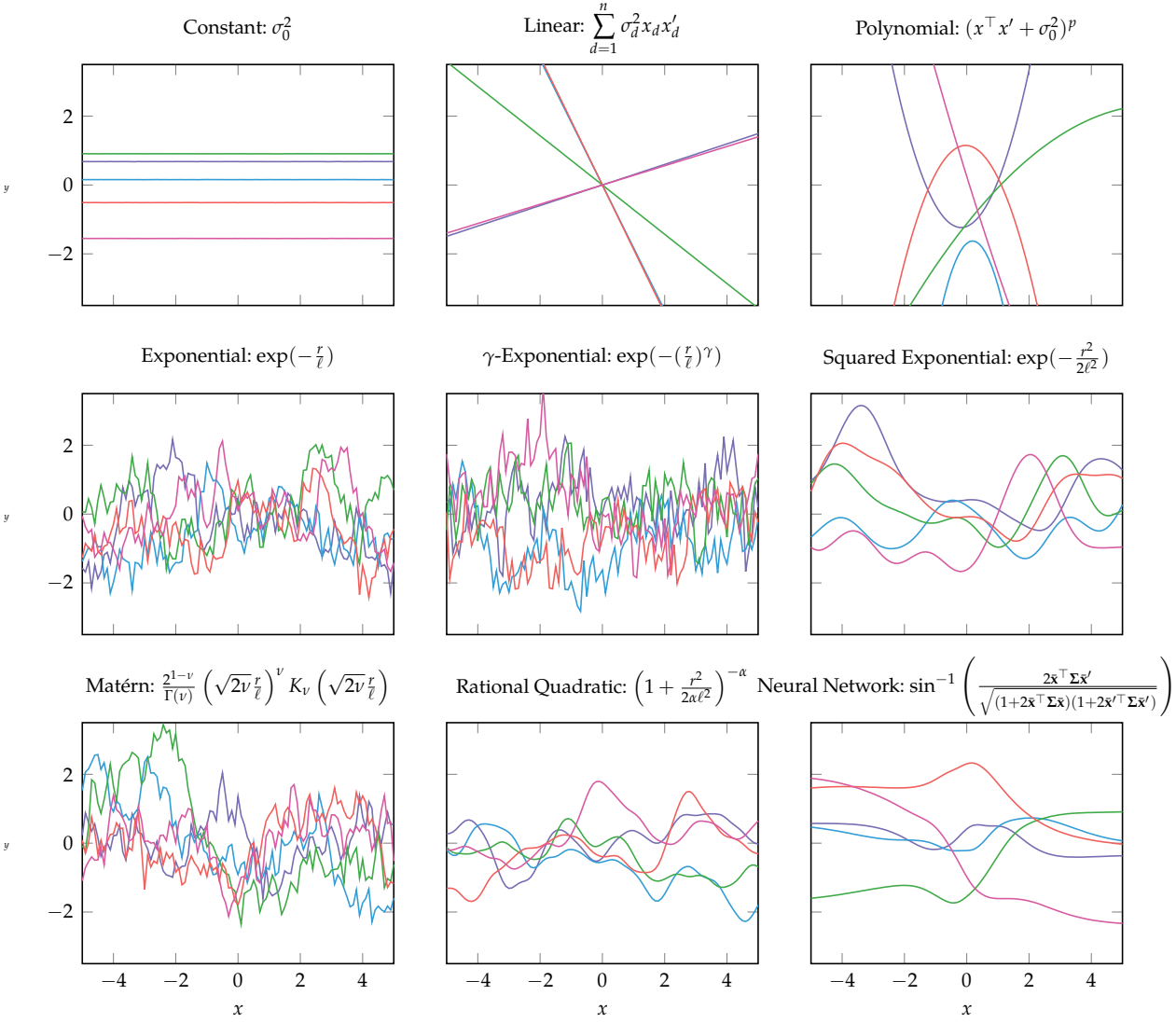


图15.3. 使用不同核函数从高斯过程中采样的函数。所示函数对应于 $\sigma_0^2 = \sigma_d^2 = \ell = 1$，$p = 2$，$\gamma = \nu = \alpha = 0.5$，及$\boldsymbol{\Sigma} = \mathrm{I}$。

本章将重点介绍具有单维设计空间的高斯过程示例，以便于绘图。然而，高斯过程可以定义在多维设计空间上，如图15.4所示。

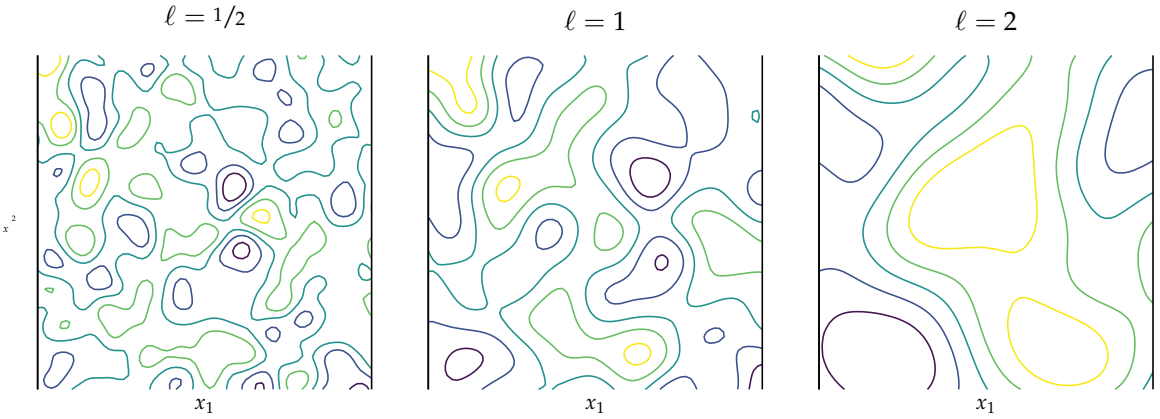


图15.4. 在二维设计空间上，从具有零均值和平方指数核的高斯过程中采样得到的函数。

正如我们将在第15.5节中看到的，高斯过程还可以纳入先验独立噪声方差，记为ν。因此，高斯过程是由均值函数和协方差函数、先验设计点及其函数评估以及噪声方差定义的。相关类型见算法15.2。

```
mutable struct GaussianProcess
    m # mean
    k # covariance function
    X # design points
    y # objective values
    ν # noise variance
end
```

算法15.2。高斯过程由均值函数`m`、协方差函数`k`、采样的设计向量`X`及其对应的值`y`以及噪声方差`ν`定义。

15.3 *Prediction*

高斯过程能够使用条件概率来表示函数上的分布。假设我们已有一组点 X 以及对应的 $\mathbf{y}$，但我们希望预测在点 X^* 处的值 $\hat{\mathbf{y}}$。联合分布

是

$$\begin{bmatrix}\hat{\mathbf{y}}\\ \mathbf{y}\end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix}\mathbf{m}(X^*)\\ \mathbf{m}(X\ )\end{bmatrix}, \begin{bmatrix}\mathbf{K}(X^*, X^*) & \mathbf{K}(X^*, X)\\ \mathbf{K}(X\ , X^*) & \mathbf{K}(X\ , X)\end{bmatrix}\right) \tag{15.10}$$

在方程 上文，我们使用了函数 m 和 K，它们被定义为 如下：

$$\mathbf{m}(X) = [m(\mathbf{x}^{(1)}), \ldots, m(\mathbf{x}^{(n)})] \tag{15.11}$$

$$\mathbf{K}(X, X') = \begin{bmatrix}k(\mathbf{x}^{(1)}, \mathbf{x}'^{(1)}) & \cdots & k(\mathbf{x}^{(1)}, \mathbf{x}'^{(m)})\\ \vdots & \ddots & \vdots\\ k(\mathbf{x}^{(n)}, \mathbf{x}'^{(1)}) & \cdots & k(\mathbf{x}^{(n)}, \mathbf{x}'^{(m)})\end{bmatrix} \tag{15.12}$$

6 在贝叶斯统计的语言中，后验分布是在给定观测值的条件下，可能的未观测值的分布。

条件分布由下式给出

$$\hat{\mathbf{y}} \mid \mathbf{y} \sim \mathcal{N}\left(\underbrace{\mathbf{m}(X^*) + \mathbf{K}(X^*, X)\mathbf{K}(X, X)^{-1}(\mathbf{y} - \mathbf{m}(X))}_{\text{mean}}, \underbrace{\mathbf{K}(X^*, X^*) - \mathbf{K}(X^*, X)\mathbf{K}(X, X)^{-1}\mathbf{K}(X, X^*)}_{\text{covariance}}\right) \tag{15.13}$$

注意，协方差并不依赖于y。这种分布通常被称为*posterior distribution*。[6]一种用于计算并从中采样的方法

由高斯过程定义的后验分布见算法 15.3。

```
function mvnrand(μ, Σ, inflation=1e-6)
    N = MvNormal(μ, Σ + inflation*I)
    return rand(N)
end
Base.rand(GP, X) = mvnrand(μ(X, GP.m), Σ(X, GP.k))
```

算法15.3。函数mvnrand从多元高斯分布中抽样，并加入膨胀因子以防止数值问题。方法rand在矩阵X中给定的设计点上抽样高斯过程GP。

预测均值可以写成关于 x 的函数：

$$\hat{\mu}(\mathbf{x}) = m(\mathbf{x}) + \mathbf{K}(\mathbf{x}, X)\mathbf{K}(X, X)^{-1}(\mathbf{y} - \mathbf{m}(X)) \tag{15.14}$$

$$= m(\mathbf{x}) + \boldsymbol{\theta}^\top \mathbf{K}(X, \mathbf{x}) \tag{15.15}$$

其中 $\theta = \mathrm{K}(X, X)^{-1}(\mathrm{y} - \mathrm{m}(X))$ 可以计算一次，并针对不同的 x 值重复使用。注意与上一章中代理模型的相似之处。高斯过程相较于之前讨论的代理模型的价值在于，它还能量化我们对预测的不确定性。

方差 预测均值的e也可以作为functi获得。 x 的 on:

$$\hat{v}(\mathbf{x}) = \mathbf{K}(\mathbf{x}, \mathbf{x}) - \mathbf{K}(\mathbf{x}, X)\mathbf{K}(X, X)^{-1}\mathbf{K}(X, \mathbf{x}) \tag{15.16}$$

在某些情况下，使用*standard deviation*（即方差的平方根）来表述方程更为方便：

$$\hat{\sigma}(\mathbf{x}) = \sqrt{\hat{v}(\mathbf{x})} \tag{15.17}$$

标准差与均值具有相同的单位。根据标准差，我们可以计算95%的*confidence region*，这是一个包含给定x下y分布相关概率质量95%的区间。对于特定的x，95%置信区域由$\hat{\mu}(\mathrm{x}) \pm 1.96\,\hat{\sigma}(\mathrm{x})$给出。人们可能想使用不同于95%的置信水平，但在本章的图表中我们将使用95%。图15.5展示了与四个函数评估拟合的高斯过程相关的置信区域图。

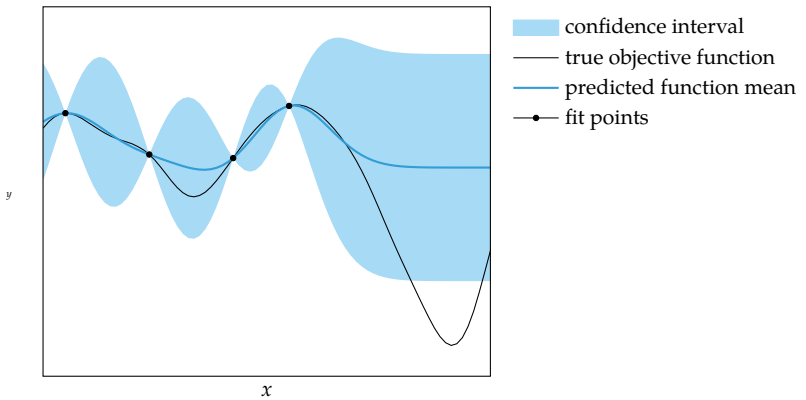


图15.5. 使用平方指数核的高斯过程及其95%置信区间。离数据点越远，不确定性越大。当我们远离数据点时，预期函数值趋近于零。

15.4 Gradient Measurements

梯度观测可以以与现有高斯过程机制一致的方式纳入高斯过程。[7] 高斯过程

扩展到同时包含函数值及其梯度：

$$\begin{bmatrix}\mathbf{y} \\ \nabla\mathbf{y}\end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix}\mathbf{m}_f \\ \mathbf{m}_\nabla\end{bmatrix}, \begin{bmatrix}\mathbf{K}_{ff} & \mathbf{K}_{f\nabla} \\ \mathbf{K}_{\nabla f} & \mathbf{K}_{\nabla\nabla}\end{bmatrix}\right) \tag{15.18}$$

tics

7 关于概述，参见例如 A. O'Hagan，“一些贝叶斯数值分析”，*Bayesian Statis-*，第4卷，J. M. Bernardo, J. O. Berger, A. P. Dawid, and A. F. M. Smith 编，第345–363页，1992年。

其中 $\mathrm{y} \sim \mathcal{N}\left(\mathrm{m}_f,\ \mathrm{K}_{ff}\right)$ 是一个传统的高斯过程，m_∇ 是梯度的均值函数，[8] $\mathrm{K}_{f\nabla}$ 是函数值之间的协方差矩阵，并且

梯度，$\mathrm{K}_{\nabla f}$是函数梯度与值之间的协方差矩阵，而$\mathrm{K}_{\nabla\nabla}$是函数梯度之间的协方差矩阵。

8 与函数值的均值一样，m_∇ 通常为零。

这些协方差矩阵是通过协方差函数构建的。高斯分布所具有的线性特性使得这些协方差函数相互关联：

$$k_{ff}(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}, \mathbf{x}') \tag{15.19}$$
$$k_{\nabla f}(\mathbf{x}, \mathbf{x}') = \nabla_{\mathbf{x}} k(\mathbf{x}, \mathbf{x}') \tag{15.20}$$
$$k_{f\nabla}(\mathbf{x}, \mathbf{x}') = \nabla_{\mathbf{x}'} k(\mathbf{x}, \mathbf{x}') \tag{15.21}$$
$$k_{\nabla\nabla}(\mathbf{x}, \mathbf{x}') = \nabla_{\mathbf{x}} \nabla_{\mathbf{x}'} k(\mathbf{x}, \mathbf{x}') \tag{15.22}$$

例15.2利用这些关系推导特定核的高阶协方差函数。

示例15.2. 针对具有梯度观测的高斯过程，推导协方差函数。

考虑平方指数协方差函数

$$k_{ff}(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{1}{2}\|\mathbf{x} - \mathbf{x}'\|^2\right)$$

我们可以使用方程（15.19）至（15.22）来获得使用带有梯度信息的高斯过程所需的其他协方差函数：

$$k_{\nabla f}(\mathbf{x}, \mathbf{x}')_i = -(\mathbf{x}_i - \mathbf{x}'_i) \exp\left(-\frac{1}{2}\|\mathbf{x} - \mathbf{x}'\|^2\right)$$
$$k_{\nabla\nabla}(\mathbf{x}, \mathbf{x}')_{ij} = -\Big((i = j) - (\mathbf{x}_i - \mathbf{x}'_i)(\mathbf{x}_j - \mathbf{x}'_j)\Big) \exp\left(-\frac{1}{2}\|\mathbf{x} - \mathbf{x}'\|^2\right)$$

提醒一下，布尔表达式，如 $(i = j)$，若为真则返回 1，若为假则返回 0。

预测可以像传统高斯过程那样完成。我们首先构造联合分布

$$\begin{bmatrix} \hat{\mathbf{y}} \\ \mathbf{y} \\ \nabla\mathbf{y} \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mathbf{m}_f(X^*) \\ \mathbf{m}_f(X\ ) \\ \mathbf{m}_\nabla(X\ ) \end{bmatrix}, \begin{bmatrix} \mathbf{K}_{ff}(X^*, X^*) & \mathbf{K}_{ff}(X^*, X\ ) & \mathbf{K}_{f\nabla}(X^*, X\ ) \\ \mathbf{K}_{ff}(X\ , X^*) & \mathbf{K}_{ff}(X\ , X\ ) & \mathbf{K}_{f\nabla}(X\ , X\ ) \\ \mathbf{K}_{\nabla f}(X\ , X^*) & \mathbf{K}_{\nabla f}(X\ , X\ ) & \mathbf{K}_{\nabla\nabla}(X\ , X\ ) \end{bmatrix}\right) \tag{15.23}$$

对于在高斯过程里，基于n维设计向量，给定m对函数与梯度评估及ℓ个查询点，协方差块的维度如下：

$$\begin{array}{ccc} \ell \times \ell & \ell \times m & \ell \times nm \\ m \times \ell & m \times m & m \times nm \\ nm \times \ell & nm \times m & nm \times nm \end{array} \tag{15.24}$$

示例15.3构建了这样一个协方差矩阵。

例15.3. 为具有梯度观测的高斯过程构建协方差矩阵。

假设我们在两个位置 x(1) 和 x(2) 处评估了一个函数及其梯度，并且我们希望预测在 x̂ 处的函数值。我们可以使用高斯过程推断出关于 ŷ、y 和 ∇y 的联合分布。协方差矩阵为：

$$\begin{bmatrix} k_{ff}(\hat{\mathbf{x}},\hat{\mathbf{x}}) & k_{ff}(\hat{\mathbf{x}},\mathbf{x}^{(1)}) & k_{ff}(\hat{\mathbf{x}},\mathbf{x}^{(2)}) & k_{f\nabla}(\hat{\mathbf{x}},\mathbf{x}^{(1)})_1 & k_{f\nabla}(\hat{\mathbf{x}},\mathbf{x}^{(1)})_2 & k_{f\nabla}(\hat{\mathbf{x}},\mathbf{x}^{(2)})_1 & k_{f\nabla}(\hat{\mathbf{x}},\mathbf{x}^{(2)})_2 \\ k_{ff}(\mathbf{x}^{(1)},\hat{\mathbf{x}}) & k_{ff}(\mathbf{x}^{(1)},\mathbf{x}^{(1)}) & k_{ff}(\mathbf{x}^{(1)},\mathbf{x}^{(2)}) & k_{f\nabla}(\mathbf{x}^{(1)},\mathbf{x}^{(1)})_1 & k_{f\nabla}(\mathbf{x}^{(1)},\mathbf{x}^{(1)})_2 & k_{f\nabla}(\mathbf{x}^{(1)},\mathbf{x}^{(2)})_1 & k_{f\nabla}(\mathbf{x}^{(1)},\mathbf{x}^{(2)})_2 \\ k_{ff}(\mathbf{x}^{(2)},\hat{\mathbf{x}}) & k_{ff}(\mathbf{x}^{(2)},\mathbf{x}^{(1)}) & k_{ff}(\mathbf{x}^{(2)},\mathbf{x}^{(2)}) & k_{f\nabla}(\mathbf{x}^{(2)},\mathbf{x}^{(1)})_1 & k_{f\nabla}(\mathbf{x}^{(2)},\mathbf{x}^{(1)})_2 & k_{f\nabla}(\mathbf{x}^{(2)},\mathbf{x}^{(2)})_1 & k_{f\nabla}(\mathbf{x}^{(2)},\mathbf{x}^{(2)})_2 \\ k_{\nabla f}(\mathbf{x}^{(1)},\hat{\mathbf{x}})_1 & k_{\nabla f}(\mathbf{x}^{(1)},\mathbf{x}^{(1)})_1 & k_{\nabla f}(\mathbf{x}^{(1)},\mathbf{x}^{(2)})_1 & k_{\nabla\nabla}(\mathbf{x}^{(1)},\mathbf{x}^{(1)})_{11} & k_{\nabla\nabla}(\mathbf{x}^{(1)},\mathbf{x}^{(1)})_{12} & k_{\nabla\nabla}(\mathbf{x}^{(1)},\mathbf{x}^{(2)})_{11} & k_{\nabla\nabla}(\mathbf{x}^{(1)},\mathbf{x}^{(2)})_{12} \\ k_{\nabla f}(\mathbf{x}^{(1)},\hat{\mathbf{x}})_2 & k_{\nabla f}(\mathbf{x}^{(1)},\mathbf{x}^{(1)})_2 & k_{\nabla f}(\mathbf{x}^{(1)},\mathbf{x}^{(2)})_2 & k_{\nabla\nabla}(\mathbf{x}^{(1)},\mathbf{x}^{(1)})_{21} & k_{\nabla\nabla}(\mathbf{x}^{(1)},\mathbf{x}^{(1)})_{22} & k_{\nabla\nabla}(\mathbf{x}^{(1)},\mathbf{x}^{(2)})_{12} & k_{\nabla\nabla}(\mathbf{x}^{(1)},\mathbf{x}^{(2)})_{22} \\ k_{\nabla f}(\mathbf{x}^{(2)},\hat{\mathbf{x}})_1 & k_{\nabla f}(\mathbf{x}^{(2)},\mathbf{x}^{(1)})_1 & k_{\nabla f}(\mathbf{x}^{(2)},\mathbf{x}^{(2)})_1 & k_{\nabla\nabla}(\mathbf{x}^{(2)},\mathbf{x}^{(1)})_{11} & k_{\nabla\nabla}(\mathbf{x}^{(2)},\mathbf{x}^{(1)})_{12} & k_{\nabla\nabla}(\mathbf{x}^{(2)},\mathbf{x}^{(2)})_{11} & k_{\nabla\nabla}(\mathbf{x}^{(2)},\mathbf{x}^{(2)})_{12} \\ k_{\nabla f}(\mathbf{x}^{(2)},\hat{\mathbf{x}})_2 & k_{\nabla f}(\mathbf{x}^{(2)},\mathbf{x}^{(1)})_2 & k_{\nabla f}(\mathbf{x}^{(2)},\mathbf{x}^{(2)})_2 & k_{\nabla\nabla}(\mathbf{x}^{(2)},\mathbf{x}^{(1)})_{21} & k_{\nabla\nabla}(\mathbf{x}^{(2)},\mathbf{x}^{(1)})_{22} & k_{\nabla\nabla}(\mathbf{x}^{(2)},\mathbf{x}^{(2)})_{21} & k_{\nabla\nabla}(\mathbf{x}^{(2)},\mathbf{x}^{(2)})_{22} \end{bmatrix}$$

条件分布遵循与方程（15.13）中相同的高斯关系：

$$\hat{\mathbf{y}} \mid \mathbf{y}, \nabla\mathbf{y} \sim \mathcal{N}(\boldsymbol{\mu}_\nabla, \boldsymbol{\Sigma}_\nabla) \tag{15.25}$$

其中：

$$\boldsymbol{\mu}_\nabla = \mathbf{m}_f(X^*) + \begin{bmatrix} \mathbf{K}_{ff}(X\ , X^*) \\ \mathbf{K}_{\nabla f}(X\ , X^*) \end{bmatrix}^\top \begin{bmatrix} \mathbf{K}_{ff}(X\ , X\ ) & \mathbf{K}_{f\nabla}(X\ , X\ ) \\ \mathbf{K}_{\nabla f}(X\ , X\ ) & \mathbf{K}_{\nabla\nabla}(X\ , X\ ) \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{y} - \mathbf{m}_f(X) \\ \nabla\mathbf{y} - \mathbf{m}_\nabla(X) \end{bmatrix} \tag{15.26}$$

$$\boldsymbol{\Sigma}_\nabla = \mathbf{K}_{ff}(X^*, X^*) - \begin{bmatrix} \mathbf{K}_{ff}(X\ , X^*) \\ \mathbf{K}_{\nabla f}(X\ , X^*) \end{bmatrix}^\top \begin{bmatrix} \mathbf{K}_{ff}(X\ , X\ ) & \mathbf{K}_{f\nabla}(X\ , X\ ) \\ \mathbf{K}_{\nabla f}(X\ , X\ ) & \mathbf{K}_{\nabla\nabla}(X\ , X\ ) \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{K}_{ff}(X\ , X^*) \\ \mathbf{K}_{\nabla f}(X\ , X^*) \end{bmatrix} \tag{15.27}$$

包含梯度观测时获得的区域进行比较 d 到
图15.6中那些没有梯度观测值的。

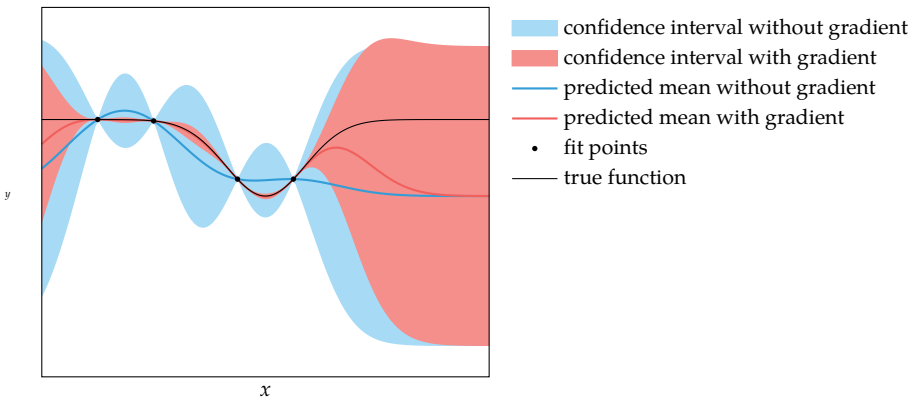


图15.6. 使用平方指数核的高斯过程，包含与不包含梯度信息。纳入梯度信息可以显著减小置信区间。

15.5 *Noisy Measurements*

到目前为止，我们假设目标函数 f 是确定性的。然而，在实际应用中，f 的评估可能包含测量噪声、实验误差或数值舍入。

我们可以将噪声评估建模为 $y = f(\mathbf{x}) + z$，其中 f 是确定性的，但 z 是零均值高斯噪声，$z \sim \mathcal{N}(0, \nu)$。噪声 ν 的方差可以调整以控制不确定性。[9]

第14.5节中涵盖的技术可用于调整噪声的方差。

新的联合分布为：

$$\begin{bmatrix}\hat{\mathbf{y}}\\ \mathbf{y}\end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix}\mathbf{m}(X^*)\\ \mathbf{m}(X\ )\end{bmatrix}, \begin{bmatrix}\mathbf{K}(X^*, X^*) & \mathbf{K}(X^*, X)\\ \mathbf{K}(X\ , X^*) & \mathbf{K}(X\ , X) + \nu\mathbf{I}\end{bmatrix}\right) \tag{15.28}$$

具有条件分布：

$$\hat{\mathbf{y}} \mid \mathbf{y}, \nu \sim \mathcal{N}(\boldsymbol{\mu}^*, \boldsymbol{\Sigma}^*) \tag{15.29}$$

$$\boldsymbol{\mu}^* = \mathbf{m}(X^*) + \mathbf{K}(X^*, X)(\mathbf{K}(X, X) + \nu\mathbf{I})^{-1}(\mathbf{y} - \mathbf{m}(X)) \tag{15.30}$$

$$\boldsymbol{\Sigma}^* = \mathbf{K}(X^*, X^*) - \mathbf{K}(X^*, X)(\mathbf{K}(X, X) + \nu\mathbf{I})^{-1}\mathbf{K}(X, X^*) \tag{15.31}$$

如上述方程所示，考虑高斯噪声是直接了当的，且后验分布可以解析计算。图15.7展示了一个含噪声的高斯过程。算法15.4实现了带有噪声测量的高斯过程预测。

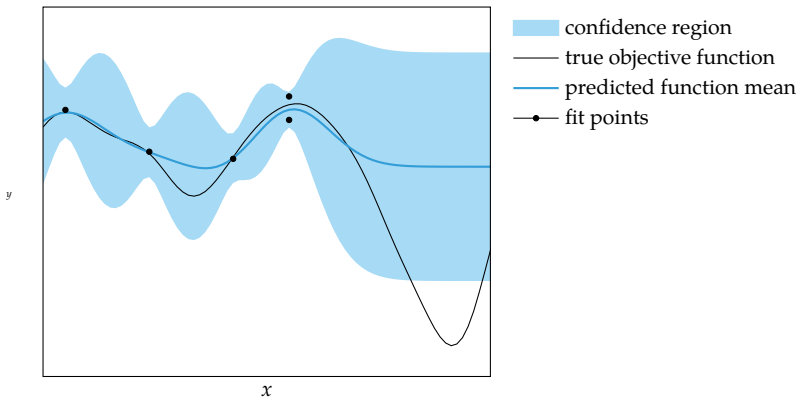


图15.7. 使用平方指数核的噪声高斯过程。

```
function predict(GP, X_pred)
    m, k, ν = GP.m, GP.k, GP.ν
    tmp = K(X_pred, GP.X, k) / (K(GP.X, GP.X, k) + ν*I)
    μₚ = μ(X_pred, m) + tmp*(GP.y - μ(GP.X, m))
    S = K(X_pred, X_pred, k) - tmp*K(GP.X, X_pred, k)
    νₚ = diag(S) .+ eps() # eps prevents numerical issues
    return (μₚ, νₚ)
end
```

算法15.4. 在高斯过程中获取f预测均值和标准差的方法。该方法接收一个高斯过程`GP`和一系列用于评估预测的点`X_pred`。它返回每个评估点处的均值和方差。

15.6 Fitting Gaussian Processes

核函数及参数的选择对评估设计点之间高斯过程的形式有很大影响。核函数及其参数可通过前一章介绍的交叉验证来选择。我们不是最小化测试数据上的平方误差，而是最大化数据的似然。[10]也就是说，我们寻找参数θ，使以下概率最大化：

函数值，$p(\mathrm{y} \mid X, \theta)$。数据的似然是观测点从模型中抽取的概率。等价地，我们可以最大化*log likelihood*，这通常更可取，因为在似然计算中乘以小概率会产生极小的值。给定一个包含n个条目的数据集$\mathcal{D}$，对数似然由下式给出

10 另外，我们也可以按照C. E. Rasmussen和C. K. I. Williams在2006年MIT出版社出版的*Gaussian Processes for Machine Learning*中所述，最大化*边际*似然。

$$\log p(\mathbf{y} \mid X, \nu, \theta) = -\frac{n}{2}\log 2\pi - \frac{1}{2}\log|\mathbf{K}_\theta(X,X) + \nu\mathbf{I}| - \frac{1}{2}(\mathbf{y} - \mathbf{m}_\theta(X))^\top(\mathbf{K}_\theta(X,X) + \nu\mathbf{I})^{-1}(\mathbf{y} - \mathbf{m}_\theta(X)) \tag{15.32}$$

其中均值和协方差函数由θ参数化。

让我们假设均值为零，使得 $\mathrm{m}_\theta(X) = 0$，且 θ 仅指高斯过程协方差函数的参数。我们可以通过梯度上升达到 *maximum likelihood estimate*。梯度则由下式给出：

$$\frac{\partial}{\partial\theta_j}\log p(\mathbf{y} \mid X, \theta) = \frac{1}{2}\mathbf{y}^\top\mathbf{K}^{-1}\frac{\partial\mathbf{K}}{\partial\theta_j}\mathbf{K}^{-1}\mathbf{y} - \frac{1}{2}\mathrm{tr}\left(\mathbf{\Sigma}_\theta^{-1}\frac{\partial\mathbf{K}}{\partial\theta_j}\right) \tag{15.33}$$

其中 $\mathbf{\Sigma}_\theta = \mathrm{K}_\theta(X、X) + \nu\mathrm{I}$。上文我们使用了矩阵导数关系

$$\frac{\partial\mathbf{K}^{-1}}{\partial\theta_j} = -\mathbf{K}^{-1}\frac{\partial\mathbf{K}}{\partial\theta_j}\mathbf{K}^{-1} \tag{15.34}$$

$$\frac{\partial\log|\mathbf{K}|}{\partial\theta_j} = \mathrm{tr}\left(\mathbf{K}^{-1}\frac{\partial\mathbf{K}}{\partial\theta_j}\right) \tag{15.35}$$

其中 tr(A) 表示矩阵 A 的*trace*，定义为主对角线元素之和。

15.7 Summary

- 高斯过程是定义在函数上的概率分布。
- 从高斯过程中采样得到的函数的平滑度受核的选择影响。
- 多元正态分布具有解析的条件分布和边缘分布。
- 我们可以计算在给定一组过去评估的情况下，目标函数在特定设计点的预测均值和标准差。
- 我们可以结合梯度观测来改进对目标值及其梯度的预测。
- 我们可以把测量噪声纳入高斯过程。
- 我们可以使用最大似然来拟合高斯过程的参数。

15.8 Exercises

练习 15.1. 在优化过程中，随着样本数量的增加，高斯过程的复杂度也会增加。这与基于回归的模型相比，有何优势?

练习 15.2. 高斯过程预测的计算复杂度如何随着数据点数量 m 的增加而增加?

练习 15.3。考虑函数 $f(x) = \sin(x)/\left(x^2 + 1\right)$ 在区间 $[-5, 5]$ 上。绘制一个带有导数信息的高斯过程的 95% 置信边界，该过程拟合于点 $\{-5, -2.5, 0, 2.5, 5\}$ 处的评估值。在 $[-5, 5]$ 内预测分布中的最大标准差是多少？为了使得一个不带导数信息的高斯过程达到相同的最大预测标准差，需要在定义域上均匀分布地进行多少次函数评估?

假设零均值函数和无噪声观测，并使用以下协方差函数：

$$k_{ff}(x, x') = \exp\left(-\frac{1}{2}\|x - x'\|_2^2\right)$$
$$k_{\nabla f}(x, x') = (x' - x)\exp\left(-\frac{1}{2}\|x - x'\|_2^2\right)$$
$$k_{\nabla\nabla}(x, x') = ((x - x')^2 - 1)\exp(-\frac{1}{2}\|x - x'\|_2^2)$$

练习15.4。推导关系式 $k_{f\nabla}(\mathrm{x}, \mathrm{x}')_i = \mathrm{cov}\left(f(\mathrm{x}), \frac{\partial}{\partial x_i'} f(\mathrm{x}')\right) = \frac{\partial}{\partial x_i'} k_{ff}(\mathrm{x}, \mathrm{x}')$。

练习15.5。假设我们有一个关于两个变量a和b的多变量高斯分布。证明在给定b的情况下，a的条件分布的方差不超过关于a的边缘分布的方差。这直观上说得通吗?

练习 15.6. 假设在采样过程中我们观察到了许多异常值，即观察到的样本并不落在高斯过程给出的置信区间内。这意味着我们选择的概率模型并不合适。我们能做些什么呢?

练习15.7。考虑函数评估对(x, y)的模型选择：

$$\{(1, 0), (2, -1), (3, -2), (4, 1), (5, 0)\}$$

使用留一交叉验证来选择在给定折叠中其他配对上的高斯过程下，最大化预测被留出配对可能性的核。假设均值为零且无噪声。从以下核中选择：

$$\exp(-\|x - x'\|) \qquad \exp(-\|x - x'\|^2) \qquad (1 + \|x - x'\|)^{-1} \qquad (1 + \|x - x'\|^2)^{-1} \qquad (1 + \|x - x'\|)^{-2}$$

16 Surrogate Optimization

上一章解释了如何使用概率代理模型，特别是高斯过程，来推断真实目标函数上的概率分布。这些分布可以用来指导优化过程朝着更好的设计点发展。[1] 本章概述了几种常见的

用于选择下一个要评估的设计点的技术。我们在这里讨论的技术会贪心地优化各种指标。[2] 我们还将讨论如何

代理模型可用于以安全的方式优化目标度量。

16.1 Prediction-Based Exploration

在*prediction-based exploration*中，我们选择代理函数的最小值点。这种方法的一个例子是我们之前在3.5节讨论的二次拟合搜索。在二次拟合搜索中，我们使用二次代理模型来拟合最后三个包围点，然后选择二次函数最小值处的点。

如果我们使用高斯过程代理模型，基于预测的优化会选择均值函数的最小值。

$$\mathbf{x}^{(m+1)} = \arg\min_{\mathbf{x} \in \mathcal{X}} \hat{\mu}(\mathbf{x}) \tag{16.1}$$

其中$\hat{\mu}(\mathrm{x})$是基于先前 m 个设计点，在设计点 x 处的高斯过程预测均值。该过程如图16.1所示。

基于预测的优化没有考虑不确定性，新样本可能生成在与现有样本非常接近的位置。在我们已经对目标值有信心的位置进行采样，是对函数评估的浪费。

1 A. Forrester, A. Sobester, and A. Keane, *Engineering Design via Surrogate Modelling: A Practical Guide* . Wiley, 2008.

2 贪心优化的一种替代方案是将问题构建为一个 *partially observable Markov decision problem*，并按照 M. Toussaint, ‘‘The Bayesian Search Game,’’ in *Theory and Principled Methods for the Design of Metaheuristics*, Y. Borenstein and A. Moraglio, eds. Springer, 2014, pp. 129–144 中所概述的那样提前规划若干步骤。另见 R. Lam, K. Willcox, and D. H. Wolpert, ‘‘Bayesian Opti- mization with a Finite Budget: An Approximate Dynamic Program- ming Approach,’’ in *Advances in Neural Information Processing Systems (NIPS)*, 2016。

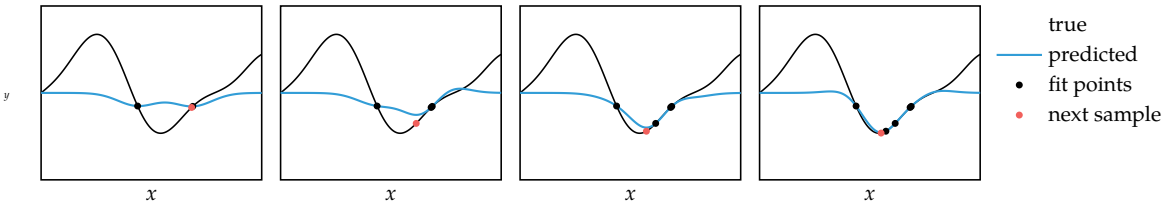


图16.1。基于预测的优化选择使目标函数均值最小的点。

16.2 *Error-Based Exploration*

Error-based exploration 旨在提高对真实函数的置信度。高斯过程可以在每个点上告诉我们均值和标准差。较大的标准差表明置信度较低，因此基于误差的探索在具有最大不确定性的设计点进行采样。

下一个样本点是：

$$x^{(m+1)} = \arg\max_{\mathbf{x}\in\mathcal{X}} \hat{\sigma}(\mathbf{x}) \tag{16.2}$$

其中$\hat{\sigma}(\mathrm{x})$是基于先前m个设计点的设计点x处高斯过程的标准差。该过程如图16.2所示。

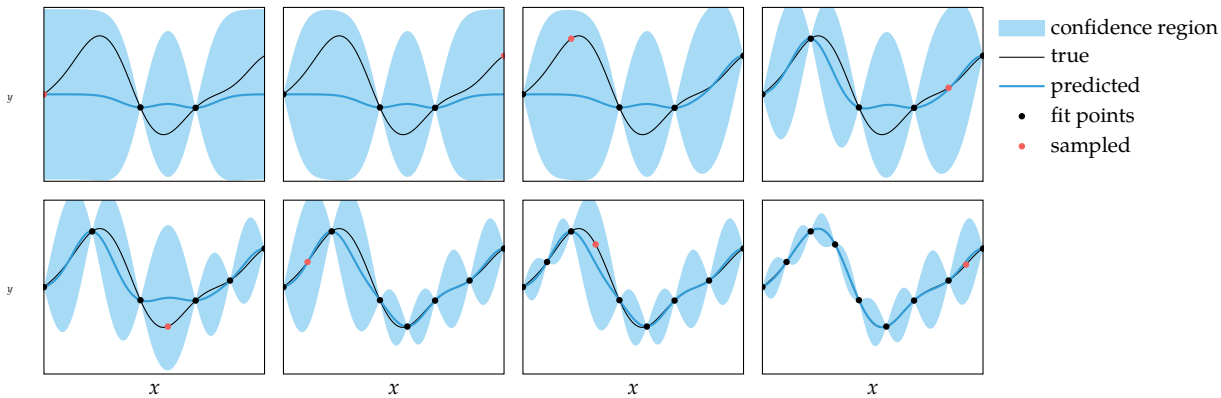


图16.2. 基于误差的探索选择具有最大不确定性的点。

高斯过程通常定义在整个 $\mathbb{R}^n$ 上。具有无界可行集的优化问题在远离采样点的地方总是具有高不确定性，这使得我们无法在整个域上对真实底层函数建立信心。因此，基于误差的探索必须限制在一个封闭区域内。

16.3 Lower Confidence Bound Exploration

基于误差的探索虽然总体上降低了目标函数的不确定性，但其样本往往位于不太可能包含全局最小值的区域。*Lower confidence bound exploration*在基于预测的优化所采用的贪婪最小化与基于误差的探索所采用的不确定性减少之间进行权衡。下一个样本最小化目标函数的*lower confidence bound*

$$LB(\mathbf{x}) = \hat{\mu}(\mathbf{x}) - \alpha\hat{\sigma}(\mathbf{x}) \tag{16.3}$$

其中 $\alpha \geq 0$ 是一个常数，用于控制 *exploration* 与 *ex- ploitation* 之间的权衡。探索涉及最小化不确定性，而利用涉及最小化预测均值。当 $\alpha = 0$ 时，我们进行基于预测的优化；当 α 接近 ∞ 时，我们进行基于误差的探索。该过程如图16.3所示。

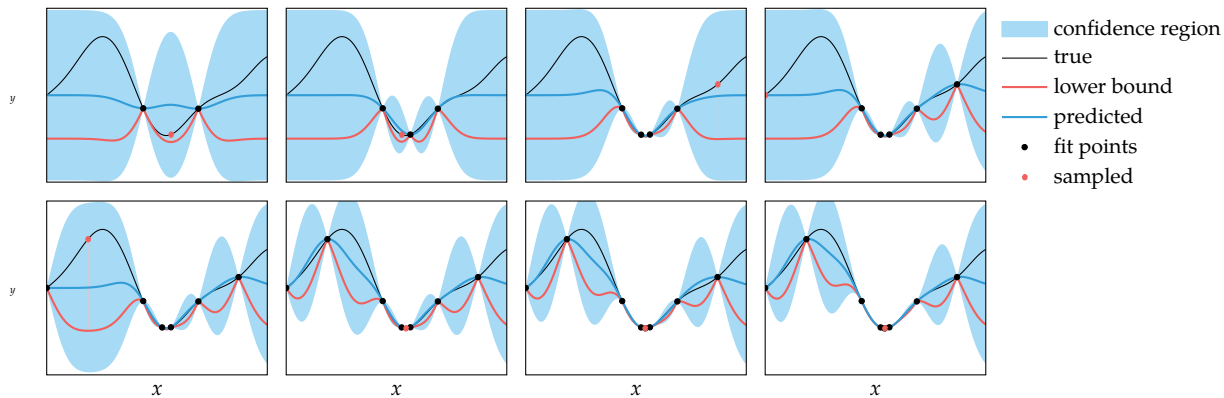


图16.3. 下置信界探索在最小化不确定性与最小化预测函数之间进行权衡。

16.4 Probability of Improvement Exploration

我们有时可以通过选择使新点比目前所见样本更好的概率最大化的设计点来获得更快的收敛速度。对于在x处采样产生$y = f(\mathrm{x})$的函数，其*improvement*为

$$I(y) = \begin{cases} y_{\min} - y & \text{if } y < y_{\min} \\ 0 & \text{otherwise} \end{cases} \tag{16.4}$$

其中y_{min}是目前为止采样的最小值。

probability of improvement 在$\hat{\sigma} > 0$的点处为

$$P(y < y_{\text{min}}) = \int_{-\infty}^{y_{\text{min}}} \mathcal{N}(y \mid \hat{\mu}, \hat{\sigma}) dy \tag{16.5}$$

$$= \Phi\left(\frac{y_{\text{min}} - \hat{\mu}}{\hat{\sigma}}\right) \tag{16.6}$$

其中 Φ 是 *standard normal cumulative distribution function* (参见附录 C.7)。此计算（算法 16.1）如图 16.4 所示。图 16.5 展示了这一过程。当$\hat{\sigma} =$ 为 0 时，即在我们有无噪声测量的点处，改进的概率为零。

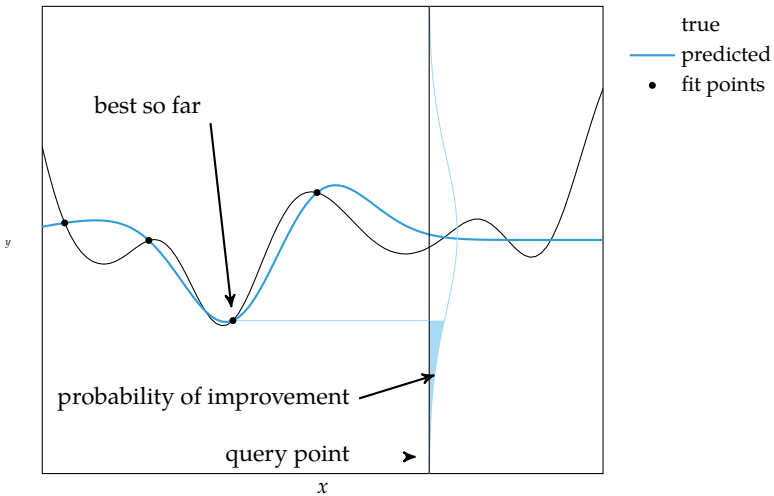


图16.4. 改进概率是指在评估某个特定点时，得到比目前最佳结果更好的结果的概率。此图展示了在查询点预测的概率密度函数，y_{min}下方的阴影区域对应改进的概率。

```
prob_of_improvement(y_min, μ, σ) = cdf(Normal(μ, σ), y_min)
```

算法16.1. 计算给定最佳y值`y_min`、均值μ和方差ν时改进的概率。

16.5 Expected Improvement Exploration

优化关注的是寻找目标函数的最小值。尽管最大化改进概率会随时间倾向于降低目标函数，但每次迭代的改进并不大。

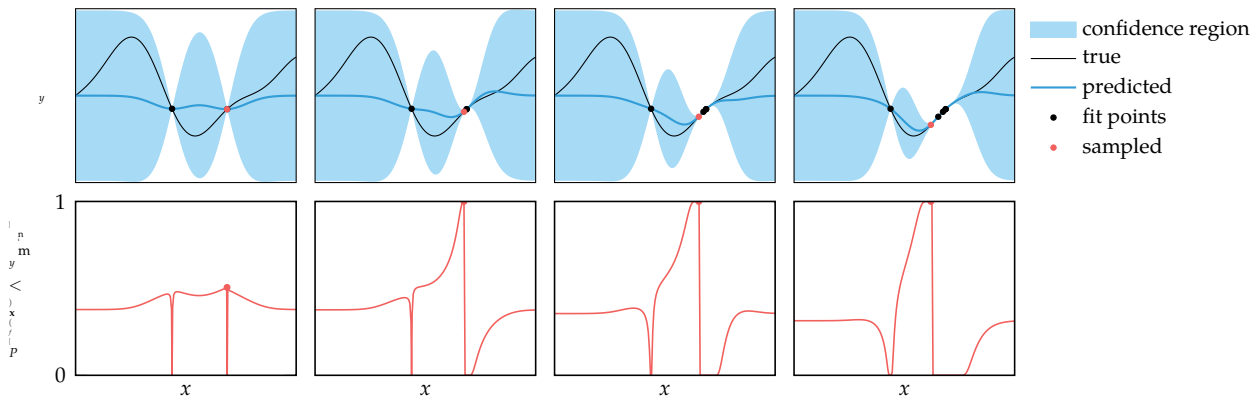


图16.5. 最大化改进概率选择最有可能产生更低目标点值的样本。

我们可以将探索重点放在使 *expected improvement* 在当前最佳函数值上最大化的点上。

通过替换

$$z = \frac{y - \hat{\mu}}{\hat{\sigma}} \qquad y'_{\text{min}} = \frac{y_{\text{min}} - \hat{\mu}}{\hat{\sigma}} \tag{16.7}$$

我们可以将方程 (16.4) 中的改进写为

$$I(y) = \begin{cases} \hat{\sigma}(y'_{\text{min}} - z) & \text{if } z < y'_{\text{min}} \text{ and } \hat{\sigma} > 0 \\ 0 & \text{otherwise} \end{cases} \tag{16.8}$$

其中 $\hat{\mu}$ 和 $\hat{\sigma}$ 是样本点 x 处的预测均值和标准差。

我们可以使用高斯过程预测的分布来计算预期改进：

$$\mathbb{E}[I(y)] = \hat{\sigma} \int_{-\infty}^{y'_{\text{min}}} (y'_{\text{min}} - z)\mathcal{N}(z \mid 0, 1)\, dz \tag{16.9}$$

$$= \hat{\sigma}\left[y'_{\text{min}} \int_{-\infty}^{y'_{\text{min}}} \mathcal{N}(z \mid 0, 1)\, dz - \int_{-\infty}^{y'_{\text{min}}} z\, \mathcal{N}(z \mid 0, 1)\, dz\right] \tag{16.10}$$

$$= \hat{\sigma}\left[y'_{\text{min}} P(z \leq y'_{\text{min}}) + \mathcal{N}(y'_{\text{min}} \mid 0, 1) - \underbrace{\mathcal{N}(-\infty \mid 0, 1)}_{=0}\right] \tag{16.11}$$

$$= (y_{\text{min}} - \hat{\mu})P(y \leq y_{\text{min}}) + \hat{\sigma}\mathcal{N}(y_{\text{min}} \mid \hat{\mu}, \hat{\sigma}^2) \tag{16.12}$$

图16.6使用算法16.2说明了这一过程。

```
function expected_improvement(y_min, μ, σ)
    p_imp = prob_of_improvement(y_min, μ, σ)
    p_ymin = pdf(Normal(μ, σ), y_min)
    return (y_min - μ)*p_imp + σ*p_ymin
end
```

Algorithm 16.2. Computing the expected improvement for a given best y value `y_min`, mean `μ`, and standard deviation `σ`.

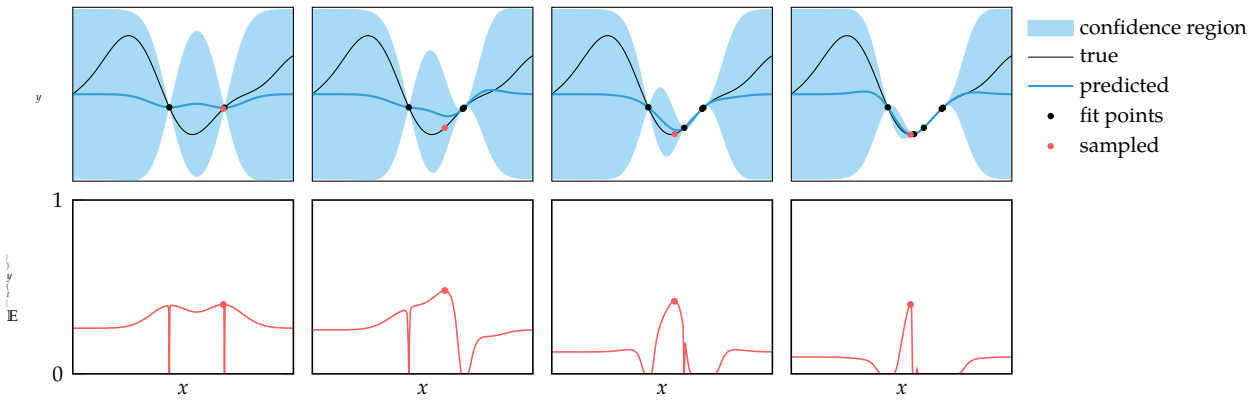


图16.6。最大化预期改进选择那些可能尽可能多地提高下界的样本。

16.6 Safe Optimization

在某些情况下，评估被认为不安全的点可能代价高昂，这些点可能对应于性能低下或不可行的点。诸如无人机控制器的飞行中调优或安全电影推荐等问题需要{v*}——在寻找最优设计点的同时，谨慎避免采样到不安全的设计。

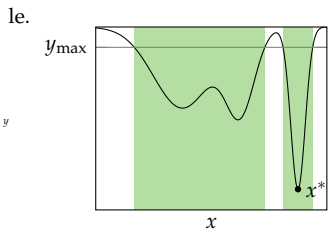


图16.7. SafeOpt 解决安全探索问题，这些问题在由最大目标函数值定义的安全区域内最小化 f。

本节概述了 *SafeOpt* 算法，[3]，该算法解决了一类安全探索问题。我们采样一系列设计点 x(1)、……、x(m)，以寻求最小值，但 f(x($^{i)}$) 不超过临界安全阈值 $y_{\max}$。此外，我们仅获得目标函数的噪声测量值，其中噪声为零均值，方差为 ν。这样的目标函数及其相关的安全区域如图16.7所示。

SafeOpt 算法使用高斯过程代理模型进行预测。在每次迭代中，我们根据来自 f 的噪声样本拟合一个高斯过程。在

3 Y. Sui, A. Gotovos, J. Burdick, and A. Krause, "用于高斯过程优化的安全探索"，载于 *International Conference on Machine Learning (ICML)*，第37卷，2015年。

i第th个样本，SafeOpt 计算置信上界和下界：

$$u_i(\mathbf{x}) = \hat{\mu}_{i-1}(\mathbf{x}) + \sqrt{\beta \hat{v}_{i-1}(\mathbf{x})} \tag{16.13}$$

$$\ell_i(\mathbf{x}) = \hat{\mu}_{i-1}(\mathbf{x}) - \sqrt{\beta \hat{v}_{i-1}(\mathbf{x})} \tag{16.14}$$

其中β的较大值会产生更宽的置信区域。此类边界如图16.8所示。

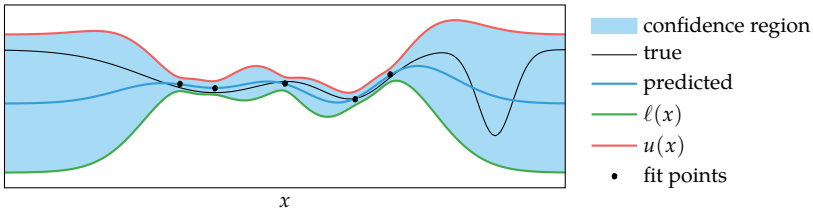


图16.8。基于高斯过程预测的函数的示意图，用于SafeOpt。

高斯过程为任何设计点预测了$f(\mathbf{x})$上的分布。由于是高斯分布，这些预测只能提供一个概率性的安全保证，最多可达到任意因子[4]：

$$P(f(\mathbf{x}) \leq y_{\max}) = \Phi\left(\frac{y_{\max} - \hat{\mu}(\mathbf{x})}{\sqrt{\hat{v}(\mathbf{x})}}\right) \geq P_{\text{safe}} \tag{16.15}$$

预测的安全区域 $\mathcal{S}$ 由提供高于所需水平 P_{safe} 的安全概率的设计点组成，如图16.9所示。安全区域也可以根据在先前采样点评估的上界构造的Lipschitz上界来定义。

4 注意与改进概率的相似性。

SafeOpt 选择一个安全样本点，以平衡定位 f 的可达最小化器和扩展安全区域的愿望。f 的潜在最小化器集合记为 $\mathcal{M}$ (图 16.10)，将有可能导致安全区域扩展的点集记为 $\mathcal{E}$ (图 16.11)。为了权衡探索和利用，我们在集合 $\mathcal{M}$ 和 $\mathcal{E}$ 中选择预测方差最大的设计点 x。[5]

潜在最小化器集合由置信下界低于最低上界的安全点组成：

$$\mathcal{M}_i = \left\{\mathbf{x} \in \mathcal{S}_i \mid \ell_i(\mathbf{x}) \leq \min_{\mathbf{x}' \in \mathcal{S}_i} u_i(\mathbf{x}')\right\} \tag{16.16}$$

5 关于该算法的一种变体，参见F. Berkenkamp、A. P. Schoellig和A. Krause，“带有高斯过程的四旋翼飞行器安全控制器优化”，载于*IEEE International Conference on Robotics and Automation (ICRA)*，2016年。

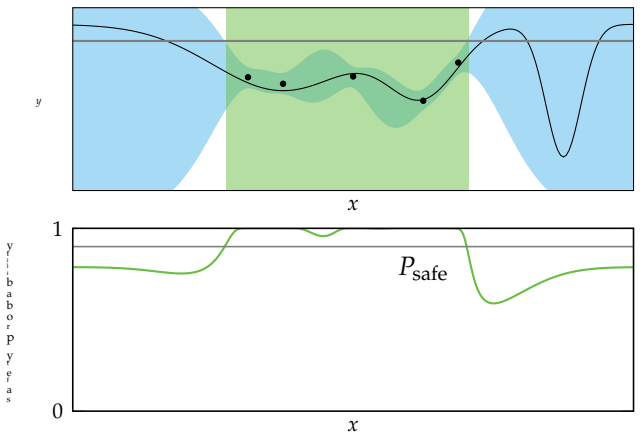


图16.9. 高斯过程预测的安全区域（绿色）。

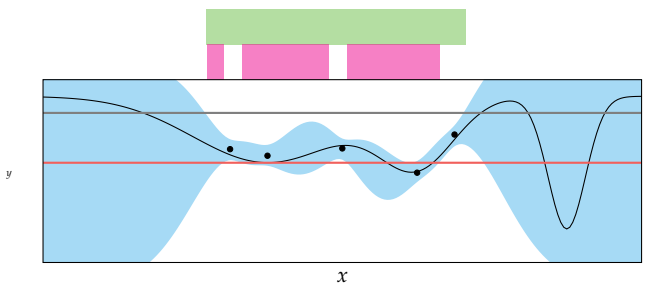


图16.10。潜在的极小值点是那些下界低于最佳安全上界的安全点。

在步骤i处，潜在扩展点集合$\mathcal{E}_i$包括那些如果加入到高斯过程中，在乐观假设下界的情况下，能够产生具有更大安全集的后验分布的安全点。这些潜在扩展点自然位于安全区域的边界附近。

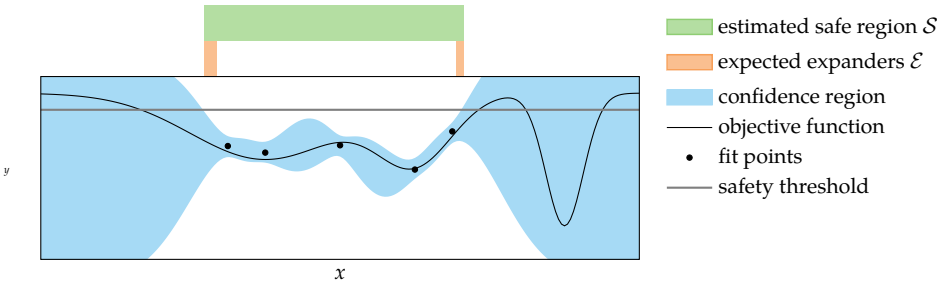


图16.11. 潜在扩展器的集合。

给定初始安全点[6] $\mathbf{x}(1)$，SafeOpt 在集合中选择设计点 $\mathcal{M}$ 以及$\mathcal{E}$，具有最大的不确定性，其量化宽度为$w_i(x) = u(x) - \ell(x)$：

$$x^{(i)} = \arg\max_{\mathbf{x} \in \mathcal{M}_i \cup \mathcal{E}_i} w_i(\mathbf{x}) \tag{16.17}$$

6 如果SafeOpt没有至少一个它知道是安全的点作为初始化，它就无法保证安全。

SafeOpt 会持续进行，直到满足终止条件。通常会将算法运行固定次数的迭代，或者直到最大宽度小于设定的阈值。

在多维空间中维护集合可能在计算上具有挑战性。SafeOpt假设一个有限设计空间$\mathcal{X}$，可以通过在连续搜索域上应用采样方法获得。增加有限设计空间的密度会使结果相对于连续空间更准确，但每次迭代耗时更长。

SafeOpt 在算法 16.3 中实现，并调用算法 16.4 来更新预测的置信区间；算法 16.5 用于计算安全区域、最小化区域和扩展区域；算法 16.6 用于选择查询点。SafeOpt 的进展在一维情况下如图 16.12 所示，在二维情况下如图 16.13 所示。

```
function safe_opt(GP, X, i, f, y_max; β=3.0, k_max=10)
    push!(GP, X[i], f(X[i])) # make first observation

    m = length(X)
    u, l = fill(Inf, m), fill(-Inf, m)
    S, M, E = falses(m), falses(m), falses(m)

    for k in 1 : k_max
        update_confidence_intervals!(GP, X, u, l, β)
        compute_sets!(GP, S, M, E, X, u, l, y_max, β)
        i = get_new_query_point(M, E, u, l)
        i != 0 || break
        push!(GP, X[i], f(X[i]))
    end

    # return the best point
    update_confidence_intervals!(GP, X, u, l, β)
    S[:] = u .≤ y_max
    if any(S)
        u_best, i_best = findmin(u[S])
        i_best = findfirst(isequal(i_best), cumsum(S))
        return (u_best, i_best)
    else
        return (NaN,0)
    end
end
```

算法16.3。将SafeOpt算法应用于一个空的GP高斯过程、一个有限设计空间X、初始安全点的索引i、目标函数f和安全阈值y_max。可选参数是置信标量β和迭代次数k_max。返回一个元组，包含最佳安全上界及其在X中的索引。

```
function update_confidence_intervals!(GP, X, u, l, β)
    μₚ, νₚ = predict(GP, X)
    u[:] = μₚ + sqrt.(β*νₚ)
    l[:] = μₚ - sqrt.(β*νₚ)
    return (u, l)
end
```

算法16.4. 更新SafeOpt中使用的下界和上界的方法，该方法采用高斯过程GP，有限搜索空间X，上下界向量u和l，以及置信度标量β。

```
function compute_sets!(GP, S, M, E, X, u, l, y_max, β)
    fill!(M, false)
    fill!(E, false)

    # safe set
    S[:] = u .≤ y_max

    if any(S)

        # potential minimizers
        M[S] = l[S] .< minimum(u[S])

        # maximum width (in M)
        w_max = maximum(u[M] - l[M])

        # expanders - skip values in M or those with w ≤ w_max
        E[:] = S .& .~M # skip points in M
        if any(E)
            E[E] .= maximum(u[E] - l[E]) .> w_max
            for (i,e) in enumerate(E)
                if e && u[i] - l[i] > w_max
                    push!(GP, X[i], l[i])
                    μₚ, νₚ = predict(GP, X[.~S])
                    pop!(GP)
                    E[i] = any(μₚ + sqrt.(β*νₚ) .≥ y_max)
                    if E[i]; w_max = u[i] - l[i]; end
                end
            end
        end
    end

    return (S,M,E)
end
```

算法16.5。一种用于更新SafeOpt中使用的安全集S、最小化集M和扩展集E的方法。这些集合都是布尔向量，指示X中相应的设计点是否在集合中。该方法还接受高斯过程GP，上界u和下界l，安全阈值y_max，以及置信标量β。

```
function get_new_query_point(M, E, u, l)
    ME = M .| E
    if any(ME)
           v = argmax(u[ME] - l[ME])
        return findfirst(isequal(v), cumsum(ME))
    else
        return 0
    end
end
```

算法16.6. SafeOpt中获取下一个查询点的方法。返回 X 中宽度最大的点的索引。

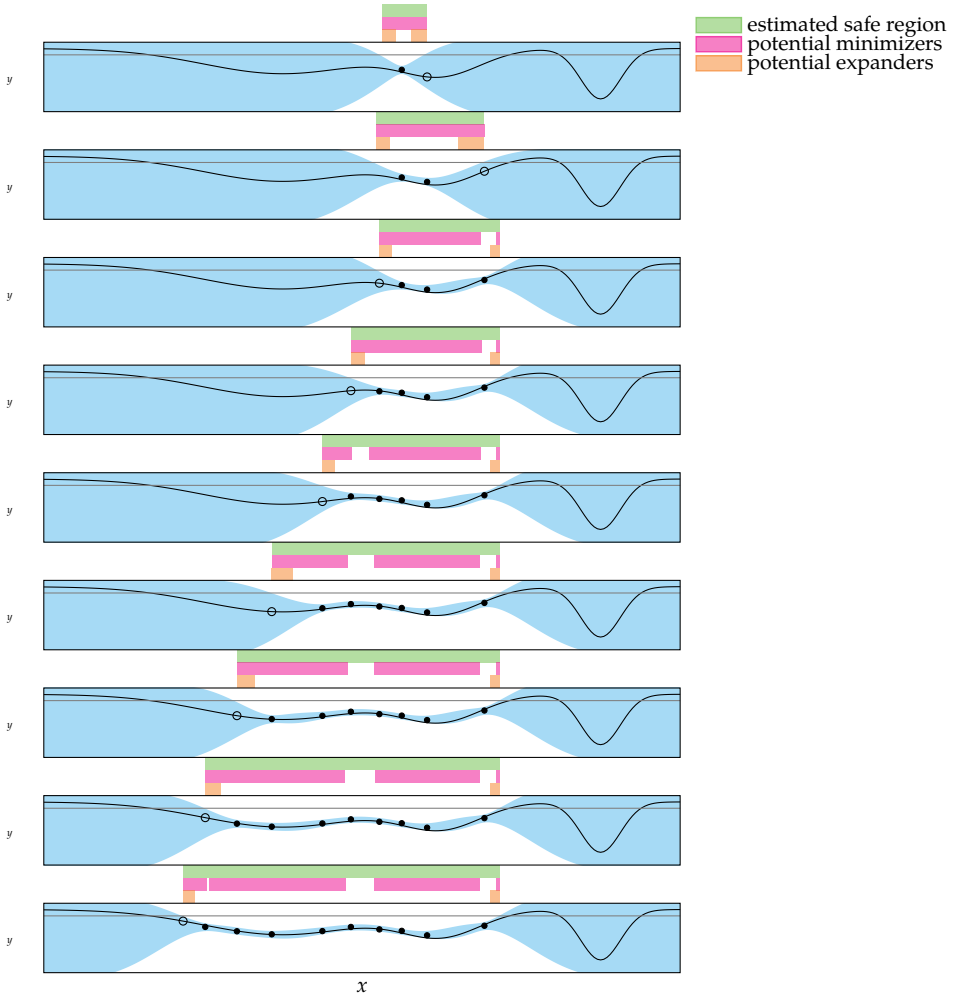


图16.12. SafeOpt在一元函数上的前八次迭代。由于需要穿越不安全区域，SafeOpt永远无法到达右侧的全局最优解。我们只能寄希望于在局部可达的安全区域内找到全局最小值。

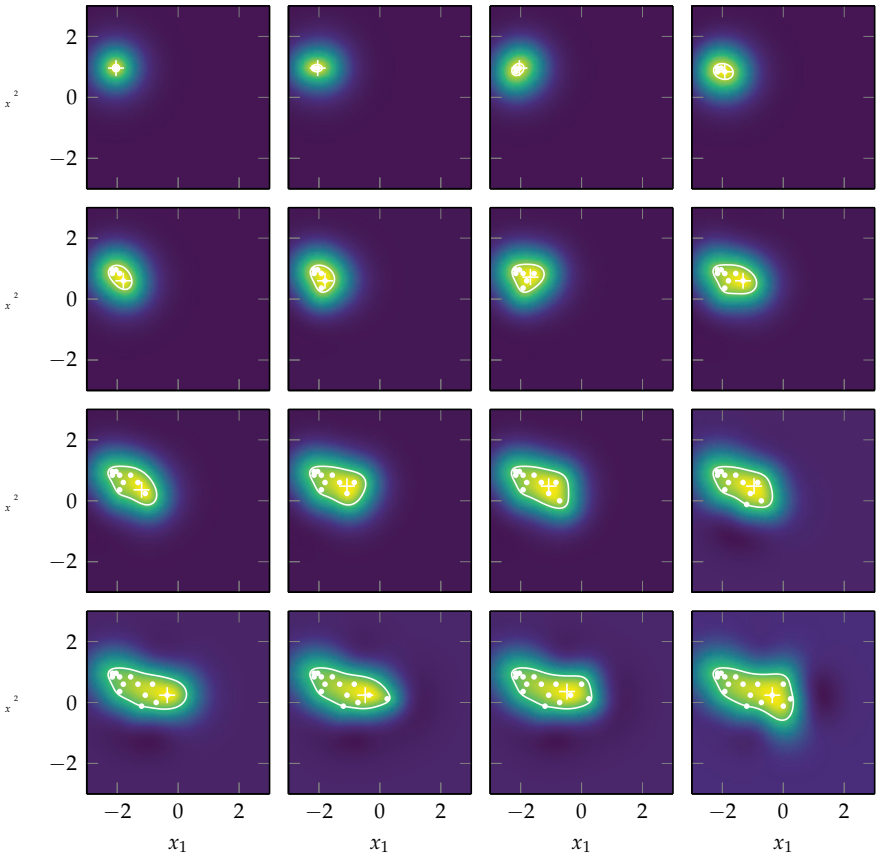
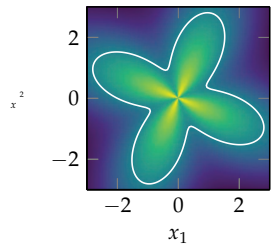


图 16.13. 将 SafeOpt 应用于 flower 函数（附录 B.4），其中 $y_{\max} = 2$，高斯过程均值为 $\mu(\mathrm{x}) = 2.5$，方差为 $\nu = 0.01$，$\beta = 10$，在搜索空间上使用 51×51 的均匀网格，初始点为 $x^{(1)} = [-2.04, 0.96]$。颜色表示上界值，十字表示具有最低上界的安全点，白色轮廓线为估计的安全区域。

目标函数用白色勾勒出真实的安全区域：



16.7 Summary

- 高斯过程可用于指导优化过程，采用多种策略，这些策略利用了诸如较低置信界、改进概率和预期改进等量值的估计。

- 有些问题不允许对不安全设计进行评估，此时我们可以采用依赖于高斯过程的安全探索策略。

16.8 Exercises

练习16.1。给出一个基于预测的优化失败的例子。

练习16.2. 在优化的背景下，低置信度边界探索与基于误差的探索之间的主要区别是什么?

练习16.3。我们有一个函数 $f(x) = (x-2)^2/40 - 0.5$，其中 $x \in [-5, 5]$，并且我们在 -1 和 1 处有评估点。假设我们使用具有零均值函数和平方指数核 $\exp(-r^2/2)$ 的高斯过程代理模型，其中 r 是两点之间的欧几里得距离。如果我们最大化改进概率，下一个要评估的 x 值是多少？如果我们最大化期望改进，下一个要评估的 x 值是多少?

17 Optimization under Uncertainty

前几章假设优化目标是最小化设计点的确定性函数。然而，在许多工程任务中，目标函数或约束可能存在不确定性。不确定性可能由多种因素引起，例如模型近似、不精确性以及参数随时间的波动。本章涵盖了多种在优化中考虑不确定性以增强鲁棒性的方法。[1]

17.1 Uncertainty

优化过程中的不确定性可能由多种原因引起。可能存在 *irreducible uncertainty*，[2]，这是系统固有的，例如反-
地面噪声、材料属性的变化以及量子效应。这些不确定性是无法避免的，我们的设计应当容纳它们。还可能存在*epistemic uncertainty*、[3]，这是由于主观缺乏

由设计者掌握的知识。这种不确定性可能源于构建设计问题时所用模型[4]中的近似，以及由
数值解法。

考虑这些不同形式的不确定性对于确保稳健设计至关重要。在本章中，我们将使用 $z \in \mathcal{Z}$ 来表示随机值向量。我们希望最小化 $f(x, z)$，但我们无法控制 z。可行性取决于设计向量 x 和不确定向量 z。本章将 x 和 z 对上的可行集引入为 $\mathcal{F}$。当且仅当 $(x, z) \in \mathcal{F}$ 时，我们具有可行性。我们将使用 $\mathcal{X}$ 作为设计空间，根据 z 的值，该空间可能包含潜在不可行的设计。

在第15.5节中，简要介绍了在利用高斯过程表示推断的目标函数背景下的不确定性优化。

1 其他参考文献包括：H.-G. Beyer 和 B. Sendhoff，“Robust OptimOverview–A Comprehensive Survey”，*Computer*，第 196 卷，第 33 期，第 3190–3218 页，2007 年。G.-J. Park、T.-H. Lee、K. H. Lee 和 K.-H. Hwang，“Robust Design: An Overview”，*AIAA Journal*，第 44 卷，第 1 期，第 181–191 页，2006 年。

2 这种不确定性形式有时被称为 *aleatory uncertainty*或 *random uncertainty*。

3 认知不确定性也称为 *reducible uncertainty*。

4 统计学家乔治·博克斯有句名言：*All models are wrong; some models are useful.* G. E. P. Box, W. G. Hunter, and J. S. Hunter, *Statistics for Experimenters: An Introduction to Design, Data Analysis, and Model Building*, 2nd ed. Wiley, 2005, p. 440.

Methods in Applied and Engineering

从含噪测量中。我们有$f(\mathrm{x}, z) = f(\mathrm{x}) + z$，附加假设是$z$来自零均值高斯分布。[5]不确定性可能以其他方式融入设计点的评估中。例如，如果目标函数的输入存在噪声，[6]我们可能会有$f(\mathrm{x}, \mathrm{z}) = f(\mathrm{x} + \mathrm{z})$。通常，$f(\mathrm{x}, \mathrm{z})$ 可以是 x 和 z 的复杂非线性函数。此外，z 可能并非来自高斯分布；实际上，它可能来自一个未知的分布。

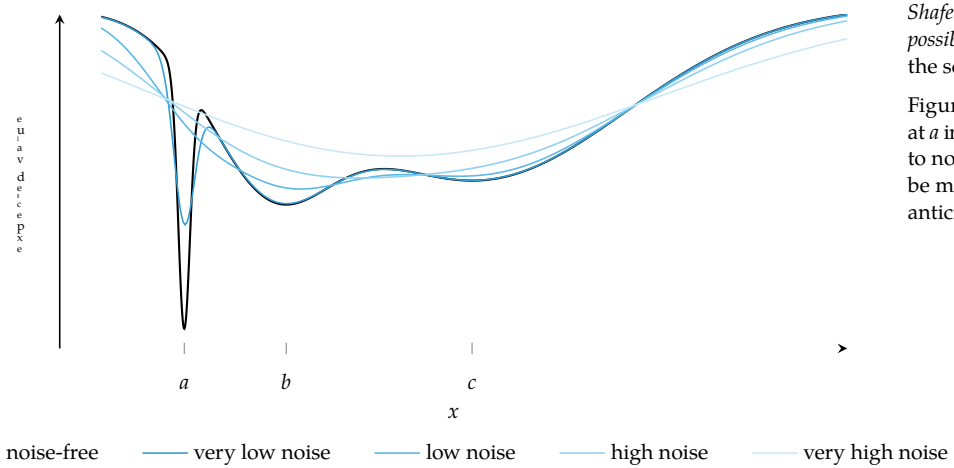
5 这里，f 的双参数版本以设计点和随机向量作为输入，而 f 的单参数版本表示设计点的确定性函数，不含噪声。

例如，在我们的设计制造过程中可能存在变异性。

图17.1展示了不确定性程度如何影响我们的设计选择。为简化起见，x是一个标量，而z选自零均值高斯分布。我们假设z对应于f输入中的噪声，因此$f(x, z) = f(x + z)$。该图显示了不同噪声水平下目标函数的期望值。无噪声时的全局最小值为a。然而，追求接近a的设计可能存在风险，因为它位于陡峭的谷底，对噪声相当敏感。即使噪声较低，选择接近b的设计也可能更好。接近c的设计对更大噪声量具有更强的鲁棒性。如果噪声非常高，最佳设计甚至可能落在b和c之间，这对应于无噪声时的局部最大值。

在优化中，有多种不同的方法来考虑不确定性。我们将讨论基于集合的不确定性和概率不确定性。[7]

7 其他表示不确定性的方法包括 *Dempster-Shafer theory*，*fuzzy-set theory*，和 *possibility theory*，这超出了the sc本书的ope。



Figur17.1. 全局最小值 at a in 无噪声情况较为敏感 to noi他。其他设计要点可能 be mo取决于 antici指定的噪声水平。

17.2 Set-Based Uncertainty

Set-based uncertainty 这些方法假设 z 属于集合 $\mathcal{Z}$，但这些方法并未对该集合内不同点的相对可能性做出假设。集合 $\mathcal{Z}$ 可以通过不同方式定义。一种方式是为 $\mathcal{Z}$ 的每个分量定义区间。另一种方式是通过一组不等式约束来定义 $\mathcal{Z}$，即 $g(x, z) \leq 0$，类似于第 10 章中对设计空间 $\mathcal{X}$ 的处理方式。

17.2.1 Minimax

在基于集合的不确定性问题中，我们常常希望最小化目标函数的最大可能值。这样的*minimax*方法[8]解决了优化问题

$$\underset{\mathbf{x}\in\mathcal{X}}{\text{minimize}}\ \underset{\mathbf{z}\in\mathcal{Z}}{\text{maximize}}\ f(\mathbf{x}, \mathbf{z}) \tag{17.1}$$

换句话说，我们想要找到一个x，最小化f，假设z的最坏情况值。

这种优化等价于定义了一个修改后的目标函数

$$f_{\text{mod}}(\mathbf{x}) = \underset{\mathbf{z}\in\mathcal{Z}}{\text{maximize}}\ f(\mathbf{x}, \mathbf{z}) \tag{17.2}$$

然后求解

$$\underset{\mathbf{x}\in\mathcal{X}}{\text{minimize}}\ f_{\text{mod}}(\mathbf{x}) \tag{17.3}$$

例17.1展示了这一优化在一个单变量问题上的应用，并说明了不同不确定性水平的影响。

在存在可行性约束的问题中，我们的优化问题变为

$$\underset{\mathbf{x}\in\mathcal{X}}{\text{minimize}}\ \underset{\mathbf{z}\in\mathcal{Z}}{\text{maximize}}\ f(\mathbf{x}, \mathbf{z}) \text{ subject to } (\mathbf{x}, \mathbf{z}) \in \mathcal{F} \tag{17.4}$$

示例 17.2 展示了在存在约束时，对可行设计点空间应用极小化极大方法的效果。

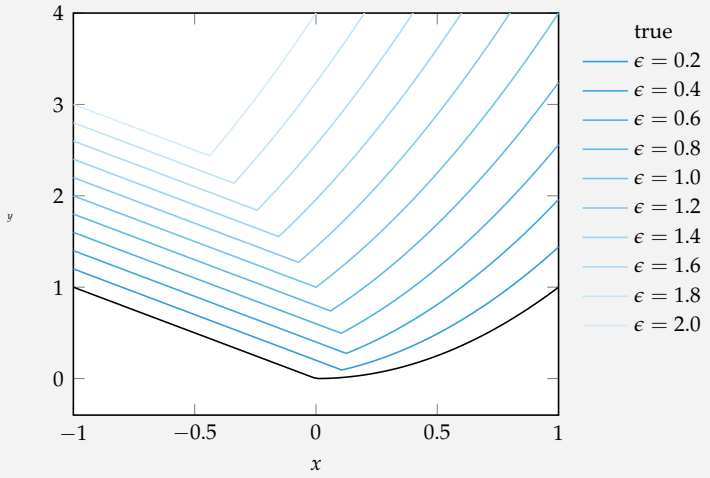
[8] 也称为 *robust counterpart* 或 *robust regularization approach*

例17.1. 基于集合不确定性下的极小极大优化方法示例。

考虑目标函数

$$f(x,z) = f(x+z) = f(\tilde{x}) = \begin{cases} -\tilde{x} & \text{if } \tilde{x} \leq 0 \\ \tilde{x}^2 & \text{otherwise} \end{cases}$$

其中$\tilde{x} = x + z$，具有基于集合的不确定性区域$z \in [-\epsilon, \epsilon]$。极小极大方法是一个关于修改后的目标函数$f_{\text{mod}}(x) =$ 最大化 $_{z\in[-\epsilon,\epsilon]} f(x, z)$ 的最小化问题。



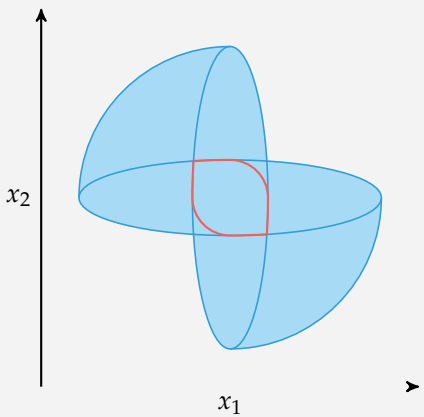
上图显示了$f_{\text{mod}}(x)$在ϵ的几个不同取值下的情况。$\epsilon = 0$的最小值与$f(x, 0)$的最小值重合。随着ϵ增大，最小值首先右移，因为x比x^2增长更快，然后左移，因为x^2比x增长更快。稳健最小化器通常不与$f(x, 0)$的最小值重合。

考虑一个形式为旋转椭圆的不确定可行集，其中 $(x, z) \in \mathcal{F}$ 当且仅当 $z \in [0, \pi/2]$ 且

$$(x_1 \cos z + x_2 \sin z)^2 + (x_1 \sin z - x_2 \cos z)^2/16 \leq 1$$

当 $z = 0$ 时，椭圆的长轴是垂直的。z 的增加会使其缓慢逆时针旋转，当达到 $z = \pi/2$ 时变为水平。下图展示了垂直和水平的椭圆，以及以蓝色表示的至少在某个 z 下可行的所有点集。

一种优化中的极小极大方法应仅考虑在z所有取值下都可行的设计点。始终可行的设计集合由通过变化z形成的所有椭圆的交集给出。该集合用红色勾勒。



示例17.2。极小极大方法应用于可行集中的不确定性。

17.2.2 *Information-Gap Decision Theory*

不同于假设不确定性集 $\mathcal{Z}$ 是固定的，另一种称为 *information-gap decision theory*[9] 的方法通过一个非负

tive标量*gap*参数ϵ。该间隙控制以某个标称值$\bar{z} = \mathcal{Z}(0)$为中心的参数化集合$\mathcal{Z}(\epsilon)$的大小。定义$\mathcal{Z}(\epsilon)$的一种方式是将其视为以标称点$\bar{z}$为中心、半径为ϵ的超球面：

$$\mathcal{Z}(\epsilon) = \{\mathbf{z} \mid \|\mathbf{z} - \bar{\mathbf{z}}\|_2 \leq \epsilon\} \tag{17.5}$$

图17.2在二维空间中说明了这一概念。

通过参数化不确定性集，我们避免局限于特定的不确定性集。不确定性集过大则会牺牲解的质量，过小则会牺牲鲁棒性。对于较大间隙仍保持可行的设计点被认为更加鲁棒。

在信息间隙决策理论中，我们试图找到允许最大间隙同时保持可行性的设计点。该设计点可通过求解以下优化问题获得：

$$\mathbf{x}^* = \arg\max_{\mathbf{x} \in \mathcal{X}} \operatorname*{maximize}_{\epsilon \in [0, \infty)} \begin{cases} \epsilon & \text{if } (\mathbf{x}, \mathbf{z}) \in \mathcal{F} \text{ for all } \mathbf{z} \in \mathcal{Z}(\epsilon) \\ 0 & \text{otherwise} \end{cases} \tag{17.6}$$

该优化侧重于寻找在存在不确定性时确保可行性的设计。事实上，方程(17.6)并未明确包含目标函数f。然而，我们可以加入$f(\mathrm{x}, \mathrm{z})$不超过某个阈值$y_{\max}$的约束。这样的性能约束有助于我们避免过度的风险规避。图17.3和例17.3展示了信息gap决策理论的应用。

17.3 *Probabilistic Uncertainty*

probabilistic uncertainty 的模型使用集合 $\mathcal{Z}$ 上的分布。概率不确定性模型比基于集合的不确定性模型提供更多信息，使设计者能够考虑设计不同结果的概率。这些分布可以使用专家知识来定义，或从数据中学习。给定 $\mathcal{Z}$ 上的分布 p，我们可以使用将在第18章讨论的方法推断出 f 输出的分布。本节将

9 F. M. Hemez和Y. Ben-Haim，“模拟相关性的信息差距鲁棒性”，*Mechanical Systems and Signal Processing*，第18卷，第6期，第1443-1467页，2004年。

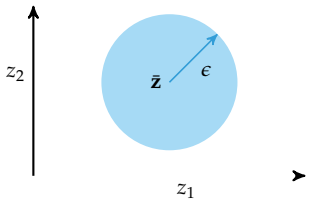


图17.2. 形式为超球面的参数化不确定性集合 $\mathcal{Z}(\epsilon)$。

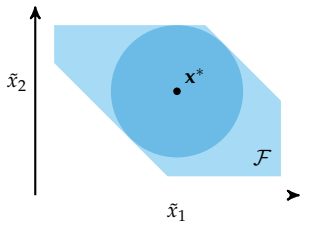
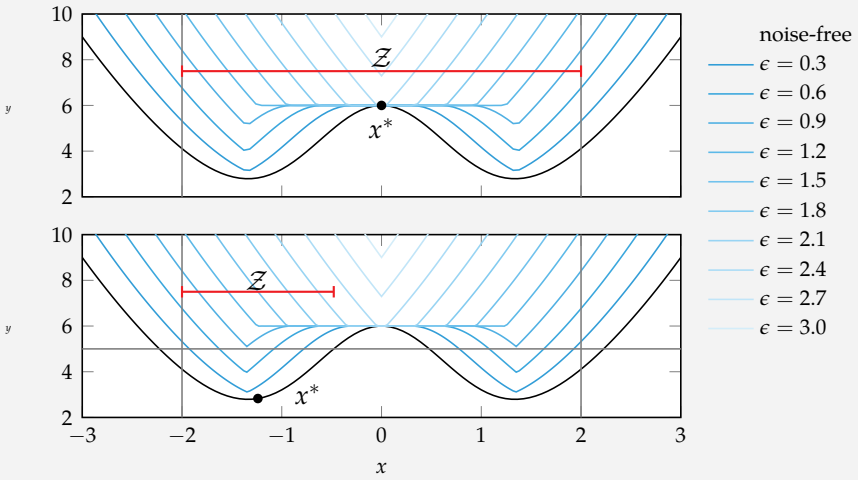


图 17.3. 信息差距决策理论应用于带有加性噪声 $f(\tilde{\mathrm{x}})$ 的目标函数，其中 $\tilde{\mathrm{x}} = \mathrm{x} + \mathrm{z}$，以及一个圆形不确定性集合

$$\mathcal{Z}(\epsilon) = \{\mathbf{z} \mid \|z\|_2 \leq \epsilon\}$$

设计 x^* 在信息差距决策理论下是最优的，因为它允许最大的 ϵ，使得所有 $\mathrm{x}^* + \mathrm{z}$ 都是可行的。

考虑 $f(x, z) = \tilde{x}^2 + 5e^{-\tilde{x}^2}$ 与 $\tilde{x} = x + z$ 的鲁棒优化，受约束 $\tilde{x} \in [-2, 2]$ 和不确定集 $\mathcal{Z}(\epsilon) = [-\epsilon, \epsilon]$ 的限制。



将信息间隙决策理论应用于此问题，会得到一个最大规模的不确定性集合，以及一个位于目标函数次优区域中心的设计方案。对最大目标函数值 $f(x, z) \leq 5$ 施加额外约束后，同一方法能够找到具有更优无噪声性能的设计。蓝线表示在给定不确定性参数 ϵ 下的最坏情况目标函数值。

示例17.3。我们可以通过在应用信息差距决策理论时对最大可接受目标函数值施加约束，来缓解过度的风险厌恶。

列出五种不同的指标，用于在给定特定设计x的情况下，将此分布转换为标量值。然后，我们可以针对这些指标进行优化。[10]

10 关于各种度量的进一步讨论可在A. Shapiro、D. Dentcheva 和 A. Ruszczyński 的著作 *Lectures on Stochastic Programming: Modeling and Theory*，第2版，SIAM，2014年中找到。

17.3.1 Expected Value

将 f 输出的分布转换为标量值的一种方法是使用 *expected value* 或 *mean*。期望值是在考虑 $f(x, z)$ 对所有 $z \in \mathcal{Z}$ 的所有输出及其相应概率时，我们可以预期的平均输出。作为设计点 x 的函数的期望值是

$$\mathbb{E}_{\mathbf{z}\sim p}[f(\mathbf{x}, \mathbf{z})] = \int_{\mathcal{Z}} f(\mathbf{x}, \mathbf{z}) p(\mathbf{z})\, d\mathbf{z} \tag{17.7}$$

期望值不一定对应于无噪声的目标函数，如例17.4所示。

计算方程（17.7）中的积分可能是无法解析完成的。人们可以使用采样或第18章讨论的其他多种更复杂的技术来近似该值。

17.3.2 Variance

除了针对函数的期望值进行优化之外，我们可能还感兴趣于选择那些其值对不确定性不过于敏感的设计点。[11] 这样的区域可以使用 f 的 *variance* 来量化：

11 有时设计者会寻找平台状区域，其中目标函数的输出相对恒定，例如生产性能一致的材料的火车调度，使它们到达时间一致。

$$\begin{aligned}\operatorname{Var}[f(\mathbf{x}, \mathbf{z})] &= \mathbb{E}_{\mathbf{z}\sim p}\left[\left(f(\mathbf{x}, \mathbf{z}) - \mathbb{E}_{\mathbf{z}\sim p}[f(\mathbf{x}, \mathbf{z})]\right)^2\right] &(17.8)\\ &= \int_{\mathcal{Z}} f(\mathbf{x}, \mathbf{z})^2 p(\mathbf{z})\, d\mathbf{z} - \mathbb{E}_{\mathbf{z}\sim p}[f(\mathbf{x}, \mathbf{z})]^2 &(17.9)\end{aligned}$$

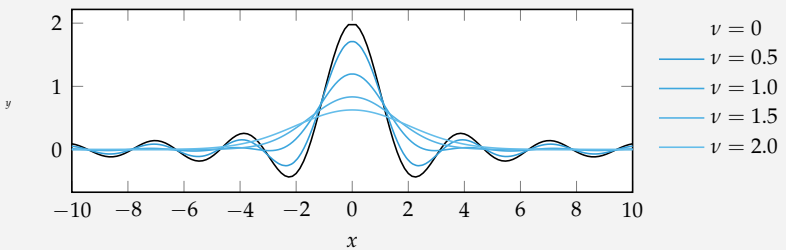
例17.4. 不确定目标函数的期望值取决于不确定性如何被纳入目标函数中。

一种常见的模型是对函数输出应用零均值高斯噪声，$f(\mathrm{x}, \mathrm{z}) = f(\mathrm{x}) + \mathrm{z}$，正如第16章中高斯过程的情况一样。期望值等同于无噪声情况：

$$\mathbb{E}_{\mathbf{z}\sim\mathcal{N}(\mathbf{0},\boldsymbol{\Sigma})}[f(\mathbf{x}) + \mathbf{z}] = \mathbb{E}_{\mathbf{z}\sim\mathcal{N}(\mathbf{0},\boldsymbol{\Sigma})}[f(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim\mathcal{N}(\mathbf{0},\boldsymbol{\Sigma})}[\mathbf{z}] = f(\mathbf{x})$$

直接将噪声添加到设计向量$f(\mathrm{x}、\mathrm{z}) = f(\mathrm{x} + \mathrm{z}) = f(\tilde{\mathrm{x}})$中也很常见。在这种情况下，期望值会受到零均值高斯噪声方差的影响。

考虑最小化$f(\tilde{x}) = \sin(2\tilde{x})/\tilde{x}$的期望值，其中$\tilde{x} = x + z$对应于从零均值高斯分布$\mathcal{N}(0, \nu)$中抽取的$z$。增加方差会增大局部函数景观对设计的影响。



上图表明，改变方差会影响最优解的位置。

我们将方差较大的设计点称为*sensitive*，将方差较小的设计点称为*robust*。敏感点和稳健点的示例如图17.4所示。我们通常感兴趣的是按期望值衡量且稳健的良好点。管理期望目标函数值与方差之间的权衡是一个多目标优化问题（见例17.5），我们可以使用第12章中讨论的技术。

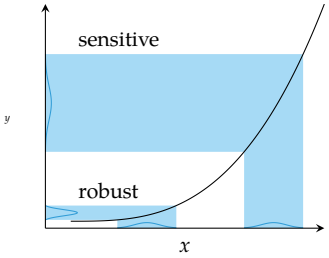


图17.4. 概率方法在模型输出上生成概率分布。设计点对不确定性可以是敏感的或稳健的。蓝色区域展示了正态分布设计上的分布如何受到目标函数的影响。

17.3.3 Statistical Feasibility

另一种可优化的替代度量是*statistical feasibility*。给定$p(\mathbf{z})$，我们可以计算设计点$\mathbf{x}$可行的概率：

$$P((\mathbf{x}, \mathbf{z}) \in \mathcal{F}) = \int_{\mathcal{Z}} ((\mathbf{x}, \mathbf{z}) \in \mathcal{F}) p(\mathbf{z}) \, d\mathbf{z} \tag{17.10}$$

这个概率可以通过采样来估计。如果我们还希望确保目标值不超过某个阈值，我们可以像信息差距决策理论那样，加入一个约束$f(\mathbf{x}, \mathbf{z}) \leq y_{\max}$。与期望值和方差指标不同，我们想要最大化这个指标。

17.3.4 Value at Risk

value at risk (*VaR*) 是在概率 α 下能够保证的最佳目标值。我们可以根据目标函数的随机输出，用 *cumula- tive distribution function*（记为 $\Phi(y)$）在数学上写出这个定义。结果小于或等于 y 的概率由 $\Phi(y)$ 给出。置信度为 α 的 VaR 是使 $\Phi(y) \geq \alpha$ 成立的最小 y 值。这个定义等价于概率分布的 α *quantile*。接近 1 的 α 对不利的离群值敏感，而接近 0 的 α 则过于乐观，并且接近最佳可能结果。

17.3.5 Conditional Value at Risk

conditional value at risk (*CVaR*)与风险价值相关。[12] CVaR是输出上概率分布的前 $1 - \alpha$ 分位数的期望值。这个量在图 17.5 中展示。

value at risk

Risk

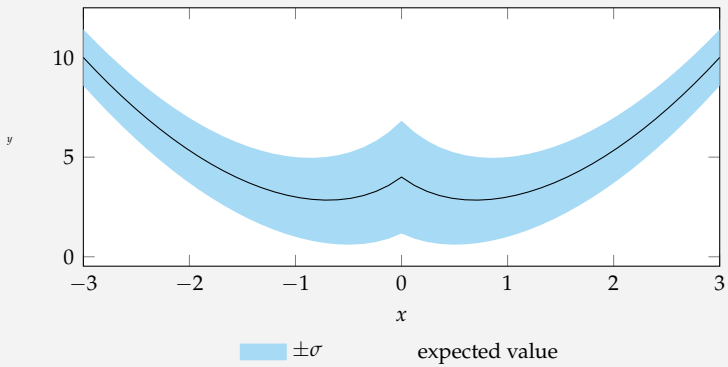
12 条件风险价值也称为*mean excess loss*、*mean shortfall*和*tail*。R. T. Rockafellar和S. Uryasev，“条件风险价值的优化”，*Journal of*，第2卷，第21–42页，2000年。

示例 17.5。在不确定性下的优化中，同时考虑期望值和方差。

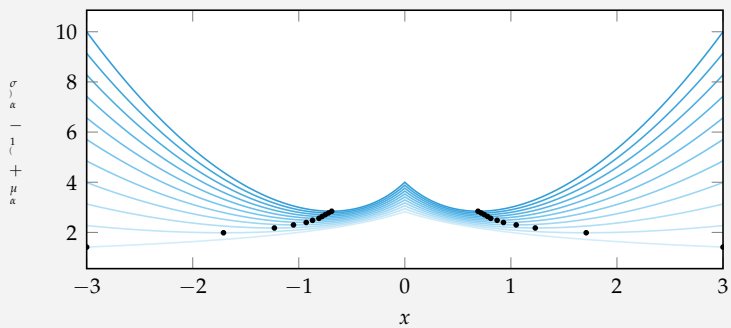
考虑目标函数 $f(x、z) = x^2 + z$，其中 z 来自依赖于 x 的 Gamma 分布。我们可以构造一个函数 `dist(x)`，它从 `Distributions.jl` 包返回一个 `Gamma` 分布：

```
dist(x) = Gamma(2/(1+abs(x)),2)
```

该分布的均值为$4/(1 + |x|)$，方差为$8/(1 + |x|)$。



我们可以找到一个稳健的优化器，使期望值和方差同时最小化。仅针对期望值进行最小化而忽略方差，会在 $x \approx \pm 0.695$ 处产生两个极小值。引入方差的惩罚项会使这些极小值偏离原点。下图展示了形式为 $\alpha\,\mathbb{E}[y \mid x] + (1-\alpha)\sqrt{\text{Var}[y \mid x]}$ 的目标函数（其中 $\alpha \in [0, 1]$）及其对应的极小值。



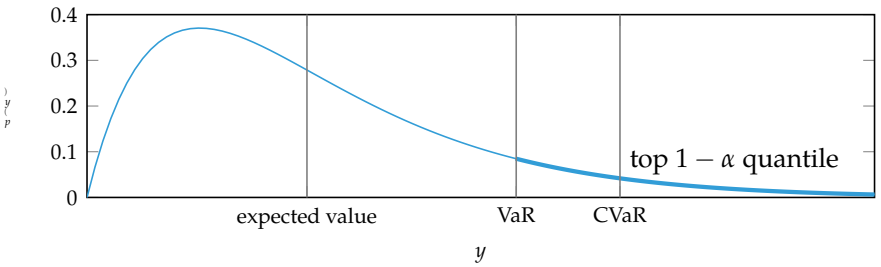


图17.5. 特定水平α下的CVaR和VaR。CVaR是最高1− α分位数的期望值，而VaR是同一分位数下的最低目标函数值。

CVaR 相较于 VaR 在理论和计算上具有一定的优势。CVaR 对目标输出分布估计误差的敏感度较低。例如，若累积分布函数在某些区间内平坦，那么 VaR 可能会因 α 的小幅变动而跳跃。此外，VaR 不考虑超过 α 分位数之外的损失，这在存在目标值极差的罕见离群点时是不理想的。[13]

17.4 Summary

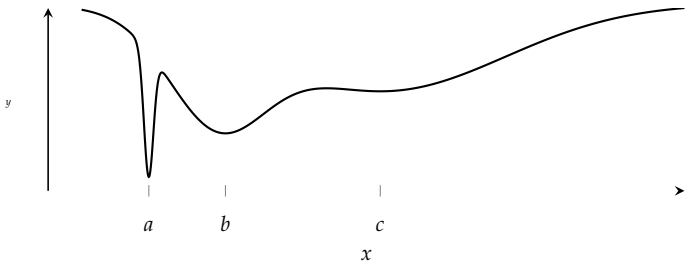
- 优化过程中的不确定性可能源于数据、模型或优化方法本身的误差。
- 在确保稳健设计时，考虑这些不确定性来源是重要的。
- 基于集合不确定性的优化包括假设最坏情况的极小极大方法，以及寻找对最大规模不确定性集具有鲁棒性的设计的信息差距决策理论。
- 概率方法通常最小化期望值、方差、不可行性风险、风险价值、条件风险价值或它们的组合。

17.5 Exercises

练习17.1。假设输入中有零均值高斯噪声，使得$f(x、z) = f(x + z)$。考虑下图中的三个点a、b和c：

13 关于性质的概述，参见 G. C. Pflug，“关于风险价值和条件风险价值的一些评论”，载于 *Probabilistic Constrained Optimization: Methodology and Applications*，S. P. Uryasev 编，Springer，2000年，第272–281页。以及 R. T. Rockafellar 和 S. Uryasev，“一般损失分布的条件风险价值”，*Journal of Banking*，第26卷，第1443–1471页，2002年。

and Finance



如果我们在最小化期望值减去标准差，那么哪个设计点是最佳的?

练习 17.2。最优解，如图 17.6 所示，常常位于约束边界上，因此对可能导致其变得不可行的不确定性很敏感。克服关于可行性的不确定性的一个方法是使约束更加严格，从而减小可行域的大小，如图 17.7 所示。

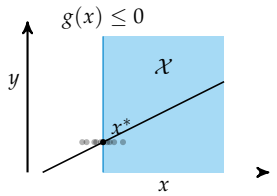


图17.6. 具有活跃约束的最优解通常对不确定性敏感。

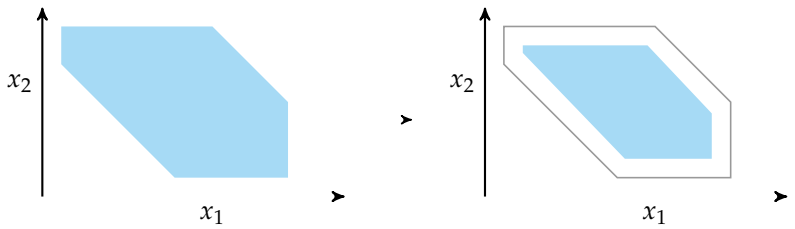


图17.7。在优化过程中施加更严格的约束可防止设计过于接近真正的可行性边界。

将形式为 $g(\mathrm{x}) \le g_{\max}$ 的约束重写为 $\gamma g(\mathrm{x}) \le g_{\max}$ 是常见的做法，其中 $\gamma > 1$ 是一个 *factor of safety*。优化使得约束值保持在 $g_{\max}/\gamma$ 以下，可提供额外的安全缓冲。

考虑一根具有正方形横截面的梁，当应力超过 $\sigma_{\max} = 1$ 时认为其失效。我们希望最小化横截面 $f(x) = x^2$，其中 x 是横截面长度。梁中的应力也是横截面长度 $g(x) = x^{-2}$ 的函数。绘制优化设计不失效的概率随安全系数从1变化到2的图：

- Maximum stress中的不确定性，$g(x,\ z) = x^{-2} + z$
- 施工公差中的不确定性，$g(x,\ z) = (x + z)^{-2}$

- 材料属性的不确定性，$g(x, z) = (1 + z)x^{-2}$

其中 z 是均值为零、方差为0.01的噪声。

练习17.3。*six-sigma*方法是统计可行性的一种特殊情况，其中改进生产或工业过程，直到其假设的高斯输出仅因超出六个标准差的异常值而违反设计要求。这一要求相当苛刻，如图17.8所示。

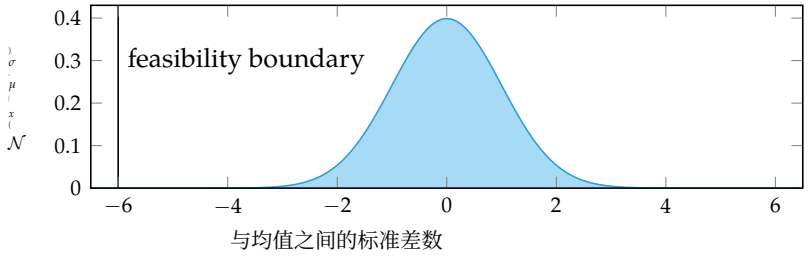


图17.8. 统计可行性可通过将目标函数均值从可行性边界移开或减少目标函数的方差来实现。

考虑优化问题

$$\begin{aligned} &\underset{\mathbf{x}}{\text{minimize}}\ x_1 \\ &\text{subject to } e^{x_1} \leq x_2 + z \leq 2e^{x_1} \end{aligned}$$

在$z \sim \mathcal{N}(0, 1)$的条件下。找到最优设计x^*，使得对于所有$|z| \leq 6$，(x, z)都是可行的。

18 Uncertainty Propagation

如前一章所述，不确定性下优化的概率方法将目标函数的一些输入建模为概率分布。本章讨论如何传播已知的输入分布，以估计与输出分布相关的量，例如目标函数的均值和方差。存在多种方法来实现{v*}，其中一些基于蒙特卡洛、泰勒级数近似、正交多项式和/高斯过程等数学概念。这些方法在所做的假设和估计质量方面有所不同。

18.1 Sampling Methods

目标函数在特定设计点处的均值和方差可以使用 *Monte Carlo integration*、[1] 来近似，这些方法近似了积分

使用来自 $\mathcal{Z}$ 上分布 p 的 m 个样本 $\mathrm{z}(1), \ldots, \mathrm{z}(m)$。这些估计也称为 *sample mean* 和 *sample variance*：

$$\mathbb{E}_{\mathbf{z}\sim p}[f(\mathbf{z})] \approx \hat{\mu} = \frac{1}{m}\sum_{i=1}^{m} f(\mathbf{z}^{(i)}) \tag{18.1}$$

$$\mathrm{Var}_{\mathbf{z}\sim p}[f(\mathbf{z})] \approx \hat{v} = \left(\frac{1}{m}\sum_{i=1}^{m} f(\mathbf{z}^{(i)})^2\right) - \hat{\mu}^2 \tag{18.2}$$

在上面的方程中，以及本章其余部分，为了符号简便，我们从$f(\mathrm{x}$ 中省略了x，z)，但对x的依赖仍然存在。在我们的优化过程中，对于每一个新的设计点x，我们重新计算均值和方差。

1 或者，可以使用拟蒙特卡洛积分来产生收敛速度更快的估计，如第13章所述。

这种基于采样的方法的一个理想特性是 p 不需要精确已知。我们可以直接从仿真或现实世界实验中获取样本。这种方法的一个潜在限制是在收敛到合适的估计之前可能需要大量样本。对于正态分布的 f，样本均值的方差为 $\text{Var}[\hat{\mu}] = \nu/m$，其中 ν 是 f 的真实方差。因此，将样本数量 m 加倍往往会使样本均值的方差减半。

18.2 Taylor Approximation

另一种估算$\hat{\mu}$和$\hat{\nu}$的方法是在固定设计点x处使用f的泰勒级数近似。[2]目前，我们将假设n分量
z 的分量相互独立且具有有限方差。我们将 z 上分布的均值记为 $\boldsymbol{\mu}$，并将 z 各个分量的方差记为 $\boldsymbol{\nu}$。[3] 以下是 $f(\mathbf{z})$ 在

点 $\mathbf{z} = \boldsymbol{\mu}$：

$$\hat{f}(\mathbf{z}) = f(\boldsymbol{\mu}) + \sum_{i=1}^{n} \frac{\partial f}{\partial z_i}(z_i - \mu_i) + \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \frac{\partial^2 f}{\partial z_i \partial z_j}(z_i - \mu_i)(z_j - \mu_j) \tag{18.3}$$

根据这个近似，我们可以解析地计算 f 的均值和方差的估计值：

$$\hat{\mu} = f(\boldsymbol{\mu}) + \frac{1}{2}\sum_{i=1}^{n} \frac{\partial^2 f}{\partial z_i^2}\nu_i \bigg|_{\mathbf{z}=\boldsymbol{\mu}} \tag{18.4}$$

$$\hat{\nu} = \sum_{i=1}^{n}\left(\frac{\partial f}{\partial z_i}\right)^2 \nu_i + \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\left(\frac{\partial^2 f}{\partial z_i \partial z_j}\right)^2 \nu_i \nu_j \bigg|_{\mathbf{z}=\boldsymbol{\mu}} \tag{18.5}$$

高阶项可以被忽略，从而获得一阶近似：

$$\hat{\mu} = f(\boldsymbol{\mu}) \qquad \hat{\nu} = \sum_{i=1}^{n}\left(\frac{\partial f}{\partial z_i}\right)^2 \nu_i \bigg|_{\mathbf{z}=\boldsymbol{\mu}} \tag{18.6}$$

我们可以放宽 z 的分量相互独立的假设，但这会使数学处理变得更加复杂。在实践中，将随机变量转换为相互独立的形式可能更容易。我们可以将一个包含 n 个相关随机变量的向量 c，其协方差矩阵为 C，转换为 m 个不相关

2 关于一般函数的均值和方差的推导，涉及n随机变量，参见H. Benaroya和S. M. Han, *Probability Models*。Taylor & Francis, 2005年。

in Engineering and Science

3 如果 z 的分量是独立的，那么协方差矩阵是对角矩阵，而 {v*} 将是由对角线元素组成的向量。

通过乘以一个正交矩阵T，其中包含对应于C的m个最大特征值的特征向量，从而得到随机变量z。我们有$z = Tc$.[4]

泰勒近似方法在算法18.1中实现。一阶和二阶近似在例18.1中进行了比较。

4 通常还会对输出进行缩放，以使协方差矩阵成为单位矩阵。这一过程被称为*whitening*。J. H. Friedman，“探索性投影寻踪”，*Journal of the American Statistical Association*，第82卷，第39–249–266页，1987年。

```
using ForwardDiff
function taylor_approx(f, μ, ν, secondorder=false)
    μhat = f(μ)
    ∇ = (z -> ForwardDiff.gradient(f, z))(μ)
    νhat = ∇.^2⋅ν
    if secondorder
        H = (z -> ForwardDiff.hessian(f, z))(μ)
        μhat += (diag(H)⋅ν)/2
        νhat += ν⋅(H.^2*ν)/2
    end
    return (μhat, νhat)
end
```

算法18.1. 一种自动计算目标函数f在设计点x处均值和方差的泰勒近似的方法，其中噪声均值向量为μ，方差向量为ν。布尔参数secondorder控制计算一阶还是二阶近似。

18.3 Polynomial Chaos

Polynomial chaos 是一种将多项式拟合到$f(z)$的评估值，并使用所得代理模型来估计均值和方差的方法。我们将在本节开始讨论如何在单变量情况下使用多项式混沌。然后，我们将这一概念推广到多变量函数，并展示如何通过积分由我们的代理模型表示的函数来获得均值和方差的估计。

18.3.1 Univariate

在一维情况下，我们用由k个多项式基函数b_1、...、b_k组成的代理模型来近似$f(z)$：

$$f(z) \approx \hat{f}(z) = \sum_{i=1}^{k} \theta_i b_i(z) \tag{18.7}$$

例18.1. 泰勒近似应用于具有二维高斯噪声的单变量设计问题。

考虑目标函数 $f(x、z) = \sin(x + z_1)\cos(x + z_2)$，其中 z_1 和 z_2 是均值为零的高斯噪声，方差分别为 0.1 和 0.2。f 关于 z 的一阶和二阶偏导数为

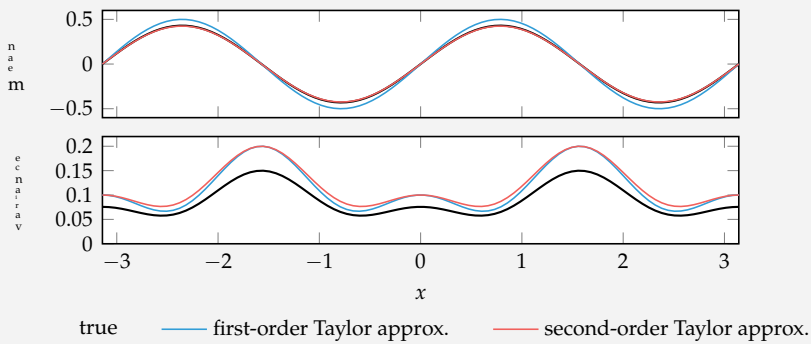
$$\frac{\partial f}{\partial z_1} = \cos(x + z_1)\cos(x + z_2) \qquad \frac{\partial^2 f}{\partial z_2^2} = -\sin(x + z_1)\cos(x + z_2)$$
$$\frac{\partial f}{\partial z_2} = -\sin(x + z_1)\sin(x + z_2) \qquad \frac{\partial^2 f}{\partial z_1 \partial z_2} = -\cos(x + z_1)\sin(x + z_2)$$
$$\frac{\partial^2 f}{\partial z_1^2} = -\sin(x + z_1)\cos(x + z_2)$$

这使我们能够构建泰勒近似：

$$\hat{\mu}(x) = 0.85\sin(x)\cos(x)$$
$$\hat{v}(x) = 0.3\sin^2(x)\cos^2(x) - 0.035\sin(x)\cos(x)$$

我们可以使用 `taylor_approx` 为给定的 `x` 通过以下方式：

```
taylor_approx(z->sin(x+z[1])*cos(x+z[2]), [0,0], [0.1,0.2])
```



与第18.1节讨论的蒙特卡洛方法不同，我们对z的采样不必从p中随机抽取。实际上，使用第13章讨论的抽样方案之一来获取样本可能是可取的。我们将在第18.3.2节讨论如何获得基系数。

我们可以使用代理模型$\hat{f}$来估计均值：

$$\begin{aligned}
\hat{\mu} &= \mathbb{E}\left[\hat{f}\right] && (18.8)\\
&= \int_{\mathcal{Z}} \hat{f}(z)p(z)\,dz && (18.9)\\
&= \int_{\mathcal{Z}} \sum_{i=1}^{k} \theta_i b_i(z)p(z)\,dz && (18.10)\\
&= \sum_{i=1}^{k} \theta_i \int_{\mathcal{Z}} b_i(z)p(z)\,dz && (18.11)\\
&= \theta_1 \int_{\mathcal{Z}} b_1(z)p(z)\,dz + \ldots + \theta_k \int_{\mathcal{Z}} b_k(z)p(z)\,dz && (18.12)
\end{aligned}$$

我们还可以估计方差：

$$\begin{aligned}
\hat{v} &= \mathbb{E}\left[\left(\hat{f} - \mathbb{E}\left[\hat{f}\right]\right)^2\right] && (18.13)\\
&= \mathbb{E}\left[\hat{f}^2\right] - \mathbb{E}\left[\hat{f}\right]^2 && (18.14)\\
&= \int_{\mathcal{Z}} \hat{f}(z)^2 p(z)\,dz - \mu^2 && (18.15)\\
&= \int_{\mathcal{Z}} \sum_{i=1}^{k}\sum_{j=1}^{k} \theta_i\theta_j b_i(z)b_j(z)p(z)\,dz - \mu^2 && (18.16)\\
&= \int_{\mathcal{Z}} \left(\sum_{i=1}^{k} \theta_i^2 b_i(z)^2 + 2\sum_{i=2}^{k}\sum_{j=1}^{i-1} \theta_i\theta_j b_i(z)b_j(z)\right) p(z)\,dz - \mu^2 && (18.17)\\
&= \int_{\mathcal{Z}} \left(\sum_{i=1}^{k} \theta_i^2 b_i(z)^2 + 2\sum_{i=2}^{k}\sum_{j=1}^{i-1} \theta_i\theta_j b_i(z)b_j(z)\right) p(z)\,dz - \mu^2 && (18.18)\\
&= \sum_{i=1}^{k} \theta_i^2 \int_{\mathcal{Z}} b_i(z)^2 p(z)\,dz + 2\sum_{i=2}^{k}\sum_{j=1}^{i-1} \theta_i\theta_j \int_{\mathcal{Z}} b_i(z)b_j(z)p(z)\,dz - \mu^2 && (18.19)
\end{aligned}$$

如果基函数选择为 p 下的 *orthogonal*，则可以高效地计算均值和方差。两个基函数 b_i 和 b_j 正交，且

关于概率密度 $p(z)$，如果

$$\int_{\mathcal{Z}} b_i(z) b_j(z) p(z)\, dz = 0 \quad \text{if } i \neq j \tag{18.20}$$

如果所选的基函数彼此正交，且第一个基函数为$b_1(z) = 1$，则均值为：

$$\begin{aligned}
\hat{\mu} &= \theta_1 \int_{\mathcal{Z}} b_1(z) p(z)\, dz + \theta_2 \int_{\mathcal{Z}} b_2(z) p(z)\, dz + \cdots + \theta_k \int_{\mathcal{Z}} b_k(z) p(z)\, dz && (18.21)\\
&= \theta_1 \int_{\mathcal{Z}} b_1(z)^2 p(z)\, dz + \theta_2 \int_{\mathcal{Z}} b_1(z) b_2(z) p(z)\, dz + \cdots + \theta_k \int_{\mathcal{Z}} b_1(z) b_k(z) p(z)\, dz && (18.22)\\
&= \theta_1 \int_{\mathcal{Z}} p(z)\, dz + 0 + \cdots + 0 && (18.23)\\
&= \theta_1 && (18.24)
\end{aligned}$$

类似地，方差为：

$$\begin{aligned}
\hat{v} &= \sum_{i=1}^{k} \theta_i^2 \int_{\mathcal{Z}} b_i(z)^2 p(z)\, dz + 2 \sum_{i=2}^{k} \sum_{j=1}^{i-1} \theta_i \theta_j \int_{\mathcal{Z}} b_i(z) b_j(z) p(z)\, dz - \mu^2 && (18.25)\\
&= \sum_{i=1}^{k} \theta_i^2 \int_{\mathcal{Z}} b_i(z)^2 p(z)\, dz - \mu^2 && (18.26)\\
&= \theta_1^2 \int_{\mathcal{Z}} b_1(z)^2 p(z)\, dz + \sum_{i=2}^{k} \theta_i^2 \int_{\mathcal{Z}} b_i(z)^2 p(z)\, dz - \theta_1^2 && (18.27)\\
&= \sum_{i=2}^{k} \theta_i^2 \int_{\mathcal{Z}} b_i(z)^2 p(z)\, dz && (18.28)
\end{aligned}$$

因此，均值直接通过将代理模型拟合到观测数据得出，而给定基函数和概率分布选择下的值 $\int_{\mathcal{Z}} b_i(z)^2 p(z)\, dz$ 时，可以非常高效地计算方差。[5] 例18.2使用这些程序来估计不同样本大小的均值和方差。

使用高斯求积可以高效地计算这种形式的5个积分，相关内容见附录C.8。

多项式混沌使用k次正交多项式基函数来近似该函数，其中$i \in \{1, \ldots, k+1\}$和$b_1 = 1$。所有正交多项式均满足递推关系：

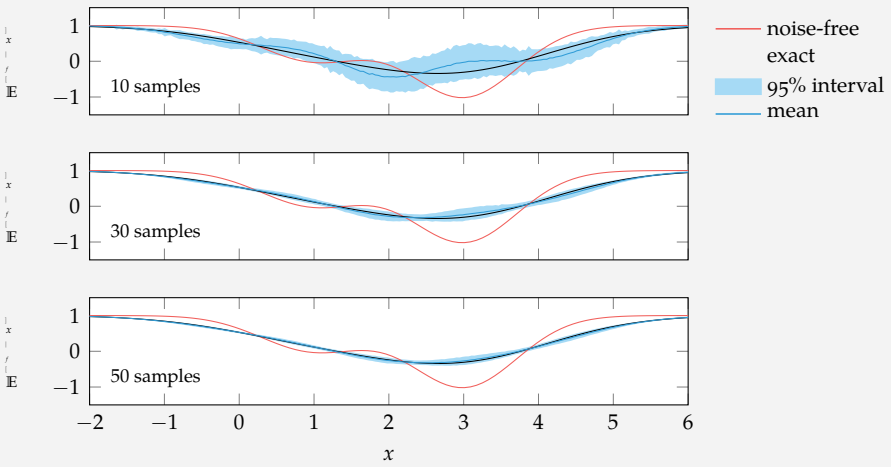
$$b_{i+1}(z) = \begin{cases} (z - \alpha_i) b_i(z) & \text{for } i = 1 \\ (z - \alpha_i) b_i(z) - \beta_i b_{i-1}(z) & \text{for } i > 1 \end{cases} \tag{18.29}$$

考虑优化（未知的）目标函数

$$f(x,z) = 1 - e^{-(x+z-1)^2} - 2e^{-(x+z-3)^2}$$

其中已知z来自零均值单位高斯分布。

目标函数、其真实期望值以及使用不同样本数量估计的期望值绘制如下。估计的期望值使用三阶Hermite多项式计算。



示例18.2. 使用多项式混沌估计未知目标函数的期望值。

使用 $b_1(z) = 1$ 和权重

$$\alpha_i = \frac{\int_{\mathcal{Z}} z\, b_i(z)^2 p(z)\, dz}{\int_{\mathcal{Z}} b_i(z)^2 p(z)\, dz}$$
$$\beta_i = \frac{\int_{\mathcal{Z}} b_i(z)^2 p(z)\, dz}{\int_{\mathcal{Z}} b_{i-1}(z)^2 p(z)\, dz} \tag{18.30}$$

递推关系可用于生成基函数。每个基函数 b_i 是次数为 $i-1$ 的多项式。若干常见概率分布的基函数在表 18.1 中给出，可使用算法 18.2 中的方法生成，并绘制在图 18.1 中。示例 18.3 说明了多项式阶数对均值和方差估计的影响。

Distribution	Domain	Density	Name	Recursive Form	Closed Form
Uniform	$[-1, 1]$	$\frac{1}{2}$	Legendre	$\text{Le}_k(x) = \frac{1}{2^k k!}\frac{d^k}{dx^k}\left[\left(x^2-1\right)^k\right]$	$b_i(x) = \sum_{j=0}^{i-1}\binom{i-1}{j}\binom{-i-2}{j}\left(\frac{1-x}{2}\right)^j$
Exponential	$[0, \infty)$	e^{-x}	Laguerre	$\frac{d}{dx}\text{La}_k(x) = \left(\frac{d}{dx} - 1\right)\text{La}_{k-1}$	$b_i(x) = \sum_{j=0}^{i-1}\binom{i-1}{j}\frac{(-1)^j}{j!}x^j$
Unit Gaussian	$(-\infty, \infty)$	$\frac{1}{\sqrt{2\pi}}e^{-x^2/2}$	Hermite	$\text{H}_k(x) = x\text{H}_{k-1} - \frac{d}{dx}\text{H}_{k-1}$	$b_i(x) = \sum_{j=0}^{\lfloor (i-1)/2 \rfloor}(i-1)!\frac{(-1)^{\frac{i-1}{2}-j}}{(2j)!(\frac{i-1}{2}-j)!}(2x)^{2j}$

表18.1. 几种常见概率分布的正交多项式基函数。

可以为任意概率密度函数和定义域解析地和数值地构造基函数。[6] *Stieltjes algorithm*[7] (算法18.3)使用方程（18.29）中的递推关系生成正交多项式。例18.4展示了多项式阶数如何影响均值和方差的估计。

6 多项式可以按非零因子缩放。按照惯例，设定 $b_1(x) = 1$。

7 T. J. Stieltjes, “关于所谓机械求积理论的一些研究”, *Annales Scientifiques de l'École Normale Supérieure*, 第409–426页, 1884年。英文概述由W. Gautschi提供, *Orthog-onal Polynomials: Computation and Approximation*。牛津大学出版社，2004年

18.3.2 *Coefficients*

方程（18.7）中的系数 $\theta_1, \ldots, \theta_k$ 可通过两种不同的方式推得。第一种方式是利用第14.3节讨论的线性回归方法，对来自 $\mathcal{Z}$ 的样本值进行拟合。第二种方式是利用基函数的正交性，产生一个适合高斯求积的积分项。

```
using Polynomials
function legendre(i)
    n = i-1
    p = Poly([-1,0,1])^n
    for i in 1 : n
        p = polyder(p)
    end
    return p / (2^n * factorial(n))
end
function laguerre(i)
    p = Poly([1])
    for j in 2 : i
        p = polyint(polyder(p) - p) + 1
    end
    return p
end
function hermite(i)
    p = Poly([1])
    x = Poly([0,1])
    for j in 2 : i
        p = x*p - polyder(p)
    end
    return p
end
```

算法18.2. 构造多项式正交基函数的方法，其中i表示b_i的构造。

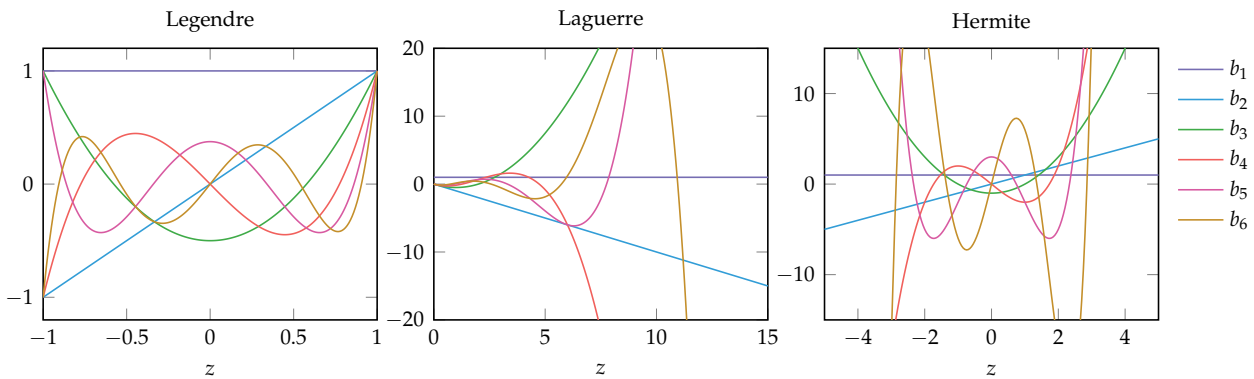


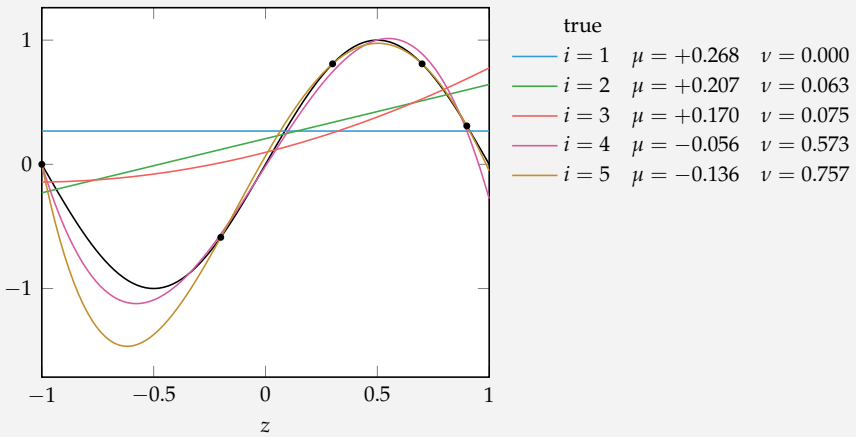
图18.1. 均匀分布、指数分布和单位高斯分布的正交基函数。

考虑函数 $f(z) = \sin(\pi z)$，其输入 z 取自域 $[-1, 1]$ 上的均匀分布。真实的均值和方差可以通过解析方法计算：

$$\mu = \int_a^b f(z)p(z)\,dz = \int_{-1}^{1} \sin(\pi z)\frac{1}{2}\,dz = 0 \tag{18.31}$$

$$\nu = \int_a^b f(z)^2 p(z)\,dz - \mu^2 = \int_{-1}^{1} \sin^2(\pi z)\frac{1}{2}\,dz - 0 = \frac{1}{2} \tag{18.32}$$

假设我们有五个 f 样本，位于 $z = \{-1、-0.2、0.3、0.7、0.9\}$。我们可以对数据拟合一个勒让德多项式来获得我们的代理模型 $\hat{f}$。不同次数的多项式得到：



示例 18.3. Legendre 多项式用于估计具有均匀分布输入的函数的均值和方差。

```
using Polynomials
function orthogonal_recurrence(bs, p, dom, ϵ=1e-6)
    i = length(bs)
    c1 = quadgk(z->z*bs[i](z)^2*p(z), dom..., atol=ϵ)[1]
    c2 = quadgk(z->  bs[i](z)^2*p(z), dom..., atol=ϵ)[1]
    α = c1 / c2
    if i > 1
        c3 = quadgk(z->bs[i-1](z)^2*p(z), dom..., atol=ϵ)[1]
        β = c2 / c3
        return Poly([-α, 1])*bs[i] - β*bs[i-1]
    else
        return Poly([-α, 1])*bs[i]
    end
end
```

算法18.3。用于根据正交递推关系构造下一个多项式基函数b_{i+1}的Stieltjes算法，其中`bs`包含$\{b_1、...、b_i\}$，`p`是概率分布，`dom`是包含z的下界和上界的元组。可选参数`ϵ`控制数值积分的绝对容差。我们使用了`Polynomials.jl`包。

我们将方程（18.7）的每一边乘以第j个基函数和我们的概率密度函数，并进行积分：

$$f(z) \approx \sum_{i=1}^{k} \theta_i b_i(z) \tag{18.33}$$

$$\int_{\mathcal{Z}} f(z) b_j(z) p(z)\, dz \approx \int_{\mathcal{Z}} \left(\sum_{i=1}^{k} \theta_i b_i(z) \right) b_j(z) p(z)\, dz \tag{18.34}$$

$$= \sum_{i=1}^{k} \theta_i \int_{\mathcal{Z}} b_i(z) b_j(z) p(z)\, dz \tag{18.35}$$

$$= \theta_j \int_{\mathcal{Z}} b_j(z)^2 p(z)\, dz \tag{18.36}$$

其中我们利用了公式（18.20）中的正交性质。

由此可知，第 j 个系数为：

$$\theta_j = \frac{\int_{\mathcal{Z}} f(z) b_j(z) p(z)\, dz}{\int_{\mathcal{Z}} b_j(z)^2 p(z)\, dz} \tag{18.37}$$

方程（18.37）的分母通常具有已知的解析解，或者可以低成本地预先计算。因此，计算该系数主要需要求解分子中的积分，这可以通过使用高斯求积进行数值计算来完成。[8]

8 高斯求积在 `QuadGK.jl` 中通过 `quadgk` 函数实现，并在附录 C.8 中涵盖。求积规则也可以通过使用由方程 (18.30) 中的系数 α_i 和 β_i 形成的三对角矩阵的特征值和特征向量来获得。G. H. Golub 和 J. H. Welsch，“高斯求积规则的计算”，*Mathematics*，第 23 卷，第 106 期，第 221–230 页，1969 年。

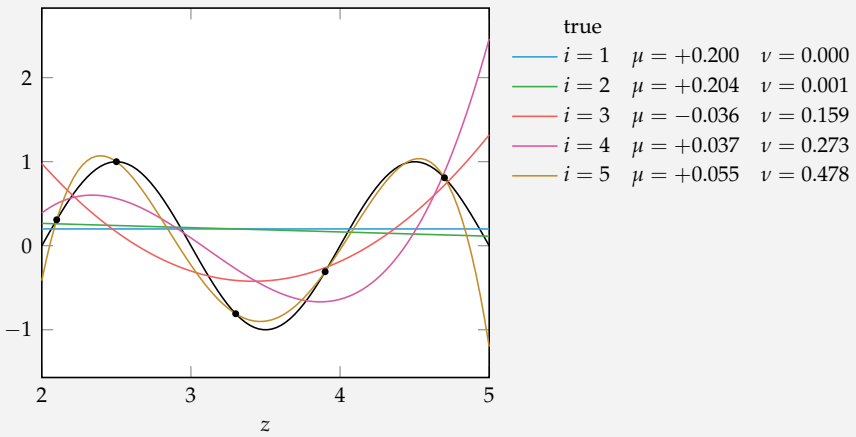
考虑函数$f(z) = \sin(\pi z)$，输入z取自截断高斯分布，其均值为3，方差为1，定义域为[2, 5]。真实的均值和方差为：

$$\mu = \int_a^b f(z)p(z)\,dz = \int_2^5 \sin(\pi z)p(z)\,dz \approx 0.104$$
$$\nu = \int_a^b f(z)^2 p(z)\,dz - \mu^2 = \int_2^5 \sin^2(\pi z)p(z)\,dz - 0.104^2 \approx 0.495$$

其中截断高斯分布的概率密度为：

$$p(z) = \begin{cases} \frac{\mathcal{N}(z|3,1)}{\int_2^5 \mathcal{N}(\tau|3,1)\,d\tau} & \text{if } z \in [2,5] \\ 0 & \text{otherwise} \end{cases}$$

假设我们在$z = \{2.1、2.5、3.3、3.9、4.7\}$处有五个$f$样本。我们可以对数据拟合正交多项式来获得我们的代理模型$\hat{f}$。不同次数的多项式产生：



示例 18.4. 使用 Stieltjes 方法构建的 Legendre 多项式，以估计具有随机变量输入的函数的均值和方差。

18.3.3 Multivariate

多项式混沌可以应用于具有多个随机输入的函数。多元基函数在m个变量上被构造为一元正交多项式的乘积：

$$b_i(\mathbf{z}) = \prod_{j=1}^{m} b_{a_j}(z_j) \tag{18.38}$$

其中a是一个分配向量，它将a_j基函数分配给j随机分量。这种基函数构造在例18.5中进行了演示。

例18.5. 使用方程(18.38)构建多变量多项式混沌基函数。

考虑一个三维多项式混沌模型，其中一个多维基函数是 $b(\mathbf{z}) = b_3(z_1)b_1(z_2)b_3(z_3)$。相应的分配向量是一个 = [3, 1, 3]。

一种构造多变量基函数的常见方法是，为每个随机变量生成单变量正交多项式，然后为所有可能的组合构建多变量基函数。[9] 此过程在算法18.4中实现。以这种方式构建基函数假设变量是独立的。相互依赖可以通过第18.2节中讨论的相同变换来解决。

[9] 这里多元指数基函数的数量随变量数量呈指数增长。

算法18.4。构建多变量基函数的方法，其中`bases1d`包含每个随机变量的单变量正交基函数列表。

```
function polynomial_chaos_bases(bases1d)
    bases = []
    for a in product(bases1d...)
        push!(bases,
            z -> prod(b(z[i]) for (i,b) in enumerate(a)))
    end
    return bases
end
```

一个具有 k 个基函数的多元多项式混沌近似仍然是项　　的线性组合

$$f(\mathbf{z}) \approx \hat{f}(\mathbf{z}) = \sum_{i=1}^{k} \theta_i b_i(\mathbf{z}) \tag{18.39}$$

其中均值和方差可以使用第18.3.1节中的方程计算，只要$b_1(\mathrm{z}) = 1$。

18.4 Bayesian Monte Carlo

第16章介绍的高斯过程是定义在函数上的概率分布。它们可以作为随机目标函数的替代模型。我们可以将先验信息，例如目标函数的期望平滑度，纳入一个称为 *Bayesian Monte Carlo* 或 *Bayes-Hermite Quadrature* 的过程中。

考虑一个高斯过程，它拟合了几个设计点x值相同但不确定点z值不同的数据点。所得到的高斯过程是基于观测数据的函数分布。当通过积分求期望值时，我们必须考虑高斯过程$p(\hat{f})$所表示的概率分布中函数的期望值：

$$\begin{aligned}
\mathbb{E}_{\mathbf{z}\sim p}[f] &\approx \mathbb{E}_{\hat{f}\sim p(\hat{f})}[\hat{f}] && (18.40)\\
&= \int_{\hat{\mathcal{F}}} \left(\int_{\mathcal{Z}} \hat{f}(\mathbf{z}) p(\mathbf{z})\, d\mathbf{z} \right) p(\hat{f})\, d\hat{f} && (18.41)\\
&= \int_{\mathcal{Z}} \left(\int_{\hat{\mathcal{F}}} \hat{f}(\mathbf{z}) p(\hat{f})\, d\hat{f} \right) p(\mathbf{z})\, d\mathbf{z} && (18.42)\\
&= \int_{\mathcal{Z}} \hat{\mu}(\mathbf{z}) p(\mathbf{z})\, d\mathbf{z} && (18.43)
\end{aligned}$$

其中$\hat{\mu}(\mathrm{z})$是高斯过程下的预测均值，而$\hat{\mathcal{F}}$是函数空间。估计的方差为

$$\begin{aligned}
\mathrm{Var}_{\mathbf{z}\sim p}[f] &\approx \mathrm{Var}_{\hat{f}\sim p(\hat{f})}[\hat{f}] && (18.44)\\
&= \int_{\hat{\mathcal{F}}} \left(\int_{\mathcal{Z}} \hat{f}(\mathbf{z}) p(\mathbf{z})\, d\mathbf{z} - \int_{\mathcal{Z}} \mathbb{E}[\hat{f}(\mathbf{z}')] p(\mathbf{z}')\, d\mathbf{z}' \right)^2 p(\hat{f})\, d\hat{f} && (18.45)\\
&= \int_{\mathcal{Z}} \int_{\mathcal{Z}} \int_{\hat{\mathcal{F}}} \left[\hat{f}(\mathbf{z}) - \mathbb{E}[\hat{f}(\mathbf{z})] \right] \left[\hat{f}(\mathbf{z}') - \mathbb{E}[\hat{f}(\mathbf{z}')] \right] p(\hat{f})\, d\hat{f} p(\mathbf{z}) p(\mathbf{z}')\, d\mathbf{z}\, d\mathbf{z}' && (18.46)\\
&= \int_{\mathcal{Z}} \int_{\mathcal{Z}} \mathrm{Cov}(\hat{f}(\mathrm{z}), \hat{f}(\mathrm{z}')) p(\mathrm{z}) p(\mathrm{z}')\, d\mathrm{z}\, d\mathrm{z}' && (18.47)
\end{aligned}$$

wher e Cov 是高斯先验下的后验协方差　　进程：

$$\mathrm{Cov}(\hat{f}(\mathbf{z}), \hat{f}(\mathbf{z}')) = k(\mathbf{z}, \mathbf{z}') - k(\mathbf{z}, Z)\mathbf{K}(Z, Z)^{-1} k(Z, \mathbf{z}') \qquad (18.48)$$

其中 Z 包含观测到的输入。

对于z为高斯分布的特殊情况，均值和方差的解析表达式存在。[10]在高斯核下，

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{1}{2}\sum_{i=1}^{n}\frac{(x_i - x_i')^2}{w_i^2}\right) \qquad \textit{relation rule} \tag{18.49}$$

高斯不确定性 $\mathrm{z} \sim \mathcal{N}(\boldsymbol{\mu}_\mathbf{z}, \boldsymbol{\Sigma}_\mathbf{z})$ 的均值为

$$\mathbb{E}_{\mathbf{z}\sim p}[f] = \mathbf{q}^\top \mathbf{K}^{-1}\mathbf{y} \tag{18.50}$$

使用

$$q_i = |\mathbf{W}^{-1}\boldsymbol{\Sigma}_\mathbf{z} + \mathbf{I}|^{-1/2}\exp\left(-\frac{1}{2}\left(\boldsymbol{\mu}_\mathbf{z} - \hat{\mu}(\mathbf{z}^{(i)})\right)^\top(\boldsymbol{\Sigma}_\mathbf{z} + \mathbf{W})^{-1}\left(\boldsymbol{\mu}_\mathbf{z} - \mathbf{z}^{(i)}\right)\right) \tag{18.51}$$

其中 $\mathrm{W} = \mathrm{diag}[w_1^2, \ldots, w_n^2]$，并且我们使用样本 $(\mathrm{z}^{(i)}, y_i)$ 针对 $i \in \{1, \ldots, m\}$ 构建了高斯过程。[11]

方差为 {v*}

$$\mathrm{Var}_{\mathbf{z}\sim p}[f] = |2\mathbf{W}^{-1}\boldsymbol{\Sigma}_\mathbf{z} + \mathbf{I}|^{-1/2} - \mathbf{q}^\top\mathbf{K}^{-1}\mathbf{q} \tag{18.52}$$

即使解析表达式不可用，对于许多问题，通过数值方法计算期望值足够经济，以至于高斯过程方法优于蒙特卡洛估计。

贝叶斯蒙特卡洛在算法18.5中实现，并在例18.6中详细说明。

```
function bayesian_monte_carlo(GP, w, μz, Σz)
    W = Matrix(Diagonal(w.^2))
    invK = inv(K(GP.X, GP.X, GP.k))
    q = [exp(-((z-μz)·(inv(W+Σz)*(z-μz)))/2) for z in GP.X]
    q .*= (det(W\Σz + I))^(-0.5)
    μ = q'*invK*GP.y
    ν = (det(2W\Σz + I))^(-0.5) - (q'*invK*q)[1]
    return (μ, ν)
end
```

算法18.5. 一种获取函数在高斯过程 `GP` 下期望值的贝叶斯蒙特卡洛估计的方法，该过程采用权重为`w`的高斯核，其中变量从均值为`μz`、协方差为`Σz`的正态分布中抽取。

10 还要求协方差函数满足 *product cor-*，即它可以写成一元正定函数 r 的乘积：

$$k(\mathbf{x}, \mathbf{x}') = \prod_{i=1}^{n} r(x_i - x_i')$$

对于多项式核和高斯混合模型，已有分析结果。C. E. Rasmussen 和 Z. Ghahramani，“贝叶斯蒙特卡洛”，载于 *Advances in Neural Information Processing Systems* (*NIPS*)，2003年。

11 见 A. Girard、C. E. Rasmussen、J. Q. Candela 和 R. Murray-Smith，“具有不确定输入的高斯过程先验——在多步超前时间序列预测中的应用”，载于 *Advances in Neural Information Processing Systems* (*NIPS*)，2003年。

再次考虑估计 $f(x、z) = \sin(x + z_1)\cos(x + z_2)$ 的期望值与方差，其中 z_1 和 z_2 是零均值高斯噪声，方差分别为 1 和 1/2：$\boldsymbol{\mu}_\mathbf{z} = [0, 0]$ 且 $\boldsymbol{\Sigma}_\mathbf{z} = \text{diag}([1, 1/2])$。

我们使用贝叶斯蒙特卡洛方法，采用单位权重为 $\text{w} = [1, 1]$ 的高斯核，针对 $x = 0$ 的情况，样本点为 $\text{Z} = \{[0, 0], [1, 0], [-1, 0], [0, 1], [0, -1]\}$。我们计算：

$$\mathbf{W} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

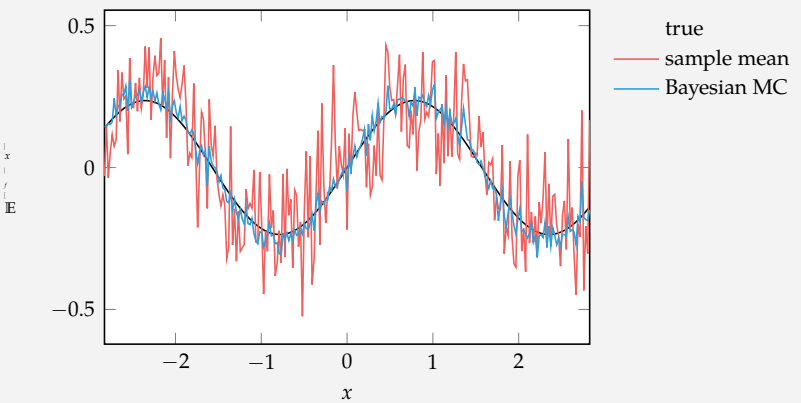
$$\mathbf{K} = \begin{bmatrix} 1 & 0.607 & 0.607 & 0.607 & 0.607 \\ 0.607 & 1 & 0.135 & 0.368 & 0.368 \\ 0.607 & 0.135 & 1 & 0.368 & 0.368 \\ 0.607 & 0.368 & 0.368 & 1 & 0.135 \\ 0.607 & 0.368 & 0.368 & 0.135 & 1 \end{bmatrix}$$

$$\mathbf{q} = [0.577, 0.4500.450, 0.417, 0.417]$$

$$\mathbb{E}_{\mathbf{z}\sim p}[f] = 0.0$$

$$\text{Var}_{\mathbf{z}\sim p}[f] = 0.327$$

下面我们使用相同的方法，绘制期望值作为 x 的函数，每个点取十个 z 的随机样本。



例18.6. 使用贝叶斯蒙特卡洛方法估计函数期望值和方差的一个示例。

该图比较了贝叶斯蒙特卡洛方法与样本均值在估计函数期望值方面的表现。对于每个被评估的 x 生成的相同随机抽样 z 值被输入到每种方法中。

18.5 Summary

- 目标函数的期望值和方差在优化涉及不确定性的问题时非常有用，但可靠地计算这些量可能具有挑战性。
- 最简单的方法之一是通过采样来估计矩，这一过程被称为蒙特卡洛积分。
- 其他方法，如泰勒近似，利用目标函数偏导数的知识。
- 多项式混沌是一种基于正交多项式的强大不确定性传播技术。
- 贝叶斯蒙特卡洛使用高斯过程，以高效地得到高斯核的矩的解析结果。

18.6 Exercises

练习 18.1. 假设我们从单变量高斯分布中抽取一个样本。我们的样本落在均值的一个标准差范围内（$x \in [\mu - \sigma, \mu + \sigma]$）的概率是多少？我们的样本小于均值加上一个标准差（$x < \mu + \sigma$）的概率是多少？

练习 18.2。设 $x^{(1)}, x^{(2)}, \ldots, x^{(m)}$ 是来自均值为 μ、方差为 ν 的分布的大小为 m 的独立同分布随机样本。证明样本均值的方差 $\text{Var}(\hat{\mu})$ 为 ν/m。

练习 18.3. 推导所有正交多项式均满足的递推关系式（18.29）。

练习 18.4。假设我们使用 m 次评估，其中 $\text{z}^{(1)}, \ldots, \text{z}^{(m)}$，针对特定设计点 x 拟合了目标函数 $f(\text{x}, \text{z})$ 的多项式混沌模型。推导一个表达式，用于估计多项式混沌系数关于设计分量 x_i 的偏导数。

练习 18.5。考虑一个目标函数$f(\mathrm{x}, \mathrm{z})$，其中设计变量为 x，随机变量为 z。如第17章所述，不确定性优化通常涉及最小化估计均值与方差的线性组合：

$$f_{\text{mod}}(\mathbf{x}, \mathbf{z}) = \alpha \hat{\mu}(\mathbf{x}) + (1 - \alpha)\hat{v}(\mathbf{x})$$

如何使用多项式混沌来估计f_{mod}关于设计变量x的梯度?

19 Discrete Optimization

前几章重点讨论了涉及连续设计变量的优化问题。然而，许多问题的设计变量本质上是离散的，例如涉及固定尺寸机械部件的制造问题或涉及离散路径选择的导航问题。*discrete optimization* 问题具有约束条件，使得设计变量必须从离散集合中选择。一些离散优化问题具有无限的设计空间，另一些则是有限的。[1] 即使对于有限的问题，

对于这类问题，尽管理论上我们可以枚举所有可能的解，但在实践中通常不具备计算可行性。本章讨论解决离散优化问题的精确方法和近似方法，这些方法避免了枚举。之前涵盖的许多方法，如模拟退火和遗传编程，很容易适应离散优化问题，但本章我们将重点放在尚未讨论的技术类别上。

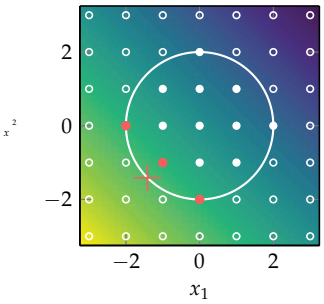
1 具有有限设计空间的离散优化有时被称为 *combinatorial optimization*。有关综述，请参见 B. Korte 和 J. Vygen, *Combinatorial Optimization: Theory and Algorithms*，第 4 版，Springer，2012 年。

Discrete optimization constrains the design to be integral. Consider the problem:

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & x_1 + x_2 \\ \text{subject to} \quad & \|\mathbf{x}\| \leq 2 \\ & \mathbf{x} \text{ is integral} \end{aligned}$$

The optimum in the continuous case is $\mathbf{x}^* = [-\sqrt{2}, -\sqrt{2}]$ with a value of $y = -2\sqrt{2} \approx -2.828$. If x_1 and x_2 are constrained to be integer-valued, then the best we can do is to have $y = -2$ with $\mathbf{x}^* \in \{[-2, 0], [-1, -1], [0, -2]\}$.

例19.1. 问题的离散版本限制了解决方案，往往导致比连续版本更差的结果。



19.1 Integer Programs

一个 *integer program* 是带有整数约束的线性规划[2]。根据整数约束，我们指的是设计变量必须来自整数集合。[3] 整数规划有时被称为 *integer linear programs*，以强调目标函数和约束条件都是线性的这一假设。

2 见第11章。

3 整数规划是一个非常成熟的领域，应用于运筹学、通信网络、任务调度以及其他学科。现代求解器，如 Gurobi 和 CPLEX，通常能够处理具有数百万变量的问题。Julia 有一些包可以访问 Gurobi、CPLEX 以及各种其他求解器。

一个标准形式的整数规划表示为：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \\ & \mathbf{x} \in \mathbb{Z}^n \end{aligned} \tag{19.1}$$

其中 $\mathbb{Z}^n$ 是 n 维整数向量的集合。

与线性规划一样，整数规划通常以等式形式求解。将整数规划转换为等式形式往往需要添加额外的松弛变量s，而这些变量无需为整数。因此，整数规划的等式形式为：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} + \mathbf{s} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \\ & \mathbf{s} \geq \mathbf{0} \\ & \mathbf{x} \in \mathbb{Z}^n \end{aligned} \tag{19.2}$$

更一般地，一个 *mixed integer program* (算法19.1) 包含连续和离散的设计组件。这样的问题，以等式形式表示，为：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \\ & \mathbf{x}_{\mathcal{D}} \in \mathbb{Z}^{\|\mathcal{D}\|} \end{aligned} \tag{19.3}$$

其中 $\mathcal{D}$ 是一组被约束为离散的设计变量索引。这里，$\mathrm{x} = [\mathrm{x}_{\mathcal{D}}, \mathrm{x}_{\mathcal{C}}]$，其中 $\mathrm{x}_{\mathcal{D}}$ 表示离散设计变量向量，$\mathrm{x}_{\mathcal{C}}$ 表示连续设计变量向量。

```
mutable struct MixedIntegerProgram
    A
    b
    c
    D
end
```

算法19.1. 一种反映等式（19.3）的混合整数线性规划类型。在此，D是被约束为离散的设计指标集。

19.2 *Rounding*

离散优化的常用方法是将设计点必须来自离散集合这一约束*relax*。这种松弛的优点在于，我们可以使用梯度下降或线性规划等技术，利用目标函数的连续性来引导搜索。找到连续解后，将设计变量*rounded*到最近的可行离散设计。

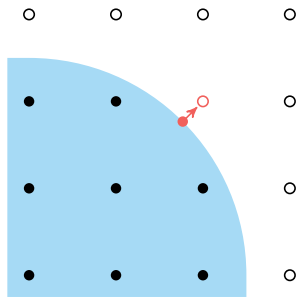


图19.1. 舍入可能导致不可行的设计点。

舍入可能存在潜在问题。如图19.1所示，舍入可能导致不可行的设计点。即使舍入后得到的点可行，也可能远离最优解，如图19.2所示。引入离散约束通常会使目标值变差，如例19.1所示。然而，对于某些问题，我们可以证明松弛解接近最优离散解。

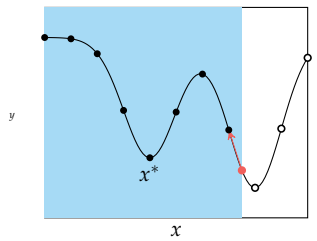


图19.2。最近的可行离散设计可能比最佳可行离散设计差得多。

我们可以通过舍入来求解整数规划，方法是去掉整数约束，求解相应的线性规划，即 LP，然后将解舍入到最接近的整数。该方法在算法19.2中实现。

```
relax(MIP) = LinearProgram(MIP.A, MIP.b, MIP.c)
function round_ip(MIP)
    x = minimize_lp(relax(MIP))
    for i in MIP.D
        x[i] = round(Int, x[i])
    end
    return x
end
```

算法19.2. 将混合整数线性规划松弛为线性规划并通过舍入求解混合整数线性规划的方法。这两种方法都接受一个混合整数线性规划MIP。通过舍入获得的解可能次优或不可行。

我们可以证明，当A是整数时，对约束$Ax \leq b$的连续解进行舍入，其结果与最优整数解永远不会相差太远。[4] 如果x_c^*是
LP 的最优解，其中 $m \times n$ 矩阵 A，则存在一个最优

4 W. Cook, A. M. Gerards, A. Schrijver, and É. Tardos, “整数线性规划中的灵敏度定理”, *Mathematical Programming*, 第34卷, 第3期, 第251–264页, 1986年。

离散解 x_d^*，其中 $\left\|\mathrm{x}_c^* - \mathrm{x}_d^*\right\|_\infty$ 小于或等于 n 乘以 A 的子矩阵行列式的最大绝对值。

向量 c 不必为整数，一个线性规划仍可能有整数最优解，因为可行域完全由 A 和 b 决定。有些方法使用线性规划的对偶形式，其可行域依赖于 c，在这种情况下，也需要 c 为整数。

在*totally unimodular*整数规划的特殊情况下，其中A、b和c的所有元素均为整数，且A是全幺模矩阵时，单纯形算法保证返回整数解。如果每个*submatrix*[5]的行列式为0、1或−1，则矩阵是全幺模的，而全幺模矩阵的逆是

也是整数。实际上，全幺模整数规划的每个顶点解都是整数。因此，对于幺模矩阵A和整数b，每个$\mathrm{Ax} = \mathrm{b}$都有整数解。

例19.2中讨论了几种矩阵及其全幺模性。算法19.3给出了判断一个矩阵或整数线性规划是否为全幺模的方法。

5 子矩阵是通过删除另一个矩阵的行和/或列而得到的矩阵。

考虑以下矩阵：

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & -1 \end{bmatrix} \quad \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \quad \begin{bmatrix} -1 & -1 & 0 & 0 & 0 \\ 1 & 0 & -1 & -1 & 0 \\ 0 & 1 & 1 & 0 & -1 \end{bmatrix}$$

左边的矩阵不是全幺模的，因为

$$\begin{vmatrix} 1 & 1 \\ 1 & -1 \end{vmatrix} = -2$$

另外两个矩阵是全幺模的。

例19.2. 全幺模矩阵的示例。

19.3 *Cutting Planes*

cutting plane method 是一种精确方法，用于在 A 不是全幺模时求解混合整数规划。[6] 现代实用的整数求解方法

程序使用*branch and cut*算法[7]，结合割平面法

6 R. E. Gomory，“线性规划整数解的算法”，*Recent Advances in Mathematical Programming*，第64卷，第289-302页，1963年。

7 M. Padberg和G. Rinaldi，“一种用于解决大规模对称旅行商问题的分支切割算法”，*SIAM Review*，第33卷，第1期，第60-100页，1991年。

```
isint(x, ϵ=1e-10) = abs(round(x) - x) ≤ ϵ
function is_totally_unimodular(A::Matrix)
    # all entries must be in [0,1,-1]
    if any(a ∉ (0,-1,1) for a in A)
        return false
    end
    # brute force check every subdeterminant
    r,c = size(A)
    for i in 1 : min(r,c)
        for a in subsets(1:r, i)
            for b in subsets(1:c, i)
                B = A[a,b]
                if det(B) ∉ (0,-1,1)
                    return false
                end
            end
        end
    end
    return true
end
function is_totally_unimodular(MIP)
    return is_totally_unimodular(MIP.A) &&
           all(isint, MIP.b) && all(isint, MIP.c)
end
```

算法19.3. 确定矩阵A或混合整数规划MIP是否全幺模的方法。若给定值为整数，则方法isint返回真。

使用分支定界法，将在下一节中讨论。割平面法的工作原理是求解松弛后的线性规划，然后添加线性约束以得到最优解。

我们以松弛LP的一个解x_c^*开始切割方法，该解必须是Ax = b的一个顶点。如果x_c^*中的$\mathcal{D}$个分量是整数，那么它也是原始混合整数规划的最优解，我们就完成了。只要x_c^*中的$\mathcal{D}$个分量不是整数，我们就找到一个超平面，使x_c^*在一侧，而所有可行的离散解在另一侧。这个*cutting plane*是一个额外的线性约束，用于排除x_c^*。然后对增广LP求解，得到新的x_c^*。

算法19.4的每次迭代都会引入切割平面，使得x_c^*的非整数分量变得不可行，同时保持最近整数解及可行集其余部分的可行性。针对这些切割平面约束修改后的整数规划问题被求解以获取新的松弛解。图19.3展示了这一过程。

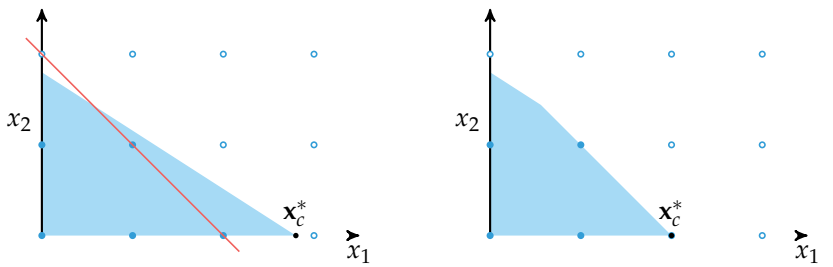


图19.3. 割平面法通过不断引入约束条件，直至线性规划的解为整数。左侧以红线表示割平面。增广线性规划的可行域见右侧。

我们希望添加约束，以剔除 x_c^* 的非整数分量。对于等式形式的线性规划，其约束为 Ax = b，回想第11.2.1节，我们可以对顶点解 x_c^* 进行划分，以得到

$$\mathbf{A}_{\mathcal{B}}\mathbf{x}_{\mathcal{B}}^* + \mathbf{A}_{\mathcal{V}}\mathbf{x}_{\mathcal{V}}^* = \mathbf{b} \tag{19.4}$$

其中 $\mathrm{x}_{\mathcal{V}}^* = 0$。因此，$\mathrm{x}_c^*$ 的非整数分量只会出现在 $\mathrm{x}_{\mathcal{B}}^*$ 中。

我们可以为每个 $b \in \mathcal{B}$ 引入一个额外的整数约束，使得 x_b^* 非整数：[8]

$$x_b^* - \lfloor x_b^* \rfloor - \sum_{v \in \mathcal{V}} \left(\bar{A}_{bv} - \lfloor \bar{A}_{bv} \rfloor \right) x_v \leq 0 \tag{19.5}$$

8 注意，$\lfloor x \rfloor$ 或 x 的 *floor* 会将 x 向下舍入到最接近的整数。

```
frac(x) = modf(x)[1]
function cutting_plane(MIP)
    LP = relax(MIP)
    x, b_inds, v_inds = minimize_lp(LP)
    n_orig = length(x)
    D = copy(MIP.D)
    while !all(isint(x[i]) for i in D)
        AB, AV = LP.A[:,b_inds], LP.A[:,v_inds]
        Abar = AB\AV
        b = 0
        for i in D
            if !isint(x[i])
                b += 1
                A2 = [LP.A zeros(size(LP.A,1));
                      zeros(1,size(LP.A,2)+1)]
                A2[end,end] = 1
                A2[end,v_inds] = (x->floor(x) - x).(Abar[b,:])
                b2 = vcat(LP.b, -frac(x[i]))
                c2 = vcat(LP.c, 0)
                LP = LinearProgram(A2,b2,c2)
            end
        end
        x, b_inds, v_inds = minimize_lp(LP)
    end
    return x[1:n_orig]
end
```

算法 19.4. 切割平面法求解给定的混合整数规划 `MIP`，并返回最优设计向量。若不存在可行解，则抛出错误。辅助函数 `frac` 返回一个数的小数部分，而 `minimize_lp` 的实现（算法 11.5）已被调整，以返回基指标 `b_inds` 和非基指标 `v_inds` 以及最优设计 `x`。

其中 $\bar{\mathrm{A}} = \mathrm{A}_{\mathcal{B}}^{-1}\,\mathrm{A}_{\mathcal{V}}$。这些切割平面仅使用 $\mathcal{V}$ 分量来切除非整数分量 x_c^*。

引入一个割平面约束会切掉松弛解 x_c^*，因为所有 x_v 都为零：

$$\underbrace{x_b^* - \lfloor x_b^* \rfloor}_{>0} - \underbrace{\sum_{v \in \mathcal{V}} \left(\bar{A}_{bv} - \lfloor \bar{A}_{bv} \rfloor\right) x_v}_{0} > 0 \tag{19.6}$$

使用额外的整数松弛变量 x_k 将割平面写为等式形式：

$$x_k + \sum_{v \in \mathcal{V}} \left(\lfloor \bar{A}_{bv} \rfloor - \bar{A}_{bv}\right) x_v = \lfloor x_b^* \rfloor - x_b^* \tag{19.7}$$

算法19.4的每次迭代都会增加约束数量和变量数量，直至求解线性规划产生整数解。仅返回对应于原始设计变量的分量。

割平面法用于求解示例19.3中的一个简单整数线性规划。

19.4 Branch and Bound

寻找离散问题（例如整数规划）全局最优解的一种方法是枚举所有可能的解。*branch and bound*[9]该方法保证能够找到最优解，而无需评估所有可能的解。许多商业整数规划求解器结合了割平面法和分支定界法的思想。该方法得名于划分解空间的*branch operation*[10]以及*bound operation*

计算分区的一个下界。

分支定界是一种通用方法，可应用于多种离散优化问题，但这里我们将重点关注如何将其用于整数规划。算法19.5提供了一种实现，该实现使用*priority queue*，这是一种将优先级与集合中元素相关联的数据结构。我们可以使用`enqueue!`操作将一个元素及其优先级值添加到优先队列中。我们可以使用`dequeue!`操作移除具有最小优先级值的元素。

9 A. H. Land 和 A. G. Doig，“一种自动解决离散规划问题的方法”，*Econometrica*，第 28 卷，第 3 期，第 497–520 页，1960 年。

10 这些子集通常是不相交的，但这并非必需。为使分支定界法有效，至少有一个子集必须拥有最优解。D. A. Bader, W. E. Hart, and C. A. Phillips，‘‘Parallel Algorithm Design for Branch and Bound,’’ in *Tutorials on Emerging Methodologies and Applications in Operations Research*, H. J. Greenberg, ed., Kluwer Academic Press, 2004.

示例 19.3. 用于求解整数规划的割平面法。

考虑整数规划：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & 2x_1 + x_2 + 3x_3 \\ \text{subject to} \quad & \begin{bmatrix} 0.5 & -0.5 & 1.0 \\ 2.0 & 0.5 & -1.5 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 2.5 \\ -1.5 \end{bmatrix} \\ & \mathbf{x} \geq \mathbf{0} \qquad \mathbf{x} \in \mathbb{Z}^3 \end{aligned}$$

松弛解为 $\mathrm{x}^* \approx [0.818, 0, 2.091]$，得到：

$$\mathbf{A}_{\mathcal{B}} = \begin{bmatrix} 0.5 & 1 \\ 2 & -1.5 \end{bmatrix} \quad \mathbf{A}_{\mathcal{V}} = \begin{bmatrix} -0.5 \\ 0.5 \end{bmatrix} \quad \bar{\mathbf{A}} = \begin{bmatrix} -0.091 \\ -0.455 \end{bmatrix}$$

根据公式（19.7），带有松弛变量 x_4 的 x_1 约束为：

$$\begin{aligned} x_4 + (-0.091 - \lfloor -0.091 \rfloor)x_2 &= \lfloor 0.818 \rfloor - 0.818 \\ x_4 - 0.909x_2 &= -0.818 \end{aligned}$$

带松弛变量 x_5 的 x_3 的约束条件为：

$$\begin{aligned} x_5 + (-0.455 - \lfloor -0.455 \rfloor)x_2 &= \lfloor 2.091 \rfloor - 2.091 \\ x_5 - 0.545x_2 &= -0.091 \end{aligned}$$

修改后的整数规划具有：

$$\mathbf{A} = \begin{bmatrix} 0.5 & -0.5 & 1 & 0 & 0 \\ 2 & 0.5 & -1.5 & 0 & 0 \\ 0 & -0.909 & 0 & 1 & 0 \\ 0 & -0.545 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} 2.5 \\ -1.5 \\ -0.818 \\ -0.091 \end{bmatrix} \quad \mathbf{c} = \begin{bmatrix} 2 \\ 1 \\ 3 \\ 0 \\ 0 \end{bmatrix}$$

求解修正后的线性规划问题，得到 $\mathrm{x}^* \approx [0.9, 0.9, 2.5, 0.0, 0.4]$。由于该点非整数，我们通过添加约束重复此过程：

$$\begin{aligned} x_6 - 0.9x_4 &= -0.9 & \qquad x_7 - 0.9x_4 &= -0.9 \\ x_8 - 0.5x_4 &= -0.5 & \qquad x_9 - 0.4x_4 &= -0.4 \end{aligned}$$

并求解第三个线性规划问题得到：$\mathrm{x}^* = [1, 2, 3, 1, 1, 0, 0, 0, 0]$，最终解为 $\mathrm{x}_i^* = [1, 2, 3]$。

该算法从一个包含原始混合整数规划单个LP松弛的优先队列开始。与该LP关联的是一个解$\mathbf{x}_c^*$和目标值$y_c = \mathbf{c}^\top \mathbf{x}_c^*$。目标值作为解的下界，因此充当优先队列中LP的优先级。在算法的每次迭代中，我们检查优先队列是否为空。如果不为空，我们出队具有最低优先级值的LP。如果与该元素关联的解具有必要的整数分量，那么我们记录它是否是迄今为止找到的最佳整数解。

如果出队的解在$\mathcal{D}$中有一个或多个非整数分量，我们从$\mathbf{x}_c^*$中选择一个距离整数值最远的分量。假设该分量对应于第i个设计变量。我们通过考虑两个新的线性规划*branch*，每个都是通过向出队的线性规划添加以下约束之一来创建的：[11]

$$x_i \leq \lfloor x_{i,c}^* \rfloor \qquad \text{or} \qquad x_i \geq \lceil x_{i,c}^* \rceil \tag{19.8}$$

11 注意，$\lceil x \rceil$ 或 x 的 *ceiling* 会将 x 四舍五入到最接近的整数。

如图19.4所示。示例19.4演示了此过程。

我们计算与这两个线性规划相关的解，它们为原始混合整数规划的价值提供了下界。如果任一解与目前所见到的最佳整数解相比降低了目标值，则将其放入优先队列。不放置那些已知劣于当前最佳整数解的解决方案，使分支定界能够修剪搜索空间。该过程持续到优先队列为空，然后我们返回最佳可行整数解。例19.5展示了如何将分支定界应用于一个小型整数规划。

考虑整数规划的一个松弛解 $\mathbf{x}_c^* = [3, 2.4, 1.2, 5.8]$，其中 $\mathbf{c} = [-1, -2, -3, -4]$。下界为

$$y \geq \mathbf{c}^\top \mathbf{x}_c^* = -34.6$$

我们对 $\mathbf{x}_c^*$ 的一个非整数坐标进行分支，通常选择距离整数值最远的那个。在这种情况下，我们选择第一个非整数坐标 $x_{2,c}^*$，它与最近整数值的距离为0.4。接着我们考虑两个新的线性规划问题，一个添加 $x_2 \leq 2$ 作为额外约束，另一个添加 $x_2 \geq 3$ 作为额外约束。

例19.4. 分支定界法中分支步骤单一应用的一个例子。

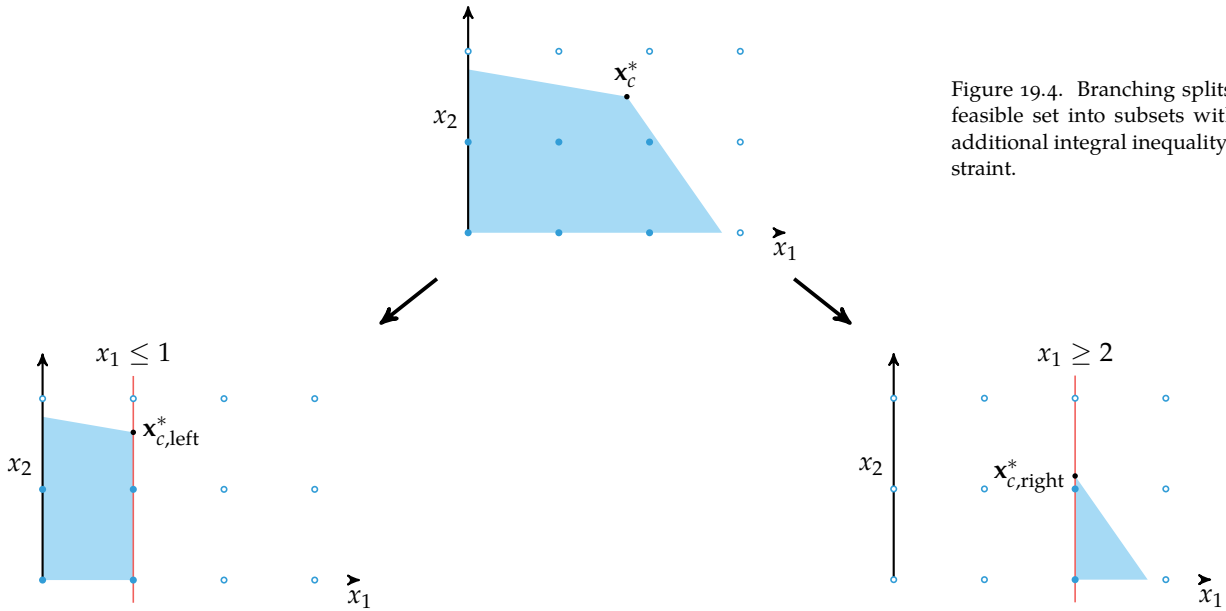


Figure 19.4. Branching splits the feasible set into subsets with an additional integral inequality constraint.

```
function minimize_lp_and_y(LP)
    try
        x = minimize_lp(LP)
        return (x, x⋅LP.c)
    catch
        return (fill(NaN, length(LP.c)), Inf)
    end
end
function branch_and_bound(MIP)
    LP = relax(MIP)
    x, y = minimize_lp_and_y(LP)
    n = length(x)
    x_best, y_best, Q = deepcopy(x), Inf, PriorityQueue()
    enqueue!(Q, (LP,x,y), y)
    while !isempty(Q)
        LP, x, y = dequeue!(Q)
        if any(isnan.(x)) || all(isint(x[i]) for i in MIP.D)
            if y < y_best
                x_best, y_best = x[1:n], y
            end
        else
            i = argmax([abs(x[i] - round(x[i])) for i in MIP.D])
            # x_i ≤ floor(x_i)
            A, b, c = LP.A, LP.b, LP.c
            A2 = [A zeros(size(A,1));
                  [j==i for j in 1:size(A,2)]' 1]
            b2, c2 = vcat(b, floor(x[i])), vcat(c, 0)
            LP2 = LinearProgram(A2,b2,c2)
            x2, y2 = minimize_lp_and_y(LP2)
            if y2 ≤ y_best
                enqueue!(Q, (LP2,x2,y2), y2)
            end
            # x_i ≥ ceil(x_i)
            A2 = [A zeros(size(A,1));
                  [j==i for j in 1:size(A,2)]' -1]
            b2, c2 = vcat(b, ceil(x[i])), vcat(c, 0)
            LP2 = LinearProgram(A2,b2,c2)
            x2, y2 = minimize_lp_and_y(LP2)
            if y2 ≤ y_best
                enqueue!(Q, (LP2,x2,y2), y2)
            end
        end
    end
    return x_best
end
```

算法19.5. 解决混合整数规划问题MIP的分支定界算法。辅助方法minimize_lp_and_y求解线性规划并返回解及其值。不可行的线性规划产生一个**NaN**解和一个**Inf**值。更复杂的实现会为了加速计算而舍弃已知解的变量。PriorityQueue类型由DataStructures.jl包提供。

我们可以使用分支定界法来求解例19.3中的整数规划问题。如前所述，松弛解为 $\mathbf{x}_c^* = [0.818, 0, 2.09]$，其值为7.909。我们在第一个分量上进行分支，得到两个整数规划问题，一个带有$x_1 \leq 0$，另一个带有$x_1 \geq 1$：

$$\mathbf{A}_{\text{left}} = \begin{bmatrix} 0.5 & -0.5 & 1 & 0 \\ 2 & 0.5 & -1.5 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{b}_{\text{left}} = \begin{bmatrix} 2.5 \\ -1.5 \\ 0 \end{bmatrix} \quad \mathbf{c}_{\text{left}} = \begin{bmatrix} 2 \\ 1 \\ 3 \\ 0 \end{bmatrix}$$

$$\mathbf{A}_{\text{right}} = \begin{bmatrix} 0.5 & -0.5 & 1 & 0 \\ 2 & 0.5 & -1.5 & 0 \\ 1 & 0 & 0 & -1 \end{bmatrix} \quad \mathbf{b}_{\text{right}} = \begin{bmatrix} 2.5 \\ -1.5 \\ 1 \end{bmatrix} \quad \mathbf{c}_{\text{right}} = \begin{bmatrix} 2 \\ 1 \\ 3 \\ 0 \end{bmatrix}$$

$x_1 \leq$ 为 0 的左 LP 不可行。$x_1 \geq$ 为 1 的右 LP 有一个松弛解，$\mathbf{x}_c^* = [1, 2, 3, 0]$，值为 13。因此，我们得到了整数解，$\mathbf{x}_i^* = [1, 2, 3]$。

例19.5. 应用分支定界法求解整数规划问题。

19.5 Dynamic Programming

Dynamic programming[12] 是一种可应用于 *optimal* 问题的技术 *substructure* 和 *overlapping subproblems*。如果一个问题的最优解可以由其子问题的最优解构造出来，那么该问题具有最优子结构。图19.5展示了一个例子。

12 动态规划这一术语由理查德·贝尔曼选定，旨在反映他所应用问题中的时变特性，并避免如*research*和*mathematics*这类词汇有时带有的负面含义。他写道：“我认为动态规划是个好名字。它甚至不会引起国会议员的反对。因此，我将它作为我各项活动的总称。” R. 贝尔曼 *Eye of the Hurricane: An Autobiography*。世界科学出版社，1984年，第159页。

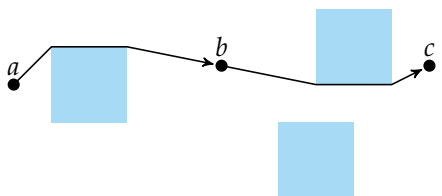


图19.5. 最短路径问题具有最优子结构，因为如果从任意a到c的最短路径经过b，那么子路径$a \to b$和$b \to c$也都是最短路径。

具有重叠子问题的问题如果采用递归方式求解，将会多次遇到相同的子问题。动态规划不是枚举指数级数量的潜在解，而是存储子问题的解，从而避免重新计算它们，或者递归地构造最优解。

单次遍历中的计算。递推关系问题常常具有重叠子问题。图19.6展示了一个例子。

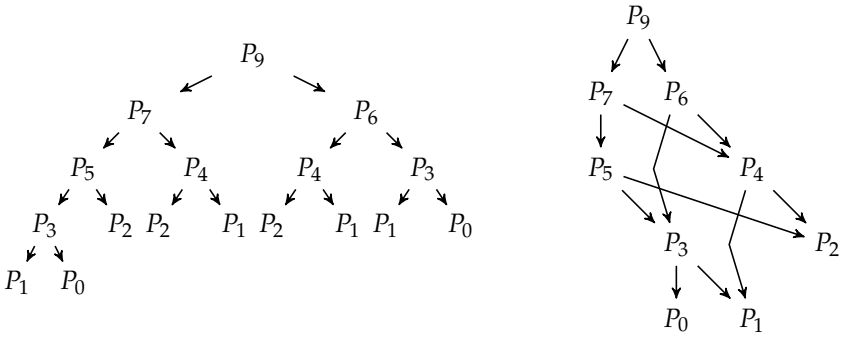


图19.6。我们可以通过递归遍历所有子项来计算第n项，即*Padovan*、*sequence* $P_n = P_{n-2} + P_{n-3}$，其中$P_0 = P_1 = P_2 =$为1（左）。更高效的方法是利用问题的重叠子结构，计算一次子项并在后续计算中重用它们的值（右）。

动态规划可以采用自顶向下或自底向上的方式实现，如算法19.6所示。自顶向下的方法从所需问题开始，递归到越来越小的子问题。子问题的解会被存储起来，这样当我们遇到一个新的子问题时，我们可以检索已计算的解，或者求解并存储以备将来使用。[13] 自底

自底向上的方法首先解决较小的子问题，并利用它们的解来获得更大问题的解。

13 以这种方式存储子问题的解称为*memoization*。

```
function padovan_topdown(n, P=Dict())
    if !haskey(P, n)
        P[n] = n < 3 ? 1 :
               padovan_topdown(n-2,P) + padovan_topdown(n-3,P)
    end
    return P[n]
end
function padovan_bottomup(n)
    P = Dict(0=>1,1=>1,2=>1)
    for i in 3 : n
        P[i] = P[i-2] + P[i-3]
    end
    return P[n]
end
```

算法19.6. 使用动态规划计算帕多瓦数列，采用自顶向下和自底向上两种方法。

*knapsack problem*是一个著名的组合优化问题，常出现在资源分配中。[14]假设我们正在为以下情况打包我们的背包：
旅行，但我们的空间有限，想要打包最有价值的物品。背包问题有几种变体。在0-1背包问题中，我们有以下优化问题：

14 背包问题是一个具有单一约束的整数规划问题，但可以通过动态规划高效求解。

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & -\sum_{i=1}^{n} v_i x_i \\ \text{subject to} \quad & \sum_{i=1}^{n} w_i x_i \leq w_{\max} \\ & x_i \in \{0,1\} \text{ for all } i \text{ in } \{1,\ldots,n\} \end{aligned} \tag{19.9}$$

我们有 n 个物品，其中第 i 个物品具有整数重量 $w_i > 0$ 和价值 v_i。设计向量 x 由二进制值组成，表示物品是否被装入。总重量不能超过我们的整数容量 $w_{\max}$，我们的目标是最大化装入物品的总价值。

有 2^n 种可能的设计向量，这使得对于较大的 n 直接枚举变得不可行。然而，我们可以使用动态规划。0-1 背包问题具有最优子结构和重叠子问题。考虑已经解决了包含 n 个物品以及若干容量直至 $w_{\max}$ 的背包问题。一个更大背包问题若增加一个重量为 w_{n+1}、容量为 $w_{\max}$ 的新物品，其解决方案要么包含该新物品，要么不包含：

- 如果新物品不值得包含，则解将具有与具有$n-1$个物品和容量$w_{\max}$的背包相同的值。

- 如果值得包含新物品，则解将具有一个背包的价值，该背包有$n-$个物品且容量为$w_{\max} - w_{n+1}$，再加上新物品的价值。

递推关系是：

$$\text{knapsack}(i, w_{\max}) = \begin{cases} 0 & \text{if } i = 0 \\ \text{knapsack}(i-1, w_{\max}) & \text{if } w_i > w_{\max} \\ \max \begin{cases} \text{knapsack}(i-1, w_{\max}) & \text{(discard new item)} \\ \text{knapsack}(i-1, w_{\max} - w_i) + v_i & \text{(include new item)} \end{cases} & \text{otherwise} \end{cases} \tag{19.10}$$

0-1背包问题可以使用算法19.7中的实现来解决。

```
function knapsack(v, w, w_max)
    n = length(v)
    y = Dict((0,j) => 0.0 for j in 0:w_max)
    for i in 1 : n
        for j in 0 : w_max
            y[i,j] = w[i] > j ? y[i-1,j] :
                     max(y[i-1,j],
                         y[i-1,j-w[i]] + v[i])
        end
    end

    # recover solution
    x, j = falses(n), w_max
    for i in n: -1 : 1
        if w[i] ≤ j && y[i,j] - y[i-1, j-w[i]] == v[i]
            # the ith element is in the knapsack
            x[i] = true
            j -= w[i]
        end
    end
    return x
end
```

算法19.7. 解决0-1背包问题的一种方法，其中物品价值为`v`，物品重量为整数`w`，背包容量为整数`w_max`。从缓存解中恢复设计向量需要额外的迭代步骤。

19.6 Ant Colony Optimization

Ant colony optimization[15] 是一种用于优化路径的随机方法，通过图。这种方法受到一些蚂蚁物种的启发，它们随机游走寻找食物，沿途留下*pheromone*轨迹。偶然发现信息素轨迹的其他蚂蚁很可能会开始跟随它，从而加强轨迹的气味。信息素会随时间缓慢蒸发，导致未使用的轨迹逐渐消失。较短的路径，由于信息素更强，被更频繁地走过，因此吸引更多蚂蚁。这样，短路径产生了正反馈，引导其他蚂蚁跟随并进一步强化这条更短的路径。

15 M. Dorigo、V. Maniezzo 和 A. Colorni，“蚁群系统：通过合作代理群体进行优化”，*IEEE Transactions on Systems, Man, and Cybernetics-Part B*，第 26 卷，第 1 期，第 29–41 页，1996 年（*IEEE Transactions on Sys-tems, Man, and Cybernetics*）。

基本最短路径问题，例如蚂蚁在蚁丘和食物源之间找到的最短路径，可以使用动态规划高效解决。

编程。蚁群优化已被用于寻找*traveling salesman problem*的近似最优解，这是一个更为困难的问题，我们希望找到恰好经过每个节点一次的最短路径。蚁群优化还被用于多车辆路径规划、寻找工厂的最佳位置以及蛋白质折叠。[16]该算法本质上是随机的

因此，它对问题随时间的变化具有抵抗性，例如交通延误导致图中有效边长变化，或网络问题导致边完全消失。

蚂蚁根据可用边的吸引力随机移动。转移 $i \to j$ 的吸引力取决于信息素水平和可选先验因子：

$$A(i \to j) = \tau(i \to j)^{\alpha} \eta(i \to j)^{\beta} \tag{19.11}$$

其中 α 和 β 分别是信息素水平 τ 和先验因子 η 的指数。[17] 对于涉及最短路径的问题，我们可以设置先验因子

到逆边长 $\ell(i \to j)$，以鼓励遍历更短的路径：$\eta(i \to j) = 1/\ell(i \to j)$。计算边吸引力的方法在算法19.8中给出。

假设一只蚂蚁位于节点 i，并且可以转移到任意节点 $j \in \mathcal{J}$。后继节点集合 $\mathcal{J}$ 包含所有有效的出邻居。[18] 有时边

被排除，如在旅行商问题中，蚂蚁被阻止两次访问同一节点。因此，$\mathcal{J}$ 既依赖于 i，也依赖于蚂蚁的历史。

16 这些及其他应用在以下参考文献中进行了讨论：M. Manfrin，“车辆路径问题的蚁群优化”，博士论文，布鲁塞尔自由大学，2004年。T. Stützle，“二次分配问题的最大最小蚁群系统”，达姆施塔特工业大学，技术报告，1997年。以及 A. Shmygelska、R. Aguirre-Hernández 和 H. H. Hoos，“二维HP蛋白质折叠问题的蚁群算法”，载于 *International Workshop on Ant Algorithms* (*ANTS*)，2002年。

17 Dorigo、Maniezzo 和 Colorni 推荐 $\alpha = 1$ 和 $\beta = 5$。

18 节点 i 的 *outgoing neighbors* 是所有满足 $i \to j$ 在图中的节点 j。在无向图中，邻居和出邻居是相同的。

```
function edge_attractiveness(graph, τ, η; α=1, β=5)
    A = Dict()
    for i in 1 : nv(graph)
        neighbors = outneighbors(graph, i)
        for j in neighbors
            v = τ[(i,j)]^α * η[(i,j)]^β
            A[(i,j)] = v
        end
    end
    return A
end
```

算法19.8. 给定图`graph`、信息素水平`τ`、先验边权重`η`、信息素指数`α`和先验指数`β`，计算边吸引力表的方法。

边转移$i \to j$的概率是：

$$P(i \to j) = \frac{A(i \to j)}{\sum_{j' \in \mathcal{J}} A(i \to j')} \tag{19.12}$$

蚂蚁通过分泌信息素影响后代蚂蚁。有几种建模信息素沉积的方法。一种常见的方法是在完成一条完整路径后建立信息素沉积。[19] 未找到

一条路径不会沉积信息素。对于最短路径问题，一只成功建立长度为ℓ路径的蚂蚁在其经过的每条边上沉积$1/\ell$的信息素。 *Life*

19 M. Dorigo、G. Di Caro和L. M. Gambardella，“离散优化的蚁群算法”，*Artificial*，第5卷，第2期，第137-172页，1999年。

```
import StatsBase: Weights, sample
function run_ant(G, lengths, τ, A, x_best, y_best)
    x = [1]
    while length(x) < nv(G)
        i = x[end]
        neighbors = setdiff(outneighbors(G, i), x)
        if isempty(neighbors) # ant got stuck
            return (x_best, y_best)
        end

        as = [A[(i,j)] for j in neighbors]
        push!(x, neighbors[sample(Weights(as))])
    end

    l = sum(lengths[(x[i-1],x[i])] for i in 2:length(x))
    for i in 2 : length(x)
        τ[(x[i-1],x[i])] += 1/l
    end
    if l < y_best
        return (x, l)
    else
        return (x_best, y_best)
    end
end
```

算法19.9。一种在旅行商问题中模拟单只蚂蚁的方法，其中蚂蚁从第一个节点出发，尝试恰好访问每个节点一次。成功完成一次巡回后，信息素水平会增加。参数包括图`G`、边长度`lengths`、信息素水平`τ`、边吸引力`A`、迄今为止找到的最佳解`x_best`及其值`y_best`。

蚁群优化还模拟了信息素蒸发，这在现实世界中自然发生。模拟蒸发有助于防止算法过早收敛到单一且可能是次优的解。信息素蒸发在所有蚂蚁模拟完成后的每次迭代结束时执行。

已完成。蒸发将每条路径上的信息素水平乘以$1-\rho$进行减少，其中$\rho \in [0, 1]$。[20]

[20] 通常使用 $\rho = 1/2$。

F或在迭代k时，m只蚂蚁的有效信息素更新 是

$$\tau(i \to j)^{(k+1)} = (1-\rho)\tau(i \to j)^{(k)} + \sum_{a=1}^{m} \frac{1}{\ell^{(a)}} \left((i \to j) \in \mathcal{P}^{(a)} \right) \tag{19.13}$$

其中 $\ell^{(a)}$ 是路径长度，$\mathcal{P}^{(a)}$ 是蚂蚁 a 经过的边的集合。

蚁群优化在算法19.10中实现，其中单个蚂蚁的模拟使用算法19.9。图19.7展示了用于解决旅行商问题的蚁群优化过程。

```
function ant_colony_optimization(G, lengths;
    m = 1000, k_max=100, α=1.0, β=5.0, ρ=0.5,
    η = Dict((e.src,e.dst)=>1/lengths[(e.src,e.dst)]
             for e in edges(G)))
    τ = Dict((e.src,e.dst)=>1.0 for e in edges(G))
    x_best, y_best = [], Inf
    for k in 1 : k_max
        A = edge_attractiveness(G, τ, η, α=α, β=β)
        for (e,v) in τ
            τ[e] = (1-ρ)*v
        end
        for ant in 1 : m
            x_best,y_best = run_ant(G,lengths,τ,A,x_best,y_best)
        end
    end
    return x_best
end
```

算法19.10。蚁群优化，该算法接收来自`LightGraphs.jl`的有向或无向图`G`以及边元组到路径长度的字典`lengths`。蚂蚁从图中的第一个节点开始。可选参数包括每次迭代的蚂蚁数量`m`、迭代次数`k_max`、信息素指数`α`、先验指数`β`、蒸发标量`ρ`以及先验边权重的字典`η`。

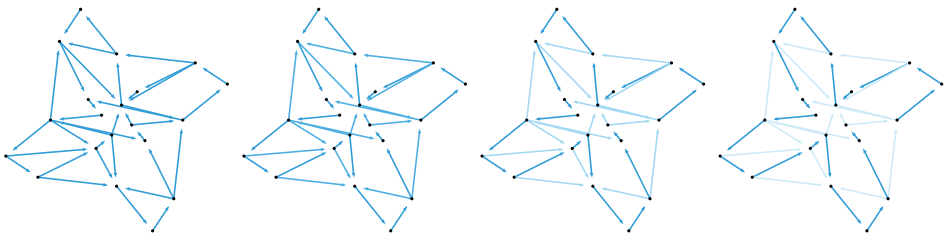


图19.7. 使用50只蚂蚁迭代解决有向图上的旅行商问题的蚁群优化。路径长度为欧几里得距离。颜色不透明度对应信息素水平。

19.7 Summary

- 离散优化问题要求设计变量从离散集合中选择。
- 松弛化，即求解离散问题的连续版本，其本身是一种寻找最优离散解的不可靠技术，但对于更复杂的算法却是核心所在。
- 许多组合优化问题可以被构建为一个整数规划，即带有整数约束的线性规划。
- 割平面法和分支定界法都可以用来高效且精确地求解整数规划问题。分支定界法非常通用，可以应用于各种各样的离散优化问题。
- 动态规划是一种强大的技术，它利用了一些问题中的最优重叠子结构。
- 蚁群优化是一种受自然启发的算法，可用于优化图中的路径。

19.8 Exercises

练习 19.1. 一个 *Boolean satisfiability problem*，通常缩写为 SAT，要求确定是否存在一个布尔设计，使得一个布尔值目标函数输出 **true**。SAT 问题是最早被证明属于 NP 完全问题这一困难类别的问题。[21] 这意味着 SAT 至少与

与其他所有解可在多项式时间内验证的问题一样困难。

考虑布尔目标函数：

$$f(\mathbf{x}) = x_1 \wedge (x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2)$$

使用枚举法寻找最优设计。对于n维设计向量，在最坏情况下必须考虑多少个设计？

练习19.2。将练习19.1中的问题形式化为整数线性规划。能否将任何布尔可满足性问题都形式化为整数线性规划？

21 S. Cook，《定理证明过程的复杂性》，载于 *ACM Symposium on Theory of Computing*，1971年。

练习19.3。为什么我们对全幺模矩阵感兴趣？此外，为什么每个全幺模矩阵只包含0、1或–1这样的元素？

练习 19.4。本章使用动态规划求解了 0-1 背包问题。说明如何将分支限界法应用于 0-1 背包问题，并使用你的方法求解具有价值 v = [9, 4, 2, 3, 5, 3] 和重量 w = [7, 8, 4, 5, 9, 4]、容量 $w_{\max} = 20$ 的背包问题。

20 Expression Optimization

前面的章节讨论了在一组固定设计变量上的优化问题。对于许多问题而言，变量数量是未知的，比如图形结构或计算机程序的优化。在这些情境下，设计可以通过属于某一语法体系的表达式来表示。本章探讨如何通过考虑设计空间的语法结构，使最优设计的搜索更为高效。

20.1 Grammars

表达式可以用一棵*symbols*树来表示。例如，数学表达式$x + \ln 2$可以使用图20.1中的树来表示，该树由符号$+$、x、ln和2组成。*Grammars*指定了可能表达式空间上的约束。

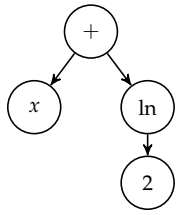


图20.1. 表达式$x + \ln 2$ 以树状表示。

文法由一组 *production rules* 表示。这些规则涉及符号以及 *types*。一个类型可以被解释为一组表达式树。一条产生式规则表示类型可能扩展为涉及符号或表达式的表达式。如果一条规则仅扩展为符号，则称为 *terminal*，因为它无法进一步扩展。非终结规则的一个例子是 $\mathbb{R} \mapsto \mathbb{R} + \mathbb{R}$，这意味着类型 $\mathbb{R}$ 可以由集合 $\mathbb{R}$ 中的元素加上集合 $\mathbb{R}$ 中的元素组成。[1]

1 本章重点讨论 *context-free grammars*，但还存在其他形式。参见 L. Kallmeyer, *Parsing Beyond*。Springer, 2010。

我们可以通过从 *start type* 开始，然后递归地应用不同的产生式规则，从一个文法生成一个表达式。当树中只包含符号时，我们就停止。图20.2展示了表达式$x + \ln 2$ 的这一过程。示例20.1展示了其在自然语言表达式中的应用。

Context-Free Grammars

语法允许的可能表达式数量可以是无限的。示例20.2展示了一个允许无限多个有效表达式的语法。

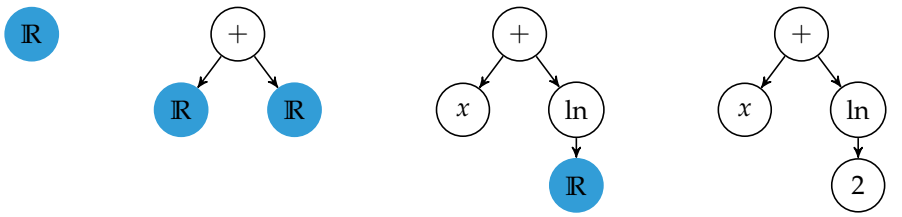


图20.2. 使用生产规则

$$\mathbb{R} \mapsto \mathbb{R} + \mathbb{R}$$
$$\mathbb{R} \mapsto x$$
$$\mathbb{R} \mapsto \ln(\mathbb{R})\ \mathbb{R} \mapsto 2$$

以生成 $x + \ln 2$。蓝色节点是未展开类型。

考虑一种允许生成简单英语陈述句的语法：

$$\mathbb{S} \mapsto \mathbb{N}\,\mathbb{V}$$
$$\mathbb{V} \mapsto \mathbb{V}\,\mathbb{A}$$
$$\mathbb{A} \mapsto \text{rapidly} \mid \text{efficiently}$$
$$\mathbb{N} \mapsto \text{Alice} \mid \text{Bob} \mid \text{Mykel} \mid \text{Tim}$$
$$\mathbb{V} \mapsto \text{runs} \mid \text{reads} \mid \text{writes}$$

类型 $\mathbb{S}$、$\mathbb{N}$、$\mathbb{V}$ 和 $\mathbb{A}$ 分别对应语句、名词、动词和副词。一个表达式是通过从类型 $\mathbb{S}$ 开始并迭代替换类型来生成的：

$$\mathbb{S}$$
$$\mathbb{N}\,\mathbb{V}$$
$$\text{Mykel}\ \mathbb{V}\ \mathbb{A}$$
$$\text{Mykel writes rapidly}$$

并非所有终端符号类别都必须使用。例如，也可以生成语句“Alice runs”。

示例20.1. 生成简单英语陈述的语法。在表达式的右侧使用|是*or*的简写。因此，规则

$$\mathbb{A} \mapsto \text{快速} \mid \text{高效}$$

等同于拥有两条规则，$\mathbb{A} \mapsto$快速和$\mathbb{A} \mapsto$高效。

表达式优化通常将表达式约束在最大深度内，或根据其深度或节点数量对表达式进行惩罚。即使语法允许有限数量的表达式，搜索空间也往往过于庞大，无法进行穷举搜索。因此，需要能够高效搜索可能表达式空间以找到优化目标函数的算法。

示例 20.2. 与语法相关的一些挑战，以四功能计算器语法为例进行说明。

考虑一个四功能计算器语法，应用加法、减法、乘法和除法于十个数字：

$$\begin{aligned}
\mathbb{R} &\mapsto \mathbb{R} + \mathbb{R} \\
\mathbb{R} &\mapsto \mathbb{R} - \mathbb{R} \\
\mathbb{R} &\mapsto \mathbb{R} \times \mathbb{R} \\
\mathbb{R} &\mapsto \mathbb{R} \,/\, \mathbb{R} \; 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \\
\mathbb{R} &\mapsto
\end{aligned}$$

可以生成无限数量的表达式，因为非终结符$\mathbb{R}$总能展开为计算器操作之一。

许多表达式会产生相同的值。加法和乘法运算符具有交换律，这意味着顺序无关紧要。例如，$a + b$ 与 $b + a$ 相同。这些运算还具有结合律，这意味着同一类型的多个运算的出现顺序无关紧要。例如，$a \times b \times c$ 与 $c \times b \times a$ 相同。其他运算保持值不变，例如加零或乘以一。

并非此语法下的所有表达式在数学上都是有效的。例如，除以零是未定义的。将零作为终结符号移除不足以防止这一错误，因为零可以通过其他运算构造出来，比如$1 - 1$。此类异常通常由目标函数处理，目标函数可以捕获异常并对其进行惩罚。

本章介绍的表达式优化例程使用 `ExprRules.jl`。可以通过列出产生式规则，使用 `grammar` 宏来定义文法，如例 20.3 所示。

许多表达式优化算法涉及以保持类型展开方式的方式操作表达式树的组件。一个`RuleNode`对象跟踪应用了哪些产生式规则，当

我们可以使用 grammar 宏来定义语法。非终结符位于等号左侧，而包含终结符和非终结符的表达式则位于右侧。该包包含一些用于更紧凑地表示语法的语法。

示例 20.3. 使用ExprRules.jl包定义语法的示例。

```
using ExprRules
grammar = @grammar begin
    R = x          # reference a variable
    R = R * A      # multiple children
    R = f(R)       # call a function
    R = _(randn()) # random variable generated on node creation
    R = 1 | 2 | 3  # equivalent to R = 1, R = 2, and R = 3
    R = |(4:6)     # equivalent to R = 4, R = 5, and R = 6
    A = 7          # rules for different return types
end;
```

进行扩展。使用指定的起始类型调用 rand 将生成一个由 RuleNode 表示的随机表达式。调用 sample 将从现有的 RuleNode 树中随机选择一个 RuleNode。节点使用 eval 进行评估。

方法return_type将节点的返回类型作为符号返回，isterminal返回该符号是否为终止符，child_types返回与该节点产生式规则相关联的非终止符符号列表，nchildren返回子节点的数量。这四个方法均以语法和节点作为输入。表达式树中的节点数通过length(node)获得，深度通过depth(node)获得。

第三种类型 NodeLoc 用于指代节点在表达式树中的位置。子树操作通常需要使用 NodeLoc。

```
loc = sample(NodeLoc, node); # uniformly sample a node loc
loc = sample(NodeLoc, node, :R, grammar); # sample a node loc of type R
subtree = get(node, loc);
```

20.2 *Genetic Programming*

遗传算法（见第9章）使用以顺序格式编码设计点的染色体。*Genetic programming*[2] 使用树来表示个体

2 J. R. Koza, *Genetic Programming:. On the Programming of Computers by Means of Natural Selection* MIT Press, 1992.

相反（图20.3），这些网络更适合表示数学函数、程序、决策树以及其他层级结构。

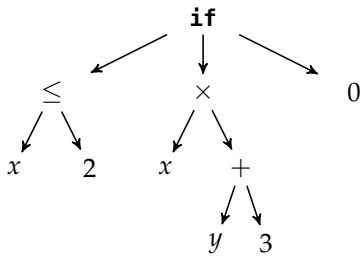


图20.3. Julia方法的树形表示：`x ≤ 2 ? x*(y+3) : 0`

与遗传算法类似，遗传程序随机初始化，并支持交叉和变异。在*tree crossover*（图20.4）中，两棵父树混合形成一棵子树。在每个父树中随机选择一个节点，并将第一个父树中所选节点的子树替换为第二个父树中所选节点的子树。树交叉适用于大小和形状不同的父树，允许任意树混合。在某些情况下，必须确保替换节点具有特定类型，例如输入到`if`语句[3]条件中的布尔值。树交叉在算法20.1中实现。

3 本书仅关注遵守语法约束的遗传操作。有时，带有这种限制的遗传编程被称为*strongly typed genetic programming*，如D. J. Montana在“强类型遗传编程”中所讨论的，*Evolutionary Computation*，第3卷，第2期，第199-230页，1995年。

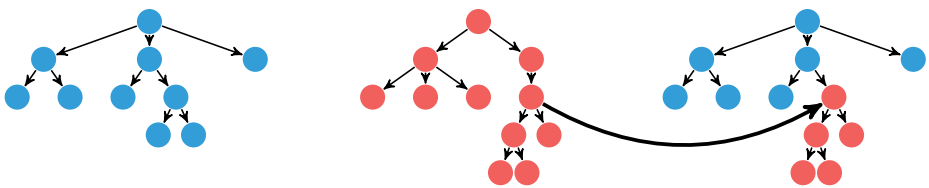


图20.4. 树交叉用于结合两棵父树以产生一棵子树。

树交叉往往会产生比父树深度更大的树。每一代都倾向于增加复杂性，这通常会导致解决方案过于复杂和运行时间变慢。我们通过在目标函数值中基于树的深度或节点数量引入小的偏置，来鼓励*parsimony*，即解决方案的简洁性。

应用*tree mutation*（图20.5）首先从树中选择一个随机节点。以该节点为根的子树被删除，并生成一个新的随机子树来替换旧的子树。与二进制染色体中的突变相比，

```
struct TreeCrossover <: CrossoverMethod
    grammar
    max_depth
end
function crossover(C::TreeCrossover, a, b)
    child = deepcopy(a)
    crosspoint = sample(b)
    typ = return_type(C.grammar, crosspoint.ind)
    d_subtree = depth(crosspoint)
    d_max = C.max_depth + 1 - d_subtree
    if d_max > 0 && contains_returntype(child,C.grammar,typ,d_max)
        loc = sample(NodeLoc, child, typ, C.grammar, d_max)
        insert!(child, loc, deepcopy(crosspoint))
    end
    child
end
```

算法20.1. 为来自ExprRules.jl的类型为RuleNode的a和b实现的树交叉。TreeCrossover结构体包含规则集grammar和最大深度max_depth。

有时，树突变通常至多发生一次，常常以大约1%的低概率发生。树突变在算法20.2中实现。

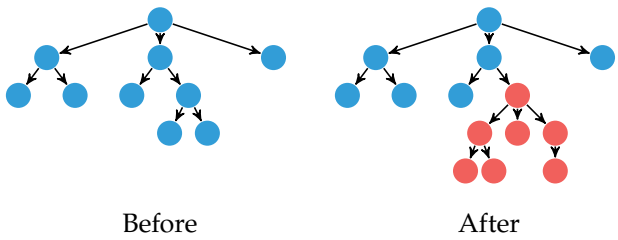


图20.5. 树变异会随机删除一棵子树，并生成一棵新子树来替代它。

```
struct TreeMutation <: MutationMethod
    grammar
    p
end
function mutate(M::TreeMutation, a)
    child = deepcopy(a)
    if rand() < M.p
        loc = sample(NodeLoc, child)
        typ = return_type(M.grammar, get(child, loc).ind)
        subtree = rand(RuleNode, M.grammar, typ)
        insert!(child, loc, subtree)
    end
    return child
end
```

算法20.2. 针对类型为`RuleNode`的个体`a`实现的树变异，源自`ExprRules.jl`。`TreeMutation`结构体包含一个规则集`grammar`和一个突变概率`p`。

Tree permutation (图20.6)是遗传变异的第二种形式。在这里，随机选择的节点的子节点被随机排列。单独的树排列通常不足以将新的遗传物质引入种群，并且经常与树变异结合使用。树排列在算法20.3中实现。

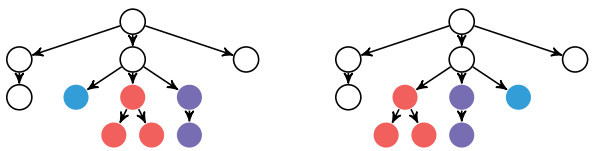


图20.6. 树排列通过随机选择一个节点来排列其子节点。

遗传编程的实现与遗传算法基本相同。通常需要在实现交叉和变异例程时更加小心，特别是在确定可以生成哪些类型的节点以及确保仅生成语法正确的树方面。在例20.4中，遗传编程用于生成一个近似π的表达式。

```
struct TreePermutation <: MutationMethod
    grammar
    p
end
function mutate(M::TreePermutation, a)
    child = deepcopy(a)
    if rand() < M.p
        node = sample(child)
        n = length(node.children)
        types = child_types(M.grammar, node)
        for i in 1 : n-1
            c = 1
            for k in i+1 : n
                if types[k] == types[i] &&
                    rand() < 1/(c+=1)

                    node.children[i], node.children[k] =
                        node.children[k], node.children[i]
                end
            end
        end
    end
    return child
end
```

算法20.3. 针对来自`ExprRules.jl`的类型为`RuleNode`的个体`a`实施的树排列，其中`p`是突变概率。

考虑仅使用四功能计算器上的运算来近似π。我们可以使用遗传规划来解决这个问题，其中节点可以是任何基本运算：加、减、乘、除，以及数字1—9。

我们使用 `ExprRules.jl` 来指定我们的语法：

```
grammar = @grammar begin
    R = |(1:9)
    R = R + R
    R = R - R
    R = R / R
    R = R * R
end
```

我们构建了一个目标函数并对大树进行惩罚：

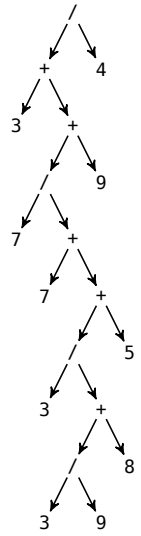
```
function f(node)
    value = eval(node, grammar)
    if isinf(value) || isnan(value)
        return Inf
    end
    Δ = abs(value - π)
    return log(Δ) + length(node)/1e3
end
```

我们最终通过调用第9.2节中的`genetic_algorithm`函数来运行我们的遗传程序：

```
srand(0)
population = [rand(RuleNode, grammar, :R) for i in 1:1000]
best_tree = genetic_algorithm(f, population, 30,
                              TruncationSelection(50),
                              TreeCrossover(grammar, 10),
                              TreeMutation(grammar, 0.25))
```

表现最佳的树显示在右侧。它计算出的值为3.141586，这与π在小数点后四位相匹配。

例20.4. 使用遗传编程仅利用数字和四种基本算术运算来估算π。



20.3 Grammatical Evolution

Grammatical evolution[4] 操作一个整数数组而不是树，允许
在语法进化中，采用了与遗传算法相同的技术。与遗传算法不同的是，语法进化中的染色体基于语法来编码表达式。语法进化受到了遗传物质的启发，遗传物质本质上是串行的，就像遗传算法中使用的染色体一样。[5]

在语法进化中，设计是整数数组，非常类似于遗传算法中使用的染色体。每个整数都是无界的，因为索引是使用模运算执行的。通过从左到右解析，整数数组可以转换为表达式树。

我们从一个起始符号和一个文法开始。假设文法中有n条规则可以应用于起始符号。应用第j条规则，其中$j = i \bmod_1 n$，且i是整数数组中的第一个整数。[6]

然后，我们考虑适用于所得表达式的规则，并基于数组中的第二个整数采用类似的模运算来选择适用的规则。这一过程重复进行，直到无法再应用任何规则，表型即构建完成。[7] 解码过程在算法20.4中实现。

并在示例20.5中详细讨论。

整数数组可能过短，从而导致转录过程超出数组长度。此时，过程不会生成无效个体并在目标函数中对其进行惩罚，而是会绕回数组开头。这种绕回效应意味着同一决策在转录过程中可能被多次读取。转录可能导致无限循环，这可以通过最大深度来防止。

遗传操作直接作用于整数设计数组。我们可以采用用于实值染色体的操作，并将其应用于整数值染色体。唯一的改变是变异必须保持真实值。使用零均值高斯扰动的整数值染色体变异方法在算法20.5中实现。

语法进化使用两个额外的遗传算子。第一个，*gene duplication*，作为DNA复制和修复中的错误自然发生。基因复制可以允许产生新的遗传物质，并且可以存储有用基因的第二份拷贝，以减少致命突变将该基因从基因库中移除的可能性。基因复制在

4 C. Ryan, J. J. Collins, and M. O. Neill, ‘‘语法进化：为任意语言进化程序’’, 载于 *European Conference on Genetic Programming*, 1998.

5 我们的序列DNA被读取并用于构建复杂的蛋白质结构。DNA通常被称为基因型——即执行遗传操作的对象。蛋白质结构是表现型——即由基因型编码并被评估其性能的对象。语法进化文献中常将整数设计向量称为基因型，将所得表达式称为表现型。

6 我们使用 $x \bmod_1 n$ 来表示1索引模：

$$((x-1) \mod n) + 1$$

这种模数类型在使用基于1的索引时很有用。对应的Julia函数是`mod1`。

7 当只有一个适用规则时，不读取遗传信息。

```
struct DecodedExpression
    node
    n_rules_applied
end
function decode(x, grammar, sym, c_max=1000, c=0)
    node, c = _decode(x, grammar, sym, c_max, c)
    DecodedExpression(node, c)
end
function _decode(x, grammar, typ, c_max, c)
    types = grammar[typ]
    if length(types) > 1
        g = x[mod1(c+=1, length(x))]
        rule = types[mod1(g, length(types))]
    else
        rule = types[1]
    end
    node = RuleNode(rule)
    childtypes = child_types(grammar, node)
    if !isempty(childtypes) && c < c_max
        for ctyp in childtypes
            cnode, c = _decode(x, grammar, ctyp, c_max, c)
            push!(node.children, cnode)
        end
    end
    return (node, c)
end
```

算法20.4。一种将整数设计向量解码为表达式的方法，其中`x`是整数向量，`grammar`是`Grammar`，`sym`是根符号。计数器`c`在递归过程中使用，参数`c_max`是规则应用最大数量的上限，以防止无限循环。该方法返回一个`DecodedExpression`，其中包含表达式树和解码过程中应用的规则数量。

考虑一种实数串的文法：

$$
\begin{aligned}
\mathbb{R} &\mapsto \mathbb{D}\,\mathbb{D}'\,\mathbb{P}\,\mathbb{E} \\
\mathbb{D}' &\mapsto \mathbb{D}\,\mathbb{D}' \mid \epsilon \\
\mathbb{P} &\mapsto .\,\mathbb{D}\,\mathbb{D}' \mid \epsilon \\
\mathbb{E} &\mapsto \mathrm{E}\,\mathbb{S}\,\mathbb{D}\,\mathbb{D}' \mid \epsilon \\
\mathbb{S} &\mapsto + \mid - \mid \epsilon \\
\mathbb{D} &\mapsto 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$

其中 $\mathbb{R}$ 是实数值，$\mathbb{D}$ 是终端小数，$\mathbb{D}'$ 是非终端小数，$\mathbb{P}$ 是小数部分，$\mathbb{E}$ 是指数，$\mathbb{S}$ 是符号。任何 ϵ 值都产生空字符串。

假设我们的设计是[205, 52, 4, 27, 10, 59, 6]，并且我们有起始符号$\mathbb{R}$。由于只有一个适用的规则，我们不使用任何遗传信息，并将$\mathbb{R}$替换为$\mathbb{D}\mathbb{D}'\mathbb{P}\mathbb{E}$。

接下来我们必须替换 {v*}。有 10 个选项。我们选择 205 对 $_1$ 取模得到 $= 5$，从而得到 $4\mathbb{D}'\mathbb{P}\mathbb{E}$

接下来我们替换 $\mathbb{D}'$，它有两个选项。我们选择索引 $52 \bmod_1 2 = 2$，这对应于 ϵ。

继续这种方式，我们生成字符串 $_{4E+8}$。

上述语法可以在 `ExprRules` 中使用以下方式实现：

```
grammar = @grammar begin
    R  =  D * De * P * E
    De  =  D * De | ""
    P  =  "." * D * De | ""
    E  =  "E" * S * D * De | ""
    S  =  "+"|"-"|""
    D  =  "0"|"1"|"2"|"3"|"4"|"5"|"6"|"7"|"8"|"9"
end
```

并且可以使用以下方式进行评估：

```
x = [205, 52, 4, 27, 10, 59, 6]
str = eval(decode(x, grammar, :R).node, grammar)
```

示例20.5. 在语法进化中，整型设计向量被解码为表达式的过程。

我们的实现是深度优先的。如果52改为51，则会应用规则 $\mathbb{D}' \mapsto \mathbb{D}\,\mathbb{D}'$，随后为新$\mathbb{D}$选择规则，最终得到$_{43950.950E+8}$。

```
struct IntegerGaussianMutation <: MutationMethod
    σ
end
function mutate(M::IntegerGaussianMutation, child)
    return child + round.(Int, randn(length(child)).*M.σ)
end
```

算法20.5. 修改后的高斯变异方法，以保持整数值染色体的整数值。每个值被标准差为σ的零均值高斯随机值扰动，然后四舍五入到最接近的整数。

chromosomes 待复制。选定区间的副本被附加到染色体后面。复制在算法20.6中实现。

```
struct GeneDuplication <: MutationMethod
end
function mutate(M::GeneDuplication, child)
    n = length(child)
    i, j = rand(1:n), rand(1:n)
    interval = min(i,j) : max(i,j)
    return vcat(child, deepcopy(child[interval]))
end
```

算法20.6。语法进化中使用的基因复制方法。

第二个遗传操作*pruning*解决了交叉过程中遇到的问题。如图20.7所示，交叉会在每条染色体中随机选择一个交叉点，并利用第一条染色体的左侧部分和第二条染色体的右侧部分构建一条新染色体。与遗传算法不同，文法进化中染色体末尾的条目可能不会被使用；在解析过程中，一旦树构建完成，剩余的条目就会被忽略。未使用的条目越多，交叉点位于非活跃区域的可能性就越大，从而无法提供新的有益材料。个体会以指定的概率被修剪，如果被修剪，其染色体将被截断，只保留活跃基因。修剪在算法20.7中实现，并在图20.8中可视化。

与遗传编程类似，语法进化也可以使用遗传算法方法。[8] 我们可以构建一个`MutationMethod`，它应用多个 mu-

在算法20.8中实现了这样一种方法，以便结合使用修剪、复制和标准变异方法。

8 基因型到表型的映射会在目标函数中发生。

语法进化存在两个主要缺点。首先，如果不将染色体解码为表达式，就很难判断其是否可行。

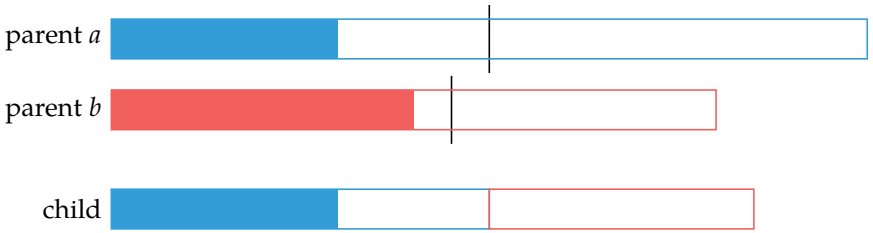


图20.7. 在语法进化中，应用于染色体的交叉可能不会影响染色体前端的活跃基因。此处展示的子代从父代{v*}继承了所有活跃的阴影基因，因此其表现实际上等同于相同的表达。剪切被开发出来以克服这一问题。

```
struct GenePruning <: MutationMethod
    p
    grammar
    typ
end
function mutate(M::GenePruning, child)
    if rand() < M.p
        c = decode(child, M.grammar, M.typ).n_rules_applied
        if c < length(child)
            child = child[1:c]
        end
    end
    return child
end
```

算法20.7。语法进化中使用的基因修剪方法。

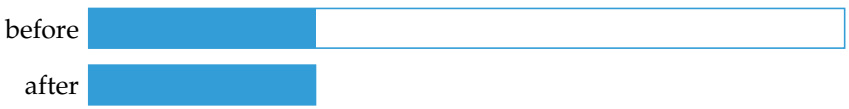


图20.8. 剪枝操作截断染色体，仅保留活跃基因。

```
struct MultiMutate <: MutationMethod
    Ms
end
function mutate(M::MultiMutate, child)
    for m in M.Ms
        child = mutate(m, child)
    end
    return child
end
```

算法20.8。一个应用存储在向量`Ms`中的所有突变方法的`MutationMethod`。

其次，染色体中的微小变化可能导致相应表达产生巨大变化。

20.4 Probabilistic Grammars

一个 *probabilistic grammar*[9] 为遗传程序的语法中的每条规则添加一个权重。3月。当从给定节点的所有适用规则中采样时，我们根据相对权重随机选择一条规则。一个表达式的概率是采样每条规则概率的乘积。算法20.9实现了概率计算。例20.6演示了从概率文法中采样一个表达式并计算其似然。

9 T. L. Booth 和 R. A. Thompson, “将概率测度应用于抽象语言”, *IEEE Transac-tions on Computers*，第 C-22 卷，第 5 期，第 442–450 页，1973 年。

```
struct ProbabilisticGrammar
    grammar
    ws
end
function probability(probgram, node)
    typ = return_type(probgram.grammar, node)
    i = findfirst(isequal(node.ind), probgram.grammar[typ])
    prob = probgram.ws[typ][i] / sum(probgram.ws[typ])
    for (i,c) in enumerate(node.children)
        prob *= probability(probgram, c)
    end
    return prob
end
```

算法20.9. 一种基于概率文法计算表达式概率的方法，其中`probgram`是一个由文法`grammar`和适用于所有规则`ws`的类型到权重映射组成的概率文法，而`node`是一个`RuleNode`表达式。

使用概率文法进行优化，通过每轮迭代中种群内的精英样本改进其权重。在每次迭代中，采样一个表达式种群并计算它们的目标函数值。最佳的部分表达式被视为精英样本，可用于更新权重。为概率文法生成一组新的权重，其中适用于返回类型$\mathbb{T}$的第i条产生式规则的权重$w_i^{\mathbb{T}}$被设置为该产生式规则在生成精英样本时被使用的次数。此更新过程在算法20.10中实现。

上述概率文法可以扩展到更复杂的概率分布，这些分布考虑了其他因素，比如表达式中的深度或子树中兄弟节点间的局部依赖关系。一种方法是使用贝叶斯网络。[10]

10 P. K. Wong, L. Y. Lo, M. L. Wong, 和 K. S. Leung, ‘‘基于语法的遗传规划与贝叶斯网络,’’ 在 *IEEE Congress on Evolutionary Computation (CEC)*, 2014.

示例20.6。从概率文法中采样一个表达式并计算该表达式的似然度。

考虑一种完全由“a”组成的字符串的概率语法：

$$\begin{aligned}
\mathbb{A} &\mapsto \mathrm{a}\,\mathbb{A} && w_1^{\mathbb{A}} = 1 \\
&\mapsto \mathrm{a}\,\mathbb{B}\,\mathrm{a}\,\mathbb{A} && w_2^{\mathbb{A}} = 3 \\
&\mapsto \epsilon && w_3^{\mathbb{A}} = 2 \\
\mathbb{B} &\mapsto \mathrm{a}\,\mathbb{B} && w_1^{\mathbb{B}} = 4 \\
&\mapsto \epsilon && w_1^{\mathbb{B}} = 1
\end{aligned}$$

其中我们为每个父类型有一组权重 w，且 ϵ 是空字符串。

假设我们生成一个以类型 $\mathbb{A}$ 开头的表达式。在三条可能规则上的概率分布为：

$$\begin{aligned}
P(\mathbb{A} \mapsto \mathrm{a}\,\mathbb{A}) &= 1/(1+3+2) &&= 1/6 \\
P(\mathbb{A} \mapsto \mathrm{a}\,\mathbb{B}\,\mathrm{a}\,\mathbb{A}) &= 3/(1+3+2) &&= 1/2 \\
P(\mathbb{A} \mapsto \epsilon) &= 2/(1+3+2) &&= 1/3
\end{aligned}$$

假设我们采样第二条规则并得到一个 $\mathbb{B}$ 一个 $\mathbb{A}$。

接下来，我们抽样一个应用于 $\mathcal{B}$ 的规则。两种可能规则的概率分布为：

$$\begin{aligned}
P(\mathbb{B} \mapsto \mathrm{a}\,\mathbb{B}) &= 4/(4+1) &&= 4/5 \\
P(\mathbb{B} \mapsto \epsilon) &= 1/(4+1) &&= 1/5
\end{aligned}$$

假设我们采样第二条规则并获得一个 ϵ 和一个 $\mathbb{A}$。

接下来，我们对$\mathbb{A}$采样一条规则。假设我们采样得到$\mathbb{A} \mapsto \epsilon$，进而获得$\epsilon$和$\epsilon$，最终生成“a”字符串“aa”。在概率文法下，生成“aa”所应用的规则序列的概率为：

$$P(\mathbb{A} \mapsto \mathrm{a}\,\mathbb{B}\,\mathrm{a}\,\mathbb{A})P(\mathbb{B} \mapsto \epsilon)P(\mathbb{A} \mapsto \epsilon) = \frac{1}{2} \cdot \frac{1}{5} \cdot \frac{1}{3} = \frac{1}{30}$$

请注意，这与获得“aa”的概率不同，因为其他产生式规则序列也可能生成它。

```
function _update!(probgram, x)
    grammar = probgram.grammar
    typ = return_type(grammar, x)
    i = findfirst(isequal(x.ind), grammar[typ])
    probgram.ws[typ][i] += 1
    for c in x.children
        _update!(probgram, c)
    end
    return probgram
end
function update!(probgram, Xs)
    for w in values(probgram.ws)
        fill!(w,0)
    end
    for x in Xs
        _update!(probgram, x)
    end
    return probgram
end
```

算法20.10. 一种基于精英表达式样本`Xs`对概率文法`probgram`应用学习更新的方法。

20.5 Probabilistic Prototype Trees

probabilistic prototype tree[11] 是一种学习方法分布的不同方法
对于表达式树中的每个节点。概率原型树中的每个节点包含一个概率向量，表示语法产生式规则上的分类分布。概率向量经过更新，以反映从连续几代表达式中获得的知识。节点的最大子节点数是语法中规则间非终结符的最大数量。[12]

11 R. Salustowicz 和 J. Schmid-huber，“概率增量程序进化”，*Evolutionary*，第5卷，第2期，第123-141页，1997年。

12 函数的参数数量是*arity*。文法规则的元数，可视为一个函数，是规则中非终结符的数量。

创建节点时，概率向量会被随机初始化。随机概率向量可以从*Dirichlet distribution*.[13]中抽取。原始实现

实现将终端初始化为标量值0.6，非终端初始化为0.4。为了处理强类型文法，我们为适用于每种父类型的规则维护一个概率向量。算法20.11定义了一个节点类型并实现了此初始化方法。

13 狄利克雷分布常用于表示离散分布上的分布。D. Barber, *Bayesian Reasoning and Machine Learning*. 剑桥大学出版社，2012年。

使用概率原型树中的概率向量对表达式进行采样。节点中的规则是从由该节点针对所需返回类型的概率向量定义的分类分布中抽取的，并对该概率向量进行归一化，

```
struct PPTNode
    ps
    children
end
function PPTNode(grammar;
    w_terminal = 0.6,
    w_nonterm = 1-w_terminal,
    )

    ps = Dict(typ => normalize!([isterminal(grammar, i) ?
                    w_terminal : w_nonterm
                    for i in grammar[typ]], 1)
              for typ in nonterminals(grammar))
    PPTNode(ps, PPTNode[])
end
function get_child(ppt::PPTNode, grammar, i)
    if i > length(ppt.children)
        push!(ppt.children, PPTNode(grammar))
    end
    return ppt.children[i]
end
```

算法20.11。一个概率原型树节点类型及相关的初始化函数，其中 `ps` 是一个字典，将返回类型对应的符号映射到适用规则上的概率向量，`children` 是一个 `PPTNode` 的列表。方法 `get_child` 在尝试访问不存在的子节点时将自动扩展树。

将关联的概率向量值转换为有效的概率分布。树按深度优先顺序遍历。该采样过程在算法20.12中实现，并在图20.9中可视化。

学习可以利用来自整个采样群体的信息，也可以利用来自精英样本的信息。设当前代中的最佳表达式为x_{best}，迄今找到的最佳表达式为x_{elite}。节点概率被更新以增加生成x_{best}.[14]的可能性。

14 原始的概率原型树实现会周期性地增加生成x_{elite}的可能性。

生成 x_{best} 的概率等于在遍历概率原型树时选择 x_{best} 中每条规则的概率之乘积。我们为 $P(x_{\text{best}})$ 计算一个目标概率：

$$P_{\text{target}} = P(x_{\text{best}}) + (1 - P(x_{\text{best}})) \cdot \alpha \cdot \frac{\epsilon - y_{\text{elite}}}{\epsilon - y_{\text{best}}} \tag{20.1}$$

其中 α 和 ϵ 是正常数。右侧的分数使得向具有更好目标函数值的表达式迈出更大的步长。目标概率可以使用算法 20.13 计算。

```
function rand(ppt, grammar, typ)
    rules = grammar[typ]
    rule_index = sample(rules, Weights(ppt.ps[typ]))
    ctypes = child_types(grammar, rule_index)

    arr = Vector{RuleNode}(undef, length(ctypes))
    node = iseval(grammar, rule_index) ?
        RuleNode(rule_index, eval(grammar, rule_index), arr) :
        RuleNode(rule_index, arr)

    for (i,typ) in enumerate(ctypes)
        node.children[i] =
            rand(get_child(ppt, grammar, i), grammar, typ)
    end
    return node
end
```

Algorithm 20.12. A method for sampling an expression from a probabilistic prototype tree. The tree is expanded as needed.

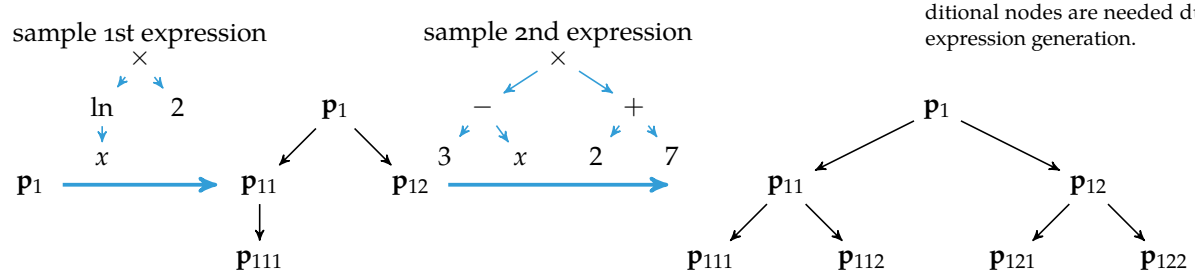


Figure 20.9. A probabilistic prototype tree initially contains only the root node but expands as additional nodes are needed during expression generation.

```
function probability(ppt, grammar, expr)
    typ = return_type(grammar, expr)
    i = findfirst(isequal(expr.ind), grammar[typ])
    p = ppt.ps[typ][i]
    for (i,c) in enumerate(expr.children)
        p *= probability(get_child(ppt, grammar, i), grammar, c)
    end
    return p
end
function p_target(ppt, grammar, x_best, y_best, y_elite, α, ϵ)
    p_best = probability(ppt, grammar, x_best)
    return p_best + (1-p_best)*α*(ϵ - y_elite)/(ϵ - y_best)
end
```

Algorithm 20.13. Methods for computing the probability of an expression and the target probability, where `ppt` is the root node of the probabilistic prototype tree, `grammar` is the grammar, `expr` and `x_best` are `RuleNode` expressions, `y_best` and `y_elite` are scalar objective function values, and α and ϵ are scalar parameters.

目标概率用于调整概率原型树中的概率向量。与所选节点相关的概率被迭代增加，直到超过目标概率：

$$P\left(x_{\text{best}}^{(i)}\right) \leftarrow P\left(x_{\text{best}}^{(i)}\right) + c \cdot \alpha \cdot \left(1 - P\left(x_{\text{best}}^{(i)}\right)\right) \text{ for all } i \text{ in } \{1, 2, \ldots\} \tag{20.2}$$

其中 $x_{\text{best}}^{(i)}$ 是在表达式 x_{best} 中应用的第 i 条规则，而 c 是一个标量。[15]

15 推荐值为 $c = 0.1$。

然后将调整后的概率向量重新归一化为1，方法是将所有未增加的向量分量的值按比例缩减至其当前值。对于增加了第i个分量的概率向量p，其调整方式如下：

$$p_j \leftarrow p_j \frac{1 - p_i}{\|\mathbf{p}\|_1 - p_i} \text{ for } j \neq i \tag{20.3}$$

学习更新在算法20.14中实现。

```
function _update!(ppt, grammar, x, c, α)
    typ = return_type(grammar, x)
    i = findfirst(isequal(x.ind), grammar[typ])
    p = ppt.ps[typ]
    p[i] += c*α*(1-p[i])
    psum = sum(p)
    for j in 1 : length(p)
        if j != i
            p[j] *= (1- p[i])/(psum - p[i])
        end
    end
    for (pptchild,xchild) in zip(ppt.children, x.children)
        _update!(pptchild, grammar, xchild, c, α)
    end
    return ppt
end
function update!(ppt, grammar, x_best, y_best, y_elite, α, c, ϵ)
    p_targ = p_target(ppt, grammar, x_best, y_best, y_elite, α, ϵ)
    while probability(ppt, grammar, x_best) < p_targ
        _update!(ppt, grammar, x_best, c, α)
    end
    return ppt
end
```

算法20.14。一种将学习更新应用于概率原型树的方法，该树的根为`ppt`，文法为`grammar`，最佳表达式为`x_best`及其目标函数值`y_best`，精英目标函数值`y_elite`，学习率为`α`，学习率乘数为`c`，参数为`ϵ`。

除了基于种群的学习外，概率原型树还可以通过变异探索设计空间。树通过变异来探索区域

围绕 x_{best}。设 p 为在生成 x_{best} 时访问的节点中的概率向量。p 中的每个分量都会按照与问题规模成比例的概率发生突变：

$$\frac{p_{\text{mutation}}}{\#\mathbf{p}\sqrt{\#\mathbf{x}_{\text{best}}}} \tag{20.4}$$

其中 p_{mutation} 是一个变异参数，#p 是 p 中的组件数量，$\#x_{best}$ 是 x_{best} 中应用的规则数量。选为变异的组件 i 根据以下方式调整：

$$p_i \leftarrow p_i + \beta \cdot (1 - p_i) \tag{20.5}$$

其中 β 控制变异的量。小概率比大概率经历更大的变异。所有变异后的概率向量必须重新归一化。变异在算法 20.15 中实现，并在图 20.10 中可视化。

```
function mutate!(ppt, grammar, x_best, p_mutation, β;
    sqrtlen = sqrt(length(x_best)),
    )
    typ = return_type(grammar, x_best)
    p = ppt.ps[typ]
    prob = p_mutation/(length(p)*sqrtlen)
    for i in 1 : length(p)
        if rand() < prob
            p[i] += β*(1-p[i])
        end
    end
    normalize!(p, 1)
    for (pptchild,xchild) in zip(ppt.children, x_best.children)
        mutate!(pptchild, grammar, xchild, p_mutation, β,
                sqrtlen=sqrtlen)
    end
    return ppt
end
```

算法20.15. 一种方法，用于变异以根节点`ppt`、文法`grammar`、最优表达式`x_best`、变异参数`p_mutation`及变异率`β`为特征的概率原型树。

最后，对概率原型树中的子树进行剪枝，以移除树中过时的部分。如果父节点包含的概率分量高于指定阈值，使得一旦被选择就会导致子节点变得无关紧要，则移除该子节点。对于终端节点来说，情况总是如此，并且可能

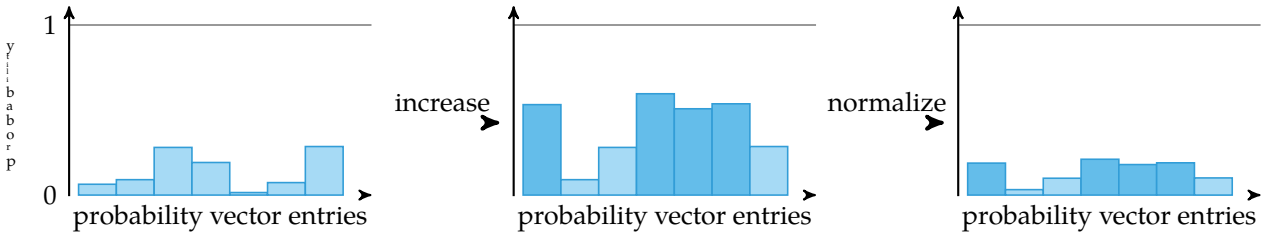


图20.10. 在概率原型树中，将一个概率向量以0.5的突变率β =进行变异。突变分量根据方程(20.5)增加，随后得到的概率向量被重新归一化。注意较小的概率获得了较大的增长。

对于非终结符也成立。剪枝算法在20.16中实现，并在例20.7中演示。

20.6 *Summary*

- 表达式优化允许优化树结构，这些结构在语法下能够表达复杂程序、结构和其他缺乏固定大小的设计。

- 语法定义了用于构造表达式的规则。
- 遗传编程将遗传算法应用于表达式树的变异和交叉操作。

- 语法进化作用于一个整数数组，该数组可以解码为一棵表达式树。

- 概率语法学习哪些规则最适合生成，而概率原型树则为表达式规则生成过程的每一次迭代学习概率。

```
function prune!(ppt, grammar; p_threshold=0.99)
    kmax, pmax = :None, 0.0
    for (k, p) in ppt.ps
        pmax′ = maximum(p)
        if pmax′ > pmax
            kmax, pmax = k, pmax′
        end
    end
    if pmax > p_threshold
        i = argmax(ppt.ps[kmax])
        if isterminal(grammar, i)
            empty!(ppt.children[kmax])
        else
            max_arity_for_rule = maximum(nchildren(grammar, r) for
                                         r in grammar[kmax])
            while length(ppt.children) > max_arity_for_rule
                pop!(ppt.children)
            end
        end
    end
    return ppt
end
```

算法20.16。一种用于修剪以ppt为根、语法为grammar且剪枝概率阈值为p_treshold的概率原型树的方法。

考虑一个在规则集上具有概率向量的节点：

$$\mathbb{R} \mapsto \mathbb{R} + \mathbb{R}$$
$$\mathbb{R} \mapsto \ln(\mathbb{R})$$
$$\mathbb{R} \mapsto 2 \mid x$$
$$\mathbb{R} \mapsto \mathbb{S}$$

如果选择2或{v*}的概率变大，那么概率原型树中的任何子节点都不太可能需要，可以将其剪枝。同样，如果选择S的概率变大，任何返回类型为ℝ的子节点都是不需要的，可以将其剪枝。

示例 20.7. 一个应应用概率原型树剪枝的例子。

20.7 Exercises

练习20.1. 使用以下语法和起始集合$\{\mathbb{R}, \mathbb{I}, \mathbb{F}\}$，可以生成多少种表达式树?

$$\begin{aligned}\mathbb{R} &\mapsto \mathbb{I} \mid \mathbb{F} \\ \mathbb{I} &\mapsto 1 \mid 2 \\ \mathbb{F} &\mapsto \pi\end{aligned}$$

练习20.2。在某一文法下可生成的、高度不超过h的表达式树的数量呈超指数增长。作为参考，计算使用该文法可生成的高度为h的表达式数量：[16]

$$\mathbb{N} \mapsto \{\mathbb{N}, \mathbb{N}\} \mid \{\mathbb{N}, \} \mid \{, \mathbb{N}\} \mid \{\} \tag{20.6}$$

[16] Let an empty expression have height 0, the expression {} have height 1, and so on.

练习 20.3. 定义一个能生成任意非负整数的文法。

练习20.4. 表达式优化方法如何处理在生成随机子树时遇到的除零值或其他异常?

练习 20.5. 考虑如下算术文法：

$$\mathbb{R} \mapsto x \mid y \mid z \mid \mathbb{R} + \mathbb{R} \mid \mathbb{R} - \mathbb{R} \mid \mathbb{R} \times \mathbb{R} \mid \mathbb{R} / \mathbb{R} \mid \ln \mathbb{R} \mid \sin \mathbb{R} \mid \cos \mathbb{R}$$

假设变量 x、y 和 z 各自具有单位，而输出期望以特定单位表示。这样的语法如何修改才能尊重单位?

练习 20.6. 考虑文法

$$\begin{aligned}\mathbb{S} &\mapsto \mathbb{NP}\ \mathbb{VP} \\ \mathbb{NP} &\mapsto \mathbb{ADJ}\ \mathbb{NP} \mid \mathbb{ADJ}\ \mathbb{N} \\ \mathbb{VP} &\mapsto \mathbb{V}\ \mathbb{ADV} \\ \mathbb{ADJ} &\mapsto \text{a} \mid \text{the} \mid \text{big} \mid \text{little} \mid \text{blue} \mid \text{red} \\ \mathbb{N} &\mapsto \text{mouse} \mid \text{cat} \mid \text{dog} \mid \text{pony} \\ \mathbb{V} &\mapsto \text{ran} \mid \text{sat} \mid \text{slept} \mid \text{ate} \\ \mathbb{ADV} &\mapsto \text{quietly} \mid \text{quickly} \mid \text{soundly} \mid \text{happily}\end{aligned}$$

与基因型 [2, 10, 19, 0, 6] 及起始符号 $\mathbb{S}$ 相对应的表现型是什么?

练习 20.7。使用遗传编程来演化时钟的齿轮比。假设所有齿轮的半径限制为从 $\mathcal{R} = \{10, 25, 30, 50, 60, 100\}$ 中选择。每个齿轮既可以附接到其父级的轴上，从而共享相同的旋转周期，也可以啮合在其父级的边缘上，从而根据父级的旋转周期和如图 20.11 所示的齿轮比来决定其旋转周期。

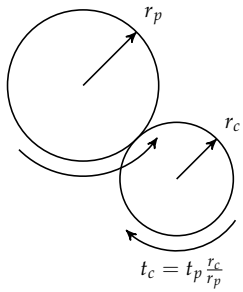


图20.11。附加在父齿轮边缘的子齿轮的旋转周期t_c取决于父齿轮的旋转周期t_p以及齿轮半径的比例。

时钟也可以包含指针，这些指针安装在父齿轮的轴上。假设根齿轮以 $t_{\text{root}} = 0$ .1 秒的周期转动，且半径为 25。目标是制作一个具有秒针、分针和时针的时钟。

根据以下内容为每个个体评分：

$$\left(\underset{\text{hands}}{\text{minimize}}\,(1 - t_{\text{hand}})^2\right) + \left(\underset{\text{hands}}{\text{minimize}}\,(60 - t_{\text{hand}})^2\right) + \left(\underset{\text{hands}}{\text{minimize}}\,(3600 - t_{\text{hand}})^2\right) + \#\text{nodes} \cdot 10^{-3}$$

其中t_{hand}是特定指针的旋转周期（以秒为单位），#nodes是表达式树中的节点数。忽略旋转方向。

练习20.8. 四个4的谜题[17]是一个数学挑战，其中我们使用
使用四个4数字和数学运算为0到100的每个整数生成表达式。例如，前两个整数可以分别由$4 + 4 - 4 - 4$和44/44产生。完成四个4的谜题。

17 W. W. R. Ball，*Mathematical Recre-*。麦克*ations and Essays*

练习20.9。考虑概率语法

$$\begin{aligned} \mathbb{R} &\mapsto \mathbb{R} + \mathbb{R} \mid \mathbb{R} \times \mathbb{R} \mid \mathbb{F} \mid \mathbb{I} & w_{\mathbb{R}} &= [1, 1, 5, 5] \\ \mathbb{F} &\mapsto 1.5 \mid \infty & p_{\mathbb{F}} &= [4, 3] \\ \mathbb{I} &\mapsto 1 \mid 2 \mid 3 & p_{\mathbb{I}} &= [1, 1, 1] \end{aligned}$$

表达式 $1.5 + 2$ 的生成概率是多少?

练习20.10. 在清除计数并对$1.5 + 2$应用学习更新后，上一个问题中的概率语法是什么?

21 Multidisciplinary Optimization

Multidisciplinary design optimization (MDO) 涉及求解跨学科的优化问题。许多现实世界的问题涉及多个学科之间复杂的相互作用，单独优化各个学科可能无法得到最优解。本章讨论了多种利用 MDO 问题结构来减少寻找良好设计所需工作量的技术。[1]

1 J. R. R. A. Martins 和 A. B. Lambe 提供了一项广泛的综述，“多学科设计优化：架构综述”，*AIAA Journal*，第 51 卷，第 9 期，第 2049–2075 页，2013 年。进一步的讨论可在 J. Sobieszczanski-Sobieski、A. Morris 和 M. van Tooren 的 *Multidis-* 中找到。Wiley，2015 年。另见 N. M. Alexandrov 和 M. Y. Hussaini 编，*Multidisciplinary Design Op-*。SIAM，1997 年。

ciplinary Design Optimization Theory and Practice

timization: State of the Art

Based Engineering

ing

21.1 Disciplinary Analyses

有许多不同的*disciplinary analyses*可能会影响设计。例如，火箭的设计可能涉及结构、空气动力学和控制等学科的分析。不同的学科有自己的分析工具，如有限元分析。这些学科分析往往相当复杂且计算成本高昂。此外，学科分析通常彼此紧密耦合，其中一个学科可能需要另一个学科分析的输出。解决这些相互依赖关系可能并非易事。

在多学科设计优化（MDO）环境中，我们仍像之前一样拥有一组设计变量，但同时还会跟踪每个学科分析的输出，即 *response variables*。[2] 我们将第i个学科分析的响应变量写为 $\mathbf{y}(i)$。通常，第i个学科分析 F_i 可以依赖于设计变量或来自任何其他学科的响应变量：

2 学科分析可以为其他学科、目标函数或约束条件提供输入。此外，它还可以为优化器提供梯度信息。

$$\mathbf{y}^{(i)} \leftarrow F_i\left(\mathbf{x}, \mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(i-1)}, \mathbf{y}^{(i+1)}, \ldots, \mathbf{y}^{(m)}\right) \tag{21.1}$$

其中 m 是学科总数。对飞机进行计算流体动力学分析的输入可能包括机翼的偏转，这

来自结构分析，该分析需要计算流体动力学提供的力。制定MDO问题的一个重要部分是考虑分析之间的这种依赖关系。

为了使对学科分析进行推理更加容易，我们引入了*assignment*的概念。一个赋值$\mathcal{A}$是一组与多学科设计优化问题相关的变量名称及其对应的值。要访问变量v，我们写作$\mathcal{A}[v]$。

学科分析是一种函数，它接受一个分配，并利用来自其他分析的设计点和响应变量来覆盖其学科的响应变量：

$$\mathcal{A}' \leftarrow F_i(\mathcal{A}) \tag{21.2}$$

其中 $F_i(\mathcal{A})$ 在 $\mathcal{A}$ 中更新 y(i) 以生成 $\mathcal{A}'$。

赋值可以用字典在代码中表示。[3] 每个变量被赋予类型为`String`的名称。变量不限于浮点数，还可以包括其他对象，如向量。例21.1展示了使用字典的实现。

3 一个*dictionary*，也称为*associative array*，是一种常见的数据结构，它允许通过键而非整数进行索引。参见附录A.1.7。

考虑一个优化问题，其中包含一个设计变量x和两个学科。假设第一个学科分析F_1计算响应变量$y^{(1)} = f_1(x, y^{(2)})$，而第二个学科分析$F_2$计算响应变量$y^{(2)} = f_2(x, y^{(1)})$。

该问题可以实现为：

```
function F1(A)
    A["y1"] = f1(A["x"], A["y2"])
    return A
end
function F2(A)
    A["y2"] = f2(A["x"], A["y1"])
    return A
end
```

赋值可以使用 $y^{(1)}$ 和 $y^{(2)}$ 的猜测值以及 x 的已知输入来初始化。例如：

```
A = Dict("x"=>1, "y1"=>2, "y2"=>3)
```

示例 21.1. 基于分配表示的多学科设计优化问题的基本代码语法。

21.2 *Interdisciplinary Compatibility*

评估设计点x的目标函数值和可行性，需要获取满足*interdisciplinary compatibility*的响应变量值，这意味着响应变量必须与学科分析保持一致。对于特定赋值，如果该赋值在所有学科分析下保持不变，则满足跨学科兼容性：

$$F_i(\mathcal{A}) = \mathcal{A} \text{ for } i \in \{1, \ldots, m\} \tag{21.3}$$

运行任何分析都会产生相同的值。找到满足跨学科兼容性的分配称为*multidisciplinary analysis*。

针对单一学科的系统优化需要一个优化器来选择设计变量，并查询学科分析以评估约束条件和目标函数，如图21.1所示。单学科优化无需考虑学科间的耦合。

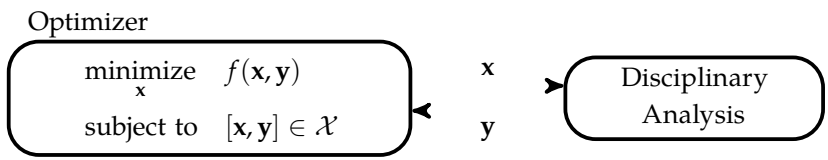


图21.1. 单一学科的优化图。梯度计算可有可无。

多学科系统优化可能引入依赖关系，此时耦合便成为一个问题。双耦合学科的示意图见图21.2。对此问题应用传统优化方法较为复杂，因为必须建立跨学科兼容性。

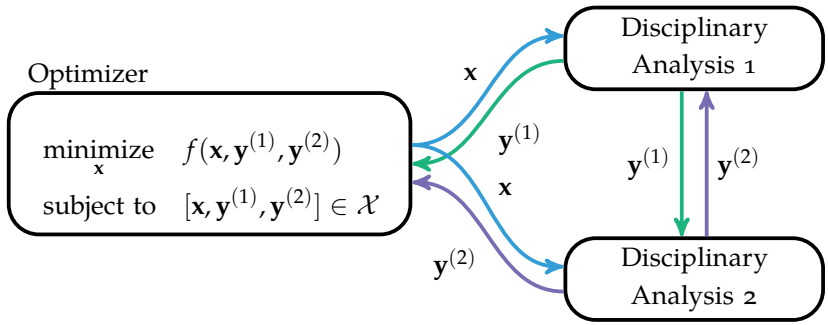


图21.2. 双学科分析中的跨学科耦合优化图。

如果多学科分析没有*dependency cycle*、[4]，那么求解为了跨学科的兼容性，这一做法直截了当。如果学科i需要学科j的任何输出，我们就说学科i依赖于学科j。这种依赖关系可用于构建*dependency graph*，其中每个节点对应一个学科，若学科i依赖于学科j，则包含边$j \to i$。图21.3展示了涉及两个学科的依赖图示例，包括有无循环的情况。

4 当学科相互依赖时，就会出现依赖循环。

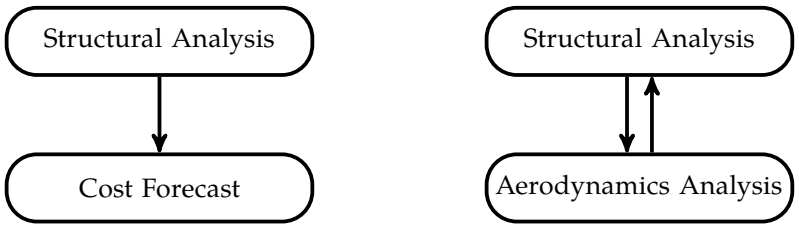


图21.3. 循环和无环依赖图。

无环依赖图。可以指定一种评估顺序，使得每个学科所需的输入可从先前评估的学科中获得。循环依赖图。结构分析依赖于空气动力学分析，反之亦然。

如果依赖图无环，则总存在一种评估顺序，遵循该顺序可确保必要的学科分析在依赖它们的学科分析之前进行评估。这种排序称为*topological ordering*，并可通过拓扑排序方法如*Kahn's algorithm*找到。[5]分析重新排序如图21.4所示。

5 A. B. Kahn，“大型网络的拓扑排序”，*Communica-tions of the ACM*，第5卷，第11期，第558-562页，1962年。

如果依赖图存在环，则不存在拓扑排序。为了解决环的问题，我们可以使用*Gauss-Seidel method*（算法21.1），它尝试通过迭代直到收敛来解决多学科分析。[6]

6 Gauss-Seidel 算法也可以编写为并行执行分析。

高斯-赛德尔算法对学科顺序敏感，如例21.2所示。不佳的顺序可能会阻止或减缓收敛。最佳顺序是那些反馈连接最少的顺序。[7]

7 在某些情况下，学科可以被划分为相互独立的 不同集群。每个连通的集群可以使用其自身的、更小规模的多学科分析来求解。

将学科合并为新的学科分析可能是有利的——将概念上相关的分析分组，同时评估紧密耦合的分析，或更有效地应用本章讨论的某些架构。学科分析可以合并形成一个新的分析，其响应变量由被合并学科的响应变量组成。新分析的形式取决于学科间的相互依赖关系。如果

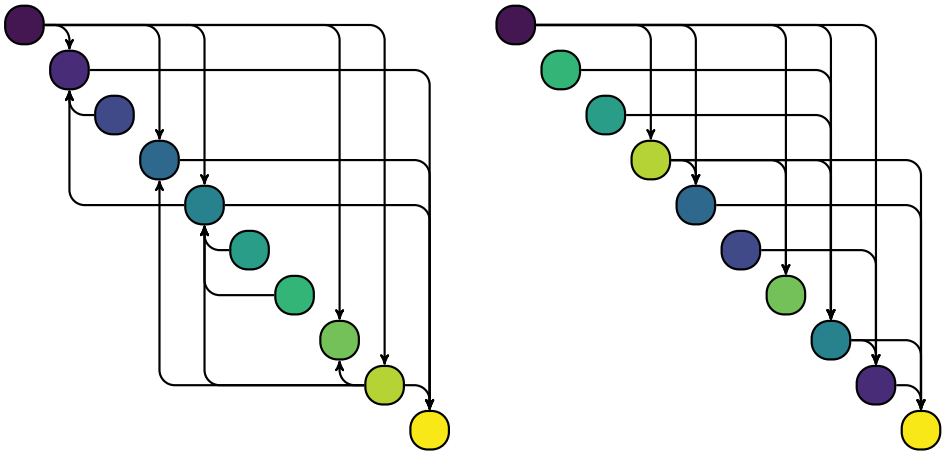


图21.4。拓扑排序可用于重新排列学科分析以消除反馈连接。

```
function gauss_seidel!(Fs, A; k_max=100, ϵ=1e-4)
    k, converged = 0, false
    while !converged && k ≤ k_max
        k += 1
        A_old = deepcopy(A)
        for F in Fs
            F(A)
        end
        converged = all(isapprox(A[v], A_old[v], rtol=ϵ)
                        for v in keys(A))
    end
    return (A, converged)
end
```

算法21.1. 用于进行多学科分析的高斯-赛德尔算法。此处，`Fs` 是一个学科分析函数向量，这些函数接收并修改一个赋值 `A`。有两个可选参数：最大迭代次数 `k_max` 和相对误差容限 `ϵ`。该方法返回修改后的赋值以及是否收敛。

例21.2。一个示例，说明了在进行多学科分析时选择合适顺序的重要性。

考虑一个多学科设计优化问题，其中有一个设计变量x和三个学科，每个学科有一个响应变量：

$$
\begin{aligned}
y^{(1)} &\leftarrow F_1(x, y^{(2)}, y^{(3)}) = y^{(2)} - x \\
y^{(2)} &\leftarrow F_2(x, y^{(1)}, y^{(3)}) = \sin(y^{(1)} + y^{(3)}) \\
y^{(3)} &\leftarrow F_3(x, y^{(1)}, y^{(2)}) = \cos(x + y^{(1)} + y^{(2)})
\end{aligned}
$$

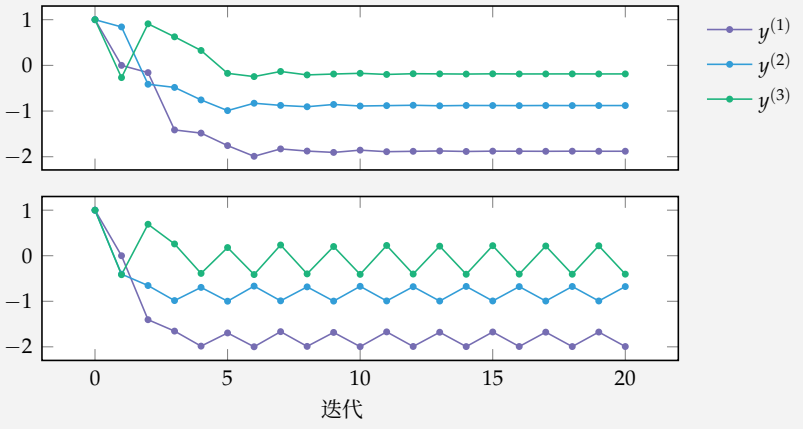
学科分析可以通过以下方式实现：

```
function F1(A)
    A["y1"] = A["y2"] - A["x"]
    return A
end
function F2(A)
    A["y2"] = sin(A["y1"] + A["y3"])
    return A
end
function F3(A)
    A["y3"] = cos(A["x"] + A["y2"] + A["y1"])
    return A
end
```

考虑对x =运行一次多学科分析 1，并将我们的分配初始化为全1：

```
A = Dict("x"=>1, "y1"=>1, "y2"=>1, "y3"=>1)
```

使用排序 F_1、F_2、F_3 运行 Gauss-Seidel 算法会收敛，但使用 F_1、F_3、F_2 运行则不会。



若合并后的学科是非循环的，则存在一种顺序，使得这些分析可以串行执行。如果合并后的学科是循环的，那么新的分析必须在内部运行多学科分析以实现兼容性。

21.3 Architectures

多学科设计优化问题可以写成：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathcal{A}) \\ \text{subject to} \quad & \mathcal{A} \in \mathcal{X} \\ & F_i(\mathcal{A}) = \mathcal{A} \text{ for each discipline } i \in \{1, \ldots, m\} \end{aligned} \tag{21.4}$$

其中目标函数 f 和可行集 $\mathcal{X}$ 均取决于设计变量和响应变量。分配中的设计变量由优化器指定。条件 $F_i(\mathcal{A}) = \mathcal{A}$ 确保第 i 个学科与 $\mathcal{A}$ 中的值一致。这最后一个条件强制实现了跨学科的兼容性。

优化多学科问题时存在若干挑战。学科分析之间的相互依赖使得分析顺序变得重要，并且常常使并行化变得困难或不可能。在直接控制所有变量的优化器与纳入利用学科特定专业知识的子优化器[8]之间存在权衡。

在本地优化值。此外，运行学科分析的成本与全局优化过多变量的成本之间存在权衡。最终，每个架构都必须确保最终解决方案中的跨学科兼容性。

[8] A *suboptimizer* is an optimization routine called within another optimization routine.

本章剩余部分将讨论应对这些挑战的各种不同优化架构。这些架构将通过示例21.3中引入的假设性拼车问题来展示。

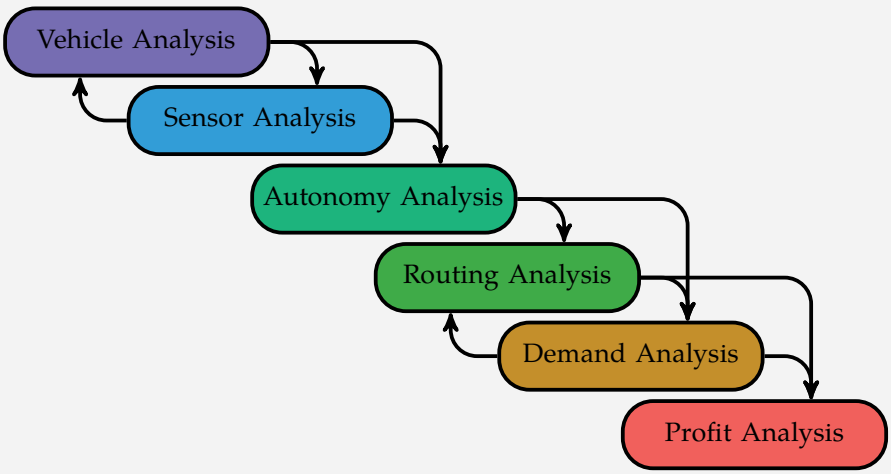
21.4 Multidisciplinary Design Feasible

multidisciplinary design feasible 架构构建 MDO 问题，使得标准优化算法可直接用于优化设计变量。对于任何给定的设计点，都会进行多学科设计分析，以获得兼容的响应值。

例21.3. 一个贯穿本章用于展示优化架构的拼车问题。

考虑一家正在开发自动驾驶车队的网约车公司。这家假设的公司同时设计车辆、其传感器套件、路线策略和定价方案。设计的这些部分分别被称为 v、s、r 和 p，其中每一部分都包含众多设计变量。例如，车辆可能包括控制结构几何形状、发动机和传动系统、电池容量以及载客量的参数。

网约车公司的目标是最大化利润。利润取决于路径算法和乘客需求的大规模仿真，而后者又取决于车辆及其传感器套件的自主性分析中的响应变量。若干分析会提取额外信息。路径算法的性能取决于定价方案生成的需求，而定价方案生成的需求又取决于路径算法的性能。车辆的续航里程和燃油效率取决于传感器套件的重量、阻力和功耗。传感器套件需要车辆几何和性能信息来满足必要的安全要求。下面给出了依赖关系图。



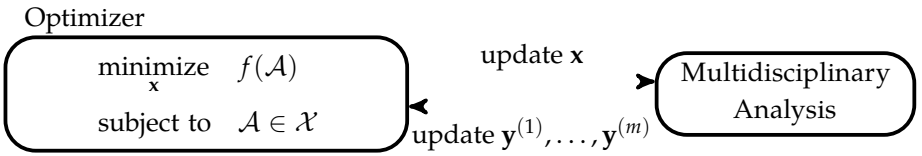


图21.5. 多学科设计可行架构。优化器选择设计点x，而多学科分析计算出一致性分配$\mathcal{A}$。结构类似于单学科优化。

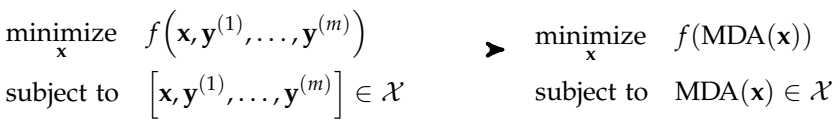


图21.6. 将MDO问题通过多学科设计分析转化为典型的优化问题，其中MDA(x)返回一个多学科兼容的分配。

图21.5给出了一个架构图。它由两个模块组成，即优化器和多学科分析。优化器是用于选择设计点的方法，目标是最小化目标函数。优化器通过向多学科分析模块传递设计点x来调用该模块，并接收一个兼容赋值$\mathcal{A}$。如果无法实现跨学科兼容，多学科分析模块会通知优化器，此类设计点被视为不可行。图21.6展示了如何使用多学科设计分析将MDO问题转化为典型的优化问题。

多学科设计可行架构的主要优势在于其概念简单，且能确保在优化的每一步都维持跨学科的兼容性。其名称反映了这样一个事实：在每个设计评估阶段都会进行多学科设计分析，从而确保系统级优化器仅考虑可行的设计。

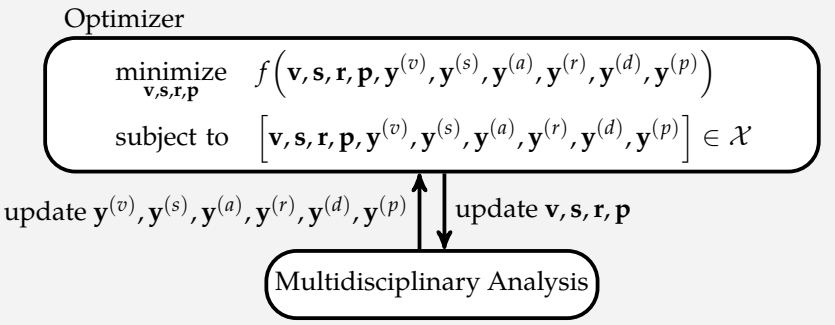
主要缺点是， multidisciplinary design analyses are ex- pensive to run， typically r equiring several iterations over all analyses. Iterative Gauss-Seidel methods may be slo w to converge or may not converge at all， de- pending on the initialization of the resp onse variables and the ordering of the disciplinary analyses.

将分析集中在一起，使得所有局部变量——通常仅与某一特定学科相关——必须被整体分析所考虑。许多实际问题涉及大量局部设计变量，例如空气动力学中的网格控制点、结构中的单元尺寸、电气工程中的组件布局，以及神经网络中的

机器学习中的权重。多学科设计可行优化要求系统优化器在所有学科中指定所有这些值，同时满足所有约束条件。

多学科设计可行架构被应用于示例21.4中的拼车问题。

多学科设计可行架构可以应用于拼车问题。架构图如下所示。



例21.4. 应用于拼车问题的多学科设计可行架构。对于每一个候选设计点，必须完成对所有响应变量的多学科分析。这往往计算量非常大。

21.5 *Sequential Optimization*

sequential optimization 架构（图21.7）是一种能够利用特定领域的工具和经验来优化子问题，但可能导致次优解的架构。此架构的引入旨在展示朴素方法的局限性，并作为与其他架构进行比较的基准。

*subproblem*是在总体优化过程的每次迭代中进行的优化程序。有时，设计变量可以从外部优化程序*system-level optimizer*中移除，并可以在子问题中更有效地优化。

i 学科的设计变量可以根据 $\mathbf{x}(i) = [\mathbf{x}_g^{(i)}, \mathbf{x}_\ell^{(i)}]$ 进行划分，其中 $\mathbf{x}_g^{(i)}$ 是 *global design variables* 与其他学科共享的，而 $\mathbf{x}_\ell^{(i)}$ 是 *local design variables* 仅由相关学科子问题使用的。[9] 响应变量可以类似地划分为全局响应

9 共享出行问题中的车辆子问题可能包括影响其他学科的全局设计变量，如车辆容量和行驶里程，但也可能包括不影响其他学科的局部设计变量，如座位配置。

变量 $\mathbf{y}(i)_g$ 和局部响应变量 $\mathbf{y}(i)_\ell$。学科自主性是

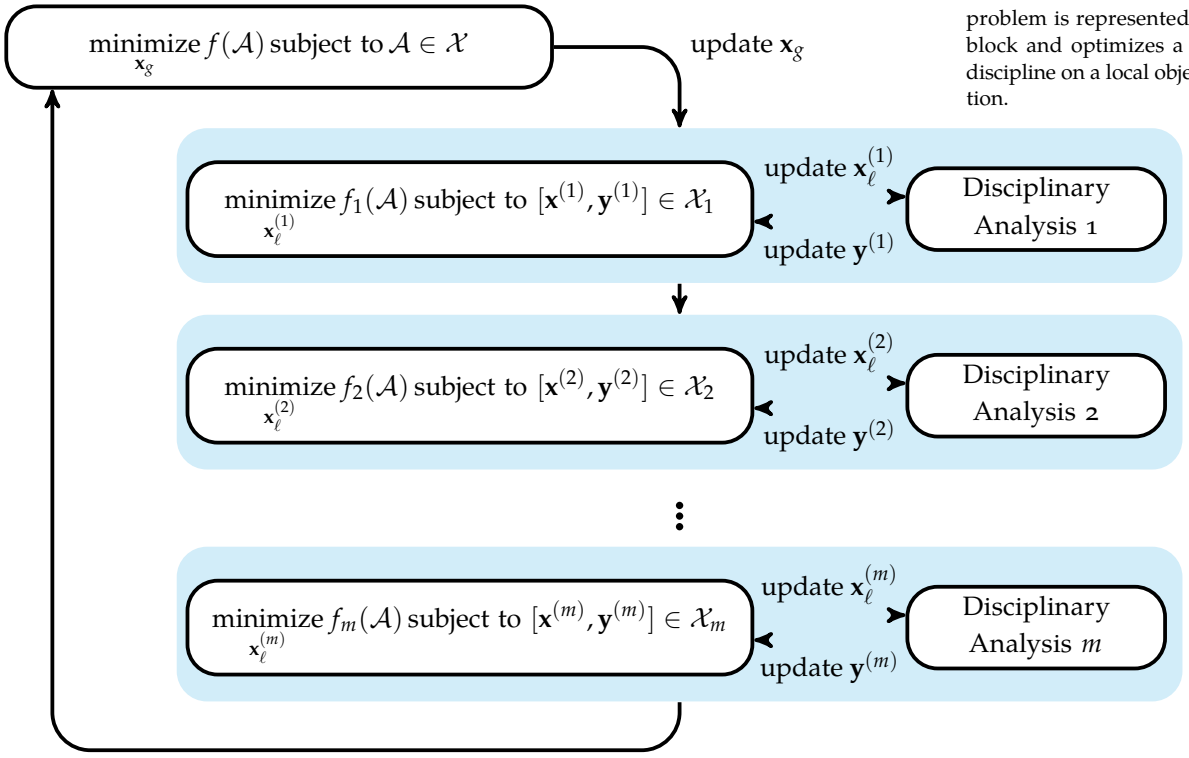


Figure 21.7. The sequential 最优 mization architecture. Each 子 problem is represented b呀，蓝色 block and optimizes a particular discipline on a local objective函数- tion.

通过在其各自学科优化器中优化局部变量来实现。必须选择局部目标函数 {v*}，使得优化它同时也有利于全局目标。顶层优化器负责针对原始目标函数优化全局设计变量 x_g。x_g 的一个实例化通过顺序优化进行评估；每个子问题依次优化，将其结果传递给下一个子问题，直到所有子问题都被评估完毕。

顺序优化利用了学科局部性；即许多变量是特定学科所独有的，不需要跨学科边界共享。顺序优化利用了每个学科解决其学科特定问题的能力。子问题优化器完全控制其学科特定的设计变量，以满足局部设计目标和约束。

除特殊情况外，顺序优化并不能导致原问题的最优解，原因与Gauss-Seidel方法不保证收敛相同。该解对局部目标函数敏感，而找到合适的局部目标函数往往是一个挑战。顺序优化不支持并行执行，且跨学科兼容性通过迭代强制实现，并不总能收敛。

例21.5将顺序优化应用于拼车问题。

21.6 *Individual Discipline Feasible*

individual discipline feasible (IDF)架构无需运行昂贵的多学科设计分析，并允许学科分析并行执行。它失去了在其整个执行过程中保持跨学科兼容性的保证，最终一致性通过优化器中的等式约束来强制实现。兼容性并非在多学科分析中强制实现，而是由优化器本身来强制实现。

IDF将*coupling variables*引入设计空间。对于每个学科，在优化问题中增加一个额外的向量c(*i*)，作为响应变量y(*i*)的别名。响应变量在由其各自领域分析计算之前是未知的；耦合变量的引入允许优化器在并行运行分析时同时向多个学科提供这些估计值。耦合变量之间的相等性

例21.5. 拼车问题的顺序优化。

顺序优化架构可以在局部优化一些变量。图21.8展示了将顺序优化应用于拼车问题的结果。

车辆、传感器系统、路由算法和定价方案的设计变量被分为局部学科特定变量和顶层全局变量。例如，车辆子问题可以优化局部车辆参数$\mathbf{v}_\ell$，如传动系统组件，而其他分析中使用的车辆容量等参数则在$\mathbf{v}_g$中全局控制。

车辆与传感器系统之间的紧密耦合在顺序优化架构中处理得不够好。虽然车辆子问题所做的更改会立即由传感器子问题处理，但传感器子问题对车辆子问题的影响直到下一次迭代才会被处理。

并非所有分析都需要各自的子问题。利润分析假定没有任何局部设计变量，因此无需子问题块即可执行。

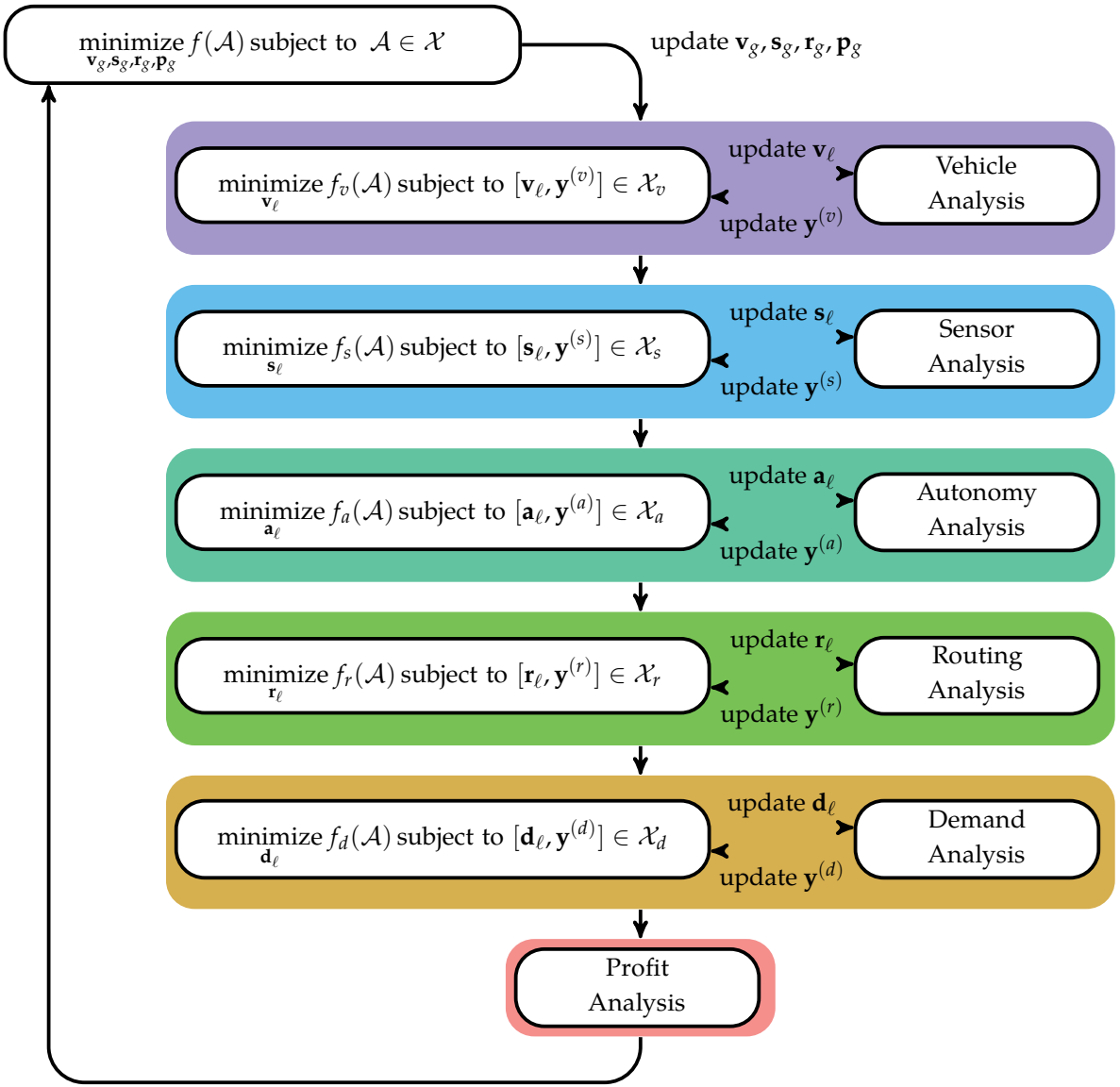


图21.8。应用于拼车问题的顺序优化架构。

耦合与响应变量通常通过迭代达到。等式是一种优化约束，$c(i) = y(i)$，针对每个学科。

图21.9展示了通用的IDF架构。系统级优化器处理耦合变量，并利用这些变量来填充一个分配方案，该方案在每次迭代中被复制到学科分析中：

$$\mathcal{A}[\mathbf{x}, \mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(m)}] \leftarrow [\mathbf{x}, \mathbf{c}^{(1)}, \ldots, \mathbf{c}^{(m)}] \tag{21.5}$$

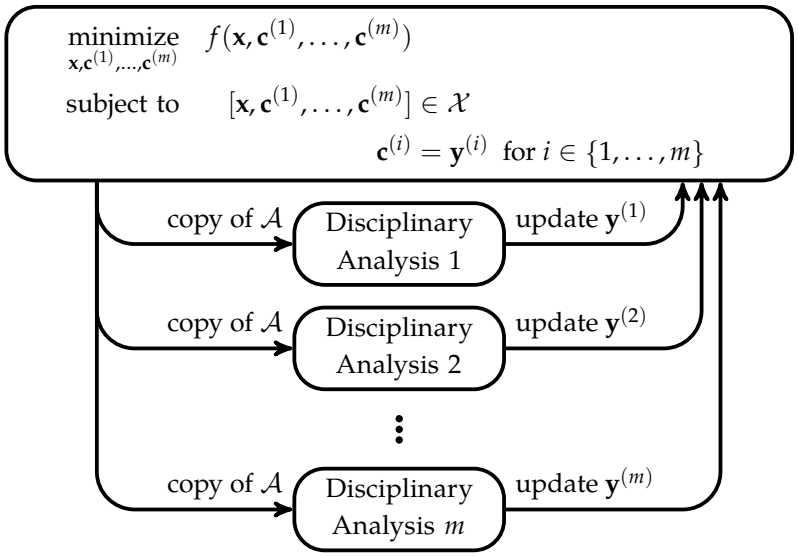


图21.9. 各学科可行的架构允许并行进行学科分析。本章假设学科分析会修改其输入，因此系统级优化器的分配副本会被传递给每个学科分析。

尽管并行执行分析增加了自由度，但IDF的缺点在于它无法像顺序优化那样利用领域特定的优化过程，因为优化仅处于顶层。此外，优化器必须满足额外的等式约束，并且有更多的变量需要优化。如图21.10所示，IDF在基于梯度的优化中可能遇到困难，因为所选择的搜索方向必须考虑到约束条件。设计变量的变化不得导致耦合变量在学科分析方面变得不可行。当学科分析成本高昂时，评估目标和约束函数的梯度非常昂贵。

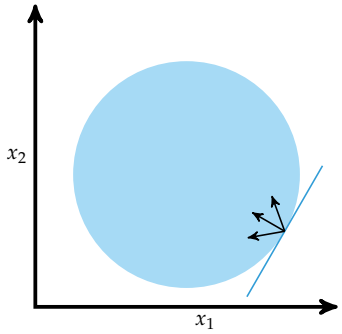


图21.10。约束边界上一点的搜索方向必须指向可行集内。

个体学科可行架构被应用于图21.11中的拼车问题。

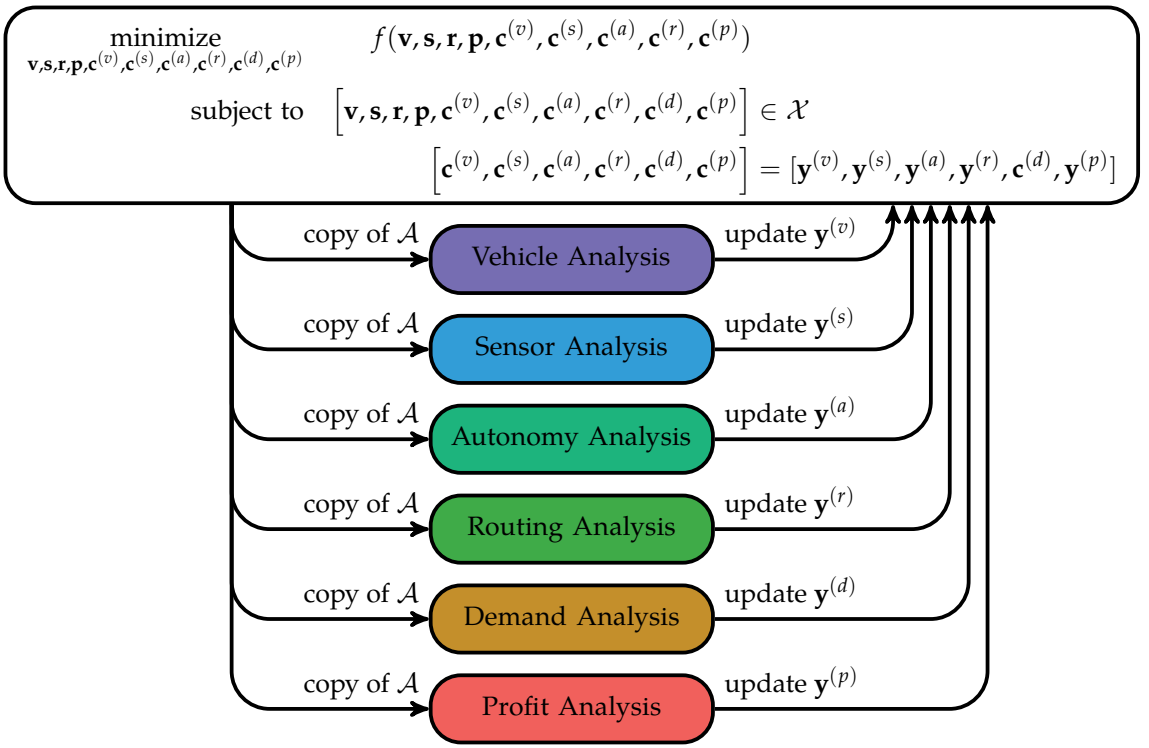


图21.11. 应用于拼车问题的个体学科可行架构。个体设计可行架构允许分析的并行执行，但系统级优化器必须优化大量变量。

21.7 Collaborative Optimization

collaborative optimization 架构（图 21.12）将问题分解为学科子问题，这些子问题对其局部设计变量和学科特定约束拥有完全控制权。子问题可以使用学科特定的工具来求解，并且可以并行优化。

第 i 个子问题具有以下形式：

$$\begin{aligned} \underset{\mathbf{x}^{(i)}}{\text{minimize}} \quad & f_i(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}) \\ \text{subject to} \quad & [\mathbf{x}^{(i)}, \mathbf{y}^{(i)}] \in \mathcal{X}_i \end{aligned} \tag{21.6}$$

其中 x(i) 包含设计变量 x 和响应变量 y(i) 的一个子集。该约束确保解决方案满足特定学科的约束。

跨学科兼容性要求全局变量x(i) $_g$和y(i) $_g$在所有学科之间保持一致。我们定义一组耦合变量$\mathcal{A}_g$，其中包括与至少在一个子问题中为全局的所有设计和响应变量相对应的变量。通过约束每个x(i) $_g$和y(i) $_g$与其对应的耦合变量匹配来强制实现一致性：

$$\mathbf{x}_g^{(i)} = \mathcal{A}_g[\mathbf{x}_g^{(i)}] \qquad \text{and} \qquad \mathbf{y}_g^{(i)} = \mathcal{A}_g[\mathbf{y}_g^{(i)}] \tag{21.7}$$

其中 $\mathcal{A}_g[\mathrm{x}_g^{(i)}]$ 和 $\mathcal{A}_g[\mathrm{y}_g^{(i)}]$ 是第 i 个学科中对应于全局设计变量和响应变量的耦合变量。该约束通过子问题目标函数来强制执行：

$$f_i = \left\| \mathbf{x}_g^{(i)} - \mathcal{A}_g[\mathbf{x}_g^{(i)}] \right\|_2^2 + \left\| \mathbf{y}_g^{(i)} - \mathcal{A}_g[\mathbf{y}_g^{(i)}] \right\|_2^2 \tag{21.8}$$

每个子问题因此寻求与耦合变量偏离最小的可行解。

子问题由系统级优化器管理，该优化器负责优化耦合变量$\mathcal{A}_g$以最小化目标函数。评估一组耦合变量实例需要运行每个学科子问题，通常并行进行。

学科子问题在优化过程中可能会偏离耦合变量。当两个或多个学科对某个变量意见不一致，或者子问题约束阻止匹配系统级优化器设定的目标值时，就会出现这种偏差。每个学科顶层约束 $f_i = 0$ 确保了最终实现耦合。

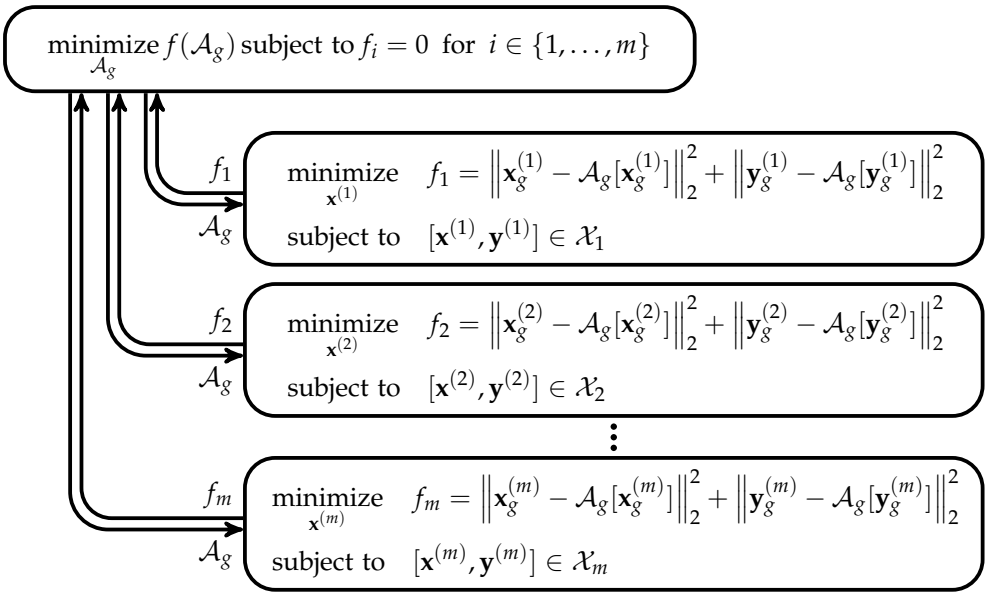


图21.12. 协作优化的设计架构。

协同优化的主要优势源于其能够将某些设计变量隔离到学科子问题中。协同优化易于应用于现实世界的多学科问题求解，因为每个学科通常是高度分离的，因此很大程度上不受其他学科中微小决策的影响。去中心化的表述允许应用传统的学科优化方法，使问题设计者能够利用现有工具和方法论。

协同优化需要在耦合变量上进行优化，这包括设计变量和响应变量。由于额外的耦合变量可能超过局部优化的好处，协同优化在高耦合问题中表现不佳。

协同优化是一种*distributed architecture*，它将单个优化问题分解为一组较小的优化问题，当它们的解组合在一起时，具有相同的解。分布式架构的优点是可以缩短求解时间，因为子问题可以并行优化。

在例21.6中，协同优化被应用于拼车问题。

协同优化架构可以通过生成六个不同的学科子问题来应用于车辆路径问题。不幸的是，拥有六个不同的子问题要求任何跨学科共享的变量都需要在全局层面进行优化。

图21.13展示了通过将车辆、传感器和自主性学科归为运输子问题，并将路线规划、需求和利润学科归为网络子问题而得到的两个学科子问题。归入每个子问题的学科紧密耦合。仅有两个子问题显著减少了系统级优化器考虑的全局变量数量，因为据推测，直接用于运输和网络两个子问题的设计变量非常少。

这些子问题各自都是多学科优化问题，本身就可以利用本章所涵盖的技术进行优化。例如，我们可以在运输子问题中采用顺序优化。我们还可以在网络子问题中增设一个协作优化的实例。

示例 21.6。将协同优化应用于拼车问题。

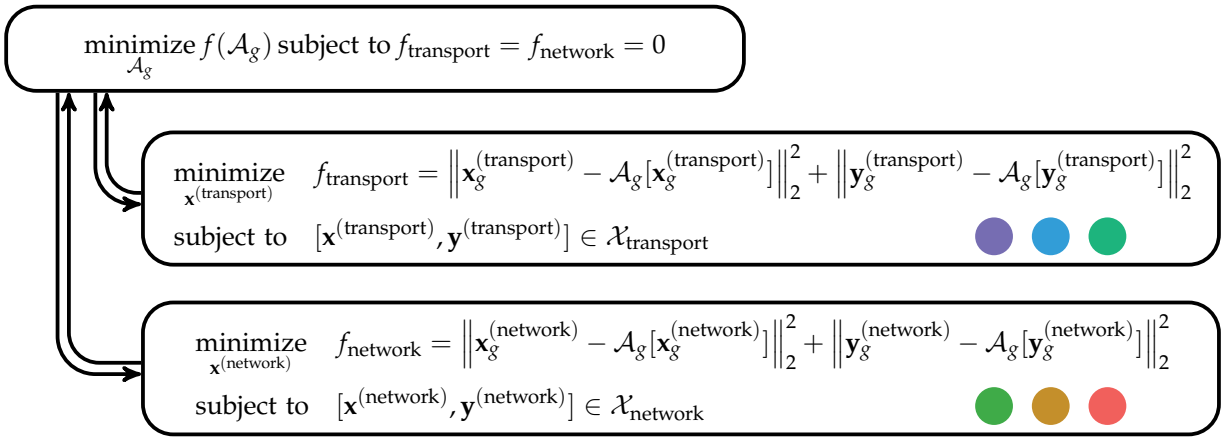


图21.13. 应用于拼车问题的协作优化架构。彩色圆圈对应于每个子问题中包含的学科分析。

21.8 Simultaneous Analysis and Design

simultaneous analysis and design (SAND) 架构通过让优化器执行分析，避免了在多个学科分析之间进行协调的核心挑战。SAND 不运行分析 $F_i(\mathcal{A})$ 来获得残差，而是在约束 $F_i(\mathcal{A}) = \mathcal{A}$ 下同时优化设计变量和响应变量。优化器负责同时优化设计变量并找到相应的响应变量。

任何学科分析都可以转化为 *residual form*。残差 $r_i(\mathcal{A})$ 用于表示作业 $\mathcal{A}$ 是否与第 i 个学科兼容。如果 $F_i(\mathcal{A}) = \mathcal{A}$，则 $r_i(\mathcal{A}) = 0$；否则，$r_i(\mathcal{A}) \neq 0$。我们可以利用学科分析得到一个残差形式：

$$r_i(\mathcal{A}) = \left\| F_i(\mathcal{A}) - \mathcal{A}[\mathbf{y}^{(i)}] \right\| \tag{21.9}$$

尽管这通常效率低下，如例21.7所示。

考虑一种求解方程 $\mathrm{A}\mathrm{y} = \mathrm{x}$ 的学科分析。该分析为 $F(\mathrm{x}) = \mathrm{A}^{-1}\mathrm{x}$，这需要进行昂贵的矩阵求逆。我们可以利用方程（21.9）构造一个残差形式：

$$r_1(\mathbf{x}, \mathbf{y}) = \|F(\mathbf{x}) - \mathbf{y}\| = \left\| \mathbf{A}^{-1}\mathbf{x} - \mathbf{y} \right\|$$

或者，我们可以利用原始约束来构造更高效的残差形式：

$$r_2(\mathbf{x}, \mathbf{y}) = \|\mathbf{A}\mathbf{y} - \mathbf{x}\|$$

示例21.7。在残差中评估学科分析通常是适得其反的。分析通常必须执行额外的工作来解决问题，而更清晰的残差形式可以更有效地验证输入是否兼容。

学科的残差形式由该学科分析所求解的一组学科方程组成。[10] 通常，评估个

—在空气动力学中，这些可能包括纳维-斯托克斯方程。在结构工程中，这些可能包括弹性方程。在电气工程中，这些可能包括电流流动的微分方程。

而不是进行学科分析。在SAND中，图21.14显示，分析工作由优化器负责。

$$\underset{\mathcal{A}}{\text{最小化}} \quad f(\mathcal{A})，\text{满足 } \mathcal{A} \in \mathcal{X}、r_i(\mathcal{A}) = 0，\text{对于每个学科}$$

图21.14。同步分析与设计将全部负担放在优化器上。它使用学科残差而非学科分析。

SAND 可以探索相对于残差方程不可行的设计空间区域，如图 21.15 所示。探索不可行区域

可以让我们更容易地遍历设计空间，并在与起始设计点的可行区域不相连的可行区域中找到解决方案。SAND 的缺点在于必须同时优化大量变量，而这些变量的导数和其他学科特定的专业知识并不容易获得。此外，SAND 的许多价值来自残差，而残差的计算可以比学科分析更高效。在实际应用中，SAND 的使用往往受到限制，因为无法修改现有的学科分析代码以生成高效的残差形式。

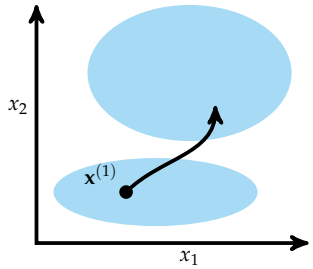


图21.15。SAND能够探索设计空间中的不可行区域，并可能弥合可行子集之间的差距。

SAND被应用于示例21.8中的拼车问题。

将同时分析与设计架构应用于拼车问题需要学科残差。这些残差可能依赖于所有设计变量和响应变量。该架构要求优化器优化所有设计变量和所有响应变量。

$$
\begin{aligned}
\underset{\mathbf{v},\mathbf{s},\mathbf{r},\mathbf{p},\mathbf{y}^{(v)},\mathbf{y}^{(s)},\mathbf{y}^{(a)},\mathbf{y}^{(r)},\mathbf{y}^{(d)},\mathbf{y}^{(p)}}{\text{minimize}} \quad & f\left(\mathbf{v},\mathbf{s},\mathbf{r},\mathbf{p},\mathbf{y}^{(v)},\mathbf{y}^{(s)},\mathbf{y}^{(a)},\mathbf{y}^{(r)},\mathbf{y}^{(d)},\mathbf{y}^{(p)}\right) \\
\text{subject to} \quad & \left[\mathbf{v},\mathbf{s},\mathbf{r},\mathbf{p},\mathbf{y}^{(v)},\mathbf{y}^{(s)},\mathbf{y}^{(a)},\mathbf{y}^{(r)},\mathbf{y}^{(d)},\mathbf{y}^{(p)}\right] \in \mathcal{X} \\
& r_v(\mathbf{v},\mathbf{s},\mathbf{r},\mathbf{p},\mathbf{y}^{(v)},\mathbf{y}^{(s)},\mathbf{y}^{(a)},\mathbf{y}^{(r)},\mathbf{y}^{(d)},\mathbf{y}^{(p)}) = 0 \\
& r_s(\mathbf{v},\mathbf{s},\mathbf{r},\mathbf{p},\mathbf{y}^{(v)},\mathbf{y}^{(s)},\mathbf{y}^{(a)},\mathbf{y}^{(r)},\mathbf{y}^{(d)},\mathbf{y}^{(p)}) = 0 \\
& r_a(\mathbf{v},\mathbf{s},\mathbf{r},\mathbf{p},\mathbf{y}^{(v)},\mathbf{y}^{(s)},\mathbf{y}^{(a)},\mathbf{y}^{(r)},\mathbf{y}^{(d)},\mathbf{y}^{(p)}) = 0 \\
& r_r(\mathbf{v},\mathbf{s},\mathbf{r},\mathbf{p},\mathbf{y}^{(v)},\mathbf{y}^{(s)},\mathbf{y}^{(a)},\mathbf{y}^{(r)},\mathbf{y}^{(d)},\mathbf{y}^{(p)}) = 0 \\
& r_d(\mathbf{v},\mathbf{s},\mathbf{r},\mathbf{p},\mathbf{y}^{(v)},\mathbf{y}^{(s)},\mathbf{y}^{(a)},\mathbf{y}^{(r)},\mathbf{y}^{(d)},\mathbf{y}^{(p)}) = 0 \\
& r_p(\mathbf{v},\mathbf{s},\mathbf{r},\mathbf{p},\mathbf{y}^{(v)},\mathbf{y}^{(s)},\mathbf{y}^{(a)},\mathbf{y}^{(r)},\mathbf{y}^{(d)},\mathbf{y}^{(p)}) = 0
\end{aligned}
$$

示例21.8. 应用于拼车问题的同步分析与设计架构。

21.9 *Summary*

- 多学科设计优化需要对多个学科进行推理，并在耦合变量之间达成一致。
- 学科分析通常可以排序以最小化依赖循环。

- 多学科设计问题可以构建为不同的体系结构，利用问题特征来改进优化过程。
- 多学科设计可行架构通过使用缓慢且可能不收敛的多学科设计分析来保持可行性和兼容性。
- 顺序优化允许每个学科优化其学科特定的变量，但并不总能得到最优设计。
- 个体学科可行架构允许并行执行分析，代价是向全局优化器添加耦合变量。
- 协作优化纳入了能够利用领域专业化局部优化某些变量的子优化器。
- 同时分析与设计架构用残差替代设计分析，使优化器能够找到兼容解，但无法直接使用学科求解技术。

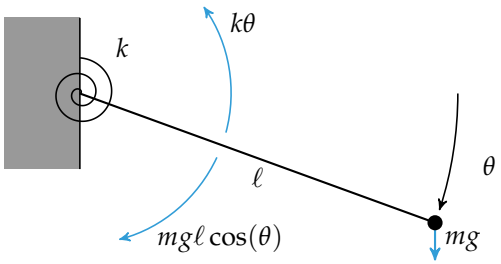
21.10 *Exercises*

练习 21.1. 提供一个多学科的实际工程问题的例子。

练习21.2。提供一个多学科问题的抽象示例，其中分析顺序很重要。

练习 21.3. 个体学科可行架构相比多学科设计可行架构和顺序优化架构，其优势是什么？

练习 21.4。考虑将多学科设计分析应用于最小化机翼的重量，其变形和载荷由不同的学科计算。我们将使用该问题的一个简化版本，将机翼表示为一个由扭转弹簧支撑的水平安装的摆。



目标是最小化弹簧刚度 k，使得摆的位移不超过目标阈值。摆长 ℓ、摆点质量 m 和重力常数 g 是固定的。

我们使用两种简化分析来代替用于计算飞机机翼变形和载荷的更复杂分析。假设摆是刚性的，则加载力矩M等于$mg\ell\cos(\theta)$。扭转弹簧抵抗变形，使得摆的角位移θ为M/k。

在多学科设计可行架构下表述弹簧摆问题，然后按照该架构解决以下问题：

$$m = 1\,\text{kg}, \ell = 1\,\text{m}, g = 9.81\,\text{m}\,\text{s}^{-2}, \theta_{\text{max}} = 10\,\text{rad}$$

练习 21.5。在个体设计可行架构下，构建弹簧摆问题。

练习 21.6。在协同优化架构下表述弹簧摆问题。给出两个学科优化问题和系统级优化问题。

A Julia

Julia 是一门免费且开源的科學程式語言。[1] 它是一
相对较新的语言，从Python、MATLAB和R等语言中汲取灵感。本书选择使用它，是因为它足够高级[2]，使得算法能够紧凑表达且可读，同时

而且速度很快。本书与Julia 1.0版本兼容。本附录介绍了理解文中代码所需的概念。

1 Julia 可从 `http://julialang.org` 获取。

2 与 C++ 等语言不同，Julia 不要求程序员操心内存管理及其他底层细节。

A.1 Types

Julia 有多种基本类型，可以表示真值、数字、字符串、数组、元组和字典等数据。用户还可以定义自己的类型。本节解释如何使用一些基本类型以及定义新类型。

A.1.1 Booleans

Julia 中的 *Boolean* 类型，写作 `Bool`，包含值 **`true`** 和 **`false`**。我们可以将这些值赋给变量。变量名可以是任意字符组成的字符串，包括 Unicode，但有一些限制。

```
done = false
α = false
```

等号左侧是变量名，右侧是值。

我们可以在Julia控制台中进行赋值。控制台会对被求值的表达式返回一个响应。

```
julia> x = true
true
julia> y = false
false
julia> typeof(x)
Bool
```

支持标准布尔运算符。

```
julia> !x     # not
false
julia> x && y # and
false
julia> x || y # or
true
```

符号表示该行的其余部分是注释，不应被求值。

A.1.2 Numbers

巨lia 支持整数和浮点数，如下所示 n 此处

```
julia> typeof(42)
Int64
julia> typeof(42.0)
Float64
```

这里，`Int64` 表示一个64位整数，而`Float64`表示一个64位浮点值。[3] 我们还可以执行标准的数学运算：

[3] On 32-bit machines, an integer literal like 42 is interpreted as an `Int32`.

```
julia> x = 4
4
julia> y = 2
2
julia> x + y
6
julia> x - y
2
julia> x * y
8
julia> x / y
2.0
julia> x ^ y
16
```

```
julia> x % y # x modulo y
0
```

请注意，`x / y` 的结果是一个 `Float64`，即使 `x` 和 `y` 是整数。我们也可以在执行赋值的同时进行这些操作。例如，`x += 1` 是 `x = x + 1` 的简写。

我们也可以进行比较：

```
julia> 3 > 4
false
julia> 3 >= 4
false
julia> 3 ≥ 4   # unicode also works
false
julia> 3 < 4
true
julia> 3 <= 4
true
julia> 3 ≤ 4   # unicode also works
true
julia> 3 == 4
false
julia> 3 < 4 < 5
true
```

A.1.3 Strings

string 是字符数组。在本教材中，除了报告某些错误外，字符串并不常用。可以使用 " 个字符来构造 `String` 类型的对象。例如：

```
julia> x = "optimal"
"optimal"
julia> typeof(x)
String
```

A.1.4 Vectors

vector 是一维数组，用于存储一系列值。我们可以使用方括号构造向量，用逗号分隔元素。这些示例中的分号用于抑制输出。

```
julia> x = [];                      # empty vector
julia> x = trues(3);                # Boolean vector containing three trues
julia> x = ones(3);                 # vector of three ones
julia> x = zeros(3);                # vector of three zeros
julia> x = rand(3);                 # vector of three random numbers between 0 and 1
julia> x = [3, 1, 4];               # vector of integers
julia> x = [3.1415, 1.618, 2.7182]; # vector of floats
```

一个 *array comprehension* 可以用来创建向量。下面，我们使用 `print` 使输出水平打印的函数。

```
julia> print([sin(x) for x = 1:5])
[0.841471, 0.909297, 0.14112, -0.756802, -0.958924]
```

我们可以检查向量的类型:

```
julia> typeof([3, 1, 4])              # 1-dimensional array of Int64s
Array{Int64,1}
julia> typeof([3.1415, 1.618, 2.7182]) # 1-dimensional array of Float64s
Array{Float64,1}
```

我们使用方括号对向量进行索引。

```
julia> x[1]       # first element is indexed by 1
3.1415
julia> x[3]       # third element
2.7182
julia> x[end]     # use end to reference the end of the array
2.7182
julia> x[end - 1] # this returns the second to last element
1.618
```

我们可以从数组中提取一定范围的元素。范围使用冒号表示法指定。

```
julia> x = [1, 1, 2, 3, 5, 8, 13];
julia> print(x[1:3])      # pull out the first three elements
[1, 1, 2]
julia> print(x[1:2:end])  # pull out every other element
[1, 2, 5, 13]
julia> print(x[end:-1:1]) # pull out all the elements in reverse order
[13, 8, 5, 3, 2, 1, 1]
```

我们可以对数组执行各种不同的操作。函数名末尾的感叹号通常用于表示该函数会改变（即修改）输入。

```
julia> print([x, x])             # concatenation
Array{Int64,1}[[1, 1, 2, 3, 5, 8, 13], [1, 1, 2, 3, 5, 8, 13]]
julia> length(x)
7
julia> print(push!(x, -1))       # add an element to the end
[1, 1, 2, 3, 5, 8, 13, -1]
julia> pop!(x)                   # remove an element from the end
-1
julia> print(append!(x, [2, 3])) # append y to the end of x
[1, 1, 2, 3, 5, 8, 13, 2, 3]
julia> print(sort!(x))           # sort the elements in the vector
[1, 1, 2, 2, 3, 3, 5, 8, 13]
julia> x[1] = 2; print(x)        # change the first element to 2
[2, 1, 2, 2, 3, 3, 5, 8, 13]
julia> x = [1, 2];
julia> y = [3, 4];
julia> print(x + y)              # add vectors
[4, 6]
julia> print(3x - [1, 2])        # multiply by a scalar and subtract
[2, 4]
julia> print(dot(x, y))          # dot product
11
julia> print(x·y)                # dot product using unicode character
11
```

将各种函数逐元素应用于向量通常是很有用的。

```
julia> print(x .* y)   # elementwise multiplication
[3, 8]
julia> print(x .^ 2)   # elementwise squaring
[1, 4]
julia> print(sin.(x))  # elementwise application of sin
[0.841471, 0.909297]
julia> print(sqrt.(x)) # elementwise application of sqrt
[1.0, 1.41421]
```

A.1.5 Matrices

matrix 是二维数组。与向量类似，它使用方括号构造。我们用空格分隔同一行中的元素，用分号分隔行。我们也可以对矩阵进行索引，并使用范围输出子矩阵。

```
julia> X = [1 2 3; 4 5 6; 7 8 9; 10 11 12];
julia> typeof(X)         # a 2-dimensional array of Int64s
Array{Int64,2}
julia> X[2]              # second element using column-major ordering
4
julia> X[3,2]            # element in third row and second column
8
julia> print(X[1,:])     # extract the first row
[1, 2, 3]
julia> print(X[:,2])     # extract the second column
[2, 5, 8, 11]
julia> print(X[:,1:2])   # extract the first two columns
[1 2; 4 5; 7 8; 10 11]
julia> print(X[1:2,1:2]) # extract a 2x2 matrix from the top left of x
[1 2; 4 5]
```

我们还可以构造各种特殊矩阵，并使用数组推导式：

```
julia> print(Matrix(1.0I, 3, 3))          # 3x3 identity matrix
[1.0 0.0 0.0; 0.0 1.0 0.0; 0.0 0.0 1.0]
julia> print(Matrix(Diagonal([3, 2, 1]))) # 3x3 diagonal matrix with 3, 2, 1 on diagonal
[3 0 0; 0 2 0; 0 0 1]
julia> print(rand(3,2))                   # 3x2 random matrix
[0.000281914 0.388884; 0.543776 0.263469; 0.337295 0.0481282]
julia> print(zeros(3,2))                  # 3x2 matrix of zeros
[0.0 0.0; 0.0 0.0; 0.0 0.0]
julia> print([sin(x + y) for x = 1:3, y = 1:2]) # array comprehension
[0.909297 0.14112; 0.14112 -0.756802; -0.756802 -0.958924]
```

矩阵运算包括以下内容：

```
julia> print(X')       # complex conjugate transpose
[1 4 7 10; 2 5 8 11; 3 6 9 12]
julia> print(3X .+ 2)  # multiplying by scalar and adding scalar
[5 8 11; 14 17 20; 23 26 29; 32 35 38]
julia> X = [1 3; 3 1]; # create an invertible matrix
julia> print(inv(X))   # inversion
[-0.125 0.375; 0.375 -0.125]
julia> det(X)          # determinant
-8.0
julia> print([X X])    # horizontal concatenation
[1 3 1 3; 3 1 3 1]
julia> print([X; X])   # vertical concatenation
[1 3; 3 1; 1 3; 3 1]
```

```
julia> print(sin.(X))  # elementwise application of sin
[0.841471 0.14112; 0.14112 0.841471]
```

A.1.6 Tuples

tuple 是一个有序的值列表，这些值可能具有不同的类型。它们通过括号构造。它们类似于数组，但不可变。

```
julia> x = (1,) # a single element tuple indicated by the trailing comma
(1,)
julia> x = (1, 0, [1, 2],  2.5029, 4.6692) # third element is a vector
(1, 0, [1, 2], 2.5029, 4.6692)
julia> x[2]
0
julia> x[end]
4.6692
julia> x[4:end]
(2.5029, 4.6692)
julia> length(x)
5
```

A.1.7 Dictionaries

dictionary 是键值对的集合。键值对用双箭头运算符表示。我们可以像数组和元组一样使用方括号对字典进行索引。

```
julia> x = Dict(); # empty dictionary
julia> x[3] = 4 # associate value 4 with key 3
4
julia> x = Dict(3=>4, 5=>1) # create a dictionary with two key-value pairs
Dict{Int64,Int64} with 2 entries:
  3 => 4
  5 => 1
julia> x[5]         # return value associated with key 5
1
julia> haskey(x, 3) # check whether dictionary has key 3
true
julia> haskey(x, 4) # check whether dictionary has key 4
false
```

A.1.8 Composite Types

composite type 是具名字段的集合。默认情况下，复合类型的实例是不可变的（即它不能改变）。我们使用 **`struct`** 关键字，然后给新类型一个名称并列出字段的名称。

```
struct A
    a
    b
end
```

添加关键字 {v*} 使得实例可以更改。

```
mutable struct B
    a
    b
end
```

复合类型使用括号构造，我们在括号之间为不同字段传入值。例如，

```
x = A(1.414, 1.732)
```

双冒号运算符可用于为字段标注类型。

```
struct A
    a::Int64
    b::Float64
end
```

此注解要求我们为第一个字段传入 `Int64`，为第二个字段传入 `Float64`。为了简洁，本文未使用类型注解，但这以性能为代价。类型注解允许 Julia 提高运行时性能，因为编译器可以针对特定类型优化底层代码。

A.1.9 Abstract Types

到目前为止，我们已经讨论了*concrete types*，这是我们能够构造的类型。然而，具体类型只是类型层次的一部分。还存在*abstracttypes types*，它们是具体类型和其他抽象类型的超类型。

我们可以使用 `supertype` 和 `subtype` 函数来探索图 A.1 中显示的 `Float64` 类型的类型层次结构。

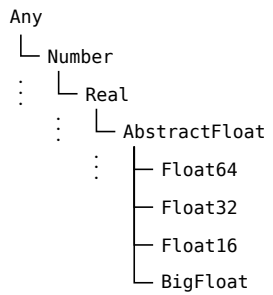


图 A.1. `Float64` 类型的类型层次结构。

```
julia> supertype(Float64)
AbstractFloat
julia> supertype(AbstractFloat)
Real
julia> supertype(Real)
Number
julia> supertype(Number)
Any
julia> supertype(Any)          # Any is at the top of the hierarchy
Any
julia> subtypes(AbstractFloat) # different types of AbstractFloats
4-element Array{Any,1}:
 BigFloat
 Float16
 Float32
 Float64
julia> subtypes(Float64)       # Float64 does not have any subtypes
0-element Array{Type,1}
```

我们可以定义自己的抽象类型。

```
abstract type C end
abstract type D <: C end # D is an abstract subtype of C
struct E <: D # E is composite type that is a subtype of D
    a
end
```

A.1.10 Parametric Types

Julia 支持 *parametric types*，这是一种可以接受参数的类型的参数。我们在字典示例中已经见过参数化类型。

```
julia> x = Dict(3=>4, 5=>1)
Dict{Int64,Int64} with 2 entries:
  3 => 4
  5 => 1
```

这将构造一个`Dict{Int64,Int64}`。参数化类型的参数列在花括号内，并以逗号分隔。对于字典类型，第一个参数指定键的类型，第二个参数指定值的类型。Julia能够根据输入推断出这一点，但我们本可以明确指定。

```
julia> x = Dict{Int64,Int64}(3=>4, 5=>1)
Dict{Int64,Int64} with 2 entries:
  3 => 4
  5 => 1
```

可以定义我们自己的参数化类型，但本文不这样做。

A.2 Functions

一个 *function* 是一种将参数值的元组映射到返回值的对象。本节讨论如何定义和使用函数。

A.2.1 Named Functions

定义 *named function* 的一种方法是使用 **function** 关键字，后跟函数名称和参数名称的元组。

```
function f(x, y)
    return x + y
end
```

我们也可以使用赋值形式简洁地定义函数。

```
julia> f(x, y) = x + y;
julia> f(3, 0.1415)
3.1415
```

A.2.2 Anonymous Functions

一个 *anonymous function* 没有被赋予名称，尽管它可以被赋值给一个命名的变量。定义匿名函数的一种方法是使用箭头运算符。

```
julia> h = x -> x^2 + 1 # assign anonymous function to a variable
#1 (generic function with 1 method)
julia> g(f, a, b) = [f(a), f(b)]; # applies function f to a and b and returns array
julia> g(h, 5, 10)
2-element Array{Int64,1}:
  26
 101
julia> g(x->sin(x)+1, 10, 20)
2-element Array{Float64,1}:
 0.4559788891106302
 1.9129452507276277
```

A.2.3 Optional Arguments

我们可以通过设置默认值来指定可选参数。

```
julia> f(x = 10) = x^2;
julia> f()
100
julia> f(3)
9
julia> f(x, y, z = 1) = x*y + z;
julia> f(1, 2, 3)
5
julia> f(1, 2)
3
```

A.2.4 Keyword Arguments

使用关键字参数定义的函数使用分号。

```
julia> f(; x = 0) = x + 1;
julia> f()
1
julia> f(x = 10)
11
julia> f(x, y = 10; z = 2) = (x + y)*z;
julia> f(1)
22
julia> f(2, z = 3)
36
julia> f(2, 3)
10
julia> f(2, 3, z = 1)
5
```

A.2.5 Function Overloading

可以使用双冒号操作符来指定传递给函数的参数类型。如果提供了多个同名函数，Julia 将执行相应的函数。

```
julia> f(x::Int64) = x + 10;
julia> f(x::Float64) = x + 3.1415;
julia> f(1)
11
julia> f(1.0)
4.141500000000001
julia> f(1.3)
4.4415000000000004
```

将使用最具体函数的实现。

```
julia> f(x) = 5;
julia> f(x::Float64) = 3.1415;
julia> f([3, 2, 1])
5
julia> f(0.00787499699)
3.1415
```

A.3 Control Flow

我们可以使用条件求值和循环来控制程序的流程。本节提供了书中使用的一些语法。

A.3.1 Conditional Evaluation

条件求值将检查布尔表达式的值，然后执行相应的代码块。最常见的方法之一是使用**if**语句。

```
if x < y
    # run this if x < y
elseif x > y
    # run this if x > y
else
    # run this if x == y
end
```

我们也可以使用*ternary operator*及其问号和冒号语法。它会在问号之前检查布尔表达式。如果表达式的值为真，则返回冒号之前的内容；否则返回冒号之后的内容。

```
julia> f(x) = x > 0 ? x : 0;
julia> f(-10)
0
julia> f(10)
10
```

A.3.2 Loops

loop 允许重复求值表达式。循环的一种类型是 while 循环。它重复求值一个表达式块，直到满足 **`while`** 关键字之后指定的条件。以下示例将对数组 `x` 中的值求和。

```
x = [1, 2, 3, 4, 6, 8, 11, 13, 16, 18]
s = 0
while x != []
    s += pop!(x)
end
```

另一种循环类型是for循环。它使用**`for`**关键字。以下示例也将对数组`x`中的值进行求和，但不会修改`x`。

```
x = [1, 2, 3, 4, 6, 8, 11, 13, 16, 18]
s = 0
for i = 1:length(x)
    s += x[i]
end
```

`=` 可以替换为 **`in`** 或 `∈`。以下代码块是等价的。

```
x = [1, 2, 3, 4, 6, 8, 11, 13, 16, 18]
s = 0
for y in x
    s += y
end
```

A.4 Packages

package 是 Julia 代码及可能包含的其他外部库的集合，可通过导入来提供额外功能。Julia 内置了包管理器。已注册包的列表可在 `https://pkg.julialang.org` 找到。要添加像 `Distributions.jl` 这样的已注册包，我们可以运行：

```
Pkg.add("Distributions")
```

要更新软件包，我们使用：

```
Pkg.update()
```

要使用一个包，我们使用关键字 **using**：

```
using Distributions
```

本文中的几个代码块指定了包导入及 **using**。一些代码块使用了未显式导入的函数。例如，var 函数由 Statistics.jl 提供，而黄金比例 φ 定义在 Base.MathConstants.jl 中。其他未包含的包有 InteractiveUtils.jl、Iterators.jl、LinearAlgebra.jl、QuadGK.jl、Random.jl 和 StatsBase.jl 。

B Test Functions

优化领域的研究人员使用多种*test functions*来评估优化算法。本节涵盖了本书中使用的若干测试函数。

B.1 Ackley's Function

Ackley's function (图 B.1) 用于测试方法陷入局部最小值的敏感性。它由两个主要部分组成——一个产生大量局部最小值的正弦分量，以及一个以原点为中心的指数钟形曲线，该曲线确定了函数的全局最小值。

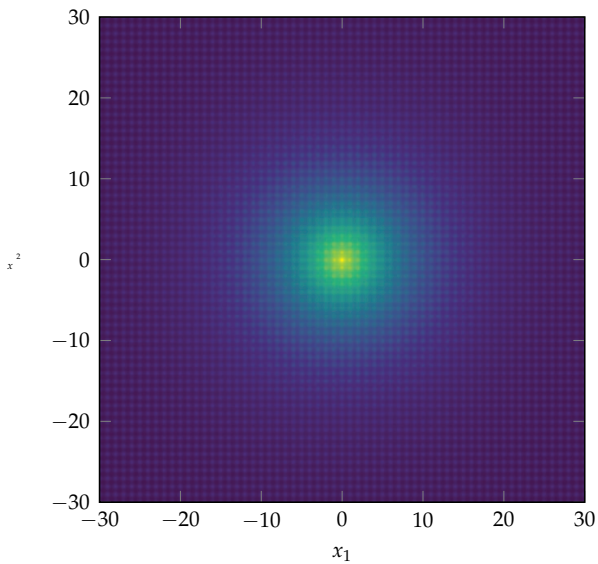


图 B.1. Ackley 函数的二维版本。全局最小值位于原点。

Ackley 函数定义于任意维度 d:

$$f(\mathbf{x}) = -a\exp\left(-b\sqrt{\frac{1}{d}\sum_{i=1}^{d}x_i^2}\right) - \exp\left(\frac{1}{d}\sum_{i=1}^{d}\cos(cx_i)\right) + a + \exp(1) \quad \text{(B.1)}$$

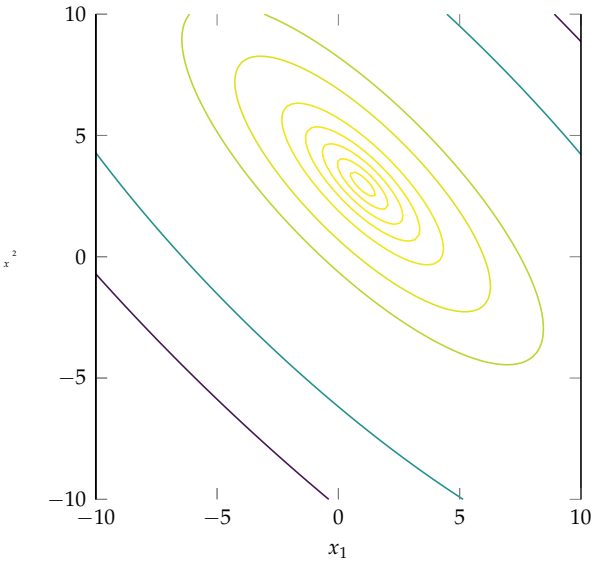
在原点处有一个全局最小值，其最优值为零。通常，$a = 20$，$b = 0.2$，且 $c = 2\pi$。Ackley 函数在算法 B.1 中实现。

```
function ackley(x, a=20, b=0.2, c=2π)
    d = length(x)
    return -a*exp(-b*sqrt(sum(x.^2)/d)) -
              exp(sum(cos.(c*xi) for xi in x)/d) + a +
end
```

算法 B.1。Ackley 函数，具有 d 维输入向量 x 和三个可选参数。

B.2 Booth's Function

Booth's function (图B.2)是一个二维二次函数。



图B.2. Booth函数，在[1, 3]处具有全局最小值。

其方程由下式给出

$$f(\mathbf{x}) = (x_1 + 2x_2 - 7)^2 + (2x_1 + x_2 - 5)^2 \tag{B.2}$$

在[1, 3]处有一个全局最小值，其最优值为零。这在算法B.2中实现。

```
booth(x) = (x[1]+2x[2]-7)^2 + (2x[1]+x[2]-5)^2
```

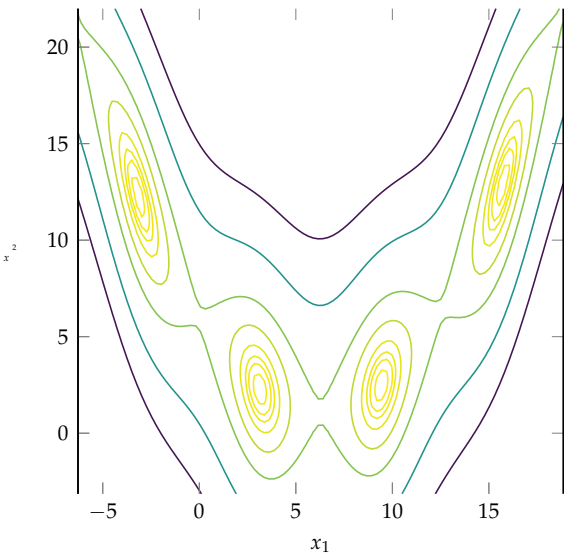
算法B.2. 布思函数，输入为二维向量x。

B.3 Branin Function

Branin function (图 B.3) 是一个二维函数,

$$f(\mathbf{x}) = a(x_2 - bx_1^2 + cx_1 - r)^2 + s(1-t)\cos(x_1) + s \tag{B.3}$$

使用推荐值$a = 1$、$b = 5.1/(4\pi^2)$、$c = 5/\pi$、$r = 6$、$s = 10$，以及$t = 1/(8\pi)$。



图B.3. Branin函数，具有四个全局最小值。

除全局最小值外，它没有局部最小值，其中 $x_1 = \pi + 2\pi m$ 对于整数 m。这些最小值中有四个是：

$$\left\{ \begin{bmatrix} -\pi \\ 12.275 \end{bmatrix}, \begin{bmatrix} \pi \\ 2.275 \end{bmatrix}, \begin{bmatrix} 3\pi \\ 2.475 \end{bmatrix}, \begin{bmatrix} 5\pi \\ 12.875 \end{bmatrix} \right\} \tag{B.4}$$

使用 $f(\mathbf{x}^*) \approx 0.397887$。它在算法 B.3 中实现。

```
function branin(x; a=1, b=5.1/(4π^2), c=5/π, r=6, s=10, t=1/(8π))
    return a*(x[2]-b*x[1]^2+c*x[1]-r)^2 + s*(1-t)*cos(x[1]) + s
end
```

算法 B.3. 具有二维输入向量 x 和六个可选参数的 Branin 函数。

B.4 Flower Function

flower function (图 B.4) 是一个二维测试函数，其等值线函数具有从原点出发的花瓣状结构。

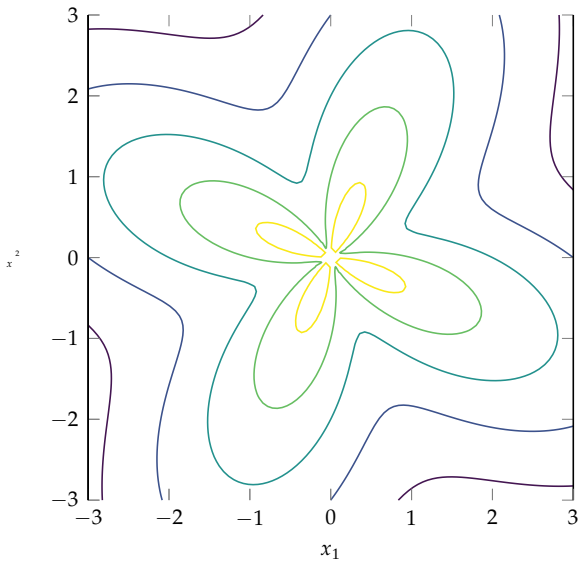


Figure B.4. The flower function.

方程是

$$f(\mathbf{x}) = a\|\mathbf{x}\| + b\sin(c\tan^{-1}(x_2, x_1)) \tag{B.5}$$

其参数通常设置为 $a = 1$、$b = 1$ 和 $c = 4$。

flower 函数在原点附近被最小化，但由于 atan 在 [0, 0] 处未定义，因此没有全局最小值。它是在算法 B.4 中实现的。

```
function flower(x; a=1, b=1, c=4)
    return a*norm(x) + b*sin(c*atan(x[2], x[1]))
end
```

算法B.4. 花函数，具有二维输入向量x和三个可选参数。

B.5 Michalewicz Function

Michalewicz function (图B.5)是一个d维优化函数，包含多个陡峭的谷地。

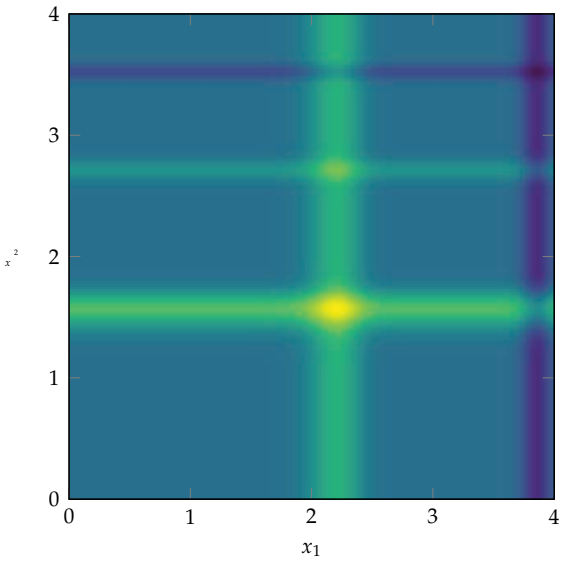


图 B.5。Michalewicz 函数。

它的方程是

$$f(\mathbf{x}) = -\sum_{i=1}^{d} \sin(x_i) \sin^{2m}\left(\frac{ix_i^2}{\pi}\right) \tag{B.6}$$

其中参数 m（通常为 10）控制陡峭程度。全局最小值取决于维数。在二维中，最小值大约位于 [2.20, 1.57]，且 $f(\mathbf{x}^*) = -1.8011$。它在算法 B.5 中实现。

```
function michalewicz(x; m=10)
    return -sum(sin(v)*sin(i*v^2/π)^(2m) for
                (i,v) in enumerate(x))
end
```

算法 B.5. 带有输入向量 x 和可选陡度参数 m 的 Michalewicz 函数。

B.6 Rosenbrock's Banana Function

Rosenbrock function (图B.6)，也称为罗森布罗克山谷或罗森布罗克香蕉函数，是Rosenbrock在1960年提出的一个著名的无约束测试函数。[1] 它在一个长而弯曲的山谷内有一个全局最小值。大多数优化算法在找到山谷方面没有问题，但在沿着山谷穿越到全局最小值时遇到困难。

1 H. H. Rosenbrock，“一种寻找函数最大值或最小值的自动方法”，*The Computer Journal*，第3卷，第3期，第175–184页，1960年。

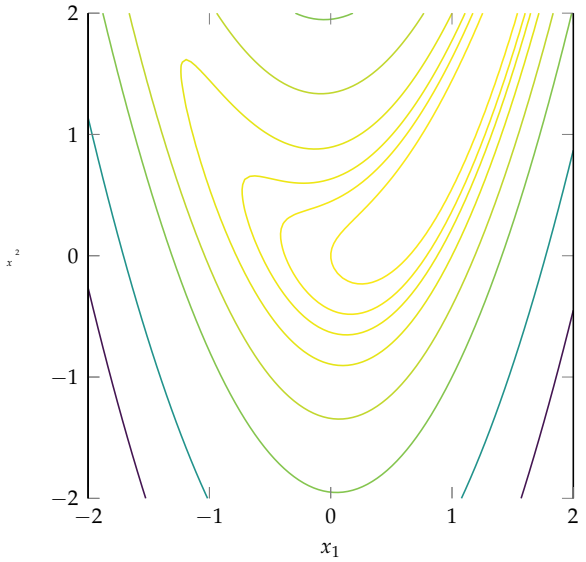


图 B.6. Rosenbrock 函数，其中 $a = 1$ 和 $b = 5$。全局最小值位于 [1, 1]。

罗森布罗克函数是

$$f(\mathbf{x}) = (a - x_1)^2 + b(x_2 - x_1^2)^2 \tag{B.7}$$

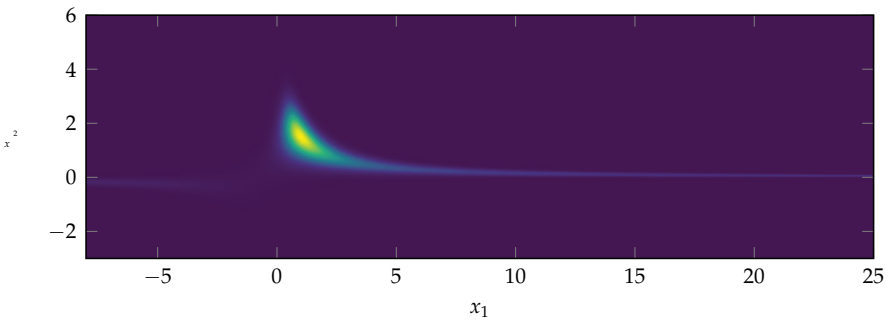
在 $[a, a^2]$ 处具有全局最小值，此时 $f(\mathbf{x}^*) = 0$。此文本使用 $a = 1$ 和 $b = 5$。Rosenbrock 函数在算法 B.6 中实现。

```
rosenbrock(x; a=1, b=5) = (a-x[1])^2 + b*(x[2] - x[1]^2)^2
```

算法 B.6. 具有二维输入向量 x 和两个可选参数的 Rosenbrock 函数。

B.7 Wheeler's Ridge

Wheeler's ridge (图 B.7) 是一个二维函数，在深而弯曲的峰中有一个单一的全局最小值。该函数有两个脊，一个沿正的第一坐标轴，一个沿负的第一坐标轴。梯度下降法会沿负轴脊发散。该函数在远离最优值和脊的地方非常平坦。

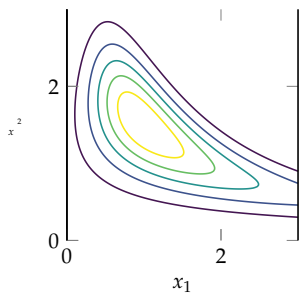


图B.7. 惠勒山脊，显示两个山脊以及包含全局最小值的峰。

该函数由下式给出：

$$f(\mathbf{x}) = -\exp(-(x_1 x_2 - a)^2 - (x_2 - a)^2) \tag{B.8}$$

其中 a 通常等于 1.5，此时 -1 的全局最优解位于 $[1, 3/2]$。



图B.8. 惠勒岭最小区域的等高线图。

```
wheeler(x, a=1.5) = -exp(-(x[1]*x[2] - a)^2 -(x[2]-a)^2)
```

算法B.7。惠勒岭回归，该算法接收一个二维设计点 x 和一个可选的标量参数 a。

Wheeler山脊在$x_1 \in [0, 3]$和$x_2 \in [0, 3]$上评估时具有平滑的等高线图（图B.8）。它实现在算法B.7中。

B.8 Circle Function

算法 B.8 是一个简单的多目标测试函数，由以下给出：

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} 1 - r\cos(\theta) \\ 1 - r\sin(\theta) \end{bmatrix} \tag{B.9}$$

其中 $\theta = x_1$ 和 r 是通过将 x_2 传递通过

$$r = \frac{1}{2} + \frac{1}{2}\left(\frac{2x_2}{1 + x_2^2}\right) \tag{B.10}$$

帕累托前沿具有 $r = 1$ 且 $\text{mod}(\theta, 2\pi) \in [0, \pi/2]$ 或 $r = -1$ 且 $\text{mod}(\theta, 2\pi) \in [\pi, 3\pi/2]$。

```
function circle(x)
    θ = x[1]
    r = 0.5 + 0.5*(2x[2]/(1+x[2]^2))
    y1 = 1 - r*cos(θ)
    y2 = 1 - r*sin(θ)
    return [y1, y2]
end
```

算法B.8。circle函数，接收一个二维设计点x并生成一个二维目标值。

C Mathematical Concepts

本附录涵盖了用于优化方法推导和分析的数学概念。这些概念贯穿本书始终。

C.1 Asymptotic Notation

Asymptotic notation 常用于刻画函数的增长。这种记号有时被称为 *big-Oh notation*，之所以使用字母O，是因为函数的增长率通常被称为其*order*。这种记号可用于描述与数值方法相关的误差，或算法的时间复杂度或空间复杂度。这种记号给出了当自变量趋近于某个值时函数的一个上界。

数学上，如果$f(x) = O(g(x))$ 作为 $x \to a$，那么$f(x)$ 的绝对值被 $g(x)$ 的绝对值乘以某个正且有限的 c 所界定，对于充分接近 a 的 x 值:

$$|f(x)| \leq c|g(x)| \quad \text{for } x \to a \tag{C.1}$$

将$f(x) = O(g(x))$ 写作等式是一种常见的等号滥用。例如，$x^2 = O(x^2)$ 和 $2x^2 = O(x^2)$，但当然，$x^2 \neq 2x^2$。在某些数学文献中，$O(g(x))$ 表示所有增长速度不超过 $g(x)$ 的函数集合。例如，可以写作 $5x^2 \in O(x^2)$。关于渐近记号的一个例子见例 C.1。

如果$f(x)$ 是项的 *linear combination*[1]，则 $O(f)$ 对应于增长最快的项。例C.2比较了几项的阶。

1 线性组合是各项的加权和。如果各项位于向量x中，则线性组合为 $w_1x_1 + w_2x_2 + \cdots = \mathrm{w}^\top \mathrm{x}$。

示例 C.1. 常数乘以函数的渐近符号表示。

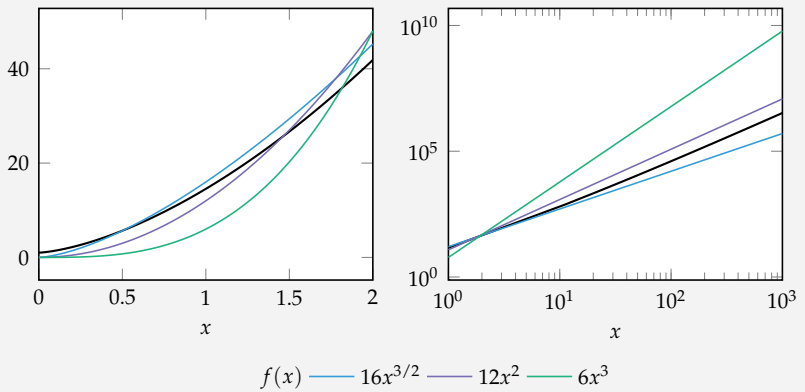
将 $f(x) = 10^6 e^x$ 视为 $x \to \infty$。这里，f 是常数10^6和e^x的乘积。常数可以直接并入边界常数 c 中：

$$\begin{aligned} |f(x)| &\leq c|g(x)| \\ 10^6|e^x| &\leq c|g(x)| \\ |e^x| &\leq c|g(x)| \end{aligned}$$

因此，$f = O(e^x)$ 作为 $x \to \infty$。

示例 C.2。寻找项之线性组合的阶的一个示例。

考虑 $f(x) = \cos(x) + x + 10x^{3/2} + 3x^2$。这里，$f$ 是各项的线性组合。当 x 趋近于无穷大时，各项 $\cos(x)$、x、$x^{3/2}$、x^2 按值递增的顺序排列。我们绘制 $f(x)$ 以及 $c|g(x)|$，其中对于每一项，c 已被选定，使得 $c|g(x = 2)|$ 超过 $f(x = 2)$。



不存在常数 c 使得对于足够大的 x，$f(x)$ 始终小于 $c|x^{3/2}|$。对于 $\cos(x)$ 和 x 也是如此。

我们发现，$f(x) = O(x^3)$，并且通常对于$m \geq 2$有$f(x) = O(x^m)$，以及诸如$f(x) = e^x$之类的其他函数类。我们通常讨论能提供最紧上界的阶。因此，当$x \to \infty$时，$f = O(x^2)$。

C.2 Taylor Expansion

函数的 *Taylor expansion*，也称为 *Taylor series*，对于理解本书涵盖的许多优化方法至关重要，因此我们在此推导它。

从*first fundamental theorem of calculus*、[2]可知

$$f(x+h) = f(x) + \int_0^h f'(x+a)\,da \tag{C.2}$$

将这一定义嵌套起来，就得到关于f的泰勒展开：

$$f(x+h) = f(x) + \int_0^h \left(f'(x) + \int_0^a f''(x+b)\,db \right) da \tag{C.3}$$

$$= f(x) + f'(x)h + \int_0^h \int_0^a f''(x+b)\,db\,da \tag{C.4}$$

$$= f(x) + f'(x)h + \int_0^h \int_0^a \left(f''(x) + \int_0^b f'''(x+c)\,dc \right) db\,da \tag{C.5}$$

$$= f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \int_0^h \int_0^a \int_0^b f'''(x+c)\,dc\,db\,da \tag{C.6}$$

$$\vdots \tag{C.7}$$

$$= f(x) + \frac{f'(x)}{1!}h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \ldots \tag{C.8}$$

$$= \sum_{n=0}^{\infty} \frac{f^{(n)}(x)}{n!}\,h^n \tag{C.9}$$

在上述公式中，x通常是固定的，函数根据h来评估。将$f(x)$在点a处的泰勒展开写为仍为x的函数往往更为方便：

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)^n \tag{C.10}$$

泰勒展开将函数表示为基于单点重复导数的多项式项的无穷和。任何解析函数都可以在其局部邻域内由其泰勒展开表示。

函数可以通过使用泰勒展开的前几项进行局部近似。图C.1展示了$\cos(x)$在$x = 1$附近的近似程度逐渐提高。包含更多项可以提高局部近似的准确性，但随着远离展开点，误差仍然会累积。

2 微积分第一基本定理将一个函数与其导数的积分联系起来：

$$f(b) - f(a) = \int_a^b f'(x)\,dx$$

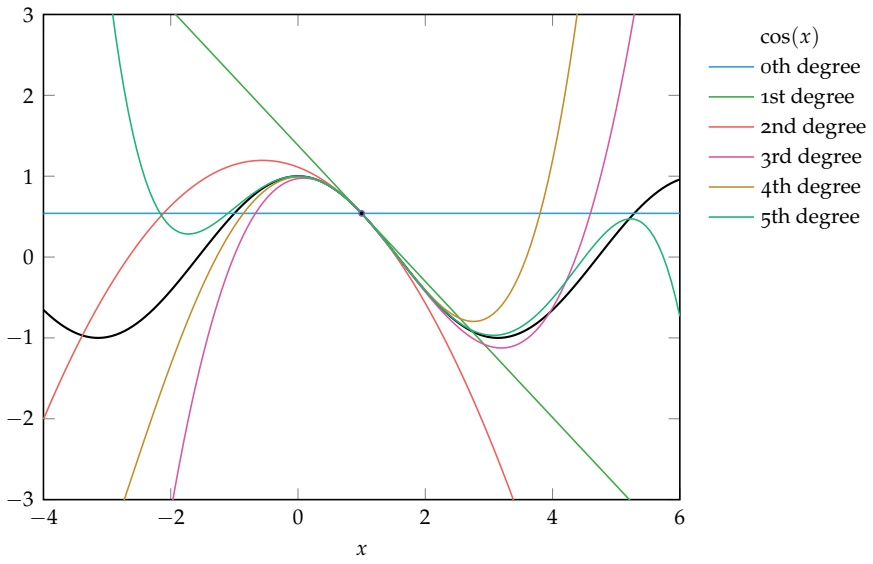


图 C.1。基于泰勒展开的前 n 项，cos(x) 在 1 附近的逐次近似。

一个线性 *Taylor approximation* 使用泰勒展开的前两项：

$$f(x) \approx f(a) + f'(a)(x-a) \tag{C.11}$$

二次泰勒近似使用前三项：

$$f(x) \approx f(a) + f'(a)(x-a) + \frac{1}{2}f''(a)(x-a)^2 \tag{C.12}$$

等等。

在多维情况下，关于a的泰勒展开推广为

$$f(\mathbf{x}) = f(\mathbf{a}) + \nabla f(\mathbf{a})^\top (\mathbf{x}-\mathbf{a}) + \frac{1}{2}(\mathbf{x}-\mathbf{a})^\top \nabla^2 f(\mathbf{a})(\mathbf{x}-\mathbf{a}) + \dots \tag{C.13}$$

前两项构成a点处的切平面。第三项则包含了局部曲率。本文将仅使用这里展示的前三项。

C.3 *Convexity*

两个向量x和y的*convex combination*是结果 的

$$\alpha\mathbf{x} + (1-\alpha)\mathbf{y} \tag{C.14}$$

对于某些 $\alpha \in [0, 1]$。凸组合可以由 m 个向量构成，

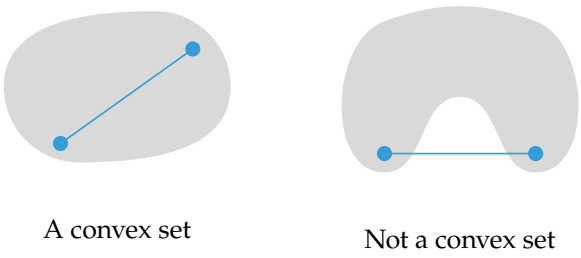
$$w_1\mathbf{v}^{(1)} + w_2\mathbf{v}^{(2)} + \cdots + w_m\mathbf{v}^{(m)} \tag{C.15}$$

使用非负权重w，且其总和为1。

一个 *convex set* 是一个集合，其中任意两点之间绘制的线段完全位于该集合内。数学上，如果满足以下条件，集合 $\mathcal{S}$ 是凸的。

$$\alpha\mathbf{x} + (1 - \alpha)\mathbf{y} \in \mathcal{S}. \tag{C.16}$$

对于所有 $\mathcal{S}$ 中的 x、y 以及所有 [0, 1] 中的 α。图 C.2 展示了一个凸集和一个非凸集。

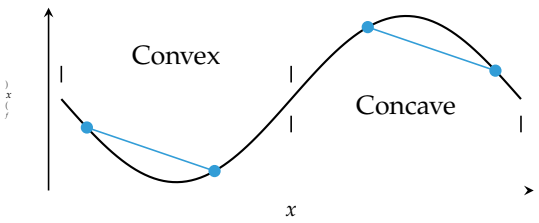


图C.2。凸集和非凸集。

一个 *convex function* 是一个定义域为凸集的 *bowl-shaped* 函数。所谓碗状，是指对于该函数，其定义域内任意两点间所画的直线都不位于函数下方。一个函数 f 在凸集 $\mathcal{S}$ 上是凸的，如果对于 $\mathcal{S}$ 中的所有 x、y 以及 [0, 1] 中的所有 α，

$$f(\alpha\mathbf{x} + (1 - \alpha)\mathbf{y}) \leq \alpha f(\mathbf{x}) + (1 - \alpha)f(\mathbf{y}) \tag{C.17}$$

函数的凸区域和凹区域如图 C.3 所示。



图C.3. 函数的凸和非凸部分。

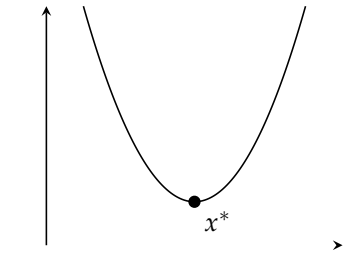
一个函数f在凸集$\mathcal{S}$上是*strictly convex*的，如果对于所有的x，y在$\mathcal{S}$中且α在(0, 1)内，

$$f(\alpha\mathbf{x} + (1 - \alpha)\mathbf{y}) < \alpha f(\mathbf{x}) + (1 - \alpha)f(\mathbf{y}) \tag{C.18}$$

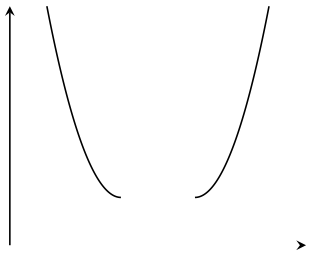
严格凸函数至多有一个最小值，而凸函数可以有平坦区域。[3]严格凸性和非严格凸性的示例如下所示

图 C.4。

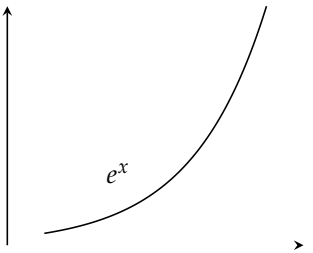
3 凸函数的优化是 S. Boyd 和 L. Vandenberghe 所著教科书 *Convex Optimization* 的主题，剑桥大学出版社，2004年。



严格凸函数具有唯一的全局最小值。



没有唯一全局最小值的凸函数。

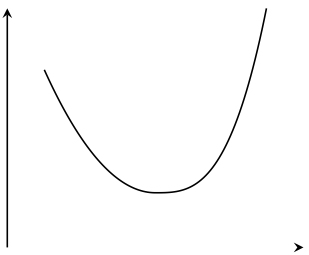


严格凸函数没有全局最小值。

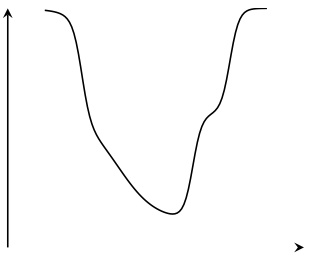
图C.4。并非所有凸函数都具有单一的全局最小值。

如果 $-f$ 是凸的，则函数 f 是 *concave*。此外，如果 $-f$ 是严格凸的，则 f 是 *strictly concave*。

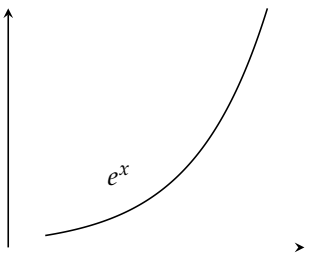
并非所有凸函数都是单峰的，也并非所有单峰函数都是凸的，如图C.5所示。



单峰且凸



单峰但非凸



Convex but nonunimodal

图C.5. 凸性和单峰性不是同一回事。

C.4 Norms

一个 *norm* 是一种为向量分配长度的函数。计算两个向量之间的距离时，我们求这两个向量之差的范数。例如，点 a 与点 b 之间使用 *Euclidean norm* 的距离是

$$\|\mathbf{a}-\mathbf{b}\|_2 = \sqrt{(a_1-b_1)^2+(a_2-b_2)^2+\cdots+(a_n-b_n)^2} \tag{C.19}$$

如果[4]，则函数f是一个范数。

1. $f(\mathrm{x})=0$ 当且仅当 a 是零向量。
2. $f(a\mathrm{x})=|a|f(\mathrm{x})$，使得长度成比例缩放。
3. $f(\mathrm{a}+\mathrm{b})\le f(\mathrm{a})+f(\mathrm{b})$，也称为*triangle inequality*。

L_p 范数是一组常用的范数，由标量 $p\ge 1$ 参数化。方程 (C.19) 中的欧几里得范数是 L_2 范数。表 C.1 中展示了几个 L_p 范数。

L_p 范数的定义如下：

$$\|\mathbf{x}\|_p = \lim_{\rho\to p}(|x_1|^\rho+|x_2|^\rho+\cdots+|x_n|^\rho)^{\frac{1}{\rho}} \tag{C.20}$$

其中该极限对于定义无穷范数是必要的，L_∞。[5]

C.5 Matrix Calculus

本节推导两个常见的梯度：$\nabla_{\mathbf{x}}\mathrm{b}^\top\mathrm{x}$ 和 $\nabla_{\mathbf{x}}\mathrm{x}^\top\mathrm{Ax}$。

为了得到$\nabla_{\mathbf{x}}\mathrm{b}^\top\mathrm{x}$，我们首先展开点积：

$$\mathbf{b}^\top\mathbf{x} = [b_1x_1+b_2x_2+\cdots+b_nx_n] \tag{C.21}$$

关于单个坐标的偏导数是：

$$\frac{\partial}{\partial x_i}\mathbf{b}^\top\mathbf{x} = b_i \tag{C.22}$$

因此，梯度为：

$$\nabla_{\mathbf{x}}\mathbf{b}^\top\mathbf{x} = \nabla_{\mathbf{x}}\mathbf{x}^\top\mathbf{b} = \mathbf{b} \tag{C.23}$$

4 从这些公理中衍生出的一些性质包括：

$$f(-\mathbf{x}) = f(\mathbf{x})$$
$$f(\mathbf{x}) \ge 0$$

5 L_∞ 范数也被称为 *max norm*、*Chebyschev dis-tance* 或 *chessboard distance*。后者名称源于国际象棋国王在两个棋盘方格之间移动所需的最少步数。

L_1: $\|\mathrm{x}\|_1 = |x_1| + |x_2| + \cdots + |x_n|$

L_2: $\|\mathrm{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$

L_∞: $\|\mathrm{x}\|_\infty = \max(|x_1|, |x_2|, \cdots, |x_n|)$

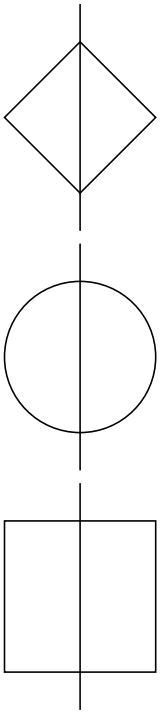


表 C.1. 常见的 L_p 范数。图示展示了二维空间中范数等高线的形状。在该范数下，等高线上的所有点到原点的距离相等。

为了得到方阵 A 的 $\nabla_{\mathbf{x}}\mathrm{x}^{\top}\mathrm{Ax}$，我们首先展开 $\mathrm{x}^{\top}\mathrm{Ax}$：

$$\mathbf{x}^{\top}\mathbf{A}\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \cdots \\ x_n \end{bmatrix}^{\top} \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \cdots \\ x_n \end{bmatrix} \tag{C.24}$$

$$= \begin{bmatrix} x_1 \\ x_2 \\ \cdots \\ x_n \end{bmatrix}^{\top} \begin{bmatrix} x_1a_{11} + x_2a_{12} + \cdots + x_na_{1n} \\ x_1a_{21} + x_2a_{22} + \cdots + x_na_{2n} \\ \vdots \\ x_1a_{n1} + x_2a_{n2} + \cdots + x_na_{nn} \end{bmatrix} \tag{C.25}$$

$$= \begin{array}{c} x_1^2a_{11} + x_1x_2a_{12} + \cdots + x_1x_na_{1n} + \\ x_1x_2a_{21} + x_2^2a_{22} + \cdots + x_2x_na_{2n} + \\ \vdots \\ x_1x_na_{n1} + x_2x_na_{n2} + \cdots + x_n^2a_{nn} \end{array} \tag{C.26}$$

关于第 i 个分量的偏导数是

$$\frac{\partial}{\partial x_i}\mathbf{x}^{\top}\mathbf{A}\mathbf{x} = \sum_{j=1}^{n} x_j\left(a_{ij} + a_{ji}\right) \tag{C.27}$$

梯度因此为：

$$\nabla_{\mathbf{x}}\mathbf{x}^{\top}\mathbf{A}\mathbf{x} = \begin{bmatrix} \sum_{j=1}^{n} x_j\left(a_{1j} + a_{j1}\right) \\ \sum_{j=1}^{n} x_j\left(a_{2j} + a_{j2}\right) \\ \vdots \\ \sum_{j=1}^{n} x_j\left(a_{nj} + a_{jn}\right) \end{bmatrix} \tag{C.28}$$

$$= \begin{bmatrix} a_{11} + a_{11} & a_{21} + a_{12} & \cdots & a_{n1} + a_{1n} \\ a_{12} + a_{21} & a_{22} + a_{22} & \cdots & a_{n2} + a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1n} + a_{n1} & a_{2n} + a_{n2} & \cdots & a_{nn} + a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \tag{C.29}$$

$$= \left(\mathbf{A} + \mathbf{A}^{\top}\right)\mathbf{x} \tag{C.30}$$

C.6 Positive Definiteness

矩阵是*positive definite*或*positive semidefinite*的概念在`线性代数`和优化中经常因各种原因出现。例如，如果矩阵A在函数$f(\mathrm{x}) = \mathrm{x}^\top \mathrm{Ax}$中是正定的，那么$f$具有唯一的全局最小值。

回想一下，一个二阶可微函数f在x_0处的二次近似为

$$f(\mathbf{x}) \approx f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)^\top (\mathbf{x} - \mathbf{x}_0) + \frac{1}{2}(\mathbf{x} - \mathbf{x}_0)^\top \mathbf{H}_0 (\mathbf{x} - \mathbf{x}_0) \tag{C.31}$$

其中 H_0 是 f 在 x_0 处求值的 Hessian 矩阵。已知 $(\mathrm{x} - \mathrm{x}_0)^\top \mathrm{H}_0 (\mathrm{x} - \mathrm{x}_0)$ 具有唯一全局最小值，这足以确定整体二次近似是否具有唯一全局最小值。[6] 。

一个对称矩阵A是正定的，如果对于除原点外的所有点，$\mathrm{x}^\top \mathrm{Ax}$均为正：对所有$\mathrm{x} \neq 0$，有$\mathrm{x}^\top \mathrm{Ax} > 0$。

一个对称矩阵 A 是半正定的，如果 $\mathrm{x}^\top \mathrm{Ax}$ 总是非负的：对于所有 x，$\mathrm{x}^\top \mathrm{Ax} \geq 0$ 。

6 分量 $f(\mathrm{x}_0)$ 仅将函数垂直移动。分量 $\nabla f(\mathrm{x}_0)^\top (\mathrm{x} - \mathrm{x}_0)$ 是一个线性项，其被二次项所主导。

C.7 Gaussian Distribution

univariate Gaussian、[7]的概率密度函数，也称为*normal distribution*，是：

$$\mathcal{N}(x \mid \mu, \nu) = \frac{1}{\sqrt{2\pi\nu}} e^{-\frac{(x-\mu)^2}{2\nu}} \tag{C.32}$$

其中 μ 是均值，ν 是方差。[8] 该分布如图 C.6 所示。

分布的 *cumulative distribution function* 将 x 映射到从该分布中抽取一个值时产生小于或等于 x 的值的概率。对于一元高斯分布，累积分布函数由下式给出：

$$\Phi(x) \equiv \frac{1}{2} + \frac{1}{2}\mathrm{erf}\left(\frac{x - \mu}{\sigma\sqrt{2}}\right) \tag{C.33}$$

其中 erf 是 *error function*：

$$\mathrm{erf}(x) \equiv \frac{2}{\sqrt{\pi}} \int_0^x e^{-\tau^2} d\tau \tag{C.34}$$

7 多元高斯分布在第8章和第15章中讨论。单变量高斯分布贯穿全书使用。

8 方差是标准差的平方。

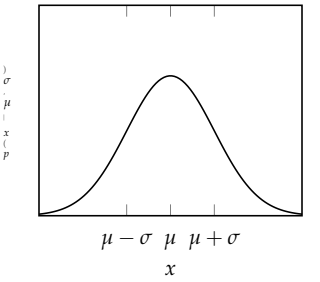


图 C.6. 单变量高斯分布，$\mathcal{N}(\mu, \nu)$。

C.8 Gaussian Quadrature

Gaussian quadrature 是一种使用函数求值的加权和来近似积分的技术。[9] 该近似的一般形式是

$$\int_a^b p(x) f(x)\, dx \approx \sum_{i=1}^{m} w_i f(x_i) \tag{C.35}$$

其中 $p(x)$ 是有限或无限上的已知非负权重函数[10]
区间 $[a, b]$。

一个 m 点 *quadrature rule* 是点 $x_i \in (a、b)$ 和权重 $w_i > 0$（对于 $i \in \{1, \ldots, m\}$）的唯一选择，它们定义了一个高斯求积近似，使得任何次数不超过 $2m - 1$ 的多项式在 $[a, b]$ 上以给定的权重函数被精确积分。

给定一个定义域和一个权函数，我们可以计算一类正交多项式。我们将使用 $b_i(x)$ 来表示一个次数为 i 的正交多项式 [11]。
任何次数为m的多项式都可以表示为直到次数m的正交多项式的线性组合。我们通过选择m个点x_i作为正交多项式p_m的零点来形成求积规则，并通过解方程组来获得权重：

$$\sum_{i=1}^{m} b_k(x_i) w_i = \begin{cases} \int_a^b p(x) b_0(x)^2\, dx & \text{for } k = 0 \\ 0 & \text{for } k = 1, \ldots, m-1 \end{cases} \tag{C.36}$$

高斯求解了方程（C.36）在区间 $[-1, 1]$ 上以及权重函数 $p(x) = 1$。对应于该情况的正交多项式是 *Legendre polynomials*。算法 C.1 实现了针对勒让德多项式的高斯求积，示例 C.3 给出了在 $[-1, 1]$ 上积分的求积规则。

我们可以使用以下变换将任何在有限区间 $[a, b]$ 上的积分转换为在 $[-1, 1]$ 上的积分

$$\int_a^b f(x)\, dx = \frac{b-a}{2} \int_{-1}^{1} f\left(\frac{b-a}{2} x + \frac{a+b}{2}\right) dx \tag{C.37}$$

因此，可以预先为勒让德多项式计算求积规则，然后将其应用于任意有限区间上的积分。[12] 例 C.4 应用了这种
一种变换和算法C.2在有限域的高斯求积法中实现了积分变换。

9 关于高斯求积的详细概述，参见 J. Stoer 和 R. Bulirsch, *Introduction to Numerical*，第3版，Springer，2002年。

10 权重函数在实践中通常是一个概率密度函数。

11 正交多项式在 chapter 18 中涵盖。

12 类似的技术可以应用于无限区间上的积分，例如使用拉盖尔多项式的$[0, \infty)$ 和使用埃尔米特多项式的$(-\infty, \infty)$。

示例 C.3。获得一个 3 项求积规则，以精确积分次数不超过 5 的多项式。

考虑勒让德多项式在$[-1, 1]$区间上的积分，权重函数为$p(x) = 1$。假设我们关注的函数能被五次多项式很好地近似。我们构造一个三点求积规则，该规则对于次数不超过5的多项式能产生精确结果。

3 阶勒让德多项式是 $\mathrm{Le}_3(x) = \frac{5}{2}x^3 - \frac{3}{2}x$，其根为 $x_1 = -\sqrt{3/5}$、$x_2 = 0$ 和 $x_3 = \sqrt{3/5}$。较低阶的勒让德多项式是 $\mathrm{Le}_0(x) = 1$、$\mathrm{Le}_1(x) = x$ 和 $\mathrm{Le}_2(x) = \frac{3}{2}x^2 - \frac{1}{2}$。权重通过求解方程组获得：

$$\begin{bmatrix} \mathrm{Le}_0(-\sqrt{3/5}) & \mathrm{Le}_0(0) & \mathrm{Le}_0(\sqrt{3/5}) \\ \mathrm{Le}_1(-\sqrt{3/5}) & \mathrm{Le}_1(0) & \mathrm{Le}_1(\sqrt{3/5}) \\ \mathrm{Le}_2(-\sqrt{3/5}) & \mathrm{Le}_2(0) & \mathrm{Le}_2(\sqrt{3/5}) \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} \int_{-1}^{1} \mathrm{Le}_0(x)^2\,dx \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 \\ -\sqrt{3/5} & 0 & \sqrt{3/5} \\ 4/10 & -1/2 & 4/10 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}$$

得到 $w_1 = w_3 = 5/9$ 和 $w_2 = 8/9$。

考虑对5次多项式 $f(x) = x^5 - 2x^4 + 3x^3 + 5x^2 - x + 4$ 进行积分。其精确值为 $\int_{-1}^{1} p(x)f(x)\,dx = 158/15 \approx 10.533$。求积法则给出了相同的值：

$$\sum_{i=1}^{3} w_i f(x_i) = \frac{5}{9} f\left(-\sqrt{\frac{3}{5}}\right) + \frac{8}{9} f(0) + \frac{5}{9} f\left(\sqrt{\frac{3}{5}}\right) \approx 10.533.$$

考虑对 $[-3, 5]$ 积分 $f(x) = x^5 - 2x^4 + 3x^3 + 5x^2 - x + 4$。我们可以利用公式 (C.37) 将其转换为对 $[-1, 1]$ 的积分：

$$\int_{-3}^{5} f(x)\,dx = \frac{5+3}{2}\int_{-1}^{1} f\left(\frac{5+3}{2}x + \frac{5-3}{2}\right)dx = 4\int_{-1}^{1} f(4x+1)\,dx$$

我们使用示例 C.3 中得到的 3 项求积规则在 $[-1, 1]$ 上对 $g(x) = 4f(4x + 1) = 4096x^5 + 3072x^4 + 1280y^3 + 768y^2 + 240y + 40$ 进行积分：

$$\int_{-1}^{1} p(x)g(x)\,dx = \frac{5}{9}g\left(-\sqrt{3/5}\right) + \frac{8}{9}g(0) + \frac{5}{9}g\left(\sqrt{3/5}\right) = 1820.8$$

示例 C.4. 有限区域上的积分可以转换为在$[-1, 1]$上的积分，并利用勒让德多项式的 quadrature 规则求解。

```
struct Quadrule
    ws
    xs
end
function quadrule_legendre(m)
    bs = [legendre(i) for i in 1 : m+1]
    xs = roots(bs[end])
    A = [bs[k](xs[i]) for k in 1 : m, i in 1 : m]
    b = zeros(m)
    b[1] = 2
    ws = A\b
    return Quadrule(ws, xs)
end
```

算法 C.1. 一种在 $[-1, 1]$ 上构造 `m` 点 Legendre 求积规则的方法。所得类型同时包含节点 `xs` 和权重 `ws`。

```
quadint(f, quadrule) =
    sum(w*f(x) for (w,x) in zip(quadrule.ws, quadrule.xs))
function quadint(f, quadrule, a, b)
    α = (b-a)/2
    β = (a+b)/2
    g = x -> α*f(α*x+β)
    return quadint(g, quadrule)
end
```

算法 C.2。函数`quadint`用于在有限域$[a, b]$上，使用给定的求积规则`quadrule`，对单变量函数`f`进行积分。

D Solutions

练习 1.1：$f(x) = x^3/3 - x$ 在 $x = 1$。
练习 1.2：它没有最小值，也就是说，该函数被称为下方无界。

练习 1.3：否。考虑最小化 $f(x) = x$，约束条件为 $x \geq 1$。
练习 1.4：函数 f 可以分解为两个分别仅依赖于各自特定坐标的函数：

$$f(x, y) = g(x) + h(y)$$

其中 $g(x) = x^2$ 和 $h(y) = y$。对于 x，$y \geq 1$，g 和 h 都严格递增。虽然 h 在 $y = 1$ 时取得最小值，但由于严格不等式 $x > y$，我们只能趋近于 $x \to 1$。因此，f 没有最小值。

练习 1.5：拐点是曲线上曲率符号改变的点。x 是拐点的必要条件是二阶导数为零。二阶导数为 $f''(x) = 6x$，仅在 $x = 0$ 时为零。

x 为拐点的一个充分条件是二阶导数在 x 附近变号。即对于 $\epsilon \ll 1$，$f''(x + \epsilon)$ 和 $f''(x - \epsilon)$ 符号相反。这对 $x = 0$ 成立，因此它是一个拐点。

因此，$x^3 - 10$上仅有一个拐点。
练习 2.1：海森矩阵的一个元素可以使用前向差分法计算：

$$H_{ij} = \frac{\partial^2 f(\mathbf{x})}{\partial x_i \partial x_j} \approx \frac{\nabla f(\mathbf{x} + h\mathbf{e}_j)_i - \nabla f(x)_i}{h}$$

其中 e_i 是第i个基向量，其中e_i =为1，其余所有项为 {e*}等于零。

因此，我们可以使用以下方式近似Hessian矩阵的第j列：

$$\mathbf{H}_{\cdot j} \approx \frac{\nabla f(\mathbf{x} + h\mathbf{e}_j) - \nabla f(x)}{h}$$

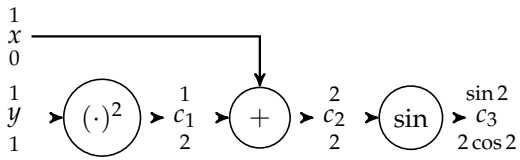
此过程可对 Hessian 矩阵的每一列重复进行。

练习2.2：它需要对目标函数进行两次评估。

练习2.3：$f'(x) = \frac{1}{x} + e^x - \frac{1}{x^2}$。当$x$接近于零时，我们发现 $x <$ 约等于 1。因此 $\frac{1}{x} >$ 近似为 1，最终$\frac{1}{x^2} > \frac{1}{x} >$ 趋向于 0，所以 $-\frac{1}{x^2}$ 占主导。

练习 2.4：根据复步方法，我们有$f'(x) \approx \text{Im}(2 + 4ih)/h = 4h/h = 4$。

练习 2.5：见下图：



练习 2.6：二阶导数可以使用一阶导数的中心差分来近似：

$$f''(x) \approx \frac{f'(x + h/2) - f'(x - h/2)}{h}$$

对于h的较小值。

将$f'(x + h/2)$和$f'(x - h/2)$的前向和后向不同估计代入，得到：

$$\begin{aligned} f''(x) &\approx \frac{\frac{f(x+h/2+h/2)-f(x+h/2-h/2)}{h} - \frac{f(x-h/2+h/2)-f(x-h/2-h/2)}{h}}{h} \\ &= \frac{f(x+h) - f(x)}{h^2} - \frac{f(x) - f(x-h)}{h^2} \\ &= \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} \end{aligned}$$

练习3.1：F　当不使用导数时，斐波那契搜索更为可取。　可用的。

练习3.2：Shubert-Piyavskii方法需要Lipschitz常数，该常数可能未知。

练习 3.3：$f(x) = x^2$。由于该函数为二次函数，经过三次求值后，二次模型将精确表示该函数。

练习 3.4：我们可以使用二分法来找到 $f'(x) = x -$ 的根。第一次更新后，我们有 [0, 500]。然后，[0, 250]。最后，[0, 125]。

练习 3.5：不，Lipschitz常数必须在区间上处处限定导数，且 $f'(1) = 2(1 + 2) = 6$。

练习3.6：不行。你能做的最好的就是使用斐波那契搜索并将不确定性缩小因子3，即到$(32 - 1)/3 = 10\frac{1}{3}$。

练习 4.1：考虑在 $f(x) = 1/x$ 上对 $x > 0$ 运行下降法。最小值不存在，下降法将永远沿正 x 方向以不断增大的步长进行。因此，仅依赖步长终止条件会导致方法永远运行。而基于梯度幅度的终止也会使其终止。

应用于 $f(x) = -x$ 的下降法也将永远沿着正 x 方向进行。该函数在下方无界，因此步长终止条件和梯度幅度终止条件都不会触发。通常会增加一个额外的终止条件来限制迭代次数。

练习 4.2：将第一个 Wolfe 条件应用于我们的目标函数得到 $6 + (-1 + \alpha)^2 \leq 7 - 2\alpha \cdot 10^{-4}$，可简化为 $\alpha^2 - 2\alpha + 2 \cdot 10^{-4}\alpha \leq 0$。解此方程可得 $\alpha \leq 2(1 - 10^{-4})$。因此，最大步长为 $\alpha = 1.9998$。

练习 5.1: $\nabla f(\mathrm{x}) = 2\mathrm{Ax} + \mathrm{b}$。

练习5.2：导数是$f'(x) = 4x^3$。从$x^{(1)} = 1$开始：

$$f'(1) = 4 \qquad \rightarrow x^{(2)} = 1 - 4 = -3 \tag{D.1}$$

$$f'(-3) = 4 * (-27) = 108 \qquad \rightarrow x^{(3)} = -3 + 108 = 105 \tag{D.2}$$

练习 5.3：对于大的 x，我们有 $f'(x) = e^x - e^{-x} \approx e^x$。因此，$f'(x^{(1)}) \approx e^{10}$ 和 $x^{(2)} \approx -e^{10}$。如果我们应用精确线搜索，$x^{(2)} = 0$。因此，如果没有线搜索，我们无法保证减小目标函数的值。

练习5.4：海森矩阵为2H，并且

$$\nabla q(\mathbf{d}) = \mathbf{d}^\top \left(\mathbf{H} + \mathbf{H}^\top\right) + \mathbf{b} = \mathbf{d}^\top (2\mathbf{H}) + \mathbf{b}.$$

当 $\mathrm{d} = 0$ 时，梯度为 b。共轭梯度法可能会发散，因为 H 不保证是正定的。

练习 5.5：Nesterov 动量法着眼于更新后你将到达的点，以计算更新本身。

练习 5.6：共轭梯度法隐式地重用了关于该函数的先前信息，因此在实践中可能享有更好的收敛性。

练习5.7：共轭梯度法最初遵循最速下降方向。梯度为

$$\nabla f(x, y) = [2x + y, 2y + x] \tag{D.3}$$

对于 $(x, y) = (1, 1)$，它是 $[1/\sqrt{2}, 1/\sqrt{2}]$。最陡下降的方向与梯度相反，$\mathrm{d}^{(1)} = [-1/\sqrt{2}, -1/\sqrt{2}]$。

海森矩阵是

$$\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \tag{D.4}$$

由于该函数是二次函数且海森矩阵正定，共轭梯度法至多两步收敛。因此，两步后得到的点即为最优点，$(x, y) = (0, 0)$，此处梯度为零。

练习 5.8：否。如果执行精确最小化，则步骤之间的下降方向是正交的，但 $[1, 2, 3]^\top [0, 0, -3] \neq 0$。

练习 6.1：二阶信息可以保证处于局部最小值，而零梯度是必要的但不足以保证局部最优性。

练习 6.2：如果我们从足够接近根的地方开始，并且能够解析地计算导数，我们更倾向于使用牛顿法。牛顿法具有更好的收敛速度。

练习6.3：$f'(x) = 2x$，$f''(x) = 2$。因此，$x^{(2)} = x^{(1)} - 2x^{(1)}/2 = 0$；也就是说，你从任意起点一步收敛。

练习6.4：由于$\nabla f(\mathrm{x}) = \mathrm{Hx}$，$\nabla^2 f(\mathrm{x}) = \mathrm{H}$，且H非奇异，因此$\mathrm{x}(2) = \mathrm{x}(1) - \mathrm{H}^{-1}\mathrm{H}\,\mathrm{x}(1) = 0$。也就是说，牛顿法一步收敛。梯度下降则发散：

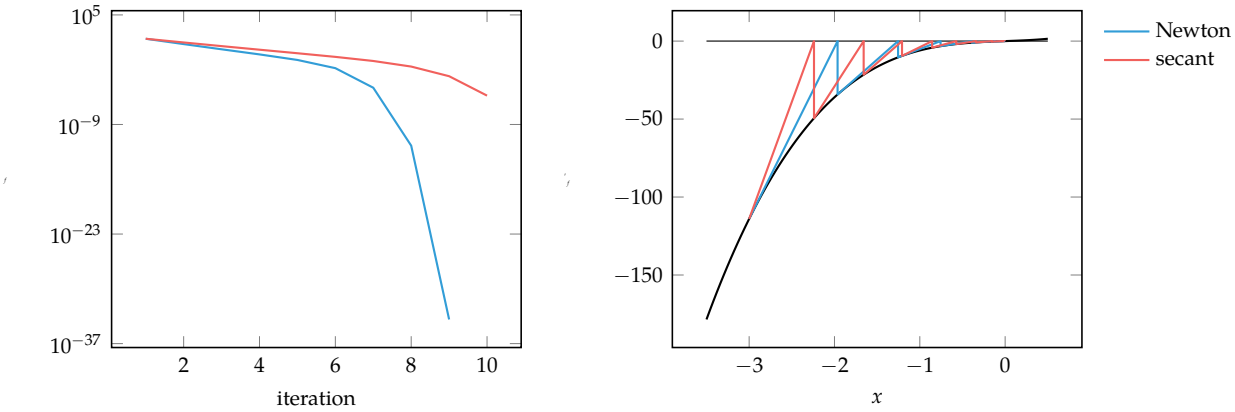
$$\mathbf{x}^{(2)} = [1, 1] - [1, 1000] = [0, -999] \tag{D.5}$$

$$\mathbf{x}^{(3)} = [0, -999] - [0, -1000 \cdot 999] = [0, 998001] \tag{D.6}$$

共轭梯度下降使用与梯度下降相同的初始搜索方向，并且由于优化目标是二次的，它在第二步就收敛到最小值。

练习 6.5：左图展示了牛顿法在九次迭代内达到浮点分辨率的收敛情况。割线法收敛较慢，因为它仅能近似导数。

右图展示了每种方法关于f'的精确切线与近似切线的投影。割线法的切线具有更高的斜率，因此过早地与x轴相交。



练习 6.6：考虑函数$f(x) = x^2 - x$上从$x^{(1)} = -1$开始的序列$x^{(k+1)} = x^{(k)}/2$。显然该序列收敛到$x = 0$，$f(x)$的值在递减，但该序列并不收敛到极小值点。练习 6.7：它不需要计算或知道 Hessian 矩阵的元素，因此不需要在每次迭代时求解线性系统。

练习 6.8：当$\delta^\top \gamma \approx 0$时，BFGS 更新不存在。在这种情况下，只需跳过更新。

练习 6.9：目标函数是二次函数，因此可以在一步内最小化。梯度为$\nabla f = [2(x_1 + 1), 2(x_2 + 3)]$，在$\mathrm{x}^* = [-1, -3]$处为零。海森矩阵是正定的，因此$\mathrm{x}^*$是最小值。

练习 6.10：新的近似具有形式

$$f^{(k+1)}(\mathbf{x}) = y^{(k+1)} + \left(\mathbf{g}^{(k+1)}\right)^\top \left(\mathbf{x} - \mathbf{x}^{(k+1)}\right) + \frac{1}{2}\left(\mathbf{x} - \mathbf{x}^{(k+1)}\right)^\top \mathbf{H}^{(k+1)} \left(\mathbf{x} - \mathbf{x}^{(k+1)}\right)$$

使用$\mathrm{x}^{(k+1)}$处的真实函数值和梯度，但需要更新的Hessian矩阵$\mathrm{H}^{(k+1)}$。这种形式自动满足$f^{(k+1)}(\mathrm{x}^{(k+1)}) = y^{(k+1)}$和$\nabla f^{(k+1)}(\mathrm{x}^{(k+1)}) = \mathrm{g}^{(k+1)}$。我们必须选择新的Hessian矩阵以满足第三个

条件：

$$\begin{aligned}\nabla f^{(k+1)}(\mathbf{x}^{(k)}) &= \mathbf{g}^{(k+1)} + \mathbf{H}^{(k+1)}\left(\mathbf{x}^{(k)} - \mathbf{x}^{(k+1)}\right) \\ &= \mathbf{g}^{(k+1)} - \mathbf{H}^{(k+1)}\left(\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\right) \\ &= \mathbf{g}^{(k+1)} - \mathbf{H}^{(k+1)}\boldsymbol{\delta}^{(k+1)} \\ &= \mathbf{g}^{(k)}\end{aligned}$$

我们可以重新排列并替换得到：

$$\mathbf{H}^{(k+1)}\boldsymbol{\delta}^{(k+1)} = \boldsymbol{\gamma}^{(k+1)}$$

回想一下，如果对于每个非零向量$\mathrm{x}^\top \mathrm{Ax} > 0$，矩阵A是正定的。如果我们将割线方程乘以$\boldsymbol{\delta}^{(k+1)}$，我们得到曲率条件：

$$\left(\boldsymbol{\delta}^{(k+1)}\right)^\top \mathbf{H}^{(k+1)}\boldsymbol{\delta}^{(k+1)} = \left(\boldsymbol{\delta}^{(k+1)}\right)^\top \boldsymbol{\gamma}^{(k+1)} > 0$$

我们寻求一个新的正定矩阵H(k+1)。所有正定矩阵都是对称的，因此计算一个新的正定矩阵需要指定$n(n+1)/2$个变量。割线方程对这些变量施加了n个条件，导致有无限多个解。为了得到唯一解，我们选择最接近H(k)的正定矩阵。这一目标引出了所需的优化问题。

练习 7.1：一阶导数有n项，而 Hessian 矩阵有 n^2 项。使用有限差分法时，每个一阶导数项需要两次求值：$f(\mathrm{x})$ 和 $f(\mathrm{x} + h\mathrm{e}^{(i)})$。使用有限差分法时，每个 Hessian 项需要三次求值。因此，为了近似梯度，你需要 $n+1$ 次求值；而为了近似 Hessian 矩阵，你大约需要 n^2 次求值。

对于大型n，近似Hessian矩阵的成本过高。直接方法可以使用n^2次函数评估采取相对更多的步骤，因为直接方法在每一步无需估计导数或Hessian矩阵。

练习7.2：考虑最小化$f(x) = xy$和$x_0 = [0, 0]$。沿任一规范方向进行都不会减少目标函数，但x_0显然不是最小值点。

练习 7.3：在每次迭代中，Hooke-Jeeves 方法沿坐标方向以步长 a 采样 $2n$ 个点。当所有点都无法带来改进且步长不超过给定容差 ϵ 时，该方法停止。虽然这通常会使 Hooke-Jeeves 方法在收敛到局部最小值 ϵ 范围内时停止，但情况未必如此。例如，一个山谷可以在到达局部最小值之前沿两个坐标方向下降得比 ϵ 更远，而 Hooke-Jeeves 方法无法检测到这一点。

练习 7.4：在满足最小厚度（以保持结构完整性）的条件下，最小化翼型的阻力。使用计算流体动力学评估翼型性能涉及求解偏微分方程。由于该函数不是解析已知的，我们不太可能得到导数的解析表达式。

练习7.5：分割矩形法在区间的中心进行采样，而不是在已知Lipschitz常数导出的界限最低的位置采样。

练习 7.6：由于有多个分量在变化，因此不可能是循环坐标搜索。这可能是鲍威尔方法。

练习 8.1：交叉熵方法每次迭代都必须拟合分布参数。遗憾的是，拟合多元正态分布没有已知的解析解。通常，人们使用迭代期望最大化算法来收敛到一个答案。

练习 8.2：如果精英样本的数量接近总样本数量，那么得到的分布将接近总体分布。对于最小化器而言，不会显著偏向最佳位置，因此收敛速度会很慢。

练习　电子 8.3：对数似然关于 t 的导数　　o ν is:

$$\begin{aligned}\frac{\partial}{\partial\nu}\ell(x \mid \mu,\nu) &= \frac{\partial}{\partial\nu}\left(-\frac{1}{2}\ln 2\pi - \frac{1}{2}\ln\nu - \frac{(x-\mu)^2}{2\nu}\right)\\ &= -\frac{1}{2\nu} + \frac{(x-\mu)^2}{2\nu^2}\end{aligned}$$

如果均值已经是最优的，第二项将为零。因此，导数是 $-1/2\nu$，减小 ν 将增加抽取精英样本的可能性。不幸的是，ν 通过任意接近零来优化。梯度更新中接近零的渐近线会导致较大的步长，而这些步长不能被采用，因为 ν 必须保持为正。

练习 8.4：在均值为 $\boldsymbol{\mu}$、协方差为 $\boldsymbol{\Sigma}$ 的多元正态分布下，设计 x 的概率密度为

$$p(\mathbf{x} \mid \boldsymbol{\sigma}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi|\boldsymbol{\Sigma}|)^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)$$

我们可以通过最大化对数似然来简化问题。[1] 该对数似然是：

[1] 对数函数对于正输入是凹函数，因此最大化 $\log f(x)$ 也使严格正的 $f(x)$ 最大化。

$$\begin{aligned}\ln p(\mathbf{x} \mid \boldsymbol{\sigma}, \boldsymbol{\Sigma}) &= -\frac{1}{2}\ln(2\pi|\boldsymbol{\Sigma}|) - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}) \\ &= -\frac{1}{2}\ln(2\pi|\boldsymbol{\Sigma}|) - \frac{1}{2}\left(\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x} - 2\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + \boldsymbol{\mu}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right)\end{aligned}$$

我们首先针对均值最大化 m 个个体的对数似然：

$$\begin{aligned}\ell\left(\boldsymbol{\mu} \mid \mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(m)}\right) &= \sum_{i=1}^{m} \ln p(\mathbf{x}^{(i)} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) \\ &= \sum_{i=1}^{m} -\frac{1}{2}\ln(2\pi|\boldsymbol{\Sigma}|) - \frac{1}{2}\left(\left(\mathbf{x}^{(i)}\right)^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}^{(i)} - 2\left(\mathbf{x}^{(i)}\right)^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + \boldsymbol{\mu}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right)\end{aligned}$$

我们使用以下事实来计算梯度：$\nabla_{\mathbf{z}} \mathrm{z}^\top \mathrm{A}\mathrm{z} = \left(\mathrm{A} + \mathrm{A}^\top\right)\mathrm{z}$，$\nabla_{\mathbf{z}} \mathrm{a}^\top \mathrm{z} = \mathrm{a}$，以及 $\boldsymbol{\Sigma}$ 是对称且正定的，因此 $\boldsymbol{\Sigma}^{-1}$ 是对称的：

$$\begin{aligned}\nabla_{\boldsymbol{\mu}} \ell\left(\boldsymbol{\mu} \mid \mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(m)}\right) &= \sum_{i=1}^{m} -\frac{1}{2}\left(\nabla_{\boldsymbol{\mu}}\left(-2\left(\mathbf{x}^{(i)}\right)^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right) + \nabla_{\boldsymbol{\mu}}\left(\boldsymbol{\mu}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right)\right) \\ &= \sum_{i=1}^{m}\left(\nabla_{\boldsymbol{\mu}}\left(\left(\mathbf{x}^{(i)}\right)^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right) - \frac{1}{2}\nabla_{\boldsymbol{\mu}}\left(\boldsymbol{\mu}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right)\right) \\ &= \sum_{i=1}^{m} \boldsymbol{\Sigma}^{-1}\mathbf{x}^{(i)} - \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\end{aligned}$$

我们将梯度设为零：

$$\begin{aligned}
\mathbf{0} &= \sum_{i=1}^{m} \mathbf{\Sigma}^{-1}\mathbf{x}^{(i)} - \mathbf{\Sigma}^{-1}\boldsymbol{\mu} \\
\sum_{i=1}^{m} \boldsymbol{\mu} &= \sum_{i=1}^{m} \mathbf{x}^{(i)} \\
m\boldsymbol{\mu} &= \sum_{i=1}^{m} \mathbf{x}^{(i)} \\
\boldsymbol{\mu} &= \frac{1}{m}\sum_{i=1}^{m} \mathbf{x}^{(i)}
\end{aligned}$$

接下来，关于逆协方差 $\mathbf{\Lambda} = \mathbf{\Sigma}^{-1}$ 进行最大化，利用 $|\mathrm{A}^{-1}| = 1/|\mathrm{A}|$ 且 $\mathrm{b}(i) = \mathrm{x}(i) - \boldsymbol{\mu}$ 的事实：

$$\begin{aligned}
\ell\left(\mathbf{\Lambda} \mid \boldsymbol{\mu}, \mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(m)}\right) &= \sum_{i=1}^{m} -\frac{1}{2}\ln\left(2\pi|\mathbf{\Lambda}|^{-1}\right) - \frac{1}{2}\left(\left(\mathbf{x}^{(i)} - \boldsymbol{\mu}\right)^{\top}\mathbf{\Lambda}\left(\mathbf{x}^{(i)} - \boldsymbol{\mu}\right)\right) \\
&= \sum_{i=1}^{m} \frac{1}{2}\ln(|\mathbf{\Lambda}|) - \frac{1}{2}\left(\mathbf{b}^{(i)}\right)^{\top}\mathbf{\Lambda}\mathbf{b}^{(i)}
\end{aligned}$$

我们利用$\nabla_{\mathbf{A}}|\mathrm{A}| = |\mathrm{A}|\mathrm{A}^{-\top}$以及$\nabla_{\mathbf{A}}\mathrm{z}^{\top}\mathrm{A}\mathrm{z} = \mathrm{z}\mathrm{z}^{\top}$这些事实来计算梯度：

$$\begin{aligned}
\nabla_{\mathbf{\Lambda}}\ell\left(\mathbf{\Lambda} \mid \boldsymbol{\mu}, \mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(m)}\right) &= \sum_{i=1}^{m} \nabla_{\mathbf{\Lambda}}\left(\frac{1}{2}\ln(|\mathbf{\Lambda}|) - \frac{1}{2}\left(\mathbf{b}^{(i)}\right)^{\top}\mathbf{\Lambda}\mathbf{b}^{(i)}\right) \\
&= \sum_{i=1}^{m} \frac{1}{2|\mathbf{\Lambda}|}\nabla_{\mathbf{\Lambda}}|\mathbf{\Lambda}| - \frac{1}{2}\mathbf{b}^{(i)}\left(\mathbf{b}^{(i)}\right)^{\top} \\
&= \sum_{i=1}^{m} \frac{1}{2|\mathbf{\Lambda}|}|\mathbf{\Lambda}|\mathbf{\Lambda}^{-\top} - \frac{1}{2}\mathbf{b}^{(i)}\left(\mathbf{b}^{(i)}\right)^{\top} \\
&= \frac{1}{2}\sum_{i=1}^{m} \mathbf{\Lambda}^{-\top} - \mathbf{b}^{(i)}\left(\mathbf{b}^{(i)}\right)^{\top} \\
&= \frac{1}{2}\sum_{i=1}^{m} \mathbf{\Sigma} - \mathbf{b}^{(i)}\left(\mathbf{b}^{(i)}\right)^{\top}
\end{aligned}$$

并将梯度设为零:

$$\begin{aligned}\mathbf{0} &= \frac{1}{2}\sum_{i=1}^{m}\boldsymbol{\Sigma} - \mathbf{b}^{(i)}\left(\mathbf{b}^{(i)}\right)^{\top} \\ \sum_{i=1}^{m}\boldsymbol{\Sigma} &= \sum_{i=1}^{m}\mathbf{b}^{(i)}\left(\mathbf{b}^{(i)}\right)^{\top} \\ \boldsymbol{\Sigma} &= \frac{1}{m}\sum_{i=1}^{m}\left(\mathbf{x}^{(i)} - \boldsymbol{\mu}\right)\left(\mathbf{x}^{(i)} - \boldsymbol{\mu}\right)^{\top}\end{aligned}$$

练习9.1：通过使选择偏向目标函数值更优的个体，来偏向最适应者的生存。

练习 9.2：突变通过随机性推动探索，因此对于避免局部极小值至关重要。如果我们怀疑存在更好的解，就需要提高突变率，并让算法有时间去发现它。

练习 9.3：增加种群大小或将搜索偏向个体最小值的系数。

练习 10.1：首先将问题重新表述为 $f(x) = x + \rho \max(-x, 0)^2$，其导数为

$$f'(x) = \begin{cases} 1 + 2\rho x & \text{if } x < 0 \\ 1 & \text{otherwise} \end{cases} \tag{D.7}$$

这个无约束目标函数可以通过将$f'(x)$ =设为0来求解，从而得到解$x^* = -\frac{1}{2\rho}$。因此，当$\rho \to \infty$时，我们有$x^* \to 0$。
练习 10.2：该问题被重新表述为 $f(x) = x + \rho(x < 0)$。只要 ρ 是有限的且 x 趋近于负无穷，无约束目标函数从下方是无界的。正确的解没有被找到，而二次罚函数方法能够接近正确的解。

练习 10.3：你可以尝试增大惩罚参数 ρ。有可能是 ρ 太小，导致惩罚项不起作用。在这种情况下，迭代点可能会进入不可行区域，在该区域中函数下降速度比惩罚项更快，从而导致方法收敛到一个不可行解。

练习 10.4：令 x_p^* 为无约束问题的解。注意 $x_p^* \not> 0$。否则，罚项将为 $\left(\min(x_p^*, 0)\right)^2 = 0$，这意味着 x_p^* 是原问题的解。现在，假设 $x_p^* = 0$。一阶

最优性条件表明$f'(x_p^*) + \mu x_p^* = 0$，这意味着$f'(x_p^*) = 0$，再次产生矛盾。因此，如果极小值点存在，它必然不可行。练习10.5：它不需要大的惩罚ρ就能产生精确解。练习10.6：当迭代点应保持可行时。

练习 10.7：考虑以下内容：

$$\begin{aligned} \underset{x}{\text{minimize}} \quad & x^3 \\ \text{subject to} \quad & x \geq 0 \end{aligned} \tag{D.8}$$

对于x^* ==0时，其取得最小值。采用惩罚方法，我们可以将其重述为

$$\underset{x}{\text{minimize}}\, x^3 + \rho(\min(x, 0))^2 \tag{D.9}$$

对于任何有限的ρ，当x趋向负无穷时，该函数下方无界。此外，当x趋向负无穷时，函数变得无限陡峭。换句话说，如果我们从过于靠左的位置开始最陡下降法，我们将有$x^3 + \rho x^2 \approx x^3$，惩罚将无效，最陡下降法将发散。

练习 10.8：你可以将寻找可行点构建为一个优化问题，该问题具有一个常数目标函数以及一个强制满足可行性的约束：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & 0 \\ \text{subject to} \quad & \mathbf{h}(\mathbf{x}) = \mathbf{0} \\ & \mathbf{g}(\mathbf{x}) \leq \mathbf{0} \end{aligned} \tag{D.10}$$

这类问题通常可以用惩罚方法解决。二次惩罚是一个常见的选择，因为它沿着可行性的方向递减。

练习 10.9：问题在$x^* = 1$处取得最小值，该点位于约束边界上。通过t变换求解，得到无约束目标函数：

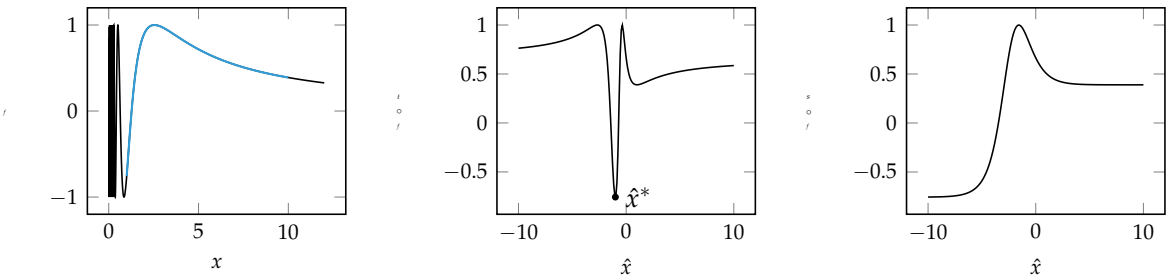
$$f_t(\hat{x}) = \sin\left(\frac{4}{5.5 + 4.5\frac{2\hat{x}}{1+\hat{x}^2}}\right) \tag{D.11}$$

其在$\hat{x} = -1$处有一个唯一的全局最小值，恰好对应于x^*。

Sigmoid变换具有一个无约束的目标函数：

$$f_s(\hat{x}) = \sin\left(\frac{4}{1 + \frac{9}{1+e^{-\hat{x}}}}\right) \tag{D.12}$$

遗憾的是，下界a，在此情况下为$x = 1$，仅当x趋近于负无穷时才达到。使用sigmoid变换得到的无约束优化问题没有解，该方法无法正确识别原问题的解。



练习 10.10：在约束条件 $x_1 \geq 1$ 下最小化 $x_1^2 + x_2^2$。

练习 10.11：我们可以根据以下内容重新排列约束条件 x_1：

$$x_1 = 6 - 2x_2 - 3x_3 \tag{D.13}$$

并将该关系代入目标函数：

$$\underset{x_2, x_3}{\text{minimize}}\, x_2^2 + x_3 - (2x_2 + 3x_3 - 6)^3 \tag{D.14}$$

练习 10.12：约束必须与目标函数对齐。目标函数的方向是 $[-1, -2]$。约束的方向是 $[a, 1]$。使它们对齐的 a 的唯一值是 $a = 0.5$。

练习10.13：变换后的目标函数是$f(x) = 1 - x^2 + \rho p(x)$，其中 p 是计数惩罚或二次惩罚：

$$p_{\text{count}}(x) = (|x| > 2) \qquad p_{\text{quadratic}}(x) = \max(|x| - 2, 0)^2 \tag{D.15}$$

计数惩罚方法不向优化过程提供任何梯度信息。若优化算法的初始点位于可行集之外，它会被拉离可行区域，因为 $1 - x^2$ 通过从原点无限向左或向右移动而最小化。计数惩罚的大幅值并非主要问题；小惩罚也可能导致类似的问题。

二次惩罚方法确实为优化过程提供了梯度信息，引导搜索向可行区域进行。对于非常大的惩罚值，二次惩罚方法会在不可行区域产生大的梯度值。在这个问题中，偏导数是：

$$\frac{\partial f}{\partial x} = -2x + \rho \begin{cases} 2(x-2) & \text{if } x > \quad 2 \\ 2(x+2) & \text{if } x < -2 \\ 0 & \text{otherwise} \end{cases} \tag{D.16}$$

对于非常大的 ρ 值，不可行区域中的偏导数也会很大，这可能会导致优化方法出现问题。如果 ρ 不大，那么不可行点可能没有得到足够的惩罚，从而导致不可行解。

练习 11.1：我们选择通过评估由约束形成的凸多面体中的每个顶点来最小化一个线性规划问题。因此，每个顶点都是潜在的极小值点。顶点由活动约束的交点定义。由于每个不等式约束可以是活动的或非活动的，假设有 n 个不等式约束，我们不需要检查超过 2^n 种约束组合。

该方法无法正确地将无界线性约束优化问题报告为无界。

练习 11.2：单纯形法保证每一步或者相对于目标函数有所改进，或者保持目标函数的当前值不变。任何线性规划问题都有有限个顶点。只要采用诸如布兰德规则这样的启发式方法，确保不发生循环，单纯形法必定会收敛到一个解。

练习 11.3：我们可以添加一个松弛变量x_3，并最小化$6x_1 + 5x_2 + x_3$，受约束于$-3x_1 + 2x_2 + x_3 = -5$和$x_3 \geq 0$。

练习 11.4：如果当前迭代点 x 是可行的，那么有 $\mathrm{w}^\top \mathrm{x} = b \geq 0$。我们希望下一个点保持可行性，因此要求 $\mathrm{w}^\top(\mathrm{x} + \alpha \mathrm{d}) \geq 0$。如果得到的 α 值为正，则该值为步长的上界。如果得到的 α 值为负，则可以忽略。

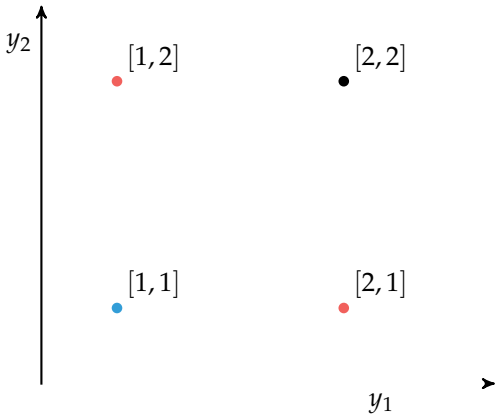
练习11.5：我们可以重写该问题：

$$\underset{\mathbf{x}}{\text{minimize}}\, \mathbf{c}^\top \mathbf{x} - \mu \sum_i \ln\left(\mathbf{A}_{\{i\}}^\top \mathbf{x}\right) \tag{D.17}$$

练习12.1：加权和法无法在帕累托前沿的非凸区域找到帕累托最优点。

练习 12.2：非种群方法只能识别 Pareto 前沿中的一个单一解点。Pareto 前沿在告知设计者一组极佳解之间的权衡方面极具价值。种群方法能够遍布 Pareto 前沿，并可作为 Pareto 前沿的一种近似。

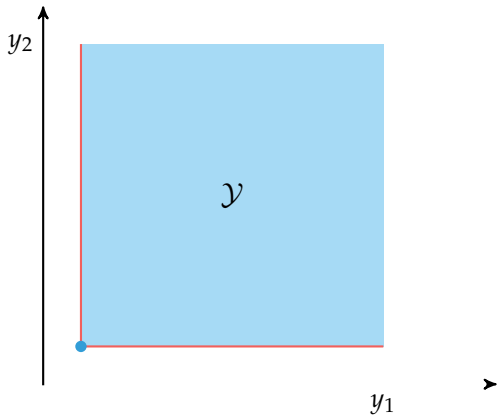
练习12.3：



唯一的帕累托最优点为[1, 1]，而[1, 2]和[2, 1]均为弱帕累托最优。

练习 12.4：向量的“梯度”是一个矩阵。二阶导数需要使用张量，而求解张量方程以找到搜索方向通常在计算上很繁重。

练习 12.5：唯一的帕累托最优点为 y = [0, 0]。左下边界上的其余点是弱帕累托最优的。



练习12.6：考虑上一题中的方形准则空间。使用$\mathbf{w} = [0, 1]$将零值赋给第一个目标，导致准则空间的整个底边具有相同的值。如上所述，只有$\mathbf{y} = [0, 0]$是帕累托最优的，其余的是弱帕累托最优的。

练习12.7：例如，如果$\mathbf{y}^{\text{goal}}$在准则集中，目标规划的目标将通过$\mathbf{y}^{\text{goal}}$得到最小化。如果$\mathbf{y}^{\text{goal}}$也不是帕累托最优的，那么解也不会是帕累托最优的。

练习 12.8：约束方法约束了除一个目标外的所有目标。通过改变约束条件，可以生成帕累托曲线。如果我们约束第一个目标，每个优化问题具有以下形式：

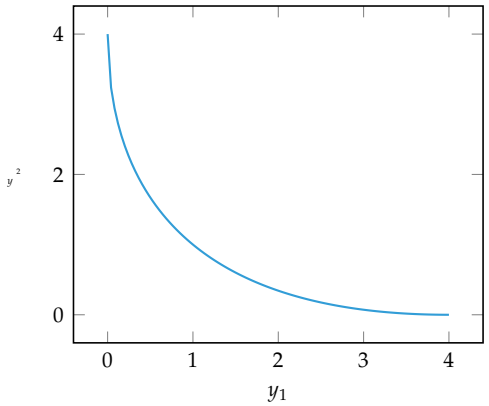
$$\underset{x}{\text{minimize}} \quad (x-2)^2 \tag{D.18}$$

$$\text{subject to} \quad x^2 \leq c \tag{D.19}$$

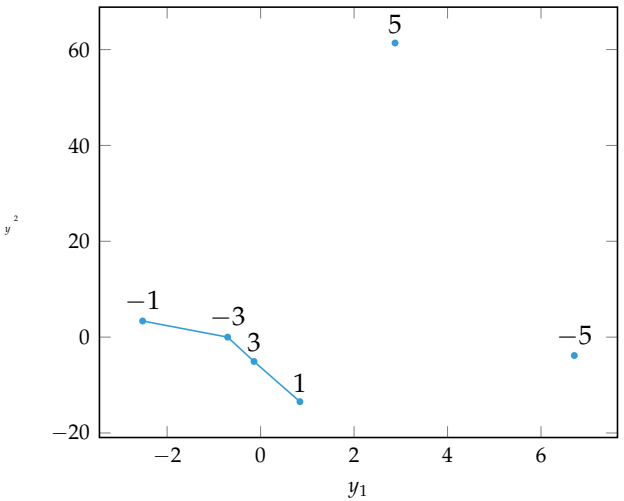
约束仅对 $c \geq 0$ 成立。这允许 x 在 $\pm\sqrt{c}$ 之间变化。第一个目标通过最小化 x 与 2 的偏差来优化。因此，对于给定的 c 值，我们得到：

$$x^* = \begin{cases} 2 & \text{if } c \geq 4 \\ \sqrt{c} & \text{if } c \in [0, 4) \\ \text{undefined} & \text{otherwise} \end{cases} \tag{D.20}$$

得到的帕累托曲线为：



练习 12.9：标准空间是目标函数值的空间。所得图表为：



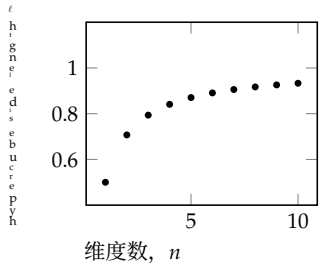
我们发现四个点位于与我们的六个样本点相对应的近似帕累托前沿上。相应的设计点为$x = \{-1、-3、3、1\}$。练习13.1：一维单位超立方体是$x \in [0, 1]$，其体积为1。在这种情况下，所需的边长ℓ为0.5。二维单位超立方体是单位正方形$x_i \in [0, 1]$，对应$i \in \{1, 2\}$，其二维体积，或面积，为1。边长为ℓ的正方形的面积为ℓ^2，因此我们求解：

$$\ell^2 = \frac{1}{2} \quad \Longrightarrow \quad \ell = \frac{\sqrt{2}}{2} \approx 0.707 \tag{D.21}$$

一个 n 维超立方体的体积为 ℓ^n。因此我们求解：

$$\ell^n = \frac{1}{2} \quad \Longrightarrow \quad \ell = 2^{-1/n} \tag{D.22}$$

边长趋近于一。



练习13.2：随机采样的点距表面ϵ距离内的概率正好是体积比。因此：

$$P(\|x\|_2 > 1-\epsilon) = 1 - P(\|x\|_2 < 1-\epsilon) = 1 - (1-\epsilon)^n \to 0 \tag{D.23}$$

作为$n \to \infty$。

练习13.3：

```
function pairwise_distances(X, p=2)
    m = length(X)
    [norm(X[i]-X[j], p) for i in 1:(m-1) for j in (i+1):m]
end
function phiq(X, q=1, p=2)
    dists = pairwise_distances(X, p)
    return sum(dists.^(-q))^(1/q)
end
X = [[cos(2π*i/10), sin(2π*i/10)] for i in 1 : 10]
@show phiq(X, 2)

phiq(X, 2) = 6.422616289332565
```

不。Morris-Mitchell 准则完全基于成对距离。将所有点按相同量进行平移不会改变成对距离，因此不会改变 $\Phi_2(X)$。

练习 13.4：一个有理数可以表示为两个整数 a/b 的分数形式。由此可知，该序列每 b 次迭代重复一次：

$$\begin{aligned}
x^{(k+1)} &= x^{(k)} + \frac{a}{b} \pmod 1 \\
x^{(k)} &= x^{(0)} + k\frac{a}{b} \pmod 1 \\
&= x^{(0)} + k\frac{a}{b} + a \pmod 1 \\
&= x^{(0)} + (k+b)\frac{a}{b} \pmod 1 \\
&= x^{(k+b)}
\end{aligned}$$

练习14.1：线性回归的目标函数是

$$\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2$$

我们求梯度并令其为零：

$$\nabla(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) = -2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) = \mathbf{0}$$

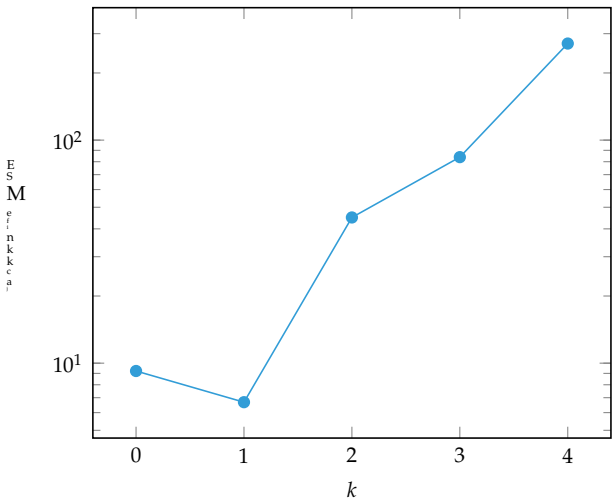
这得到了正规方程

$$\mathbf{X}^\top\mathbf{X}\boldsymbol{\theta} = \mathbf{X}^\top\mathbf{y}$$

练习14.2：通常，当可获得更多数据时，应使用描述性更强的模型。如果只有少量样本可用，这类模型容易过拟合，此时应使用更简单的模型（自由度更少）。

练习 14.3：当前模型可能拥有非常大量的参数。在这种情况下，所产生的线性系统将过于庞大，并且所需内存会随参数空间呈二次增长。诸如随机梯度下降之类的迭代方法所需内存与参数空间的大小成线性关系，有时是唯一可行的解决方案。

练习14.4：留一交叉验证估计是通过运行k折交叉验证获得的，其中k等于X中的样本数量。这意味着我们必须为每个多项式次数运行4折交叉验证。



线性模型获得了最低的均方误差，$k = 1$。我们在完整数据集上拟合一个新的线性模型以获得参数：

```
X = [[1],[2],[3],[4]]
y = [0,5,4,6]
bases = polynomial_bases(1, 1)
B = [b(x) for x in X, b in bases]
θ = pinv(B)*y
@show θ
```

```
θ = [-0.5, 1.7]
```

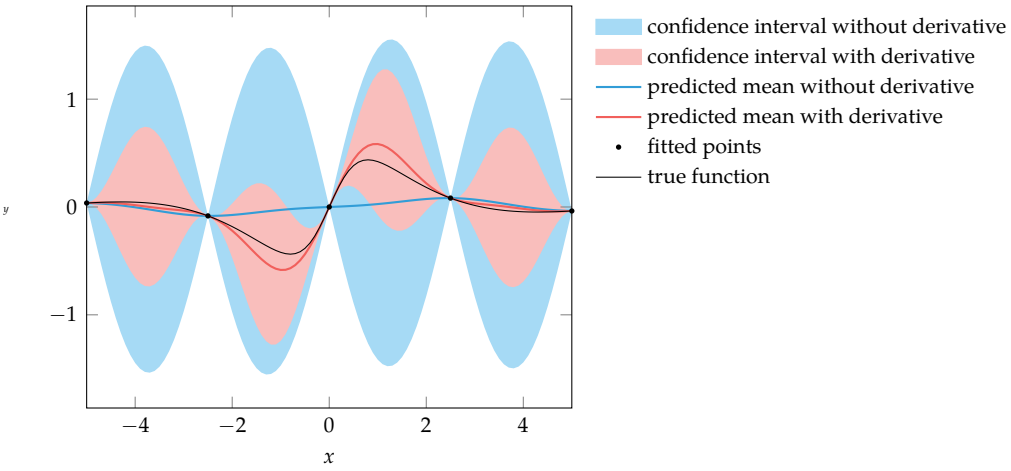
练习15.1：高斯过程是*nonparametric*，而线性回归模型是*parametric*。这意味着模型自由度随着数据量的增加而增长，使得高斯过程在优化过程中能够在偏差和方差之间保持平衡。

练习15.2：获取高斯过程的条件分布需要解方程(15.13)。最昂贵的操作是求$m \times m$矩阵K(X, X)的逆，其复杂度为$O(m^3)$。

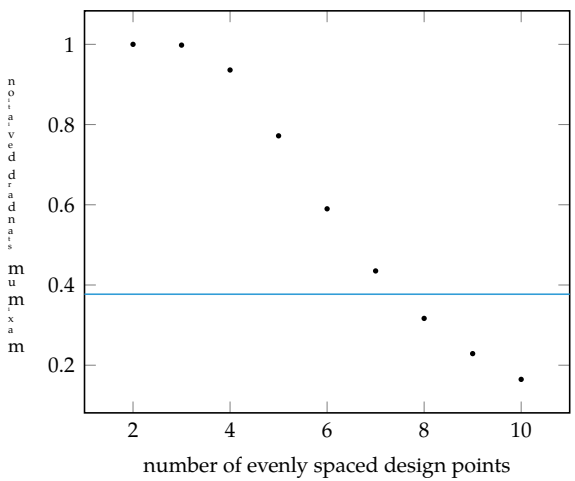
练习15.3：f的导数是

$$\frac{(x^2+1)\cos(x) - 2x\sin(x)}{(x^2+1)^2}$$

下面我们绘制了包含和不包含导数信息的高斯过程的预测分布。对于包含导数信息的高斯过程，在区间$[-5, 5]$上预测分布的最大标准差约为0.377，出现在$x \approx \pm 3.8$处。



引入导数信息会显著缩小置信区间，因为可用来指导预测的信息更多了。下面我们绘制了在区间 $[-5, 5]$ 上，对于没有导数信息且具有不同数量均匀间隔评估点的高斯过程，预测分布中的最大标准差。至少需要八个点，才能超越带有导数信息的高斯过程。



练习 15.4：这可以根据以下内容推导得出：

$$
\begin{aligned}
k_{f\nabla}(\mathbf{x},\mathbf{x}')_i &= \operatorname{cov}\left(f(\mathbf{x}), \frac{\partial}{\partial x_i'} f(\mathbf{x}')\right) \\
&= \mathbb{E}\left[(f(\mathbf{x}) - \mathbb{E}[f(\mathbf{x})])\left(\frac{\partial}{\partial x_i'} f(\mathbf{x}') - \mathbb{E}\left[\frac{\partial}{\partial x_i'} f(\mathbf{x}')\right]\right)\right] \\
&= \mathbb{E}\left[(f(\mathbf{x}) - \mathbb{E}[f(\mathbf{x})])\left(\frac{\partial}{\partial x_i'} f(\mathbf{x}') - \frac{\partial}{\partial x_i'}\mathbb{E}\left[f(\mathbf{x}')\right]\right)\right] \\
&= \mathbb{E}\left[(f(\mathbf{x}) - \mathbb{E}[f(\mathbf{x})])\frac{\partial}{\partial x_i'}\left(f(\mathbf{x}') - \mathbb{E}\left[f(\mathbf{x}')\right]\right)\right] \\
&= \frac{\partial}{\partial x_i'}\mathbb{E}\left[(f(\mathbf{x}) - \mathbb{E}[f(\mathbf{x})])\left(f(\mathbf{x}') - \mathbb{E}\left[f(\mathbf{x}')\right]\right)\right] \\
&= \frac{\partial}{\partial x_i'}\operatorname{cov}\left(f(\mathbf{x}), f(\mathbf{x}')\right) \\
&= \frac{\partial}{\partial x_i'} k_{ff}(\mathbf{x},\mathbf{x}')
\end{aligned}
$$

其中我们使用了$\mathbb{E}\left[\frac{\partial}{\partial x} f\right] = \frac{\partial}{\partial x}\mathbb{E}[f]$。我们可以确信这是真的：

$$
\begin{aligned}
\mathbb{E}\left[\frac{\partial}{\partial x} f\right] &= \mathbb{E}\left[\lim_{h\to 0}\frac{f(x+h) - f(x)}{h}\right] \\
&= \lim_{h\to 0}\mathbb{E}\left[\frac{f(x+h) - f(x)}{h}\right] \\
&= \lim_{h\to 0}\frac{1}{h}(\mathbb{E}[f(x+h)] - \mathbb{E}[f(x)]) \\
&= \frac{\partial}{\partial x}\mathbb{E}[f(x)]
\end{aligned}
$$

前提是目标函数可微。

练习15.5：让我们将联合高斯分布写作：

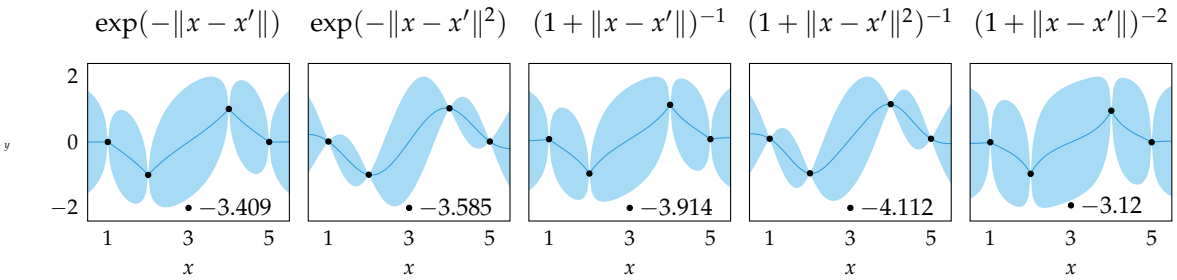
$$
\begin{bmatrix} a \\ b \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu_a \\ \mu_b \end{bmatrix}, \begin{bmatrix} \nu_a & \nu_c \\ \nu_c & \nu_b \end{bmatrix}\right) \tag{D.24}
$$

a 上的边缘分布是 $\mathcal{N}(\mu_a、\ \nu_a)$，其方差为 ν_a。a 的条件分布的方差为 $\nu_a - \nu_c^2/\nu_b$。我们知道 ν_b 必须为正，才能使原始协方差矩阵为正定。因此，ν_c^2/ν_b 为正，且 $\nu_a - \nu_c^2/\nu_b \leq \nu_a$。

条件分布的方差不会大于边缘分布的方差，这是直观的，因为条件分布融合了更多关于 {v*} 的信息。如果 a 和 b 相关，那么知道 b 的值会使我们了解到 a 的值，并减少我们的不确定性。

练习15.6：我们可以使用泛化误差估计或通过最大化观测数据的似然来调整核函数的参数或切换核函数。

练习15.7：最大化似然函数的乘积等价于最大化对数似然之和。以下是使用每个核函数时，给定其他点情况下第三个点的对数似然：



计算 在所有五个折上对这些值求和，得到总对数似然 oods:

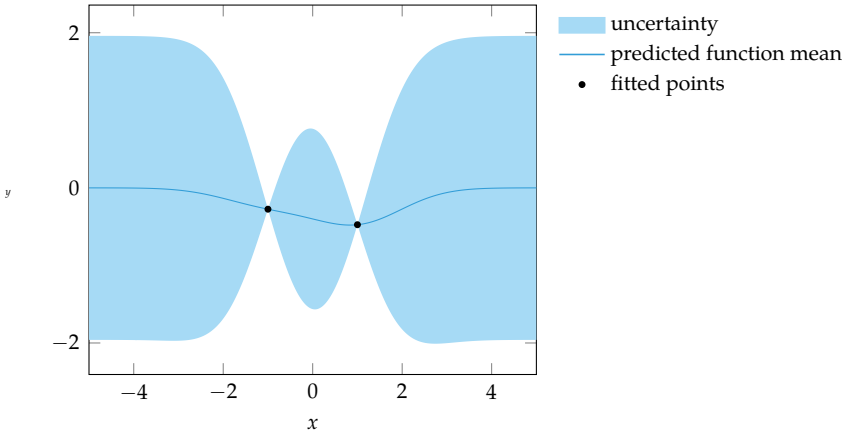
$$\exp(-\|x-x'\|) \to -8.688$$
$$\exp(-\|x-x'\|^2) \to -9.010$$
$$(1+\|x-x'\|)^{-1} \to -9.579$$
$$(1+\|x-x'\|^2)^{-1} \to -10.195$$
$$(1+\|x-x'\|)^{-2} \to -8.088$$

由此可见，最大化留一交叉验证似然的核函数是合理二次核$(1+\|x-x'\|)^{-2}$。

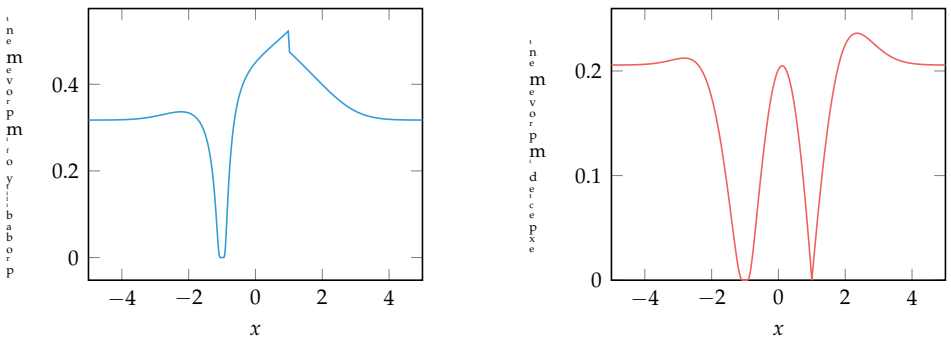
练习 16.1：基于高斯过程的预测优化可能会重复采样同一点。假设我们有一个零均值高斯过程，从单个点 x(1) 开始，这给了我们一些 $y^{(1)}$。预测均值在 x(1) 处有一个唯一的全局最小值。基于预测的优化将继续在 x(1) 处采样。

练习 16.2：基于误差的探索在降低方差方面浪费了精力，并且没有主动寻求最小化函数。

练习 16.3：高斯过程看起来像这样：



改进概率和预期改进如下所示：



最大改进概率为 $x = 0.98$（对应 $P = 0.523$）。

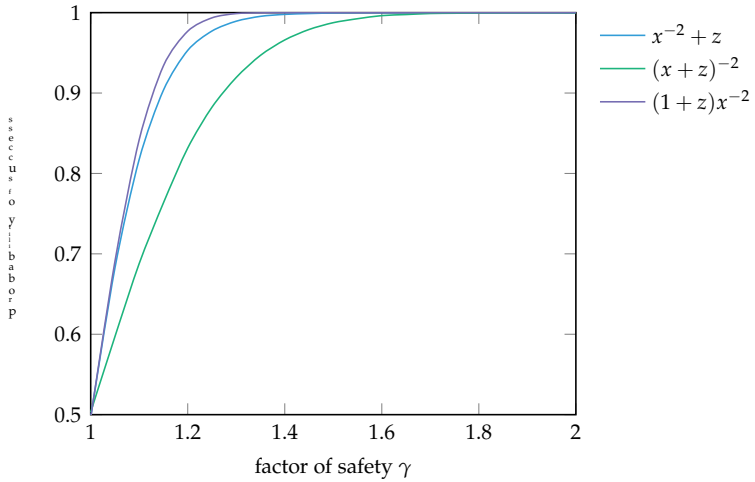
在 $E = 0.236$ 时，最大期望改进为 $x = 2.36$。

练习17.1：目标是最小化 $\mathbb{E}_{z\sim\mathcal{N}}[f(x+z)] - \sqrt{\text{Var}_{z\sim\mathcal{N}}[f(x+z)]}$。对应于均值的第一项在设计点 a 处达到最小。对应于标准差的第二项也在设计点 a 处达到最大，因为在该位置对设计进行扰动会导致输出产生大的变化。因此，最优设计是 $x^* = a$。

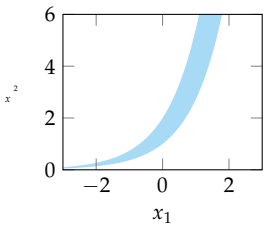
练习17.2：确定性优化问题is：

$$\begin{aligned} \underset{x}{\text{minimize}} \quad & x^2 \\ \text{subject to} \quad & \gamma x^{-2} \leq 1 \end{aligned}$$

最优横截面长度作为安全系数的函数是$x = \sqrt{\gamma}$。因此，我们可以在每个不确定性公式中用$\sqrt{\gamma}$代替横截面长度，并评估设计不失败的概率。注意，当安全系数为1时，由于正态分布的对称性，所有设计都有50%的失败概率。



练习 17.3：图 D.1 展示了无噪声的可行域。在没有噪声的情况下，最优解位于 x_1 为负无穷处。我们存在噪声，并选择不接受任何绝对值大于 6 的异常值。这类异常值大约在 1.973×10^{-7}% 的时间内出现。



图D.1. 无噪声可行区域。

x_2 的可行区域位于 e^{x_1} 与 $2e^{x_1}$ 之间。噪声是对称的，因此 x_2 最稳健的选择是 $1.5e^{x_1}$。

x_2 的可行区域宽度为 e^{x_1}，它随 x_1 增加。目标函数也随 x_1 增加，因此最优的 x_1 是使可行区域宽度至少为 12 的最小值。这导致 $x_1 = \ln 12 \approx 2.485$ 和 $x_2 = 18$。

练习18.1：累积分布函数可用于计算这些值：

```
julia> using Distributions
julia> N = Normal(0,1);
julia> cdf(N, 1) - cdf(N, -1)
0.6826894921370861
```

```
julia> cdf(N, 1)
0.841344746068543
```

因此，我们的样本大约有68.3%的时间落在均值的一个标准差范围内，并且有84.1%的时间低于均值以上一个标准差。

练习18.2： 我们首先代入样本的定义 均值：

$$\begin{aligned}\text{Var}(\hat{\mu}) &= \text{Var}\left(\frac{x^{(1)} + x^{(2)} + \cdots + x^{(m)}}{m}\right) \\ &= \text{Var}\left(\frac{1}{m}x^{(1)} + \frac{1}{m}x^{(2)} + \cdots + \frac{1}{m}x^{(m)}\right)\end{aligned}$$

两个独立变量之和的方差等于这两个变量方差的和。由此可得：

$$\begin{aligned}\text{Var}(\hat{\mu}) &= \text{Var}\left(\frac{1}{m}x^{(1)}\right) + \text{Var}\left(\frac{1}{m}x^{(2)}\right) + \cdots + \text{Var}\left(\frac{1}{m}x^{(m)}\right) \\ &= \frac{1}{m^2}\text{Var}\left(x^{(1)}\right) + \frac{1}{m^2}\text{Var}\left(x^{(2)}\right) + \cdots + \frac{1}{m^2}\text{Var}\left(x^{(m)}\right) \\ &= \frac{1}{m^2}(v + v + \cdots + v) \\ &= \frac{1}{m^2}(mv) \\ &= \frac{v}{m}\end{aligned}$$

练习18.3：正交多项式的三项递推关系是其构造与使用的核心。推导的关键在于注意到z的倍数可以从一个基转换到另一个基：

$$\int_{\mathcal{Z}} (zb_i(z))b_j(z)p(z)\,dz = \int_{\mathcal{Z}} b_i(z)\left(zb_j(z)\right)p(z)\,dz$$

我们必须展示

$$b_{i+1}(z) = \begin{cases}(z - \alpha_i)b_i(z) & \text{for } i = 1 \\ (z - \alpha_i)b_i(z) - \beta_i b_{i-1}(z) & \text{for } i > 1\end{cases}$$

生成正交多项式。

我们注意到 $b_{i+1} - zb_i$ 是一个次数最多为 i 的多项式，因此可以将其写成前 i 个正交多项式的线性组合：

$$b_{i+1}(z) - zb_i(z) = -\alpha_i b_i(z) - \beta_i b_{i-1}(z) + \sum_{j=0}^{i-2} \gamma_{ij} b_j(z)$$

对于常数 α_i、β_i 和 γ_{ij}。

将两边同时乘以 b_i 和 p，然后进行积分得到：

$$\int_{\mathcal{Z}} (b_{i+1}(z) - zb_i(z)) b_i(z) p(z)\, dz = \int_{\mathcal{Z}} \left(-\alpha_i b_i(z) - \beta_i b_{i-1}(z) + \sum_{j=0}^{i-2} \gamma_{ij} b_j(z) \right) b_i(z) p(z)\, dz$$

$$\begin{aligned}
\int_{\mathcal{Z}} b_{i+1}(z) b_i(z) p(z)\, dz - \int_{\mathcal{Z}} z b_i(z) b_i(z) p(z)\, dz = &- \int_{\mathcal{Z}} \alpha_i b_i(z) b_i(z) p(z)\, dz - \\
&- \int_{\mathcal{Z}} \beta_i b_{i-1}(z) b_i(z) p(z)\, dz + \\
&+ \sum_{j=0}^{i-2} \int_{\mathcal{Z}} \gamma_{ij} b_j(z) b_i(z) p(z)\, dz \\
- \int_{\mathcal{Z}} z b_i^2(z) p(z)\, dz = &- \alpha_i \int_{\mathcal{Z}} b_i^2(z) p(z)\, dz
\end{aligned}$$

生成我们关于 α_i 的表达式：

$$\alpha_i = \frac{\int_{\mathcal{Z}} z b_i^2(z) p(z)\, dz}{\int_{\mathcal{Z}} b_i^2(z) p(z)\, dz}$$

通过将两边同时乘以 b_{i-1} 和 p 然后积分，可以得到带有 $i \geq 1$ 的 β_i 的表达式。

将两边乘以b_k，其中$k < i - 1$为，同样得到：

$$- \int_{\mathcal{Z}} z b_i(z) b_k(z) p(z)\, dz = \gamma_{ik} \int_{\mathcal{Z}} b_k^2(z) p(z)\, dz$$

移位性质可以应用得到：

$$\int_{\mathcal{Z}} z b_i(z) b_k(z) p(z)\, dz = \int_{\mathcal{Z}} b_i(z) (z b_k(z)) p(z)\, dz = 0$$

由于$zb_k(z)$是次数至多为$i - 1$的多项式，并且由正交性，该积分为零。由此可知所有γ_{ik}均为零，三项递推关系得以建立。

练习18.4：我们可以通过设计组件x_i对f的偏导数来推导梯度近似值：

$$\frac{\partial}{\partial x_i} f(\mathbf{x}, \mathbf{z}) \approx b_1(\mathbf{z}) \frac{\partial}{\partial x_i} \theta_1(\mathbf{x}) + \cdots + b_k(\mathbf{z}) \frac{\partial}{\partial x_i} \theta_k(\mathbf{x})$$

如果我们有 m 个样本，我们可以将这些偏导数写成矩阵形式：

$$\begin{bmatrix} \frac{\partial}{\partial x_i} f(\mathbf{x}, \mathbf{z}^{(1)}) \\ \vdots \\ \frac{\partial}{\partial x_i} f(\mathbf{x}, \mathbf{z}^{(m)}) \end{bmatrix} \approx \begin{bmatrix} b_1(\mathbf{z}^{(1)}) & \cdots & b_k(\mathbf{z}^{(1)}) \\ \vdots & \ddots & \vdots \\ b_1(\mathbf{z}^{(m)}) & \cdots & b_k(\mathbf{z}^{(m)}) \end{bmatrix} \begin{bmatrix} \frac{\partial}{\partial x_i} \theta_1(\mathbf{x}) \\ \vdots \\ \frac{\partial}{\partial x_i} \theta_k(\mathbf{x}) \end{bmatrix}$$

我们可以使用伪逆来求解$\frac{\partial}{\partial x_i}\theta_1(\mathrm{x})$、……、$\frac{\partial}{\partial x_i}\theta_k(\mathrm{x})$的近似值：

$$\begin{bmatrix} \frac{\partial}{\partial x_i} \theta_1(\mathbf{x}) \\ \vdots \\ \frac{\partial}{\partial x_i} \theta_k(\mathbf{x}) \end{bmatrix} \approx \begin{bmatrix} b_1(\mathbf{z}^{(1)}) & \cdots & b_k(\mathbf{z}^{(1)}) \\ \vdots & \ddots & \vdots \\ b_1(\mathbf{z}^{(m)}) & \cdots & b_k(\mathbf{z}^{(m)}) \end{bmatrix}^+ \begin{bmatrix} \frac{\partial}{\partial x_i} f(\mathbf{x}, \mathbf{z}^{(1)}) \\ \vdots \\ \frac{\partial}{\partial x_i} f(\mathbf{x}, \mathbf{z}^{(m)}) \end{bmatrix}$$

练习18.5：估计的均值和方差具有依赖于设计变量的系数：

$$\begin{aligned} \hat{\mu}(\mathbf{x}) &= \theta_1(\mathbf{x}) \\ \hat{v}(\mathbf{x}) &= \sum_{i=2}^{k} \theta_i^2(\mathbf{x}) \int_{\mathcal{Z}} b_i(\mathbf{z})^2 p(\mathbf{z})\, d\mathbf{z} \end{aligned} \tag{D.25}$$

f_{mod} 关于第 i 个设计分量的偏导数为

$$\frac{\partial}{\partial x_i} f_{\text{mod}}(\mathbf{x}) = \alpha \frac{\partial \theta_1(\mathbf{x})}{\partial x_i} + 2(1-\alpha) \sum_{i=2}^{k} \theta_i(\mathbf{x}) \frac{\partial \theta_i(\mathbf{x})}{x_i} \int_{\mathcal{Z}} b_i(\mathbf{z})^2 p(\mathbf{z})\, d\mathbf{z} \tag{D.26}$$

计算方程 (D.26) 需要系数关于 x 的梯度，这将在练习 18.4 中进行估计。

练习 19.1：枚举法尝试所有设计。每个组件可以是真或假，因此在最坏情况下会产生 2^n 种可能的设计。这个问题有 $2^3 = 8$ 种可能的设计。

```
f(x) = (!x[1] || x[3]) && (x[2] || !x[3]) && (!x[1] || !x[2])
using IterTools
for x in IterTools.product([true,false], [true,false], [true,false])
    if f(x)
       @show(x)
       break
    end
end
```

```
x = (false, true, true)
```

练习19.2：布尔可满足性问题仅寻求一个有效的解。因此，我们将c设为零。

约束条件更有趣。与所有整数线性规划一样，x被限制为非负且为整数。此外，我们令1对应**true**，0对应**false**，并引入约束条件$x \leq 1$。

接下来，我们查看目标函数并注意到，$\wedge$ 的“与”语句将 f 划分为多个独立的布尔表达式，每个表达式都必须为真。我们将这些表达式转换为线性约束条件：

$$\begin{aligned} x_1 &\implies x_1 \geq 1 \\ x_2 \vee \neg x_3 &\implies x_2 + (1 - x_3) \geq 1 \\ \neg x_1 \vee \neg x_2 &\implies (1 - x_1) + (1 - x_2) \geq 1 \end{aligned}$$

其中每个表达式都必须得到满足（≥ 1），而否定变量 $\neg x_i$ 只是 $1 - x_i$。

生成的整数线性规划为：

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & 0 \\ \text{subject to} \quad & x_1 \geq 1 \\ & x_2 - x_3 \geq 0 \\ & -x_1 - x_2 \geq -1 \\ & \mathbf{x} \in \mathbb{N}^3 \end{aligned}$$

这种方法是通用的，可用于将任何布尔可满足性问题转换为整数线性规划。

练习 19.3：全幺模矩阵的逆矩阵也是整数矩阵。对于 A 是全幺模矩阵且 b 是整数的整数规划问题，可以使用单纯形法精确求解。

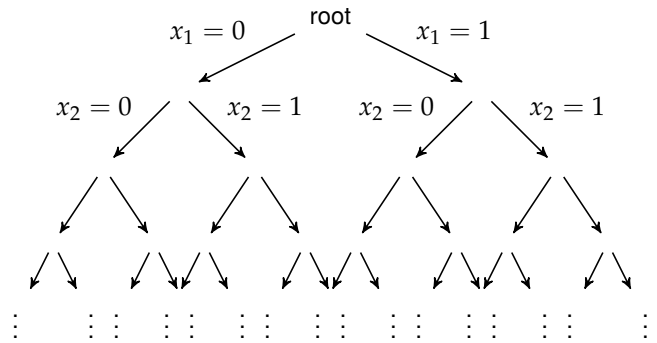
如果一个矩阵的每个非奇异方阵子矩阵都是单模的，则该矩阵是全单模的。单个矩阵元素就是一个方阵子矩阵。1×1矩阵的行列式是其单个元素的绝对值。单元素子矩阵只有当其行列式为±1时才是单模的，这仅当元素为±1时发生。单元素子矩阵也可以是非奇异的，这允许0。不允许其他元素，因此每个全单模矩阵只包含0、1或–1这些元素。

练习19.4：分支定界法要求我们能在设计上执行分支和界定操作。[2] 正在做出的决策

在0-1背包问题中，是否包含每个物品是一个选择。因此，每个物品代表一个分支：要么包含该物品，要么排除该物品。

2 P. J. Kolesar，“背包问题的分支定界算法”，*Management Science*，第13卷，第9期，第723–735页，1967年。

为每个这样的枚举构建一棵树，按照：



每个节点代表一个子问题，其中某些项目已被包含或排除。该子问题关联的决策变量、值和权重已被移除，并且容量实际上因包含项的总体重量而减少。

分支定界法利用边界操作的信息避免了构建整棵树。通过求解背包子问题的松弛版本可以构造出一个边界。这个*fractional knapsack*问题允许分配物品的小数值，$0 \leq x_i \leq 1$。

松弛背包问题可以通过贪心方法高效求解。每次选择价值与重量比最大的下一个物品来逐个添加物品。如果剩余容量足够，该物品被完全分配为 $x_i = 1$。如果不够，则分配一个分数值，使得剩余容量被饱和，并且所有剩余物品的 $x_i =$ 为 0。

我们首先对第一项进行分支。$x_1 = 0$ 的子树具有以下子问题：

$$\begin{aligned} \underset{\mathbf{x}_{2:6}}{\text{minimize}} \quad & -\sum_{i=2}^{6} v_i x_i \\ \text{subject to} \quad & \sum_{i=2}^{6} w_i x_i \leq 20 \end{aligned}$$

而带有 $x_1 = 1$ 的子树则有子问题：

$$\begin{aligned} \underset{\mathbf{x}_{2:6}}{\text{minimize}} \quad & -9 - \sum_{i=2}^{6} v_i x_i \\ \text{subject to} \quad & \sum_{i=2}^{6} w_i x_i \leq 13 \end{aligned}$$

我们可以使用贪心方法为两个子树构造一个下界。我们按价值与重量之比对其余物品进行排序：

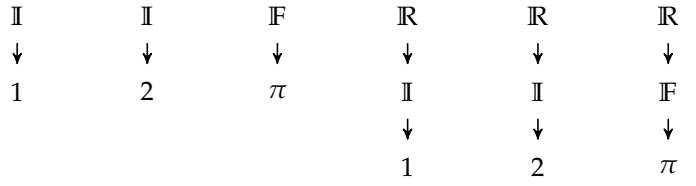
item:	6	4	5	3	2
ratio:	3/4	3/5	5/9	2/4	4/8
	0.75	0.6	0.556	0.5	0.5

对于以 $x_1 = 0$ 为根的子树，我们完全分配项目 6、4 和 5。然后，由于剩余容量为 2，我们部分分配项目 3，因此设 $x_3 = 2/4 = 0.5$。下界因此为 $-(3 + 5 + 3 + 0.5 \cdot 2) = -12$。

对于以$x_1 = 1$为根的子树，我们分配物品6和4，并将物品5部分分配给$x_5 = 4/9$。因此下界为$-(3 + 5 + (4/9) \cdot 3) \approx -18.333$。

以 $x_1 = 1$ 为根的子问题具有更好的下界，因此算法继续通过分裂该子问题来推进。最终解为 x = [1, 0, 0, 0, 1, 1]。

练习20.1：可以生成六种表达式树：



练习20.2：高度为0的表达式仅有一个，即空表达式。我们将其记作$a_0 = 1$。同样地，高度为1的表达式也只有一个，即表达式{}。我们将其记作$a_1 = 1$。对于深度2，存在三个表达式；深度3则有21个，以此类推。

假设我们已经构造了高度不超过 h 的所有表达式。所有高度为 $h + 1$ 的表达式都可以使用一个根节点，并根据以下规则选择左右子表达式来构造：

1. 高度为 h 的左表达式和高度小于 h 的右表达式
2. 一个高度为h的右表达式和一个高度小于h的左表达式
3. 高度h的左右表达式

由此可见，高度为$h + 1$ 的表达式数量为：[3]

3 这对应于OEIS序列A001699。

$$a_{h+1} = 2a_h(a_0 + \cdots + a_{h-1}) + a_h^2$$

练习20.3：可以使用以下语法及起始符号 $\mathbb{I}$：

$$\mathbb{I} \mapsto \mathbb{D} + 10 \times \mathbb{I}$$
$$\mathbb{D} \mapsto 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

练习20.4：构建无异常的语法可能颇具挑战性。此类问题可通过在目标函数中捕获异常并适当惩罚来避免。

练习 20.5：在表达式优化过程中，必须限制所操作变量类型的原因有很多。许多运算符仅对特定输入有效[4]，而矩阵乘法要求维度

4 通常不会对负数取平方根。

输入必须兼容。变量的物理维度是另一个问题。语法必须对输入值的单位以及可对其执行的有效操作进行推理。

例如，$x \times y$，其中x的单位是$\mathrm{kg}^a\mathrm{m}^b\mathrm{s}^c$，而$y$的单位是$\mathrm{kg}^d\mathrm{m}^e\mathrm{s}^f$，将生成单位为$\mathrm{kg}^{a+d}\mathrm{m}^{b+e}\mathrm{s}^{c+f}$的值。对$x$取平方根将生成单位为$\mathrm{kg}^{a/2}\mathrm{m}^{b/2}\mathrm{s}^{c/2}$的值。此外，诸如sin之类的操作只能应用于无单位的输入。

处理物理单位的一种方法是将一个n元组与表达式树中的每个节点相关联。该元组记录相对于允许的基本单位的指数，这些基本单位由用户指定。如果涉及的基本单位是质量、长度和时间，那么每个节点将有一个3元组（a，b，c）来表示单位$\text{kg}^a\text{m}^b\text{s}^c$。在分配产生式规则时，相关的文法必须考虑这些单位。[5]

5 更全面的概述，例如参见 A. Ratle 和 M. Sebag，“遗传编程与领域知识：超越语法引导机器发现的局限性”，载于 *International Conference on Parallel Problem Solving from Nature*，2000年。

练习20.6：可以使用`ExprRules.jl`通过字符串组合来编码该语法：

```
grammar = @grammar begin
    S = NP * " " * VP
    NP = ADJ * " " * NP
    NP = ADJ * " " * N
    VP = V  * " " * ADV
    ADJ = |(["a", "the", "big", "little", "blue", "red"])
    N = |(["mouse", "cat", "dog", "pony"])
    V = |(["ran", "sat", "slept", "ate"])
    ADV = |(["quietly", "quickly", "soundly", "happily"])
end
```

我们可以使用我们的表型方法来获得解 n.

```
eval(phenotype([2,10,19,0,6], grammar, :S)[1], grammar)
```

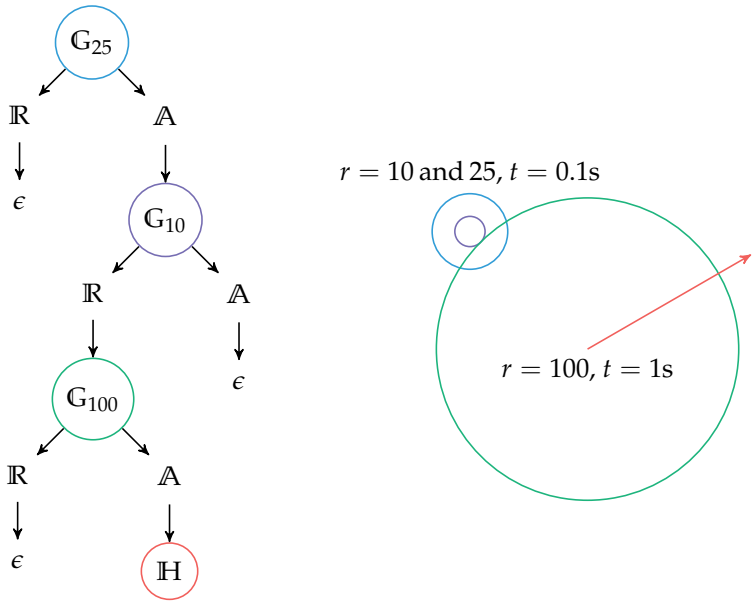
表型是“小狗吃得快”。

练习 20.7：我们为时钟问题定义一个文法。令 $\mathbb{G}_r$ 表示半径为 r 的齿轮的符号，令 $\mathbb{A}$ 表示轴，令 $\mathbb{R}$ 表示轮缘，令 $\mathbb{H}$ 表示指针。我们的文法是：

$$\begin{aligned}\mathbb{G}_r &\mapsto \mathbb{R}\,\mathbb{A} \mid \epsilon \\ \mathbb{R} &\mapsto \mathbb{R}\,\mathbb{R} \mid \mathbb{G}_r \mid \epsilon \\ \mathbb{A} &\mapsto \mathbb{A}\,\mathbb{A} \mid \mathbb{G}_r \mid \mathbb{H} \mid \epsilon\end{aligned}$$

这允许每个齿轮拥有任意数量的轮缘和轴子节点。表达式 ϵ 是一个空终端。

根据以下方式可以构造一个只有单根秒针的时钟：



注意，语法并不计算齿轮的旋转周期。这由目标函数处理。可以编写一个递归过程来返回所有指针的旋转周期。随后，利用旋转周期列表来计算目标函数值。

练习 20.8：本章所涵盖的任何方法均可用于完成四个 4 的谜题。一种简单的方法是对合适的文法使用蒙特卡洛采样。对恰好包含四个 4 的采样表达式进行求值，如果符合要求，则记录下来。重复此过程，直到为每个整数找到一个表达式。

其中一种合适的语法是：[6]

$$\mathbb{R} \mapsto 4 \mid 44 \mid 444 \mid 4444 \mid \mathbb{R}+\mathbb{R} \mid \mathbb{R}-\mathbb{R} \mid \mathbb{R}\times\mathbb{R} \mid \mathbb{R}/\mathbb{R} \mid \mathbb{R}^{\mathbb{R}} \mid \lfloor\mathbb{R}\rfloor \mid \lceil\mathbb{R}\rceil \mid \sqrt{\mathbb{R}} \mid \mathbb{R}! \mid \Gamma(\mathbb{R})$$

我们将求值表达式四舍五入到最接近的整数。向上舍入的表达式可以包含在向上取整操作中，向下舍入的表达式可以包含在向下取整操作中，因此所有此类表达式都是有效的。

6 伽马函数 $\Gamma(x)$ 是阶乘函数的扩展，它接受实数和复数值输入。对于正整数 x，它产生$(x-1)!$。

练习20.9：通过应用产生式规则得到该表达式：

$$\begin{aligned}
&\mathbb{R} \mapsto \mathbb{R} + \mathbb{R} && P = 1/12 \\
&\mathbb{R} \mapsto \mathbb{F} && P = 5/12 \\
&\mathbb{F} \mapsto 1.5 && P = 4/7 \\
&\mathbb{R} \mapsto \mathbb{I} && P = 5/12 \\
&\mathbb{I} \mapsto && P = 1/3
\end{aligned}$$

其概率为

$$\frac{1}{12}\frac{5}{12}\frac{4}{7}\frac{5}{12}\frac{1}{3} = \frac{25}{9072} \approx 0.00276$$

练习20.10：学习更新会清除所有计数，然后在每次应用每个产生式规则时将其递增。被应用的五个规则各递增一次，结果为：

$$\begin{aligned}
&\mathbb{R} \mapsto \mathbb{R} + \mathbb{R} \mid \mathbb{R} \times \mathbb{R} \mid \mathbb{F} \mid \mathbb{I} && w_{\mathbb{R}} = [1, 0, 1, 1] \\
&\mathbb{F} \mapsto 1.5 \mid \infty && p_{\mathbb{F}} = [1, 0] \\
&\mathbb{I} \mapsto 1 \mid 2 \mid 3 && p_{\mathbb{I}} = [0, 1, 0]
\end{aligned}$$

练习21.1：在满足翼型结构稳定性约束的条件下，最大化翼型的升阻比。

练习 21.2：考虑一个问题，其学科依赖图是一棵（有向）树：如果优化从根开始并按照拓扑顺序进行，那么在对树进行一遍遍历后就会收敛。

练习 21.3：它可以并行执行纪律分析。

练习21.4：多学科设计可行架构下的弹簧摆问题是：

$$\begin{aligned}
\underset{k}{\text{minimize}} \quad & f(\text{MDA}(k)) && \text{subject to} \quad k > 0 \\
& \theta \leq \theta_{\max} \\
& \text{MDA}(k) \text{ converged}
\end{aligned}$$

其中MDA(k)对这两个学科分析执行多学科分析：一个载荷分析计算$\mathcal{A}[M] = mg\ell\cos(\mathcal{A}[\theta])$，以及一个位移分析计算$\mathcal{A}[\theta] = \mathcal{A}[M]/\mathcal{A}[k]$。多学科设计分析是否收敛被作为一个额外的响应变量包含在内，以强制实现收敛。

求解优化问题得到 $k \approx 55.353$ /N。

练习21.5：个体设计可行架构下的弹簧摆问题是：

$$\begin{aligned}
\underset{k,\theta_c,M_c}{\text{minimize}} \quad & k \\
\text{subject to} \quad & k > 0 \\
& \theta_c = F_{\text{displacement}}(k, M_c) \\
& M_c = F_{\text{loads}}(\theta_c) \\
& \theta \leq \theta_{\max}
\end{aligned}$$

其中θ_c和M_c是受优化器控制的附加耦合变量。两个学科分析可以并行执行。

练习21.6：弹簧摆的两个学科优化问题

协同优化架构下的问题包括：

$$\begin{aligned}
\underset{k,M}{\text{minimize}} \quad & J_{\text{displacement}} = (k_g - \mathcal{A}[k_g])^2 + (M_g - \mathcal{A}[M_g])^2 + (F_{\text{displacement}}(k_g, M_g) - \theta)^2 \\
\text{subject to} \quad & \theta_g \leq \theta_{\max} \\
& k > 0
\end{aligned}$$

和

$$\underset{\theta_g}{\text{minimize}} \quad J_{\text{loads}} = (\theta_g - \theta)^2 + (F_{\text{loads}}(\theta_g) - M)^2$$

其中下标 g 表示全局变量。全局变量为 $\mathcal{A}_g = \{k_g、\theta_g、M_g\}$。系统级优化问题为：

$$\begin{aligned}
\underset{k_g,\theta_g,M_g}{\text{minimize}} \quad & k_g \\
\text{subject to} \quad & J_{\text{structures}} = 0 \\
& J_{\text{loads}} = 0
\end{aligned}$$

Bibliography

1. N. M. Alexandrov 和 M. Y. Hussaini 编，*Multidisciplinary Design Optimization: State of the Art*。SIAM，1997（参见第387页）。 2. S. Amari，“自然梯度在学习中高效工作”，*Neural Computation*，第10卷，第2期，第251–276页，1998（参见第90页）。 3. 亚里士多德，*Metaphysics*，由 W. D. Ross 翻译。公元前350年，第I卷，第5部分（参见第2页）。 4. L. Armijo，“具有Lipschitz连续一阶偏导数的函数的最小化”，*Pacific Journal of Mathematics*，第16卷，第1期，第1–3页，1966（参见第56页）。 5. J. Arora，*Introduction to Optimum Design*，第4版。Academic Press，2016（参见第4页）。 6. R. K. Arora，*Optimization: Algorithms and Applications*。Chapman and Hall/CRC，2015（参见第6页）。 7. T. W. Athan 和 P. Y. Papalambros，“关于多目标优化中折衷解的加权准则方法的注记”，*Engineering Optimization*，第27卷，第2期，第155–176页，1996（参见第221页）。 8. C. Audet 和 J. E. Dennis Jr.，“用于约束优化的网格自适应直接搜索算法”，*SIAM Journal on Optimization*，第17卷，第1期，第188–217页，2006（参见第105、126页）。 9. D. A. Bader、W. E. Hart 和 C. A. Phillips，“分支定界的并行算法设计”，载于 *Tutorials on Emerging Methodologies and Applications in Operations Research*，H. J. Greenberg 编，Kluwer Academic Press，2004（参见第346页）。 10. W. W. R. Ball，*Mathematical Recreations and Essays*。Macmillan，1892（参见第385页）。 11. D. Barber，*Bayesian Reasoning and Machine Learning*。Cambridge University Press，2012（参见第377页）。 12. A. G. Baydin、R. Cornish、D. M. Rubio、M. Schmidt 和 F. Wood，“使用超梯度下降的在线学习率自适应”，载于 *International Conference on Learning Representations* (*ICLR*)，2018（参见第82页）。 13. A. D. Belegundu 和 T. R. Chandrupatla，*Optimization Concepts and Applications in Engineering*，第2版。Cambridge University Press，2011（参见第6页）。

14. R. Bellman, “论动态规划理论”，*Proceedings of the National Academy of Sciences of the United States of America*，第38卷，第8期，第716–719页，1952年（引用自第3页）。15. R. Bellman，*Eye of the Hurricane: An Autobiography*。World Scientific，1984年（引用自第351页）。16. H. Benaroya和S. M. Han，*Probability Models in Engineering and Science*。Taylor & Francis，2005年（引用自第322页）。17. F. Berkenkamp、A. P. Schoellig和A. Krause，“基于高斯过程的四旋翼飞行器安全控制器优化”，载于*IEEE International Conference on Robotics and Automation* (*ICRA*)，2016年（引用自第297页）。18. H.-G. Beyer和B. Sendhoff，“鲁棒优化概述——全面综述”，*Computer Methods in Applied Mechanics and Engineering*，第196卷，第33期，第3190–3218页，2007年（引用自第307页）。19. T. L. Booth和R. A. Thompson，“将概率测度应用于抽象语言”，*IEEE Transactions on Computers*，第C-22卷，第5期，第442–450页，1973年（引用自第375页）。20. C. Boutilier、R. Patrascu、P. Poupart和D. Schuurmans，“使用极小极大决策准则的基于约束的优化与效用 elicitation”，*Artificial Intelligence*，第170卷，第8-9期，第686–713页，2006年（引用自第231页）。21. G. E. P. Box、W. G. Hunter和J. S. Hunter，*Statistics for Experimenters: An Introduction to Design, Data Analysis, and Model Building*，第2版。Wiley，2005年（引用自第235、307页）。22. S. Boyd和L. Vandenberghe，*Convex Optimization*。Cambridge University Press，2004年（引用自第6、178、438页）。23. D. Braziunas和C. Boutilier，“基于极小极大后悔的广义可加效用 elicitation”，载于*Conference on Uncertainty in Artificial Intelligence* (*UAI*)，2007年（引用自第231页）。24. D. Braziunas和C. Boutilier，“因子化效用的 elicitation”，*AI Magazine*，第29卷，第4期，第79–92页，2009年（引用自第230页）。25. R. P. Brent，*Algorithms for Minimization Without Derivatives*。Prentice Hall，1973年（引用自第51页）。26. S. J. Colley，*Vector Calculus*，第4版。Pearson，2011年（引用自第19页）。27. V. Conitzer，“使用比较查询 elicitation 单峰偏好”，*Journal of Artificial Intelligence Research*，第35卷，第161–191页，2009年（引用自第228页）。28. S. Cook，“定理证明过程的复杂性”，载于*ACM Symposium on Theory of Computing*，1971年（引用自第358页）。

29. W. Cook, A. M. Gerards, A. Schrijver, 和 É. Tardos, “整数线性规划中的敏感性定理”, *Mathematical Programming*, 第34卷, 第3期, 第251–264页, 1986年（见第341页）。 30. A. Corana, M. Marchesi, C. Martini, 和 S. Ridella, “使用‘模拟退火’算法最小化连续变量的多模态函数”, *ACM Transactions on Mathematical Software*, 第13卷, 第3期, 第262–280页, 1987年（见第130、132页）。 31. G. B. Dantzig, “单纯形法的起源”, 载于 *A History of Scientific Computing*, S. G. Nash 编, ACM, 1990年, 第141–151页（见第195页）。 32. S. Das 和 P. N. Suganthan, “差分进化：最新进展综述”, *IEEE Transactions on Evolutionary Computation*, 第15卷, 第1期, 第4–31页, 2011年（见第157页）。 33. W. C. Davidon, “用于最小化的变量度量方法”, 阿贡国家实验室, 技术报告 ANL-5990, 1959年（见第92页）。 34. W. C. Davidon, “用于最小化的变量度量方法”, *SIAM Journal on Opti- mization*, 第1卷, 第1期, 第1–17页, 1991年（见第92页）。 35. A. Dean, D. Voss, 和 D. Dragulji , *Design and Analysis of Experiments*, 第2版。Springer, 2017年（见第235页）。 36. K. Deb, A. Pratap, S. Agarwal, 和 T. Meyarivan, “一种快速且精英的多目标遗传算法：NSGA-II”, *IEEE Transactions on Evolutionary Computation*, 第6卷, 第2期, 第182–197页, 2002年（见第223页）。 37. T. J. Dekker, “通过逐次线性插值寻找零点”, 载于 *Constructive Aspects of the Fundamental Theorem of Algebra*, B. Dejon 和 P. Henrici 编, Interscience, 1969年（见第51页）。 38. R. Descartes, “几何学”, 载于 *Discours de la Méthode*。1637年（见第2页）。 39. E. D. Dolan, R. M. Lewis, 和 V. Torczon, “关于模式搜索的局部收敛性”, *SIAM Journal on Optimization*, 第14卷, 第2期, 第567–583页, 2003年（见第103页）。 40. M. Dorigo, G. Di Caro, 和 L. M. Gambardella, “用于离散优化的蚁群算法”, *Artificial Life*, 第5卷, 第2期, 第137–172页, 1999年（见第356页）。 41. M. Dorigo, V. Maniezzo, 和 A. Colorni, “蚁群系统：通过合作代理的群体进行优化”, *IEEE Transactions on Systems, Man, and Cybernetics, Part B* (*Cybernetics*), 第26卷, 第1期, 第29–41页, 1996年（见第354、355页）。 42. J. Duchi, E. Hazan, 和 Y. Singer, “在线学习和随机优化的自适应次梯度方法”, *Journal of Machine Learning Research*, 第12卷, 第2121–2159页, 2011年（见第77页）。 43. R. Eberhart 和 J. Kennedy, “一种使用粒子群理论的新优化器”, 载于 *International Symposium on Micro Machine and Human Science*, 1995年（见第159页）。

44. B. Efron, “自助法：对刀切法的另一种审视”，*The Annals of Statis- tics*，第 7 卷，第 1–26 页，1979 年（引用自第 270 页）。45. B. Efron，“估计预测规则的错误率：对交叉验证的改进”，*Journal of the American Statistical Association*，第 78 卷，第 382 期，第 316–331 页，1983 年（引用自第 273 页）。46. B. Efron 和 R. Tibshirani，“对交叉验证的改进：.632+ 自助法”，*Journal of the American Statistical Association*，第 92 卷，第 438 期，第 548–560 页，1997 年（引用自第 273 页）。47. 欧几里得，*The Elements*，D. E. Joyce 译。公元前 300 年（引用自第 2 页）。48. R. Fletcher，*Practical Methods of Optimization*，第 2 版。Wiley，1987 年（引用自第 92 页）。49. R. Fletcher 和 M. J. D. Powell，“一种快速收敛的最小化下降方法”，*The Computer Journal*，第 6 卷，第 2 期，第 163–168 页，1963 年（引用自第 92 页）。50. R. Fletcher 和 C. M. Reeves，“通过共轭梯度进行函数最小化”，*The Computer Journal*，第 7 卷，第 2 期，第 149–154 页，1964 年（引用自第 73 页）。51. J. J. Forrest 和 D. Goldfarb，“线性规划的最陡边单纯形算法”，*Mathematical Programming*，第 57 卷，第 1 期，第 341–374 页，1992 年（引用自第 201 页）。52. A. I. J. Forrester、A. Sóbester 和 A. J. Keane，“通过代理模型进行多保真度优化”，*Proceedings of the Royal Society of London A: Mathematical, Physical and Engineering Sciences*，第 463 卷，第 2088 期，第 3251–3269 页，2007 年（引用自第 244 页）。53. A. Forrester、A. Sobester 和 A. Keane，*Engineering Design via Surrogate Modelling: A Practical Guide*。Wiley，2008 年（引用自第 291 页）。54. J. H. Friedman，“探索性投影寻踪”，*Journal of the American Statistical Association*，第 82 卷，第 397 期，第 249–266 页，1987 年（引用自第 323 页）。55. W. Gautschi，*Orthogonal Polynomials: Computation and Approximation*。牛津大学出版社，2004 年（引用自第 328 页）。56. A. Girard、C. E. Rasmussen、J. Q. Candela 和 R. Murray-Smith，“输入不确定的高斯过程先验——在多步提前时间序列预测中的应用”，载于 *Advances in Neural Information Processing Systems* (*NIPS*)，2003 年（引用自第 335 页）。57. D. E. Goldberg，*Genetic Algorithms in Search, Optimization, and Machine Learning*。Addison-Wesley，1989 年（引用自第 148 页）。58. D. E. Goldberg 和 J. Richardson，“用于多模态函数优化的共享遗传算法”，载于 *International Conference on Genetic Algorithms*，1987 年（引用自第 227 页）。

59. G. H. Golub 和 J. H. Welsch，“高斯求积规则的计算，” *Mathemat- ics of Computation*，第23卷，第106期，第221–230页，1969年（引用自第331页）。 60. R. E. Gomory，“线性规划整数解的一种算法，” *Recent Advances in Mathematical Programming*，第64卷，第269–302页，1963年（引用自第342页）。 61. I. Goodfellow、Y. Bengio 和 A. Courville，*Deep Learning*。MIT Press，2016年（引用自第4页）。 62. A. Griewank 和 A. Walther，*Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*，第2版。SIAM，2008年（引用自第23页）。 63. S. Guo 和 S. Sanner，“基于成对比较查询的实时多属性贝叶斯偏好引出，” 见 *International Conference on Artificial Intelligence and Statistics* (*AISTATS*)，2010年（引用自第228页）。 64. B. Hajek，“最优退火的冷却调度，” *Mathematics of Operations Research*，第13卷，第2期，第311–329页，1988年（引用自第130页）。 65. T. C. Hales，“蜂窝猜想，” *Discrete & Computational Geometry*，第25卷，第1–22页，2001年（引用自第2页）。 66. J. H. Halton，“算法247：根式逆拟随机点序列，” *Communications of the ACM*，第7卷，第12期，第701–702页，1964年（引用自第248页）。 67. N. Hansen，“CMA进化策略：教程，” *ArXiv*，第1604.00772号，2016年（引用自第138页）。 68. N. Hansen 和 A. Ostermeier，“在进化策略中适应任意正态变异分布：协方差矩阵适应，” 见 *IEEE International Conference on Evolutionary Computation*，1996年（引用自第140页）。 69. F. M. Hemez 和 Y. Ben-Haim，“非线性瞬态测试与仿真相关性的信息差距鲁棒性，” *Mechanical Systems and Signal Process- ing*，第18卷，第6期，第1443–1467页，2004年（引用自第312页）。 70. G. Hinton 和 S. Roweis，“随机邻域嵌入，” 见 *Advances in Neural Information Processing Systems* (*NIPS*)，2003年（引用自第125页）。 71. R. Hooke 和 T. A. Jeeves，“数值和统计问题的直接搜索解法，” *Journal of the ACM* (*JACM*)，第8卷，第2期，第212–229页，1961年（引用自第102页）。 72. H. Ishibuchi 和 T. Murata，“一种多目标遗传局部搜索算法及其在流水车间调度中的应用，” *IEEE Transactions on Systems, Man, and Cybernetics*，第28卷，第3期，第392–403页，1998年（引用自第225页）。 73. V. S. Iyengar、J. Lee 和 M. Campbell，“Q-EVAL：使用查询评估多属性项目，” 见 *ACM Conference on Electronic Commerce*，2001年（引用自第229页）。

74. D. R. Jones, C. D. Perttunen, 和 B. E. Stuckman, “无需Lipschitz常数的Lipschitz优化”, *Journal of Optimization Theory and Application*, 第79卷, 第1期, 第157-181页, 1993年（引用自第108页）。75. D. Jones 和 M. Tamiz, *Practical Goal Programming*。Springer, 2010年（引用自第219页）。76. A. B. Kahn, “大型网络的拓扑排序”, *Communications of the ACM*, 第5卷, 第11期, 第558-562页, 1962年（引用自第390页）。77. L. Kallmeyer, *Parsing Beyond Context-Free Grammars*。Springer, 2010年（引用自第361页）。78. L. V. Kantorovich, “解决某些类极值问题的新方法”, 载于 *Proceedings of the USSR Academy of Sciences*, 第28卷, 1940年（引用自第3页）。79. A. F. Kaupe Jr, “算法178：直接搜索”, *Communications of the ACM*, 第6卷, 第6期, 第313-314页, 1963年（引用自第104页）。80. A. Keane 和 P. Nair, *Computational Approaches for Aerospace Design*。Wiley, 2005年（引用自第6页）。81. J. Kennedy, R. C. Eberhart, 和 Y. Shi, *Swarm Intelligence*。Morgan Kaufmann, 2001年（引用自第158页）。82. D. Kingma 和 J. Ba, “Adam：一种随机优化方法”, 载于*International Conference on Learning Representations* (*ICLR*), 2015年（引用自第79页）。83. S. Kiranyaz, T. Ince, 和 M. Gabbouj, *Multidimensional Particle Swarm Optimization for Machine Learning and Pattern Recognition*。Springer, 2014年, 第2.1节（引用自第2页）。84. S. Kirkpatrick, C. D. Gelatt Jr., 和 M. P. Vecchi, “通过模拟退火进行优化”, *Science*, 第220卷, 第4598期, 第671-680页, 1983年（引用自第128页）。85. T. H. Kjeldsen, “非线性规划中Kuhn-Tucker定理的语境化历史分析：第二次世界大战的影响”, *Historia Mathematica*, 第27卷, 第4期, 第331-361页, 2000年（引用自第176页）。86. L. Kocis 和 W. J. Whiten, “低差异序列的计算研究”, *ACM Transactions on Mathematical Software*, 第23卷, 第2期, 第266-294页, 1997年（引用自第249页）。87. P. J. Kolesar, “背包问题的分支定界算法”, *Manage-ment Science*, 第13卷, 第9期, 第723-735页, 1967年（引用自第475页）。88. B. Korte 和 J. Vygen, *Combinatorial Optimization: Theory and Algorithms*, 第5版。Springer, 2012年（引用自第339页）。89. J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of NaturalSelection*。MIT Press, 1992年（引用自第364页）。90. K. W. C. Ku 和 M.-W. Mak, “探索拉马克和鲍德温学习在进化递归神经网络中的影响”, 载于*IEEE Congress on Evolutionary Computation* (*CEC*), 1997年（引用自第162页）。

91. L. Kuipers 和 H. Niederreiter，*Uniform Distribution of Sequences*。Dover，2012（引用自第239页）。92. J. C. Lagarias、J. A. Reeds、M. H. Wright 和 P. E. Wright，“Nelder-Mead单纯形法在低维情况下的收敛特性”，*SIAM Journal on Optimization*，第9卷，第1期，第112–147页，1998年（引用自第105页）。93. R. Lam、K. Willcox 和 D. H. Wolpert，“有限预算下的贝叶斯优化：一种近似动态规划方法”，载于 *Advances in Neural Information Processing Systems* (*NIPS*)，2016年（引用自第291页）。94. A. H. Land 和 A. G. Doig，“一种求解离散规划问题的自动方法”，*Econometrica*，第28卷，第3期，第497–520页，1960年（引用自第346页）。95. C. Lemieux，*Monte Carlo and Quasi-Monte Carlo Sampling*。Springer，2009年（引用自第245页）。96. J. R. Lepird、M. P. Owen 和 M. J. Kochenderfer，“面向多目标工程设计优化的贝叶斯偏好启发”，*Journal of Aerospace Information Systems*，第12卷，第10期，第634–645页，2015年（引用自第228页）。97. K. Levenberg，“一种求解最小二乘中某些非线性问题的方法”，*Quarterly of Applied Mathematics*，第2卷，第2期，第164–168页，1944年（引用自第61页）。98. S. Linnainmaa，“将算法的累积舍入误差表示为局部舍入误差的泰勒展开”，硕士论文，赫尔辛基大学，1970年（引用自第30页）。99. M. Manfrin，“车辆路径问题的蚁群优化”，博士论文，布鲁塞尔自由大学，2004年（引用自第355页）。100. R. T. Marler 和 J. S. Arora，“工程多目标优化方法综述”，*Structural and Multidisciplinary Optimization*，第26卷，第6期，第369–395页，2004年（引用自第211页）。101. J. R. R. A. Martins 和 A. B. Lambe，“多学科设计优化：架构综述”，*AIAA Journal*，第51卷，第9期，第2049–2075页，2013年（引用自第387页）。102. J. R. R. A. Martins、P. Sturdza 和 J. J. Alonso，“复步导数近似”，*ACM Transactions on Mathematical Software*，第29卷，第3期，第245–262页，2003年（引用自第25页）。103. J. H. Mathews 和 K. D. Fink，*Numerical Methods Using MATLAB*，第4版。Pearson，2004年（引用自第24页）。104. K. Miettinen，*Nonlinear Multiobjective Optimization*。Kluwer Academic Publishers，1999年（引用自第211页）。105. D. J. Montana，“强类型遗传规划”，*Evolutionary Computation*，第3卷，第2期，第199–230页，1995年（引用自第365页）。

106. D. C. Montgomery，*Design and Analysis of Experiments*。Wiley，2017（见第235页）。107. M. D. Morris和T. J. Mitchell，“计算实验的探索性设计”，*Journal of Statistical Planning and Inference*，第43卷，第3期，第381–402页，1995（见第242页）。108. K. P. Murphy，*Machine Learning: A Probabilistic Perspective*。MIT Press，2012（见第254、265页）。109. S. Narayanan和S. Azarm，“关于改进设计优化的多目标遗传算法”，*Structural Optimization*，第18卷，第2-3期，第146–155页，1999（见第227页）。110. S. Nash和A. Sofer，*Linear and Nonlinear Programming*。McGraw-Hill，1996（见第178页）。111. J. A. Nelder和R. Mead，“一种用于函数最小化的单纯形方法”，*The Computer Journal*，第7卷，第4期，第308–313页，1965（见第105页）。112. Y. Nesterov，“一种求解具有收敛速率$O(1/k_2)$的凸规划问题的方法”，*Soviet Mathematics Doklady*，第27卷，第2期，第543–547页，1983（见第76页）。113. J. Nocedal，“具有有限存储的拟牛顿矩阵更新”，*Mathematics of Computation*，第35卷，第151期，第773–782页，1980（见第94页）。114. J. Nocedal和S. J. Wright，*Numerical Optimization*，第2版。Springer，2006（见第57、189页）。115. J. Nocedal和S. J. Wright，“信赖域方法”，载于*Numerical Optimization*。Springer，2006，第66–100页（见第65页）。116. A. O’Hagan，“一些贝叶斯数值分析”，*Bayesian Statistics*，第4卷，J. M. Bernardo、J. O. Berger、A. P. Dawid和A. F. M. Smith编，第345–363页，1992（见第282页）。117. M. Padberg和G. Rinaldi，“一种用于求解大规模对称旅行商问题的分支切割算法”，*SIAM Review*，第33卷，第1期，第60–100页，1991（见第342页）。118. P. Y. Papalambros和D. J. Wilde，*Principles of Optimal Design*。Cambridge University Press，2017（见第6页）。119. G.-J. Park、T.-H. Lee、K. H. Lee和K.-H. Hwang，“稳健设计：概述”，*AIAA Journal*，第44卷，第1期，第181–191页，2006（见第307页）。120. G. C. Pflug，“关于风险价值和条件风险价值的一些评论”，载于*Probabilistic Constrained Optimization: Methodology and Applications*，S. P. Uryasev编。Springer，2000，第272–281页（见第318页）。121. S. Piyavskii，“一种寻找函数绝对极值的算法”，*USSR Computational Mathematics and Mathematical Physics*，第12卷，第4期，第57–67页，1972（见第45页）。

122. E. Polak 和 G. Ribière，“关于共轭方向方法收敛性的注记”，
Revue Française d'informatique et de Recherche Opérationnelle, Série Rouge，第 3 卷，第 1 期，第 35–43 页，1969 年（引自第 73 页）。123. M. J. D. Powell，“一种无需计算导数即可寻找多变量函数最小值的有效方法”，*Computer Journal*，第 7 卷，第 2 期，第 155–162 页，1964 年（引自第 100 页）。124. W. H. Press、S. A. Teukolsky、W. T. Vetterling 和 B. P. Flannery，*Numerical Recipes in C: The Art of Scientific Computing*。Cambridge University Press，1982 年，第 2 卷（引自第 100 页）。125. C. E. Rasmussen 和 Z. Ghahramani，“贝叶斯蒙特卡洛”，载于 *Advances in Neural Information Processing Systems* (*NIPS*)，2003 年（引自第 335 页）。126. C. E. Rasmussen 和 C. K. I. Williams，
Gaussian Processes for Machine Learning。MIT Press，2006 年（引自第 277、278、287 页）。127. A. Ratle 和 M. Sebag，“遗传编程与领域知识：超越语法引导机器发现的局限”，载于 *International Conference on Parallel Problem Solving from Nature*，2000 年（引自第 478 页）。128. I. Rechenberg，
Evolutionsstrategie Optimierung technischer Systeme nach Prinzipien der biologischen Evolution。Frommann-Holzboog，1973 年（引自第 137 页）。129. R. G. Regis，“关于正生成集与正基的性质”，*Optimization and Engineering*，第 17 卷，第 1 期，第 229–262 页，2016 年（引自第 103 页）。130. A. M. Reynolds 和 M. A. Frye，“果蝇中的自由飞行气味追踪与最优间歇无标度搜索一致”，*PLoS ONE*，第 2 卷，第 4 期，e354，2007 年（引自第 162 页）。131. R. T. Rockafellar 和 S. Uryasev，“条件风险价值的优化”，*Journal of Risk*，第 2 卷，第 21–42 页，2000 年（引自第 316 页）。132. R. T. Rockafellar 和 S. Uryasev，“一般损失分布的条件风险价值”，*Journal of Banking and Finance*，第 26 卷，第 1443–1471 页，2002 年（引自第 318 页）。133. H. H. Rosenbrock，“一种寻找函数最大值或最小值的自动方法”，*The Computer Journal*，第 3 卷，第 3 期，第 175–184 页，1960 年（引自第 430 页）。134. R. Y. Rubinstein 和 D. P. Kroese，
The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation, and Machine Learning。Springer，2004 年（引自第 133 页）。135. D. E. Rumelhart、G. E. Hinton 和 R. J. Williams，“通过反向传播误差学习表示”，*Nature*，第 323 卷，第 533–536 页，1986 年（引自第 30 页）。136. C. Ryan、J. J. Collins 和 M. O. Neill，“语法进化：为任意语言进化程序”，载于 *European Conference on Genetic Programming*，1998 年（引自第 370 页）。

137. T. Salimans、J. Ho、X. Chen 和 I. Sutskever，“进化策略作为强化学习的可扩展替代方案”，*ArXiv*，第 1703.03864 号，2017 年（参见第 137 页）。138. R. Salustowicz 和 J. Schmidhuber，“概率增量程序进化”，*Evolutionary Computation*，第 5 卷，第 2 期，第 123–141 页，1997 年（参见第 377 页）。139. J. D. Schaffer，“多目标优化与向量评估遗传算法”，载于 *International Conference on Genetic Algorithms and Their Applications*，1985 年（参见第 221 页）。140. C. Schretter、L. Kobbelt 和 P.-O. Dehaye，“用于低差异采样的黄金比例序列”，*Journal of Graphics Tools*，第 16 卷，第 2 期，第 95–104 页，2016 年（参见第 247 页）。141. A. Shapiro、D. Dentcheva 和 A. Ruszczyński，*Lectures on Stochastic Programming: Modeling and Theory*，第 2 版。SIAM，2014 年（参见第 314 页）。142. A. Shmygelska、R. Aguirre-Hernández 和 H. H. Hoos，“用于二维 HP 蛋白质折叠问题的蚁群算法”，载于 *International Workshop on Ant Algorithms* (*ANTS*)，2002 年（参见第 355 页）。143. B. O. Shubert，“一种寻求函数全局最大值的序列方法”，*SIAM Journal on Numerical Analysis*，第 9 卷，第 3 期，第 379–388 页，1972 年（参见第 45 页）。144. D. Simon，*Evolutionary Optimization Algorithms*。Wiley，2013 年（参见第 162 页）。145. J. Sobieszczanski-Sobieski、A. Morris 和 M. van Tooren，*Multidisciplinary Design Optimization Supported by Knowledge Based Engineering*。Wiley，2015 年（参见第 387 页）。146. I. M. Sobol，“关于立方体中点的分布与积分的近似计算”，*USSR Computational Mathematics and Mathematical Physics*，第 7 卷，第 4 期，第 86–112 页，1967 年（参见第 249 页）。147. D. C. Sorensen，“带有模型信赖域修改的牛顿法”，*SIAM Journal on Numerical Analysis*，第 19 卷，第 2 期，第 409–426 页，1982 年（参见第 61 页）。148. K. Sörensen，“元启发式——被揭露的隐喻”，*International Transactions in Operational Research*，第 22 卷，第 1 期，第 3–18 页，2015 年（参见第 162 页）。149. T. J. Stieltjes，“关于所谓机械求积理论研究”，*Annales Scientifiques de l'École Normale Supérieure*，第 1 卷，第 409–426 页，1884 年（参见第 328 页）。150. J. Stoer 和 R. Bulirsch，*Introduction to Numerical Analysis*，第 3 版。Springer，2002 年（参见第 89、443 页）。151. M. Stone，“统计预测的交叉验证选择与评估”，*Journal of the Royal Statistical Society*，第 36 卷，第 2 期，第 111–147 页，1974 年（参见第 269 页）。152. T. Stützle，“用于二次分配问题的最大最小蚁群系统”，达姆施塔特技术大学，技术报告，1997 年（参见第 355 页）。

153. Y. Sui、A. Gotovos、J. Burdick 和 A. Krause，“高斯过程优化中的安全探索”，载于 *International Conference on Machine Learning* (*ICML*)，第 37 卷，2015 年（参见第 296 页）。154. H. Szu 和 R. Hartley，“快速模拟退火”，*Physics Letters A*，第 122 卷，第 3-4 期，第 157–162 页，1987 年（参见第 130 页）。155. G. B. Thomas，*Calculus and Analytic Geometry*，第 9 版。Addison-Wesley，1968 年（参见第 22 页）。156. V. Torczon，“论模式搜索算法的收敛性”，*SIAM Journal of Optimization*，第 7 卷，第 1 期，第 1–25 页，1997 年（参见第 103 页）。157. M. Toussaint，“贝叶斯搜索博弈”，载于 *Theory and Principled Methods for the Design of Metaheuristics*，Y. Borenstein 和 A. Moraglio 编。Springer，2014 年，第 129–144 页（参见第 291 页）。158. R. J. Vanderbei，*Linear Programming: Foundations and Extensions*，第 4 版。Springer，2014 年（参见第 189 页）。159. D. Wierstra、T. Schaul、T. Glasmachers、Y. Sun 和 J. Schmidhuber，“自然进化策略”，*ArXiv*，第 1106.4487 号，2011 年（参见第 139 页）。160. H. P. Williams，*Model Building in Mathematical Programming*，第 5 版。Wiley，2013 年（参见第 189 页）。161. D. H. Wolpert 和 W. G. Macready，“优化中的没有免费午餐定理”，*IEEE Transactions on Evolutionary Computation*，第 1 卷，第 1 期，第 67–82 页，1997 年（参见第 6 页）。162. P. K. Wong、L. Y. Lo、M. L. Wong 和 K. S. Leung，“基于语法的遗传规划与贝叶斯网络”，载于 *IEEE Congress on Evolutionary Computation* (*CEC*)，2014 年（参见第 375 页）。163. X.-S. Yang，*Nature-Inspired Metaheuristic Algorithms*。Luniver Press，2008 年（参见第 159、160 页）。164. X.-S. Yang，“优化简史”，载于 *Engineering Optimization*。Wiley，2010 年，第 1–13 页（参见第 2 页）。165. X.-S. Yang 和 S. Deb，“通过莱维飞行的布谷鸟搜索”，载于 *World Congress on Nature & Biologically Inspired Computing* (*NaBIC*)，2009 年（参见第 161 页）。166. P. L. Yu，“多目标决策问题中的锥凸性、锥极点和非支配解”，*Journal of Optimization Theory and Applications*，第 14 卷，第 3 期，第 319–377 页，1974 年（参见第 219 页）。167. Y. X. Yuan，“信赖域算法的最新进展”，*Mathematical Program- ming*，第 151 卷，第 1 期，第 249–281 页，2015 年（参见第 61 页）。168. L. Zadeh，“最优性和非标量值性能标准”，*IEEE Transactions on Automatic Control*，第 8 卷，第 1 期，第 59–60 页，1963 年（参见第 218 页）。

169. M. D. Zeiler, “ADADELTA：一种自适应学习率方法”，*ArXiv*，第1212.5701号，2012年（参见第78页）。

Index

0.632 自助法估计，273

抽象类型, 418Ackley 函数, 425　活跃的, 175Adadelta, 78Adagrad, *see* 自适应次梯度Adam, *see* 自适应矩估计　自适应矩估计, 79　自适应次梯度, 77　加性递推, 247　仿射子空间, 193　偶然不确定性, 307　代数, 2　算法, 1　算法, 2　锚点, 102　匿名函数, 420　蚁群优化, 354　近似线搜索, 55　元数, 77Armijo 条件, 55Armijo 线搜索, 56　数组推导式, 414　赋值, 388　关联数组, 388　渐近记号, 433　增广拉格朗日方法, 383

自动微分，27 辅助线性规划，202

回溯线搜索，56 后向差分，20 反向传播，30 鲍德温学习，164 障碍方法，183 基函数，255 批次，125 贝叶斯-埃尔米特求积，334 贝叶斯蒙特卡洛，334 BGFS，*see* Broyden-Fletcher-Goldfarb-Shanno 大O符号，433 二进制串染色体，150 比内公式，38 二分法，49 黑箱，99 布兰德规则，202 布尔，411 布尔可满足性问题，358 布思函数，426 自助法，270 自助样本，270 绑定操作，346 碗，9，11 碗形，437 括号，35

括号，35 括号阶段，57 分支，348 分支定界法，346 分支切割法，342 分支操作，346 Branin函数，427 Brent-Dekker，49 Broyden-Fletcher-Goldfarb-Shanno，92

微积分, 3 柯西分布, 148 天花板函数, 348 中心差分, 20 特征长度尺度, 278 切比雪夫距离, 439 棋盘距离, 439 染色体, 148 圆函数, 432 CMA-ES, *see* 协方差矩阵自适应

协同优化, 403 组合优化, 339 完全交叉验证, 269 复步法, 25 复合类型, 418 计算图, 27 凹的, 438 具体类型, 418

条件分布，276 条件风险价值，316 置信区域，282 共轭梯度，70 约束，6 约束方法，216 上下文无关文法，361 等高线图，11 等高线，11 凸组合，436 凸函数，437 凸集，437 坐标下降，99 互质，248 耦合变量，398 协方差函数，277 协方差矩阵自适应，138 协方差矩阵自适应进化策略，*see* 协方差矩阵自适应协变梯度，90 准则空间，213 临界点，173 临界点，7 交叉熵，133 交叉熵方法，133 交叉，148 交叉点，153 布谷鸟搜索，161 累积分布函数，316, 442 曲率条件，56, 98 切割平面，344 切割平面方法，342

CVaR，*see* 条件风险价值周期，202 循环坐标搜索，99

Dantzig 规则，202 Davidon-Fletcher-Powell，92 衰减步长因子，55

决策质量改进，231 Dempster-Shafer理论，308 依赖循环，390 依赖图，390 导数，19 无导数，99 下降方向，53 下降方向法，53 设计者，4 设计矩阵，254 设计点，5 设计选择，230 设计变量，5 DFP，*see* Davidon-Fletcher-Powell 字典，388, 417 差分进化，157 DIRECT，*see* 分割矩形 方向导数，22 直接法，99 Dirichlet分布，377 学科分析，387 差异，239 离散因子，235 离散优化，339 分布式架构，404 分割矩形，108 支配，212 点表示法，28 对偶，177 对偶证书，206 对偶可行性，176 对偶函数，177 对偶间隙，178 对偶数，28 对偶值，177 动态排序，105 动态规划，351

精英样本，133 进入索引，200 认知不确定性，307

等式形式，193 等价类共享，227 基于误差的探索，292 误差函数，442 欧几里得范数，439 交换算法，244 期望改进，295 期望值，314 专家响应，229 开发，293 探索，293 指数退火计划，130 指数加权准则，221

安全系数，319 快速退火，130 可行集，5 斐波那契搜索，37 有限差分法，23 萤火虫算法，159 微积分第一基本定理，

435 一阶，69 一阶必要条件，9 第一 Wolfe 条件，57 适应度比例选择，151 适应度共享，227 Fletcher-Reeves，73 下取整，344 花函数，428 前向累积，27 前向差分，20 前向传播，30 傅里叶级数，258 分数背包，475 全因子，235 函数，420 模糊集理论，308

伽马函数，278 间隙，312 高斯分布，275

高斯过程，275，277 高斯求积，443 高斯-赛德尔方法，390 基因复制，370 一般形式，191 泛化误差，265 广义拉格朗日量，177 广义模式搜索，103 遗传算法，148 遗传局部搜索，162 遗传编程，364 全局设计变量，396 全局最小化器，7 全局优化方法，45 目标规划，219 黄金比例，38 黄金分割搜索，39 梯度，21 梯度下降，69 文法，361 语法进化，370 贪婪启发式，202 贪婪局部搜索，244 网格搜索，235

半空间，192 哈尔顿序列，248 海森矩阵，21 留出法，267 胡克-吉夫斯方法，102 混合方法，162 超梯度，82 超梯度下降，80 超参数，36 超平面，21

图像，213 改进，293 非活动，175 个体学科可行，398 个体，147 拐点，8

信息差距决策理论，312 初始化阶段，198, 202 初始种群，147 整数线性规划，340 整数规划，340 跨学科兼容性，389 内点法，183 介值定理，49 不可约不确定性，307 迭代，53

Kahn算法，390 核，277 k折交叉验证，268 KKT条件，176 背包问题，353 Kullback–Leibler散度，133

L_2 正则化，263 拉格朗日乘子，173 拉格朗日乘子，171 拉格朗日量，173 拉马克学习，164 拉丁超立方采样，238 拉丁方，238 L-BFGS，*see* 有限内存 BFGS 跳越 Halton 方法，249 学习率，54，55 留一自助估计，270 留一交叉验证，269 离开索引，200 勒贝格测度，239 勒让德多项式，443 莱维飞行，162 字典序方法，217 有限内存 BFGS，94 线性组合，433 线性模型，254 线性规划，189 线性回归，254 线搜索，54 利普希茨连续，45

局部设计变量，396 局部最大值，11 局部最小值，7 局部模型，53 对数退火计划，130 对数似然，287 循环，423 低差异序列，245 置信下界，293 置信下界探索，

293

边缘分布, 276 Matérn 核, 278 矩阵, 415 矩阵范数, 98 最大似然估计, 287 最大最小不等式, 177 最大范数, 439 MDO, *see* 多学科设计优化 均值, 314 平均超额损失, 316 均值函数, 277 平均短缺, 316 均方误差, 265 模因算法, 162 记忆化, 352 网格自适应直接搜索, 126 Metropolis 准则, 128 Michalewicz 函数, 429 极小极大, 309 极小极大决策, 231 极小极大遗憾, 231 最小化器, 5 最小比率检验, 200 混合整数规划, 340 第二类修正贝塞尔函数, 278 动量, 75 蒙特卡洛积分, 245, 321

Morris-Mitchell准则，242 多学科分析，389 多学科设计可行，393 多学科设计优化，387 多目标优化，211 多变量，53 变异，148 变异率，155 相互共轭，72

命名函数，420 自然进化策略，137 自然梯度，90 必要条件，8 Nelder-Mead 单纯形法，105 Nesterov 动量，76 神经网络核，278 牛顿法，88 牛顿记号，28 小生境，227 小生境技术，227 没有免费午餐定理，6 非支配排序，223 非参数，465 范数，439 正态分布，275, 442 正规方程，274 数值微分，23

目标函数，5 运算符重载，32 机会主义，105 最优子结构，351 优化阶段，198，202 顺序，433 正交，325 出邻，355 过度约束，193 过拟合，265 重叠子问题，351

包，423 配对查询选择，229 成对距离，239 参数化，465 参数化类型，419 帕累托曲线，213 帕累托过滤器，225 帕累托前沿，213 帕累托最优性，211, 213 简约性，365 偏导数，21 部分可观测马尔可夫决策过程，291 粒子，158 粒子群优化，158 模式，103 模式搜索，99 罚函数法，178 信息素，354 旋转，200 Polak-Ribière，73 多面体方法，230 多项式基函数，257 多项式混沌，323 多面体，196 种群方法，147 正定，442 半正定，442 正生成集，103 可能性理论，308 后验分布，281 Powell 方法，100 基于预测的探索，291 偏好引出，228 原始，175 原始形式，177 原始值，177 主要解析中心，230 优先队列，346 概率语法，375 概率原型树，377

概率不确定性, 312 改进概率, 294 乘积相关规则, 335 产生式规则, 361 提议分布, 133 剪枝, 373 伪逆, 255 伪随机, 125

Q-Eval，229 二次收敛，89 二次拟合搜索，43 二次罚函数，181 二次规划，189 求积法则，443 分位数，316 拟蒙特卡洛方法，245 拟牛顿法，91 拟随机序列，245

径向函数, 259 随机重启, 66 随机采样, 236 随机子采样, 267 随机不确定性, 307 实值染色体, 150 可减少不确定性, 307 回归, 254 遗憾, 231 正则化项, 263 松弛, 341 残差形式, 406 响应变量, 387 受限步长方法, 61 反向累积, 27, 30 RMSProp, 78 鲁棒, 316 鲁棒对等方法, 309 鲁棒正则化, 309 求根方法, 49 均方根误差, 273 根, 49

罗森布罗克函数，430 旋转估计，268 轮盘赌选择，151 舍入，341

鞍点, 11 安全探索, 296 SafeOpt, 296 样本均值, 321 样本方差, 321 抽样计划, 15, 235 割线方程, 98 割线法, 91 二阶, 87 二阶必要条件, 9 二阶充分条件, 11 第二Wolfe条件, 57 自对偶单纯形算法, 208 敏感, 316 序贯优化, 396 基于集合的不确定性, 309 Shubert-Piyavskii方法, 45 单纯形, 105 单纯形算法, 195 模拟退火, 128 同时分析与设计, 406 单点交叉, 153 正弦基函数, 258 六西格玛, 320 松弛变量, 194 Sobol序列, 249 解, 5 空间填充, 235 空间填充度量, 239 空间填充子集, 244 规格, 4 平方指数核, 278 标准差, 282 样本标准差, 108 标准形式, 191

标准正态累积分布函数，294 起始类型，361 驻点，9, 69 统计可行性，316 步长，54 斯蒂尔杰斯算法，328 随机梯度下降，125 随机方法，125 分层抽样，238 严格局部极小值点，8 严格凹，438 严格凸，437 字符串，413 强回溯线搜索，57 强局部极小值，8 强局部极小值点，8 强类型遗传编程，365 强沃尔夫条件，57 强沃尔夫条件，57 子矩阵，342 子优化器，393 子种群，221 子问题，396 充分接近，89 充分条件，8 充分下降，55 上确界，239 代理模型，235 代理模型，253 符号微分，20 符号，361 系统级优化器，396 尾部在险价值，316

出租车搜索，99 泰勒近似，436 泰勒展开，435 泰勒级数，435 温度，128

终端，361 三元运算符，422 测试函数，425 测试集，267 拓扑排序，390 全幺模，342 锦标赛选择，151 迹，287 训练误差，265 训练集，267 旅行商问题，355 树交叉，365 树变异，365 树排列，367 三角不等式，439 截断选择，151 信赖域，61 元组，417 两点交叉，153 类型，361

不确定性传播, 321 未校正样本标准差, 108 均匀交叉, 153 均匀投影计划, 237 单峰函数, 35 单峰性, 35 单变量, 7 单变量函数, 7 单变量高斯, 442 乌托邦点, 214

风险价值，316 van der Corput 序列，248 VaR，*see* 风险价值方差，314 向量，413 向量评估遗传算法，221 向量优化，211 顶点，196

弱局部极小值，8 弱帕累托最优，213 加权指数和，219 加权最小-最大方法，220

加权和法，218 加权切比雪夫法，220 Wheeler 岭，431 白化，323

沃尔夫条件，57

零阶，99变焦相位，60